

Zynq UltraScale+ MPSoC Software Developer Guide

UG1137 (v10.0) June 26, 2019



Revision History

The following table shows the revision history for this document.

Date	Version	Revision
06/26/2019	v10.0	- In Chapter 4: - Updated Table 4-3. - In Chapter 7: - Updated Table 7-6 to Table 7-17. - In Chapter 10: - Added the CSU/PMU Register Access section. - Updated Table 10-10. - In Chapter 11: - Updated the Configuration Object section. - Updated the Power Management Initialization section. - Updated Appendix A to Appendix L. - Added a new library, Appendix M, XilMailbox Library v1.0.
01/18/2019	v9.0	- In Chapter 2: - Updated Boot Modes section. - Updated the System-Level Protections section with a reference to UG1085 - In Chapter 3: - Added Device Tree Generator section. - In Chapter 4: - Removed XilRSA references. - In Chapter 8: - Updated the Configuring XMPU Registers section. - In Chapter 10: - Updated PMU firmware Build Flags table with Efuse_Access and PM_LOG_LEVEL. Power Management Framework section updated. Restart Tracker Structure Members table updated. PMU firmware Metrics table updated. - In Chapter 12: Warm Restart section updated with a note about on-chip memory (OCM). - In Chapter 16: - Removed content and updated the chapter with a short description and added a reference to the Bootgen user guide.

06/22/2018	v8.0	• In Chapter 7: ◦ Added a note that SHA-2 will be deprecated from 2019.1 release with a recommendation to use SHA-3 • In Chapter 8: ◦ Added Enhanced RSA Key Revocation Support • In Chapter 10 ◦ Updated PMU firmware Signals PLL Lock Errors on PS_ERROR_OUT section and PMU firmware Loading Options
05/04/2018	v7.0	• In Chapter 8: ◦ Added Bif File for Obfuscated Form (Gray) key stored in eFUSE ◦ Updated deprecation of SHA-2 authentication • In Chapter 12: ◦ Added Warm Restart section • In Chapter 16: ◦ Updated Boot Image format documentation
01/19/2018	v6.0	• In Chapter 5: ◦ Added a note in Bare Metal Application Development section. • In Chapter 7: ◦ Updated Boot Flow section ◦ Updated Boot Modes section • In Chapter 8: ◦ Updated BIF File with Multiple AESKEY Files section. • In Chapter 13: ◦ Updated Ethernet Flow figures. • In Chapter 16: ◦ Updated example for <code>[fsbl_config]</code> parameter

11/15/2017	v5.0	• In Chapter 1: ◦ Updated Prerequisites section. • In Chapter 2: ◦ Updated Boot Process section. ◦ Updated Security section. • In Chapter 4: ◦ Updated FreeRTOS Software Stack section. • In Chapter 7: ◦ Added FSBL Build Process section. ◦ Added Setting FSBL Compilation Flags section. ◦ Updated Boot Modes section. • In Chapter 8: ◦ Updated Boot Time Security section. • In Chapter 9: ◦ Updated Platform Management in PS section. ◦ Updated PMU firmware section. • Added a new Chapter 10, Platform Management Unit Firmware. • In Chapter 11: ◦ Updated Zynq UltraScale+ MPSoC Power Management Software Architecture section. ◦ Updated Using the API for Power Management section. ◦ Updated Configuration Object section. ◦ Updated Power Management Initialization section. ◦ Updated Using the API for Power Management section. ◦ Updated XilPM Implementation Details section. • In Chapter 16: ◦ Updated BIF File Parameters section. ◦ Updated Boot Image Format section. ◦ Updated Boot Header Table. • Updated Appendix A to Appendix L. • Updated Appendix N, Additional Resources and Legal Notices.
------------	------	---

05/03/2017	v4.0	• In Chapter 2: ◦ Added Boot Process. • In Chapter 4: ◦ Updated Figure 4-2. ◦ Added information about Linux software stack exception levels EL0-EL3. • In Chapter 5: ◦ Updated Figure 5-1. • In Chapter 7: ◦ Moved Boot Flow here from Chapter 2. ◦ Added QSPI24 and QSPI32 Boot Modes and eMMC18 Boot Mode. ◦ Added more information to JTAG Boot Mode. ◦ Added USB Boot Mode. ◦ Updated Figure 7-6. ◦ Added FSBL_USB_EXCLUDE to Table 7-3. • In Chapter 8: ◦ Removed figure showing flow diagram for secured booting. ◦ Removed section on library support; this is now covered in Appendix I, XilSecure Library v4.0. ◦ Added examples to Encryption. ◦ Added more information to Authentication. ◦ Added Bitstream Authentication Using External Memory. ◦ Added System Memory Management Unit. ◦ Added A53 Memory Management Unit. ◦ Added R5 Memory Protection Unit. • Added Chapter 11, Power Management Framework. This content was previously in the <i>Power Management Framework User Guide: For Zynq UltraScale+ MPSoC Devices</i> (UG1199). • In Chapter 16: ◦ Added parameters and descriptions in Table 16-1. ◦ Added Boot Image Format. ◦ Added additional bit descriptions in Table 16-9. • Added Appendixes for OS & Libraries content (Appendixes A-K).
12/15/2016	v3.0	Added content to Introduction in Chapter 1. Corrected text in Boot Modes in Chapter 7. Changed link references to the <i>Zynq UltraScale+ MPSoC Technical Reference Manual</i> (UG1085). Corrected and added links to Appendix N, Additional Resources and Legal Notices.

10/05/2016	v2.0	Chapter 2: Removed JTAG and MDM from Figure 2-2. Clarified Secure and Non-Secure Boot Modes in in Chapter 2. Removed Interrupt Features. Chapter 3: Added Hardware IDE feature list. Added Vivado Design Suite. Modified Supported features in Xilinx Software Development Kit. Added a link to the SDK_Download. Replaced PetaLinux figure with Table 3-1. Chapter 4: Replaced Figure 4-2. Added FreeRTOS Software Stack. Chapter 5: Removed Developing Open Source Software. Chapter 6: Changed the title Chapter 6, Software Design Paradigms. Added Frameworks for Multiprocessor Development section. Chapter 7: Modified SD Mode diagram, Figure 7-3. Modified NAND Mode diagram Figure 7-5. Removed Keys organization in the CSU in Chapter 7, System Boot and Configuration. Removed Wake UP Mechanisms Chapter 7, System Boot and Configuration. Added Pre-Boot Sequence in Chapter 7 Chapter 8: Changed the title of Chapter 8, Security Features and reorganized sections. Revised text and renamed Xilinx Peripheral Protection Unit. Revised text in Encryption. Removed Encryption Key Types and Key Registers table. Replaced with a cross-reference to the <i>Zynq UltraScale+ MPSoC Technical Reference Manual</i> (UG1085). Made changes to Arm Trusted Firmware. Removed text from Protecting Memory with XMPU. Removed sections "Protection Checking" and "Error Handling" and "Using Peripheral Protection" from Security Features. Added Library Support. Chapter 9: Added reference to ATF [Ref 41]. Removed text under Wake Up Mechanisms. Added Power Management Framework. Modified PMU firmware. Removed Chapter 12, DMA. Chapter 13: Removed QEMU feature table. Added content to Boards and Kits. Removed Chapter 15, System Coherency. Moved Appendix A to Chapter Chapter 16, Boot Image Creation. Removed -interface option from Bootgen Command Options. Removed Virtualization section. Replaced with a reference to the <i>Zynq UltraScale+ MPSoC Technical Reference Manual</i> (UG1085). Fields and Offsets table removed. Replaced with a reference to the <i>Zynq UltraScale+ MPSoC Technical Reference Manual</i> (UG1085). Added that boot access is programmable. Removed several Wiki sites from Appendix N, Additional Resources and Legal Notices
11/18/2015	v1.0	Initial Public Access release.

Table of Contents

Revision History	2
Chapter 1: About This Guide	
Introduction	12
Intended Audience and Scope of this Document	13
Prerequisites	13
Chapter 2: Programming View of Zynq UltraScale+ MPSoC Devices	
Introduction	15
Hardware Architecture Overview	16
Boot Process	17
Virtualization	19
System Level Reset Requirements	20
Security	21
Safety and Reliability	24
Memory Overview for APU and RPU Executables	28
Chapter 3: Development Tools	
Introduction	30
Vivado Design Suite	30
Xilinx Software Development Kit	32
Arm GNU Tools	34
Device Tree Generator	35
PetaLinux Tools	35
Linux Software Development using Yocto	36
Chapter 4: Software Stack	
Introduction	39
Bare Metal Software Stack	39
Linux Software Stack	42
Third-Party Software Stack	46

Chapter 5: Software Development Flow

Overview of Software Development Flow	47
Bare Metal Application Development	48
Application Development Using PetaLinux Tools	50
Linux Application Development Using SDK	51

Chapter 6: Software Design Paradigms

Introduction	54
Frameworks for Multiprocessor Development	54
Symmetric Multiprocessing (SMP)	55
Asymmetric Multiprocessing (AMP)	56

Chapter 7: System Boot and Configuration

Introduction	61
Boot Process Overview	61
Boot Flow	61
Boot Image Creation	63
Boot Modes	64
Detailed Boot Flow	70
Disabling FPD in Boot Sequence	73
Setting FSBL Compilation Flags	73
FSBL Build Process	77

Chapter 8: Security Features

Introduction	101
Boot Time Security	101
Bitstream Authentication Using External Memory	113
Run-Time Security	116
Arm Trusted Firmware	116
FPGA Manager Solution	120
Xilinx Memory Protection Unit	122
Xilinx Peripheral Protection Unit	123
System Memory Management Unit	123
A53 Memory Management Unit	123
R5 Memory Protection Unit	124

Chapter 9: Platform Management

Introduction	125
Platform Management in PS	125

Chapter 10: Platform Management Unit Firmware

Introduction	131
Features	131
PMU firmware Architecture	132
Execution Flow	133
Handling Inter-Process Interrupts in PMU firmware	135
PMU firmware Modules	139
Error Management (EM) Module	142
Power Management (PM) Module	148
Scheduler	149
Safety Test Library	150
CSU/PMU Register Access	150
Timers	151
Configuration Object	153
PMU firmware Loading Options	157
PMU firmware Usage	162
Debugging PMU firmware	169
PMU firmware Memory Layout and Footprint	176
Dependencies	177

Chapter 11: Power Management Framework

Introduction	178
Zynq UltraScale+ MPSoC Power Management Overview	180
Zynq UltraScale+ MPSoC Power Management Software Architecture	183
Using the API for Power Management	196
XilPM Implementation Details	204
Linux	206
Arm Trusted Firmware (ATF)	220
PMU firmware	223

Chapter 12: Reset

Introduction	226
System-Level Reset	226
Block-Level Resets	226
Application Processing Unit Reset	227
Real Time Processing Unit Reset	227
Full Power Domain Reset	228
Warm Restart	228

Chapter 13: High-Speed Bus Interfaces

Introduction	256
USB 3.0	256
Gigabit Ethernet Controller	259
PCI Express	263

Chapter 14: Clock and Frequency Management

Introduction	268
Changing the Peripheral Frequency	268

Chapter 15: Target Development Platforms

Introduction	270
QEMU	270
Boards and Kits	270

Chapter 16: Boot Image Creation

Introduction	271
--------------------	-----

Appendix A: XilPM Library Parameters

XilPM Argument Value Definitions	272
XilPM Error Codes	279

Appendix B: Xilinx Standard C Libraries

Xilinx Standard C Libraries	282
-----------------------------------	-----

Appendix C: Standalone Library v7.0

Overview	288
----------------	-----

Appendix D: XilMFS Library v2.3

Overview	398
----------------	-----

Appendix E: LwIP2.1.1 Library v1.0

Introduction	412
--------------------	-----

Appendix F: XilFlash Library v4.6

Overview	433
----------------	-----

Appendix G: XilSf Library v5.13

Overview	444
----------------	-----

Appendix H: XilFFS Library v4.1

Overview	465
----------------	-----

Appendix I: XilSecure Library v4.0

Overview	471
----------------	-----

Appendix J: XilSKey Library v6.7

Overview	496
----------------	-----

Appendix K: XilPM Library v2.5

Overview	589
----------------	-----

Appendix L: XilFPGA Library v5.0

Overview	623
----------------	-----

Appendix M: XilMailbox Library v1.0

Overview	636
----------------	-----

Appendix N: Additional Resources and Legal Notices

Xilinx Resources	641
Solution Centers	641
Documentation Navigator and Design Hubs	641
References	642
Please Read: Important Legal Notices	644

About This Guide

Introduction

This document provides the software-centric information required for designing and developing system software and applications for the Xilinx® Zynq® UltraScale+™ MPSoC devices. The Zynq UltraScale+ MPSoC family has different products, based upon the following system features:

- Application processing unit (APU):
 - Dual or Quad-core Arm Cortex™-A53 MPCore™
 - CPU frequency up to 1.5 GHz
- Real-time processing unit (RPU):
 - Dual-core Arm Cortex-R5F MPCore
 - CPU frequency up to 600 MHz
- Graphics processing unit (GPU):
 - Arm Mali™-400 MP2
 - GPU frequency up to 667 MHz
- Video codec unit (VCU):
 - Simultaneous Encode and Decode through separate cores
 - H.264 high profile level 5.2 (4Kx2K-60)
 - H.265 (HEVC) main, main10 profile, level 5.1, high Tier, up to 4Kx2K-60 rate
 - 8 and 10-bit encoding
 - 4:2:0 and 4:2:2 chroma sampling

For more details, see the Zynq UltraScale+ MPSoC Product Table [\[Ref 5\]](#) and the Product Advantages [\[Ref 6\]](#).

Intended Audience and Scope of this Document

The purpose of this guide is to enable software developers and system architects to become familiar with:

- Xilinx software development tools
- Available programming options
- Xilinx software components that include device drivers, middleware stacks, frameworks, and example applications
- Platform management unit firmware (PMUFW), Arm® Trusted Firmware (ATF), OpenAMP, PetaLinux tools, Xen Hypervisor, and other tools developed for the Zynq UltraScale+ MPSoC device

Prerequisites

This document assumes that you are:

- Experienced with embedded software development
- Familiar with Armv7 and Armv8 architecture
- Familiar with Xilinx development tools such as the Vivado® Integrated Design Environment (IDE), the Xilinx software developers kit (SDK), compilers, debuggers, and operating systems.

This document includes the following chapters:

- [Chapter 2, Programming View of Zynq UltraScale+ MPSoC Devices](#): Briefly explains the architecture of the Zynq UltraScale+ MPSoC hardware. Xilinx recommends you to go through and understand each feature of this chapter.
- [Chapter 3, Development Tools](#): Provides a brief description about the Xilinx software development tools. This chapter helps you to understand all the available features in the software development tools. It is recommended for software developers to go through this chapter and understand the procedure involved in building and debugging software applications.
- [Chapter 4, Software Stack](#): Provides a description of various software stacks such as bare metal software, RTOS-based software and the full-fledged Linux stack provided by Xilinx for developing systems with the Zynq UltraScale+ MPSoC device.
- [Chapter 5, Software Development Flow](#): Walks you through the software development process. It also provides a brief description of the APIs and drivers supported in the Linux OS and bare metal.

- [Chapter 6, Software Design Paradigms](#): Helps you understand different approaches to develop software on the heterogeneous processing systems. After reading this chapter, you will have a better understanding of programming in different processor modes like symmetric multi-processing (SMP), asymmetric multi-processing (AMP), virtualization, and a hybrid mode that combines SMP and AMP.
- [Chapter 7, System Boot and Configuration](#): Describes the booting process using different booting devices in both secure and non-secure modes.
- [Chapter 8, Security Features](#): Describes the Zynq UltraScale+ MPSoC devices features you can leverage to enhance security during application boot- and run-time.
- [Chapter 9, Platform Management](#): Describes the features available to manage power consumption, and how to control the various power modes using software.
- [Chapter 10, Platform Management Unit Firmware](#): Describes the features and functionality of PMU firmware developed for Zynq UltraScale+ MPSoC device
- [Chapter 11, Power Management Framework](#): Describes the functionality of Xilinx's Power Management Framework (PMF) that supports a flexible power management control through the platform management unit (PMU).
- [Chapter 12, Reset](#): Explains the system and module-level resets.
- [Chapter 13, High-Speed Bus Interfaces](#): Explains the configuration flow of the high-speed interface protocols.
- [Chapter 14, Clock and Frequency Management](#): Briefly explains the clock and frequency management of peripherals in Zynq UltraScale+ MPSoC devices.
- [Chapter 15, Target Development Platforms](#): Explains about the different development platforms available for the Zynq UltraScale+ MPSoC device, such as quick emulators (QEMU), and the Zynq UltraScale+ MPSoC boards and kits.
- [Chapter 16, Boot Image Creation](#): Describes Bootgen, a standalone tool for creating a bootable image for Zynq UltraScale+ MPSoC devices. Bootgen is included in the SDK.
- [Appendix A-Appendix L](#) describe the available libraries and board support packages to help you develop a software platform.
- [Appendix N, Additional Resources and Legal Notices](#): Provides links to additional information that is cited throughout the document.

Programming View of Zynq UltraScale+ MPSoC Devices

Introduction

The Zynq® UltraScale+™ MPSoC device supports a wide range of applications that require heterogeneous multiprocessing. Heterogeneous multiprocessing system consists of multiple single and multi-core processors of differing types. It supports the following features:

- Multiple levels of security
- Increased safety
- Advanced power management
- Superior processing, I/O, and memory bandwidth
- A design approach, based on heterogeneous multiprocessing presents design challenges, which includes:
 - Meeting application performance requirements within a specified power envelope
 - Optimizing memory access within heterogeneous multiprocessing system
 - Providing low-latency, coherent communications between various processing engines
 - Managing and optimizing system power consumption in all operational modes

Xilinx provides comprehensive tools for hardware and software development on the Zynq UltraScale+ MPSoC device, and various software modules such as operating systems, heterogeneous system softwares, and security management modules.

The Zynq UltraScale+ MPSoC device is a heterogeneous device that includes the Arm® Cortex™-A53, high-performance, energy-efficient, 64-bit application processor, and also the 32-bit Arm Cortex-R5F dual-core real-time processor.

Hardware Architecture Overview

The Zynq UltraScale+ MPSoC devices provide power savings, programmable acceleration, I/O, and memory bandwidth. These features are ideal for applications that require heterogeneous multiprocessing.

Figure 2-1 shows the Zynq UltraScale+ MPSoC architecture with next-generation programmable engines for security, safety, reliability, and scalability.

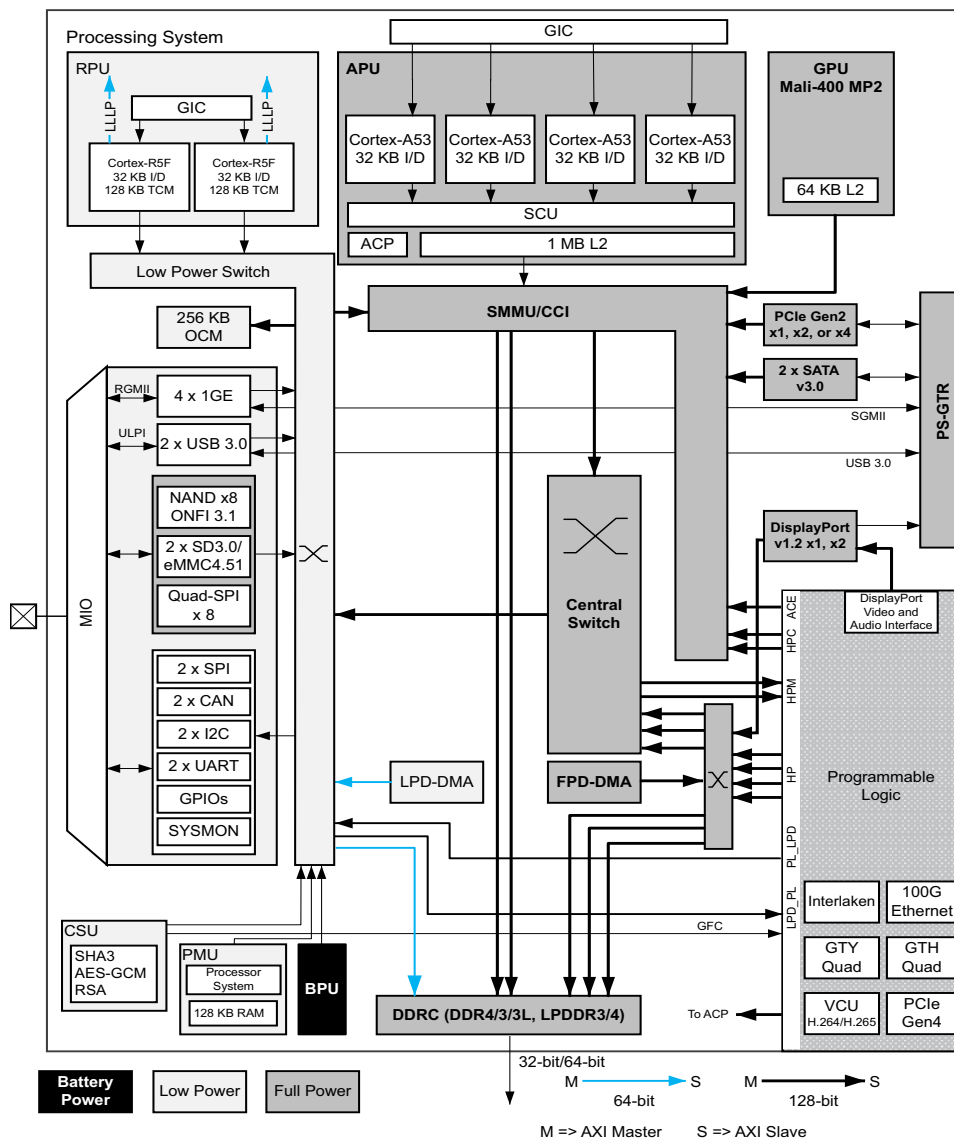


Figure 2-1: Zynq UltraScale+ MPSoC Device Hardware Architecture

The Zynq UltraScale+ MPSoC device features are as follows:

- Cortex-R5F dual-core real-time processor unit (RPU)
- Arm Cortex-A53 64-bit quad/dual-core processor unit (APU)
- Mali-400 MP2 graphic processing unit (GPU)
- External memory interfaces: DDR4, LPDDR4, DDR3, DDR3L, LPDDR3, 2x Quad-SPI, and NAND
- General connectivity: 2x USB 3.0, 2x SD/SDIO, 2x UART, 2x CAN 2.0B, 2x I2C, 2x SPI, 4x 1GE, and GPIO
- Security: Advanced Encryption Standard (AES), RSA public key encryption algorithm, and Secure Hash Algorithm-3 (SHA-3)
- AMS system monitor: 10-bit, 1 MSPS ADC, temperature, voltage, and current monitor
- The processor subsystem (PS) has five high-speed serial I/O (HSSIO) interfaces supporting the protocols:
 - PCIe: base specification, version 2.1 compliant, and Gen2x4
 - SATA 3.0
 - DisplayPort: Implements a DisplayPort source-only interface with video resolution up to 4k x 2k
 - USB 3.0: Compliant to USB 3.0 specification implementing a 5 Gb/s line rate
 - Serial GMII: Supports a 1 Gb/s SGMII interface
- Platform Management Unit (PMU) for functions that include power sequencing, safety, security, and debug.

For more details, see the following sections of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11]: [APU](#), [RPU](#), [PMU](#), [GPU](#), and inter-processor interrupt (IPI).

For additional components, see the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

Boot Process

The platform management unit (PMU) and configuration security unit (CSU) manage and perform the multi-staged booting process. You can boot the device in either secure or non-secure mode. See [Boot Process Overview](#) or, see the Boot and Configuration chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

Boot Modes

You can use any of the following as the boot mode for booting from external devices:

- Quad SPI flash memory (QSPI24, QSPI32)
- eMMC18
- NAND
- Secure Digital Interface Memory (SD0, SD1)
- JTAG
- USB

The BootROM does not directly support booting from SATA, Ethernet, or PCI Express (PCIe). The boot security does not rely on, and is largely orthogonal to TrustZone (TZ). The BootROM (running on the Platform Management Unit) performs the security resources management (for example, key management) and establishes root-of-trust. It authenticates FSBL, locks boot security resources, and transfers chain-of-trust control to FSBL (either on APU or RPU).

To understand more about the boot process in the different boot modes, see the 'Boot and Configuration' chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [\[Ref 11\]](#).

QSPI24 and QSPI32

The QSPI boot mode supports the following:

- x1, x2, and x4 read modes for single Quad SPI flash memory (QSPI24) and x8 for dual QSPI
- Image search for MultiBoot
- I/O mode is not supported in FSBL

Note: Single Quad-SPI memory (x1, x2 and x4) is the only boot mode that supports execute-in-place (XIP).

For additional information, see [QSPI24 and QSPI32 Boot Modes](#).

eMMC18

The eMMC18 boot mode supports:

- FAT 16 and FAT 32 file systems for reading the boot images.
- Image search for MultiBoot. The maximum number of searchable files as part of an image search for MultiBoot is 8,191.

For additional information, see [eMMC18 Boot Mode](#).

NAND

The NAND boot supports the following:

- 8-bit widths for reading the boot images
- Image search for MultiBoot

For additional information, see [NAND Boot Mode](#).

SD

The SD boot supported version is 3.0. This version supports:

- FAT 16/32 file systems for reading the boot images.
- Image search for MultiBoot. The maximum number of searchable files as part of an image search for MultiBoot is 8,191.

For additional information, see [SD Boot Mode](#).

JTAG

You can download any software images needed for the PS and hardware images needed for the PL using JTAG.



IMPORTANT: *In JTAG mode, you can boot the Zynq UltraScale+ MPSoC device in **non-secure mode** only.*

For additional information, see [JTAG Boot Mode](#).

USB

USB boot mode supports USB 3.0. It does not support MultiBoot, image fallback, or XIP. It supports both secure and non-secure boot mode. It is not supported for systems without DDR. USB boot mode is disabled by default. For additional information, see [USB Boot Mode](#).

Virtualization

Virtualization allows multiple software stacks to run simultaneously on the same processor, which enhances the productivity of the Zynq UltraScale+ MPSoC device. The role of virtualization varies from system to system. For some designers, virtualization allows the

processor to be kept fully loaded at all times, saving power and maximizing performance. For others systems, virtualization provides the means to partition the various software stacks for isolation or redundancy.

For more information, see [System Virtualization](#) in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

The support for virtualization applies only to an implementation that includes Arm exception level-2 (EL2). Armv8 supports virtualization extension to achieve full virtualization with near native guest operating system's performance. There are three key hardware components for virtualization:

- CPU virtualization
- Interrupt virtualization
- System MMU for I/O virtualization

System Level Reset Requirements

The system-level reset term is used to describe the system or subsystem level resets. 'System' reset (different from system-level resets) is a specific type of system-level reset. [Table 2-1](#) provides summary of system-level resets, which are described in details in subsequent sections.

Table 2-1: System-Level Resets

Reset Type	Description
External POR	The external POR reset is triggered by external pin assertion. There are a number of software only registers which are not reset by the POR resets. At first POR boot, a safety system (requiring HFT1 by PS & PL) can be configured such that a subsequent POR only resets PS (and not PL).
Internal POR	Internal POR reset can be triggered by software register write, or by safety errors. With the exception of error status register (which are reset by external POR, but not by internal POR), internal POR resets the same thing as external reset does. Internal-POR cannot be guaranteed without silicon validation (due to in-rush power concern), so internal-POR is for internal purpose unless validated.
System Reset	System reset is to be able to reset system excluding debug logic. To simplify system reset, there are few other things (xBIST, scan clear, power gating) which are not reset by this reset. Also, boot mode information is not reset by system reset. The system reset can be triggered by external pin (SRST), or software register write, or by safety errors.

Table 2-1: System-Level Resets (Cont'd)

Reset Type	Description
PS Only Reset	The PS only reset is to reset the PS while the PL remains active. This reset can be triggered by hardware error signals or by software register write. This reset is a subset of system reset (excluding the PL reset). If the PS reset is triggered by an error signal, then the error is also transmitted to the PL.
FPD Reset	The FPD reset resets all of the FPD power domain. It can be triggered by errors or software register write. If the FPD reset is triggered by an error signal, then the error is also transmitted to LPD & PL.
RPU Reset	The RPU Reset is to reset the RPU in case of errors. While each of the R5 core can be independently reset, but in lockstep, only R5_0 needs to be reset to reset both the R5 cores. This reset can be triggered by errors or software register write.

Security

The increasing ubiquity of Xilinx® devices makes protecting the intellectual property (IP) within them as important as protecting the data processed by the device. As security threats have increased, the range of security threats or potential weaknesses that must be considered to deploy secure products has grown as well.

The Zynq UltraScale+ MPSoC provides the following features to help secure applications running on the SoC:

- Encryption and authentication of configuration files.
- Hardened crypto accelerators for use by the user application.
- Secure methods of storing cryptographic keys.

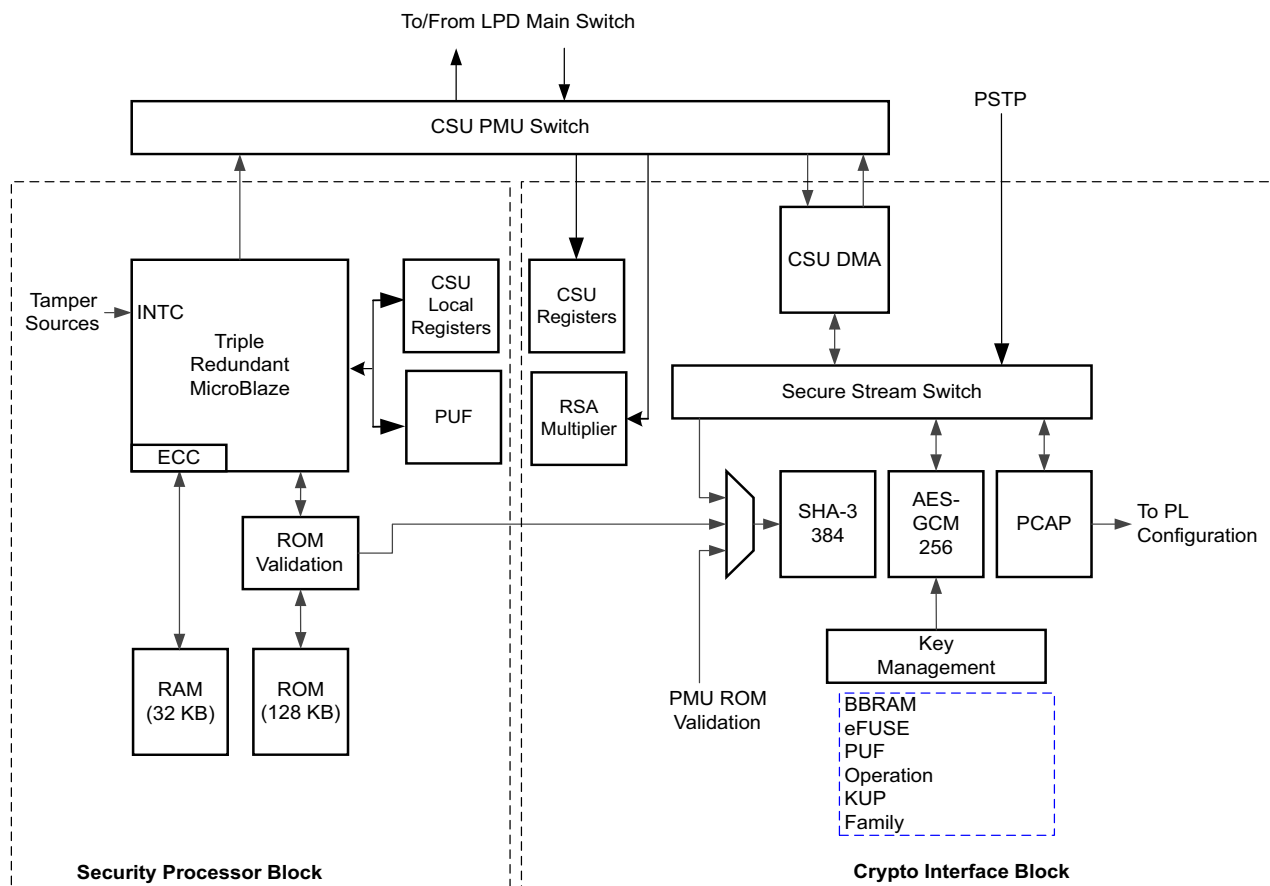
Methods for detecting and responding to tamper events. See '[Security](#)' chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [\[Ref 11\]](#) for more information.

Configuration Security Unit

The following are some of the important responsibilities of the configuration security unit (CSU):

- Secure boot.
- Tamper monitoring and response.
- Secure key storage and management.
- Cryptographic hardware acceleration.

The CSU comprises two main blocks as shown in Figure 2-2. On the left is the secure processor block that contains a triple redundant processor for controlling boot operation. It also contains an associated ROM, a small private RAM, and the necessary control/status registers required to support all secure operations. The block on the right is the crypto interface block (CIB) and contains the AES-GCM, DMA, SHA, RSA, and PCAP interfaces.



X15318-032817

Figure 2-2: Configuration and Security Unit Architecture

- After boot, the CSU provides tamper response monitoring. These crypto interfaces are available during runtime. To understand how to use these features, see [Appendix L, XilFPGA Library v5.0](#). See the 'Security' chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11] for more information.
- Secure Processor Block:** The triple-redundant processor architecture enhances the CSU operations during single event upset (SEU) conditions.
- Crypto Interface Block (CIB):** Consists of AES-GCM, DMA, SHA-3/384, RSA, and PCAP interfaces.
- AES-GCM:** The AES-GCM core has a 32-bit word-based data interface, with 256-bits of key support.

- **Key Management:** To use the AES, a key must be loaded into the AES block. The key is selected by CSU bootROM.
- **SHA-3/384:** The SHA-3/384 engine is used to calculate a hash value of the input image for authentication.
- **RSA-4096 Accelerator:** Facilitates RSA authentication.

To understand boot image encryption or authentication, refer to the following:

- [Chapter 7, System Boot and Configuration](#)
- [Chapter 16, Boot Image Creation.](#)
- The 'Security' chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [\[Ref 11\]](#).
- [Boot and Configuration](#) information in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [\[Ref 11\]](#).

System-Level Protections

The system-level protection mechanism involves the following areas:

- Zynq UltraScale+ MPSoC system software stack relies on the Arm Trusted Firmware (ATF). Protection can be enhanced even further by configuring the XMPU and XPPU to provide the system-level run-time security.
 - Protection against buggy or malicious software (erroneous software) from corrupting system memory or causing a system failure.
 - Protection against incorrect programming, or malicious devices (erroneous hardware) from corrupting system memory or causing a system failure.
 - Memory (DDR, OCM) and peripherals (peripheral control, SLCRs) are protected from illegal accesses by erroneous software or hardware to protect the system.
- The Xilinx® memory protection unit (XMPU) enforces memory partitioning and TrustZone (TZ) protection for memory and FPD slaves. The XMPU can be configured to isolate a master or a given set of masters to a developer-defined set of address ranges.
- The Xilinx peripheral protection unit (XPPU) provides LPD peripheral isolation and inter-processor interrupt (IPI) protection. The XPPU can be configured to permit one or more masters to access an LPD peripheral. For more information, see the '[XPPU Protection of Slaves](#)' section of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [\[Ref 11\]](#).

Safety and Reliability

The Zynq UltraScale+ MPSoC architecture includes features that enhance the reliability of safety critical applications to give users and designers increased confidence in their systems. The key features are as follows:

- Memory and cache error detection and correction
- RPU safety features
- System-wide safety features

To understand how to use these features, see [Chapter 8, Security Features](#).

Safety Features

The Cortex-A53 MPCore processor supports cache protection in the form of ECC on all RAM instances in the processor using the following separate protection elements:

- SCU-L2 cache protection
- CPU cache protection

These elements enable the Cortex-A53 MPCore processor to detect and correct a 1-bit error in any RAM, and to detect 2-bit errors.

Cortex-A53 MPCore RAMs are protected against single-event-upset (SEU) such that the processor system can detect and then, take specific action to continue making progress without data corruption. Some RAMs have parity single-error detect (SED) capability, while others have ECC single-error correct, double-error detect (SECCDED) capability.

The RPU includes two major safety features:

- Lock-step operation, shown in [Figure 2-3](#).
- Error checking and correction, described further in [Error Checking and Correction](#).

Lock-Step Operation

Cortex-R5F processors support lock-step operation mode, which operates both RPU CPU cores as a redundant CPU configuration called safety mode.

The Cortex-R5F processor set to operate in the lock-step configuration exposes only one CPU interface.

Because Cortex-R5F processor only supports the static split and lock configuration, switching between these modes is permitted only while the processor group is held in

power-on reset (POR). The input signals **SLCLAMP** and **SLSPLIT** control the mode of the processor group.

These signals control the multiplex and clamp logic in the lock-step configuration. When the Cortex-R5F processors are in the lock-step mode (shown in the following figure), there must be code in the reset handler to manage that the distributor within the GIC dispatches interrupts only to **CPU0**.

The RPU includes a dedicated interrupt controller for Cortex™-R5F MPCore processors. This Arm® PL390 generic interrupt controller (GIC) is based on the GICv1 specification.

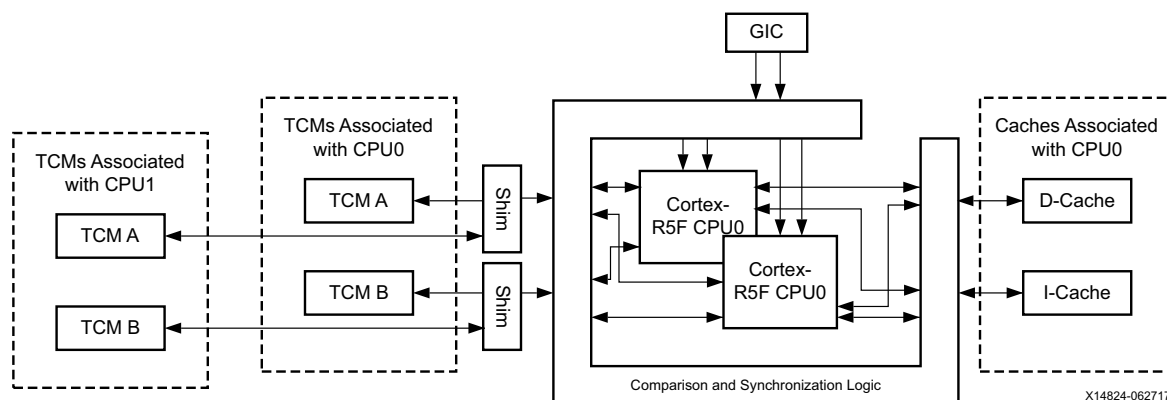


Figure 2-3: RPU Lock-Step Operation

Tightly coupled memories (TCMs) are mapped in the local address space of each Cortex-R5F processor; however, they are also mapped in the global address space where any master can access them provided that the XPPU is configured to allow such accesses.

The following table lists the address maps from the RPU point of view:

Table 2-2: RPU Address Maps

Operation Mode	Memory	R5_0 View (Start Address)	R5_1 View (Start Address)	Global Address View (Start Address)
Split Mode	R5_0 ATCM (64 KB)	0x0000_0000	N/A	0xFFE0_0000
	R5_0 BTCM (64 KB)	0x0002_0000	N/A	0xFFE2_0000
	R5_0 instruction cache	I-Cache	N/A	0xFFE4_0000
	R5_0 data cache	D-Cache	N/A	0xFFE5_0000
Split Mode	R5_1 ATCM (64KB)	N/A	0x0000_0000	0xFFE9_0000
	R5_1 BTCM (64KB)	N/A	0x0002_0000	0xFFEB_0000
	R5_1 instruction cache	I-Cache	N/A	0xFFEC_0000
	R5_1 data cache	D-Cache	N/A	0xFFED_0000

Table 2-2: RPU Address Maps (Cont'd)

Operation Mode	Memory	R5_0 View (Start Address)	R5_1 View (Start Address)	Global Address View (Start Address)
Lock-step Mode	R5_0 ATCM (128KB)	0x0000_0000	N/A	0xFFE0_0000
	R5_0 BTCM (128KB)	0x0002_0000	N/A	0xFFE2_0000
	R5_0 instruction cache	I-Cache	N/A	0xFFE4_0000
	R5_0 data cache	D-Cache	N/A	0xFFE5_0000

Error Checking and Correction

The Cortex-R5F processor supports error checking and correction (ECC) schemes of data. The data has similar properties although the size of the data chunk to which the ECC scheme applies is different.

For each aligned data chunk, the processor computes and stores a number of redundant code bits with the data. This enables the processor to detect up to two errors in the data chunk or its code bits, and correct any single error in the data chunk or its associated code bits. This is also referred to as a single-error correction, double-error detection (SEC-DED) ECC scheme.

System-Wide Safety Features

The system-wide safety features are designed to address error-free operation of the Zynq UltraScale+ MPSoC.

These features include the following:

- [Platform Management Unit](#)
- [PMU Triple-Redundancy](#)

The following sections describe these features.

Platform Management Unit

The platform management unit (PMU) in the Zynq UltraScale+ MPSoC device executes the code loaded from ROM and RAM within a flat memory space, implements power safety routines to prevent tampering of PS voltage rails, performs logic built-in self-test (LBIST), and responds to a user-driven power management sequence.

The PMU also includes some registers to control the functions that are typically very critical to the operation and safety of the device. Some of the registers related to safety are as follows:

- **GLOBAL_RESET**: Contains reset for safety-related blocks.

- **SAFETY_GATE**: Gates hardware features from accidental enablement.
- **SAFETY_CHK**: Checks the integrity of the interconnect data lines by using target registers for safety applications by periodically writing to and reading from these registers.

PMU Triple-Redundancy

The power management unit (PMU) contains triple-redundant embedded processors for a high-level of system reliability and strong SEU resilience. PMU controls the power-up, reset, and monitoring of resources within the entire system. The PMU performs multiple tasks including the following tasks:

- Initializing the system during boot
- Managing power gating and retention states for different power domains and islands
- Communicating the supply voltage settings to the external power control devices
- Managing sleep states including the deep-sleep mode and processing of wake functions

More details about PMU are available in [Chapter 9, Platform Management](#).

Interrupts

The generic interrupt controller (GIC) handles interrupts. Both the APU and the RPU have a separate dedicated GIC for interrupt handling.

The RPU includes an Arm PL390 GIC, which is based upon the GICv1 specification due to its flexibility and protection.

The APU includes a GICv2 controller. The GICv2 is a centralized resource for supporting and managing interrupts in multi-processor systems. It aids the GIC virtualization extensions that support the implementation of the GIC in systems supporting processor virtualization.

The Zynq UltraScale+ MPSoC device embeds an inter-processor interrupt (IPI) block that aids in communication between the heterogeneous processors. Because PMUs can communicate with different processors simultaneously, the PMU has four IPIs connected to the GIC of the PMU.

For more information on IPI routing to different processors, see the “Interrupts” chapter in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [\[Ref 11\]](#).

Memory Overview for APU and RPU Executables

The following tables give the configurable memory regions for APUs and RPUs.

Note the following:

- In RPU lock-step mode ([Lock-Step Operation](#)), R5_0_ATCM_MEM_0 and R5_0_BTCM_MEM_0 memory address are mapped to R5_0_ATCM_LSTEP and R5_0_BTCM_LSTEP memory ranges respectively in the system address map.
- In RPU split mode, R5_x_ATCM_MEM_0 and R5_x_BTCM_MEM_0 memory address are mapped to R5_x_ATCM_SPLIT and R5_x_BTCM_SPLIT memory ranges respectively in the system address map.
- QSPI memory is accessible when QSPI controller is in linear mode.

See the [System Addresses](#) chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085)[[Ref 11](#)] for more information.

See Real-time Processing Unit (RPU) and On-Chip Memory (OCM) sections of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085)[[Ref 11](#)] for more information on RPU, R5 and OCM.

Table 2-3: Configurable Memory Regions for APUs

Memory Type	Start Address	Size
DDR Low	0x00000000	2 GB
DDR High	0x80000000	2 GB
OCM	0xFFFFC000	256 KB
QSPI	0xC0000000	512 MB

Table 2-4: Configurable Memory Regions for RPU Lock-Step Mode

Memory Type	Start Address	Size
DDR Low	0x100000	2047 MB
OCM	0xFFFFC000	256 KB
QSPI	0xC0000000	512 MB
R5_0_ATCM_MEM_0	0x00000	64 KB
R5_0_BTCM_MEM_0	0x20000	64 KB
R5_TCM_RAM_0_MEM	0x00000	256 KB

Table 2-5: Configurable Memory Regions for RPU Split Mode

Memory Type	Start Address	Size
R5_0		
DDR Low	0x100000	2047 MB
OCM	0xFFFFC0000	256 KB
QSPI	0xC0000000	512 MB
R5_0_ATCM_MEM_0	0x00000	64 KB
R5_0_BTCM_MEM_0	0x20000	64 KB
R5_1		
DDR Low	0x100000	2047 MB
OCM	0xFFFFC0000	256 KB
QSPI	0xC0000000	512 MB
R5_1_ATCM_MEM_0	0x00000	64 KB
R5_1_BTCM_MEM_0	0x20000	64 KB

Note: BootROM always copies First Stage Boot Loader (FSBL) from 0xFFFFC0000 and it is not configurable. If FSBL is compiled for a different load address, Bootgen may refuse it as CSU bootROM (CBR) does not parse partition headers in the boot image but merely copies the FSBL code at a fixed OCM memory location (0xFFFC0000). See [Chapter 7, System Boot and Configuration](#) for more information on Bootgen.

Development Tools

Introduction

This chapter focuses on Xilinx® tools and flows available for programming software for Zynq® UltraScale+™ MPSoC devices. However, the concepts are generally applicable to third-party tools as the Xilinx tools incorporate familiar components such as an Eclipse-based integrated development environment (IDE) and the GNU compiler tool chain.

This chapter also provides a brief description about the open source tools available that you can use for open source development on different processors of the Zynq UltraScale+ MPSoC device.

A comprehensive set of tools for developing and debugging software applications on Zynq UltraScale+ MPSoC devices includes:

- Hardware IDE
- Software IDEs
- Compiler toolchain
- Debug and trace tools
- Embedded OS and software libraries
- Simulators (for example: QEMU)
- Models and virtual prototyping tools (for example: emulation board platforms)

Third-party tool solutions vary in the level of integration and direct support for Zynq UltraScale+ MPSoC devices.

The following sections provide a summary of the available Xilinx development tools.

Vivado Design Suite

The Xilinx Vivado® Design Suite contains tools that are encapsulated in the Vivado integrated design environment (IDE). The IDE provides an intuitive graphical user interface (GUI) with powerful features.

The Vivado Design Suite supersedes the Xilinx ISE software with additional features for system-on-a-chip development and high-level synthesis. It delivers a SoC-strength, IP- and system-centric, next generation development environment built exclusively by Xilinx to address the productivity bottlenecks in system-level integration and implementation.

All of the tools and tool options in Vivado Design Suite are written in native Tool Command Language (Tcl) format, which enables use both in the Vivado IDE or the Vivado Design Suite Tcl shell. Analysis and constraint assignment is enabled throughout the entire design process. For example, you can run timing or power estimations after synthesis, placement, or routing. Because the database is accessible through Tcl, changes to constraints, design configuration, or tool settings happen in real time, often without forcing re-implementation.

The Vivado IDE uses a concept of opening designs in memory. Opening a design loads the design netlist at that particular stage of the design flow, assigns the constraints to the design, and then applies the design to the target device. This provides the ability to visualize and interact with the design at each design stage.



IMPORTANT: *The Vivado IDE supports designs that target 7 series and newer devices only.*

You can improve design performance and ease of use through the features delivered by the Vivado Design Suite, including:

- The Processor Configuration Wizard (PCW) within the IP Integrator with graphical user interfaces to let you create and modify the PS within the IP Integrator block design.



VIDEO: *For a better understanding of the PCW, see the Quick Take Video: [Vivado Processor Configuration Wizard Overview](#).*

- Register transfer level (RTL) design in VHDL, Verilog, and SystemVerilog
- Quick integration and configuration of IP cores from the Xilinx IP Catalog to create block designs through the Vivado IP Integrator
- Vivado synthesis
- C-based sources in C, C++, and SystemC
- Vivado implementation for place and route
- Vivado serial I/O and logic analyzer for debugging
- Vivado power analysis
- SDC-based Xilinx® Design Constraints (XDC) for timing constraints entry
- Static timing analysis
- Flexible floorplanning
- Detailed placement and routing modification

- Bitstream generation
- Vivado Tcl Store, which you can use to add to and modify the capabilities in Vivado

You can download the Vivado Design Suite from the *Xilinx Vivado Design Suite – HLx Editions* [\[Ref 3\]](#).

Xilinx Software Development Kit

The Xilinx Software Development Kit (SDK) provides a complete environment for creating software applications targeted for Xilinx embedded processors. It includes a GNU-based compiler toolchain, JTAG debugger, flash programmer, middleware libraries, bare metal BSPs, and drivers for Xilinx IP. SDK also includes a robust IDE for C/C++ bare metal and Linux application development and debugging. Based upon the open source Eclipse platform, SDK incorporates the C/C++ Development Toolkit (CDT).

SDK lets you create software applications using a unified set of Xilinx tools for the Arm® Cortex™-A53 and Cortex-R5F processors, as well as Xilinx MicroBlaze™ processors. SDK provides various methods to create applications, as follows:

- Bare metal and FreeRTOS applications for MicroBlaze
- Bare metal, Linux, and FreeRTOS applications for APU
- Bare metal and FreeRTOS applications for RPU
- User customization of PMU firmware
- Library examples are provided with the SDK tool (ready to load sources and build), as follows:
 - OpenCV
 - OpenAMP RPC
 - FreeRTOS "HelloWorld"
 - lwIP
 - Performance tests (Dhrystone, memory tests, peripheral tests)
 - RSA authentication for preventing tampering or modification of images and bitstream
 - First stage boot loader (FSBL) for APU or RPU.

You can export a block design, hardware design files, and bitstream files to the SDK export directory directly from the Vivado Project Navigator. For more information regarding the Vivado Design Suite, see the *Vivado Design Suite Documentation* [\[Ref 22\]](#).

All processes necessary to successfully complete this export process are run automatically. The Xilinx SDK creates a new hardware platform project within the workspace containing the following files:

- `.project`: Project file
- `psu_init.tcl`: PS initialization script
- `psu_init.c, psu_init.h`: PS initialization code
- `psu_init.html`: Register summary viewer
- `system.hdf`: Hardware definition file

The compiler can be switched as follows:

- 32-bit or 64-bit (applications that are targeted to Cortex-A53)
- 32-bit only (applications targeted to Cortex-R5F, and Xilinx MicroBlaze devices)

For the list of build procedures, see the *Xilinx Software Developer Kit Help* [Ref 24], where built-in help content lets you explore further after you launch the SDK tool.

Also, Xilinx SDK provides the following tools for use in Xilinx embedded software development:

- **Xilinx System Debugger (XSDB)**: Provides a system debugger GUI and a command-line interface to the Xilinx `hw_server`. XSDB also provides various low-level debugging features not directly available in the Xilinx SDK.
- **FPGA programmer**: Programs the Xilinx device with the bitstream.
- **Flash programmer**: Used for burning bitstreams and software application images into external, parallel NOR flash devices.
- **Linker script generator**: Used for mapping your application image across the hardware memory space.
- **Boot image generator**: Used to create a boot image by combining boot loader, bitstream, user applications, and optional authentication and encryption enabled.
- **Xilinx software command-line tool (XSCT)**: The Xilinx software command-line tool is based on Tcl and is delivered in the SDK install. This tool provides the Tcl prompt and you can use all of the supported commands.

The Xilinx software command-line tool is a scriptable command-line interface to run Xilinx SDK commands, XSDB commands, and HSI commands. XSCT can be started from the Start menu or by executing the `xsct.bat` file, available in the `<SDK installation directory>/bin` folder. All the commands supported by XSCT are grouped under their respective categories.

The Xilinx SDK provides a separate perspective for each task to ease the software development process. Perspectives available for C/C++ developers are, as follows:

- **C/C++ Perspective views:** Helps you to view, create, and build the software C/C++ projects. By default, it consists of an editor area and other views, such as Xilinx SDK projects, C/C++ projects to show the software projects present in the workspace, a navigation console, properties, tasks, make targets, outline, and search.
- **System Debugger:** Helps you debug a software application. The system debugger from open source is customized and integrated with SDK.
- **System Performance Monitor:** Assists you in characterizing and evaluating the performance of hardware and software systems by providing the performance summary views of the MicroBlaze devices, and the PL and PS of the Zynq UltraScale+ MPSoC device.
- **Remote System Explorer:** Lets you connect and work with a variety of remote systems.

Xilinx SDK supports Linux application development but does not explicitly target Linux kernel development and debug; however, both Xilinx PetaLinux tools and third-party partners provide such tools and capabilities.

For a detailed explanation on the Xilinx SDK features, and to understand the SDK design flow, see the *Xilinx Software Developer Kit Help* [Ref 24].

You can download the SDK tool from the *Embedded Design Tools Download* [Ref 26].

Arm GNU Tools

The Arm GNU open source toolchain is adopted for the Xilinx software development platform. The GNU tools for Linux hosts are available as part of Xilinx software development kit (SDK). This section details the open source GNU tools and Linux tools available for the processing clusters in the Zynq UltraScale+ MPSoC device.

The following table lists some of the Xilinx Arm GNU tools available for programming the APU, RPU, and embedded MicroBlaze processors.

Table 3-1: Xilinx Arm GNU Tools

Tool	Description
<code>aarch64-linux-gnu-gcc</code> <code>aarch64-linux-gnu-g++</code>	GNU C/C++ compiler.
<code>aarch64-linux-gnu-as</code>	GNU assembler.
<code>aarch64-linux-gnu-ld</code>	GNU linker.
<code>aarch64-linux-gnu-ar</code>	A utility for creating, modifying, and extracting from archives.
<code>aarch64-linux-gnu-objcopy</code>	Copies and translates object files.
<code>aarch64-linux-gnu-objdump</code>	Displays information from object files.

Table 3-1: Xilinx Arm GNU Tools (Cont'd)

Tool	Description
<code>aarch64-linux-gnu-size</code>	Lists the section sizes of an object or archive file.
<code>aarch64-linux-gnu-gprof</code>	Displays profiling information.
<code>aarch64-linux-gnu-gdb</code>	The GNU debugger.

Device Tree Generator

The device tree (DT) data structure consists of nodes with properties that describe a hardware. The Linux kernel uses the device tree to support a wide range of hardware configurations.

In FPGAs, it is possible to have different combinations of peripheral logics, each using a different configuration. For all the different combinations, the device tree generator (DTG) generates the `.dts/.dtsi` device tree files.

The following is a list of the `dts/dtsi` files generated by the device tree generator:

- `pl.dtsi`: Contains all the memory mapped peripheral logic (PL) IPs.
- `pcw.dtsi`: Contains the dynamic properties for the PS IPs.
- `system-top.dts`: Contains the memory, boot arguments, and command line parameters.
- `zynqmp.dtsi`: Contains all the PS specific and the CPU information.
- `zynqmp-clk-ccf.dtsi`: Contains all the clock information for the PS peripheral IPs.

For more information, see the [Build Device Tree Blob](#) page on the *Xilinx Wiki* [Ref 38].

PetaLinux Tools

The PetaLinux tools offer everything necessary to customize, build, and deploy open source Linux software to devices.

PetaLinux tools include the following:

- Build tools such as GNU, `petalinux-build`, and `make` to build the kernel images and the application software.
- Debug tools such as GDB, `petalinux-boot`, and `oprofile` for profiling.

The following table shows the supported PetaLinux tools.

Table 3-2: PetaLinux Supported Tools

Tools	Description
GNU	Arm GNU tools.
<code>petalinux-build</code>	Used to build software image files.
Make	Make build for compiling the applications.
GDB	GDB tools for debugging.
<code>petalinux-boot</code>	Used to boot Linux.
QEMU	Emulator platform for the Zynq UltraScale+ MPSoC device.
OProfile	Used for profiling.

See the following documentation for more details:

- *PetaLinux Tools* documentation [\[Ref 2\]](#)
- *Zynq UltraScale+ MPSoC: Embedded Design Tutorial* (UG1209) [\[Ref 13\]](#)
- *Zynq UltraScale+ MPSoC OpenAMP Getting Started Guide* (UG1186) [\[Ref 16\]](#)

Linux Software Development using Yocto

Xilinx offers the `meta-xilinx` Yocto/OpenEmbedded recipes to enable those customers with in-house Yocto build systems to configure, build, and deploy Linux for Zynq UltraScale+ MPSoC devices.

The `meta-xilinx` layer also provides a number of BSPs for common boards which use Xilinx devices.

The `meta-xilinx` layer provides additional support for Yocto/OE, adding recipes for various components. See the *meta-xilinx* link [\[Ref 33\]](#).

You can develop Linux software on Cortex-A53 using open source Linux tools. This section explains the Linux Yocto tools and its project development environment.

The following table lists the Yocto tools.

Table 3-3: Yocto Tools

Tool Type	Name	Description
Yocto build tools	Bitbake	Generic task execution engine that allows shell and Python tasks to be run efficiently, and in parallel, while working within complex inter-task dependency constraints.
Yocto profile and trace tools	Perf	Profiling and tracing tool that comes bundled with the Linux Kernel.
	Ftrace	Refers to the ftrace function tracer but encompasses a number of related tracers along with the infrastructure used by all the related tracers.
	Oprofile	System-wide profiler that runs on the target system as a command-line application.
	Sysprof	System-wide profiler that consists of a single window with three panes, and buttons, which allow you to start, stop, and view the profile from one place.
	Blktrace	A tool for tracing and reporting low-level disk I/O.

Yocto Project Development Environment

Developers can configure the Yocto project development environment to support developing Linux software for Zynq UltraScale+ MPSoC devices through Yocto recipes provided from the Xilinx GIT server. You can use components from the Yocto project to design, develop, and build a Linux-based software stack.

Figure 3-1 shows the complete Yocto project development environment. The Yocto project has wide range of tools which can be configured to download the latest Xilinx kernel and build with some enhancements made locally in the form of local projects.

You can also change the build and hardware configuration through BSP.

Yocto combines a compiler and other tools to build and test images. After the images pass the quality tests and package feeds required for SDK generation are received, the Yocto tool launches SDK for application development.

The important features of the Yocto project are, as follows:

- Provides a recent Linux kernel along with a set of system commands and libraries suitable for the embedded environment.
- Makes available system components such as X11, GTK+, Qt, Clutter, and SDL (among others) so you can create a rich user experience on devices that have display hardware. For devices that do not have a display or where you wish to use alternative UI frameworks, these components need not be installed.
- Creates a focused and stable core compatible with the OpenEmbedded project with which you can easily and reliably build and develop Linux software.

- Supports a wide range of hardware and device emulation through the quick emulator (QEMU). See the *Zynq UltraScale+ MPSoC QEMU User Guide (UG1169)* [Ref 8] for more information.



IMPORTANT: *Enabling Full Yocto of Xilinx QEMU is not available.*

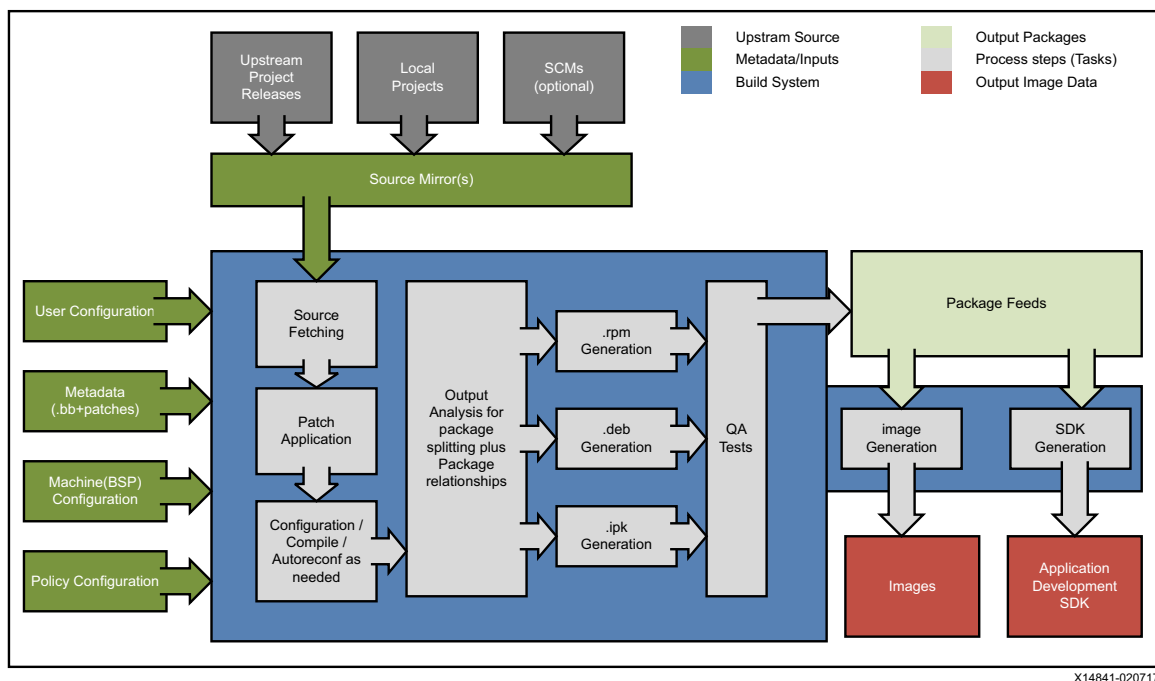


Figure 3-1: Yocto Project Development Environment

You can download the Yocto tools and the Yocto project development environment from the *Yocto Project Organization* [Ref 44].

For more information about Xilinx-provided Yocto features, see [Yocto Features](#) in the *PetaLinux Tools Documentation: Reference Guide (UG1144)* [Ref 27].

Software Stack

Introduction

This chapter provides an overview of the various software stacks available for the Zynq® UltraScale+™ MPSoC devices.

For more information about the various software development tools used with this device, see [Chapter 3, Development Tools](#). For more information about bare metal and Linux software application development, see [Chapter 5, Software Development Flow](#).

Bare Metal Software Stack

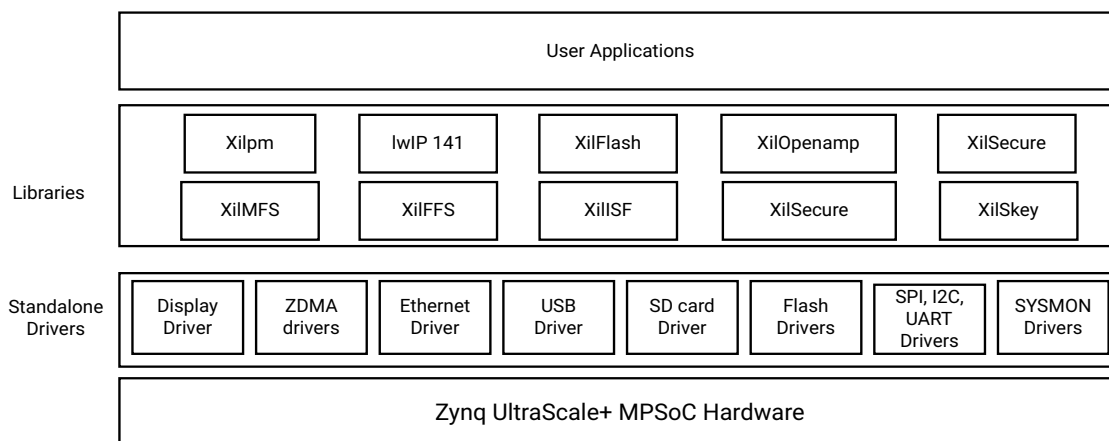
Xilinx provides a bare metal software stack called the standalone board support package (BSP) as part of the Xilinx® SDK tools. The Standalone BSP gives you a simple, single-threaded environment that provides basic features such as standard input/output and access to processor hardware features. The BSP and included libraries are configurable to provide the necessary functionality with the least overhead. The [Xilinx Software Development Kit](#) in [Chapter 3, Development Tools](#) provides more overview information. You can locate the standalone drivers at the following path:

```
<Xilinx Installation  
Directory>\SDK\<version>\data\embeddedsd\XilinxProcessorIPLib\drivers
```

You can locate libraries at the following path:

```
<Xilinx Installation  
Directory>\SDK\<version>\data\embeddedsd\lib\sw_services
```

The following figure illustrates the bare metal software stack in the APU.



X17169-112618

Figure 4-1: Bare-Metal Software Development Stack

Note: The software stack of libraries and drivers layer for bare metal in RPU is same as that of APU.

The key components of this bare metal stack are:

- Software drivers for peripherals including core routines needed for using the Arm® Cortex™-A53, Arm Cortex-R5F processors in the PS as well as the Xilinx MicroBlaze™ processors in the PL.
- Bare metal drivers for PS peripherals and optional PL peripherals.
- Standard C libraries: `libc` and `libm`, based upon the open source `Newlib` library, ported to the Arm Cortex-A53, Arm Cortex-R5F, and the MicroBlaze processors.
- Additional middleware libraries that provide networking, file system, and encryption support.
- Application examples including the first stage boot loader (FSBL) and test applications.

The C Standard Library (`libc`)

`libc` library contains standard functions that all C programs can use. Table 4-1 lists the `libc` modules:

Table 4-1: `libc.a` Functions and Descriptions

Header File	Description
<code>alloca.h</code>	Allocates space in the stack
<code>assert.h</code>	Diagnostics code
<code>ctype.h</code>	Character operations

Table 4-1: Libc.a Functions and Descriptions (Cont'd)

Header File	Description
<code>errno.h</code>	System errors
<code>inttypes.h</code>	Integer type conversions
<code>math.h</code>	Mathematics
<code>setjmp.h</code>	Non-local goto code
<code>stdint.h</code>	Standard integer types
<code>stdio.h</code>	Standard I/O facilities
<code>stdlib.h</code>	General utilities functions
<code>time.h</code>	Time function

The C Standard Library Mathematical Functions (libm)

Table 4-2 lists the `libm` mathematical C modules:

Table 4-2: libm.a Function Types and Function Listing

Function Type	Supported Functions
Algebraic	<code>cbrt</code> , <code>hypot</code> , <code>sqrt</code>
Elementary transcendental	<code>asin</code> , <code>acos</code> , <code>atan</code> , <code>atan2</code> , <code>asinh</code> , <code>acosh</code> , <code>atanh</code> , <code>exp</code> , <code>expm1</code> , <code>pow</code> , <code>log</code> , <code>log1p</code> , <code>log10</code> , <code>sin</code> , <code>cos</code> , <code>tan</code> , <code>sinh</code> , <code>cosh</code> , <code>tanh</code>
Higher transcendental	<code>j0</code> , <code>j1</code> , <code>jn</code> , <code>y0</code> , <code>y1</code> , <code>yn</code> , <code>erf</code> , <code>erfc</code> , <code>gamma</code> , <code>lgamma</code> , and <code>gamma_r</code>
Integral rounding	<code>ceil</code> , <code>floor</code> , <code>rint</code>
IEEE standard recommended	<code>copysign</code> , <code>fmod</code> , <code>ilogb</code> , <code>nextafter</code> , <code>remainder</code> , <code>scalbn</code> , and <code>fabs</code>
IEEE classification	<code>isnan</code>
Floating point	<code>logb</code> , <code>scalb</code> , <code>significand</code>
User-defined error handling routine	<code>matherr</code>

Standalone BSP

The libraries available with the standalone BSP are as follows:

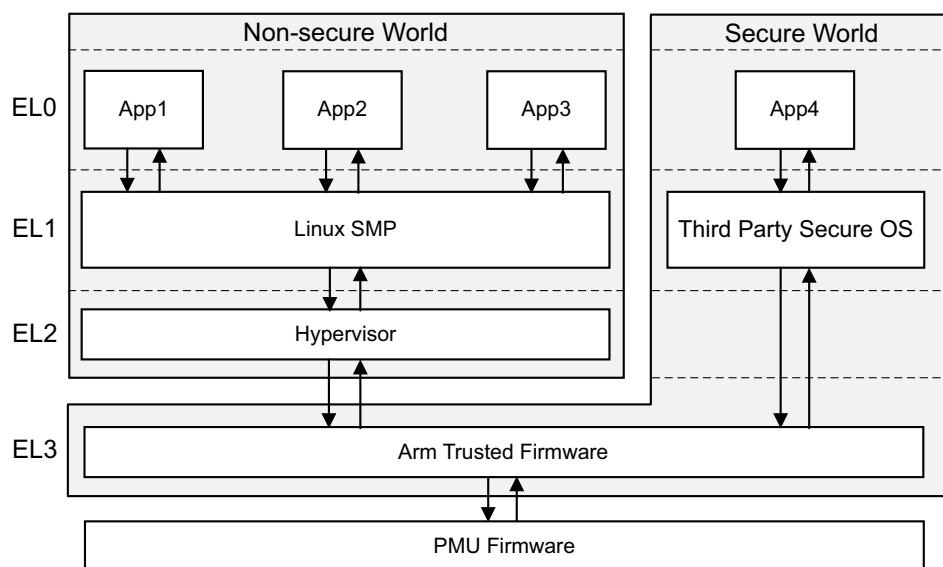
- `XilFatFS`: Is a `LibXil` FATFile system and provides read/write access to files stored on a Xilinx system ACE compact flash.
- `XilFFS`: Generic Fat File System Library.
- `XilFlash`: Xilinx flash library for Intel/AMD CFI compliant parallel flash.
- `XilISF`: In-System Flash library that supports the Xilinx in-system flash hardware.
- `XilMFS`: Memory file system.

- XilSecure: Xilinx Secure library provides support to access secure hardware (AES, RSA and SHA) engines.
- XilSkey: Xilinx secure key library.
- lwIP Library: An open source TCP/IP protocol suite that provides access to the core lwIP stack and BSD (Berkeley Software Distribution) sockets style interface to the stack.

These libraries are documented in [Appendix B, Xilinx Standard C Libraries](#).

Linux Software Stack

The Linux OS supports the Zynq UltraScale+ MPSoC device. With the sole exception of the Arm GPU, Xilinx provides open source drivers for all peripherals in the PS as well as key peripherals in the PL. The following figure illustrates the full software stack in APU, including Linux and an optional hypervisor.



X18968-07121

Figure 4-2: Linux Software Development Stack

The Armv8 exception model defines exception levels EL0–EL3, where:

- EL0 has the lowest software execution privilege. Execution at EL0 is called unprivileged execution.
- Increased exception levels, from 1 to 3, indicate an increased software execution privilege.
- EL2 provides support for processor virtualization. You may optionally include an open source or commercial hypervisor in the software stack.

- EL3 provides support for a secure state. The Cortex-A53 MPCore processor implements all the exception levels (EL0-EL3) and supports both execution states (AArch64 and AArch32) at each exception level.

You can leverage the Linux software stack for the Zynq UltraScale+ MPSoC device in multiple ways. The following are some of your options:

- **PetaLinux Tools:** The PetaLinux tools include a branch of the Linux source tree, U-Boot as well as Yocto-based tools to make it easy to build complete Linux images including the kernel, the root file system, device tree, and applications for Xilinx devices. See the *PetaLinux Product Page* [\[Ref 2\]](#) for more information. The PetaLinux tools work with the same open source Linux components described immediately below.
- **Open Source Linux and U-Boot:** The Linux Kernel sources including drivers, board configurations, and U-Boot updates for the Zynq UltraScale+ MPSoC device are available from the Xilinx *Github* link [\[Ref 31\]](#), and on a continuing basis from the main Linux kernel and U-Boot trees as well. Yocto board support packages are also available from the main Yocto tree.
- **Commercial Linux Distributions:** Some commercial distributions also include support for Xilinx UltraScale+ MPSoC devices and they include advanced tools for Linux configuration, optimization, and debug. You can find more information about these from the Xilinx *Embedded Computing* page [\[Ref 32\]](#).

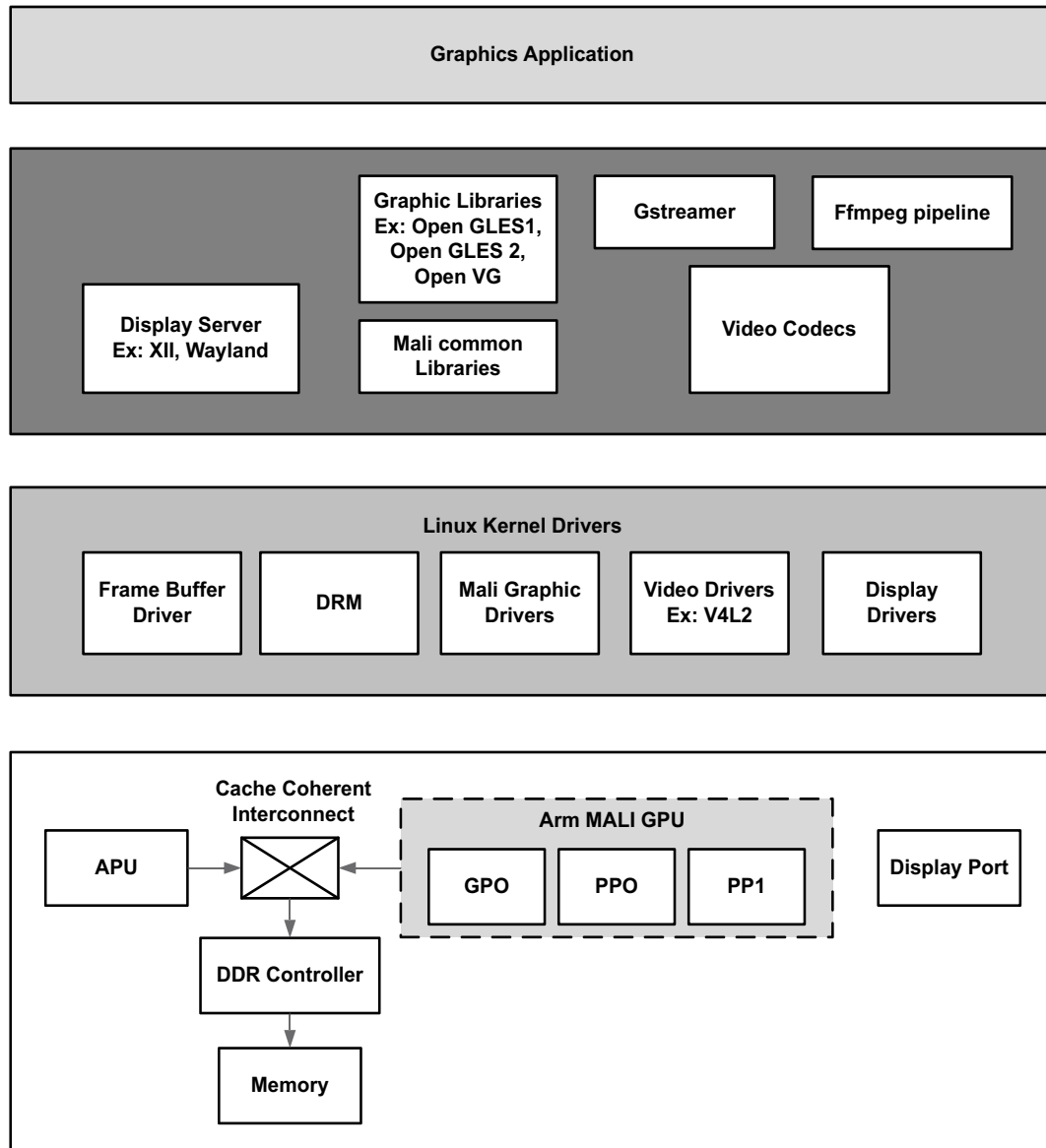
Multimedia Stack Overview

This section describes the multimedia software stack in the Zynq UltraScale+ MPSoC device.

The GPU and a high performance DisplayPort accelerate the graphics application. The GPU provides hardware acceleration for 2D and 3D graphics by including one geometry processor (GP) and two pixel processors (PP0 and PP1), each having a dedicated memory management unit (MMU). The cache coherency between the APU and the GPU is achieved by cache-coherent interconnect (CCI), which supports the AXI coherency extension (ACE) only.

CCI in-turn connects the APU and the GPU to the DDR controller, which arbitrates the DDR access.

The following figure shows the multimedia stack.



X14795-071317

Figure 4-3: Multimedia Stack

The Linux kernel drivers for multimedia enables the hardware access by the applications running on the processors.

Table 4-3 lists the multimedia drivers through the middleware stack that consists of the libraries and framework components the applications use.

Table 4-3: Libraries and Framework Components

Component	Description
Display server	Coordinates the input and output from the applications to the operating system.
Graphics library	The Zynq UltraScale+ MPSoC device architecture supports OpenGL ES 1.1 and 2.2, and Open VG 1.1.
Mali-400 MP2 common libraries	Mali-400 MP2 graphic libraries. For more details on how to switch between different EGL backends, refer to Xilinx MALI Driver .
Gstreamer	A freeware multimedia framework that allows a programmer to create a variety of media handling components.
Video codecs	Video encoders and decoders.

Table 4-4 lists the Linux kernel graphics drivers.

Table 4-4: Linux Kernel Drivers

Drivers	Description
Frame buffer driver	Kernel graphics driver exposing its interface through <code>/dev/fb*</code> . This interface implements limited functionality (allowing you to set a video mode and drawing to a linear frame buffer).
Direct rendering manager (DRM)	Serves in rendering the hardware between multiple user space components.
MALI-400 MP2 graphics drivers	Provides the hardware access to the GPU hardware.
Video drivers	Video capture and output device pipeline drivers based on the V4L2 framework. The Xilinx Linux V4L2 pipeline driver represents the whole pipeline with multiple sub-devices. You can configure the pipeline through the media node, and you can perform control operations, such as stream on/off, through the video node. Device nodes are created by the pipeline driver. The pipeline driver also includes the wrapper layer of the DMA engine API, and this enables it to read/write frames from RAM.
Display port drivers	Enables the hardware access to the display port, based on DRM framework.

FreeRTOS Software Stack

Xilinx provides a FreeRTOS board support package (BSP) as a part of the Xilinx SDK tool. The FreeRTOS BSP provides you a simple, multi-threading environment with basic features such as, standard input/output and access to processor hardware features. The BSP and the included libraries are highly configurable to provide you the necessary functionality with the least overhead. The FreeRTOS software stack is similar to the bare metal software stack, except that it contains the FreeRTOS library. Xilinx device drivers included with the standalone libraries can typically be used within FreeRTOS provided that only a single thread requires access to the device. Xilinx bare metal drivers are not aware of Operating Systems. They do not provide any support for mutexes to protect critical sections, nor do they provide any mechanism for semaphores to be used for synchronization. While using the driver API with FreeRTOS kernel, you must take care of this aspect.

The following figure illustrates the FreeRTOS software stack for RPU.

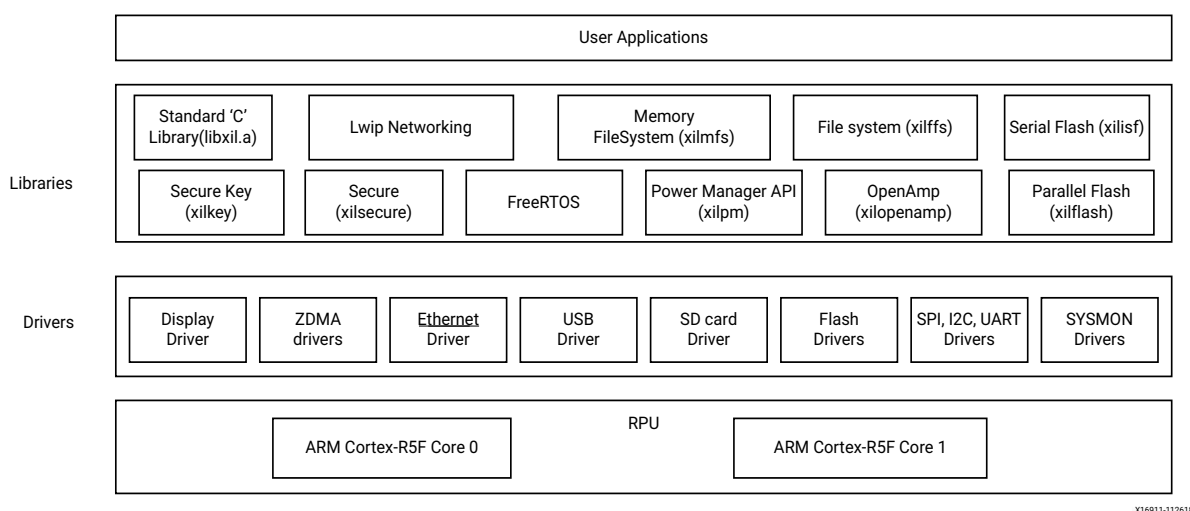


Figure 4-4: FreeRTOS Software Stack

Note: The FreeRTOS software stack for APU is same as that for RPU except that the libraries support both 32-bit and 64-bit for APU.

Third-Party Software Stack

Many other embedded software solutions are also available from the Xilinx partner ecosystem. More information is available from the Xilinx website, *Embedded Computing* [Ref 32] and the website, *Xilinx Third Party Tools* [Ref 4].

Software Development Flow

Overview of Software Development Flow

This chapter explains the bare metal software development for RPU and APU using the Xilinx® Software Development Kit (SDK) as well as Linux software development for APU using PetaLinux tools and SDK.

The following figure depicts the top-level software architecture of the Zynq® UltraScale+™ MPSoC device.

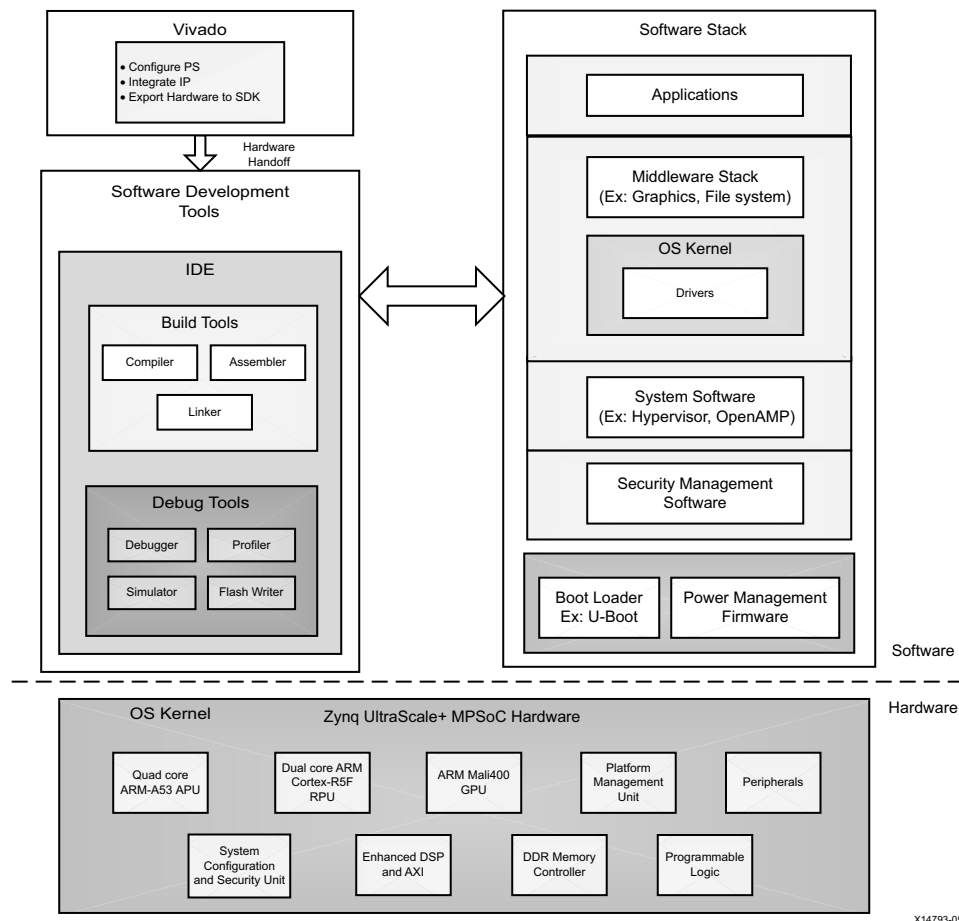
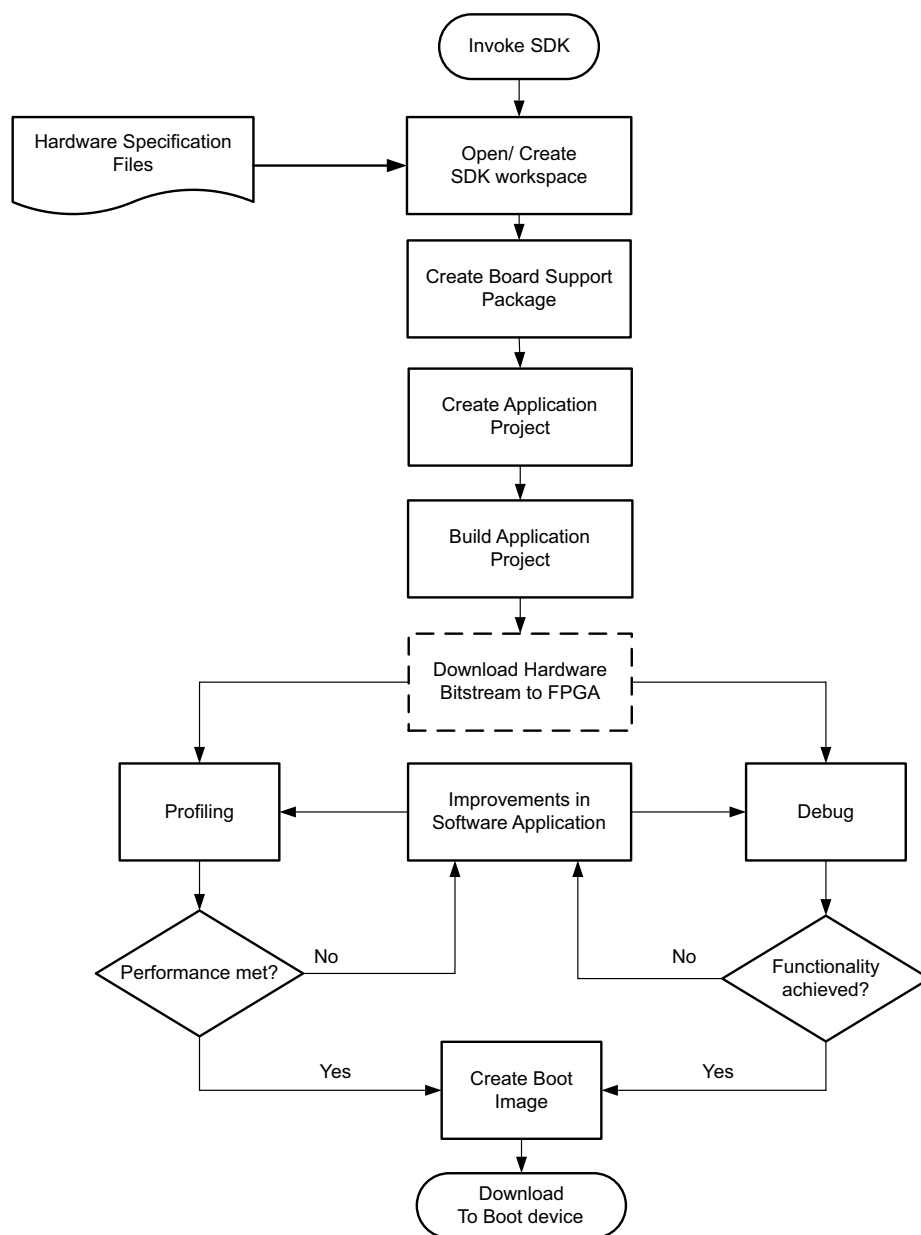


Figure 5-1: Software Development Architecture

Bare Metal Application Development

This section assists you in understanding the design flow of bare metal application development for APU and RPU using SDK.

The following figure shows the top-level design flow in SDK.



X14817-071217

Figure 5-2: Bare Metal Application Development Flow

Developing bare metal applications involves the following steps:

1. Opening and creating an SDK workspace for bare metal applications: See [Creating a Standalone Application Project](#).
2. Importing the hardware platform information: See [Importing a Hardware Platform Specification File](#).
3. Select target processor (Cortex-A53, Cortex-R5F, or PMU MicroBlaze™).
4. Creating a board support package (BSP): See [Creating a Board Support Package](#).
5. Alter the BSP configuration settings (optional): See [Changing Build Configuration](#).
6. Adding custom IP driver support (optional): See [Using the Board Support Package Drivers Page](#).
7. Creating application projects: See [Creating Application Projects](#).
8. Building application projects: See [Building Projects](#).
9. Debugging user applications: See [Debugging Projects](#).
10. Running user applications: See [Running Projects](#)
11. Profiling user applications: See [Software Profiling](#).
12. Modeling system performance: See [System Performance Modeling](#).
13. Creating boot image: See [Creating a Boot Image](#).

For more details on these steps, see the *MPSoC PetaLinux Software Development* [\[Ref 34\]](#).

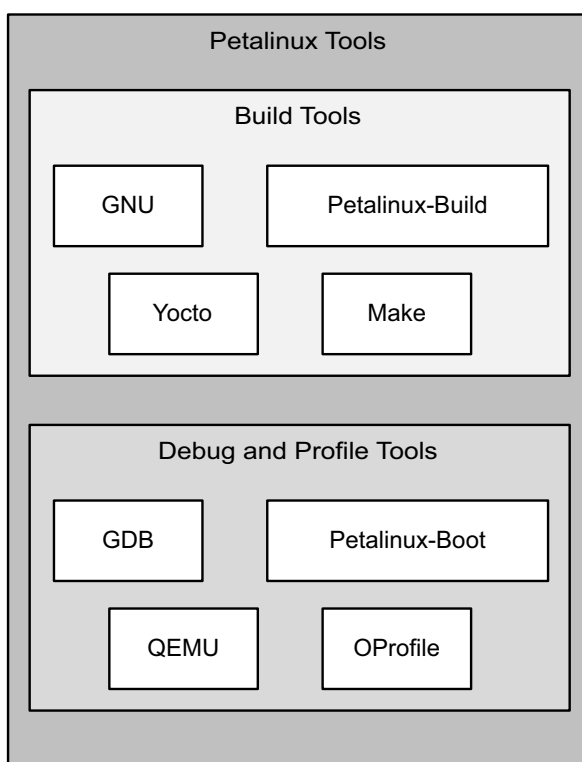
For more details on QEMU, see the *Zynq UltraScale+ MPSoC QEMU User Guide (UG1169)* [\[Ref 8\]](#).

Note: Cortex-R5F and Cortex-A53 32-bit bare metal software do not support 64-bit addressed data transfer using device DMA.

Note: By default, all standalone applications will run only on APU0. The other APU cores will be off.

Application Development Using PetaLinux Tools

Software development flow in the PetaLinux tools environment involves many stages. To simplify understanding, the following figure shows a chart with all the stages in the PetaLinux tools application development.



X14815-063017

Figure 5-3: PetaLinux Tool-Based Software Development Flow

Linux Application Development Using SDK

Xilinx software design tools facilitate the development of Linux user applications. This section provides an overview of the development flow for Linux application development.

The following figure illustrates the typical steps involved to develop Linux user applications using SDK.

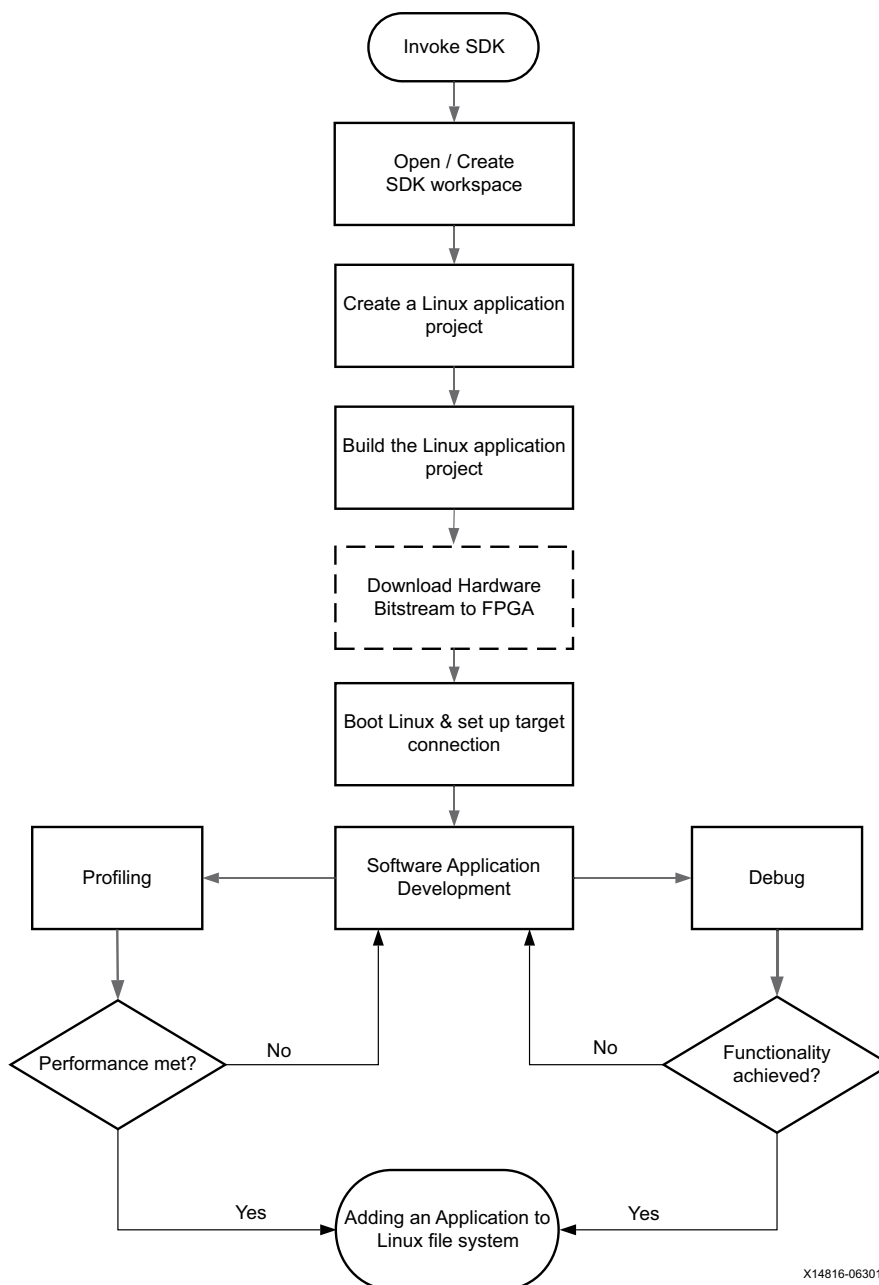


Figure 5-4: Linux Application Development Flow

The development and execution of a PetaLinux application involve the following steps:

1. Setting up the PetaLinux tools working environment: See [PetaLinux Working Environment Setup](#).
2. Creating a PetaLinux project or a user application: See [Create a New PetaLinux Project](#).
3. Configuring and customizing the PetaLinux tools for your project
4. Importing a SDK Linux C/C++ application project into the PetaLinux workspace: See [PetaLinux Tools Installation Steps](#).
5. Building a PetaLinux image: See [Build System Image](#).
6. Running a PetaLinux image on a platform (QEMU or Board): See [Building User Applications](#).
7. Debugging a PetaLinux image. There are several options for debugging. See the *PetaLinux Tools Documentation Reference Guide* (UG1144) [\[Ref 27\]](#).

For more details on each of above steps, see the *MPSoC PetaLinux Software Development* link [\[Ref 34\]](#).

For more details on QEMU, see the *Zynq UltraScale+ MPSoC QEMU User Guide* (UG1169) [\[Ref 8\]](#).

For a detailed explanation of the SDK features, and to understand the SDK design flow with a "Hello World" example, see the *SDK Help* [\[Ref 24\]](#). Also, see *Xilinx Software Development Kit: System Performance* (UG1145) [\[Ref 28\]](#).

Creating an Application Project

SDK provides a template-based application generator for included sample programs, from a basic "Hello World," an empty application, or an FSBL application. The Xilinx C or C++ application wizard invokes the application generator.

You can also create an empty application or import existing Linux applications for porting. Code development tools include editors, search, re-factoring, and features available in the base Eclipse platform and CDT plug-in.

Building the Application

SDK application projects can be user-managed (user-created makefiles) or automatically managed (SDK created makefiles). For user-managed projects, you can maintain the makefile and initiate application builds. For automatically managed projects, SDK updates the makefile as needed when source files are added or removed, source files are compiled when changes are saved and the ELF is built automatically; in Eclipse CDT terminology, the application project is a managed makefile project. Where possible, SDK infers or sets default build options based on the hardware platform and BSP used, including compiler, linker, and library path options.

Running the Application

You can create an SDK run configuration to copy the compiled application to the file system and run the application. With Linux running on the Zynq UltraScale+ MPSoC device platform, the run configuration copies the executable to the file system using `sftp` if the Linux environment includes `SSH`. A terminal view is available to interact with the application using `STDIN` and `STDOUT`.

You can also run the application using a command line shell. Use the following commands as needed:

- `sftp` to copy the executable
- `ssh` in Linux to run the executable

Adding Driver Support for Custom IP in the PL

SDK supports Linux BSP generation for peripherals in the PS as well as custom IP in the PL. When generating a Linux BSP, SDK produces a device tree, which is a data structure describing the hardware system that passes to the kernel when you boot.

Device drivers are available as part of the kernel or as separate modules, and the device tree defines the set of hardware functions available and features enabled.

Additionally, you can add dynamic, loadable drivers. The Linux kernel supports these drivers. Custom IP in the PL are highly configurable, and the device tree parameters define both the set of IP available in the system and the hardware features enabled in each IP.

See [Chapter 3, Development Tools](#), for additional overview information on the Linux Kernel and boot sequence.

Adding an Application to a Linux File System

You can add the compiled user application, and the required shared libraries to the Linux file system, as follows:

- While Linux is running on the Zynq UltraScale+ MPSoC device platform, you can copy the files using `sftp` if the Linux environment includes `SSH`.
- In SDK, a remote system explorer (RSE) plug-in lets you copy files using a drag-and-drop mechanism.
- In workflows outside of SDK, add the application and the libraries to the file system folder before creating the file system image and programming it to flash.

For more details on developing a Linux application, see “Creating a Linux Application Project” in the *SDK Help* [\[Ref 24\]](#).

Software Design Paradigms

Introduction

The Xilinx® Zynq® UltraScale+™ MPSoC device architecture supports heterogeneous multiprocessor engines targeted at different tasks. The main approaches for developing software to target these processors are by using the following:

- **Frameworks for Multiprocessor Development:** Describes the frameworks available for development on the Zynq UltraScale+ MPSoC device.
- **Symmetric Multiprocessing (SMP):** Using SMP with PetaLinux is the most simple flow for developing an SMP with a Linux platform for the Zynq UltraScale+ MPSoC device.
- **Asymmetric Multiprocessing (AMP):** AMP is a powerful mode to use multiple processor engines with precise control over what runs on each processor. Unlike SMP, there are many different ways to use AMP. This section describes two ways of using AMP with varying levels of complexity.

The following sections describe these development methods in more detail.

Frameworks for Multiprocessor Development

Xilinx provides multiple frameworks for Zynq UltraScale+ MPSoC devices to facilitate the application development on the heterogeneous processors and Xilinx 7 series FPGAs. The following bullets explain these frameworks:

- **Hypervisor Framework:** Xilinx provides the Xen hypervisor, a critical item needed to support virtualization on APU of Zynq UltraScale+ MPSoC. The [Use of Hypervisors](#) section covers more details.
- **Authentication Framework:** The Zynq UltraScale+ MPSoC device supports authentication and encryption features as a part of authentication framework. To understand more about the authentication framework, see [Boot Time Security in Chapter 8](#).

- **TrustZone Framework:** The TrustZone technology allows and maintains isolation between secure and non-secure processes within the same system.

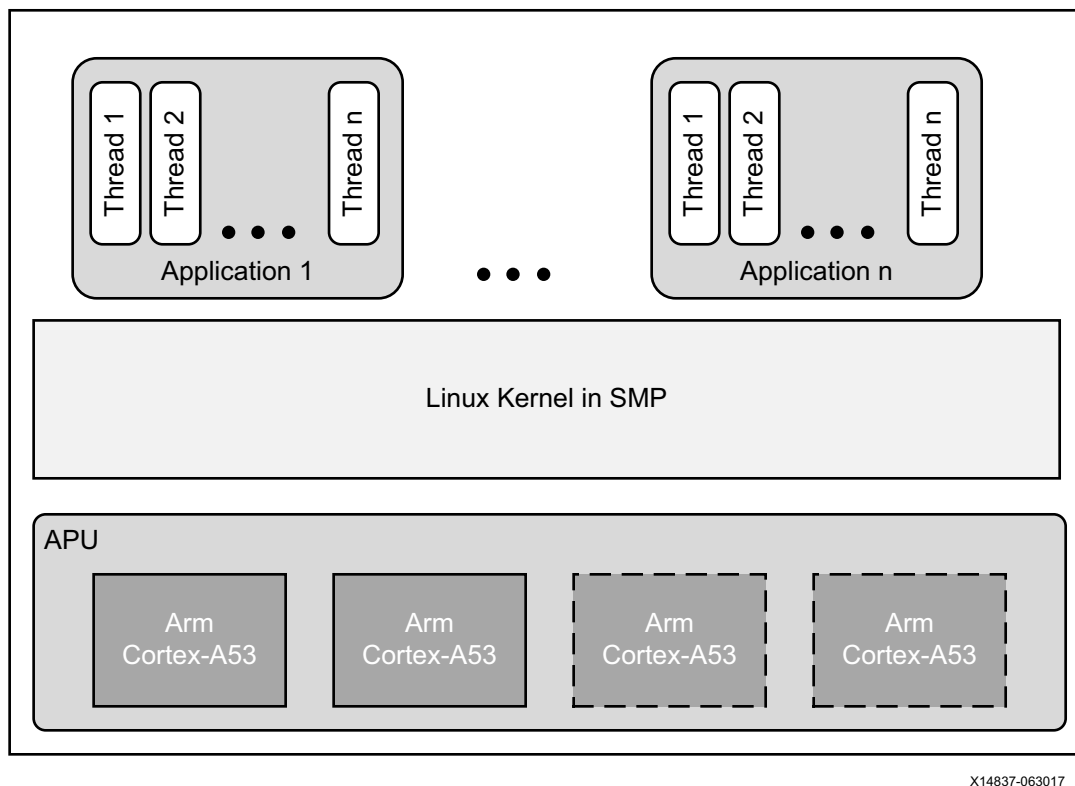
Xilinx provides the trustzone support through the Arm Trusted Firmware (ATF) to maintain the isolation between secure and non-secure worlds. To understand more about ATF, see [Arm Trusted Firmware in Chapter 8](#).

- **Multiprocessor Communication Framework:** Xilinx provides the OpenAMP framework for Zynq UltraScale+ MPSoC devices to allow communication between the different processing units. For more details, see the Zynq UltraScale+ MPSoC *OpenAMP Framework for Zynq Devices: Getting Started Guide* (UG1169) [\[Ref 16\]](#).
- **Power Management Framework:** The power management framework allows software components running across different processing units to communicate with the power management unit.

Symmetric Multiprocessing (SMP)

SMP enables the use of multiple processors via a single operating system instance. The operating system handles most of the complexity of managing multiple processors, caches, peripheral interrupts, and load balancing.

The APU in the Zynq UltraScale+ MPSoC devices contains four homogeneous cache coherent Arm Cortex™-A53 processors that support SMP mode of operation using an OS (Linux or VxWorks). Xilinx and its partners provide operating systems that make it easy to leverage SMP in the APU. The following diagram shows an example of Linux SMP with multiple applications running on a single OS.



X14837-063017

Figure 6-1: Example SMP Using Linux

This would not be the best mode of operation when there are hard, real-time requirements as it ignores Linux application core affinity which should be available to developers with the existing Xilinx software.

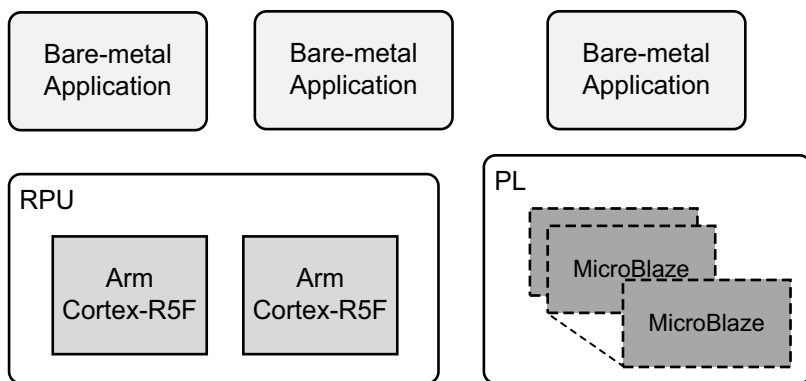
Asymmetric Multiprocessing (AMP)

AMP uses multiple processors with precise control over what runs on each processor. Unlike SMP, there are many different ways to use AMP. This section describes two ways of using AMP with varying levels of complexity.

In AMP, a software developer must decide what code has to run on each processor before compiling and creating a boot image that includes the software executable for each CPU. Using AMP with the Arm Cortex-R5F processors (SMP is not supported in Cortex-R5F) in the RPU enables developers to meet highly demanding, hard real-time requirements as opposed to soft real-time requirements.

You can develop the applications independently, and program those applications to communicate with each other using inter-processing communication (IPC) options. See this [link](#) to the “Interrupts” chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11] for further description of this feature.

You can also apply this AMP method to applications running on MicroBlaze™ processors in the PL or even in the APU. The following diagram shows an AMP example with applications running on the RPU and the PL without any communication with each other.



X19225-071317

Figure 6-2: AMP Example using Bare-Metal Applications Running on RPU and PL

OpenAMP

The OpenAMP framework provides mechanisms to do the following:

- Load and unload firmware
- Communicate between applications using a standard API

The following diagram shows an example of an OpenAMP and the hard real-time capabilities of the RPU using the OpenAMP framework.

In this case, Linux applications running on the APU perform the loading and unloading of RPU applications. This allows developers to load different processing dedicated algorithms to the RPU processing engines as needed with very deterministic performance.

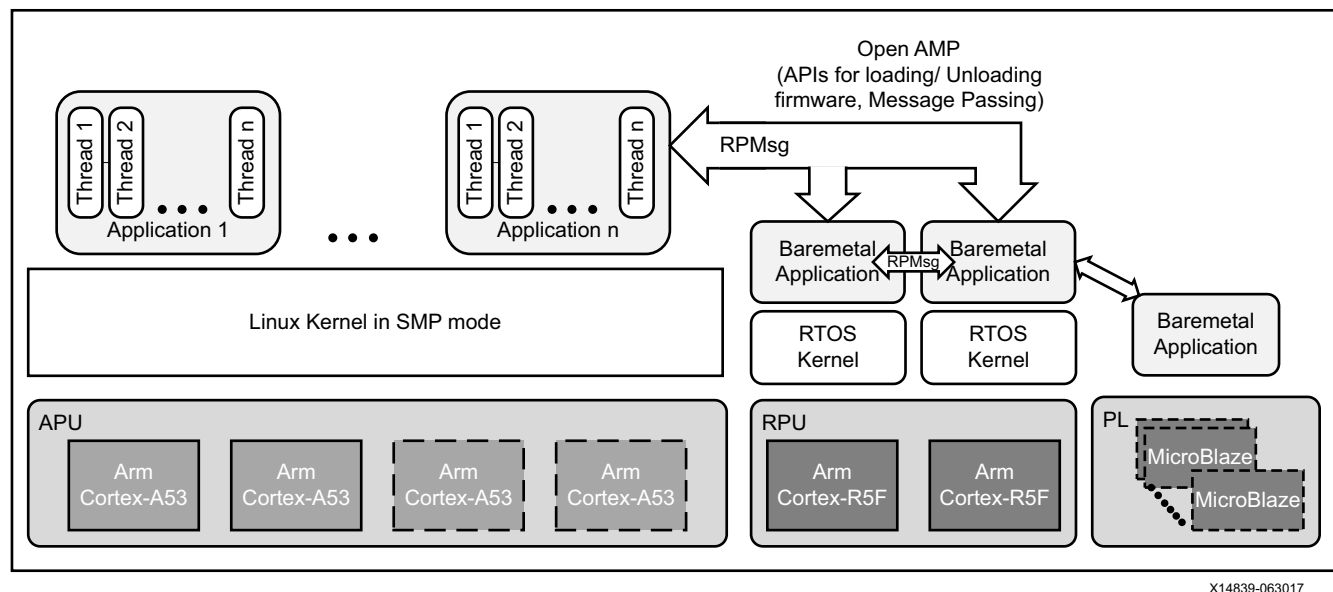


Figure 6-3: Example with SMP and AMP using OpenAMP Framework

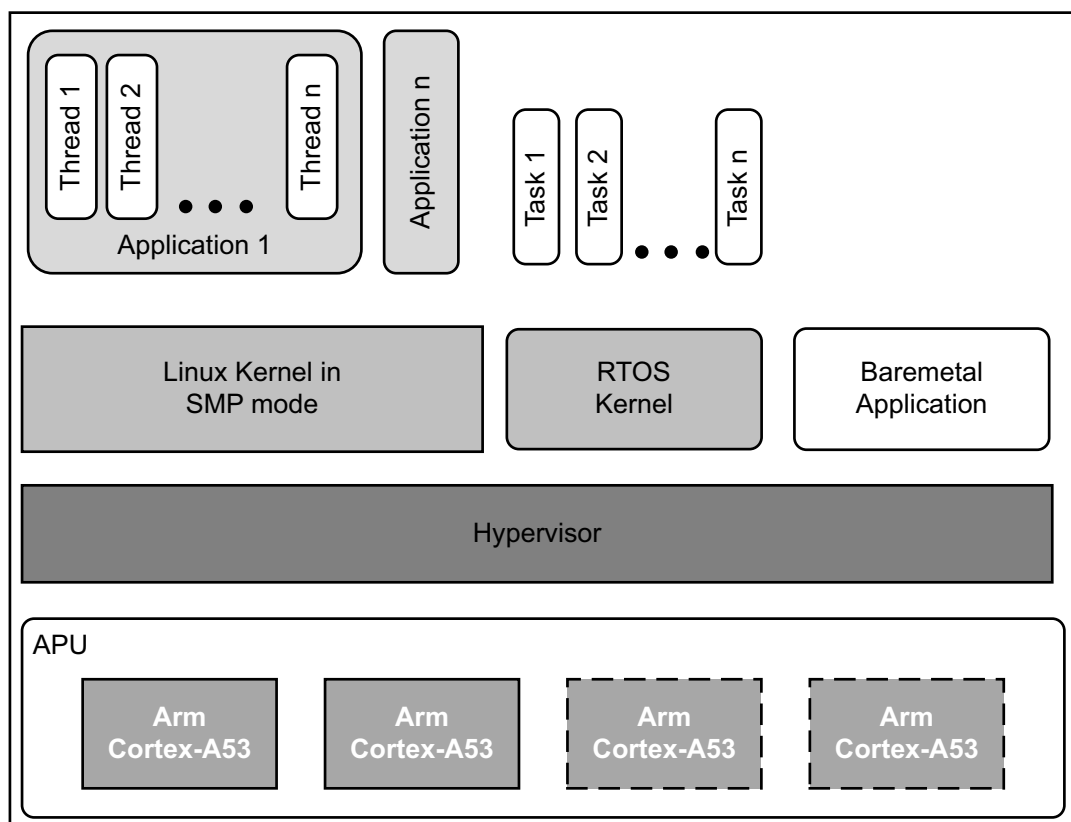
See the *Zynq UltraScale+ MPSoC OpenAMP Getting Started Guide* (UG1186) [Ref 16] for more information about the OpenAMP Framework.

Virtualization with Hypervisor

The Zynq UltraScale+ MPSoC devices include a hardware virtualization extension on the Arm Cortex-A53 processors, interrupt controller, and Arm System MMU (SMMU) that provides flexibility to combine various operating system combinations, including SMP and AMP, within the APU.

The following diagram shows an example of an SMP-capable OS, like Linux working along with Real-Time Operating System (RTOS) as well as a bare metal application using a single hypervisor.

This enables independent development of applications in their respective mode of operation.



X14840-063017

Figure 6-4: Example with Hypervisor

Although the hardware virtualization included within Zynq UltraScale+ MPSoC and its hypervisors allow the standard operating systems and their applications to function with low to moderate effort, the addition of a hypervisor does bring design complexity to low-level system functions such as power management, FPGA bitstream management, OpenAMP software stack, and security accelerator access which must utilize additional underlying layers of system firmware. Hence, Xilinx recommends that the developers must initiate early effort into these aspects of system architecture and implementation.

For more details on using Hypervisor like the Xen Hypervisor, see *MPSoC Xen Hypervisor* website [Ref 41].

Use of Hypervisors

Xilinx distributes a port for the Xen open source hypervisor in the Xilinx Zynq UltraScale+ MPSoC device. Xen hypervisor provides the ability to run multiple operating systems on the same computing platform. Xen hypervisor, which runs directly on the hardware, is responsible for managing CPU, memory, and interrupts. Multiple numbers of OS can run on top of the hypervisor. These operating systems are called *domains* (also called as *virtual machines* (VMs)).

The Xen hypervisor provides the ability to concurrently run multiple operating systems and their standard applications with relative ease. However, Xen does not provide a generic interface which gives the guest an operating system access to system functions. Hence, you need to follow the cautions mentioned in this section.

Xen hypervisor controls one domain, which is domain 0, and one or more guest domains. The control domain has special privileges, such as the following:

- Capability to access the hardware directly
- Ability to handle access to the I/O functions of the system
- Interaction with other virtual machines.

It also exposes a control interface to the outside world, through which the system is controlled. Each guest domain runs its own OS and application. Guest domains are completely isolated from the hardware.

Running multiple Operating Systems using Xen hypervisor involves setting up the host OS and adding one or more guest OS.

Note: Xen hypervisor is available as a selectable component within the PetaLinux tools; Xen hypervisor can also be downloaded from Xilinx GIT. With Linux and Xen software that is made available by Xilinx, it is possible to build custom Linux guest configurations. Guest OS other than Linux require additional software and effort from third-parties. See the *PetaLinux Product Page* [Ref 2].

System Boot and Configuration

Introduction

Zynq® UltraScale+™ MPSoC devices support the ability to boot from different devices such as a QSPI flash, an SD card, USB Device Firmware Upgrade (DFU) host, and the NAND flash drive. This chapter details the booting process using different booting devices in both secure and non-secure modes.

Boot Process Overview

The platform management unit (PMU) and configuration security unit (CSU) manage and perform the multi-staged booting process. You can boot the device in either secure (using authenticated boot image) or non-secure (using an unauthenticated boot image) mode. The boot stages are as follows:

- Pre-configuration stage: The PMU primarily controls pre-configuration stage that executes PMU ROM to setup the system. The PMU handles all of the processes related to reset and wake-up.
 - Configuration stage: This stage is responsible for loading the first-stage boot loader (FSBL) code for the PS into the on-chip RAM (OCM). It supports both secure and non-secure boot modes. Through the boot header, you can execute FSBL on the Cortex-R5F processor or the Cortex-A53 processor. In the Cortex-R5F processor, lock-step is also supported.
 - Post-configuration stage: After FSBL execution starts, the Zynq UltraScale+ MPSoC device enters the post configuration stage.
-

Boot Flow

There are two boot flows in the Zynq UltraScale+ MPSoC architecture: secure and non-secure. The following sections describe some of the example boot sequences in which you bring up various processors and execute the required boot tasks.

Note: The figures in these sections show the complete boot flow, including all mandatory and optional components.

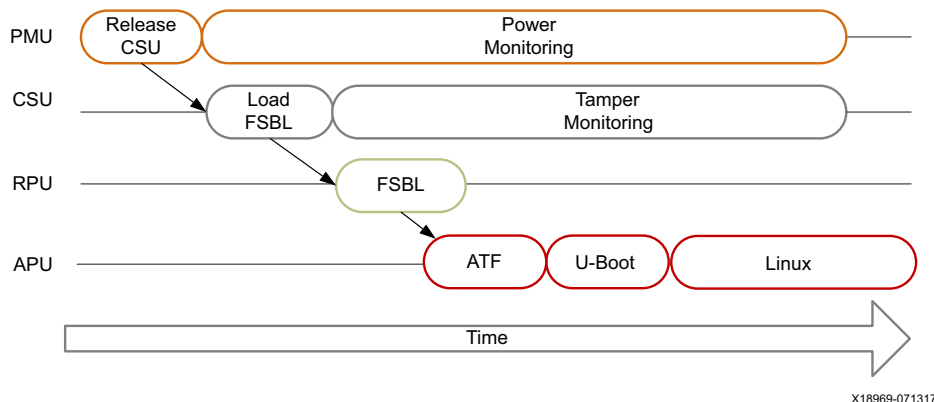


Figure 7-1: Boot Flow Example

Non-Secure Boot Flow

In non-secure boot mode, the PMU releases the reset of the configuration security unit (CSU), and enters the PMU server mode where it monitors power. After the PMU releases the CSU from its reset, it loads the FSBL into OCM. PMU firmware loads into PMU RAM. PMU RAM will execute from PMU RAM in parallel to FSBL in OCM. In this example, FSBL is loaded into RPU and CTF. U-Boot and Linux are loaded into the APU. You can load the FSBL into either RPU or APU. Other boot configurations allow the RPU to start and operate wholly independent of the APU and vice-versa.

- In APU, ATF will be executed after the FSBL handoff to ATF. ATF handoff, typically a second stage boot loader like U-Boot executes and loads an OS, such as Linux.
- In RPU, FSBL hands off to a software application.
- Linux, in turn, loads the executable software.

Note: The OS manages the multiple Cortex™-A53 processors in symmetric multi-processing (SMP) mode.

Secure Boot Flow

In the secure boot mode, the PMU releases the reset of the configuration security unit (CSU) and enters the PMU server mode where it monitors power.

After the PMU releases the CSU from reset, the CSU checks to determine if authentication is required by the FSBL or the user application.

The CSU does the following:

- Performs an authentication check and proceeds only if the authentication check passes. Then checks the image for any encrypted partitions.
- If the CSU detects partitions that are encrypted, the CSU performs decryption and loads the FSBL into the OCM.

For more information on CSU, see the [Configuration Security Unit](#) section.

In the APU, the FSBL hands off to ATF. ATF then executes the U-Boot and loads an OS such as Linux. Then Linux, in turn, loads the executable software. Similarly, FSBL checks for authentication and encryption of each partition it tries to load. The partitions are only loaded by FSBL on successful authentication and decryption (if previously encrypted).

Note: In the secure boot mode, `psu_coresight_0` is not supported as a stdout port.

Boot Image Creation

The Bootgen utility, which is available as a part of the SDK, creates a single boot image file suitable for booting applications developed in the Zynq UltraScale+ MPSoC device. Bootgen creates the image by building the required boot header, appending tables that describe the following partitions, and processing the input data files (ELF files, FPGA bitstreams, and other binary files) to partitions. Bootgen has features for assigning specific destination memory addresses or imposing alignment requirements for each partition. Bootgen also supports the encryption, authentication, and performing checksums on each partition.

The utility is driven by a configuration file known as the boot image format (BIF) file with a file extension of `*.bif`.



IMPORTANT: *The .bif file must contain the bitstream above ATF.*

For advanced authentication flows, you can use the Bootgen utility to output intermediate hash files that can be signed offline. Otherwise, Bootgen uses the provided private keys to sign the authentication certificates included in the boot image.

Building a boot image involves the following steps:

1. Create a BIF file.
2. Run the Bootgen utility to create a binary file.
3. (For QEMU): Convert the binary file to an image format corresponding to the boot device.

For more information regarding Bootgen, see [Chapter 16, Boot Image Creation](#).

Boot Modes

See [Table 7-4](#) for a brief list of available boot modes. Refer to this [link](#) to the “Boot and Configuration” chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11] for a comprehensive table of the available boot modes.

QSPI24 and QSPI32 Boot Modes

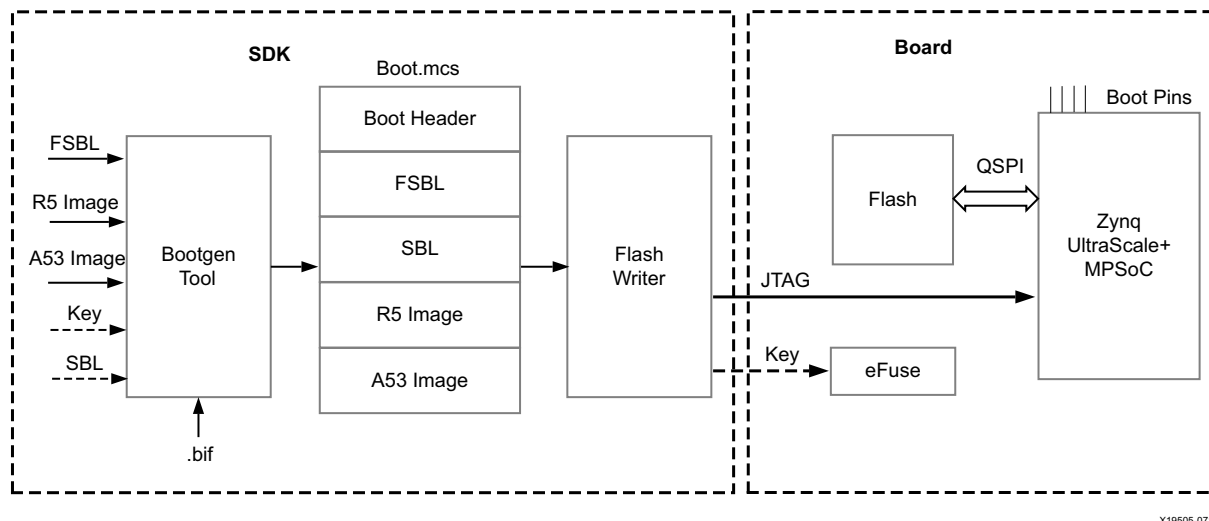
The QSPI24 and QSPI32 boot modes support the following:

- x1, x2, and x4 read modes for single Quad SPI flash memory 24 (QSPI24) and single Quad SPI flash memory 32 (QSPI32)
- x8 read mode for dual QSPI.
- Image search for MultiBoot
- I/O mode for BSP drivers (no support in FSBL)

The BootROM searches the first 256 Mb in x8 mode. In QSPI24 and QSPI32 boot modes (where the QSPI24/32 device is < 128 Mb), to use MultiBoot, place the multiple images so that they fit in memory locations less than 128 Mb. The pin configuration for QSPI24 boot mode is 0x1.

Note: QSPI dual stacked (x8) boot is not supported. Only QSPI Single Transmission Rate (STR) is supported. Single Quad-SPI memory (x1, x2 and x4) is the only boot mode that supports execute-in-place (XIP).

The following figure shows an example for booting in QSPI mode.



X19505-071317

Figure 7-2: Booting in QSPI Mode

To create a QSPI24/QSPI32 boot image, provide the following files to the Bootgen tool:

- An FSBL ELF
- A secondary boot loader (SBL), such as U-Boot, or a Cortex-R5F and/or a Cortex-A53 application ELF
- Authentication and encryption key (optional)

For more information on Authentication and Encryption, see [Chapter 8, Security Features](#).

Bootgen generates the `boot.mcs` and a `boot.bin` binary file that you can write into the QSPI24/QSPI32 flash using the flash writer. MCS is an Intel hex-formatted file that includes an checksum for reliability.

Note: The pin configuration for QSPI24 boot mode is 0x2.

SD Boot Mode

SD boot (version 3.0) supports the following:

- FAT 16/32 file systems for reading the boot images.
- Image search for MultiBoot with a maximum number of files for MultiBoot are 8,192.

The following figure shows an example for booting Linux in SD mode.

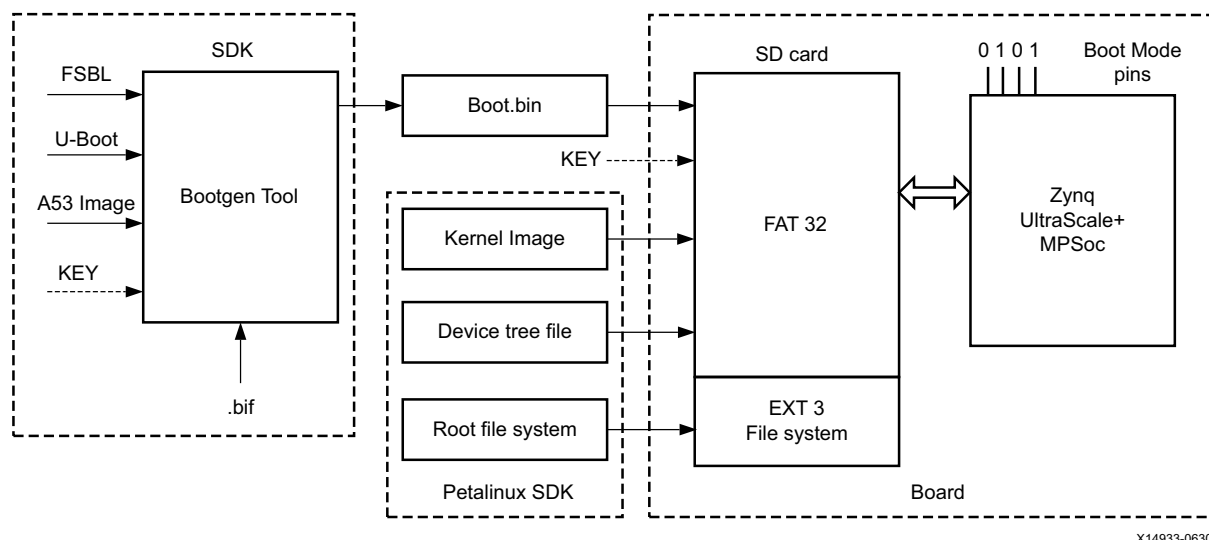


Figure 7-3: Booting in SD Mode

To create an SD boot image, provide the following files to Bootgen:

- An FSBL ELF
- A Cortex-R5F and/or an Cortex-A53 application ELF
- Optional authentication and encryption key

The Bootgen tool generates the `boot.bin` binary file. You can write the `boot.bin` file into an SD card using a SD card reader.

In PetaLinux, do the following:

1. Build the Linux kernel image, device tree file, and the root file system.
2. Copy the files into the SD card.

The formatted SD card then contains the `boot.bin`, the kernel image, and the device tree file in the **FAT32** partition; the root file system resides in the **EXT 3** partition.



IMPORTANT: To boot from SD1, configure the boot pins to 0x5. To boot from SD0, configure the boot pins to 0x3. To boot from SD with a level shifter, configure the boot pins to 0xE.

eMMC18 Boot Mode

eMMC18 boot (version 4.5) supports the following:

- FAT 16/32 file systems for reading the boot images.
- Image search for MultiBoot with a maximum number of files for MultiBoot are 8,192.

The following figure shows an example for booting Linux in eMMC18 mode.

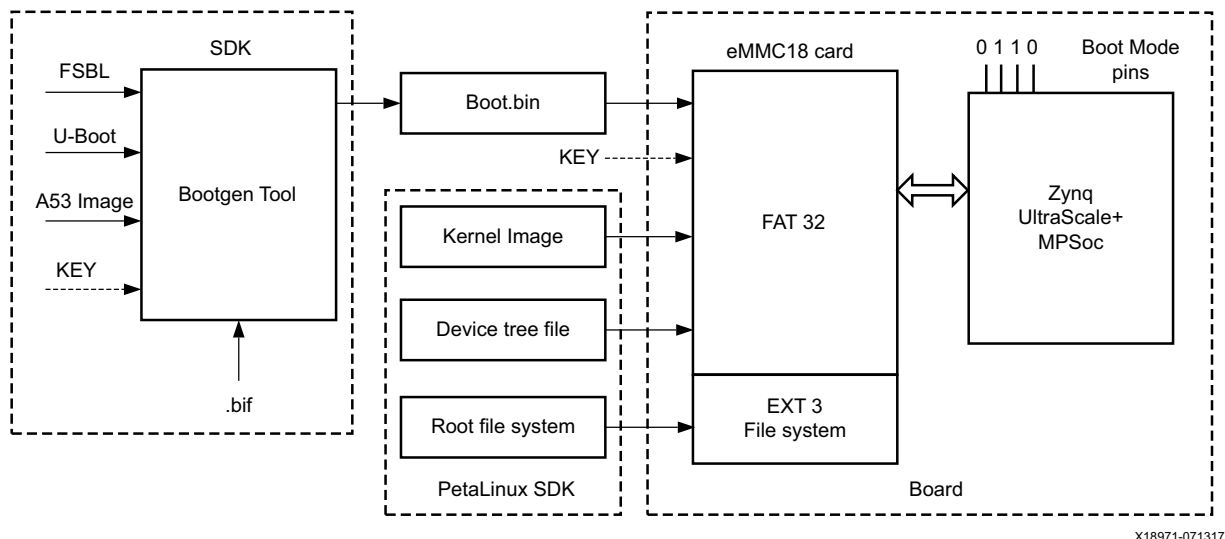


Figure 7-4: Booting in eMMC18 Mode

To create an eMMC18 boot image, provide the following files to Bootgen:

- An FSBL ELF
- A Cortex-R5F and/or a Cortex-A53 application ELF
- Optional authentication and encryption key

The Bootgen tool generates the boot.bin binary file. You can write the boot.bin file into an eMMC18 card using an eMMC18 card reader.

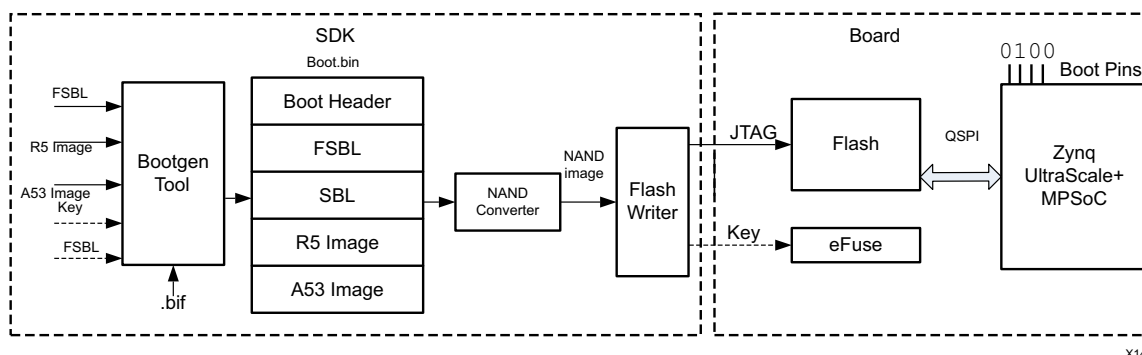
In PetaLinux, do the following:

- Build the Linux kernel image, device tree file, and the root file system.
- Copy the files into the eMMC18 card.

The formatted eMMC18 card then contains the boot.bin, the kernel image, and the device tree files in the FAT32 partition; the root file system resides in the EXT3 partition.

NAND Boot Mode

The NAND boot only supports 8-bit widths for reading the boot images, and image search for MultiBoot. The following figure shows an example for booting Linux in NAND mode.



X14934-071317

Figure 7-5: Booting in NAND Mode

To create a NAND boot image, provide the following files to Bootgen:

- An FSBL ELF
- A Cortex-R5F application ELF and/or an Cortex-A53 application ELF
- Optional authentication/encryption key

The Bootgen tool generates the `boot.bin` binary file. You can then write the NAND bootable image into the NAND flash using the flash writer.



IMPORTANT: To boot from NAND, configure the boot pins to `0x4`.

JTAG Boot Mode

You can individually download any software images needed for the PS and hardware images for the PL using JTAG.

For JTAG boot mode settings, see this [link](#) in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11]



IMPORTANT: Secure boot is not supported in the JTAG mode.

USB Boot Mode

The USB boot mode supports only USB 2.0. In USB boot mode, both the secure and non-secure boot modes are supported. However, DDR-less systems, MultiBoot image, fallback image or XIP and are not supported.

Note: USB boot mode is disabled by default in FSBL. To enable the USB boot mode, configure the `FSBL_USB_EXCLUDE_VAL` to 0 in `xfbsbl_config.h` file.

Table 7-1: USB Boot Mode Details

Mode pins	0x7
MIO pins	MIO[63:52]
Non secure	Yes
Secure	Yes
Signed	Yes
Mode	Slave

USB boot mode requires a host PC with dfu-utils installed in it. The host and device need to be connected through a USB device cable. The host must contain one `boot.bin` for BootROM code, which contains only `fsbl.elf` and another `boot_all.bin` for FSBL. On powering on the board in USB boot mode, issue the following commands:

- For Linux:
 - `dfu-util -D boot.bin`
This downloads the file to the device, which then runs FSBL.
 - `dfu-util -D boot_all.bin`
This downloads the file to the device. FSBL carries out the required processing.
- For Windows:
 - `dfu-util.exe -D boot.bin`
This downloads the file to the device, which then runs FSBL.
 - `dfu-util.exe -D boot_all.bin`
This downloads the file to the device. FSBL carries out the required processing.

The size limit of `boot.bin` and `boot_all.bin` are the sizes of OCM and DDR, available respectively.

Secondary Boot Mode

There is a provision to have two boot devices in the Zynq UltraScale+ MPSoC architecture. The primary boot mode is the boot mode used by BootROM to load FSBL and optionally PMU FW. The secondary boot mode is the boot device used by FSBL to load all the other partitions. The supported secondary boot modes are QSPI24, QSPI32, SD0, eMMC, SD1, SD1-ls, NAND and USB.



IMPORTANT: If secondary boot mode is specified, it should be different from the primary boot device. For example, if QSPI32 is the primary boot mode, QSPI24 cannot be the secondary boot mode. Instead, you can have SD0, eMMC, SD1, SD1-ls, NAND, USB as secondary boot modes. All combinations of boot devices are supported as primary and secondary boot devices.

Note: By default, the secondary boot mode is the same as primary boot mode and there will be only one boot image.

See [What is Secondary Boot Mode](#) in FSBL wiki page for more information.

Detailed Boot Flow

The platform management unit (PMU) in the Zynq UltraScale+ MPSoC device is responsible for handling the primary pre-boot tasks.

PMU ROM will execute from a ROM during boot to configure a default power state for the device, initialize RAMs, and test memories and registers. After the PMU performs these tasks and relinquishes system control to the configuration security unit (CSU), it enters a service mode. In this mode, the PMU responds to interrupt requests made by system software through the register interface or by hardware through the dedicated I/O to perform platform management services.

Pre-Boot Sequence

The following table lists the tasks performed by the PMU in the pre-Boot sequence.

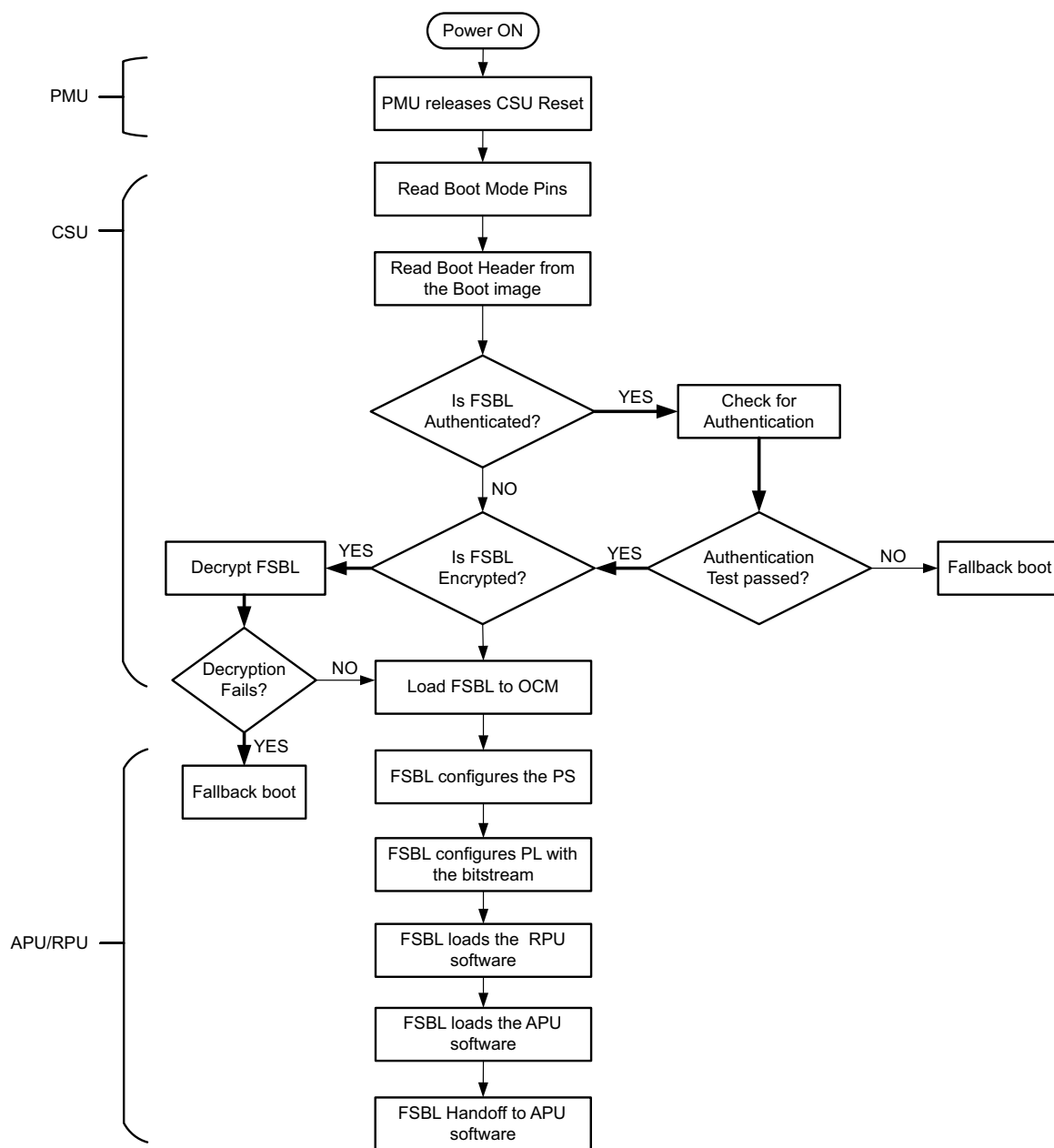
Table 7-2: Pre-Boot Sequence

Pre-Boot Task	Description
0	Initialize MicroBlaze™ processor. Capture key states.
1	Scan, and clear LPD and FPD.
2	Initialize the System Monitor.
3	Initialize the PLL used for MBIST clocks.
4	Zero out the PMU RAM.
5	Validate the PLL. Configure the MBIST clock.
6	Validate the power supply.
7	Repair FPD memory (if required).
8	Zeroize the LPD and FPD and initialize memory self-test.
9	Power-down any disabled IPs.
10	Either release CSU or enter error state.
11	Enter service mode.

As soon as the CSU reset is released, it executes the CSU bootROM and performs the following sequence:

1. Initializes the OCM.
2. Determines the boot mode by reading the boot mode register, which captures the boot-mode pin strapping at the POR.
3. The CSU continues with the FSBL load and the optional PMU firmware code. The firmware code is the software code that can be executed by the PMU unit. The code executes from the PMU's RAM. See [Chapter 9, Platform Management](#) for more information.

Figure 7-6 shows the detailed boot flow diagram.



X14935-070717

Figure 7-6: Detailed Boot Flow Example

Disabling FPD in Boot Sequence

Perform the following to avoid an FPD lockout, where FPD Power is applied momentarily:

- Apply the power until the completion of BootROM execution.
- To power down the FP during FSBL execution, set FPD bit '22' of `PMU_GLOBAL_REQ_PWRDWN_STATUS` register.
- To bring the FP domain up in a later stage of the boot process, set the `PMU_GLOBAL_REQ_PWRUP_STATUS` bit to '22'.

Perform the following in cases where the FPD power is not applied before the FSBL boots:

- Power up the R5.
- A register is set indicating the FPD is locked pending POR as the reset or clear sequence cannot execute on the FPD.
- R5 can read the FP locked status from `PMU_GLOBAL_REQ_ISO_STATUS` register bit '4'.
- At this stage, `PMU_GLOBAL_REQ_PWRUP_STATUS` bit '22' will not be set.
- To bring the FPD node back up, power must be supplied to the node and a POR needs to be issued.

Setting FSBL Compilation Flags

You can set compilation flags using the C/C++ settings in SDK FSBL project, as shown in [Figure 7-7](#).

Note: There is no need to change any of the FSBL source files or header files to include these flags.

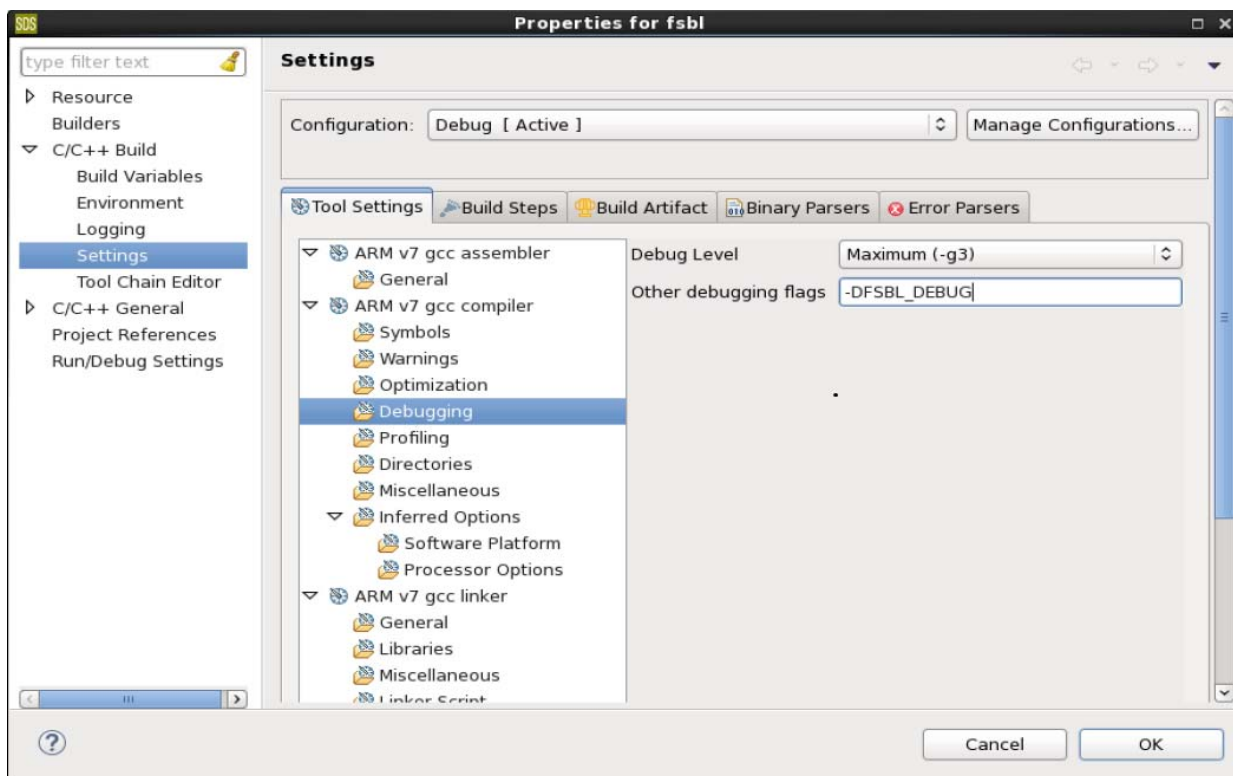


Figure 7-7: FSBL Debug Flags

The following table lists the FSBL compilation flags.

Table 7-3: FSBL Compilation Flags

Flag	Description
FSBL_DEBUG	Prints basic information and error prints, if any.
FSBL_DEBUG_INFO	Enables prints with format specifiers in addition to the basic information.
FSBL_DEBUG_DETAILED	Prints information with all data exchanged.
FSBL_NAND_EXCLUDE	Excludes NAND support code.
FSBL_QSPI_EXCLUDE	Excludes QSPI support code.
FSBL_SD_EXCLUDE	Excludes SD support code.
FSBL_RSA_EXCLUDE	Excludes authentication code.
FSBL_AES_EXCLUDE	Excludes decryption code.
FSBL_BS_EXCLUDE	Excludes bitstream code.
FSBL_SHA2_EXCLUDE	Excludes SHA-2 code.
FSBL_WDT_EXCLUDE	Excludes WDT support code.

Table 7-3: FSBL Compilation Flags (Cont'd)

Flag	Description
FSBL_USB_EXCLUDE	Excludes USB code. This is set to 1 by default. Set this value to 0 to enable USB boot mode.
FSBL_FORCE_ENC_EXCLUDE_VAL	Excludes forcing encryption of all partitions when ENC_ONLY fuse is programmed. By default, this is set to 0. FSBL forces to enable encryption for all the partitions when ENC_ONLY is programmed.

See **I'm unable to build FSBL due to size issues, how can I reduce its footprint** section in [FSBL wiki page](#) for more information.



IMPORTANT: *SHA-2 is deprecated from the current release. Xilinx recommends using SHA-3 instead of SHA-2.*

Enabling Debug Prints

Use the following steps to enable debug prints:

1. Define FSBL_DEBUG_INFO symbol. Right click on **FSBL application project** in SDK -> select **C/C++ Build Settings** -> **Tool Settings** tab -> **Symbols** (under Arm A53 gcc compiler).
2. Click **Add (+)** icon and type **FSBL_DEBUG_INFO**.
3. Click **OK** to close the Properties screen.

See [FSBL wiki page](#) for more information on debugging FSBL.

Fallback and MultiBoot Flow

In the Zynq UltraScale+ MPSoC device, the CSU bootROM supports MultiBoot and fallback boot image search where the configuration security unit (CSU) bootROM searches through the boot device looking for a valid image to load. The sequence is as follows:

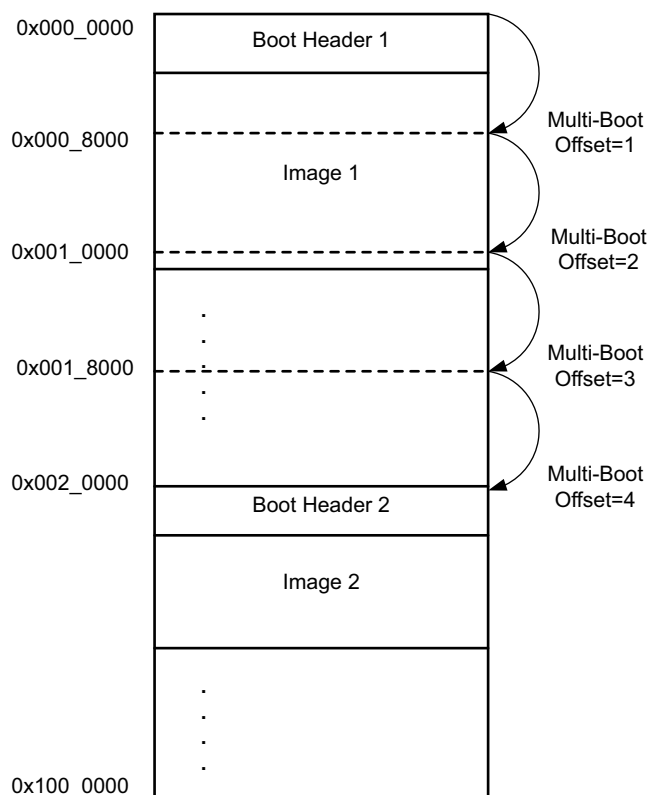
- BootROM searches for a valid image identification string (**XLNX** as image ID) at offsets of 32 KB in the flash.
- After finding a valid identification value, validates the checksum for the header.
- If the checksum is valid, the bootROM loads the image. This allows for more than one image in the flash.

In MultiBoot:

- FSBL or the user application must initiate the boot image search to choose a different image from which to boot.

- To initiate this image search, FSBL or the user application updates the MultiBoot offset to point the intended boot image, and generates a soft reset by writing into the **CRL_APB** register.

The following figure shows an example of the fallback and MultiBoot flow.



X14936-071217

Figure 7-8: MultiBoot Flow

Note: The same flow is applicable to both Secure and Non-secure boot methods.

In the example fallback boot flow figure, the following sequence occurs:

- Initially, the CSU bootROM loads the boot image found at **0x000_0000**.
- If this image is found to be corrupted or the decryption and authentication fails, CSU bootROM increments the MultiBoot offset by 1 and searches for a valid boot image at **0x000_8000** (32 KB offset).
- If the CSU bootROM does not find the valid identification value, it again increments the MultiBoot offset by 1, and searches for a valid boot image at the next 32 KB aligned address.
- The CSU bootROM repeats this until a valid boot image is found or the image search limit is reached. In this example flow, the next image is shown at **0x002_0000** where the MultiBoot offset is 4.

- In the example MultiBoot flow, to load the second image that is at the address `0x002_0000`, and set the MultiBoot offset to 4 with the FSBL or the user application. When the MultiBoot offset is updated, soft reset the system.

The following table shows the MultiBoot image search range for different booting devices.

Table 7-4: Boot Devices and MultiBoot Image Search Range

Boot Device	MultiBoot Image Search Range
QSPI Single (24-bit)	16 MB
QSPI Dual (24-bit)	32 MB
QSPI Single (32-bit)	256 MB
QSPI Dual (32-bit)	512 MB
NAND	128 MB
SD/EMMC	8,191 boot files
USB	Not applicable

FSBL Build Process

First Stage Boot Loader configures the FPGA with a bitstream (if it exists) and loads either the Operating System (OS) Image or Standalone (SA) Image or Second Stage Boot Loader image from the non-volatile memory (NAND/SD/eMMC/QSPI) to RAM(DDR/TCM/OCM). It takes the Cortex-R5F processor or the Cortex-A53 processor unit out of reset. It supports multiple partitions. Each partition can be a code image or a bitstream. Each of these partitions, if required, will be authenticated and/or decrypted. After authenticating and/or decrypting, the FSBL is loaded into OCM and handed off by the CSU BootROM.

Note: If you are creating a custom FSBL, you should be aware that the OCM size is 256 KB and is available to CSU BootROM. The FSBL size is close to 170 KB and it would fit in the OCM. While using the USB boot mode, you should make sure that the PMU firmware is loaded by the FSBL and not by the CSU BootROM. This is because the size of `boot.bin` loaded by the CSU BootROM should be less than 256 KB.

Creating an FSBL from the Software Development Kit

Use the following steps to create an FSBL using the Software Development Kit (SDK):

1. Launch the SDK with the following command:

```
xsdk
```

2. Select **File-->New-->Application Project** to open a New Project window. Provide a name for the FSBL project.
3. In the **Target Hardware** section, select a pre-defined hardware platform for ZynqMP (e.g. `ZCU102_hw_platform`).

Alternatively, to create a new/custom platform from a .hdf file, click on **New** to create a new hardware platform. In the next (New Hardware Project) window, enter the Project name and under the **Target Hardware Specification** click on **browse** and select the HDF file. A new hardware platform is created.

4. In the **New Project** window, select either the psu_cortexa53_0 processor or psu_cortexr5_0 processor. If you select psu_cortexa53, select the compiler to be either 64-bit (default) or 32-bit from drop down menu.
5. Click **Next** and select **Zynq MP FSBL**.
6. Click **Finish** to generate the A53/R5 FSBL. This populates and builds the FSBL code along with BSP.

Note: FSBL can only be run from A53_0 (AArch32 and AArch64), R5_0, or R5_Lockstep.

See [FSBL wiki page](#) for more information on FSBL.

Note: Debug prints in FSBL are disabled by default (except for FSBL banner). See [Enabling Debug Prints](#) for more information.

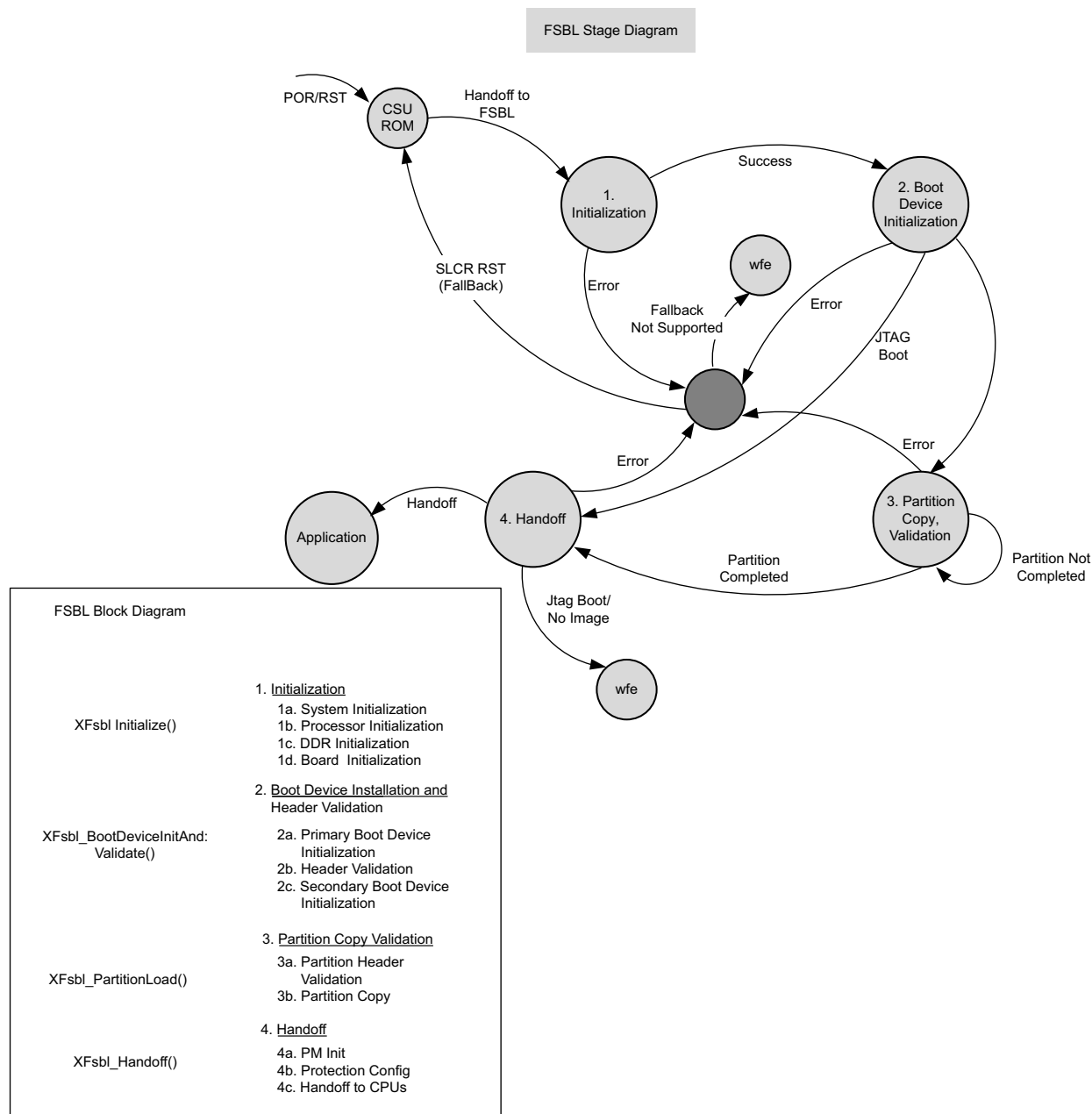
To modify the source files (FSBL or BSP), browse the source file, update, and save the file. Build the project. The Debug/Release folder of the FSBL project includes .elf file.

Phases of FSBL Operation

FSBL operation includes the following four stages:

- Initialization
- Boot device initialization
- Partition loading
- Handoff

[Figure 7-9](#) shows the stages of FSBL operation:



X19962-101917

Figure 7-9: Stages of FSBL

Initialization

Initialization consists of the following four internal stages:

1. [XFsbI_SystemInit](#)
2. [XFsbI_ProcessorInit](#)
3. [Initialize DDR](#) (not required for APU only reset)

4. XFsbl_BoardInit

XFsbI_SystemInit

This function powers up PL for 1.0 and 2.0 silicon and removes PS-PL isolation. It initializes clocks and peripherals as specified in psu-init. This function is not called in APU only reset.

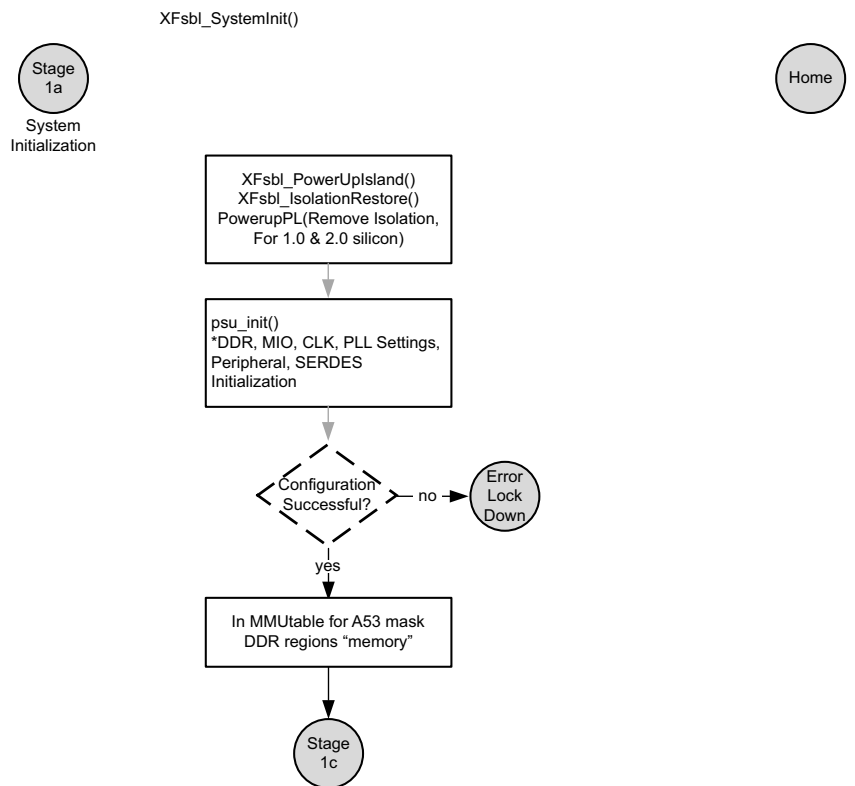


Figure 7-10: FSBL System Initialization

XFsbI_ProcessorInit

Processor initialization will start in this stage. It will set up the Instruction and Data caches, L2 caches, MMU settings, stack pointers in case of A53 and I/D caches, MPU settings, memory regions, stack pointers, and TCM settings for R5F. Most of these settings will be performed in BSP code initialization. IVT vector is changed to the start of OCM for A53 and to start of TCM (0x0 in lowvec and 0xffff0000 in highvec) in case of R5F.

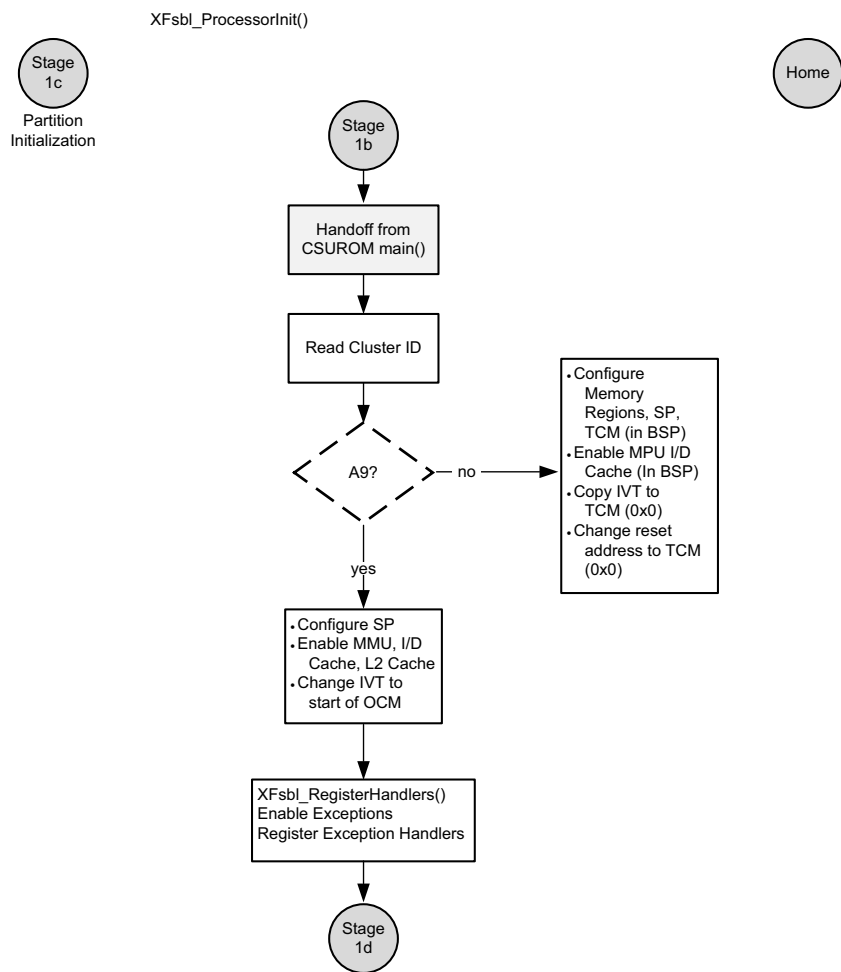
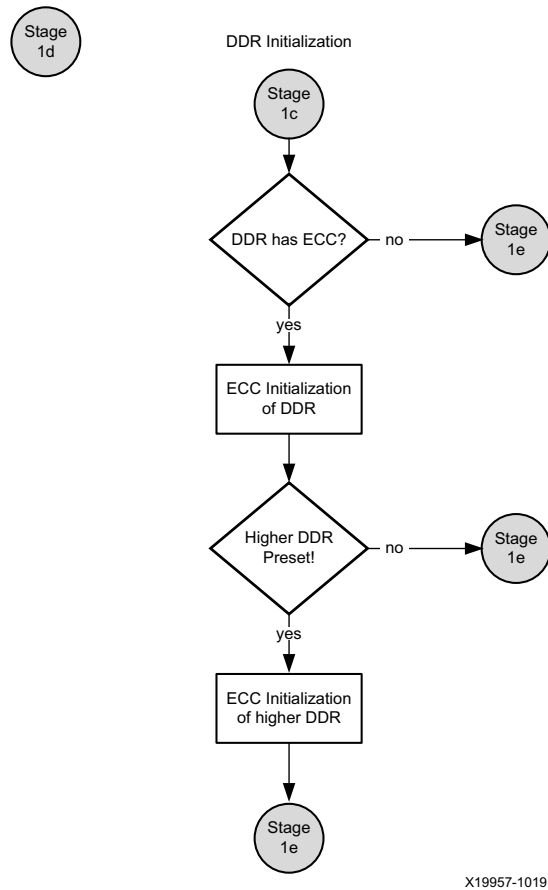


Figure 7-11: Processor Initialization

Initialize DDR

DDR would be initialized in this stage. This function is not called in APU only reset.



X19957-101917

Figure 7-12: DDR Initialization

XFsbI_BoardInit

This function performs required board specific initializations. Most importantly, it configures GT lanes and IIC.

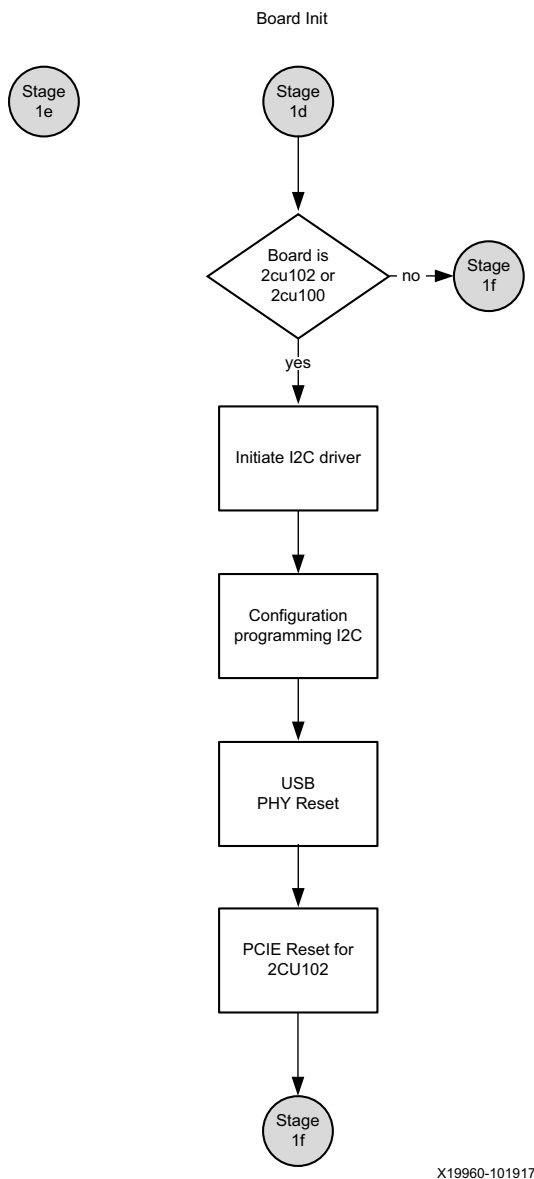
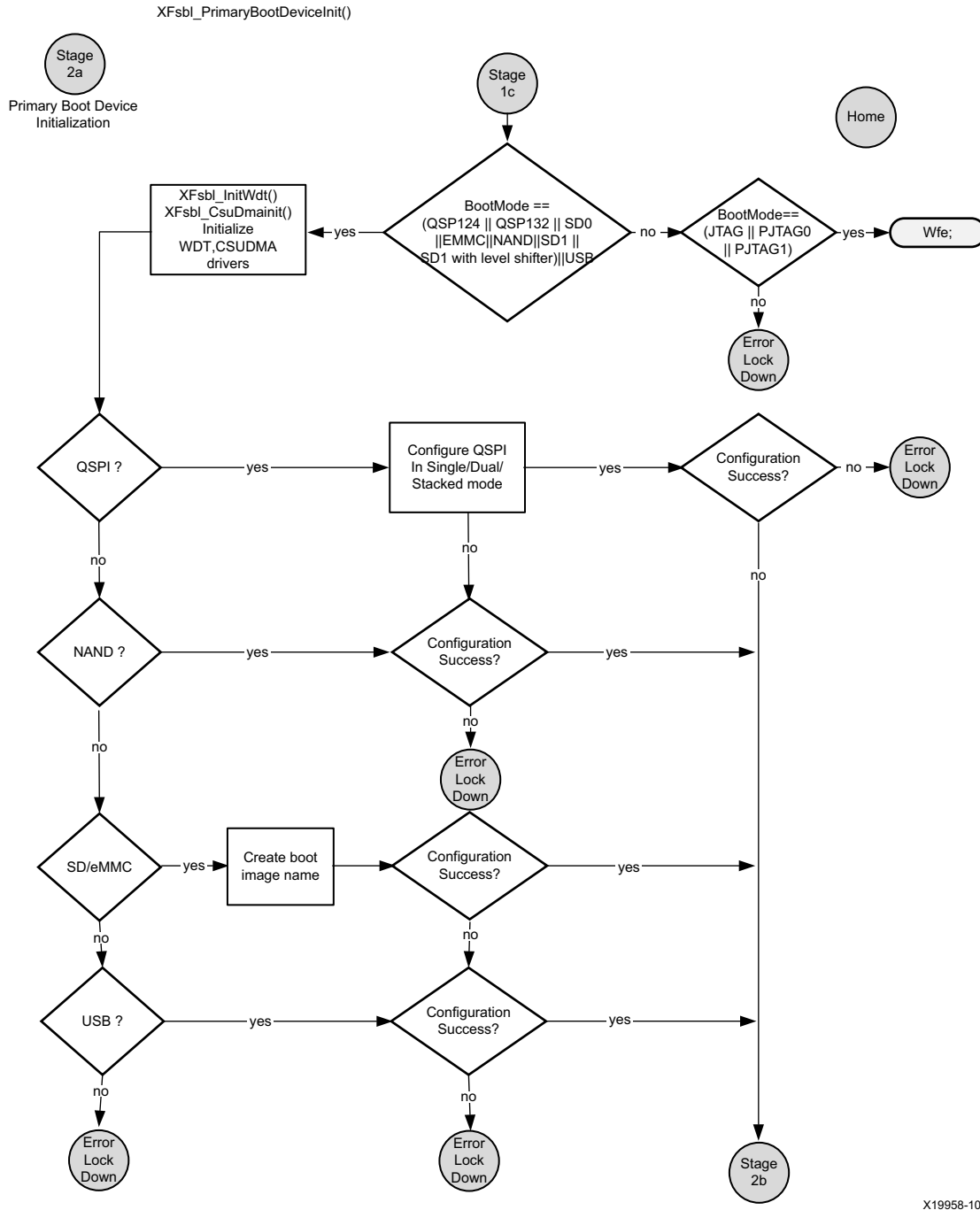


Figure 7-13: Board Initialization

Boot Device initialization

XFsbI_PrimaryBootDeviceInit

This stage involves reading boot mode register to identify the primary boot device and initialize the corresponding device. Each boot device driver provides init, copy and release functions which are initialized to DevOps function pointers in this stage.



X19958-101917

Figure 7-14: Primary Boot Device Initialization

XFsbI_ValidateHeader

Using the copy functions provided, the FSBL reads the boot header attributes and image offset address. It reads the EFUSE bit to check for authentication. It reads the image header and validates the image header table. It then reads the **Partition Present Device** attribute of image header. A non-zero value indicates a secondary boot device. A zero value indicates that the secondary boot device is the same as the primary boot device.

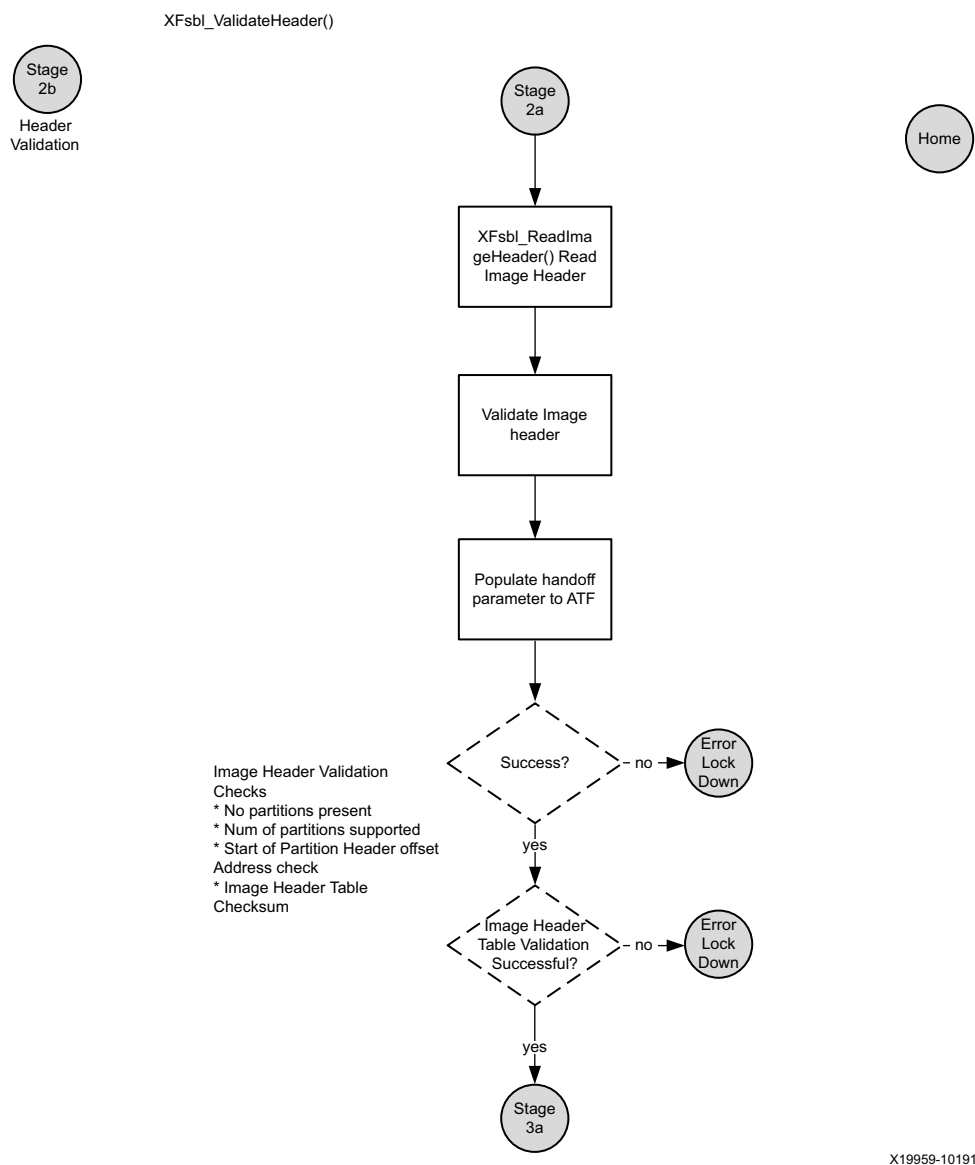
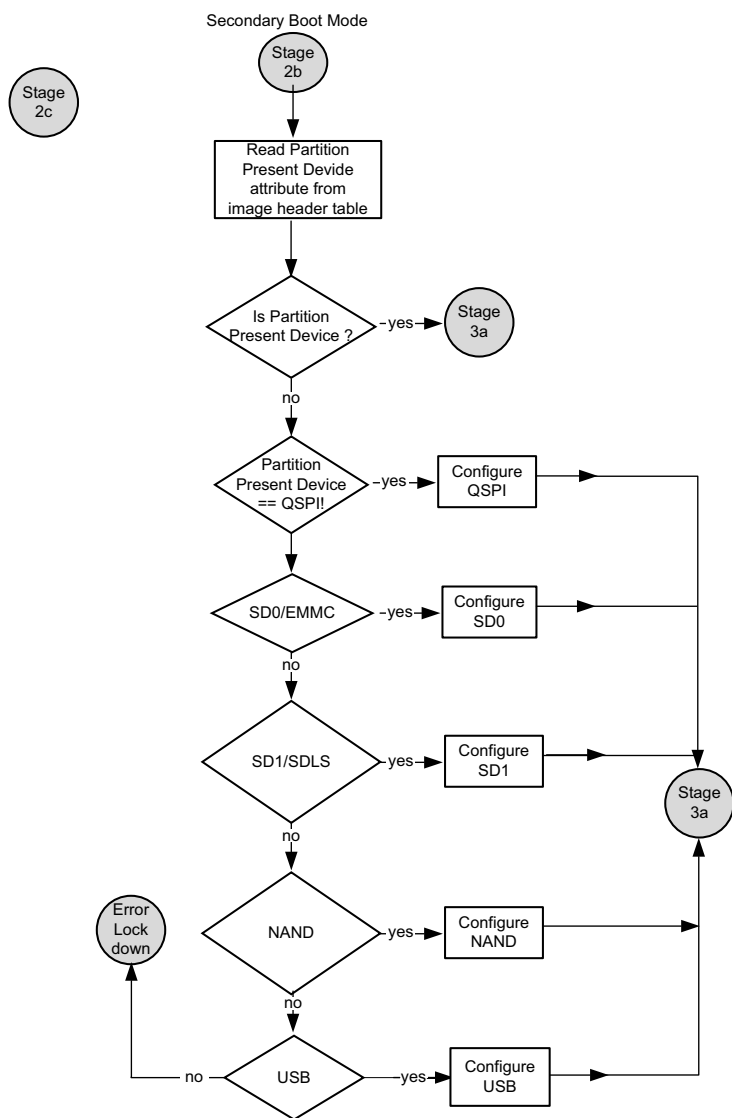


Figure 7-15: Validating Header

XFsbI_SecondaryBootDeviceInit

This function is called in case of a non-zero value of **Partition Present Device** attribute of image header table. It initializes the secondary boot device driver and the secondary boot device would be used to load all partitions by FSBL.



X19961-101917

Figure 7-16: Secondary Boot Mode

XFsbI_SetATFHandoffParams

ATF is assumed to be the next loadable partition after FSBL. It is capable of loading all other partitions and hence it is passed a handoff structure.

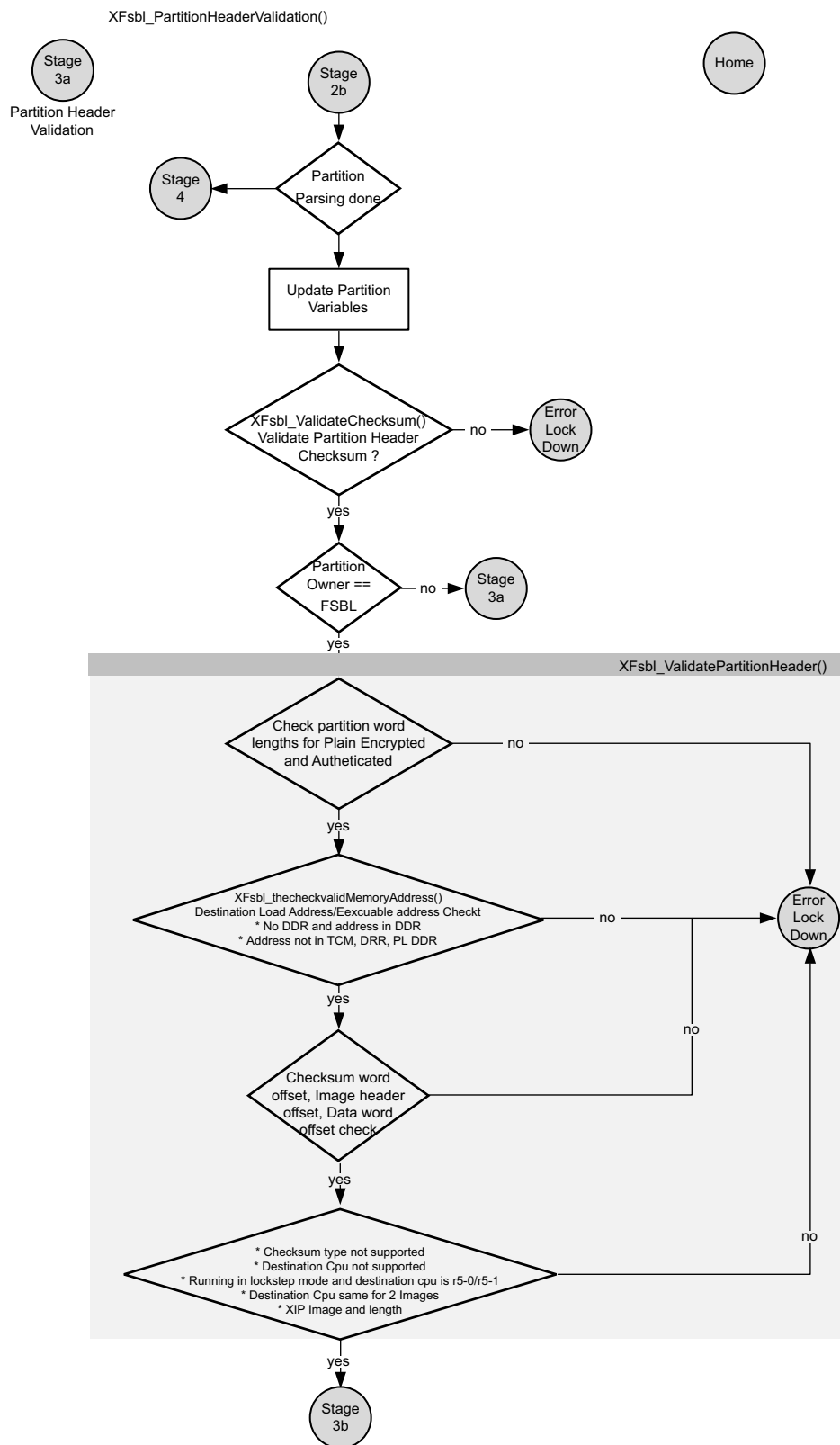
The first partition of an application will have a non-zero execution address. All the remaining partitions of that application will have 0 as execution address. Hence look for the non-zero execution address for partition which is not the first one and ensure the CPU is A53.

This function sets the handoff parameters to the Arm Trusted Firmware (ATF). The first argument is taken from the FSBL partition header. A pointer to the handoff structure containing these parameters is stored in the PMU_GLOBAL.GLOBAL_GEN_STORAGE6 register, which the ATF reads. The structure is filled with magic characters 'X', 'L', 'N', and 'X' followed by the total number of partitions and execution address of each partition.

Partition Loading

XFsbbl_PartitionHeaderValidation

Partition header is validated against various checks. All the required partition variables are updated at this stage. If the partition owner is not FSBL, partition will be ignored and will continue with the other partitions.

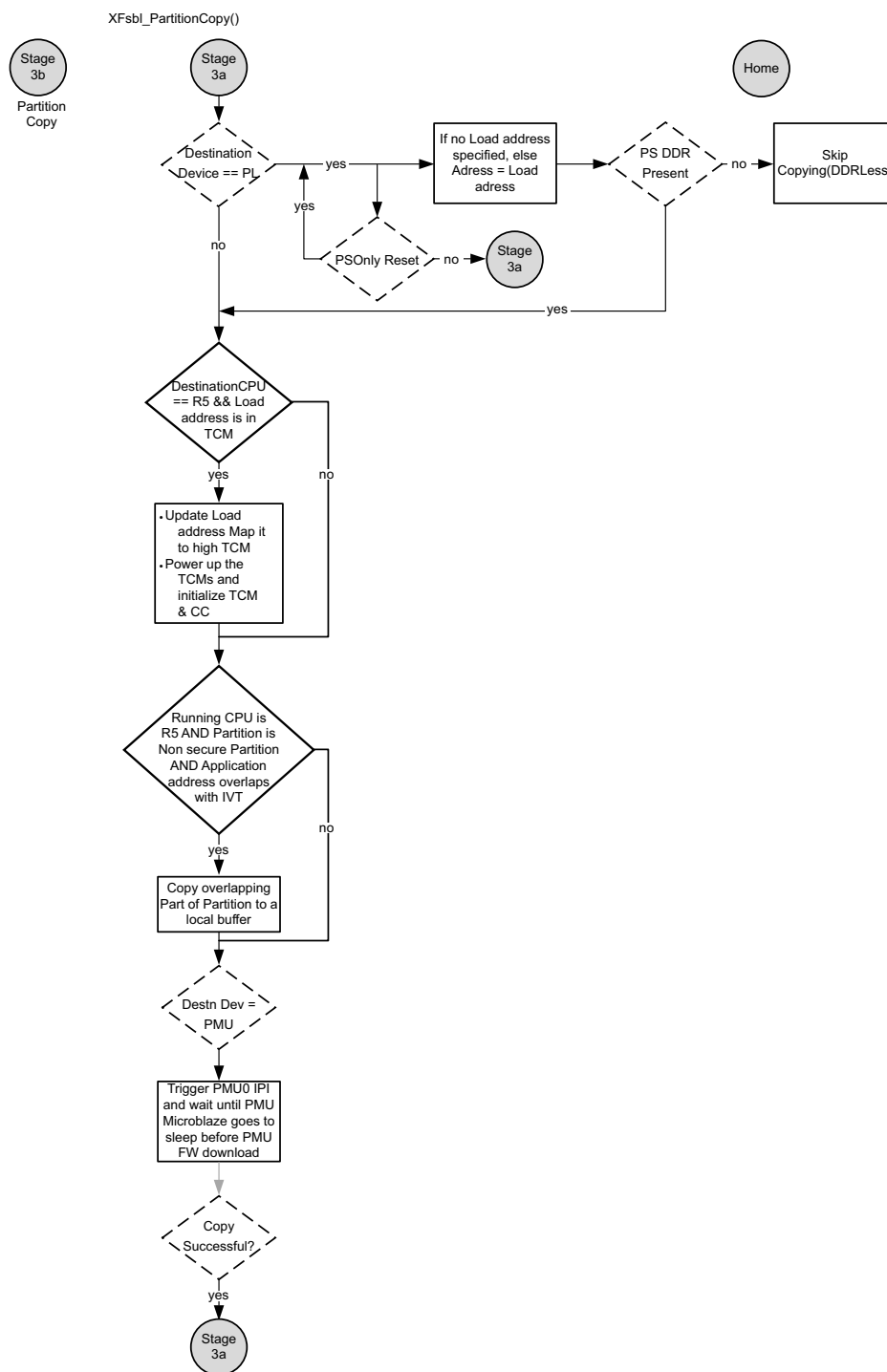


X19951-101917

Figure 7-17: Partition Header Validation

XFsbI_PartitionCopy

Partition will be copied to the DDR or TCM or OCM or PMU RAM.

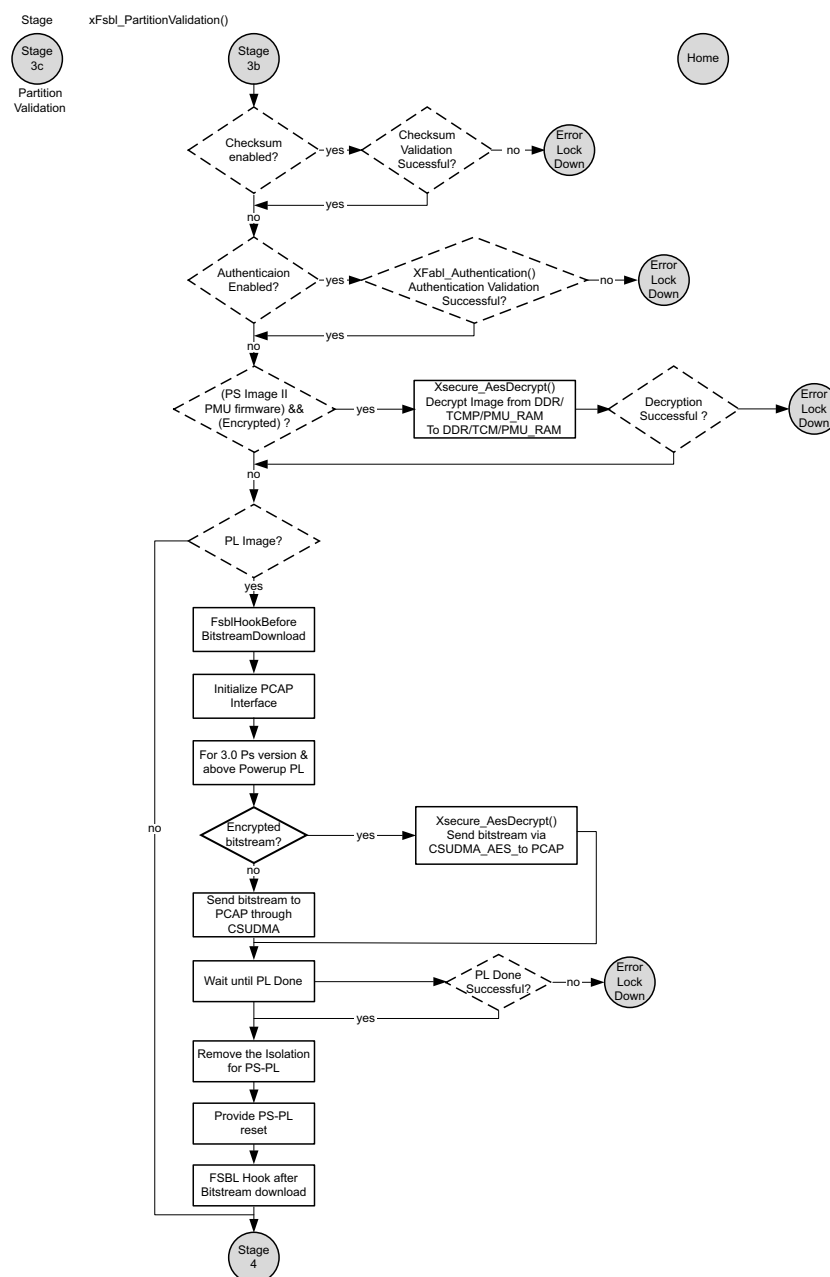


X19950-101917

Figure 7-18: Partition Copy

XFsbI_PartitionValidation

Partition will be validated based on the partition attributes. If checksum bit is enabled, then the partition will be validated first for checksum correctness and then, based on the authentication flag, it would be authenticated. If encryption flag is set, then the partition will be decrypted and then copied to the destination.



X19949-101917

Figure 7-19: Partition Validation Function

Handoff

In this stage, `protection_config` functions from `psu_init` will be executed and then, any handoff functionality is executed. Also PS-PL isolation is removed unconditionally. R5F will be brought out of reset if there is any partition supposed to run on it. R5F will be configured to boot in lowvec mode or highvec mode as per the settings provided by you while building the boot image. The handoff address in lowvec mode is 0x0 and 0xffff0000 in highvec mode. Lowvec/Highvec information should be specified by you while building the boot image. After all the other PS images are done, then the running CPU image will be handed off to that CPU with an update on the PC value. If there is no image for the running CPU, it will be in `wfe` loop.

Running the processor does not pass any parameters to any other processor. Any communication between various partitions can happen by reading from (or writing to) the PMU global registers.

Handoff on the running processor involves updating Program Counter (PC) of the running processor, as is done in the case of APU Reset. Handoff to other processors involves updating their PCs and bringing the processors out of reset.

XFsbI_PmInit

This function initializes and configures the Inter Processor Interrupts (IPI). It then writes the PM configuration object address to an IPI buffer and triggers an IPI to the target. The PMU firmware then reads and configures the device nodes as specified in the configuration object.

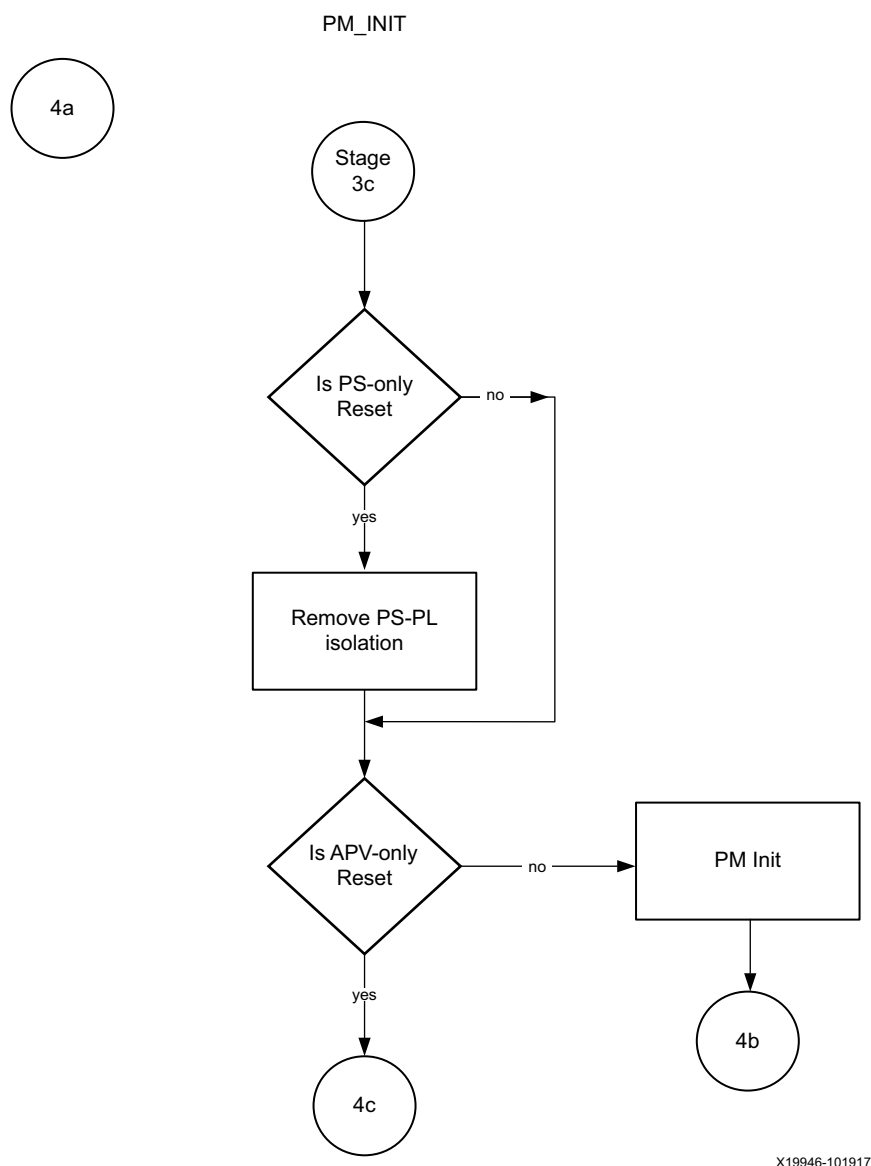


Figure 7-20: PM Initialization

Protection Configuration

In this stage, `protection_config` functions from `psu_init` will be executed. The Protection Configuration is currently bypassed by default. Isolation is currently limited to OCM only. The bypassing or the application of protection happens in this stage.

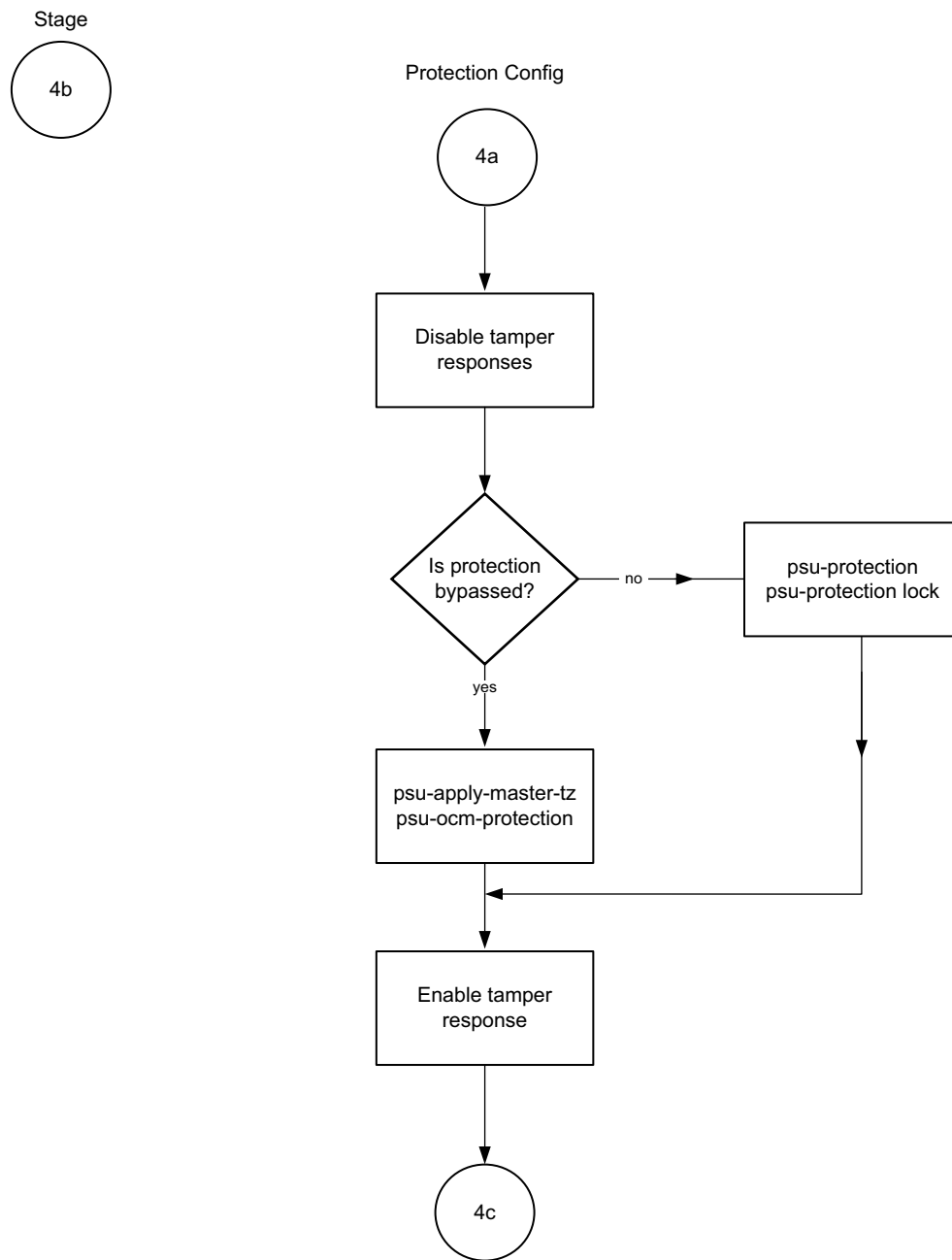


Figure 7-21: Protection Configuration

Handoff

Handoff on the running processor involves updating Program Counter (PC) of the running processor, as is done in the case of APU Reset. Handoff to other processors involves updating their PCs and bringing the processors out of reset. A53 FSBL will bring R5 out of reset if there is any partition to run on it. R5 will be configured to boot in lowvec mode or highvec mode as per the settings provided by you while building the boot image. The handoff address in lowvec mode is 0x0 and 0xffff0000 in highvec mode.

You must specify Lowvec/Highvec information while building the boot image. After all the other PS images are done, then running the cpu image will be handed off to that cpu with an update on the PC value. If there no image for the running cpu, it will be in wfe loop.

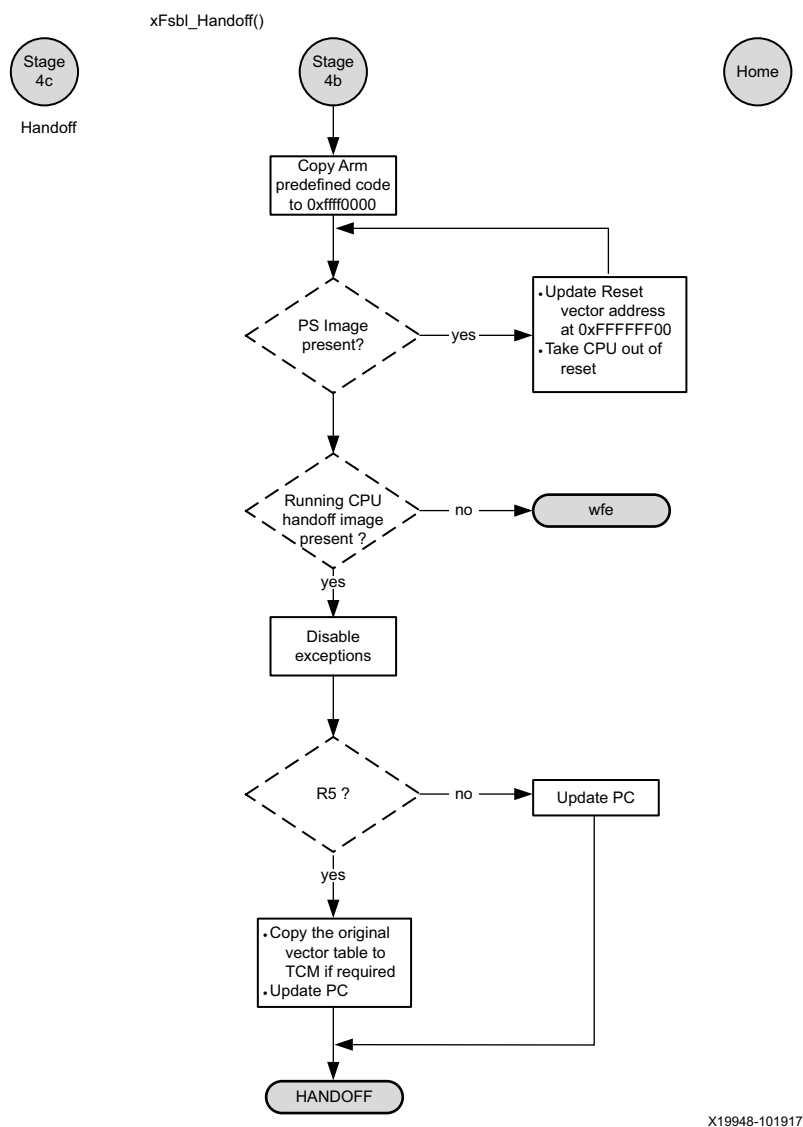


Figure 7-22: Handoff

Supported Handoffs

Table 7-5 shows the various combinations of handoffs that are supported in FSBL

Table 7-5: Supported Handoffs

FSBL	Application	Processor Cores	Execution Address
64-bit	64-bit	All (i.e. A53-0, A53-1, A53-2, A53-3)	Any Address
64-bit	32-bit	A53-1, A53-2, A53-3	0x0

Table 7-5: Supported Handoffs (Cont'd)

FSBL	Application	Processor Cores	Execution Address
32-bit	32-bit	A53-0	Any Address
32-bit	32-bit	A53-1, A53-2, A53-3	0x0
32-bit	64-bit	A53-1, A53-2, A53-3	Any Address

Error Lock Down

XFsb1_ErrorLockDown function handles errors in FSBL. This function is called whenever the return value of a function is unsuccessful. This function updates error status register and then loops indefinitely, if fallback is not supported.

In case the boot mode supports fallback, MultiBoot offset register is updated and then waits for a WDT reset to occur. On reboot, BootROM and FSBL read the image from the new address calculated from MultiBoot offset, thus loading a new image.

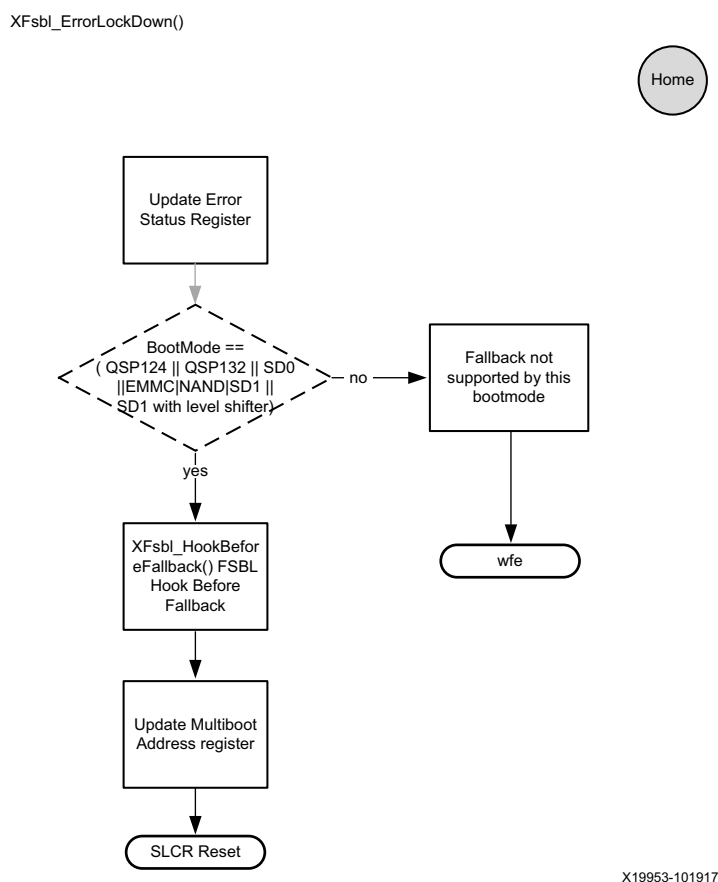


Figure 7-23: Error Lock Down Function

Miscellaneous Functions

The following functions are available in FSBL:

- [XFsbl_PrintArray](#)
- [XFsbl_Strcpy](#)
- [XFsbl_Strcat](#)
- [XFsbl_Strcmp](#)
- [XFsbl_MemCpy](#)
- [XFsbl_PowerUpIsland](#)
- [XFsbl_IsolationRestore](#)
- [XFsbl_SetTlbAttributes](#)
- [XFsbl_GetSiliconIdName](#)
- [XFsbl_GetProcEng](#)
- [XFsbl_CheckSupportedCpu](#)
- [XFsbl_AdmaCopy](#)
- [XFsbl_GetDrvNumSD](#)
- [XFsbl_MakeSdFileName](#)

XFsbl_PrintArray

Use this function to print an entire array in bytes as specified by the debug type.

```
void XFsbl_PrintArray (u32 DebugType, const u8 Buf[], u32 Len, const char *Str);
```

Table 7-6: XFsbl_PrintArray Parameters in FSBL

Parameters	Description
DebugType	Printing of the array is performed as defined by the debug type.
Buf	Pointer to the buffer to be printed
Len	Length of the bytes to be printed
Str	Pointer to the data that is printed

XFsbl_Strcpy

Use this function to copy the source string to the destination string.

```
char *XFsbl_Strcpy(char *DestPtr, const char *SrcPtr)
```

Table 7-7: XFsbl_Strcpy Parameters in FSBL

Parameters	Description
DestPtr	Pointer to the buffer to be printed
SrcPtr	Pointer to the buffer containing the source string

XFsbI_Strcat

Use this function to append the second string to the first string.

```
char* XFsbl_Strcat(char* Str1Ptr, const char* Str2Ptr)
```

Table 7-8: XFsbl_Strcat Parameters in FSBL

Parameters	Description
Str1Ptr	Pointer to the original string to which string pointed to by Str2Ptr would be appended
Str2Ptr	Pointer to the second string

XFsbI_Strcmp

Use this function to compare the strings.

```
s32 XFsbl_Strcmp( const char* Str1Ptr, const char* Str2Ptr)
```

Table 7-9: XFsbl_Strcmp Parameters in FSBL

Parameters	Description
Str1Ptr	Pointer to the first string
Str2Ptr	Pointer to the second string

XFsbI_MemCpy

Use this function to copy the memory contents pointed to by SrcPtr to the memory pointed to by DestPtr. Len is number of bytes to be copied.

```
void* XFsbl_MemCpy(void * DestPtr, const void * SrcPtr, u32 Len)
```

Table 7-10: XFsbl_MemCpy Parameters in FSBL

Parameters	Description
SrcPtr	Pointer to the memory contents to be copied
DestPtr	Pointer to the destination
Len	Length of the bytes to be printed

XFsbI_PowerUpIsland

Use this function to check the power state of one or more power islands and powers them up if required.

```
u32 XFsbI_PowerUpIsland(u32 PwrIslandMask)
```

Table 7-11: XFsbI_PowerUpIsland Parameters in FSBL

Parameters	Description
PwrIslandMask	Mask of island that needs to be powered up

XFsbI_IsolationRestore

Use this function to request isolation restore through PMU firmware.

```
u32 XFsbI_IsolationRestore(u32 IsolationMask);
```

Table 7-12: XFsbI_IsolationRestore Parameters in FSBL

Parameters	Description
IsolationMask	Mask of the entries for which isolation is to be restored

XFsbI_SetTlbAttributes

Use this function to set the memory attributes for a section in the translation table.

```
void XFsbI_SetTlbAttributes(INTPTR Addr, UINTPTR attrib);
```

Table 7-13: XFsbI_SetTlbAttributes Parameters in FSBL

Parameters	Description
Addr	Address for which the attributes are to be set
Attrib	Attributes for the memory region

XFsbI_GetSiliconIdName

This function reads the CSU_ID_CODE register and calculates the SvdId of the device. It returns the corresponding deviceId name.

```
const char *XFsbI_GetSiliconIdName(void);
```

XFsbI_GetProcEng

This function determines and returns the engine type. Currently only CG, EG, and EV engine types are supported.

```
const char *XFsbI_GetProcEng(void);
```

XFsbI_CheckSupportedCpu

This function checks if a given CPU is supported by this variant of Silicon. Currently it checks if it is CG part and disallows handoff to A53_2/3 cores.

```
u32 XFsbI_CheckSupportedCpu(u32 CpuId);
```

Table 7-14: XFsbI_CheckSupportedCpu Parameters in FSBL

Parameters	Description
Cpuld	Checks if the processor is A53_2 or A53_3 or not.

XFsbI_AdmaCopy

This function copies data memory to memory using ADMA. You must take care of cache invalidation and flushing. ADMA also should be configured to simple DMA before calling this function.

```
u32 XFsbI_AdmaCopy(void * DestPtr, void * SrcPtr, u32 Size);
```

Table 7-15: XFsbI_AdmaCopy Parameters in FSBL

Parameters	Description
DestPtr	Pointer to the destination buffer to which data needs to be copied
SrcPtr	Pointer to the source buffer from which data needs to be copied
Size	Number of bytes of data that needs to be copied

XFsbI_GetDrvNumSD

This function is used to obtain drive number based on design and boot mode.

```
u32 XFsbI_GetDrvNumSD(u32 DeviceFlags);
```

Table 7-16: XFsbI_GetDrvNumSD Parameters in FSBL

Parameters	Description
Device flags	Contains the boot mode information, that is, one of SD0, SD1, eMMC, or SD1-LS boot modes

XFsbI_MakeSdFileName

This function returns the file name of the boot image. The name is deduced from the parameters.

```
void XFsbI_MakeSdFileName(char*XFsbI_SdEmmcFileName, u32 MultiBootReg, u32 DrvNum);
```

Table 7-17: XFsbl_MakeSdFileName Parameters in FSBL

Parameters	Description
XFsbL_SdEmmcFileName	Contains the final file name
Multiboot reg	The value of the MultiBoot register gets appended to the file name, if its value is non zero
DrvNum	Differentiates between SD0 and SD1 logical drives

Hooks In FSBL

Hooks are the functions that can be defined by you. FSBL provides blank functions and executes them from certain strategic locations. The following table shows the currently available hooks.

Table 7-18: Hooks in FSBL

Hook purpose/location	Hook Function Name
Before PL bitstream loading	XFsbL_HookBeforeBSDownload()
After PL bitstream loading	XFsbL_HookAfterBSDownload()
Before (the first) Handoff (to any application)	XFsbL_HookBeforeHandoff()
Before fallback	XFsbL_HookBeforeFallback()
To add more initialization code, in addition to that in psu_init or to replace psu_init with custom initialization	XFsbL_HookPsuInit()

See [FSBL wiki Page](#) for more information on FSBL.

Security Features

Introduction

This chapter details the Zynq® UltraScale+™ MPSoC device features that you can leverage to address security during boot time and run time of an application. The Secure Boot mechanism is described in detail in this [link](#) to the Security chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11]

The system protection unit (SPU) provides the following hardware features for run-time security of an application running on Zynq UltraScale+ MPSoC devices:

- [Xilinx Memory Protection Unit](#) (XMPU)
- [Xilinx Peripheral Protection Unit](#) (XPPU)
- [System Memory Management Unit](#) (SMMU)
- [A53 Memory Management Unit](#)
- [R5 Memory Protection Unit](#)

One of the runtime security features is access controls on the PMU and CSU global registers from Linux. These registers are classified into two lists: The white list (accessible all the time by default) and the black list (accessible only when a compile time flag is set). For more details, see [CSU/PMU Register Access](#).

Boot Time Security

This section details the various boot image formats for authentication and encryption.

Encryption

Zynq UltraScale+ MPSoC devices has AES-GCM hardware engine that supports confidentiality of your boot images, and can also be used by you post-boot to encrypt and decrypt your data.

The AES crypto engine has access to a diverse set of key sources. For more information on the key sources, see *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

The red key is used to encrypt the image. During the generation of the Boot file (`BOOT.bin`), the red key, and the initialization vector (IV) must be provided to the Bootgen tool in `.nky` file format.

PMU firmware can be loaded by CSU bootROM or FSBL. The CSUROM treats the FSBL and PMU firmware as separate partitions and hence, decrypts each of them individually. If both the FSBL and PMU firmware are encrypted, the AES Key/IV will be reused, which is a violation of the standard.



IMPORTANT: *If both the FSBL and PMU firmware are encrypted, the PMU firmware must be loaded by the FSBL (and not the CSU BootROM) to avoid reusing the AES Key/IV pair. For more information, see Xilinx Answer [70622](#).*

The following BIF file is for encrypted image, where PMU firmware is loaded by FSBL

```
the_ROM_image:
{
  [aeskeyfile] bbram.nky
  [keysrc_encryption] bbram_red_key
  [bootloader, encryption=aes, destination_cpu=a53-0] ZynqMP_Fsbl.elf
  [destination_cpu = pmu, encryption=aes] pmufw.elf
}
```

BIF File with BBRAM Red Key

The following BIF file sample shows the red key stored in BBRAM.

```
the_ROM_image: {
  [aeskeyfile]  bbram.nky
  [keysrc_encryption]  bbram_red_key
  [bootloader, encryption=aes, destination_cpu=a53-0]  ZynqMP_Fsbl.elf
  [destination_cpu = a53-0, encryption=aes]  App_A53_0.elf
}
```

BIF File with eFUSE Red Key

The following BIF file sample shows the red key stored in eFUSE.

```
the_ROM_image: {
  [aeskeyfile]  efuse.nky
  [keysrc_encryption]  efuse_red_key
  [bootloader, encryption=aes, destination_cpu=a53-0]  fsbl.elf
  [destination_cpu = a53-0, encryption=aes]  App_A53_0.elf
}
```

BIF File with an Operational Key

For creating a boot image using Bootgen with an operational key, you must provide the tool with the operational key, along with the red key and IV in an `.nky` file. Bootgen places this operational key in a header and encrypts it with the device red key. The result is what is called an encrypted secure header. The main advantage of this is that it minimizes the use of the device key, thus limiting its exposure.

For more details, refer to "Minimizing Use of the AES Boot Key (OP Key Option)" in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

```
the_ROM_image:
{
  [aeskeyfile]  bbram.nky
  [fsbl_config]  opt_key
  [keysrc_encryption]  bbram_red_key
  [bootloader, encryption=aes, destination_cpu=a53-0]  ZynqMP_Fsbl.elf
  [destination_cpu = a53-0, encryption=aes]  App_A53_0.elf
}
```

Using Op Key to Protect the Device Key in a Development Environment

The following steps provide a solution in a scenario where two development teams Team-A (secure team), which manages the secret red key and Team-B (not so secure team) work collaboratively to build an encrypted image without sharing the secret red key. Team-A manages the secret red key. Team-B builds encrypted images for development and test. However, it does not have access to the secret red key.

Team-A encrypts the boot loader with the device key (using the **Op Key** option) and delivers the encrypted bootloader to Team-B. Team-B encrypts all the other partitions using the **Op Key**.

Team-B takes the encrypted partitions that they created and the encrypted boot loader they received from the Team-A and uses Bootgen to 'stitch' everything together into a single boot.bin.

The following procedures describe the steps to build an image:

Procedure 1

In the initial step, Team-A encrypts the boot loader with the device Key using the **opt_key** option, delivers the encrypted boot loader to Team-B. Now, Team-B can create the complete image at a go with all the partitions and the encrypted boot loader using the operational key as device key.

1. Encrypt boot loader with device key:

```
bootgen -arch zynqmp -image stage1.bif -o fsbl_e.bin -w on -log error
```

Example stage1.bif:

```
stage1:
{
  [aeskeyfile] aes.nky
  [fsbl_config] opt_key
  [keysrc_encryption] bbram_red_key
  [bootloader,destination_cpu=a53-0,encryption=aes] fsbl.elf
}
```

Example **aes.nky** for stage1:

```
Device xc7z020clg484;
Key 0 AD00C023E238AC9039EA984D49AA8C819456A98C124AE890ACEF002100128932;
IV 0 F7F8FDE08674A28DC6ED8E37;
Key Opt 229C993D1310DD27B6713749B6D07FCF8D3DCA01EC9C64778CBAF457D613508F;
```

2. Attach the encrypted boot loader and rest of the partitions with the operational key as device key to form a complete image:

```
bootgen -arch zynqmp -image stage2a.bif -o final.bin -w on -log error
```

Example of stage2.bif:

```
stage2:
{
  [aeskeyfile] aes-opt.nky
  [bootimage] fsbl_e.bin
  [destination_cpu=a53-0,encryption=aes] hello.elf
  [destination_cpu=a53-1,encryption=aes] hello1.elf
}
```

Example `aes-opt.nky` for stage2:

```
Device xc7z020clg484;
Key 0 229C993D1310DD27B6713749B6D07FCF8D3DCA01EC9C64778CBAF457D613508F;
IV 0 F7F8FDE08674A28DC6ED8E37;
```

Procedure 2

in the initial step, Team-A encrypts the boot loader with the device key using the `opt_key` option and delivers the encrypted boot loader to Team-B. Now, Team-B can create encrypted images for each partition independently, using the operational key as the device key. Finally, Team-B can use Bootgen to stitch all the encrypted partitions and the encrypted boot loader, to get the complete image.

1. Encrypt boot loader with device key:

```
bootgen -arch zynqmp -image stage1.bif -o fsbl_e.bin -w on -log error
```

Example stage1.bif:

```
stage1:
{
  [aeskeyfile] aes.nky
  [fsbl_config] opt_key
  [keysrc_encryption] bbram_red_key
  [bootloader,destination_cpu=a53-0,encryption=aes] fsbl.elf
}
```

Example `aes.nky` for stage1:

```
Device xc7z020clg484;
Key 0 AD00C023E238AC9039EA984D49AA8C819456A98C124AE890ACEF002100128932;
IV 0 F7F8FDE08674A28DC6ED8E37;
Key Opt 229C993D1310DD27B6713749B6D07FCF8D3DCA01EC9C64778CBAF457D613508F;
```

2. Encrypt the rest of the partitions with operational key as device key:

```
bootgen -arch zynqmp -image stage2a.bif -o hello_e.bin -w on -log error
```

Example of stage2a.bif:

```
stage2a:
{
  [aeskeyfile] aes-opt.nky
  [destination_cpu=a53-0,encryption=aes] hello.elf
}
bootgen -arch zynqmp -image stage2b.bif -o hello1_e.bin -w on -log error
```

Example of stage2b.bif:

```
stage2b:
{
  [aeskeyfile] aes-opt.nky
  [destination_cpu=a53-1,encryption=aes] hello1.elf
}
```

Example of `aes-opt.nky` for stage2a and stage2b:

```
Device xc7z020clg484;
Key 0 229C993D1310DD27B6713749B6D07FCF8D3DCA01EC9C64778CBAF457D613508F;
IV 0 F7F8FDE08674A28DC6ED8E37;
```

3. Use Bootgen to stitch the above to form a complete image:

Use bootgen to stitch the above, to form a complete image.

Example of stage3.bif:

```
stage3:
{
  [bootimage]fsbl_e.bin
  [bootimage]hello_e.bin
  [bootimage]hello1_e.bin
}
```

Note: Key Opt of `aes.nky` is same as Key 0 in `aes-opt.nky` and IV 0 must be same in both nky files.

BIF File for Black Key Stored in eFUSE

For customers who would like to have the device key stored encrypted when not in use, the physical unclonable function (PUF) can be used. Here, the actual red key is encrypted with the PUF key encryption key (KEK), which is an encryption key that is generated by the PUF. The device will decrypt the black key to get the actual red key, so you need to provide the required inputs to Bootgen. The black key can be stored in either eFUSE or the Boot Header. Shutter value indicates the time for which the oscillator values can be captured for PUF. This value must always be `0x100005E`.

For more details, refer to "Storing Keys in Encrypted Form (Black)" in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

The following example shows storage of the black key in eFUSE.

```
the_ROM_image:
{
  [pskfile]PSK.pem
  [sskfile]SSK.pem
  [aeskeyfile]red.nky
  [keysrc_encryption] efuse_blk_key
  [fsbl_config] shutter=0x0100005E
  [auth_params] ppk_select=0
  [bootloader, encryption = aes, authentication = rsa,
destination_cpu=a53-0]fsbl.elf
  [bh_key_iv] black_key_iv.txt
}
```

BIF File for Black Key Stored in Boot Header

The following BIF file sample shows boot header black key encryption:

```
the_ROM_image:
{
  [aeskeyfile] redkey.nky
  [keysrc_encryption] bh_blk_key
  [bh_keyfile] blackkey.txt
  [bh_key_iv] black_key_iv.txt
  [fsbl_config] pufhd_bh , puf4kmode , shutter=0x0100005E, bh_auth_enable
  [pskfile] PSK.pem
  [sskfile] SSK.pem
  [bootloader,authentication=rsa , encryption=aes, destination_cpu=a53-0] fsbl.elf
  [puf_file] hlprdata4k.txt
}
```

Note: Authentication of boot image is compulsory for using black key encryption.

To generate or program eFUSE with black key, see Zynq eFUSE PS API in [Appendix J, XilSKey Library v6.7](#).

Bif File for Obfuscated Form (Gray) key stored in eFUSE

If you would like to have the device key store in obfuscated form, you can encrypt the actual red key with the family key which is an encryption key. Device will decrypt the obfuscated key to get the actual red key. Hence, you need to provide the required inputs to Bootgen. The obfuscated key can be stored in either eFUSE or the Boot Header.

For more details, see Storing Keys in Obfuscated Form (Gray) section in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085)[\[Ref 11\]](#).

Note: The Family key is the same for all devices within a given ZynqMP SoC family. This solution allows you to distribute the Obfuscated key to contract manufacturer's without disclosing the actual user key.

The following example shows storage of the obfuscated key in eFUSE:

```
the_ROM_image:
{
  [aeskeyfile] red.nky
  [keysrc_encryption] efuse_gry_key
  [bh_key_iv] bhkeyiv.txt
  [bootloader, encryption=aes, destination_cpu=a53-0] fsbl.elf
}
```

The following example shows storage of the obfuscated form (gray) key in boot header:

the_ROM_image:

```
{
    [aeskeyfile] red.nky
    [keysrc_encryption] bh_gry_key
    [bh_key_iv] bhkeyiv.txt
    [bh_keyfile] bhkey.txt
    [bootloader, encryption=aes, destination_cpu=a53-0] fsbl.elf
}
```

To generate obfuscated key with family key:

Use Xilinx tools (Bootgen) to create the Obfuscated key. However, the family key is not distributed with the Xilinx development tools. It is provided separately. The family key received from Xilinx should be provided in the `bif` as shown in the example below.



IMPORTANT: To receive the family key, please contact secure.solutions@xilinx.com.

Sample `bif` to generate Obfuscated key:

```
all:
{
    [aeskeyfile] aes.nky
    [familykey] familyKey.cfg
    [bh_key_iv] bhiv.txt
}
```

Using Bootgen to Generate Keys

If you are using Bootgen to create keys, NIST approved KDF is used, which is Counter Mode KDF with CMAC as the PRF.

With a Single Key/IV pair:

- If seed is specified - Key Generation is based on Seed.
- If seed is NOT specified - Key Generation is based on Key0.

If an empty file is mentioned, Bootgen generates a seed with time based randomization. This is not a standard like the KDF. This seed will in turn be the input for KDF to generate the Key/IV pairs.

BIF File with Multiple AESKEY Files

The following BIF file samples show the encryptions using aeskey files:

One AES key / partition

You may specify multiple nky files, one for each partition in the image. The partitions are encrypted using the key that is specified before the partition.

```
sample_bif:
{
    [aeskeyfile] test1.nky
    [bootloader, encryption=aes] fsbl.elf
    [aeskeyfile] test2.nky
    [encryption=aes] hello.elf
    [aeskeyfile] test3.nky
    [encryption=aes] app.elf
}
```

The `fsbl.elf` partition is encrypted using the keys from `test1.nky` file. If we assume that the `hello.elf` file has two partitions since it has two loadable sections, then both the partitions are encrypted using keys from `test2.nky` file. The `app.elf` partition is encrypted using keys from `test3.nky` file.

One AES key / each partition (multiple loadable sections scenario)

You may specify multiple nky files, one for each partition in the image. The partitions are encrypted using the key that is specified before the partition. You are allowed to have unique key files for each of the partition created due to multiple loadable sections by having key file names appended with '.1', '.2'...'n' in the same directory of the key file meant for that partition.

```
sample_bif:
{
    [aeskeyfile] test1.nky
    [bootloader, encryption=aes] fsbl.elf
    [aeskeyfile] test2.nky
    [encryption=aes] hello.elf
    [aeskeyfile] test3.nky
    [encryption=aes] app.elf
}
```

The `fsbl.elf` partition is encrypted using the keys from `test1.nky` file. Assume that the `hello.elf` file has three partitions since it has three loadable sections, and `hello.elf.0` is encrypted using the keys from `test2.nky` file, `hello.elf.1` is encrypted using the keys from `test2.1.nky`, and `hello.elf.2` is encrypted using the keys from `test2.2.nky` file. The `app.elf` partition is encrypted using keys from `test3.nky` file.

Using the same `.nky` across multiple partitions, re uses the AES Key and AES Key/IV Pair in each partition. Using the AES key across multiple partitions increases the exposure of the key and may be a security vulnerability. Using the same AES Key/IV Pair across multiple partitions is a violation of the standard. To avoid the re-use of AES Key/IV pair, Bootgen

increments the IV with the partition number. To avoid the re-use of both AES Key and AES Key/IV pair, Bootgen allows you to provide multiple `.nky` files, one for each partition.



IMPORTANT: *To avoid key re-use, support for single nky file across multiple partitions will be deprecated.*



CAUTION! *Using a single .nky file with multiple partitions means that the same key is being used in each partition - which can be a security vulnerability. A warning is issued in the current release with the plan to generate an error in future releases.*

Note: Key0/IV0 - should be the same in all the nky files.

If you specify multiple keys and if the number of keys are less than the number of blocks to be encrypted, it is ERRORED OUT.

If you need to specify multiple Key/IV pairs, you must specify (number-of-blocks+1) pairs.

The extra Key/IV pair is for SH. Ex: If blocks=4;8;16 - you have to specify 4+1=5 Key/IV pairs.

Authentication

The SHA hardware accelerator included in the Zynq UltraScale+ MPSoC implements the SHA-3 algorithm and produces a 384-bit digest. It is used together with the RSA accelerator to provide image authentication and the AES-GCM is used to decrypt the image. These blocks (SHA-3/384,RSA and AES-GCM) are hardened and part of crypto interface block (CIB).

Authentication flow treats the FSBL as raw data, where it makes no difference whether the image is encrypted or not. There are two level of keys: primary key (PK) and secondary Key (SK).

Each key has two complementary parts: secret key and public key:

- PK contains primary public key (PPK) and primary secret key (PSK).
- SK contains secondary public key (SPK) and secondary secret key (SSK).

The hardened RSA block in the CIB is a Montgomery multiplier for acceleration of the big math required for RSA. The hardware accelerator can be used for signature generation or verification. The ROM code only supports signature verification. Secret keys are only used in the signature generation stage when the certificate is generated.



IMPORTANT: *Signature generation is not done on the device, but in software during preparation of the boot image.*

To better understand the format of the authentication certificate, see *Bootgen User Guide* (UG1283) [Ref 23].

The PPK and SPK keys authenticate a partition.

PSK and SSK are used to sign the partition.

The equations for each signature (SPK, boot header, and boot image) are listed here:

- SPK signature. The 512 bytes of the SPK signature is generated by the following calculation:

```
SPK signature = RSA(PSK, padding || SHA(SPK+ auth_header)).
```

- Boot header signature. The 512 bytes of the boot header signature is generated by the following calculation:

```
Boot header signature = RSA(SSK, padding || SHA(boot header)).
```

- Boot image signature. The 512 bytes of the boot image signature is generated by the following calculation:

```
BI signature = RSA(SSK, padding || SHA(PFW + FSBL + authentication certificate)).
```

Note: For SHA-3 authentication, always use Keccak SHA3 to calculate hash on boot header, PPK hash and boot image. NIST-SHA3 is used for all other partitions which are not loaded by ROM.

Bootgen supports RSA signature generation only. The modulus, exponentiation and precalculated $R^2 \text{ Mod } N$ are required.

Software is supported only for RSA public key encryption, for encrypting the signature RSA engine requires modulus, exponentiation and pre-calculated $R^2 \text{ Mod } N$, all these are extracted from keys.

BIF File with SHA-3 Boot Header Authentication and PPK0

The following BIF file sample supports the BH RSA option. This option supports integration and test prior to the system being fielded. For more details, see "Integration and Test Support (BH RSA Option)" in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11]

The BIF file is for SHA-3 boot header authentication, where actual PPK hash is not compared with the eFUSE stored value.

```
the_ROM_image: {
  [fsbl_config] bh_auth_enable
  [auth_params] ppk_select=0; spk_id=0x00000000
  [pskfile] primary_4096.pem
  [sskfile] secondary_4096.pem
  [bootloader, authentication=rsa, destination_cpu=a53-0] fsbl.elf
  [pmufw_image, authentication=rsa] xpfw.elf
}
```

BIF File with SHA-3 eFUSE RSA Authentication and PPK0

The following BIF file sample shows eFUSE RSA authentication using PPK0 and SHA-3.

```
the_ROM_image:
{
  [auth_params]ppk_select=0;spk_id=0x584C4E58
  [pskfile]psk.pem
  [sskfile]ssk.pem
  [bootloader, authentication = rsa, destination_cpu=a53-0]zynqmp_fsbl.elf
  [destination_cpu = a53-0, authentication = rsa]Application.elf
}
```

Enhanced RSA Key Revocation Support

The RSA key provides the ability to revoke the secondary keys of one partition without revoking them for all partitions.

Note: Primary key should be the same across all partitions.

This is achieved by using **USER_FUSE0** to **USER_FUSE7** eFuses (one can revoke up to 256 keys, if all are not required for their usage) with the new BIF parameter **spk_select**.

The following BIF file sample shows enhanced user fuse revocation:

Image header and FSBL uses different SSK's for authentication(ssk1.pem and ssk2.pem respectively) with the following bif input.

```
the_ROM_image:

{
  [auth_params]ppk_select = 0
  [pskfile]psk.pem
  [sskfile]ssk1.pem
  [bootloader, authentication = rsa, spk_select = spk-efuse, spk_id = 0x12345678,
  sskfile = ssk2.pem]zynqmp_fsbl.elf
  [destination_cpu =a53-0, authentication = rsa, spk_select = user-efuse, spk_id = 200,
  sskfile = ssk3.pem]Application1.elf
  [destination_cpu =a53-0, authentication = rsa, spk_select = spk-efuse, spk_id =
  0x12345678, sskfile = ssk4.pem]Application2.elf
}
```

Same SSK will be used for both Image header and FSBL (ssk2.pem), if separate SSK is not mentioned.

the_ROM_image:

```
{
  [auth_params]ppk_select = 0
  [pskfile]psk.pem
  [bootloader, authentication = rsa, spk_select = spk-efuse, spk_id = 0x12345678,
  sskfile = ssk2.pem]zynqmp_fsbl.elf
  [destination_cpu =a53-0, authentication = rsa, spk_select = user-efuse, spk_id = 200,
  sskfile = ssk3.pem]Application1.elf
  [destination_cpu =a53-0, authentication = rsa, spk_select = spk-efuse, spk_id =
  0x12345678, sskfile = ssk4.pem]Application2.elf
}
```

spk_select = spk-efuse indicates that **spk_id** eFuse will be used for that partition.

spk_select = user-efuse indicates that user eFuse will be used for that partition. Partitions loaded by CSU ROM will always use spk_efuse.

Note: The **spk_id** eFuse specifies which key is valid. Hence, the ROM checks the entire field of **spk_id** eFuse against the SPK ID to make sure its a bit for bit match.

The user eFuse specifies which key ID is not valid (has been revoked). Hence, the firmware (non-ROM) checks to see if a given user eFuse that represents the SPK ID has been programmed.

Bitstream Authentication Using External Memory

Authentication of bitstream is different from all other partitions. The FSBL can be wholly contained within the OCM, and therefore authenticated and decrypted inside of the device. For the bitstream, the size of the file is so large that it cannot be wholly contained inside the device and external memory must be used. The use of external memory creates a challenge to maintain security because an adversary may have access to this external memory.

The following section describes how the bitstream is authenticated securely using external memory.

Bootgen

When bitstream is requested for authentication, Bootgen divides the whole bitstream into 8 MB blocks and has an authentication certificate for each block.

If a bitstream is not in multiples of 8 MB, the last block contains the remaining bitstream data.

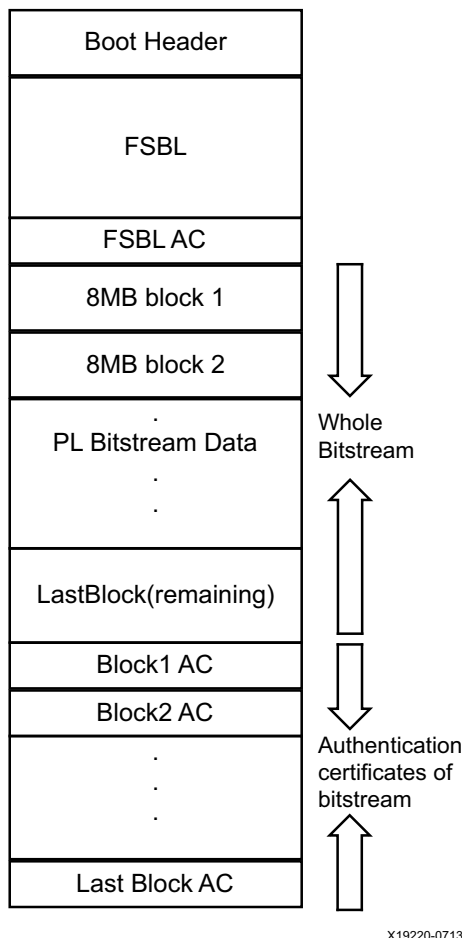


Figure 8-1: Bitstream Blocks

When authentication and encryption are both enabled, encryption is first done on the bitstream. Then Bootgen divides the encrypted data into blocks and places an authentication certificate for each block.

Software

To securely authenticate the bitstream partition, FSBL uses the ATF section's OCM memory to copy the bitstream in chunks from FLASH or DDR. Therefore, while creating a boot image, the bitstream partition should be before ATF partition. Otherwise ATF memory is over-written while processing the bitstream partition.

The workflow for the DDR and DDR-less systems is nearly identical. The only difference is that for systems with the DDR, FSBL copies the entire bitstream partition (bitstream and authentication certificates) to the DDR from the FLASH devices, because DDR is faster to access. FSBL then, each time, copies a chunk of bitstream from the DDR. For the DDR-less systems, FSBL copies a chunk of bitstream directly from the FLASH devices.

The following is the software workflow for authenticating the bitstream:

1. FSBL identifies the availability of the DDR on the system based on the XFSBL_PS_DDR macro. FSBL has two buffers in OCM, **ReadBuffer** buffer of size 56 KB and **HashsOfChunks []** to store intermediate hashes calculated for each 56 KB of 8 MB blocks.
2. FSBL copies a 56 KB chunk from the first 8 MB block to **ReadBuffer**.
3. FSBL calculates hash on 56 KB and stores in **HashsOfChunks**.
4. FSBL repeats the previous steps until the entire 8 MB of block is completed.
Note: 56 KB is taken for performance; it can be of any size.
5. FSBL authenticates the bitstream.
6. Once the authentication is successful, FSBL starts copying 56 KB starting from the first block which is located in DDR/FLASH to **ReadBuffer**, calculates the hash, and then compares it with the hash stored at **HashsOfChunks**.
7. If hash comparison is successful, FSBL transmits data to PCAP via DMA (for unencrypted bitstream) or AES (if encryption is enabled).
8. FSBL repeats the previous two steps until the entire 8MB block is completed.
9. Repeats the entire process for all the blocks of bitstream.

Note: If there is any failure at any stage, PL is reset and FSBL is exited.

The bitstream is directly routed to PCAP via CSU DMA by configuring secure stream switch.

For a DDR system, the whole encrypted bitstream is copied to DDR. For DDR-less system, decryption is copied to OCM(ATF section) in chunks.

Note: Xilinx recommends that you have a bitstream partition immediately after the FSBL partition in the boot image.

Run-Time Security

Run-time security involves protecting the system against incorrectly programmed or malicious devices corrupting the system memory or causing a system failure.

To protect the system, it is important to secure memory and the peripherals during a software execution. The Zynq UltraScale+ MPSoC devices provide memory and peripheral protection through the following blocks:

- [Arm Trusted Firmware](#)
- [Xilinx Memory Protection Unit](#)
- [Xilinx Peripheral Protection Unit](#)
- [System Memory Management Unit](#)
- [A53 Memory Management Unit](#)
- [R5 Memory Protection Unit](#)

One of the runtime security features is access controls on the PMU and CSU global registers from Linux. These registers are classified into two lists:

- The white list which is accessible all the time by default.
- The black list which is accessible only when a compile time flag is set.

For more details see

Arm Trusted Firmware

The Zynq UltraScale+ MPSoC device incorporates the standard execution model advocated for Armv8 cores. This model runs the normal operating system at a less privileged state, requiring it to request access to security-sensitive hardware or registers using a proxy software called as secure monitor code (SMC). The specific SMC provided by Xilinx for the Zynq UltraScale+ MPSoC device is a part of Linaro Arm Trusted Firmware (ATF). Xilinx neither requires nor provides a Trusted OS as secure boot functionality is available through the CSU and PMU as previously described. However, the ATF provided by Xilinx does include hooks which allow customers to add their own Trusted OS for incorporation of additional trusted applications. ATF includes a secure monitor for switching between the secure and the non-secure world.

The primary purpose of ATF is to ensure that the system modules (drivers, applications) do not have access to a resource unless absolutely necessary. For example, Linux should be prevented from accessing the region where the public key is stored in the SoC. Likewise, the driver for a crypto block does not need to know the current session key; the session key

could be programmed by the key negotiation algorithm and stored in a secure location within the crypto block.

PSCI is the interface from non-secure software to firmware implementing power management use-cases (for example, secondary CPU boot, hotplug, and idle).

It might be necessary for supervisory systems running at exception levels to perform actions, such as restoring context and switches to the power state of core.

Non-secure software can access ATF runtime services using the Arm secure monitor call (SMC) instruction.

In the Arm architecture, synchronous control transfers between the non-secure state to a secure state through SMC exceptions, which are generated by the SMC instruction, and handled by the secure monitor. The operation of the secure monitor is determined by the parameters passed in through registers.

Two types of calls are defined:

- Fast calls to execute atomic secure operations
- Standard calls to start preemptive secure operations

Two calling conventions for the SMC instruction defines two function identifiers for the SMC instruction define two calling conventions:

- **SMC32**: A 32-bit interface that either 32-bit or 64-bit client code can use. **SMC32** passes up to six 32-bit arguments.
- **SMC64**: A 64-bit interface used only by 64-bit client code that passes up to six 64-bit arguments.

You define the SMC function identifiers based upon the calling convention. When you define the SMC function identifier, you pass that identifier into every SMC call in register **R0** or **W0**, which determines the following:

- Call type
- Calling convention
- Secure function to invoke

ATF implements a framework for configuring and managing interrupts generated in either security state. It implements a subset of the trusted board boot requirements (TBBR) and the platform design document (PDD) for Arm reference platforms.

The cold boot path is where the TBBR sequence starts when the platform is powered on, and runs up to the stage where it hands-off control to firmware running in the non-secure world in DRAM. The cold boot path starts when you physically turn on the platform.

- You chose one of the CPUs released from reset as the primary CPU, and the remaining CPUs are considered secondary CPUs.
- The primary CPU is chosen through platform-specific means. The cold boot path is mainly executed by the primary CPU, other than essential CPU initialization executed by all CPUs.
- The secondary CPUs are kept in a safe platform-specific state until the primary CPU has performed enough initialization to boot them.

For a warm boot, the CPU jumps to a platform-specific address in the same processor mode as it was when released from reset.

Table 8-1 lists the ATF functions:

Table 8-1: ATF Functions

ATF Functions	Description
<code>bl31_arch_setup();</code>	Generic architectural setup from EL3.
<code>bl31_platform_setup();</code>	Platform setup in BL1.
<code>bl31_lib_init();</code>	Simple function to initialize all BL31 helper libraries.
<code>cm_init();</code>	Context management library initialization routine.
<code>dcsw_op_all(DCCSW);</code>	Cleans caches before re-entering the non-secure software world.
<code>(*bl32_init)();</code>	Function pointer to initialize the BL32 image.
<code>runtime_svc_init();</code>	Calls the initialization routine in the descriptor exported by a runtime service. After a descriptor is validated, its start and end owning entity numbers and the call type are combined to form a unique <code>oen</code> . The unique <code>oen</code> is an index into the <code>rt_svc_descs_indices</code> array. This index stores the index of the runtime service descriptor.
<code>validate_rt_svc_desc();</code>	Simple routine to sanity check a runtime service descriptor before it is used.
<code>get_unique_oen();</code>	Gets a unique <code>oen</code> .
<code>bl31_prepare_next_image_entry();</code>	Programs EL3 registers and performs other setup to enable entry into the next image after BL31 at the next <code>ERET</code> .
<code>bl31_get_next_image_type();</code>	Returns the <code>next_image_type</code> .
<code>bl31_plat_get_next_image_ep_info(image_type);</code>	Returns a reference to the <code>entry_point_info</code> structure corresponding to the image that runs in the specified security state.
<code>get_security_state();</code>	Gets the security state.

Table 8-1: ATF Functions (Cont'd)

ATF Functions	Description
<code>cm_init_context()</code>	Initializes a <code>cpu_context</code> for the first use by the current CPU, and sets the initial entry point state as specified by the <code>entry_point_info</code> structure.
<code>cm_get_context_by_mpidr()</code>	Returns a pointer to the most recent <code>cpu_context</code> structure for the CPU identified by MPIDR that was set as the context for the specified Security state. NULL is returned if no such structure has been specified.
<code>get_scr_el3_from_routing_model()</code>	Returns the cached copy of the <code>SCR_EL3</code> which contains the routing model (expressed through the <code>IRQ</code> and <code>FIQ</code> bits) for a security state that is stored through a previous call to <code>set_routing_model()</code> .
<code>get_el3state_ctx()</code>	Populates <code>EL3</code> state so that <code>ERET</code> jumps to the correct entry.
<code>get_gpregs_ctx()</code>	Stores the <code>X0-X7</code> value from the entry point into the context.
<code>cm_prepare_el3_exit()</code>	Prepares the CPU system registers for first entry into the secure or the non-secure software world. • If execution is requested to <code>EL2</code> or <code>hyp</code> mode, <code>SCTLR_EL2</code> is initialized. • If execution is requested to the non-secure <code>EL1</code> or <code>svc</code> mode, and the CPU supports <code>EL2</code>; then <code>EL2</code> is disabled by configuring all necessary <code>EL2</code> registers. For all entries, the <code>EL1</code> registers are initialized from the <code>cpu_context</code> .
<code>cm_get_context(security_state);</code>	Gets the context of the security state.
<code>el1_sysregs_context_restore</code>	Restores the context of the system registers.
<code>cm_set_next_context</code>	Programs the context used for exception return. This initializes the <code>SP_EL3</code> to a pointer to a <code>cpu_context</code> set for the required security state.
<code>bl31_late_platform_setup();</code>	Sets up the platform.
<code>bl31_register_bl32_init</code>	Initializes the pointer to <code>BL32_init</code> function.
<code>bl31_set_next_image_type</code>	Accessor function to help runtime services determine which image to execute after <code>BL31</code> .

For more information about ATF, see *Arm Trusted Firmware documentation*[\[Ref 40\]](#).

FPGA Manager Solution

The FPGA Manager in the Zynq UltraScale+ MPSoC provides an interface to download different types of bitstreams (full, partial, authenticated, encrypted and so on) during runtime from Linux environment. The key features of the FPGA Manager are as follows:

- Full bitstream loading
- Partial Reconfiguration (partial bitstream loading)
- Encrypted full/partial bitstream loading
- Authenticated full/partial bitstream loading
- Authenticated and encrypted full/partial bitstream loading
- Readback of configuration registers
- Readback of bitstream (configuration data)

FPGA Manager Architecture

The following figure shows the architecture of the FPGA Manager.

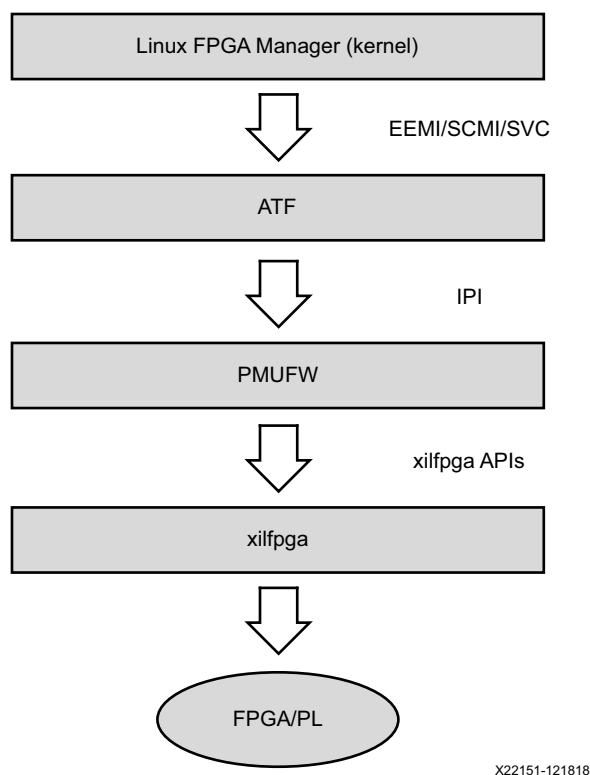
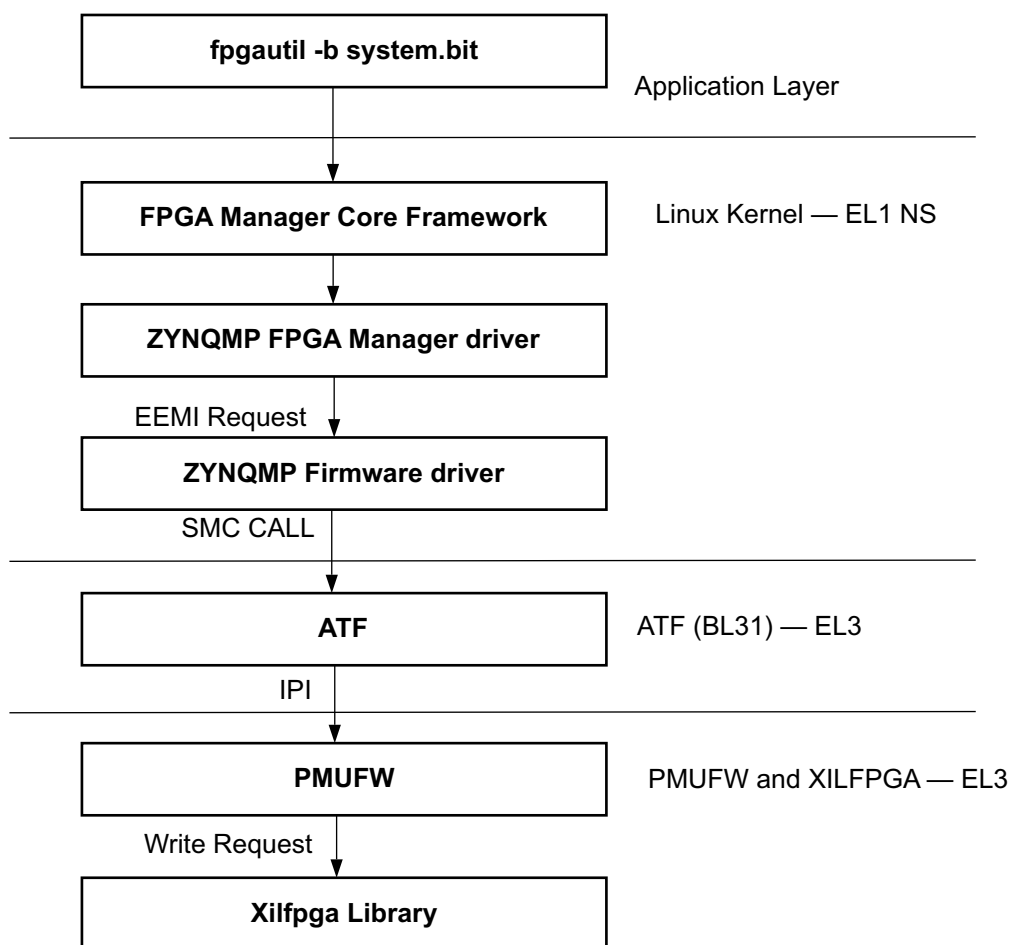


Figure 8-2: FPGA Manager Architecture Block Diagram

Execution Flow

FPGA manager provides an abstraction for the user to load bitstream using Linux. The the xilfpga library initializes the PCAP, CSUDMA and other hardware. For more details about xilfpga, see the [Appendix L, XilFPGA Library v5.0](#) section.

To load a bitstream, the FPGA manager allocates the required memory and invokes the EEMI API using the FPGA LOAD API ID. This request is a blocking call. The FPGA Manger waits for response from the ATF and response is provided to the fpga core layer which passes it to the application. This is described in the following figure:



X22152-121818

Figure 8-3: FPGA Manager Flow

Xilinx Memory Protection Unit

The Xilinx® memory protection unit (XMPU) is a region-based memory protection unit. For more details, see “System Protection Unit” chapter in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

Protecting Memory with XMPU

To understand more about XMPU features and functionality, refer to “System Protection Unit” chapter in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

Configuring XMPU Registers

The XMPU is configurable either one-time or through trust-zone access from a secure master (PMU, APU TrustZone secure master, or RPU when configured as secure master). At boot time, XMPU can be configured and its configuration can be locked such that it can only be reconfigured at next power-on reset. If the configuration is not locked, then XMPU can be reconfigured any number of times by secure master accesses. If you choose to configure the XMPU dynamically, you must also consider many aspects including the idling of active devices and the AXI bus.

For more information on using the XMPU please see *Isolation Methods in Zynq UltraScale+ MPSoCs* (XAPP1320) [\[Ref 10\]](#).

Xilinx Peripheral Protection Unit

To understand more about Xilinx peripheral protection unit (XPPU) features and functionality, see this [link](#) to the "Xilinx Peripheral Protection Unit" section of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [\[Ref 11\]](#).

For more information on using the XMPU please see *Isolation Methods in Zynq UltraScale+ MPSoCs* (XAPP1320) [\[Ref 10\]](#).

System Memory Management Unit

The system memory management unit (SMMU) offers isolation services. The SMMU provides address translation for an I/O device to identify more than its actual addressing capability. In absence of memory isolation, I/O devices can corrupt system memory. The SMMU provides device isolation to prevent DMA attacks. To offer isolation and memory protection, it restricts device access for DMA-capable I/O to a pre-assigned physical space.

To understand more about SMMU features and functionality, see this [link](#) to the "System Memory Management Unit" section of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [\[Ref 11\]](#).

A53 Memory Management Unit

The memory management unit (MMU) controls table-walk hardware that accesses translation tables in main memory. The MMU translates virtual addresses to physical addresses. The MMU provides fine-grained memory system control through a set of

virtual-to-physical address mappings and memory attributes held in page tables. These are loaded into the translation lookaside buffer (TLB) when a location is accessed.

To understand more about MMU features and functionality, see this [link](#) to the “Memory Management Unit” section of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

R5 Memory Protection Unit

The memory protection unit (MPU) enables you to partition memory into regions and set individual protection attributes for each region. When the MPU is disabled, no access permission checks are performed, and memory attributes are assigned according to the default memory map. The MPU has a maximum of 16 regions.

To understand more about MPU features and functionality, see this [link](#) to the “Memory Protection Unit” section of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

Platform Management

Introduction

Zynq® UltraScale+™ MPSoC devices are designed for high performance and power-sensitive applications in a wide range of markets. The system power consumption depends on how intelligently software manages the various subsystems – turning them on and off only when they are needed and, also at a finer level, trading off performance for power. This chapter describes the features available to manage power consumption, and how to control the various power modes using software.

Platform Management in PS

To increase the scalability in the platform management unit (PMU), the Zynq UltraScale+ MPSoC device supports multiple power domains such as:

- Full Power Domain
- Low Power Domain
- Battery Power Domain
- PL Power Domain

For details on the [PMU](#) and the optional PMU firmware (PMU firmware) functionality, see the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [\[Ref 11\]](#).

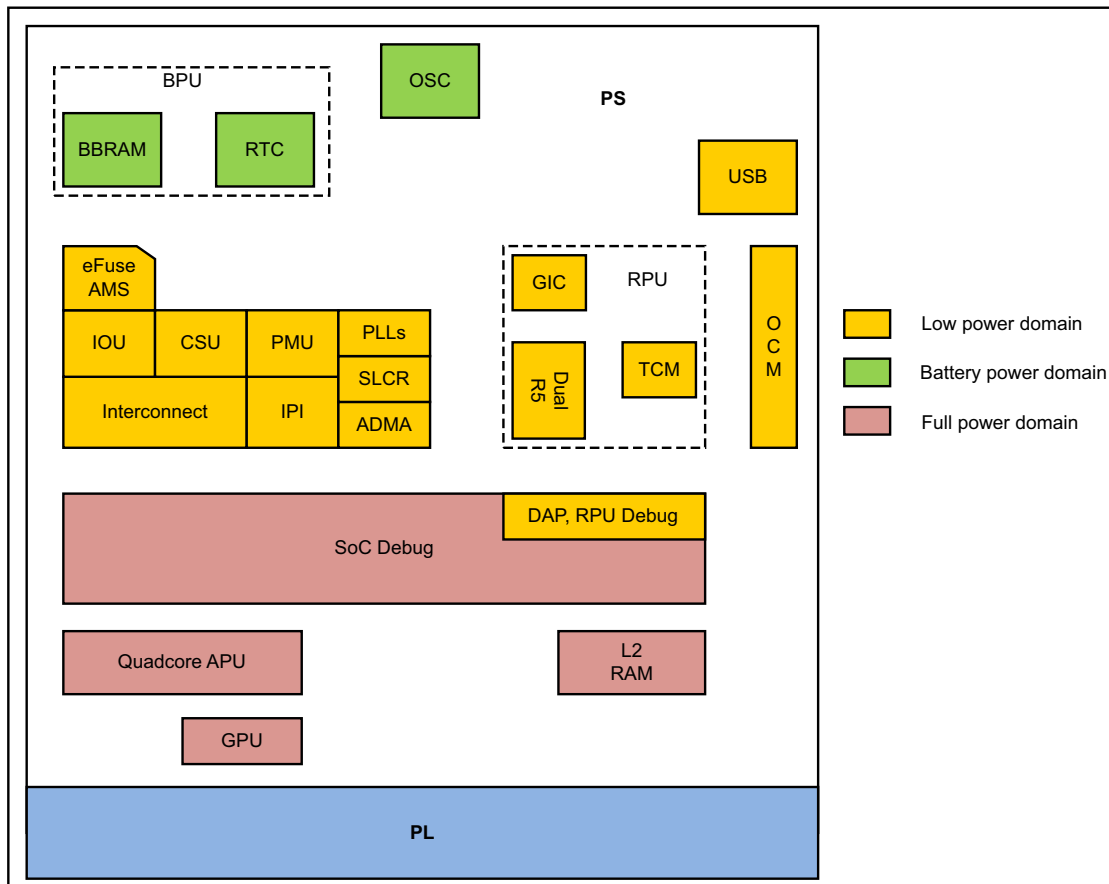
For more information on dynamically changing the PS clocks, see [Chapter 14, Clock and Frequency Management](#).

The PS block offers high levels of functionality and performance. At the same time, there is a strong need to optimize the power consumption of this block with respect to the functionality and performance that is necessary at each stage of the operation.

The Zynq UltraScale+ MPSoC device has multiple power rails. Each rail can be turned off independently, or can use a different voltage. Many of the blocks on a specific power rail implement power-gating, which allows blocks to be gated off independently.

Examples of these power-gated domains are the: Arm® Cortex™-A53 and the Cortex-R5F processors, GPU pixel processors (PP), large RAMs, and individual USBs.

The following figure shows a block diagram of the platform management at the PS level.



X19226-071317

Figure 9-1: Platform Management at the PS Level

From the power perspective, Zynq UltraScale+ MPSoC devices offers the following modes of operation at the PS level:

- Full-power operation mode
- Low-power operation mode
- Deep-sleep mode
- Shutdown mode
- Battery-power mode

The following sections describe these modes.

Full-Power Operation Mode

In the full-power operation mode (shown as full power domain in [Figure 9-1](#)), the entire system is up and running. Total power dissipation depends on the number of components that are running: their states and their frequencies. In this mode, dynamic power will likely dominate the total power dissipation.

To optimize static and dynamic power in full-power mode, all large modules have their own *power islands* to allow them to be shut down when they are not being used.

To understand about full-power operation mode, see this [link](#) to the “Platform Management Unit” chapter in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [[Ref 11](#)].

Low-Power Operation Mode

In the low-power operation mode, a subset of the PS (shown as low-power domain in the [Figure 9-1](#)) is powered up that includes: the PMU, RPU, CSU, and the IOU.

In this mode, the ability to change system frequency allows power dissipation to be tuned. The CSU must be running continuously to monitor the system security against SEU and tampering. In this mode, the ability to change system frequency allows power dissipation to be tuned.

The low-power mode includes all lower-domain peripherals*. Among the blocks within the low-power mode, PLLs, dual Cortex-R5F, USBs, and the TCM and OCM block RAMs offer power gating.

You can control power gating to different blocks through software by configuring the `LPD_SLCR` registers. See the *SLCR_Registers* link [[Ref 12](#)] for more information on `LPD_SLCR` register.

* SATA, PCIe, and DisplayPort blocks are within the full power domain (FPD).

Deep-Sleep Operation Mode

Deep-Sleep is a special mode in which the PS is suspended and waiting a wake-up signal. The wake can be triggered by the MIO, the USB, or the RTC.

Upon wake, the PS does not have to go through the boot process, and the security state of the system is preserved. The device consumes the lowest power during this mode while still maintaining its boot and security state.

In this mode, all the blocks outside the low-power domain, such as the system monitor and PLLs, are powered down. In LPD, Cortex-R5F is powered down. Because this mode has to preserve the context, TCM and OCM are in a retention state.

Shutdown Mode

Shutdown mode powers down the entire APU core. This mode is applicable to APU only. During shutdown, the entire processor state, including its caches, is completely lost; therefore, software is required to save all states before requesting the PMU to power down the APU core.

When a CPU is shutdown, it is expected that any interrupt from a peripheral that is associated with that CPU to initiate its power up; therefore, the interrupt lines to an APU core are also routed to the PMU interrupt controller, and are enabled when the APU core is powered down.

The *Embedded Energy Management Interface Specification* (UG1200) [Ref 10] describe the APIs to invoke shutdown.

For more details, see this [link](#) to the “Platform Management Unit Programming Model” section in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 8].

Battery-Powered Mode

When the system is OFF, limited functionality within the PS must stay ON by operating on a battery. The following features operate within the battery-powered domain PS (shown in the [Figure 9-1](#)):

- Battery-backed RAM (BBRAM) to hold key for secure configuration
- Real-time clock (RTC) including the crystal I/O

The Zynq UltraScale+ MPSoC device includes only one battery-powered domain and only the functions those are implemented in the PS can be battery backed-up. The required I/O for the battery-powered domain includes the battery power pads and the I/O pads for the RTC crystal.

Wake Up Mechanisms

To understand about wake up mechanisms, see this [link](#) to the “Platform Management Unit Operation” section of “Chapter 6, Platform Management Unit” of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

Platform Management for Memory

The Zynq UltraScale+ MPSoC devices include large RAMs like L2 cache, OCM, and TCM. These RAMs support various power management features such as: clock gating, power gating, and memory retention modes.

- TCM and OCM support independent power gating and retention modes.
- The L2 cache controller supports dynamic clock gating, retention, and shutdown modes to reduce power consumption at a finer granularity.

DDR Controller

The DDR controller implements the following mechanisms to reduce its power consumption:

- **Clock Stop:** When enabled, the DDR PHY can stop the clocks to the DRAM.
 - For DDR2 and DDR3, this feature is only effective in self-refresh mode.
 - For LPDDR2, this feature becomes effective during idle periods, power-down mode, self-refresh mode, and deep power-down mode.
- **Pre-Charge Power Down:** When enabled, the DDRC dynamically uses pre-charge power down mode to reduce power consumption during idle periods. Normal operation continues when a new request is received by the controller.
- **Self-Refresh:** The DDR controller can dynamically put the DRAM into self-refresh mode during idle periods. Normal operation continues when a new request is received by the controller.

In this mode, DRAM contents are maintained even when the DDRC core logic is fully powered down; this allows stopping the DDR3X clock and the DCI clock that controls the DDR termination.

Platform Management for Interconnects

The Interconnect lays across multiple power rails and power islands which can be on or off at different times. To ease the implementation, in most cases, the clocks for two power domains that communicate with one another must be asynchronous; consequently, requiring synchronizers on their interconnection.

To ease timing, the power domain is placed exactly at the clock crossing. The synchronizer must be implemented as two separate pieces with each placed in one of the two domains that are connected through the synchronizer, creating a bridge.

The bridge consists of a slave interface and a master interface with each lying entirely within a single power and clock domain. The clock frequencies at the interfaces can vary independent of each other, and each half can be reset independent of the other half.

Level shifters or clamping, or both, must be implemented between the two halves of the bridge for multi-voltage implementation or power-off.

Also, the bridge keeps track of open transactions, as follows:

- When the bridge receives a power-down request from the PMU, it logs that request.
- All new transactions return an error while the previously open transactions are being processed as usual until the transaction counter becomes 0. At that point, the bridge acknowledges to the PMU that it is safe to shut down the master or slave connected to the bridge.
- The entire Interconnect shuts down only when all bridges within that interconnect are idle.

For more details, see this [link](#) to the “PMU Interconnect” sub-section in the “Platform Management Unit” chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

PMU firmware

Every system configuration that is supported by Xilinx includes PMU firmware in addition to the functions of power-up and sleep management. The PMU can execute user programs that implement advanced system monitoring and power management algorithms. In this mode, an application or a real-time processor copies the power management program into the PMU internal RAM through an inbound LPD switch. The PMU executes software that implements the required reset, power management, system monitoring, and interrupt controls within all Xilinx supported system configurations.

For more details, see this [link](#) to the “Platform Management Unit Programming Model” section in “Chapter 6” of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

You can use the Xilinx® SDK to create custom PMU firmware. It provides the source code for the PMU firmware template and the necessary library support. For details on how to create an SDK project, see [Chapter 5, Software Development Flow](#).

Power Management Framework

The *Embedded Energy Management Interface Specification* (UG1200) [Ref 10] describe how to use the power API functions.

Note: There is no difference between bare metal, FreeRTOS, or Linux-specific power management Xilinx EEMI API offerings.

Platform Management Unit Firmware

Introduction

The Platform Management Unit (PMU) in Zynq® UltraScale+™ MPSoC devices is located within the Low-power sub-system. The PMU consists of a MicroBlaze processor which loads executable code from 32 KB ROM and 128 KB RAM into flat memory space. The PMU controls the power-up, reset, and monitoring of resources within the system including inter-processor interrupts and power management registers. The ROM is preloaded with PMU bootROM (PBR) which performs pre-boot tasks and enters a service mode. PMU_FW must be loaded to provide advanced system functionality for each of the Xilinx supported use-cases. This chapter explains the features and functionality of PMU firmware developed for Zynq UltraScale+ MPSoC device.

Features

The following are the key features of PMU firmware:

- Provides modular functionality: PMU firmware is designed to be modular. It enables you to add a new functionality in the form of a module
- Provides easy customization of modules
- Easily configurable to include only the required functionality for a user application
- Support communication with other components in the system over IPI (Inter-Processor Interrupt)
- Run time configurability for EM module
- Support for various Power Management features

PMU firmware Architecture

Figure 10-1 shows the architecture block diagram of PMU firmware. PMU firmware is designed to be modular and enables adding new functionality in the form of modules. Each functionally distinct feature is designed as a module so that the PMU firmware can be configured to include only the required functionality for a user application. This type of modular design allows easy addition of new features and optimizes memory footprint by disabling unused modules.

PMU firmware can be divided into **Base Firmware** and **Modules**. PMU Base Firmware does initialization of modules, registering events for the modules, and provides all the common functions that may be required by the modules. These common functions can be categorized into the following APIs:

1. PMU firmware Core APIs
 - a. Scheduler
 - b. Event Manager
 - c. IPI Manager
2. PMU firmware General APIs
 - a. BSP/Utility APIs
 - b. Reset Services APIs
 - c. ROM Services APIs

These APIs can be used by the modules in PMU firmware to perform the specified actions as required.

Platform Management Overview

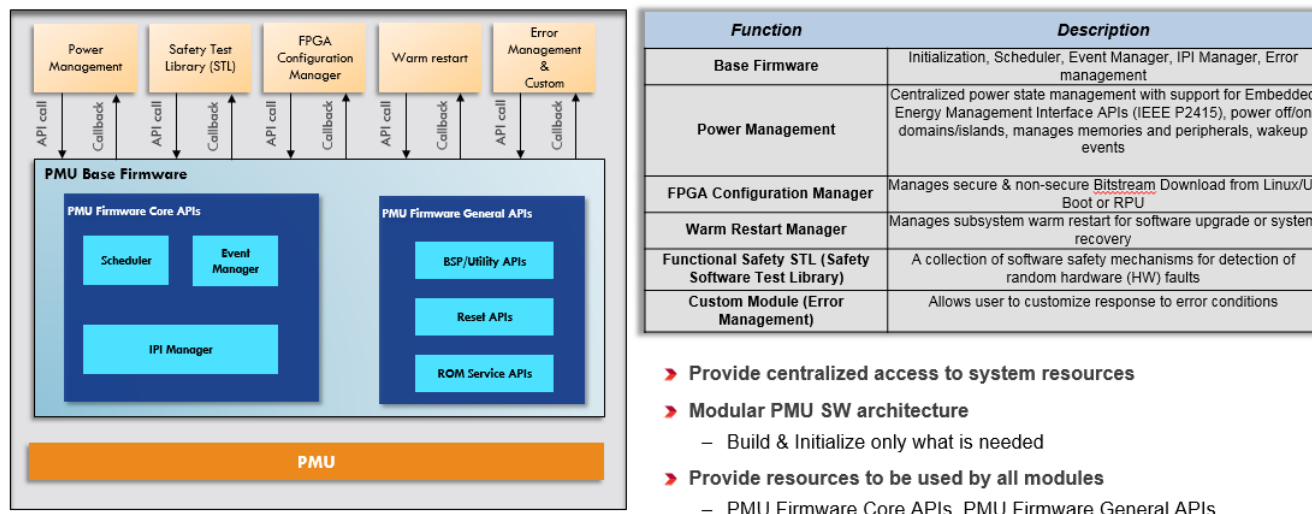


Figure 10-1: PMU firmware Architecture Block diagram

Execution Flow

The initialization in PMU firmware takes place in a normal context. Interrupts are disabled to avoid un-intended interruptions and prevent usage of the system resources before they are properly initialized. After initialization completes, interrupts are enabled and the required tasks are scheduled to be executed. The system enters in to a sleep state. The system wakes up only when an event occurs or the scheduled tasks are triggered and the corresponding handlers are executed. Figure 10-2 shows the state transitions for PMU firmware.

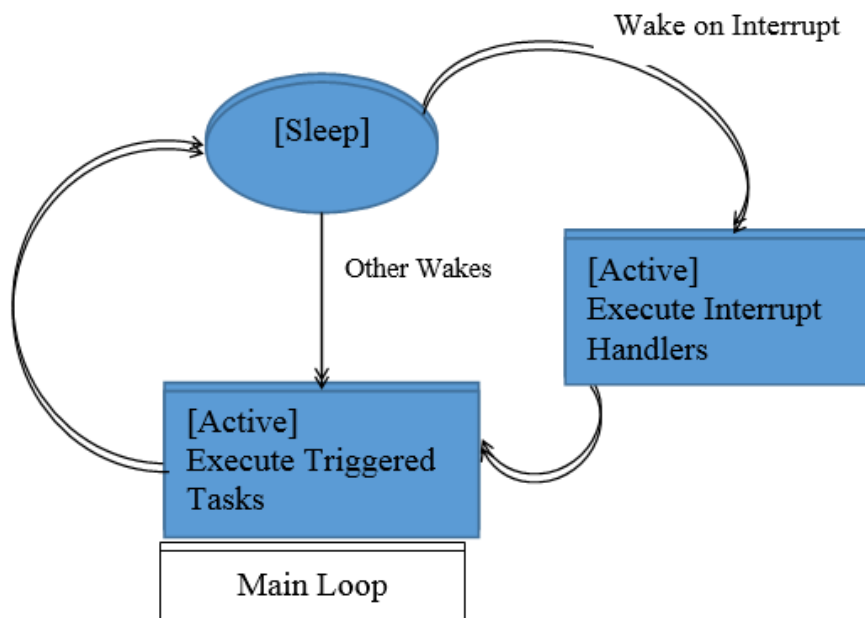


Figure 10-2: State Transitions for PMU firmware in Main Loop

PMU firmware execution flow consists of the following three phases:

1. **Initialization phase:** This phase consists of PMU firmware starting up, performing self-tests and validations, initializing the hardware, creating and initializing modules. Interrupts are disabled during this phase and are enabled at the end.
2. **Post initialization:** In this phase, PMU firmware enters service mode, wherein it enters into sleep and waits for an interrupt.
3. **Waking up:** PMU firmware enters the interrupt context and services the interrupt. After completing this task, it goes back to sleep.

Figure 10-3 depicts the execution flow for PMU firmware.

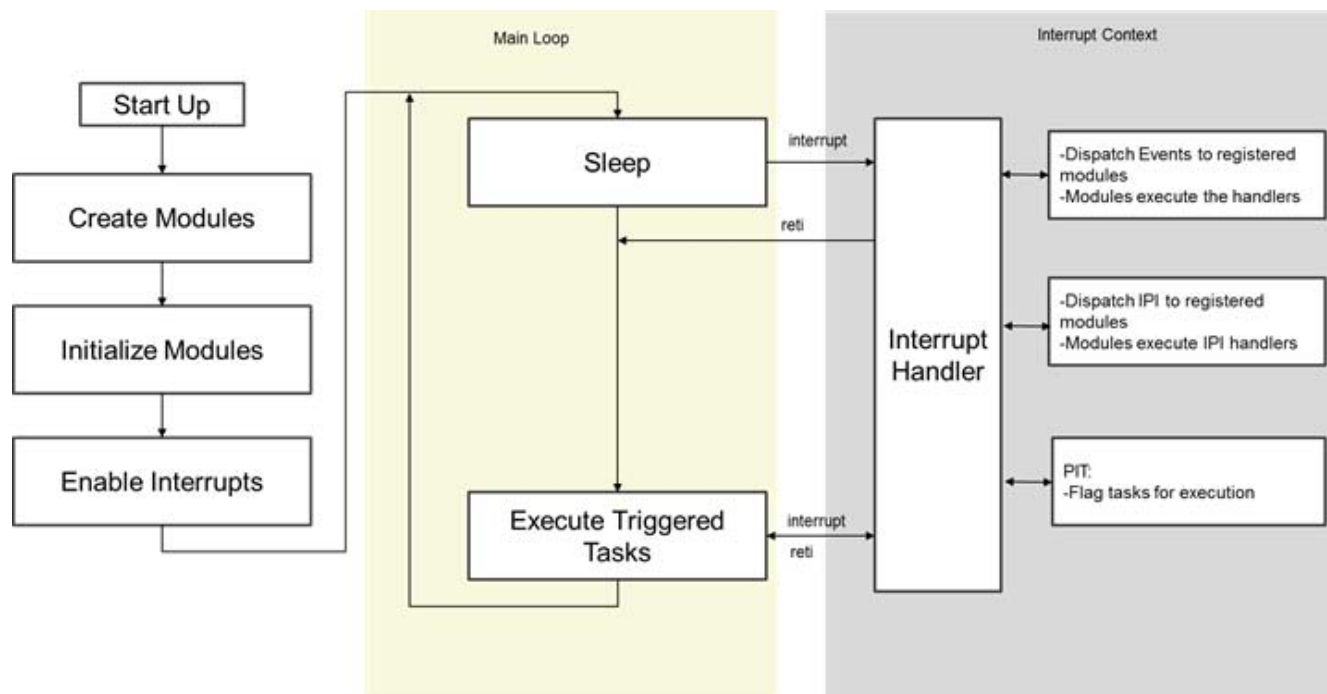


Figure 10-3: Execution Context View for PMU firmware

Handling Inter-Process Interrupts in PMU firmware

IPI is a key interface between PMU firmware and non-PMU entities on the SoC. PMU includes four Inter-Processor Interrupts (IPI) assigned to it and one set of buffers. PMU firmware uses IPI-0 and associated buffers for communication by default, which is initiated by other masters on SoC to PMU. PMU firmware uses IPI-1 and associated buffers for callbacks from PMU to other masters and for communication initiated by PMU firmware.

Figure 10-4 shows the IPI handling stack with interfaces between different components involved in this process. PMU firmware uses IPI driver to send and receive the messages. An IPI manager layer in Base Firmware is implemented over the driver and it takes care of dispatching the IPI message to the registered module handlers based on IPI ID in the first word of the message. Table 10-1 displays the message format for IPI.

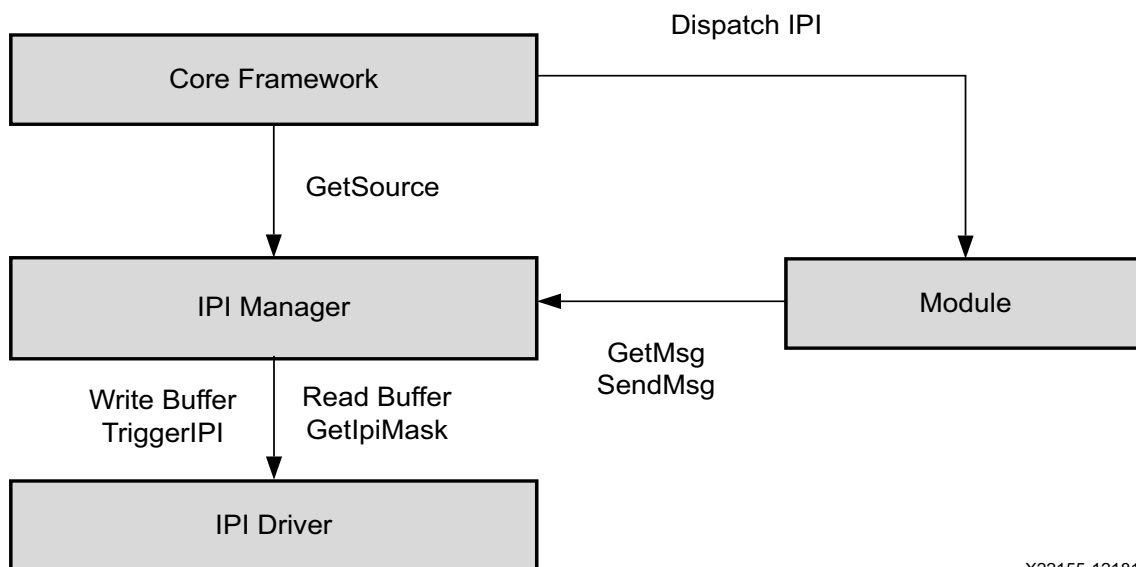
Table 10-1: IPI Message Format

Word	Content	Description
0	Header	<target_module_id, api_id>

Table 10-1: IPI Message Format (Cont'd)

Word	Content	Description
1	Payload	Module dependent payload
2		
3		
4		
5		
6	Reserved	Reserved - for future use
7	Checksum	

IPI-1 is used for the callbacks from PMU to other masters and for communication initiated by PMU firmware. Currently, PM and EM modules use IPIs and this can be taken as reference for implementing custom modules which require IPI messaging.



X22155-121818

Figure 10-4: IPI Handler Stack with Interfaces

PMU firmware provides wrapper APIs around IPI driver functions to send and receive IPI messages. During initialization, PMU firmware initializes the IPI driver and enables IPI interrupt from the masters which are IPI assigned.

Send IPI Message

XPfw_IpiWriteMessage() API is used to send IPI message to target. This function internally calls the IPI driver write API with buffer type as Message buffer.

Parameters

Table 10-2: Send IPI Message

Parameter	Description
ModPtr	Module pointer from where the IPI message is being sent. In IPI message, target_module_id field will be updated with the Module IPI ID information which is present in Module pointer.
DestCpuMask	Destination target IPI ID
MsgPtr	Message Pointer
MsgLen	Message Length

Return

XST_SUCCESS: If message is sent successfully.

XST_FAILURE: If message fails.

Send IPI response

XPfw_IpiWriteResponse() API is used to send the response to the master which sent an IPI message. This function internally calls the IPI driver write API with buffer type as Response buffer.

Parameters

Table 10-3: Send IPI Response

Parameter	Description
ModPtr	Module pointer to check which module received this IPI response
SrcCpuMask	Source IPI ID to read IPI response
MsgPtr	Response Message Pointer
MsgLen	Response Message Length

Return

XST_SUCCESS: If IPI response is read successfully.

XST_FAILURE: If response fails.

Read IPI Message

`XPfw_IpiReadMessage()` is used to read the IPI message received when IPI interrupt comes. This function internally calls the IPI driver read API with buffer type as Message buffer.

Parameters

Table 10-4: Read IPI Message

Parameter	Description
SrcCpuMask	Source IPI ID to read the IPI message
MsgPtr	Message Pointer
MsgLen	Message Length

Return

XST_SUCCESS: If IPI message is read successfully.

XST_FAILURE: If message fails.

Read IPI Response

`XPfw_IpiReadResponse()` is used to read the IPI response for the message sent. This function internally calls the IPI driver read API with buffer type as Response buffer.

Parameters

Table 10-5: Read IPI Response

Parameter	Description
ModPtr	Module pointer to check which module received this IPI response
SrcCpuMask	Source IPI ID to read IPI response
MsgPtr	Response Message Pointer
MsgLen	Response Message Length

Return

XST_SUCCESS: If IPI response is read successfully.

XST_FAILURE: If response fails.

Triggering an IPI

`XPfw_IpiTrigger()` is used to trigger an IPI to the destination. This function internally calls the IPI driver trigger. This function should be called after the IPI message writes IPI buffer.

Parameters

Table 10-6: Triggering an IPI

Parameter	Description
DestCpuMask	Destination target IPI ID

Return

XST_SUCCESS: If IPI is triggered successfully.

XST_FAILURE: If trigger fails.

PMU firmware Modules

PMU firmware consists of the following modules:

1. Error Management (EM)
2. Power Management (PM)
3. Scheduler
4. Safety Test Library (STL)

PMU firmware has a module data structure (`XPfw_Module_t`) which contains the information about the module. This data structure is defined for each module when the module is created. Table 10-7 shows its members.

Table 10-7: Module Data Structure Members

Member	Range	Additional Information
ModId	0.. 31	
CfgInitHandler	Init handler function pointer	Default to NULL
IpiHandler	Handler for IPI manager	Default to NULL
EventHandler	Handler for registered events of the module	Default to NULL
IpId	16-bit IPI ID	Unique to each module

PMU firmware also has a core data structure which contains the list and the details of all modules. Table 10-8 shows its members.

Table 10-8: Core Data Structure Members

Member	Range	Additional Information
ModList array	0.. 31	Module list array (of 32 elements) of Module structure
Scheduler	Scheduler structure	Scheduler task owned by the module
ModCount	0.. 31	
IsReady	Core is ready/dead	
Mode	Safety Diagnostics mode/Normal mode	

Base PMU firmware supports a few APIs that are used by these modules. Also, if you want to create a custom module, these APIs can be used from `xpfw_core.h`.

Creating a Module

`XPfw_CoreCreateMod()` API is called during the startup to create a module. PMU firmware can have maximum of 32 modules. This function checks if the module count reached the maximum count. If not, it fills in the details to core structure ModList and returns this module data structure to the caller. Otherwise, it returns NULL.

Setting up handlers for the Module

Each module can be provided with three handlers which are called during the respective phases as described below:

Table 10-9: Module Handlers

Module Handler	Purpose	API for Registering the Handler	Execution context
Init	Called during the init of the core to configure the module, register for events or add scheduler tasks. This can be used to load the configuration data into the module if required.	<code>XPfw_CoreSetCfgHandler(const XPfw_Module_t *ModPtr, XPfwModCfgInitHandler_t CfgHandler);</code>	StartUp
Event Handler	Called when an event occurs (module should have registered for this event, preferably during the init phase)	<code>XPfw_CoreSetEventHandler(const XPfw_Module_t *ModPtr, XPfwModEventHandler_t EventHandler);</code>	Interrupt
IPI Handler	Called when an IPI message with respective module-id arrives	<code>XPfw_CoreSetIpiHandler(const XPfw_Module_t *ModPtr, XPfwModIpiHandler_t IpiHandler, u16 Ipild);</code>	Interrupt

PMU firmware Build Flags

In PMU firmware, each module can be enabled/disabled based on your requirement. This is achieved by using build flags. [Table 10-10](#) describes the important build flags in PMU firmware and its usage. Please see `xpfw_config.h` file in PMU firmware sources for a complete list of build flags.

Table 10-10: PMU firmware Build Flags

Flag	Description	Prerequisites	Default Setting
XPFW_DEBUG_DETAILED	Enables detailed debug prints in PMU firmware. This feature is supported in 2017.3 release and above		Disabled
PM_LOG_LEVEL	Enables print based debug functions for PM module. Possible values are: - Alerts - Errors - Warnings - Information Higher numbers include the debug scope of lower number, i.e. enabling 3 (warnings) also enables 1 (alerts) and 2 (errors)		Disabled
ENABLE_EM	Enables Error Management Module	ENABLE_SCHEDULER	Disabled
ENABLE_ESCALATION	Enables escalation of sub-system restart to SRST/PS-Only if the first restart attempt fails	ENABLE_RECOVERY	Disabled
ENABLE_RECOVERY	Enables WDT based restart of APU sub-system	ENABLE_EM, ENABLE_PM, ENABLE_SCHEDULER	Disabled
ENABLE_PM	Enables Power Management Module		Enabled
ENABLE_NODE_IDLING	Enables idling and reset of nodes before force shutdown of a sub-system		Disabled
ENABLE_SCHEDULER	Enables Scheduler module		Disabled

Table 10-10: PMU firmware Build Flags (Cont'd)

Flag	Description	Prerequisites	Default Setting
ENABLE_WDT	Enables CSU WDT based restart of system used by PMU	ENABLE_SCHEDULER, ENABLE_EM	Disabled
ENABLE_STL	Enables STL Module	None	Disabled
ENABLE_RTC_TEST	Enables RTC event handler test module	None	Disabled
ENABLE_SAFETY	Enables CRC calculation for IPI messages	None	Disabled
ENABLE_FPGA_LOAD	Enables FPGA bit stream loading feature	ENABLE PM	Enabled
ENABLE_SECURE	Enables security features	ENABLE PM	Enabled
IDLE_PERIPHERALS	Enables idling peripherals before PS-only or System reset	ENABLE PM	Disabled
ENABLE_POS	Enables Power Off Suspend feature	ENABLE PM	Disabled
EFUSE_ACCESS	Enables efuse access feature	ENABLE PM	Disabled
ENABLE_UNUSED_RPU_PWR_DWN	Powers down RPU(s) and slaves if they are not running after receiving PmlnitFinalize		Enabled

Error Management (EM) Module

Error Management Hardware

Zynq UltraScale+ MPSoC has a dedicated error handler to aggregate and handle fatal errors across the SoC. See the TRM/Arch Spec for more information.

All fatal errors routed to Error Manager can either set to be handled by HW (and trigger a SRST/PoR/PS error out) or trigger an interrupt to PMU.

Error Management in PMU firmware

Error management module initializes and handles the errors that are generated by hardware and provides an option for you to customize these handlers. In hardware, there are two error status registers which hold the type of error that occurred. Also any error can be enabled/disabled from interrupting the PMU Microblaze. For each of the errors, you can decide what action should be taken when the error occurs. The possible scenarios would be one or a combination of the following choices:

1. Asserting of **PS_ERROR_OUT** signal on the device
2. Generation of an interrupt to the PMU processor
3. Generation of a system reset (SRST)
4. Generation of a power-on-reset (POR)

PMU firmware provides APIs to register custom error handlers or assign a default (SRST/PoR/PS error out) action in response to an Error. When PMU firmware starts, it sets an error action as interrupt to PMU for some of the errors and PS error out for others as per the **ErrorTable []** structure defined in **xpfw_error_manager.c**.

Error Management API Calls

This section describes the APIs supported by Error Management module in PMU firmware.

Setting up Error Action

XPfw_EmSetAction() API is used to setup an action for the specified error.

Parameters

Table 10-11: **XPfw_EmSetAction**

Parameter	Description
ErrorId	ErrorId is ID for error as defined in Table 10-19
ActionId	ActionId is one of the actions defined in Table 10-20
ErrorHandler	ErrorHandler is the handler to be called in case where action is interrupt to PMU

Return

XST_SUCCESS: If error action is set properly.

XST_FAILURE: If error action fails.

Removing Error Action

XPfw_EmDisable() API is used to remove error action for the specified error.

Parameters

Table 10-12: **XPfw_EmDisable**

Parameter	Description
ErrorId	ErrorId is ID for error to remove error action

Return

XST_SUCCESS: If successful.

XST_FAILURE: If action fails.

Processing an error

XPfw_EmProcessError() API processes the errors that occur. If the respective error is registered with an error handler, then this function will call the respective handler to take appropriate action.

Parameters

Table 10-13: XPfw_EmProcessError

Parameter	Description
ErrorType	Type of error received (EM_ERR_TYPE_1: For errors in PMU GLOBAL_ERROR_STATUS_1 EM_ERR_TYPE_2: For errors in PMU GLOBAL_ERROR_STATUS_2)

Return

XST_SUCCESS: If successful.

XST_FAILURE: If action fails.

IPI Handling by EM module

Along with the PM module, error management module also uses IPI-0 channel for message exchange. APU and RPU 0/1 masters can communicate to this module using IPI. The **target_module_id** in IPI message differentiates which module needs to take an action based on the message received. The **target_module_id** for IPI handler registered for EM module is 0xE. Currently, PMU firmware supports only the messages shown in Table 10-14 using IPI.

Table 10-14: IPI Messages Supported by PMU firmware

S.No	IPI Message	IPI Message ID/API ID
1	Set error action	0x1
2	Remove error action	0x2
3	Send errors occurred	0x3

Set error action

When this IPI message is received from any target to PMU firmware, PMU firmware sets the error action for the error ID received in the message. If processing of the message is successful, it sends SUCCESS (0x0) response to the target. Otherwise FAILURE (0x1) response will be sent. The message format for the same is as below:

Table 10-15: Message Format for Error Action

Word	Description
0	<target_module_id, api_id>
1	Error ID. See Table 10-19 for the Error ID's supported.
2	Error Action. See Table 10-20 for the Error Actions supported.

Remove error action

When this IPI message is received from any target to PMU firmware, EM module IPI handler will remove the error action for the error ID received. And after processing the message, it will send SUCCESS/FAILURE response to the target respectively. The message format for the same is as below:

Table 10-16: Message Format for Removing Error Action

word	Description
0	<target_module_id, api_id>
1	Error ID. See Table 10-19 for the Error ID's supported.

Send errors occurred

PMU firmware saves the errors that occur in the system and sends to the target upon request. The message format is as below:

Table 10-17: Message Format for Sending Errors Occurred

Word	Description
0	<target_module_id, api_id>

Table 10-18 shows the response message sent by PMU firmware.

Table 10-18: Response Message by PMU firmware

Word	Description
0	<target_module_id, Success/Failure>
1	Error_1 (Bit description is as ERROR_STATUS_1 register in PMU Global registers. If a bit is set to 1, then it means the respective error as described in ERROR_STATUS_1 has occurred)
2	Error_2 (Bit description is as ERROR_STATUS_2 register in PMU Global registers. If a bit is set to 1, then it means the respective error as described in ERROR_STATUS_2 has occurred)
3	PMU RAM Correctable ECC Count

EM Error ID Table

Table 10-19: EM Error IDs Table

Error ID	Error Number	Error Description	Default Error Action
EM_ERR_ID_CSU_ROM	1	Errors logged by CSU bootROM (CBR)	PS Error Out
EM_ERR_ID_PMU_PB	2	Errors logged by PMU bootROM (PBR) in the pre-boot stage	PS Error Out
EM_ERR_ID_PMU_SERVICE	3	Errors logged by PBR in service mode	PS Error Out
EM_ERR_ID_PMU_FW	4	Errors logged by PMU firmware	PS Error Out
EM_ERR_ID_PMU_UC	5	Un-Correctable errors logged by PMU HW. This includes PMU ROM validation Error, PMU TMR Error, uncorrectable PMU RAM ECC Error, and PMU Local Register Address Error	PS Error Out
EM_ERR_ID_CSU	6	CSU HW related Errors	PS Error Out
EM_ERR_ID_PLL_LOCK	7	Errors set when a PLL loses lock (These need to be enabled only after the PLL locks-up)	PS Error Out
EM_ERR_ID_PL	8	PL Generic Errors passed to PS	PS Error Out
EM_ERR_ID_TO	9	All Time-out Errors [FPS_TO, LPS_TO]	PS Error Out
EM_ERR_ID_AUX3	10	Auxiliary Error 3	PS Error Out
EM_ERR_ID_AUX2	11	Auxiliary Error 2	PS Error Out
EM_ERR_ID_AUX1	12	Auxiliary Error 1	PS Error Out
EM_ERR_ID_AUX0	13	Auxiliary Error 0	PS Error Out
EM_ERR_ID_DFT	14	CSU System Watch-Dog Timer Error	System Reset
EM_ERR_ID_CLK_MON	15	Clock Monitor Error	PS Error Out
EM_ERR_ID_XMPU	16	XPMU Errors [LPS XMPU, FPS XMPU]	Interrupt to PMU
EM_ERR_ID_PWR_SUPPLY	17	Supply Detection Failure Errors	PS Error Out
EM_ERR_ID_FPD_SWDT	18	FPD System Watch-Dog Timer Error	'Interrupt to PMU' if ENABLE_RECOVERY flag is defined. Otherwise, 'System Reset'
EM_ERR_ID_LPD_SWDT	19	LPD System Watch-Dog Timer Error	System Reset
EM_ERR_ID_RPU_CCF	20	Asserted if any of the RPU CCF errors are generated	PS Error Out
EM_ERR_ID_RPU_LS	21	Asserted if any of the RPU CCF errors are generated	Interrupt to PMU
EM_ERR_ID_FPD_TEMP	22	FPD Temperature Shutdown Alert	PS Error Out
EM_ERR_ID_LPD_TEMP	23	LPD Temperature Shutdown Alert	PS Error Out

Table 10-19: EM Error IDs Table (Cont'd)

Error ID	Error Number	Error Description	Default Error Action
EM_ERR_ID_RPU1	24	RPU1 Error including both Correctable and Uncorrectable Errors	PS Error Out
EM_ERR_ID_RPU0	25	RPU0 Error including both Correctable and Uncorrectable Errors	PS Error Out
EM_ERR_ID_OCM_ECC	26	OCM Uncorrectable ECC Error	PS Error Out
EM_ERR_ID_DDR_ECC	27	DDR Uncorrectable ECC Error	PS Error Out

EM Error Action Table

Table 10-20: EM Error Action Table

Error Action	Error Action Number	Error Action Description
EM_ACTION_POR	1	Trigger a Power-On-Reset
EM_ACTION_SRST	2	Trigger a System Reset
EM_ACTION_CUSTOM	3	Call the custom handler registered as ErrorHandler parameter
EM_ACTION_PSERR	4	Trigger a PS-Error Out action

PMU firmware Signals PLL Lock Errors on PS_ERROR_OUT

When EM module is enabled, it is recommended to enable SCHEDULER also. During FSBL execution of `psu_init`, it is expected to get the PLL lock errors. To avoid these errors during EM module initialization, PMU firmware will not enable PLL Lock errors. It waits for `psu_init` completion by FSBL using a scheduler task. After FSBL completes execution of `psu_init`, PMU firmware will enable all PLL Lock errors.

In `xpfw_error_management.c`, you can see the following default behavior of the PMU firmware for PLL Lock Errors:

```
[EM_ERR_ID_PLL_LOCK] = { .Type = EM_ERR_TYPE_2, .RegMask =
PMU_GLOBAL_ERROR_STATUS_2_PLL_LOCK_MASK, .Action = EM_ACTION_NONE, .Handler =
NullHandler},
```

where, `PMU_GLOBAL_ERROR_STATUS_2_PLL_LOCK_MASK` is #defined with `0X00001F00` value, which means that all the PLL Lock Errors are enabled. Hence, if the design do not use any PLL/PLLs that are not locked, this triggers the `PS_ERROR_OUT` signal. It means that the `PMU_GLOBAL.ERROR_STATUS_2` register (bits [12:8]) signals that one or more PLLs are NOT locked and that triggers the `PS_ERROR_OUT` signal.

To analyze further and see if this is really an issue is to fully understand the status of the PLL in the design.

For example, if the design only uses `IO_PLL` and `DDR_PLL` and `PMU_GLOBAL.ERROR_STATUS_2` register signals 0x1600 value, it means that the `RPU_PLL`, `APU_PLL` and `Video_PLL` Lock errors have occurred. Looking at a few more registers, you can really understand the status of the PLLs.

PLL_STATUS

- PLL_STATUS (CRL_APB) = FF5E0040: 00000019
- PLL_STATUS (CRF_APB) = FD1A0044: 0000003A

Table 10-21: PLL_STATUS

PLL STATUS	ERROR_STATUS_2
IOPLL is locked and stable	Bit [8] is for IO_PLL = 0
RPLL is stable and NOT locked (which means bypassed)	Bit [9] is for RPU_PLL = 1
APPL is stable and NOT locked (which means bypassed)	Bit [10] is for APU_PLL = 1
DPLL is locked and stable	Bit [11] is for DDR_PLL = 0
VPLL is stable and NOT locked (which means bypassed)	Bit [12] is for Video_PLL = 1

Hence, if the design only uses `IO_PLL` and `DDR_PLL`, then it is not really an error to have `RPU_PLL`, `APU_PLL` and `Video_PLL` in NOT locked status.

Xilinx recommends you to customize the `PMU_GLOBAL.ERROR_STATUS_2.PLL_LOCK_MASK` to cover only the PLL of interest so that you can have a meaningful `PS_ERROR_OUT` signal.

Example:

```
#define PMU_GLOBAL_ERROR_STATUS_2_PLL_LOCK_MASK ((u32)0X00000900U) will only
signal on PS_ERROR_OUT IO_PLL and DDR_PLL errors.
```

Power Management (PM) Module

Zynq UltraScale+ MPSoC Power Management framework is based on an implementation of the Embedded Energy Management Interface (EEMI). This framework allows software components running across different processing units (PUs) on a chip or device to issue or respond to requests for power management.

The Power Management module is implemented within the PMU firmware as an event-driven module. Events processed by the Power Management module are called power management events. All power management events are triggered via interrupts.

When handling an interrupt the PMU firmware determines whether the associated event shall be processed by the Power Management module. Accordingly, if the PMU firmware

determines that an event is power management related and if the Power Management module is enabled, the PMU firmware triggers it to process the event.

For example, all the PS and PL interrupts can be routed to the PMU via the GIC Proxy. When the application processors (APU or RPU) are temporarily suspended, the PMU handles the GIC Proxy interrupt and wakes up the application processors to service the original interrupts. The PMU firmware does not actually service these interrupts, although you are free to customize the PMU firmware so that these interrupts are serviced by the PMU instead of by the application processors. For more information, see the 'Interrupts' chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11]..

When processing a power management event the Power Management controller may exploit the PMU ROM handlers for particular operations regarding the state control of hardware resources.

Warm restart and FPGA configuration manager are part of Power Management module. PMU firmware includes XilFPGA and XilSecure libraries to support the functionalities of PL FPGA configuration and to access secure features respectively. See [Chapter 11, Power Management Framework](#) for more information.

Note: Since the Power Management module uses base firmware APIs such as IPI manager/event manager, it is not possible to run standalone power management features without PMU firmware. See [PM Examples](#) wiki page for XilPM based design examples.

Scheduler

A scheduler is required by modules like STL in order to support periodic tasks like register coverage, scrubbing, etc. PMU firmware also uses scheduler for LPD WDT functionality. This will be explained in the following section. PMU MicroBlaze has 4 PITs (0-3) and Scheduler uses PIT1. The scheduler supports up to 10 tasks. Table shows the Scheduler's task list data structure with members.

Table 10-22: Scheduler data structure members

Member	Values/Range	Additional information
Task ID	0.. 9	0 - Highest priority
Interval	Task interval in Milliseconds	
OwnerId	0.. 9	Modules that owns this task
Status	Enabled/Disabled	
Callback	Function pointer	Default to NULL

Note: By default, Scheduler functionality is disabled. To enable the same, ENABLE_SCHEDULER build flag needs to be defined.

Safety Test Library

Safety Test Library (STL) is a collection of software safety mechanisms complementing hardware safety features for the detection of random hardware (HW) faults. PMU firmware has a placeholder for STL initialization during PMU firmware startup. This is enabled when ENABLE_STL build flag is defined. The software library and the safety documentation can be seen at the [Safety Lounge](#).

CSU/PMU Register Access

The following section discusses how to Read/Write the CSU and PMU global registers and provides a list of White and Black registers.

Register Write:

```
$ echo <address> <mask> <value> > /sys/firmware/zynqmp/config_reg
```

Register Read:

```
$ echo <address> > /sys/firmware/zynqmp/config_reg
```

```
$ cat /sys/firmware/zynqmp/config_reg
```

CSU and PMU global registers are categorized into two lists:

- By default, the White list registers can be accessed all the time. The following is a list of white registers.
 - CSU Module:
 - Csu_status
 - Csu_multi_boot
 - Csu_tamper_trig
 - Csu_ft_status
 - Jtag_chain_status
 - Idcode
 - Version
 - Csu_rom_digest(0:11)
 - Aes_status
 - Pcap_status

- PMU Global Module :
 - Global_control
 - Global_Gen_Storage0 - 6
 - Pers_Glob_Gen_Storage0-6
 - Req_Iso_Status
 - Req_SwRst_Status
 - Csu_Br_Error
 - Safety_Chk
- The Black list registers can accessed when a compile time flag is set.

Every other register in both the CSU Module and the PMU_GLOBAL Module that is not covered in the above white list will be a black register. RSA and RSA_CORE module registers are black registers.

The `#define` option (`SECURE_ACCESS_VAL`) provides access to the black list. To access black list registers, build the PMUFW with `SECURE_ACCESS_VAL` flag set.

Timers

Zynq UltraScale+ MPSoCs have two system watchdog timers, one each for full-power domain (FPD) and low-power domain (LPD). Each of these WDT provides error condition information to the error manager. EM module can be configured to set a specific error action when FPD or LPD WDT expires. This section describes the usage of these watchdog timers and the PMU firmware functionality when these watchdog timers expire.

FPD WDT

FPD WDT can be used to reset the APU or the FPD. PMU firmware error management module can configure the error action to be taken when the FPD WDT error occurs. PMU firmware implemented a recovery mechanism for FPD WDT error. This mechanism is disabled by default. The same can be enabled by defining **ENABLE_RECOVERY** build flag.

The EM module in PMU firmware sets FPD WDT error action as 'system reset' when recovery mechanism is not enabled. In this case, PMU firmware doesn't initialize and configure the FPD WDT. It is left for Linux driver to initialize and start the WDT if required. When WDT expires, system restart happens.

When **ENABLE_RECOVERY** flag is defined, PMU firmware sets FPD WDT error action as 'interrupt to PMU' and registers a handler to be called when this error occurs. In this case, when PMU firmware comes up, it initializes and starts the WDT. It also initializes and sets the timer mode of TTC to interval mode.

PMU firmware configures FPD WDT expiry time to 60 seconds. And if WDT error occurs, PMU firmware gets an interrupt and it calls the registered handler. PMU firmware has a restart tracker structure to track the restart phase and other information for a master. APU is the only master currently using this structure. Following are its members:

Table 10-23: Restart Tracker Structure Members

Member	Description
Master	Master whose restart cycle is to be tracked
RestartState	Track different phases in restart cycle
RestartScope	Restart scope upon FPD WDT error interrupt
WdtBaseAddress	Base address for WDT assigned to this master
WdtTimeout	Timeout value for WDT
ErrorId	Error Id corresponding to the WDT
WdtPtr	Pointer to WDT for this master
TtcDeviceId	TTC timer device ID
TtcPtr	Pointer to TTC for this master
TtcTimeout	Timeout to notify master for event
TtcResetId	Reset line ID for TTC

When WDT error occurs, WDT error handler is called and PMU firmware performs the following:

1. It checks if master is APU and error ID is FPD WDT. Then, it checks if restart state is in progress or not. If restart state is not in progress, then it changes the restart state to in progress and increments the restart count to track the number of times a master is restarted.
2. Later, it restarts the WDT so that the PMU firmware knows when the WDT error is not due to APU application.
3. Then, it idles APU by sending an IPI to ATF via timer interrupt **TTC3_0**.

Note: This is only true for Linux, and not for bare metal where there is no ATF.

4. If the first restart attempt fails, then PMU firmware escalates restart to either system-reset or PS-only reset if **ENABLE_ESCALATION** flag is defined. If **ENABLE_ESCALATION** is not defined, PMU firmware restarts the APU. Otherwise, PMU firmware performs the following:
 - a. First, PMU firmware checks if PL is configured or not.
 - b. If PL is configured, PMU firmware initiates PS-only restart. Otherwise, it initiates system-reset.

Note: Ensure that the WDT heartbeat application is running in Linux.

CSU WDT

The CSU WDT is configured to be used by PMU firmware that if PMU firmware application hangs for some reason, then the system would restart. This functionality is enabled only when **ENABLE_WDT** flag is defined.

EM modules sets CSU WDT error action as 'System Reset' Initialization of CSU WDT depends on bringing WDT out of reset which is performed by **psu_init** from FSBL. FSBL writes the status of **psu_init** completion to PMU Global general storage register 5, so that PMU firmware can check for its completion before initializing CSU WDT. When **ENABLE_WDT** flag is defined during PMU firmware initialization, it adds a task to scheduler to be triggered for every 100 milli-seconds until **psu_init** completion status is updated by FSBL. After **psu_init** is completed, this task will be removed from scheduler tasks list and PMU firmware initializes CSU WDT and configures it to 90 milli-seconds. It also starts a scheduler task to restart the WDT for every 50 milli-seconds. Whenever CSU WDT error occurs due to PMU firmware code hanging, this error is handled in hardware to trigger 'System Reset' and the system will restart.

Following are the dependencies to use this WDT functionality:

1. EM module needs to be enabled by defining **ENABLE_EM** flag
2. **ENABLE_WDT** flag needs to be defined to use CSU WDT
3. Scheduler module needs to be enabled by defining **ENABLE_SCHEDULER** to add a task to scheduler to check for **psu_init** completion and to restart WDT periodically.

Configuration Object

The configuration object is a binary data object used to allow updating data structures in the PMU firmware power management module at boot time. The configuration object must be copied into memory by a processing unit on the Zynq UltraScale+ MPSoC. The memory region containing the configuration object must be accessible by the PMU.

The PMU is triggered to load the configuration object via the following API call:

```
XPm_SetConfiguration(address);
```

The address argument represents the start address of the memory where the configuration object is located. The PMU determines the size of the configuration object based on its content.

Once the PMU loads the configuration object it updates its data structures which are used to manage the states of hardware resources (nodes). Partial configurations are not possible. If the configuration object does not provide information as defined in this document or provides partial information, the consistency of PMU firmware power management data

cannot be guaranteed. The creator of the configuration object must ensure the consistency of the information provided in the configuration object. The PMU does not change the state of nodes once the configuration object is loaded. The PMU also does not check whether the information about current states of nodes provided in the configuration object really matches the current state of the hardware. Current state is a state of a hardware resource at the moment of processing the configuration object by the PMU.

The configuration object specifies the following:

- List of masters available in the system
- All the slave nodes the master is currently using and current requirement of the master for the slave configuration
- All the slave nodes the master is allowed to use and default requirement of the master for the slave configuration
- For each power node, which masters are allowed to request/release/power down
- For each reset line, which masters are allowed to request the change of a reset line value
- Which shutdown mode the master is allowed to request and shutdown timeout value for the master
- Which masters are allowed to set configuration after the configuration is already set upon the system boot by the FSBL

PM Configuration Object Generation

PM Configuration Object is generated as follows:

1. Specify the custom PM framework Configuration using the PCW tool
2. PCW generates the HDF file
3. At build time, the HDF Parser parses the HDF file and insert the configuration object into the FSBL code

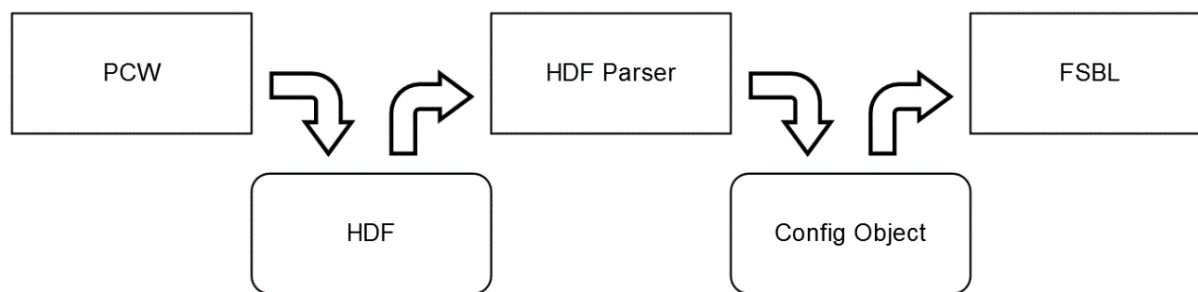


Figure 10-5: Configuration Object Generation

Initial Configuration at Boot

The configuration object shall be loaded prior to calling any EEMI API, except the following APIs:

- Get API version
- Set configuration
- Get Chip ID

Until the first configuration object is loaded the PM controller is configured to initially expect the EEMI API calls from the APU or RPU master, via IPI_APU or IPI_RPU_0 IPI channels, respectively. In other words, the first configuration object has to be loaded by APU or RPU.

After the first configuration object is loaded, the next loading of the configuration object can be triggered by a privileged master. Privileged masters are defined in the configuration object that was loaded the last.

Following are the steps at boot level:

1. FSBL sends the configuration object to PMU with the Set Configuration API
2. PMU parses the configuration object and configures
3. PMU powers off all the nodes which are unused after all the masters have completed the initialization

All other requests prior to the first Set Configuration API call will be rejected by PMU firmware.

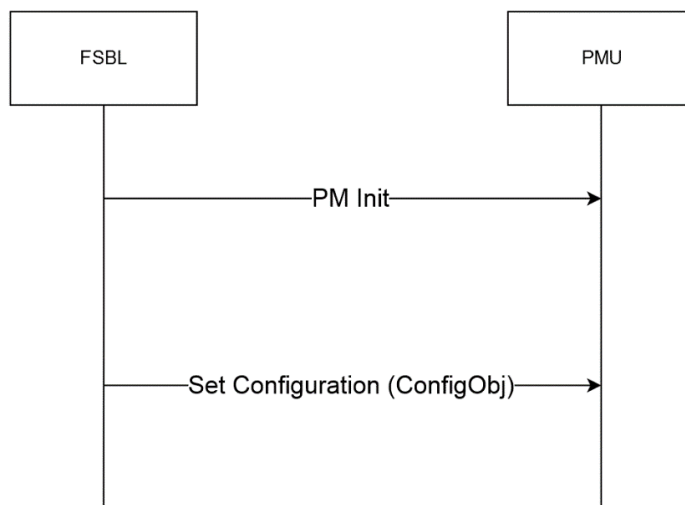


Figure 10-6: Initial Configuration at Boot

PMU firmware Loading Options

PMU firmware can be loaded by either FSBL or CSU BootROM (CBR). Both these flows are supported by Xilinx. Loading PMU firmware using FSBL has the following benefits:

- Possible quick boot time, when PMU firmware is loaded after bitstream.
- In use cases where you want two BIN files - stable and upgradable, PMU firmware can be part of the upgradable (by FSBL) image.



IMPORTANT:

CBR loads FSBL. If CBR also loads PMU firmware, it means that the secure headers for both FSBL and PMU firmware are decrypted with same Key-IV pair, which is a security vulnerability (security rule is: no two partitions should use the same Key-IV pair). This is addressed in FSBL, not in CBR. Hence, you should avoid CBR loading PMU firmware in secure (decryption) cases.

For DDR self-refresh over Warm restart, FSBL and PMU firmware must be loaded first (in any order) before all other images (e.g. bitstream).

For Power Off Suspend, PMU firmware must be loaded first (i.e. by CSU) before FSBL.

Loading PMU firmware in JTAG Boot Mode

PM operations depend on the configuration object loaded by FSBL from 2017.1 release onwards. Hence, In JTAG boot mode, it is mandatory to load PMU FW before loading FSBL. In device boot modes, loading of configuration object to PMU firmware by FSBL is handled both in CBR loading PMU firmware and FSBL loading PMU firmware options.

Use the following steps to boot in JTAG mode:

1. Disable security gates to view PMU Microblaze. PMU Microblaze is not visible in xsdb for Silicon v3.0 and above.
2. Load PMU FW and run
3. Load FSBL and run
4. Continue with U-Boot/Linux/user specific application

Following is a complete Tcl script:

```
#Disable Security gates to view PMU MB target
targets -set -filter {name =~ "PSU"}

#By default, JTAGsecurity gates are enabled
#This disables security gates for DAP, PLTAP and PMU.
mwr 0xffca0038 0x1ff
after 500
```

```
#Load and run PMU FW
targets -set -filter {name =~ "MicroBlaze PMU"}
dow xpfw.elf
con
after 500

#Reset A53, load and run FSBL
targets -set -filter {name =~ "Cortex-A53 #0"}
rst -processor
dow fsbl_a53.elf
con

#Give FSBL time to run
after 5000
stop

#Other SW...
dow u-boot.elf
dow bl31.elf
con

#Loading bitstream to PL
Targets -set -nocase -filter {name =~ "**PL*"}
fpga download.bit
```

Loading PMU firmware in NON-JTAG Boot Mode

When PMU firmware is loaded in a non-JTAG Boot mode on a 1.0 Silicon, an error message 'Error: Unhandled IPI received' may be logged by PMU firmware at startup, which can be safely ignored. This is due to the IPI0 ISR not being cleared by PMU ROM. This is fixed in 2.0 and later versions of Silicon.

Using FSBL to load PMU FW

1. Build PMU firmware application in SDK
2. Build an FSBL in the SDK for A53. (R5 can also be used)
3. Create a hello_world example for A53
4. Select **Xilinx** > **Create Boot Image**
5. Create a new bif file. Choose
 - a. Architecture: **ZynqMP**
 - b. You will see A53 fsbl and hello_world example by default in partitions. Also, we need PMU firmware.
 - c. Click on **Add**, then provide pmufw.elf path. Also select Partition type as **datafile**, Destination device as **PS**, and Destination CPU as **PMU**.
 - d. Click **OK**.

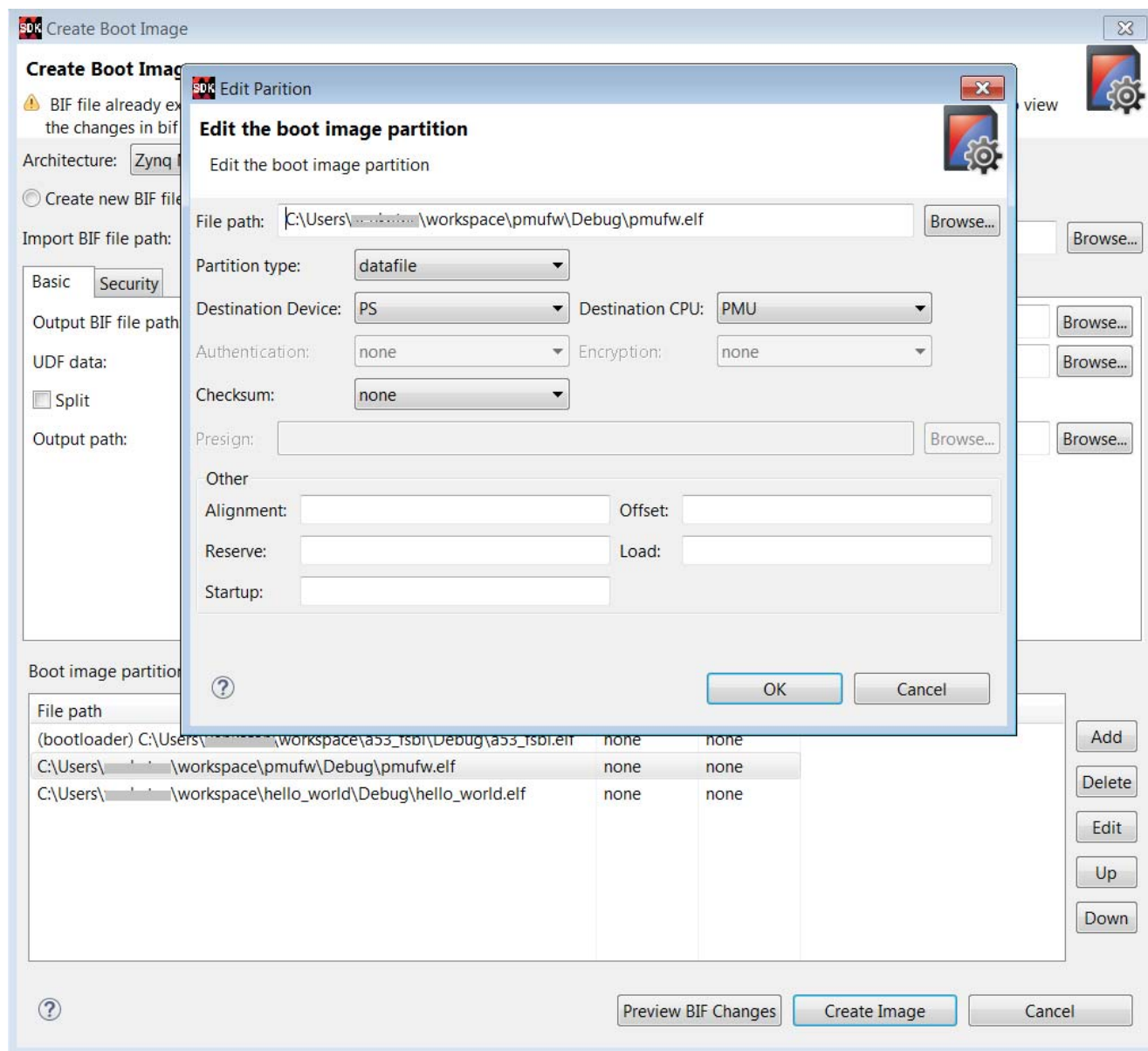


Figure 10-7: Boot Image Creation

6. After adding pmufw as partition. Click on **pmufw partition** and then, click **UP**.
7. Make sure to select the following partition order:
 - a. A53 FSBL
 - b. PMU firmware
 - c. Hello World application
8. Click on **Create Image**. You will see BOOT.bin created in a new **bootimage** folder in your example project.
9. View the .BIF file to confirm the partition order.

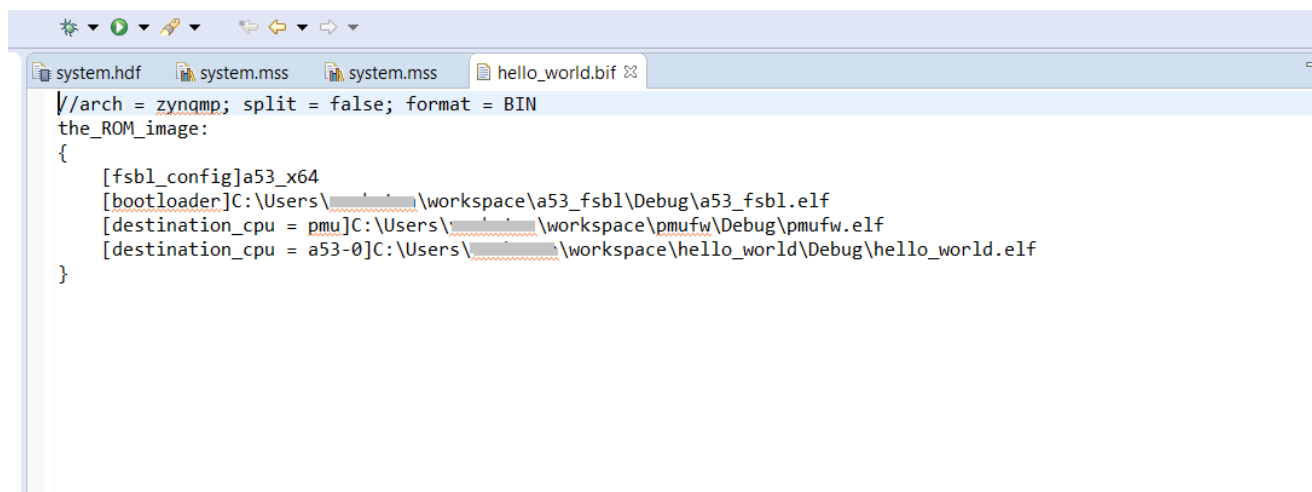


Figure 10-8: BIF file

10. Now copy this BOOT.bin into SD card.
11. Boot the ZCU102 board in SD boot mode. You can see the **fsbl** -> **pmufw** -> **hello_world** example prints in a sequence.

Using CBR to load PMU FW

When PMU firmware is loaded by CBR, it is executed prior to FSBL. So the MIOs, Clocks and other initializations are not done at this point. Consequently, the PMU firmware banner and other prints may not be seen prior to FSBL. Post FSBL execution, the PMU firmware prints can be seen as usual.

To make CBR load PMU firmware:

1. Change the BOOT.bin boot partitions.
2. Perform the steps listed in [Loading PMU firmware in NON-JTAG Boot Mode](#).
3. Create a new bif file. Choose the following:
 - a. Architecture: **ZynqMP**
 - b. You will see A53 fsbl and hello_world example by default in partitions. Also, we need pmufw.
 - c. Click **Add** and then provide the pmufw.elf path. Select the Partition type as **pmu** (loaded by bootrom).
 - d. Click **OK**.

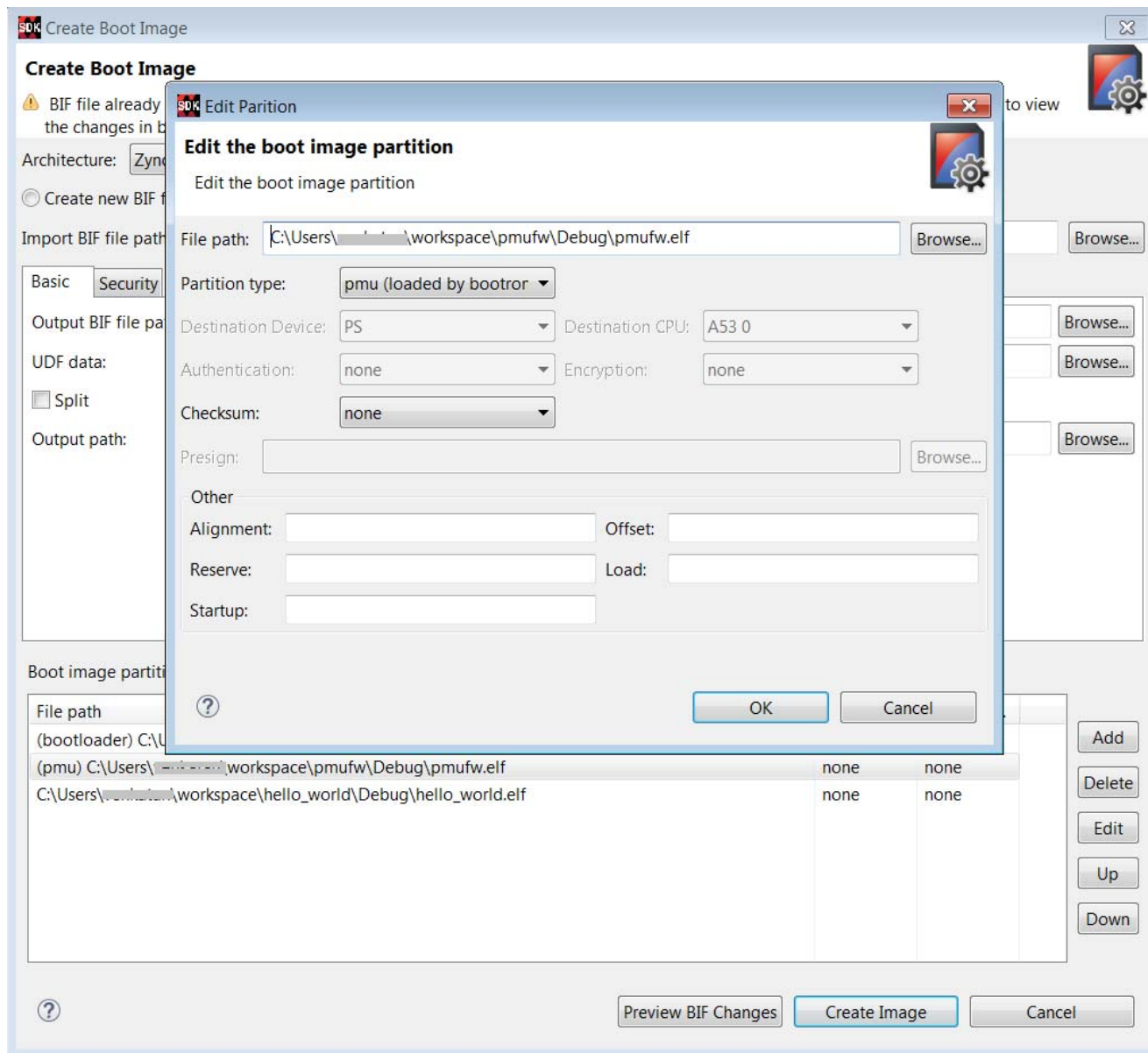


Figure 10-9: Creating Boot Image Partition

- e. Click on Create Image. You will see BOOT.bin created in a new folder named **bootimage** in your example project.
- f. You can also view .BIF to confirm the partition order.

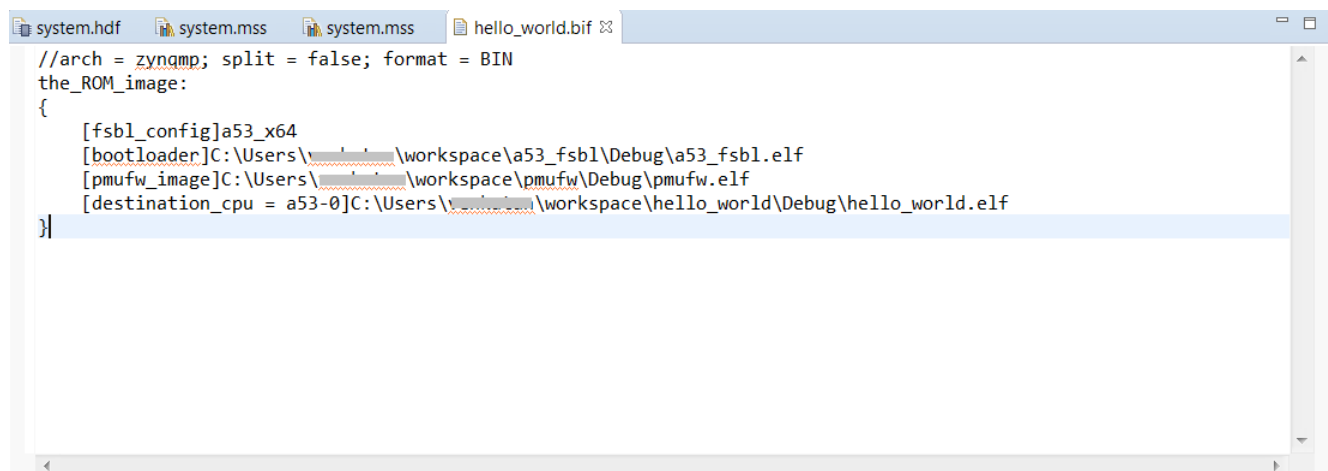


Figure 10-10: Viewing BIF File

- g. Now copy this BOOT.bin into SD card.
- h. Boot the ZCU102 board in SD boot mode. You can see the **pmufw** -> **fsbl** -> **hello_world** example prints in a sequence.

PMU firmware Usage

This section describes the usage of PMU firmware with examples.

Enable/Disable Modules

This section describes how to enable/disable PMU firmware build flags both in SDK and PetaLinux.

In SDK

1. PM Module build flag are defined in <pmu firmware application>/src/xpfw_config.h file. You can make changes to this file to enable or disable any build flags by modifying PMU firmware code include options section.
2. Use the following steps to enable all the other build flags:
 - a. Right click on the PMU firmware project and select **C/C++ Build Settings** option. The Properties window appears.
 - b. Select **Settings** from **C/C++ Build** options. In **Tool Settings** tab, select **Symbols** from **Microblaze gcc compiler** option.
 - c. Click **Add** under **Defined symbols (-D)**.
 - d. Enter flag name to define and click **OK**.

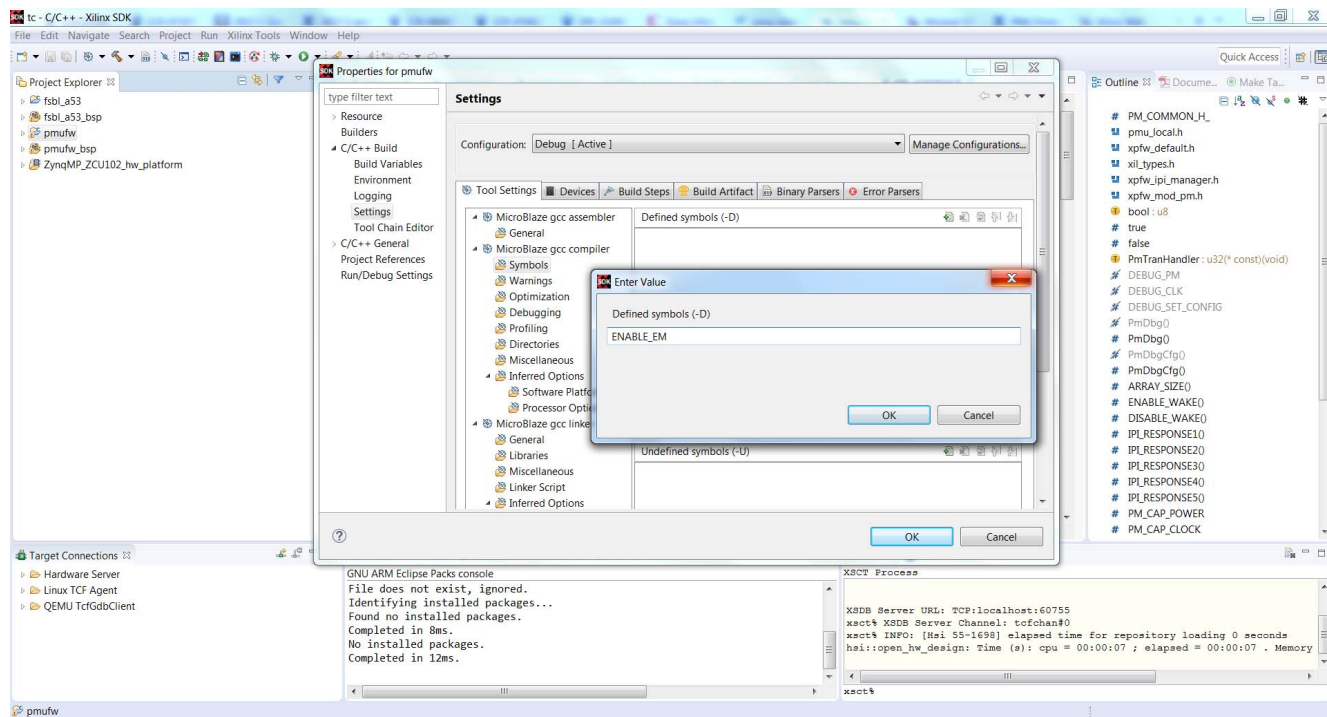


Figure 10-11: Enable Build Flags in SDK

- e. Again, click **OK** on properties window and the application will start building with the new compile flags defined.

In PetaLinux

1. Create a PetaLinux project
2. Open `<plnx-project-root>/project-spec/meta-user/recipes-bsp/pmu/pmu-firmware_%.bbappend` file and add the following line:

```
YAML_COMPILER_FLAGS_append = -DENABLE_EM
```

The above line enables EM module. To enable any flag, it should be prefixed with '-D'

3. After any change to the YAML compiler flags, force a clean state before rebuilding the application.

Custom Module Usage

Each set of user defined routines performing a specific functionality should be designed to be a module in PMU firmware. These files must be self-contained. However, these files can use declarations from `xpfw_core.h`. Each module can register with the following interfaces. If any of the handler is not needed by the module, it can be skipped from being registered.

1. Config Handler: Called during initialization
2. Event Handler: Called when a registered event is triggered
3. IPI Handler: Called when an IPI message arrives with the registered IPI ID

Creating a custom module

To create a custom module, add the following code to PMU firmware:

```
/* IPI Handler */
static void CustomIpiHandler(const XPfw_Module_t *ModPtr, u32 IpiNum, u32 SrcMask,
const u32* Payload, u8 Len)
{
    /**
     * Code to handle the IPI message received
     */
}

/* CfgInit Handler */
static void CustomCfgInit(const XPfw_Module_t *ModPtr, const u32 *CfgData,
u32 Len)
{
    /**
     * Code to configure the module, register for events or add scheduler tasks
     */
}

/* Event Handler */
static void CustomEventHandler(const XPfw_Module_t *ModPtr, u32 EventId)
{
    /**
     * Code to handle the events received
     */
}

/*
 * Create a Mod and assign the Handlers. We will call this function
 * from XPfw_UserStartup()
 */
void ModCustomInit(void)
{
    const XPfw_Module_t *CustomModPtr = XPfw_CoreCreateMod();

    (void) XPfw_CoreSetCfgHandler(CustomModPtr, CustomCfgInit);
    (void) XPfw_CoreSetEventHandler(CustomModPtr, CustomEventHandler);
    (void) XPfw_CoreSetIpiHandler(CustomModPtr, CustomIpiHandler, (u16)IPI_ID);
}
```

Registering for an Event

All interrupts that come into PMU are exposed to user as Events with specific EVENTIDs defined in xpfw_events.h. Any module can register for an event (usually in CfgHandler) and the module's EventHandler will be called when an event is triggered.

To register for an RTC Event:

```
Status = XPfw_CoreRegisterEvent (ModPtr,XPFW_EV_RTC_SECONDS);
```

Example of an EventHandler

```
void RtcEventHandler(const XPfw_Module_t *ModPtr, u32 EventId)
{
    xil_printf("MOD%d:EVENTID: %d\r\n", ModPtr->ModId, EventId);
    if(XPFW_EV_RTC_SECONDS == EventId){
        /* Ack the Int in RTC Module */
        Xil_Out32(RTC_RTC_INT_STATUS,1U);
        xil_printf("RTC: %d \r\n", Xil_In32(RTC_CURRENT_TIME));
    }
}
```

Error Management Usage

This sections describes the usage of the EM module to configure the error action to be taken for the errors that comes to PMU firmware (the errors generated in the system which are mapped to PMU MB).

Example for Error Management (Custom Handler)

For this example, OCM uncorrectable error (EM_ERR_ID_OCM_ECC) is considered. A custom handler is registered for this error in PMU firmware and the handler in this case just prints out the error message. In a more realistic case, the corrupted memory may be reloaded, but this example is just limited to clearing the error and printing a message.

Adding the Error Handler for OCM Uncorrectable ECC in PMU firmware:

```
+++ b/lib/sw_apps/zynqmp_pmufw/src/xpfw_mod_em.c
@@ -140,6 +140,14 @@ void FpdSwdtHandler(u8 ErrorId)
    XPfw_RecoveryHandler(ErrorId);
}

/* OCM Uncorrectable Error Handler */
+static void OcmErrHandler(u8 ErrorId)
+{
+    XPfw_Printf(DEBUG_DETAILED, "EM: OCM ECC error detected\n");
+    /* Clear the Error Status in OCM registers */
+    XPfw_Write32(0xFF960004, 0x80);
+}
+
/* CfgInit Handler */
static void EmCfgInit(const XPfw_Module_t *ModPtr, const u32 *CfgData,
    u32 Len)
@@ -162,6 +170,8 @@ static void EmCfgInit(const XPfw_Module_t *ModPtr, const u32
    *CfgData,
    }
}

+    XPfw_EmSetAction(EM_ERR_ID_OCM_ECC, EM_ACTION_CUSTOM, OcmErrHandler);
+
```

```
if (XPfw_RecoveryInit() == XST_SUCCESS) {
    /* This is to enable FPD WDT and enable recovery mechanism when
```

Inject OCM Uncorrectable ECC error using debugger (xsdb):

```

;# Enable ECC UE interrupt in OCM_IEN
mwr -force 0xFF96000C [expr 1<<7]

;# Write to Fault Injection Data 0 Register OCM_FI_D0
;# Errors will be injected in the bits which are set, here its bit0, bit1
mwr -force 0xFF96004C 3

;# Enable ECC and Fault Injection
mwr -force 0xFF960014 1
;
;# Clear the Count Register : OCM_FI_CNTR
mwr -force 0xFF960074 0
;# Now write data to OCM for the fault to be injected
;# Since OCM does a RMW for 32-bit transactions, it should trigger error here
mwr -force 0xFFFE0000 0x1234

;# Read back to trigger error again
mrd -force 0xFFFE0000

```

Example for Error Management (PoR as a response to Error)

Some error may be too fatal and the system recovery from those errors may not be feasible without doing a Reset of entire system. In such cases PoR or SRST can be used as actions. In this example we use PoR reset as a response to the OCM ECC double-bit error.

Here is the code that adds the PoR as action:

```

@@ -162,6 +162,8 @@ static void EmCfgInit(const XPfw_Module_t *ModPtr, const u32
 *CfgData,
    }
}

+    XPfw_EmSetAction(EM_ERR_ID_OCM_ECC, EM_ACTION_POR, NULL);
+

    if (XPfw_RecoveryInit() == XST_SUCCESS) {
        /* This is to enable FPD WDT and enable recovery mechanism when

```

The Tcl script to inject OCM ECC error is same as the one for above example. Once you trigger the error, a PoR occurs and you may see that all processors are in reset state similar to how they would be in a fresh power-on state. PMU RAM also gets cleared off during a PoR. Hence, PMU firmware needs to be reloaded.

Example for Error Management (PS Error out as a response to Error)

If you need to communicate outside of system when any error occurs, **PS ERROR OUT** response can be set for that respective error. So, when that error occurs, error will be propagated outside and **PS_ERROUT** signal LED will glow. In this example we use **PS ERROR OUT** as a response to the OCM ECC double-bit error.

Following is the code that adds the PS ERROR OUT as action:

```
@@ -162,6 +162,8 @@ static void EmCfgInit(const XPfw_Module_t *ModPtr, const u32
*CfgData,
    }

+    XPfw_EmSetAction(EM_ERR_ID_OCM_ECC, EM_ACTION_PSERR, NULL);
+

    if (XPfw_RecoveryInit() == XST_SUCCESS) {
        /* This is to enable FPD WDT and enable recovery mechanism when
```

The Tcl script to inject OCM ECC error is same as the one for above example. Once you trigger the error, a **PS_ERROUT** LED will glow on board.

IPI Messaging Usage

This section describes the usage of IPI messaging from PMU firmware to RPU0 and RPU0 to PMU firmware. PMU firmware, while initializing IPI driver, also enables IPI interrupt from the IPI channel assigned master.

From PMU firmware to RPU0

See [Zynq UltraScale Plus MPSoC - IPI Messaging Example](#) for more information.

Note: You need to enable EM module in PMU firmware to run this example.

From RPU0 to PMU firmware

See [Zynq UltraScale Plus MPSoC - IPI Messaging Example](#) for IPI messaging example from RPU to PMU.



IMPORTANT: Since the example in the wiki page shows how to trigger IPI from PMU to RPU0 and vice versa, to trigger an IPI to/from APU or RPU1, you need to change the destination CPU mask to the intended master.

Adding a Task to Scheduler

Tasks are functions which take void arguments and return void. Currently PMU firmware has no way to check that the task returns in a pre-determined time, so this needs to be ensured by the task design. Let us consider a task which prints out a message:

```
void TaskPrintMsg(void)
{
    xil_printf("Task has been triggered\r\n");
}
```

If we want to schedule the above task to occur every 500ms, the following code can be used. The TaskModPtr is a pointer for module which is scheduling the task.

```
Status = XPfw_CoreScheduleTask(TaskModPtr, 500U, TaskPrintMsg);
if(XST_SUCCESS == Status) {
    xil_printf("Task has been added successfully !\r\n");
}
else {
    xil_printf(Error: Failed to add Task !\r\n");
}
```

Reading FPD Locked Status from RPU

Register 0xFFD600F0 is a local register to PMU firmware, in which bit 31 displays whether FPD is locked or not locked. (If bit 31 is set to 1, then FPD is locked. It remains isolated until POR is asserted). You can verify the FPD locked status by reading this register through PMU firmware. This can be achieved by an MMIO read call to PMU firmware. Use the following steps to read FPD locked status from R5:

1. Create an empty application for R5 processor. Enable xilpm library in BSP settings.
2. Create a new.c file in the project and add the following code:

```
#include "xipipsu.h"
#include "pm_api_sys.h"
#define IPI_DEVICE_ID XPAR_XIPIPSU_0_DEVICE_ID
#define IPI_PMU_PM_INT_MASK XPAR_XIIPS_TARGET_PSU_PMU_0_CH0_MASK

#define MMIO_READ_API_ID20U
#define FPD_LOCK_STATUS_REG 0xFFD600F0

int main(void)
{
    XIpiPsu IpiInstance;
    XIpiPsu_Config *Config;
    s32 Status;
    u32 Value;

    /* Initialize IPI peripheral */
    Config = XIpiPsu_LookupConfig(IPI_DEVICE_ID);
    if (Config == NULL) {
        xil_printf("Config Null\r\n");
        goto END;
    }
```

```

    }

    Status = XIpiPsu_CfgInitialize(&IpiInstance, Config,
        Config->BaseAddress);
    if (0x0U != Status) {
        xil_printf("Config init failed\r\n");
        goto END;
    }

    /* Initialize the XilPM library */
    Status = XPm_InitXilpm(&IpiInstance);
    if (0x0U != Status) {
        xil_printf("XilPM init failed\r\n");
        goto END;
    }
    /* Read using XPm_MmioRead() */
    Status = XPm_MmioRead(FPD_LOCK_STATUS_REG, &Value);
    if (0x0U != Status)
    {
        xil_printf("XilPM MMIO Read failed\r\n");
        goto END;
    }
    xil_printf("Value read from 0x%x: 0x%x\r\n", FPD_LOCK_STATUS_REG, Value);

END:
    xil_printf("Exit from main\r\n");
}

```

Note: This application must be run after FSBL is successfully executed. This application cannot run successfully, if FSBL fails to send configuration object to PMU firmware.

Debugging PMU firmware

PMU FW is built with -Os and LTO optimizations by default. You must disable the optimization to debug application.

Steps to Disable Optimization for PMU firmware

Since removing optimization results increases the code size and can result in not being able to build PMU FW, disable some of the features that you do not use by removing build flags from project settings. Some of the features are disabled in `xpfw_config.h` file of PMU FW.

To disable/modify Optimizations, remove/modify the highlighted options (-Os -flto -ffat -lto -objects) as shown in [Figure 10-12](#) in the Miscellaneous section of pmu_fw properties window.

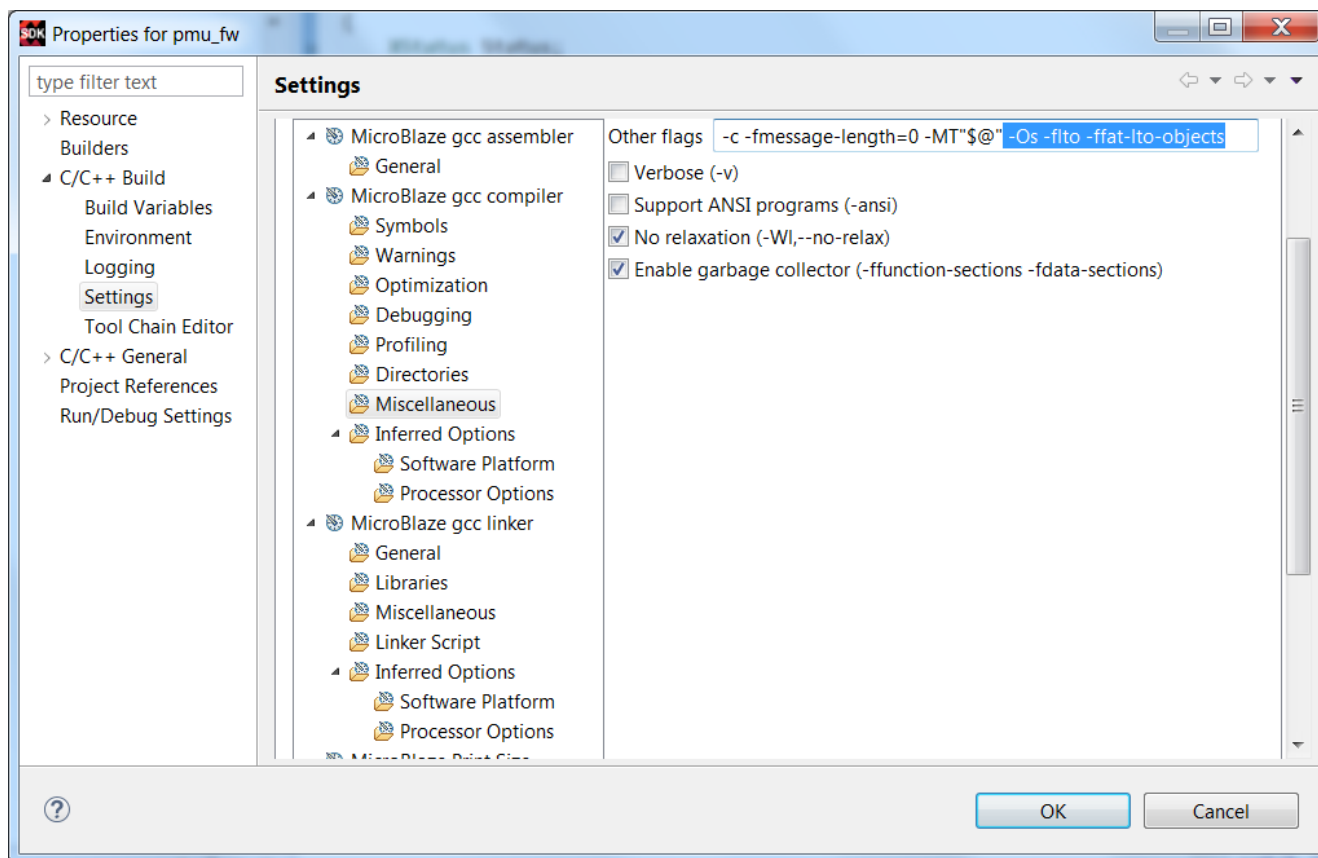


Figure 10-12: Modifying Options



IMPORTANT: The compiler Optimization screen in SDK cannot sense the optimization selected using Miscellaneous flags screen (above). The Optimization screen still shows `-O0` as shown in Figure 10-13, which can be misleading to you as a user.

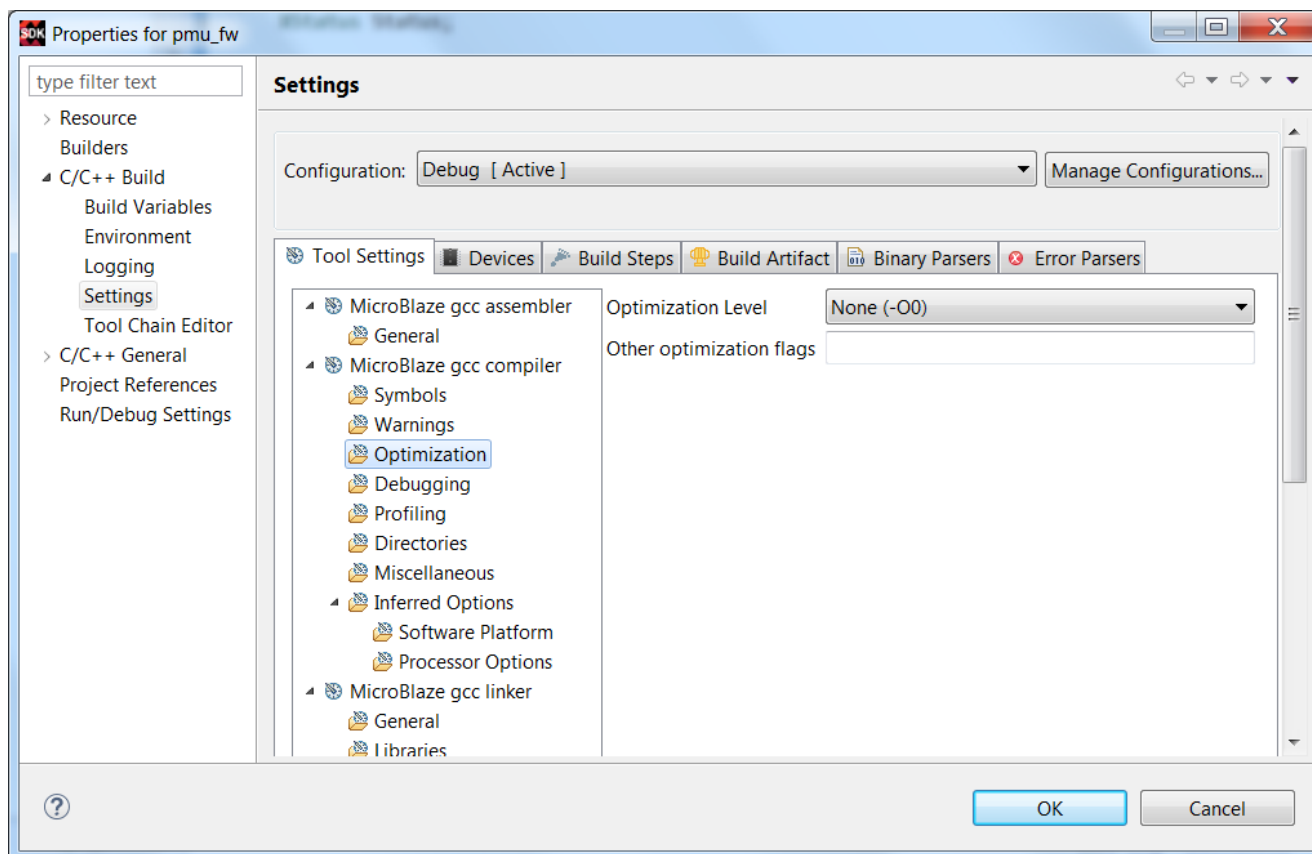


Figure 10-13: Optimization Level Setting

Steps to Debug PMU firmware Application Using SDK

1. Right click on the application, click **Debug As** and then, click on **Debug Configurations**.
2. Right click on **System Debugger** and click **New**. You get a New Configuration. Click on **Debug**.

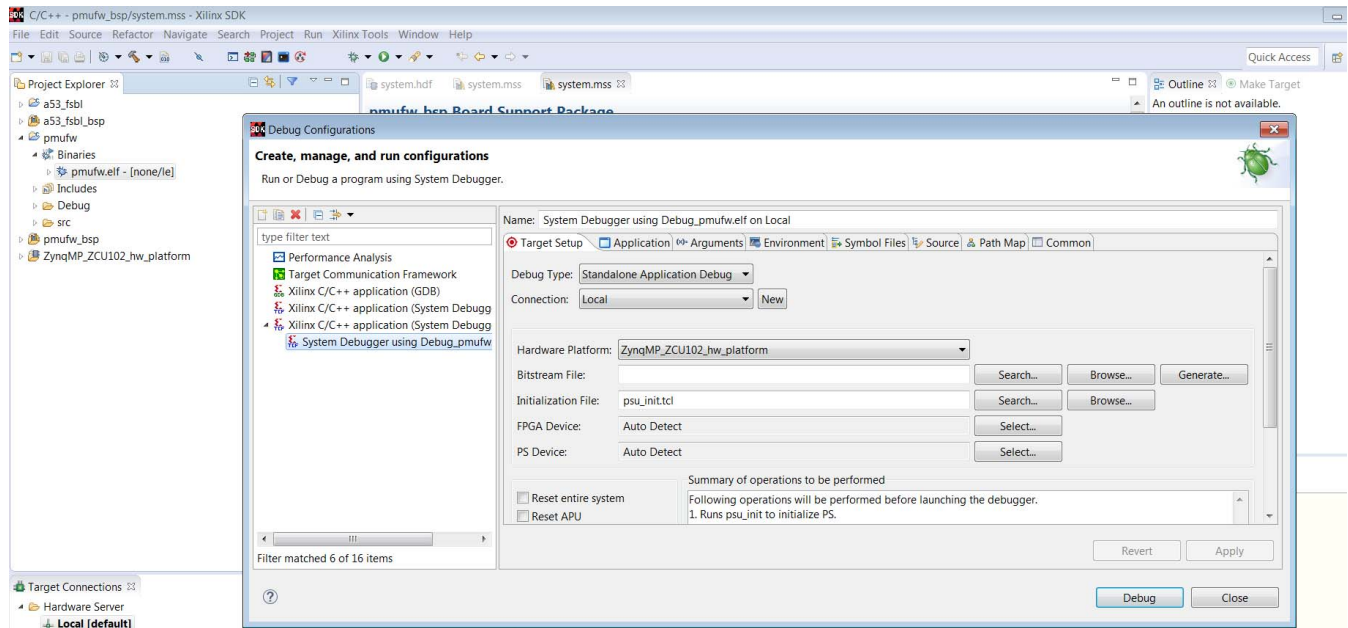


Figure 10-14: Debugging PMUFirmware Using SDK

3. Choose the Debug perspective. Click **OK**. Click **Yes** to confirm the Perspective Switch.

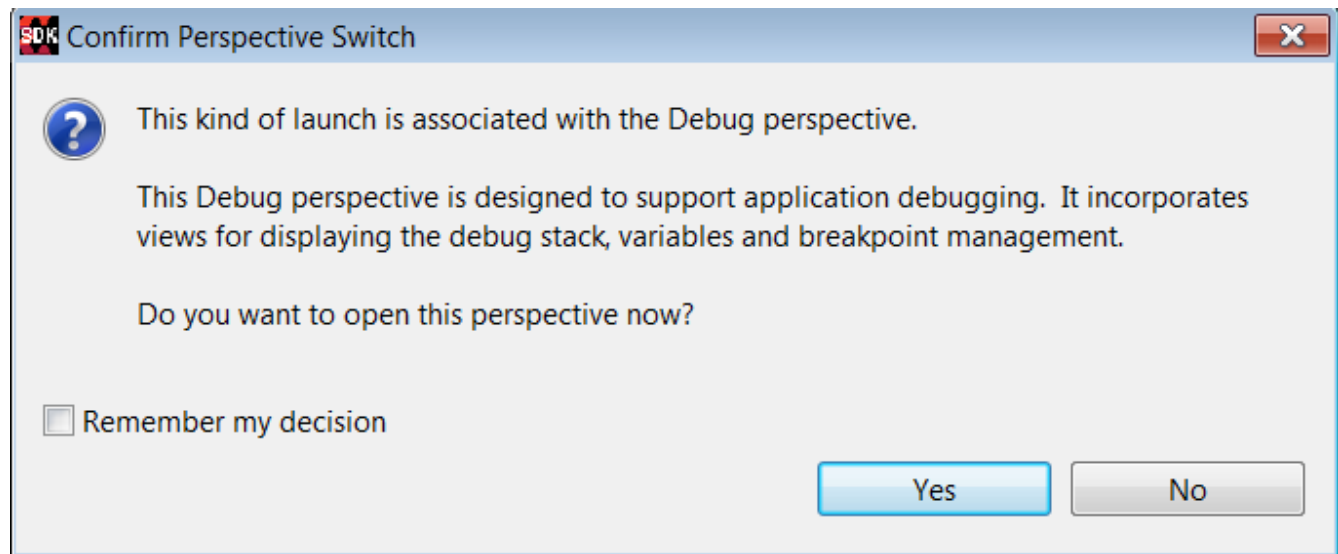


Figure 10-15: Perspective Switch Confirmation Dialog

4. You will now see the debug perspective and PMU firmware will run.

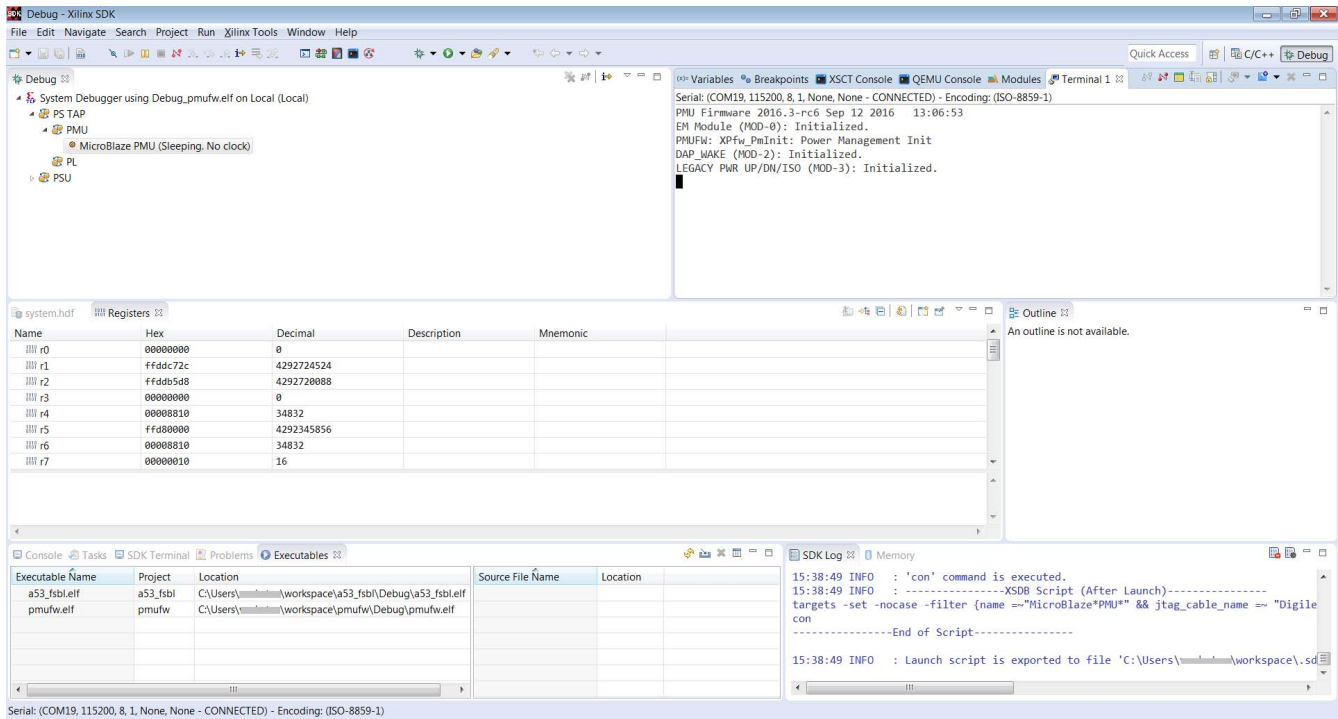


Figure 10-16: PMUFirmware Debug Perspective

- Place the breakpoints to control the flow and rerun for debugging.

Diverting PMU firmware Logs

Use the following steps to divert PMU firmware prints to different UART.

- After creating PMU firmware application, right click on **PMU firmware BSP** and select **Board Support Package Settings**.
- Board Support Package Settings window appears. Select **Standalone** under **Overview**.
- For **stdin** and **stdout**, list down the values and select any UART to which the prints needs to be diverted. By default, **psu_uart_0** is selected.

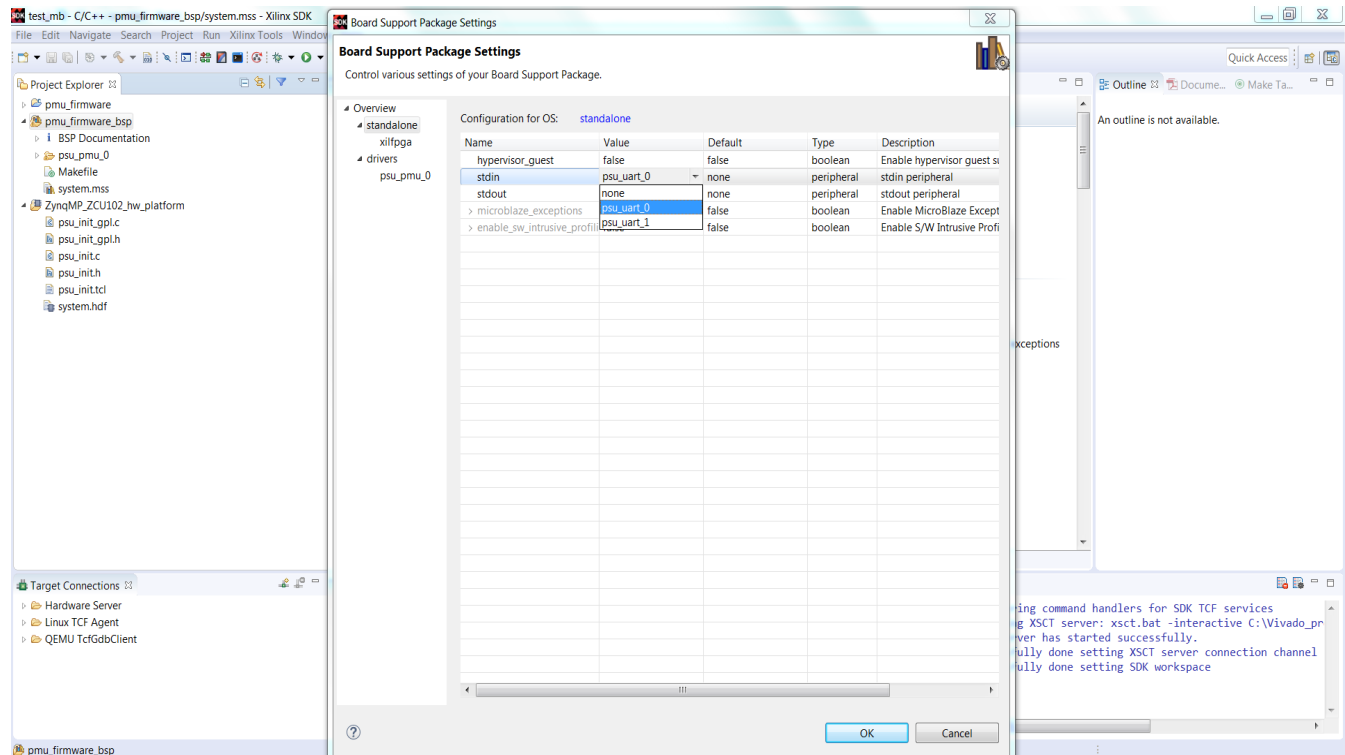


Figure 10-17:

- Click **OK**. Now PMU firmware BSP and application will get re-generated.
- In Tera Term, select **UART interface 0** or **UART interface 1** based on your setting in BSP and run the application on target.

Note: If hardware design holds a single UART, it must be shared between the targets.

Considerations for PMU Logging

The following console ports are available for PMU logging:

- UART0
- UART1

If PMU logging is enabled, the UART that is used by the PMU for logging must never be disabled, clock gated or powered down. Otherwise, the PMU may hang. That will eventually cause the entire system to hang. PMU logging also increases power consumption during Deep Sleep since the UART's PLL and clock will not be disabled.



IMPORTANT: The default device tree that comes with PetaLinux specifies all the UART devices available on the board. It means that the Linux kernel will power down any UART if it is not being used. Hence, if a UART is being used for PMU logging, it should be removed from the Linux device tree.

Any standalone application running on the RPU should not attempt to change the power state of the UART that is used for PMU logging.

By default, PMU logging is disabled. Make sure that the UART being used for PMU logging is not affected by the APU or RPU.

PMU firmware Memory Layout and Footprint

This section contains the approximate details of PMU firmware Memory Layout and also the Memory Footprint with various modules enabled.

In PMU RAM, some part is reserved for PBR leaving around 125.7 KB for PMU firmware. [Figure 10-18](#) shows the memory layout of PMU RAM.

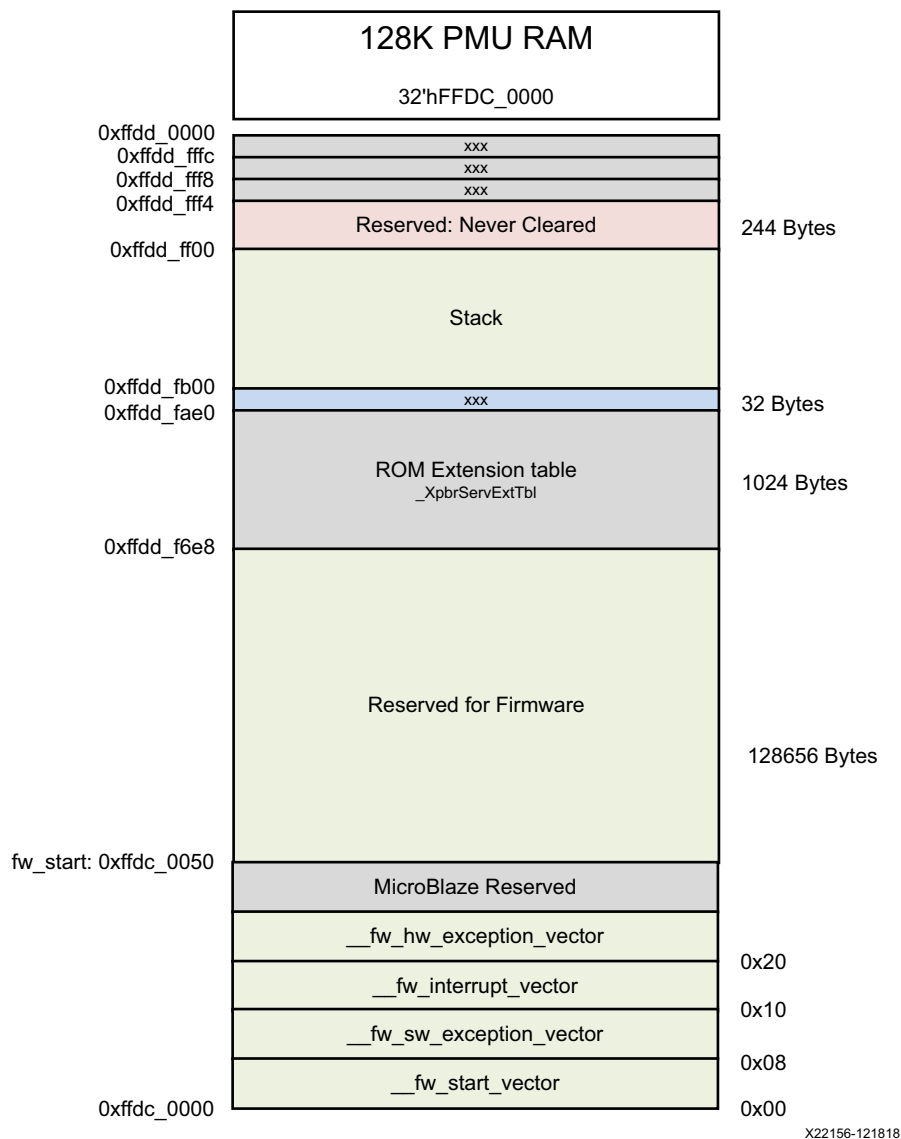


Figure 10-18: PMU firmware Memory Layout

In PMU firmware, only PM module is enabled by default along with Base Firmware and all the other modules are disabled. See [PMU firmware Build Flags](#) section to know about the default setting of a module.

Note: All the metrics are with compilation optimized for size -Os. This optimization setting is enabled by default in SDK. To disable the same, follow the steps mentioned in [Steps to Disable Optimization for PMU firmware](#) section.

Table 10-24: PMU firmware Metrics

S.No	Feature/Component	Size Occupied (KB)	Free Space (KB)	Additional Notes	Remarks
1	PMU firmware without detailed debug prints enabled	112.5	15.5	This is with base PMU firmware and PM module	
2	PMU firmware with detailed debug prints enabled	123	5	Detailed debug prints are enabled when XPFW_DEBUG_DETAILED and DEBUG_MODE flags are defined	This estimation is with combination of (1) and (2)
3	PMU firmware with Error Management Module enabled	115.4	12.6	Error Management module is enabled when ENABLE_EM and ENABLE_SCHEDULER flags are defined	This estimation is with combination of (1) and (3)
4	PMU firmware with Restart functionality enabled	117.5	10.5	Restart functionality is enabled when ENABLE_RECOVERY, ENABLE_ESCALATION and CHECK_HEALTHY_BOOT flags are defined along with EMABLE_EM and ENABLE_SCHEDULER flags	This estimation is with combination of (1) and (4)

Dependencies



RECOMMENDED: It is recommended to have all the software components (FSBL, PMU firmware, ATF, U-Boot and Linux) of the same release tag (e.g.: 2017.3).

Power Management Framework

Introduction

The Zynq® UltraScale+™ MPSoC is the industry's first heterogeneous multiprocessor SoC (MPSoC) that combines multiple user programmable processors, FPGA, and advanced power management capabilities.

Modern power efficient designs requires usage of complex system architectures with several hardware options to reduce power consumption as well as usage of a specialized CPU to handle all power management requests coming from multiple masters to power on, power off resources and handle power state transitions. The challenge is to provide an intelligent software framework that complies to industry standard (IEEE P2415) and is able to handle all requests coming from multiple CPUs running different operating systems. Xilinx has created the Power Management Framework (PMF) to support a flexible power management control through the platform management unit (PMU).

This Power Management Framework handles several use case scenarios. For example, Linux provides basic power management capabilities such as idle, hotplug, suspend, resume, and wakeup. The kernel relies on the underlining APIs to execute power management decisions, but most RTOSes do not have this capability. Therefore they rely on user implementation, which is made easier with use of the Power Management Framework.

Industrial applications such as embedded vision, Advanced Driver Assistance, surveillance, portable medical, and Internet of Things (IoT) are ramping up their demand for high-performance heterogeneous SoCs, but they have a tight power budget. Some of the applications are battery operated, and battery life is a concern. Some others such as cloud and data center have demanding cooling and energy cost, not including their need to reduce environmental cost. All of these applications benefit from a flexible power management solution.

Key Features

The following are the key features of the Power Management Framework.

- Provides centralized power state information through use of a Power Management Unit (PMU)
- Supports Embedded Energy Management Interface (EEMI) APIs (IEEE P2415)
- Manages power state of all devices
- Provides support for Linux power management, including:
 - Linux device tree power management
 - ATF/PSCI power management support
 - Idle
 - Hotplug
 - Suspend
 - Resume
 - Wakeup process management
- Provides direct control of the following power management features with more than 24 APIs:
 - Processor unit suspend and wake up management
 - Memories and peripherals management

Power Management Software Architecture

The Zynq UltraScale+ MPSoC architecture employs a dedicated programmable unit (PMU) that controls the power-up, power-down, monitor, and wakeup mechanisms of all system resources. The customer benefits from a system that is better equipped on handling power management administration for a multiprocessor heterogeneous system. However it is inherently more complex. The goal of the Power Management Framework is to abstract this complexity, exposing only the APIs you need to be aware of to meet your power budget goal.

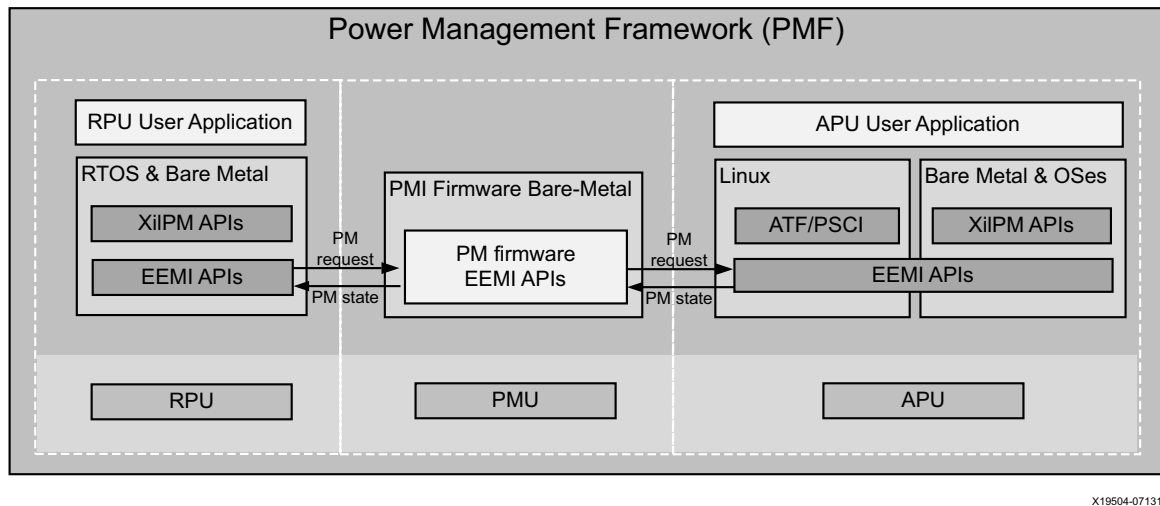


Figure 11-1: Power Management Framework

The intention of the EEMI is to provide a common API that allows all software components to power manage cores and peripherals. At a high level, EEMI allows you to specify a high-level power management goal such as suspending a complex processor cluster or just a single core. The underlying implementation is then free to autonomously implement an optimal power-saving approach.

The Linux device tree provides a common description format for each device and its power characteristics. Linux also provides basic power management capabilities such as idle, hotplug, suspend, resume, and wakeup. The kernel relies on the underlining APIs to execute power management decisions.

Users can also create their own power management applications using the XilPM library, which provides access to more than 24 APIs.

Zynq UltraScale+ MPSoC Power Management Overview

The Zynq UltraScale+ MPSoC power management framework is a set of power management options, based upon an implementation of the Embedded Energy Management Interface (EEMI). The power management framework allows software components running across different processing units (PUs) on a chip or device to issue or respond to requests for power management.

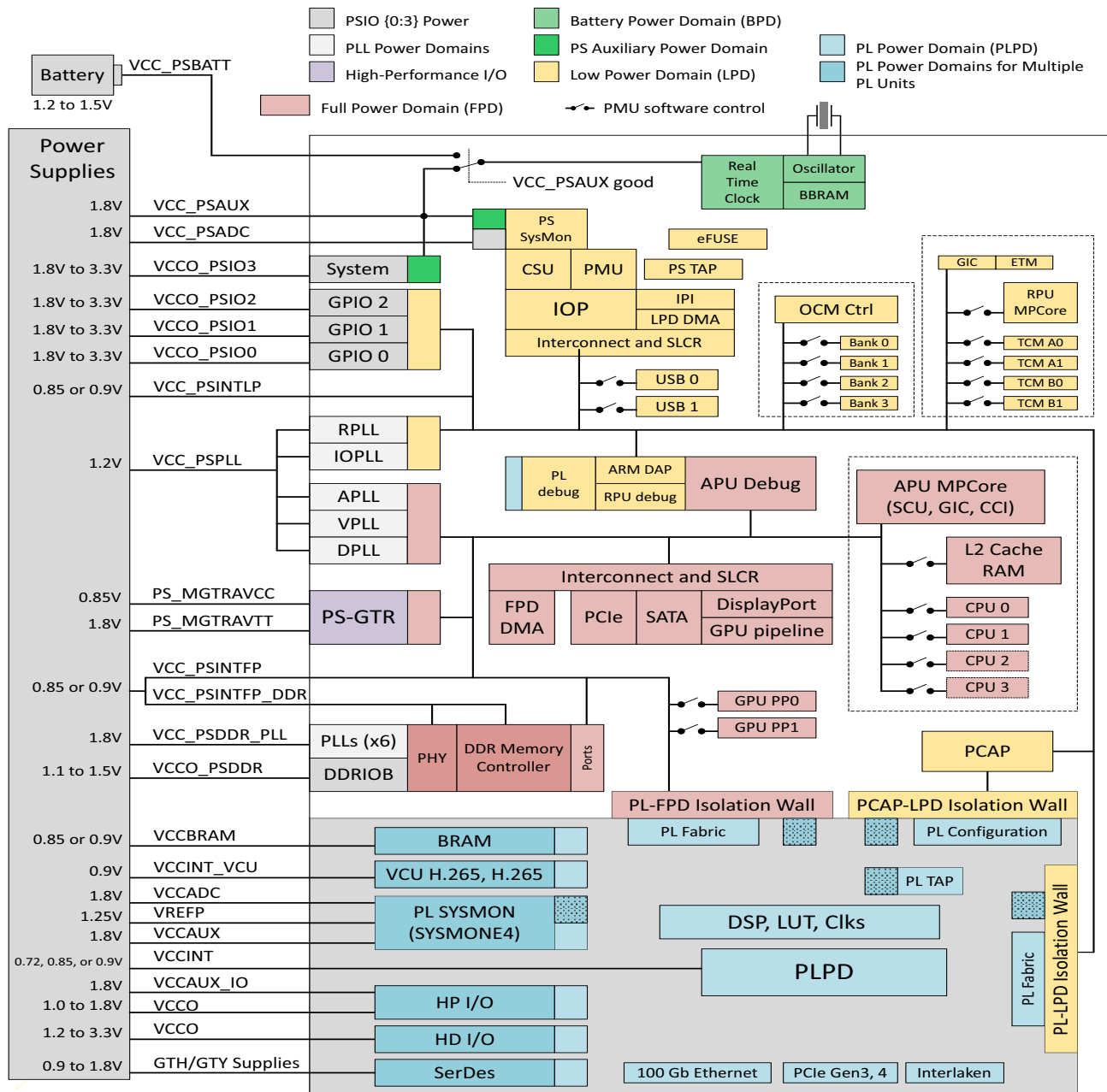
Zynq UltraScale+ MPSoC Power Management Hardware Architecture

The Zynq UltraScale+ MPSoC device is divided into four major power domains:

- Full power domain (FPD): Contains the Arm Cortex™-A53 application processor unit (APU) as well as a number of peripherals typically used by the APU.
- Low power domain (LPD): Contains the Arm Cortex-R5F real-time processor unit (RPU), the platform management unit (PMU), and the configuration security unit (CSU), as well as the remaining on-chip peripherals.
- Programmable logic (PL) power domain: Contains the PL.
- Battery-power domain: Contains the real-time clock (RTC) as well as battery-backed RAM (BBRAM).

Other power domains listed in the following figure are not actively managed by the power framework. Designs that want to take advantage of the Power Management switching of power domains must keep some power rails discrete. This allows individual rails to be powered off with the power domain switching logic. For more details, see the “PCB Power Distribution and Migration in UltraScale+ FPGAs” in the *UltraScale Architecture PCB Design User Guide* (UG583)[[Ref 21](#)].

The following diagram illustrates the Zynq UltraScale+ MPSoC device power domains and islands.



X19927-101717

Figure 11-2: Zynq UltraScale+ MPSoC Power Domain and Islands

Because of the heterogeneous multi-core architecture of the Zynq UltraScale+ MPSoC device, no single processor can make autonomous decisions about power states of individual components or subsystems.

Instead, a collaborative approach is taken, where a power management API delegates all power management control to the platform management unit (PMU). It is the key component coordinating the power management requests received from the other processing units (PUs), such as the APU or the RPU, and the coordination and execution from other processing units through the power management API.



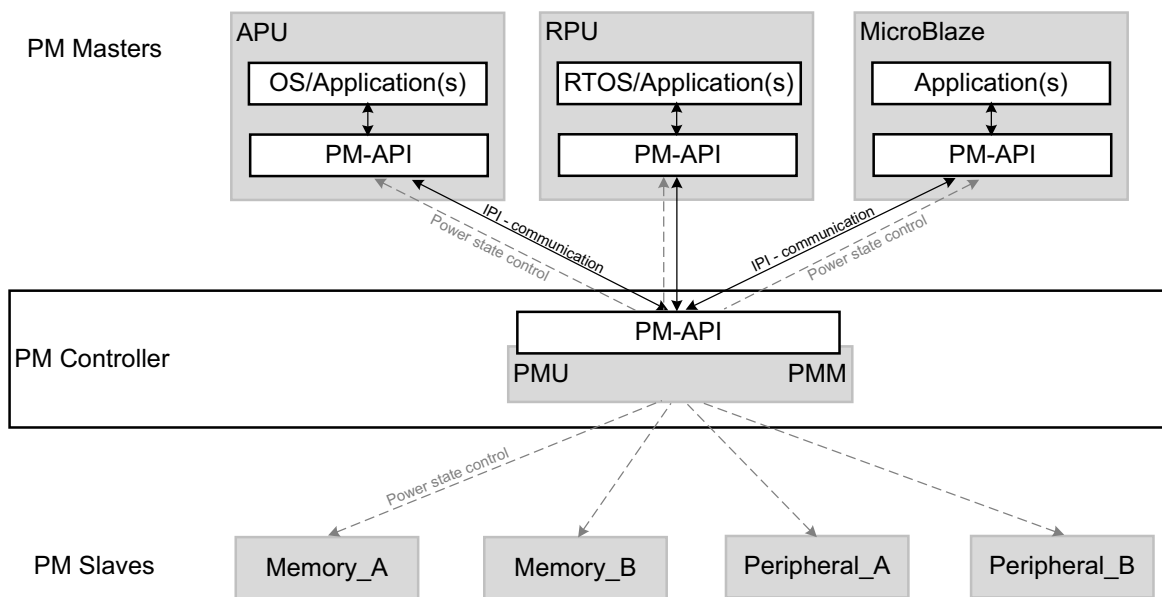
IMPORTANT: *In the EEMI implementation for Zynq UltraScale+ MPSoC, the platform management unit (PMU) serves as the power management controller for the different processor units (PUs), such as the APU and the RPU. These APU/RPU act as a power management (PM) master node and make power management requests. Based on those requests, the PMU controls the power states of all PM slave nodes as well as the PM masters. Unless otherwise specified, the terms "PMU" and "power management controller" are interchangeable.*

The Zynq UltraScale+ MPSoC device also supports inter-processor interrupts (IPIs), which are used as the basis for power-management related communication between the different processors. See this [link](#) to the "Interrupts" chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11] for more detail on this topic.

Zynq UltraScale+ MPSoC Power Management Software Architecture

To enable multiple processing units to cooperate in terms of power management, the software framework for the Zynq UltraScale+ MPSoC device provides an implementation of the power management API for managing heterogeneous multiprocessing systems.

The following figure illustrates the API-based power management software architecture.



X19503-071317

Figure 11-3: API-Based Power Management Software Architecture

Power Management Framework Overview

The Zynq UltraScale+ MPSoC device power management framework (PMF) is based on an implementation of EEMI, see the *Embedded Energy Management API Specification* (UG1200) [Ref 17]. It includes APIs that consist of functions available to the processor units (PUs) to send messages to the power management controller, as well as callback functions in for the power management controller to send messages to the PUs. The APIs can be grouped into the following functional categories:

- Suspending and waking up PUs
- Slave device power management, such as memories and peripherals
- Miscellaneous
- Direct-access

API Calls and Responses

Power Management Communication using IPIs

In the Zynq UltraScale+ MPSoC device, the power management communication layer is implemented using inter-processor interrupts (IPIs), provided by the IPI block. See this [link](#) to the “Interrupts” chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11] for more details on IPIs.

Each PU has a dedicated IPI channel with the power management controller, consisting of an interrupt and a payload buffer. The buffer passes the API ID and up to five arguments. The IPI interrupt to the target triggers the processing of the API, as follows:

- When calling an API function, a PU generates an IPI to the power management unit (PMU), prompting the execution of necessary power management action.
- The PMU performs each PM action atomically, meaning that the action cannot be interrupted.
- To support PM callbacks, which are used for notifications from the PMU to a PU, each PU implements handling of these callback IPIs.

Acknowledge Mechanism

The Zynq UltraScale+ MPSoC power management framework (PMF) supports blocking and non-blocking acknowledges. In most API calls that offer an acknowledge argument, the caller can choose one of the following three acknowledge options:

- REQUEST_ACK_N0: No acknowledge requested
- REQUEST_ACK_BLOCKING: Blocking acknowledge requested
- REQUEST_ACK_NON_BLOCKING: Non-blocking acknowledge using callback requested

Multiple power management API calls are serialized because each processor unit (PU) uses a single IPI channel for the API calls. After one request is sent to the power management controller, the next one can be issued only after the power management controller has completed servicing the first one. Therefore, no matter which acknowledge mechanism is used, the caller can be blocked when issuing subsequent requests.

No Acknowledge

If no acknowledge is requested (REQUEST_ACK_N0), the power management controller processes the request without returning an acknowledge to the caller, otherwise an acknowledgment is sent.

Blocking Acknowledge

After initiating a PM request with the `(REQUEST_ACK_BLOCKING)` specified, a caller remains blocked as long as the power management controller does not provide the acknowledgment.

The platform management unit (PMU) writes the acknowledge values into the response portion of the IPI buffer before it clears the IPI interrupt. The caller reads the acknowledge values from the IPI buffer after the IPI observation register shows that the interrupt is cleared, which is when PMU has completed servicing the issued IPI. The IPI for the PU is disabled until the PMU is ready to handle the next request.

Non-Blocking Acknowledge

After initiating a PM request with the `(REQUEST_ACK_NON_BLOCKING)` specified, a caller does not wait for the platform management unit (PMU) to process that request. Moreover, the caller is free to perform some other activities while waiting for the acknowledge from the PMU.

After the PMU completes servicing the request, it writes the acknowledge values into the IPI buffer. Next, the PMU triggers the IPI to the caller PU to interrupt its activities, and to inform it about the sent acknowledge.

Non-blocking acknowledges are implemented using a callback function that is implemented by the calling PU, see `XPm_NotifyCb` Callback.

For more information about `XPm_NotifyCb`, see [Appendix K, XilPM Library v2.5](#).

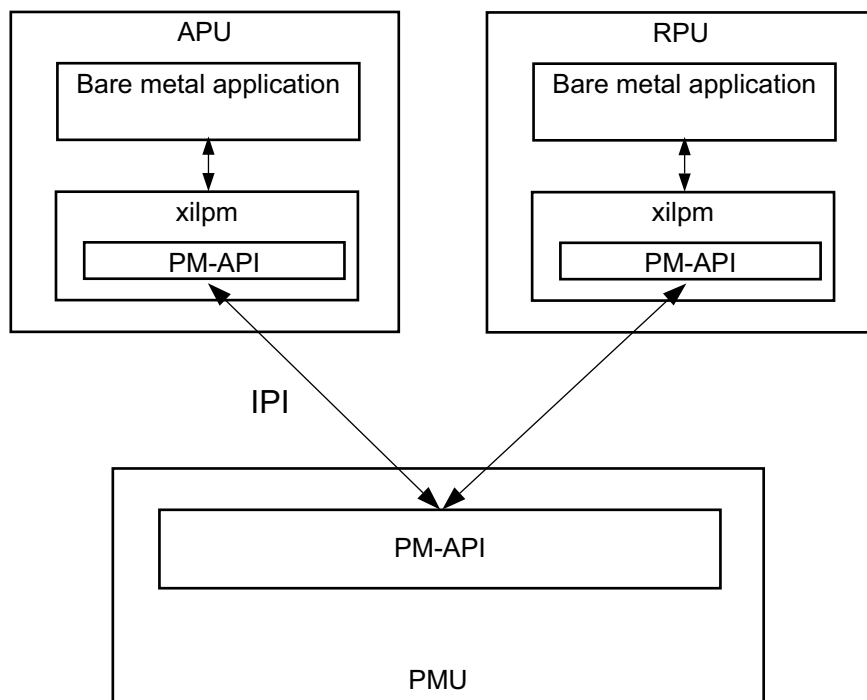
Power Management Framework Layers

There are different API layers in the power management framework (PMF) implementation for Zynq UltraScale+ MPSoC devices, which are, as follows:

- **Xilpm:** This is a library layer used for standalone applications in the different processing units, such as the APU and RPU.
- **ATF:** The Arm Trusted Firmware (ATF) contains its own implementation of the client-side PM framework. It is currently used by Linux operating systems.
- **PMU firmware:** The power management unit firmware (PMUFW) runs on the power management unit (PMU) and implements of the power management API.

For more details, see this [link](#) in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

The following figure shows the interaction between the APU, the RPU, and the PMF APIs.



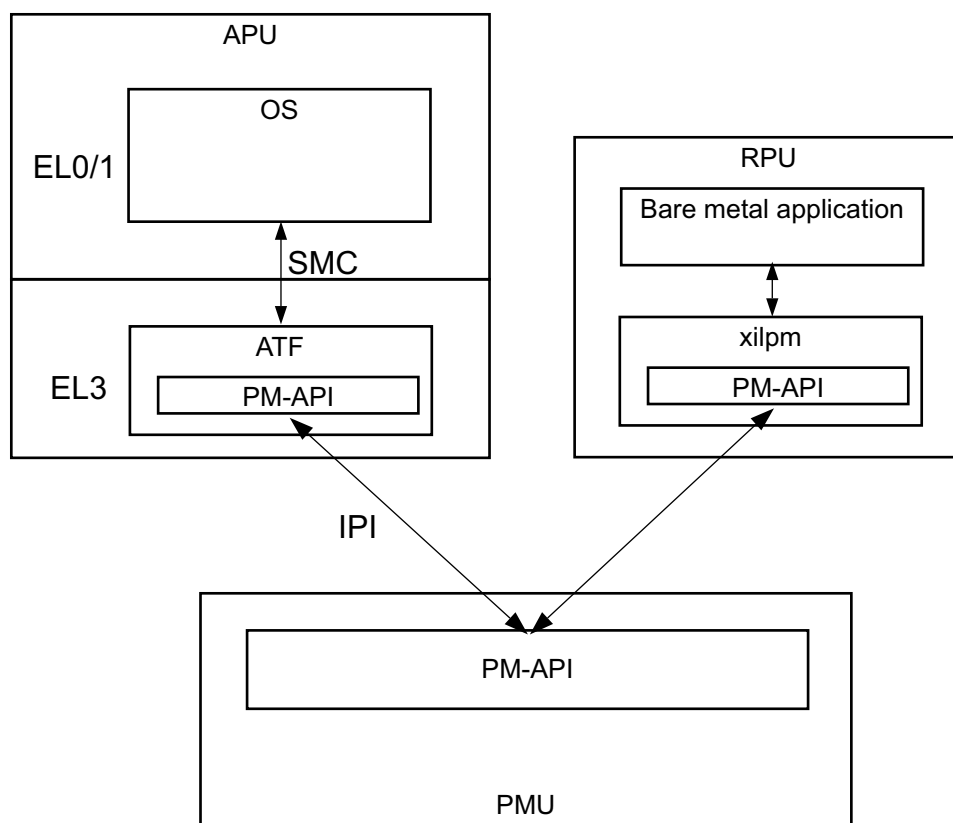
X19094-071317

Figure 11-4: API Layers Used with Bare-Metal Applications Only

If the APU is running a complete software stack with an operating system, the `Xilpm` library is not used. Instead, the ATF running on EL3 implements the client-side power management API, and provides a secure monitor call (SMC)-based interface to the upper layers.

Figure 11-5 illustrates this behavior. See the *Armv8 manuals* [Ref 46] for more details on the Armv8 architecture and its different execution modes.

The following figure illustrates the PMF layers that are involved when running a full software stack on the APU.



X19093-071317

Figure 11-5: PM Framework Layers Involved When Running a Full Software Stack on the APU

Typical Power Management API Call Flow

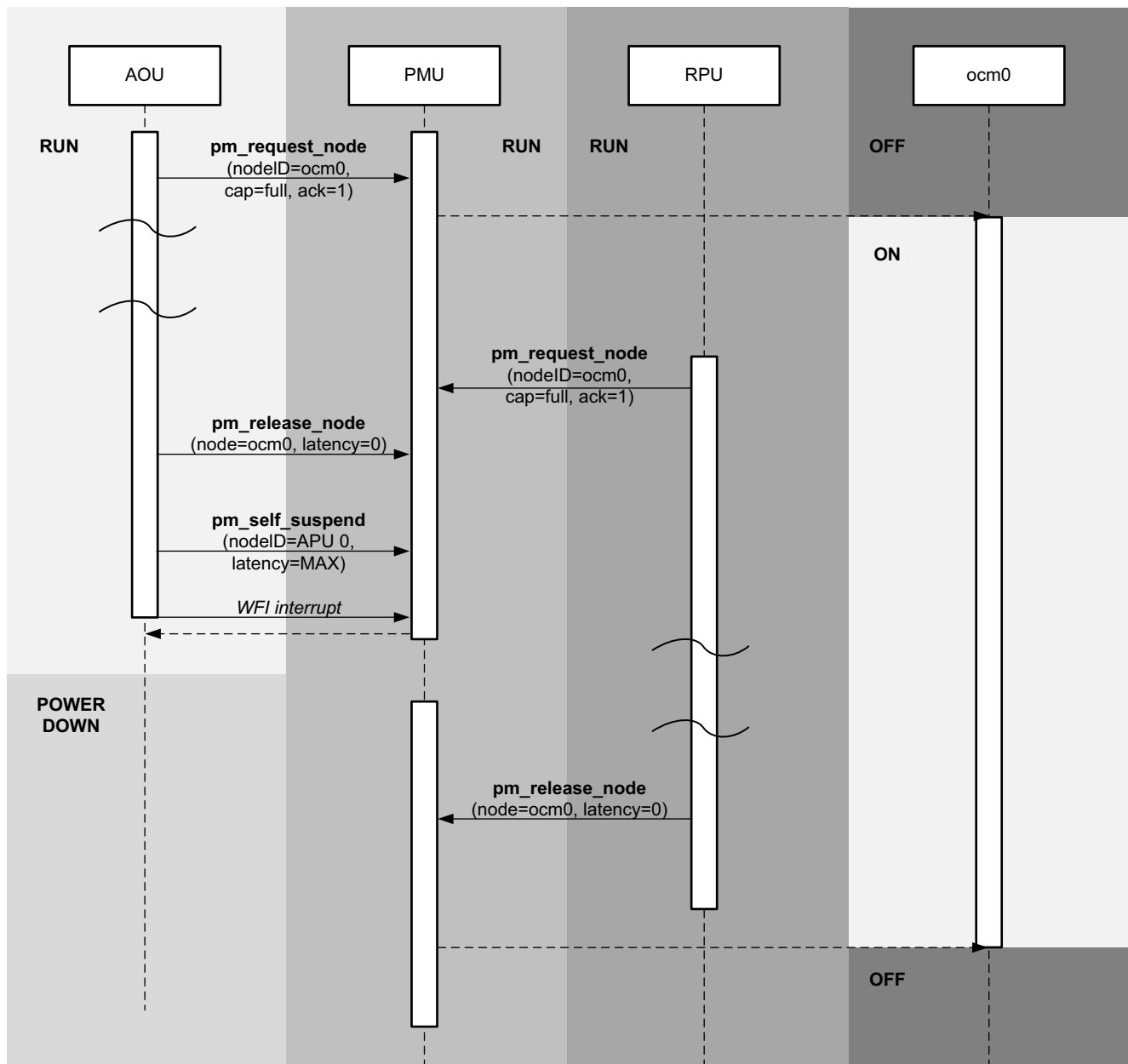
Any entity involved in power management is referred to as a *node*. The following sections describe how the power management framework (PMF) works with slave nodes allocated to the APU and the RPU.

Generally, the APU or the RPU inform the power management controller about their usage of a slave node, by requesting for it. They then inform the power management controller about the capability requirement needed from the slave node. At this point, the power management controller powers up the slave node so that it can be initialized by the APU or the RPU.

Requesting and Releasing Slave Nodes

When a PU requires a slave node, either peripheral or memory, it must request that slave node using the power management API. After the slave node has performed its function and is no longer required, it may be released, allowing the slave node to be powered off.

The following figure shows the call flow for a use-case in which the APU and the RPU are sharing an OCM memory bank, ocm0.



X20022-110217

Figure 11-6: PM Framework Call Sequence for APU and RPU Sharing an OCM Memory Bank

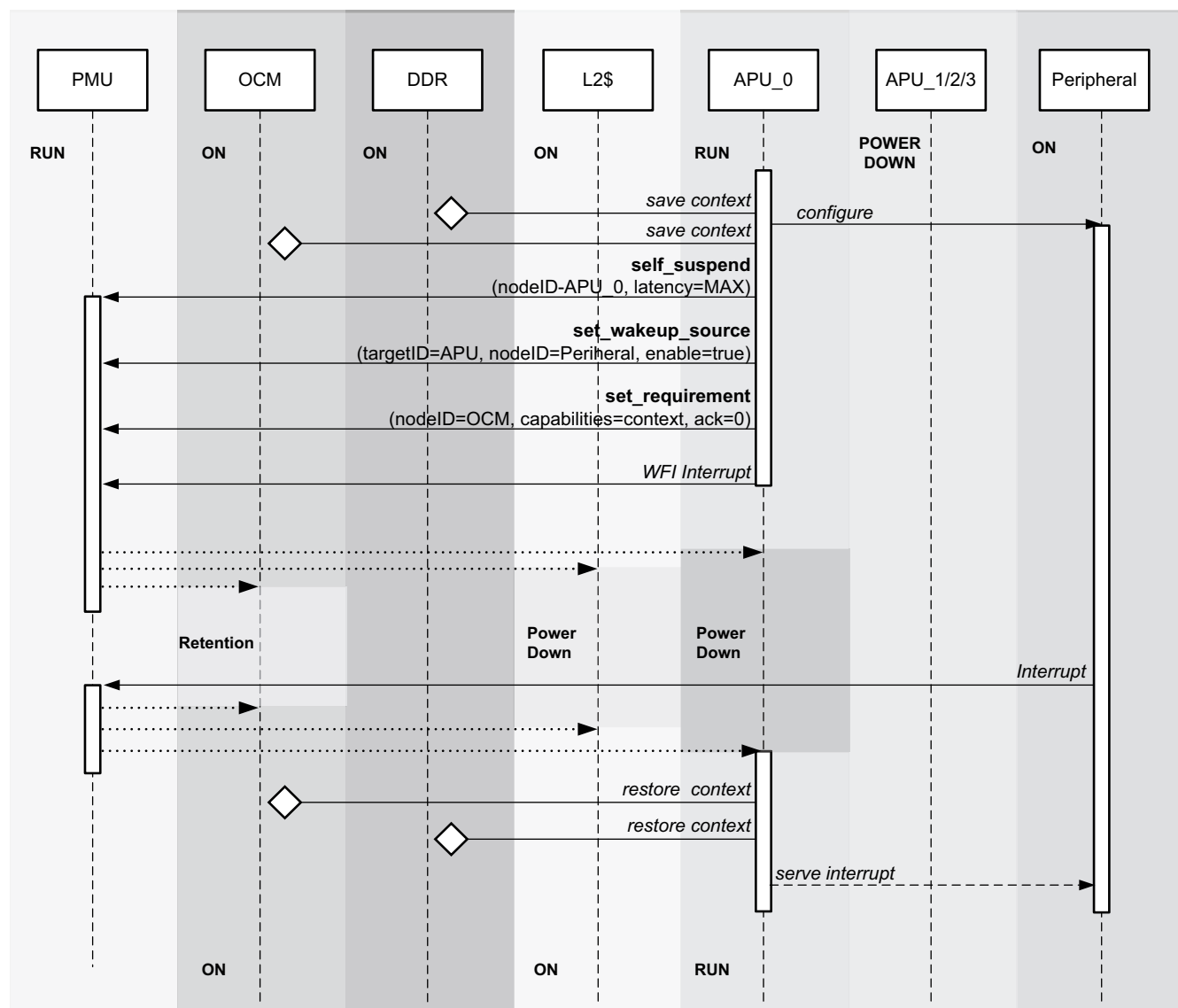
Note: The OCM0 memory remains powered on after the APU calls `XStatus XPm_ReleaseNode`, because the RPU has also requested the same slave node. It is after the RPU also releases the OCM0 node that the PMU powers off the OCM0 memory.

Processor Unit Suspend and Resume

To allow a processor unit (PU) to be powered off, as opposed to just entering an idle state, an external entity is required to take care of the power-down and power-up transitions.

For the Zynq UltraScale+ MPSoC device, the platform management unit (PMU) is the responsible entity for performing all power state changes.

The processor unit (PU) notifies the PMU that a power state transition is being requested. The following figure illustrates the process.



X20023-110217

Figure 11-7: APU Suspend and Resume Procedure

The [Self-Suspending a CPU/PU](#) section provides more details on the suspend or resume procedure. Each PU usually depends on a number of slave nodes to be able to operate.

Sub-system Power Management

Isolation Configuration

The Zynq UltraScale+ MPSoC can be partitioned into sub-systems, so that they can be managed independently by the power management framework. For example, you can define a Linux sub-system and a Real-time sub-system. The Linux sub-system may include the APU (as the PM master) and a number of peripherals (as the PM slaves). The Real-time sub-system may include the RPU and a number of other peripherals. Each sub-system can be powered up, powered down, restarted or suspended without affecting the other sub-systems. A sub-system has only one PM Master, and may include both FPD and LPD peripherals.

You can create your own sub-systems using the Vivado PCW tool. [Figure 11-8](#) shows the PCW screen shots of a valid configuration, which contains only an APU sub-system and no RPU sub-systems.

Please review **Known Limitations** under the **Isolation Configuration** Section of PG201.

☒ Enable Isolation
 ☒ Enable Secure Debug
 ☐ Lock Unused Memory

Search:

Name	Start Address	Size	Unit	TZ Settings	Access Settl...	End Address	Type
LINUX							
Masters							
SD1				Secure			
GEM3				NonSecure			
APU							
PCIe				NonSecure			
DP				NonSecure			
GPU				NonSecure			
Coresight							
SATA0				NonSecure			
SATA1				NonSecure			
USB0				NonSecure			
FPD_DMA				NonSecure			
DAP							
QSPI				NonSecure			
Slaves							
Memory							
DDR_LOW	0x0	2	GB	NonSecure	Read/Write	0x7FFFFFFF	DDR
QSPI_Linear...	0xC0000000	524288	KB	NonSecure	Read/Write	0xDFFFFFFF	LPD
Peripherals							
CAN1	0xFF070000	64	KB	NonSecure	Read/Write	0xFF07FFFF	LPD
GEM3	0xFF0E0000	64	KB	NonSecure	Read/Write	0xFF0EFFFF	LPD
GPIO	0xFF0A0000	64	KB	NonSecure	Read/Write	0xFF0AFFFF	LPD
I2C0	0xFF020000	64	KB	NonSecure	Read/Write	0xFF02FFFF	LPD
I2C1	0xFF030000	64	KB	NonSecure	Read/Write	0xFF03FFFF	LPD
SWDT0	0xFF150000	64	KB	NonSecure	Read/Write	0xFF15FFFF	LPD
TTC0	0xFF110000	64	KB	NonSecure	Read/Write	0xFF11FFFF	LPD
UART0	0xFF000000	64	KB	NonSecure	Read/Write	0xFF00FFFF	LPD
UART1	0xFF010000	64	KB	NonSecure	Read/Write	0xFF01FFFF	LPD
TTC1	0xFF120000	64	KB	NonSecure	Read/Write	0xFF12FFFF	LPD
TTC2	0xFF130000	64	KB	NonSecure	Read/Write	0xFF13FFFF	LPD
TTC3	0xFF140000	64	KB	NonSecure	Read/Write	0xFF14FFFF	LPD
Control and Status Registers							

Figure 11-8: PCW Configuration

Name	Start Address	Size	Unit	TZ Settings	Access Setti...	End Address	Type
Linux							
Masters							
Slaves							
Memory							
Peripherals							
Control and Status Registers							
USB3_0	0xFF9D0000	64	KB	NonSecure	Read/Write	0xFF9DFFFF	LPD
USB3_0_XHCI	0xFE200000	1024	KB	NonSecure	Read/Write	0xFE2FFFFF	LPD
Coresight	0xFE800000	8192	KB	NonSecure	Read/Write	0xFEFFFFF	LPD
LPD_DMA_0	0xFFA80000	64	KB	NonSecure	Read/Write	0xFFA8FFFF	LPD
LPD_DMA_1	0xFFA90000	64	KB	NonSecure	Read/Write	0xFFA9FFFF	LPD
LPD_DMA_2	0xFFAA0000	64	KB	NonSecure	Read/Write	0xFFAAFFFF	LPD
LPD_DMA_3	0xFFAB0000	64	KB	NonSecure	Read/Write	0xFFABFFFF	LPD
LPD_DMA_4	0xFFAC0000	64	KB	NonSecure	Read/Write	0xFFACFFFF	LPD
LPD_DMA_5	0xFFAD0000	64	KB	NonSecure	Read/Write	0xFFADFFFF	LPD
LPD_DMA_6	0xFFAE0000	64	KB	NonSecure	Read/Write	0xFFAEFFFF	LPD
LPD_DMA_7	0xFFAF0000	64	KB	NonSecure	Read/Write	0xFFAFFFFF	LPD
QSPI	0xFF0F0000	64	KB	NonSecure	Read/Write	0xFF0FFFFF	LPD
SD1	0xFF170000	64	KB	NonSecure	Read/Write	0xFF17FFFF	LPD
AMS	0xFFA50000	64	KB	NonSecure	Read/Write	0xFFA5FFFF	LPD
APM1	0xFFA00000	64	KB	NonSecure	Read/Write	0xFFA0FFFF	LPD
APM2	0xFFA10000	64	KB	NonSecure	Read/Write	0xFFA1FFFF	LPD
APM_FPD_LPD	0xFFA30000	64	KB	NonSecure	Read/Write	0xFFA3FFFF	LPD
APM_INTC_I0U	0xFFA20000	64	KB	NonSecure	Read/Write	0xFFA2FFFF	LPD
IOU_GPV	0xFE000000	1024	KB	NonSecure	Read/Write	0xFE0FFFFF	LPD
IPI_CTRL	0xFF380000	512	KB	NonSecure	Read/Write	0xFF3FFFFF	LPD
LPD_GPV	0xFE100000	1024	KB	NonSecure	Read/Write	0xFE1FFFFF	LPD
RTC	0xFFA60000	64	KB	NonSecure	Read/Write	0xFFA6FFFF	LPD

Figure 11-9: PCW Configuration Contd

APU_secure							
Masters							
SD1				Secure			
APU							
Slaves							
Memory							
OCM	0xFFFC0000	256	KB	Secure	Read/Write	0xFFFFFFF	OCM
Control and Status Registers							
CRF_APB	0xFD1A0000	1280	KB	Secure	Read/Write	0xFD20FFFF	FPD
CRL_APB	0xFF5E0000	2560	KB	Secure	Read/Write	0xFF5FFFFF	LPD
EFUSE	0xFFCC0000	64	KB	Secure	Read/Write	0xFFCCFFFF	LPD
IOU_SLCR	0xFF180000	768	KB	Secure	Read/Write	0xFF23FFFF	LPD
PMU Firmware							
Masters							
PMU							
Slaves							
Peripherals							
UART0	0xFF000000	64	KB	NonSecure	Read/Write	0xFF00FFFF	LPD
Control and Status Registers							
CRF_APB	0xFD1A0000	1280	KB	Secure	Read/Write	0xFD20FFFF	FPD
DDR_XMPU0...	0xFD000000	64	KB	Secure	Read/Write	0xFD00FFFF	FPD
DDR_XMPU1...	0xFD010000	64	KB	Secure	Read/Write	0xFD01FFFF	FPD
DDR_XMPU2...	0xFD020000	64	KB	Secure	Read/Write	0xFD02FFFF	FPD
DDR_XMPU3...	0xFD030000	64	KB	Secure	Read/Write	0xFD03FFFF	FPD
DDR_XMPU4...	0xFD040000	64	KB	Secure	Read/Write	0xFD04FFFF	FPD
DDR_XMPU5...	0xFD050000	64	KB	Secure	Read/Write	0xFD05FFFF	FPD
FPD_SLCR	0xFD610000	512	KB	Secure	Read/Write	0xFD68FFFF	FPD
FPD_XMPU...	0xFD050000	64	KB	Secure	Read/Write	0xFD05FFFF	FPD
LPD_XPPU	0xFF980000	64	KB	Secure	Read/Write	0xFF98FFFF	LPD
CRL_APB	0xFF5E0000	2560	KB	Secure	Read/Write	0xFF5FFFFF	LPD
EFUSE	0xFFCC0000	64	KB	Secure	Read/Write	0xFFCCFFFF	LPD
IOU_SLCR	0xFF180000	768	KB	Secure	Read/Write	0xFF23FFFF	LPD
LPD_SLCR	0xFF410000	640	KB	Secure	Read/Write	0xFF4AFFFF	LPD
OCM_XMPU...	0xFFA70000	64	KB	Secure	Read/Write	0xFFA7FFFF	LPD
RPU	0xFF9A0000	64	KB	Secure	Read/Write	0xFF9AFFFF	LPD

Figure 11-10: PCW Configuration Contd

Note: The PCW tool is also used to isolate some peripherals from each other for security purposes. See *Zynq UltraScale+ MPSoC: Embedded Design Tutorial* (UG1209)[Ref 13] and *Zynq UltraScale+ MPSoC Processing System LogiCORE IP Product Guide* (PG201)[Ref 14] for details on how to set up isolation between peripherals.

Configuration Object

The sub-system configuration is captured in a Configuration Object, which is generated by the Vivado and PetaLinux toolchain. The Configuration Object contains:

- The PM Masters that are present in the system (APU and/or RPU). Any PM Master not specified in the Configuration Object will be powered down by the PMU.
- Configurable permissions for each PM Master, such as:
 - Which PM Master can use which PM Slave (A PM Master can use all the PM Slaves that belong in the same sub-system.)
 - Access to MMIO address regions.
 - Access to peripheral reset lines.
- Pre-allocated PM Slaves. The PM Master can use these PM Slaves without requesting for them first. These PM Slaves are needed by the PM Master in order to boot. The toolchain makes sure that the APU can access the L2 cache and DDR banks without first requesting for them. The same is true for the RPU accessing all the TCM banks.

During boot, the Configuration Object is passed from the FSBL to the PMU firmware. For more details, see the [Configuration Object](#).

Note: Isolation is not required for the Configuration Object to be created. You can create subsystems to customize the Configuration Object and then uncheck the isolation checkbox.

Power Management Initialization

Power management is disabled during boot and all the peripherals are powered up at this time. That is because it is often necessary to allow for possible, and temporary, inter-dependencies between peripherals during boot and initialization. When FSBL is finished with initializing the peripherals and loading the application binaries, it passes the Configuration Object to the PMU. The PMU is now aware of all the sub-systems and their associated PM Masters and PM Slaves. PM Masters and PM Slaves that are not included in the Configuration Object are never used, and are powered down by the PMU.

A PM Master is not likely to use all the PM Slaves at all times. Therefore, a PM Slave should be powered up only when it is being used. The PM Master must notify the PMU before and after using a PM Slave. This functionality is implemented in the PetaLinux kernel. This requirement hinders developers starting with a new RPU application, when the focus is on functionality and not power optimization. Therefore, it is convenient for the PMU to also support PM-incapable Masters that do not provide notifications when they are using the PM Slaves. This is done by keeping all the PM Slaves in the sub-system powered up until the

PM Master sends the `PmInitFinalize` request to the PMU. A PM-incapable Master will never send this request, which means that its PM Slaves will remain powered up at all times or until this PM Master itself is powered down.

A PM-capable Master sends this request after initializing the sub-system. The PMU then begins powering down the PM Slaves in this sub-system whenever they are not being used.

As a result, when there is an RPU master present in the system but it is not running any application, the PMU firmware will consider it as a PM incapable master and hence will never power down the RPU and its slaves. From the 2018.3 release and onwards, this behavior is fixed and allows you to power down unused RPUs. This change is protected by the compilation flag `ENABLE_UNUSED_RPU_PWR_DWN` and is enabled by default. When this flag is enabled, the unused RPU and allocated slaves will be powered down if not in use.

Note: If you do not want to power down RPU by default, set the `ENABLE_UNUSED_RPU_PWR_DWN` flag to 0 while compiling the PMU firmware. For the JTAG boot mode there is no impact on behavior change even though `ENABLE_UNUSED_RPU_PWR_DWN` flag is 1.

Note: Sub-systems may overlap each other. This means that some PM Slaves may belong to more than one sub-system (for example, DDR, OCM, and so on). If a PM Slave is in more than one sub-system, the PMU does not power down this PM Slave until it has been released by all its PM Masters, or until all these PM Masters have powered down themselves.

Default Configuration

By default, Isolation Configuration is disabled, and the tool chain generates a configuration with three sub-systems. Each has a PM Master: APU, R5-0 and R5-1. All three sub-systems contain all the PM Slaves (meaning that the sub-systems completely overlap each other.) This is the default configuration generated by PCW when the “Enable Isolation” box is unchecked. The default PetaLinux kernel configuration is PM-capable, but R5-0 and R5-1 must be also running “PM-capable” applications, or be powered down. Otherwise, the PMU will not power down any PM Slaves.

Note: You can create a configuration that does not allow the processors to boot and run. If you are a beginner, use the APU-only configuration as described in [Isolation Configuration](#) section and customize it as necessary.

RPU Lock-step vs. Split Mode

The toolchain infers the RPU run modes from the PCW Isolation Configuration as follows:

- No RPU present in any subsystem: Configuration Object contains no RPU.
- Only R5-0 present in subsystem(s): Configuration Object contains R5-0 running in lock-step mode.
- Both R5-0 and R5-1 in subsystems: Configuration Object contains R5-0 and R5-1 running in split mode.

- Only R5-1 present in subsystem(s): Configuration Object contains R5-1 running in split mode.

The default Configuration Object contains two RPU PM Masters: R5-0 and R5-1, and the PMU assumes that the R5-0 and R5-1 are running in split mode. However, the boot image actually determines whether the RPU runs in lock-step or split mode at boot time. The RPU run mode from the boot image must match the number of RPU PM Masters in the Configuration Object. Otherwise, the power management framework will not work properly.

Note: If you intend to use the R5 in lock-step mode, you need to ensure that the Isolation Configuration is enabled in PCW, and only R5-0 (not R5-1) is present in a subsystem.

Sharing Devices

Sharing access to devices between APU and RPU is possible but must always be done with great care. The access and operation of a device depend on its clock (if applicable), its configuration and its power state (on, off, retention, and so on.) The PMU makes sure the device is in the lowest power state that will satisfy the requirement of all the PM Masters, but it is up to the APU and RPU to set up the clock and configuration of the device.

Extra care must be taken when a device is shared between the APU running Linux and the RPU. Linux is not aware that another entity might be using one of its devices, and will clock-gate, power-gate and disable the device whenever it is not being used. The options available are:

- Disable Linux runtime power management of the device. See <https://www.kernel.org/doc/Documentation/ABI/testing/sysfs-devices-power>. This will keep the device running even when Linux is not using it, but the device will still be clock-gated and disabled when Linux goes to sleep.
- Implement a special driver for the device.

Any devices not used by the APU should be removed from the device tree.

Using the API for Power Management

Introduction

This chapter contains detailed instructions on how to use the Xilinx® power management framework (PMF) APIs to carry out common power management tasks.

Implementing Power Management on a Processor Unit

The Xilpm library provides the functions that the standalone applications executing on a processor can use to initiate the power management API calls.

See the *Xilinx Software Developer Kit Help* (UG782) [Ref 24] for information on how to include the `xilpm` library in a project.

Initializing the Xilpm Library

Before initiating any power management API calls, you must initialize the `xilpm` library by calling `XPm_InitXilpm`, and passing a pointer to a properly initialized inter-processor interrupt (IPI) driver instance.

See this [link](#) to the “Interrupts” chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11]. for more information regarding IPIs.

For more information about `XPm_InitXilpm`, see [Appendix K, XilPM Library v2.5](#).

Working with Slave Devices

The Zynq® UltraScale+™ MPSoC device power management framework (PMF) contains functions dedicated to managing slave devices (also referred to as PM slaves), such as memories and peripherals. Processor units (PUs) use these functions to inform the power management controller about the requirements (such as capabilities and wake-up latencies) for those devices. The power management controller manages the system so that each device resides in the lowest possible power state, meeting the requirements from all eligible PUs.

Requesting and Releasing a Node

A PU uses the `XPm_RequestNode` API to request the access to a slave device and assert its requirements on that device. The power management controller manages the requested device's power-on and active state, provided the PU and the slave belong to the same sub-system.

After a device is no longer used, the PU typically calls the `XPm_ReleaseNode` function to allow the PM controller to re-evaluate the power state of that device, and potentially place it into a low-power state. It also then allows other PUs to request that device.

For more information about `XPm_ReleaseNode`, see [Appendix K, XilPM Library v2.5](#).

Changing Requirements

When a PU is using a PM slave, its requirement on the slave's capability may change. For example, an interface port may go into a low power state, or even be completely powered off, if the interface is not being used. The PU may use `XPm_SetRequirement` to change the capability requirement of the PM slave. Typically, the PU would not release the PM slave if it will be changing the requirement again in the future.

The following example call changes the requirement for the node argument to require wake-interrupts only:

```
XPm_SetRequirement(node, PM_CAP_WAKEUP, 0, REQUEST_ACK_NO);
```



IMPORTANT: *Setting requirements of a node to zero is not equivalent to releasing the PM slave. By releasing the PM slave, a PU may be allowing other PUs to use the device exclusively.*

When multiple PUs share a PM slave (this applies mostly to memories), the power management controller selects a power state of the PM slave that satisfies all requirements of the requesting PUs.

The requirements on a PM slave include capability as well as latency requirements. Capability requirements may include a top capability state, some intermediate capability states, an inactive state (but with the configuration retained), and the off state. Latency requirement specifies the maximum time allowed for the PM slave to switch to the top capability state from any other state. If this time limit cannot be met, the power management controller will leave the PM slave in the top capability state regardless of other capability requirements.

For more information about `XPm_SetRequirement`, see [Appendix K, XilPM Library v2.5](#).

Self-Suspending a CPU/PU

A PU can be a cluster of CPUs. The APU is a PU, that has four CPUs. An RPU has two CPUs, but it is considered as two PUs when running in the split mode, and one PU when it is running in the lock-step mode.

To suspend itself, a CPU must inform the power management controller about its intent by calling the `XPm_SelfSuspend` function. The following actions then occur:

- After the `XPm_SelfSuspend()` call is processed, none of the future interrupts can prevent the CPU from entering a sleep state. To manage such behavior in the case of the APU and RPU, after the `XPm_SelfSuspend()` call has completed, all of the interrupts to a CPU are directed to the power management controller as GIC wake interrupts.
- The power management controller then waits for the CPU to finalize the suspend procedure. The PU informs the power management controller that it is ready to enter a sleep state by calling `XPm_SuspendFinalize`.
- The `XPm_SuspendFinalize()` function is architecture-dependent. It ensures that any outstanding power management API call is processed, then executes the architecture-specific suspend sequence, which also signals the suspend completion to the power management controller.
- For Arm processors such as the APU and RPU, the `XPm_SuspendFinalize()` function uses the wait for interrupt (WFI) instruction, which suspends the CPU and triggers an interrupt to the power management controller.

- When the suspend completion is signaled to the power management controller, the power management controller places the CPU into reset, and may power down the power island of the CPU, provided that no other component within the island is currently active.
- Interrupts enabled through the GIC interface of the CPU are redirected to the power management controller (PMC) as a GIC wake interrupt assigned to that particular CPU. Because the interrupts are redirected, the CPU can only be woken up using the power management controller.
- Suspending a PU requires suspending all of its CPUs individually.

For more information about `XPM_SelfSuspend` and `XPm_SuspendFinalize`, see [Appendix K, XilPM Library v2.5](#).

Resuming Execution

A CPU can be woken up either by a wake interrupt triggered by a hardware resource or by an explicit wake request using the `XPM_RequestWakeup` API.

The CPU starts executing from the resume address provided with the `XPm_SelfSuspend` call.

For more information about `XPM_RequestWakeup` and `XPm_SelfSuspend`, see [Appendix K, XilPM Library v2.5](#).

Setting up a Wake-up Source

The power management controller can power down the entire FPD if none of the FPD devices are in use and existing latency requirements allow this action. If the FPD is powered off and the APU is to be woken up by an interrupt triggered by a device in the LPD, the GIC Proxy must be configured to allow propagation of FPD wake events. The APU can ensure this by calling `XPM_SetWakeUpSource` for all devices that might need to issue wake interrupts.

Hence, prior to suspending, the APU must call `XPm_SetWakeUpSource(NODE_APU, node, 1)` to add the required slaves as a wake-up source. The APU can then set the requirements to zero for all slaves it is using. After the APU finalizes its suspend procedure, and provided that no other PU is using any resource in the FPD, the PM controller powers off the entire FPD and configures the GIC proxy to enable propagation of the wake event of the LPD slaves.

For more information about `XPM_SetWakeUpSource`, see [Appendix K, XilPM Library v2.5](#).

Aborting a Suspend Procedure

If a PU decides to abort the suspend procedure after calling the `XPM_SetSelfSuspend` function, it must inform the power management controller about the aborted suspend by calling the `XPm_AbortSuspend` function.

For more information about `XPM_SetSelfSuspend` and `XPm_AbortSuspend`, see [Appendix K, XilPM Library v2.5](#).

Handling PM Slaves During the Suspend Procedure

A PU that suspends itself must inform the power management controller about its changed requirements on the peripherals and memories in use. If a PU fails inform the power management controller, all of the used devices remain powered on. Typically, for memories you must ensure that their context is preserved by using the following function:

```
XPm_SetRequirement(node, PM_CAP_CONTEXT, 0, REQUEST_ACK_NO);
```

When setting requirements for a PM slave during the suspend procedure; after calling `XPm_SelfSuspend`, the setting is deferred until the CPU finishes the suspend. This deference ensures that devices that are needed for completing the suspend procedure can enter a low power state after the calling CPU finishes suspend.

A common example is instruction memory, which a CPU can access until the end of a suspend. After the CPU suspends a memory, that memory can be placed into retention. All deferred requirements reverse automatically before the respective CPU is woken up.

When an entire PU suspends, the last awake CPU within the PU must manage the changes to the devices.

For more information about `XPm_SelfSuspend`, see [Appendix K, XilPM Library v2.5](#).

Example Code for Suspending an APU/RPU

There the following is an example of source code for suspending the APU or RPU:

```
/* Base address of vector table (reset-vector) */
extern void *_vector_table;
/* Inform PM controller that APU_0 intends to suspend */
XPm_SelfSuspend(NODE_APU_0, MAX_LATENCY, 0,
(u64)&_vector_table);
/**
 * Set requirements for OCM banks to preserve their context.
 * The PM controller will defer putting OCMs into retention
until the suspend is finalized
 */
XPm_SetRequirement(NODE_OCM_BANK_0, PM_CAP_CONTEXT, 0,
REQUEST_ACK_NO);
XPm_SetRequirement(NODE_OCM_BANK_1, PM_CAP_CONTEXT, 0,
REQUEST_ACK_NO);
XPm_SetRequirement(NODE_OCM_BANK_2, PM_CAP_CONTEXT, 0,
REQUEST_ACK_NO);
XPm_SetRequirement(NODE_OCM_BANK_3, PM_CAP_CONTEXT, 0,
REQUEST_ACK_NO);

/* Flush data cache */
Xil_DCacheFlush();
/* Inform PM controller that suspend procedure is completed */
XPm_SuspendFinalize();
```

Suspending the Entire FPD Domain

To power-down the entire full power domain, the power management controller must suspend the APU at a time when none of the FPD devices is in use. After this condition is met, the power management controller can power-down the FPD automatically. The power management controller powers down the FPD if no latency requirements constrain this action, otherwise the FPD remains powered on.

Forcefully Powering Down the FPD

There is the option to force the FPD to power-down by calling the function `XPM_ForcePowerdown`. This requires that the requesting PU has proper privileges configured in the power management controller. The power management controller releases all PM Slaves used by the APU automatically.

Note: This force method is typically not recommended, especially when running complex operating systems on the APU because it could result in loss of data or system corruption, due to the OS not suspending itself gracefully.



IMPORTANT: Use the `XPm_RequestSuspend` API.

For more information about `XPM_ForcePowerdown`, see [Appendix K, XilPM Library v2.5](#).

Interacting With Other Processing Units

Suspending a PU

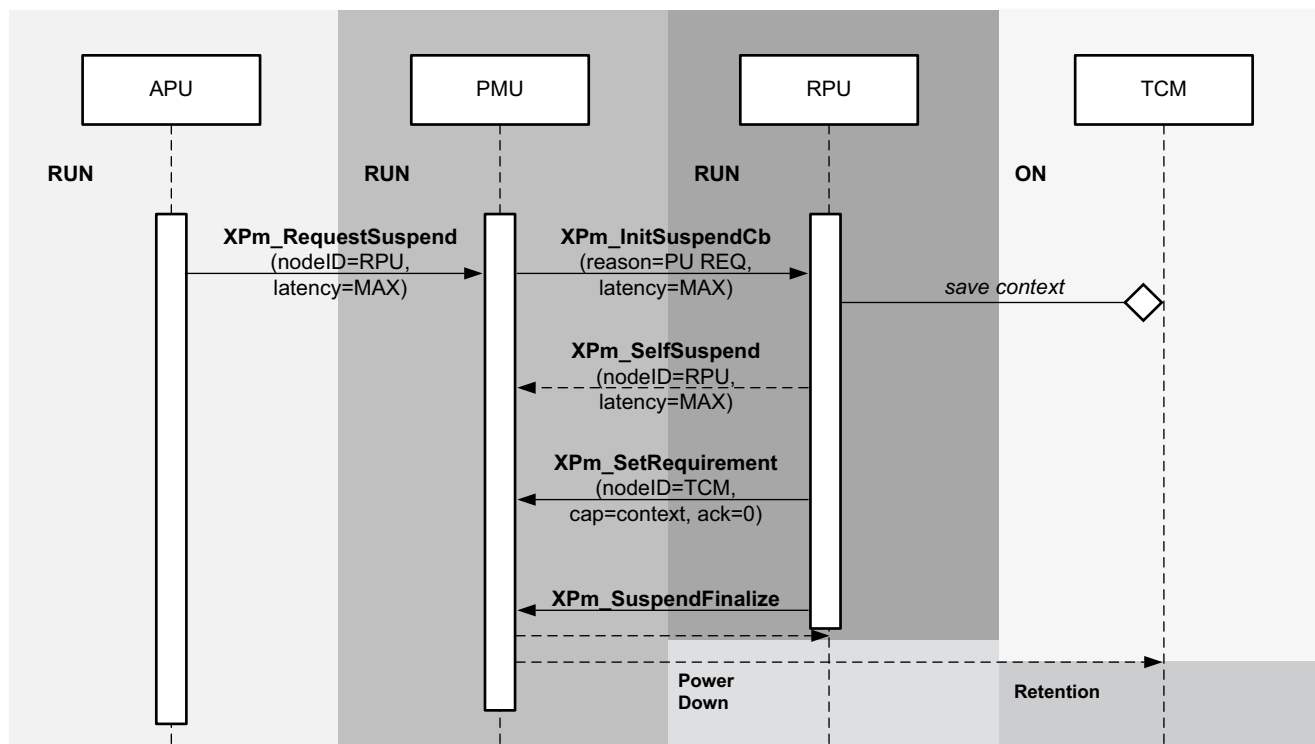
A PU can request that another PU be suspended by calling `XPm_RequestSuspend`, and passing the targeted node name as an argument.

This causes the power management controller to call `XPm_InitSuspendCb()`, which is a callback function implemented in the target PU. The target PU then initiates its own suspend procedure, or call `XPm_AbortSuspend` and specify the abort reason. For example, you can request an APU to suspend with the following command:

```
XPm_RequestSuspend(NODE_APU, REQUEST_ACK_NON_BLOCKING, MAX_LATENCY, 0);
```

The following diagram shows the general sequence triggered by a call to the `XPm_RequestSuspend`.

For more information about `XPm_RequestSuspend`, `XPm_InitSuspendCb`, and `XPm_AbortSuspend`, see [Appendix K, XilPM Library v2.5](#).



X20024-110217

Figure 11-11: APU initiating suspend for the RPU by calling `XPm_RequestSuspend`

Waking a PU

Additionally, a PU can request the wake-up of one of its CPUs or of another PU by calling `XPm_RequestWakeup`.

- When processing the call, the power management controller causes a target CPU or PU to be awakened.
- If a PU is the target, only one of its CPUs is woken-up by this request.
- The CPU chosen by the power management controller is considered the primary CPU within the PU.

The following is an example of a wake-up request:

```
XPm_RequestWakeup (NODE_APU_1, REQUEST_ACK_NO);
```

For more information about `XPm_RequestWakeup`, see [Appendix K, XiIPM Library v2.5](#).

XilPM Implementation Details

The system layer of the PM framework is implemented on the Zynq UltraScale+ MPSoC using inter-processor interrupts (IPIs). To issue an EEMI API call, a PU will write the API data (API ID and arguments) into the IPI request buffer and then trigger the IPI to the PMU.

After the PM controller processes the request it will send the acknowledge depending on the particular EEMI API and provided arguments.

Payload mapping for API calls to PMU

Each EEMI API call is uniquely identified by the following data:

- EEMI API identifier (ID)
- EEMI API arguments

Please see Appendix A for a list of all API identifiers as well as API argument values.

Prior to initiating an IPI to the PMU, the PU shall write the information about the call into the IPI request buffer. Each data written into the IPI buffer is a 32-bit word. Total size of the payload is six 32-bit words - one word is reserved for the EEMI API identifier, while the remaining words are used for the arguments. Writing to the IPI buffer starts from offset zero. The information is mapped as follows:

- Word [0]EEMI API ID
- Word [1:5]EEMI API arguments

The IPI response buffer is used to return the status of the operation as well as up to 3 values.

- Word [0]success or error code
- Word [1:3]value 1..3

Payload mapping for API callbacks from the PMU

The EEMI API includes callback functions, invoked by the PM controller, sent to a PU.

- Word [0]EEMI API Callback ID
- Word [1:5]EEMI API arguments

Refer to [Appendix K, XilPM Library v2.5](#) for a list of all API identifiers as well as API argument values.

Issuing EEMI API calls to the PMU

Before issuing an API call to the PMU, a PU must wait until its previous API call is processed by the PMU. A check for completion of a PMU action can be implemented by reading the corresponding IPI observation register.

An API call is issued by populating the IPI payload buffer with API data and triggering an IPI interrupt to the PMU. In case of a blocking API call, the PMU will respond by populating the response buffer with the status of the operation and up to 3 values. See Appendix B for a list of all errors that can be sent by the PMU if a PM operation was unsuccessful. The PU must wait until the PMU has finished processing the API call prior to reading the response buffer, to ensure that the data in the response buffer is valid.

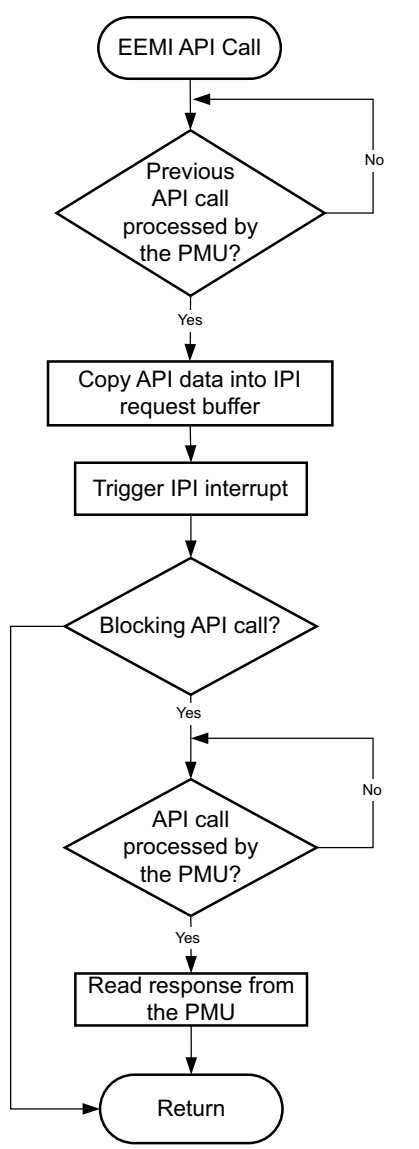


Figure 11-12: Example Flow of Issuing API Call to the PMU

Handling API callbacks from the PMU

The PMU invokes callback functions to the PU by populating the IPI buffers with the API callback data and triggering an IPI interrupt to the PU. In order to receive such interrupts, the PU must properly initialize the IPI block and interrupt controller. A single interrupt is dedicated to all callbacks. For this reason, element 0 of the payload buffer contains the API ID, which the PU should use to identify the API callback. The PU should then call the respective API callback function, passing in the arguments obtained from locations 1 to 4 of the IPI request buffer.

An implementation of this behavior can be found in the XilPM library.

Linux

Linux executes on the EL1 level, and the communication between Linux and the ATF software layer is realized using SMC calls.

Power management features based on the EEMI API have been ported to the Linux kernel, ensuring that the Linux-centric power management features utilize the EEMI services provided by the PMU.

Additionally, the EEMI API can be access directly via debugfs for debugging purposes. Note that direct access to the EEMI API through debugfs will interfere with the kernel power management operations and may cause unexpected problems.

All the Linux power management features presented in this chapter are available in the PetaLinux default configuration.

User Space PM Interface

System Power States

The user may request to change the power state of a system or the entire system. The PMU Facilitates the switching of the system or sub-system to the new power state.

Shutdown

The user may shutdown the APU sub-system with the standard 'shutdown' command.

To shut down the entire system, the user must shut down all the other sub-systems prior to shutting down the APU sub-system. For example, use the following command to power down the PL.

```
echo pm_release_node 69 > /sys/kernel/debug/zynqmp-firmware/pm
```

Use this command to power up the PL again:

```
echo pm_request_node 69 > /sys/kernel/debug/zynqmp-firmware/pm
```

For information about how to shut down the PL sub-system, see the *Libmetal and OpenAMP User Guide* (UG1186) [Ref 16].

Reboot

The user can use the `reboot` command to reset the APU, the PS or the System. By default, the `reboot` command resets the System.

You can change the scope of the `reboot` command to APU or PS if required.

To change the reboot scope to APU:

```
echo subsystem > /sys/firmware/zynqmp/shutdown_scope
```

To change the reboot scope to PS:

```
echo ps_only > /sys/firmware/zynqmp/shutdown_scope
```

To change the reboot scope to System:

```
echo system > /sys/firmware/zynqmp/shutdown_scope
```

The reboot scope is set to System again after the reset.

Suspend

The kernel is suspended when the CPU and most of the peripherals are powered down. The system run states needed to resume from suspend is stored in the DRAM, which is put into self-refresh mode.

Kernel configurations required:

- Power management options
 - [*] Suspend to RAM and standby
 - [*] User space wakeup sources interface
 - [*] Device power management core functionality
- Device Drivers
 - SoC (System On Chip) specific Drivers
 - Xilinx SoC drivers
 - Zynq MPSoC SoC
 - [*] Enable Xilinx Zynq MPSoC Power Management driver
 - [*] Enable Zynq MPSoC generic PM domains

- Firmware Drivers
 - Zynq MPSoC Firmware Drivers
 - *- Enable Xilinx Zynq MPSoC firmware interface

Note that any device can prevent the kernel from suspending.

See also https://wiki.archlinux.org/index.php/Power_management/Suspend_and_hibernate

To suspend the kernel:

```
$ echo mem > /sys/power/state
```

Wake-up Source

The kernel resumes from the suspend mode when a wake-up event occurs. The following wake-up sources can be used:

- UART

If enabled as a wake-up source, a UART input will trigger the kernel to resume from the suspend mode.

Kernel configurations required:

- Same as [Suspend](#).

For example, to wake up the APU on UART input:

```
$ echo enabled > /sys/devices/platform/amba/ff000000.serial/tty/ttyPS0/power/wakeup
```

- RTC

If enabled as a wake-up source, the kernel will resume from the suspend mode when the RTC timer expires. Note that the RTC wake-up source is enabled by default.

Kernel configurations required:

- Same as [Suspend](#).

For example, to set RTC to wake up the APU after 10 seconds:

```
$ echo +10 > /sys/class/rtc/rtc0/wakealarm
```

- GPIO

If enabled as a wake-up source, a GPIO event will trigger the kernel to resume from the suspend mode.

Kernel configurations required:

- Device Drivers

- Input device support, [*]

Generic input layer (needed for keyboard, mouse, ...) (INPUT [=y])

[*] Keyboards (INPUT_KEYBOARD [=y])

[*] GPIO Buttons (CONFIG_KEYBOARD_GPIO=y)

[*] Polled GPIO buttons

For example, to wake up the APU on the GPIO pin:

```
$ echo enabled > /sys/devices/platform/gpio-keys/power/wakeup
```

Power Management for the CPU

CPU Hotplug

The user may take one or more APU cores on-line and off-line as needed via the CPU Hotplug control interface.

Kernel configurations required:

- Kernel Features
 - [*] Support for hot-pluggable CPUs

See also:

- <https://www.kernel.org/doc/Documentation/cpu-hotplug.txt>
- <http://lxr.free-electrons.com/source/Documentation/devicetree/bindings/arm/idle-states.txt>

For example, to take CPU3 off-line:

```
$ echo 0 > /sys/devices/system/cpu/cpu3/online
```

CPU Idle

If enabled, the kernel may cut power to individual APU cores when they are idling.

Kernel configurations required:

- CPU Power Management
 - CPU Idle
 - [*] CPU idle PM support
 - Arm CPU Idle Drivers

[*] Generic Arm/Arm64 CPU idle Driver

See also:

- <https://www.kernel.org/doc/Documentation/cpuidle/core.txt>
- <https://www.kernel.org/doc/Documentation/cpuidle/driver.txt>
- <https://www.kernel.org/doc/Documentation/cpuidle/governor.txt>
- <https://www.kernel.org/doc/Documentation/cpuidle/sysfs.txt>

Below is the sysfs interface for cpuidle.

```
$ ls -lR /sys/devices/system/cpu/cpu0/cpuidle/

/sys/devices/system/cpu/cpu0/cpuidle/:
drwxr-xr-x  2 root  root          0 Jun 10 21:55 state0
drwxr-xr-x  2 root  root          0 Jun 10 21:55 state1

/sys/devices/system/cpu/cpu0/cpuidle/state0:
-r--r--r--  1 root  root        4096 Jun 10 21:55 desc
-rw-r--r--  1 root  root        4096 Jun 10 21:55 disable
-r--r--r--  1 root  root        4096 Jun 10 21:55 latency
-r--r--r--  1 root  root        4096 Jun 10 21:55 name
-r--r--r--  1 root  root        4096 Jun 10 21:55 power
-r--r--r--  1 root  root        4096 Jun 10 21:55 residency
-r--r--r--  1 root  root        4096 Jun 10 21:55 time
-r--r--r--  1 root  root        4096 Jun 10 21:55 usage

/sys/devices/system/cpu/cpu0/cpuidle/state1:
-r--r--r--  1 root  root        4096 Jun 10 21:55 desc
-rw-r--r--  1 root  root        4096 Jun 10 21:55 disable
-r--r--r--  1 root  root        4096 Jun 10 21:55 latency
-r--r--r--  1 root  root        4096 Jun 10 21:55 name
-r--r--r--  1 root  root        4096 Jun 10 21:55 power
-r--r--r--  1 root  root        4096 Jun 10 21:55 residency
-r--r--r--  1 root  root        4096 Jun 10 21:55 time
-r--r--r--  1 root  root        4096 Jun 10 21:55 usage
```

where:

- desc: Small description about the idle state (string)
- disable: Option to disable this idle state (bool)
- latency: Latency to exit out of this idle state (in microseconds)
- name: Name of the idle state (string)
- power: Power consumed while in this idle state (in milliwatts)
- time: Total time spent in this idle state (in microseconds)
- usage: Number of times this state was entered (count)

Below is the sysfs interface for cpuidle governors.

```
$ ls -lR /sys/devices/system/cpu/cpuidle/
/sys/devices/system/cpu/cpuidle/:
-r--r--r-- 1 root root      4096 Jun 10 21:55 current_driver
-r--r--r-- 1 root root      4096 Jun 10 21:55 current_governor_ro
```

CPU Freq

If enabled, the CPU cores may switch between different operation clock frequencies.

Kernel configurations required:

- CPU Frequency scaling
 - [*] CPU Frequency scaling
 - Default CPUFreq governor
 - Userspace
- CPU Power Management
 - [*] CPU Frequency scaling
 - Default CPUFreq governor
 - Userspace
 - <*> Generic DT based cpufreq driver

Look up the available CPU speeds:

```
$ cat /sys/devices/system/cpu/cpu*/cpufreq/scaling_cpu_freq
```

Select the 'userspace' governor for CPU frequency control:

```
$ echo userspace > /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
```

Look up the current CPU speed (same for all cores):

```
$ cat /sys/devices/system/cpu/cpu*/cpufreq/scaling_cpu_freq
```

Change the CPU speed (same for all cores):

```
$ echo <freq> > /sys/devices/system/cpu/cpu0/cpufreq/scaling_setspeed
```

For details on adding and changing CPU frequencies, see the [Linux kernel documentation on Generic Operating Points](#).

Power Management for the Devices

Clock Gating

Stop device clocks when they are not being used (also called Common Clock Framework.)

Kernel configurations required:

- Common Clock Framework
 - [*] Support for Xilinx ZynqMP Ultrascale+ clock controllers

Runtime PM

Power off devices when they are not being used. Note that individual drivers may or may not support run-time power management.

Kernel configurations required:

- Power management options
 - [*] Suspend to RAM and standby
- Device Drivers
 - SoC (System On Chip) specific Drivers
 - [*] Xilinx Zynq MPSoC driver support

Global General Storage Registers

Four 32-bit storage registers are available for general use. Their values are not preserved across after software reboots. [Table 11-1](#) lists the global general storage registers.

Table 11-1: Global General Storage Registers

Device Node	MMIO Register	MMIO Address	Valid Value Range
/sys/firmware/zynqmp/ggs0	GLOBAL_GEN_STORAGE0	0xFFD80030	0x00000000 - 0xFFFFFFFF
/sys/firmware/zynqmp/ggs1	GLOBAL_GEN_STORAGE1	0xFFD80034	0x00000000 - 0xFFFFFFFF
/sys/firmware/zynqmp/ggs2	GLOBAL_GEN_STORAGE2	0xFFD80038	0x00000000 - 0xFFFFFFFF
/sys/firmware/zynqmp/ggs3	GLOBAL_GEN_STORAGE3	0xFFD8003C	0x00000000 - 0xFFFFFFFF

Read the value of a global storage register:

```
$cat /sys/firmware/zynqmp/ggs0
```

Write the mask and value of a global storage register:

```
$echo 0xFFFFFFFF 0x1234ABCD > /sys/firmware/zynqmp/ggs0
```


Persistent Global General Storage Registers

Four 32-bit persistent global storage registers are available for general use. Their values are preserved across after software reboots. Table 11-2 lists the persistent global general storage registers.

Table 11-2: Persistent Global General Storage Registers

Device Node	MMIO Register	MMIO Address	Valid Value Range
/sys/firmware/zynqmp/pggs0	PERS_GLOB_GEN_STORAGE0	0xFFD80050	0x00000000 - 0xFFFFFFFF
/sys/firmware/zynqmp/pggs1	PERS_GLOB_GEN_STORAGE1	0xFFD80054	0x00000000 - 0xFFFFFFFF
/sys/firmware/zynqmp/pggs2	PERS_GLOB_GEN_STORAGE2	0xFFD80058	0x00000000 - 0xFFFFFFFF
/sys/firmware/zynqmp/pggs3	PERS_GLOB_GEN_STORAGE3	0xFFD8005C	0x00000000 - 0xFFFFFFFF

Read the value of a persistent global storage register:

```
$cat /sys/firmware/zynqmp/pggs0
```

Write the mask and value of a persistent global storage register:

```
$echo 0xFFFFFFFF 0x1234ABCD > /sys/firmware/zynqmp/pggs0
```

Demo

A demo script is included with the PetaLinux pre-built images, which performs a few simple power management tasks:

- System Suspend
- CPU Hotplug
- CPU Freq
- System Reboot
- System Shutdown

To start the demo, type the following command:

```
$ hellopm
```

Debug Interface

The PM platform driver exports a standard debugfs interface to access all EEMI services. The interface is intended for testing only and does not contain any checking regarding improper usage, and the number, type and valid ranges of the arguments. The user should be aware that invoking EEMI services directly via this interface can very easily interfere with the kernel power management operations, resulting in unexpected behavior or system crash. ZynqMP debugfs interface is disabled by default in `defconfig`. It needs to be enabled explicitly as mentioned below.

Kernel configurations required (in this order):

- Kernel hacking
 - Compile-time checks and compiler options
 - [*] Debug Filesystem
- Firmware Drivers
 - Zynq MPSoC Firmware Drivers
 - [*] Enable Xilinx Zynq MPSoC firmware interface
 - [*] Enable Xilinx Zynq MPSoC firmware debug APIs

You may invoke any EEMI API except for:

- Self Suspend
- System Shutdown
- Force Power Down the APU
- Request Wake-up the APU

Command-line Input

The user may invoke an EEMI service by writing the EEMI API ID, followed by up to 4 arguments, to the debugfs interface node.

API ID

Function ID can be EEMI API function name or ID number, type string or type integer, respectively.

Arguments

The number and type of the arguments directly depend on the selected API function. All arguments must be provided as integer types and represent the ordinal number for that specific argument type from the EEMI argument list. For more information about function descriptions, type and number of arguments see the EEMI API Specification.

Example

The following example shows how to invoke a request_node API call for NODE_USB_0.

```
$ echo "pm_request_node 22 1 100 1" > /sys/kernel/debug/zynqmp-firmware/pm
```

Command List

Get API Version

Get the API version.

```
$ echo pm_get_api_version > /sys/kernel/debug/zynqmp-firmware/pm
```

Request Suspend

Request another PU to suspend itself.

```
$ echo pm_request_suspend <node> > /sys/kernel/debug/zynqmp-firmware/pm
```

Self Suspend

Notify PMU that this PU is about to suspend itself.

```
$ echo pm_self_suspend <node> > /sys/kernel/debug/zynqmp-firmware/pm
```

Force Power Down

Force another PU to power down.

```
$ echo pm_force_powerdown <node> > /sys/kernel/debug/zynqmp-firmware/pm
```

Abort Suspend

Notify PMU that the attempt to suspend has been aborted.

```
$ echo pm_abort_suspend > /sys/kernel/debug/zynqmp-firmware/pm
```

Request Wake-up

Request another PU to wake up from suspend state.

```
$ echo pm_request_wakeup <node> <set_address> <address> >
/sys/kernel/debug/zynqmp-firmware/pm
```

Set Wake-up Source

Set up a node as the wake-up source.

```
$ echo pm_set_wakeup_source <target> <wkup_node> <enable> >
/sys/kernel/debug/zynqmp-firmware/pm
```

Request Node

Request to use a node.

```
$ echo pm_request_node <node> > /sys/kernel/debug/zynqmp-firmware/pm
```

Release Node

Free a node that is no longer being used.

```
$ echo pm_release_node <node> > /sys/kernel/debug/zynqmp-firmware/pm
```

Set Requirement

Set the power requirement on the node.

```
$ echo pm_set_requirement <node> <capabilities> > /sys/kernel/debug/zynqmp-firmware/pm
```

Set Max Latency

Set the maximum wake-up latency requirement for a node.

```
$ echo pm_set_max_latency <node> <latency> > /sys/kernel/debug/zynqmp-firmware/pm
```

Get Node Status

Get status information of a node. (Any PU can check the status of any node, regardless of the node assignment.)

```
$ echo pm_get_node_status <node> > /sys/kernel/debug/zynqmp-firmware/pm
```

Get Operating Characteristic

Get operating characteristic information of a node.

```
$ echo pm_get_operating_characteristic <node> > /sys/kernel/debug/zynqmp-firmware/pm
```

Reset Assert

Assert/de-assert on specific reset lines.

```
$ echo pm_reset_assert <reset> <action> > /sys/kernel/debug/zynqmp-firmware/pm
```

Reset Get Status

Get the status of the reset line.

```
$ echo pm_reset_get_status <reset> > /sys/kernel/debug/zynqmp-firmware/pm
```

Get Chip ID

Get the chip ID.

```
$ echo pm_get_chipid > /sys/kernel/debug/zynqmp-firmware/pm
```

Get pin control functions

Get current selected function for given pin.

```
$ echo pm_pinctrl_get_function <pin-number> > /sys/kernel/debug/zynqmp-firmware/pm
```

Set pin control functions

Set requested function for given pin.

```
$ echo pm_pinctrl_set_function <pin-number> <function-id> >
/sys/kernel/debug/zynqmp-firmware/pm
```

Get configuration parameters for the pin

Get value of requested configuration parameter for given pin.

```
$ echo pm_pinctrl_config_param_get <pin-number> <parameter to get> >
/sys/kernel/debug/zynqmp-firmware/pm
```

Set configuration parameters for the pin

Set value of requested configuration parameter for given pin.

```
$ echo pm_pinctrl_config_param_set <pin-number> <parameter to set> <param value> >
/sys/kernel/debug/zynqmp-firmware/pm
```

Control device and configurations

Control device and configurations and get configurations values.

```
$ echo pm_ioctl <node id> <ioctl id> <arg1> <arg2> >
/sys/kernel/debug/zynqmp-firmware/pm
```

Query Data

Request data from firmware.

```
$ echo pm_query_data <query id> <arg1> <arg2> <arg3> >
/sys/kernel/debug/zynqmp-firmware/pm
```

Enable Clock

Enable the clock for a given clock node id.

```
$ echo pm_clock_enable <clock id> > /sys/kernel/debug/zynqmp-firmware/pm
```

Disable Clock

Disable the clock for a given clock node id.

```
$ echo pm_clock_disable <clock id> > /sys/kernel/debug/zynqmp-firmware/pm
```

Get Clock State

Get the state of clock for a given clock node id.

```
$ echo pm_clock_getstate <clock id> > /sys/kernel/debug/zynqmp-firmware/pm
```

Set Clock Divider

Set the divider value of clock for a given clock node id.

```
$ echo pm_clock_setdivider <clock id> <divider value> >
/sys/kernel/debug/zynqmp-firmware/pm
```

Get Clock Divider

Get the divider value of clock for a given clock node id.

```
$ echo pm_clock_getdivider <clock id> > /sys/kernel/debug/zynqmp-firmware/pm
```

Set Clock Rate

Set the clock rate for a given clock node id.

```
$ echo pm_clock_setrate <clock id> <clock rate> >
/sys/kernel/debug/zynqmp-firmware/pm
```

Get Clock Rate

Get the clock rate for a given clock node id.

```
$ echo pm_clock_getrate <clock id> > /sys/kernel/debug/zynqmp-firmware/pm
```

Set Clock Parent

Set the parent clock for a given clock node id.

```
$ echo pm_clock_setparent <clock id> <parent clock id> >
/sys/kernel/debug/zynqmp-firmware/pm
```

Get Clock Parent

Get the parent clock for a given clock node id.

```
$ echo pm_clock_getparent <clock id> > /sys/kernel/debug/zynqmp-firmware/pm
```

Note: Clock id definitions are available in the following txt file of the clock bindings documentation: Documentation/devicetree/bindings/clock/xlnx,zynqmp-clk.txt

PM Platform Driver

The Zynq UltraScale+ MPSoC power management for Linux is encapsulated in a power management driver, power domain driver and platform firmware driver. The system-level API functions are exported and as such, can be called by other Linux modules with GPL compatible license. The function declarations are available in the following location:

`include/linux/firmware/xilinx/zynqmp/firmware.h`

The function implementations are available in the following location:

`drivers/firmware/xilinx/zynqmp/firmware*.c`

Provide the correct node in the Linux device tree for proper driver initialization. The firmware driver relies on the 'firmware' node to detect the presence of PMU firmware, determine the calling method (either 'smc' or 'hvc') to the PM-Framework firmware layer and to register the callback interrupt number.

The 'firmware' node contains following properties:

- Compatible: Must contain 'xlnx,zynqmp-firmware'
- Method: The method of calling the PM framework firmware. It should be 'smc'.

Note: Additional information is available in the following txt file of Linux Documentation: `Documentation/devicetree/bindings/firmware/xilinx/xlnx,zynqmp-firmware.txt`

Example:

```
firmware {
    zynqmp_firmware: zynqmp-firmware {
        compatible = "xlnx,zynqmp-firmware";
        method = "smc";
    };
};
```

Note: power domain driver and power management driver binding details are available in the following files of Linux Documentation:

`Documentation/devicetree/bindings/soc/xilinx/xlnx,zynqmp-power.txt`

`Documentation/devicetree/bindings/power/zynqmp-genpd.txt`

Note: xilPM do not support the following EEMI APIs. For current release, they are only supported for linux through ATF.

- query_data
- ioctl
- clock_enable
- clock_disable
- clock_getstate
- clock_setdivider

- clock_getdivider
- clock_setrate
- clock_getrate
- clock_setparent
- clock_getparent
- pinctrl_request
- pinctrl_release
- pinctrl_set_function
- pinctrl_get_function
- pinctrl_set_config
- pinctrl_get_config

Arm Trusted Firmware (ATF)

The Arm Trusted Firmware (ATF) executes in EL3. It supports the EEMI API for managing the power state of the slave nodes, by sending PM requests through the IPI-based communication to the PMU.

ATF Application Binary Interface

All APU executable layers below EL3 may indirectly communicate with the PMU via the ATF. The ATF receives all calls made from the lower ELs, consolidates all requests and send the requests to the PMU.

Following Arm's SMC Calling Convention, the PM communication from the non-secure world to the ATF is organized as SiP Service Calls, using a predefined SMC function identifier and SMC sub-range ownership as specified by the calling convention.

Note that the EEMI API implementation for the APU is compliant with the SMC64 calling convention only.

EEMI API calls made from the OS or hypervisor software level pass the 32-bit API ID as the SMC Function Identifier, and up to four 32-bit arguments as well. As all PM arguments are 32-bit values, pairs of two are combined into one 64 bit value.

The ATF returns up to five 32-bit return values:

- Return status, either success or error and reason
- Additional information from the PM controller

Checking the API version

Before using the EEMI API to manage the slave nodes, the user must check that EEMI API version implemented in the ATF matches the version implemented in the PMU firmware. EEMI API version is 32 bit value separated in higher 16 bits of MAJOR and lower 16 bits of MINOR part. Both fields must be the same between the ATF and the PMU firmware.

How to check EEMI API version

The EEMI version implemented in the ATF is defined in the local EEMI_API_VERSION flag. The rich OS may invoke the **PM_GET_API_VERSION** function to retrieve the EEMI API version from the PMU. If the versions are different, this call will report an error.

Note: This EEMI API call is version independent; every EEMI version implements it.

Checking the Chip ID

Linux or other rich OS can invoke the PM_GET_CHIPID function via SMC to retrieve the chip ID information from the PMU.

The return values are:

1. CSU idcode register (see TRM).
2. CSU version register (see TRM).

For more details, see the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

PSCI

Power State Coordination Interface is a standard interface for controlling the system power state of Arm processors, such as suspend, shutdown, and reboot. For the PSCI specifications, see <http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.den0022c/index.html>.

ATF handles the PSCI requests from Linux. ATF supports PSCI v0.2 only (with no backward compatible support for v0.1).

The Linux kernel comes with standard support for PSCI. For information regarding the binding between the kernel and the ATF/PSCI, see <https://www.kernel.org/doc/Documentation/devicetree/bindings/arm/psci.txt>.

Table 11-3: PSCI v0.2 Functions Supported by the ATF

Functions	Description	Supported
PSCI Version	Return the version of PSCI implemented.	Yes
CPU Suspend	Suspend execution on a core or higher level topology node. This call is intended for use in idle subsystems where the core is expected to return to execution through a wakeup event.	Yes
CPU On	Power up a core. This call is used to power up cores that either: • Have not yet been booted into the calling supervisory software. • Have been previously powered down with a CPU_OFF call.	Yes
CPU Off	Power down the calling core. This call is intended for use in hotplug. A core that is powered down by CPU_OFF can only be powered up again in response to a CPU_ON.	Yes
Affinity Info	Enable the caller to request status of an affinity instance.	Yes
Migrate (Optional)	This is used to ask a uniprocessor Trusted OS to migrate its context to a specific core.	Yes
Migrate Info Type (Optional)	This function allows a caller to identify the level of multicore support present in the Trusted OS.	Yes
Migrate Info Up CPU (Optional)	For a uniprocessor Trusted OS, this function returns the current resident core.	Yes
System Off	Shut down the system.	Yes
System Reset	Reset the system.	Yes
PSCI Features	Introduced in PSCI v1.0. Query API that allows discovering whether a specific PSCI function is implemented and its features.	Yes
CPU Freeze (Optional)	Introduced in PSCI v1.0. Places the core into an IMPLEMENTATION DEFINED low-power state. Unlike CPU_OFF it is still valid for interrupts to be targeted to the core. However, the core must remain in the low power state until it a CPU_ON command is issued for it.	No
CPU Default Suspend (Optional)	Introduced in PSCI v1.0. Will place a core into an IMPLEMENTATION DEFINED low-power state. Unlike CPU_SUSPEND the caller need not specify a power state parameter.	No
Node HW State (Optional)	Introduced in PSCI v1.0. This function is intended to return the true HW state of a node in the power domain topology of the system.	Yes
System Suspend (Optional)	Introduced in PSCI v1.0. Used to implement suspend to RAM. The semantics are equivalent to a CPU_SUSPEND to the deepest low-power state.	Yes
PSCI Set Suspend Mode (Optional)	Introduced in PSCI v1.0. This function allows setting the mode used by CPU_SUSPEND to coordinate power states.	No

Table 11-3: PSCI v0.2 Functions Supported by the ATF (Cont'd)

Functions	Description	Supported
PSCI Stat Residency (Optional)	Introduced in PSCI v1.0. Returns the amount of time the platform has spent in the given power state since cold boot.	Yes
PSCI Stat Count (Optional)	Introduced in PSCI v1.0. Return the number of times the platform has used the given power state since cold boot.	Yes

PMU firmware

The EEMI service handlers are implemented in the PMU firmware, as one of the modules called PM Controller (There are other modules running in the PMU firmware to handle other types of services). For more details, see the [Chapter 10, Platform Management Unit Firmware](#).

Power Management Events

The PM Controller is event-driven, and all of the operations are triggered by one of the following events:

- EEMI API events triggered via IPI0 interrupt.
- Wake events triggered via GPI1 interrupt.
- Sleep events triggered via GPI2 interrupt.
- Timer event triggered via PIT2 interrupt.

EEMI API Events

EEMI API events are software-generated events. The events are triggered via IPI interrupt when a PM master initiates an EEMI API call to the PMU. The PM Controller handles the EEMI request and may send back an acknowledgment (if one is requested.) An EEMI request often triggers a change in the power state of a node or a master, with some exceptions.

Wake Events

Wake events are hardware-generated events. They are triggered by a peripheral signaling that a PM master should be woken-up. All wake events are triggered via the GPI1 interrupt.

The following wake events are supported by the PM controller:

- GIC wake events which signal that a CPU shall be woken up due to an interrupt triggered by a hardware resource to the associated GIC interface. The following GIC wake events are supported:
 - APU[3:0]An event for each APU processor
 - RPU[1:0]An event for each RPU processor
- FPD wake event directed by the GIC Proxy. This wake event is triggered when any of the wake sources enabled prior to suspending. The purpose of this event is to trigger a wake-up of APU master when FPD is powered down. If FPD is not powered down, none of the wake signals would propagate through FPD wake. Instead, the wake would propagate through GIC wake if the associated interrupt at the GIC is properly enabled. All wake events targeted to the RPU propagate via the associated GIC wake.

Sleep Events

Sleep events are software-generated events. The events are triggered by a CPU after it finalizes the suspend procedure with the aim to signal to the PMU that it is ready to be put in a low power state. All sleep events are triggered via GPI2 interrupt.

The following sleep events are supported:

- APU[3:0]An event for each APU processor
- RPU[1:0]An event for each RPU processor

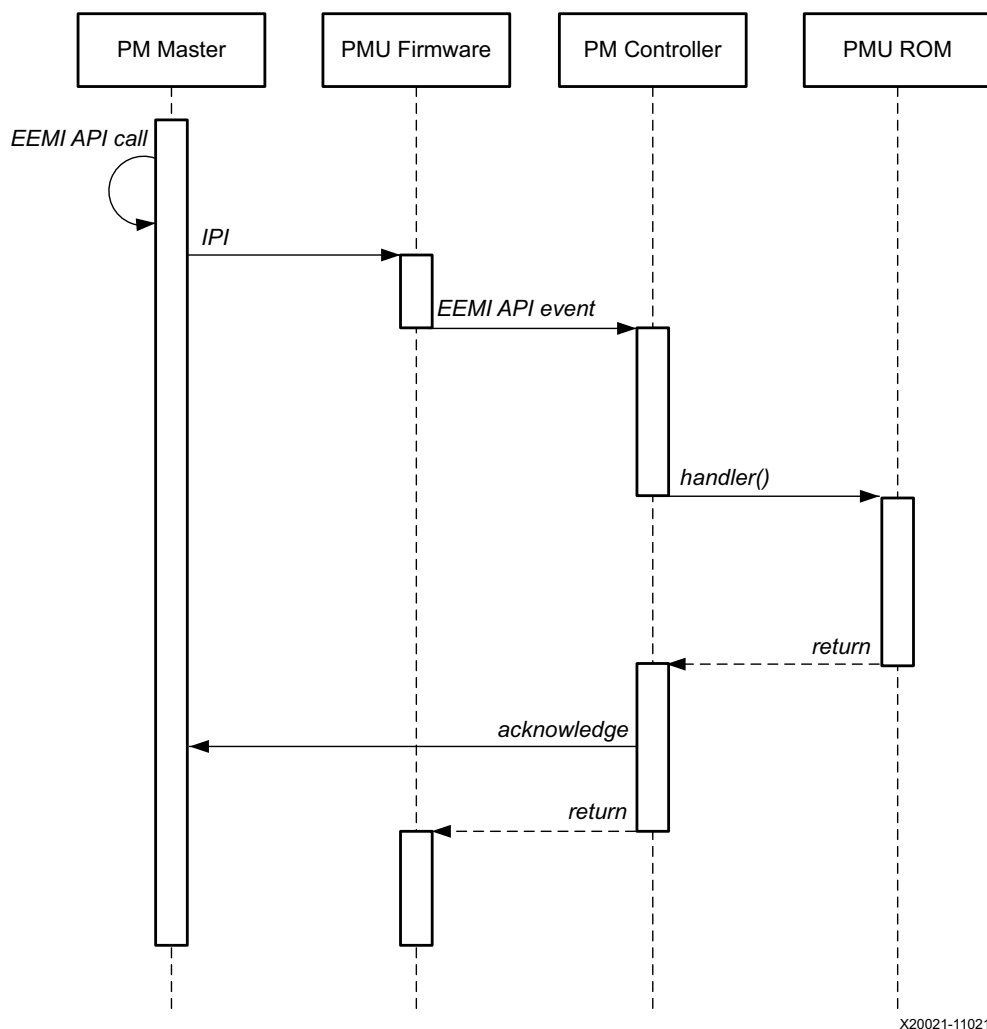
When the PM controller PM Controller receives the sleep event for a particular CPU, the CPU is put into a low power state.

Timer Event

Timer event is hardware-generated event. It is triggered by a hardware timer when a period of time expires. The event is used for power management timeout accounting and it is triggered via PIT2 interrupt.

General flow of an EEMI API Call

The following diagram illustrates the sequence diagram of a typical API call, starting with the call initiated by a PM Master (such as another PU):



X20021-110217

Figure 11-13: EEMI API Call Sequence Diagram

The previous diagram shows four actors, where the first one represents the PM Master, i.e. either the RPU, APU, or a MicroBlaze™ processor core. The remaining 3 actors are the different software layers of the PMU.

First the PMU firmware receives the IPI interrupt. Once the interrupt has been identified as a power management related interrupt, the IPI arguments are passed to the Power Management Module. The PM controller then processes the API call. If necessary it may call the PMU ROM in order to perform power management actions, such as power on or off a power island, or a power domain.

Reset

Introduction

The Zynq[®] UltraScale+[™] MPSoC device reset block is responsible for handling both internal and external reset inputs to the system, and to meet the reset requirements for all the peripherals and the APU and RPU. The reset block generates resets for the programmable logic part of the device, and allows independent reset assertion for PS and PL blocks.

This chapter explains the reset mechanisms involved in the system reset and the individual module resets.

System-Level Reset

The Zynq UltraScale+ MPSoC devices let you reset individual blocks such as the APU, RPU, or even individual power domains like the FPD and LPD. There are multiple, system-level reset options, as follows:

- Power-on reset (POR)
- System reset (**SRST_B**)
- Debug system reset

For more details on the system-level reset flow, see this [link](#) to the “Reset System” chapter in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

Block-Level Resets

The PS-only reset can be implemented as a subset of system-reset; however, the user must provide software that ensures PS-to-PS AXI transactions are gracefully terminated before initiating a PS-only reset.

PS-Only Reset

The PS-only reset re-boots the PS while that PL remains active. You can trigger the PS-only reset by hardware error signal(s) or a software register write. If the PS-only reset is due to an error signal, then the error can be indicated to the PL also, so that the PL can prepare for the PR restart.

The PS-only reset sequence can be implemented as follows:

- [ErrorLogic] Error interrupt is asserted whose action requires PS-only reset. This request is sent to PMU as an interrupt.
- [PMU-FW] Set PMU Error (= > PS-only reset) to indicate to PL.

See the *PS Only Reset* section in the "Reset System" chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11] describes the PS-only reset sequence.

Application Processing Unit Reset

You can independently reset each of the APU CPU core in the software.

The APU MPCore reset can be triggered by FPD, WDT, or a software register write; however, APU MPCore is reset without gracefully terminating requests to and from the APU. The intent is that you use the FPD in case of catastrophic failures in the FPD. The APU reset is primarily for software debug.

The *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11] describes the APU reset sequence.

APU-Only Reset

APU-only reset is supported in qspi24, qspi32, sd0, sd1, sd-ls boot modes. However, APU-only reset is not supported in qspi24 mode on systems with a flash size that is greater than 16 MB.

Real Time Processing Unit Reset

Each Cortex™-R5F core can be independently reset. In lockstep mode, only the `Cortex-R5F_0` needs to be reset to reset both Cortex-R5F cores. It can be triggered by errors or a software register write. The Cortex-R5F reset can be triggered due to a lockstep error to be able to reset and restart the RPU.

It needs to gracefully terminate Cortex-R5F ingress and egress transactions before initiating reset of corresponding Cortex-R5F.

Full Power Domain Reset

The FPD-reset resets all of the FPD power domain and can be triggered by errors or a software register write. If the FPD reset is due to error signal, then the error must be indicated to both the LPD and the PL.

The FPD reset can be implemented by leveraging the FPD power-up sequence; however, it needs to gracefully terminate FPD ingress and egress AXI transactions before initiating reset of FPD. FPD reset sequence can be PL Reset.

The Zynq UltraScale+ MPSoC devices has general-purpose output pins from the PMU block that can be used to reset the blocks in PL. Additionally, GPIO using the EMIO interface can also be used to reset PL logic blocks. For a detailed description of the reset flow, see the [this link](#) to the “Reset System” chapter in the *Zynq UltraScale+ MPSoC Technical Reference Manual* [Ref 11].

For more information on the software APIs for reset, see the [PMU firmware](#) in [Chapter 9, Platform Management](#).

Warm Restart

Zynq UltraScale+ MPSoC is a highly complex piece of silicon, capable of running multiple subsystems on the chip simultaneously. As such, Zynq UltraScale+ supports various types of reset. This varies from the simplest system reset to the much more complicated subsystem restart. In any system or subsystem that has a processor component and a programmable logic component, reset must entail both reset to the hardware as well as software. Reset to the hardware includes the following:

- Resetting of the processor and all peripherals associated with the system/subsystem
- Cleaning up of the memory as needed
- Making sure that the interconnect is in a clean state that is capable of routing traffic.

Reset to the software results in the processor starting from the reset vector. However, designer must make sure that a valid and clean code for the system/subsystem is located at the reset vector in order to bring the system back to a clean running state.

Resets for Zynq UltraScale+ are broadly divided into two categories. They are:

- Full system resets
- Subsystem restarts

Full system resets include the following:

- Power-On-Reset (POR)
- System-reset
- PS-only-reset

Subsystem restarts include APU subsystems and RPU subsystem restarts.

Full system resets are quite straight forward. Hardware is brought back to the reset state and software starts executing ROM code, with a minor behavior difference between the reset types. There are subtleties to PS-only reset which will be discussed in later sections.

Subsystem restart is more complicated. A subsystem in Zynq UltraScale+ is composed of all the components of a particular operating system. [Figure 12-1](#) shows both Vivado's view of the PS as well as example subsystems as defined by the OS. The default IP configuration menu in Vivado provides a flattened view, consisting of all available PS components. In the example, these components are partitioned into three separate subsystems, each running an independent operating system. Each subsystem consists of a processor, list of peripherals and memory. The example shows the following subsystems:

- RPU based subsystem running uC/OS-II
- RPU based subsystem running FreeRTOS
- APU based subsystem running Linux

Subsystems can be configured in the Isolation Configuration view that is inside the Vivado PCW (PS Configuration Wizard), when the Advanced Mode check box is enabled.

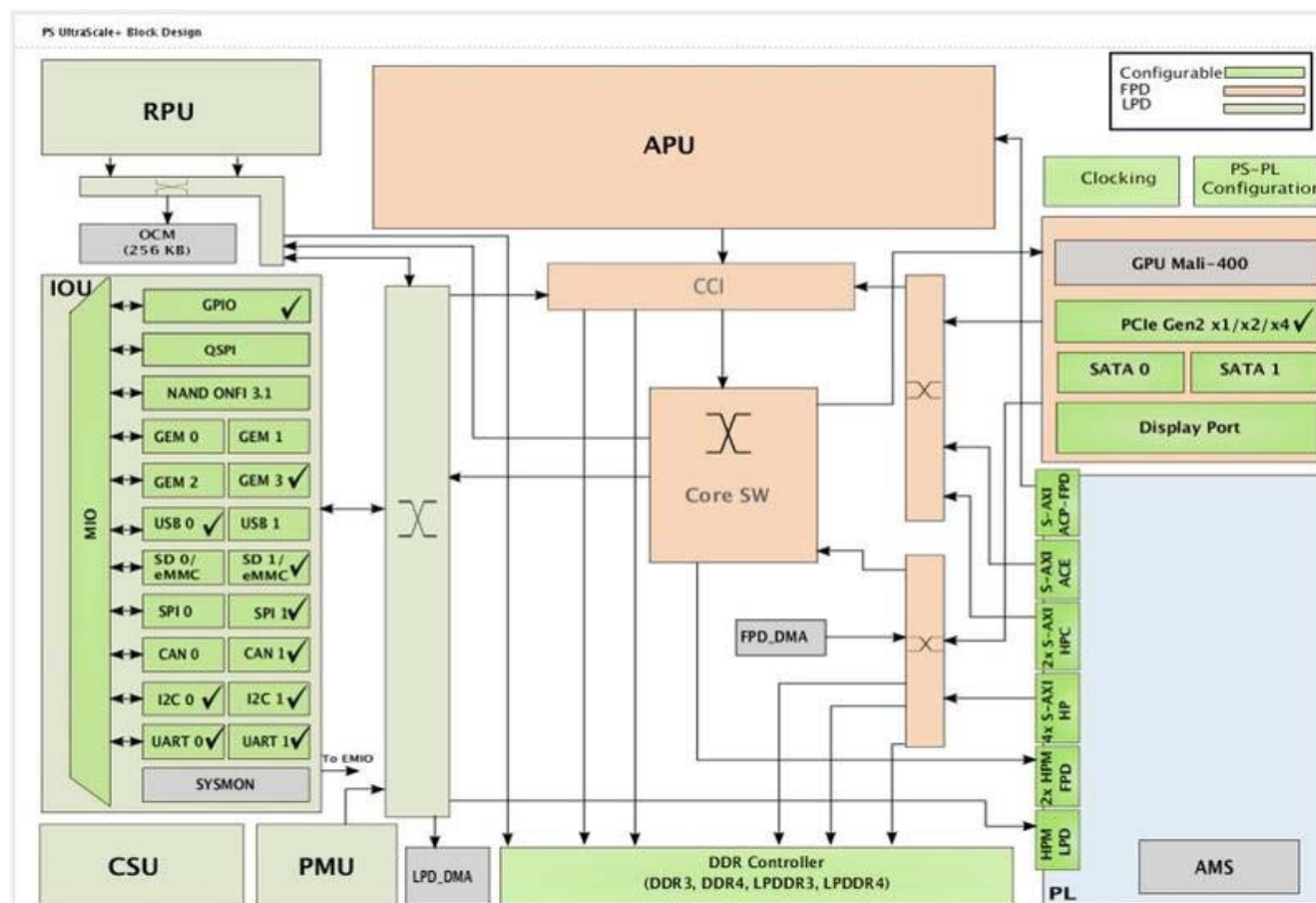


Figure 12-1: Vivado IP Configuration Menu

During subsystem restart, the entire subsystem is restarted from a clean state without affecting the running of the other active subsystems defined in MPSoC. For example, during an APU subsystem restart, an APU subsystem running Linux is restarted as far back as FSBL, while the RPU subsystem running FreeRTOS and uC/OS-II continues to function undisturbed. Similarly for a RPU subsystem restart, an APU subsystem continues to function undisturbed.

Subsystem restarts are managed by the platform management unit (PMU). To restart each subsystem, PMU must first ensure that all on-going AXI-transactions are terminated and that no new transactions are issued. In the subsystems shown in Figure 12-2, the interconnects, which connects the components of the subsystem, are not explicitly shown. However, each subsystem includes multiple interconnects and the same interconnects are used by all three subsystems. If the PMU firmware resets all the components in a subsystem while leaving unfinished transactions in the interconnect, the AXI master and slave might both be in the reset state. However, the unfinished AXI transactions will remain in the interconnect, thus blocking all subsequent traffic. Stuck transactions in the interconnect causes the system to freeze as these connections are shared. It is therefore imperative that the PMU ensures all transactions are completely finished before resetting each and every components in the subsystem, including the processor.

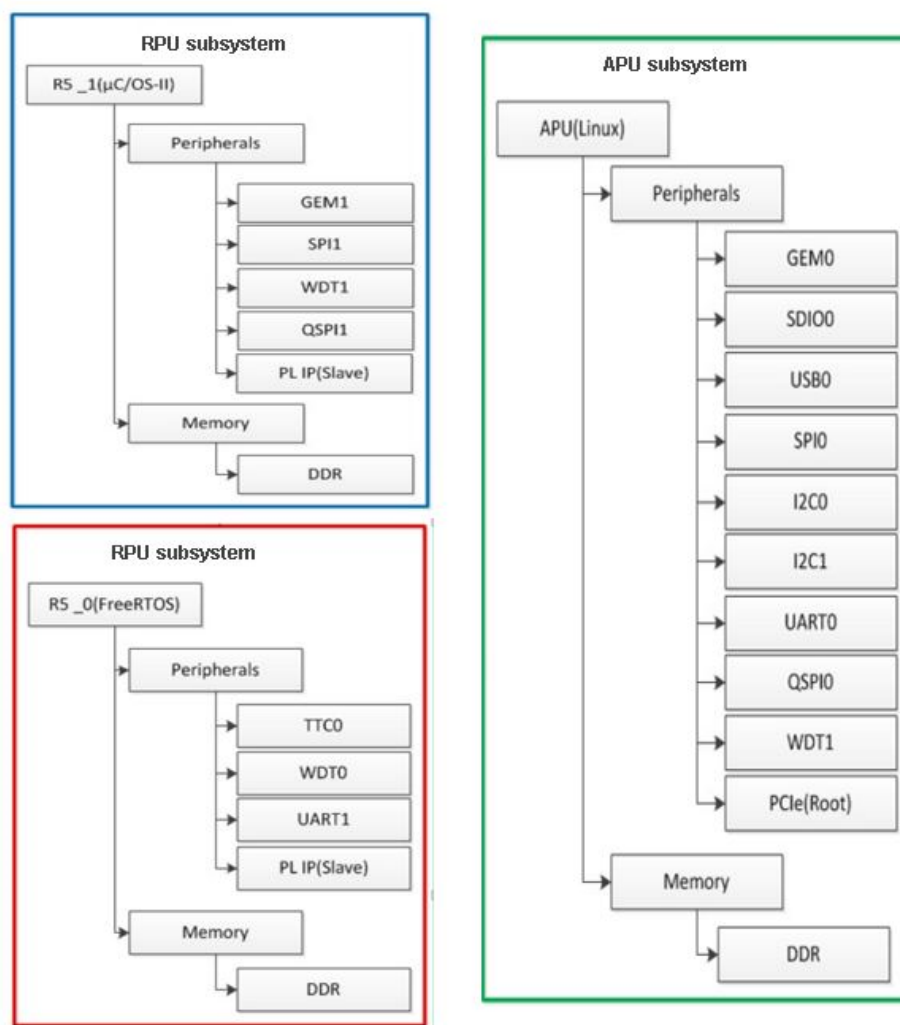


Figure 12-2: Subsystem Components for Various Operating Systems

Before releasing the processor from reset, the PMU must ensure that the code in the reset vector will result in a clean system restart. In the case of the RPU subsystem running standalone applications, this means either loading a clean copy of the application elf or making sure that the application code is re-entrant. In the case of the APU subsystem running Linux, this means starting from a re-entrant copy of FSBL.

Note: The on-chip memory (OCM) module contains 256 KB of RAM starting at `0xFFFFC000`. The OCM is mainly used by the FSBL and ATF components. The FSBL uses the OCM region from `0xFFFFC000` to `0xFFFFE9FFF`. The last 512B of this region is used by the FSBL to share the handoff parameters corresponding to applications that the ATF hands off. The ATF uses the rest of the OCM i.e. from `0xFFFFEA000` to `0xFFFFFFFF`.

The current implementation of a warm reset requires the FSBL to be in the OCM to support the PMU firmware hand off to (already existing) the FSBL without actually restarting. Hence, the OCM is completely used and no other application is allowed to use it when a warm restart is enabled.

Supported Use Cases

APU Subsystem Restart

For an APU subsystem only restart, you must define the APU subsystem using PCW in the Vivado design tools. The PMU executes the function to restart the APU subsystem. First, the PMU idles all components in the APU subsystem. When all is quiet, the PMU will reset each component, including the APU processors. When the reset is released, it will re-execute the FSBL code in the OCM. The task carried out by the FSBL for restart differs only slightly than that of the POR.

Note: The FSBL is re-entrant. Hence, the APU can simply re-execute the FSBL without having to reload a clean copy.

Figure 12-3 shows the APU subsystem restart process.

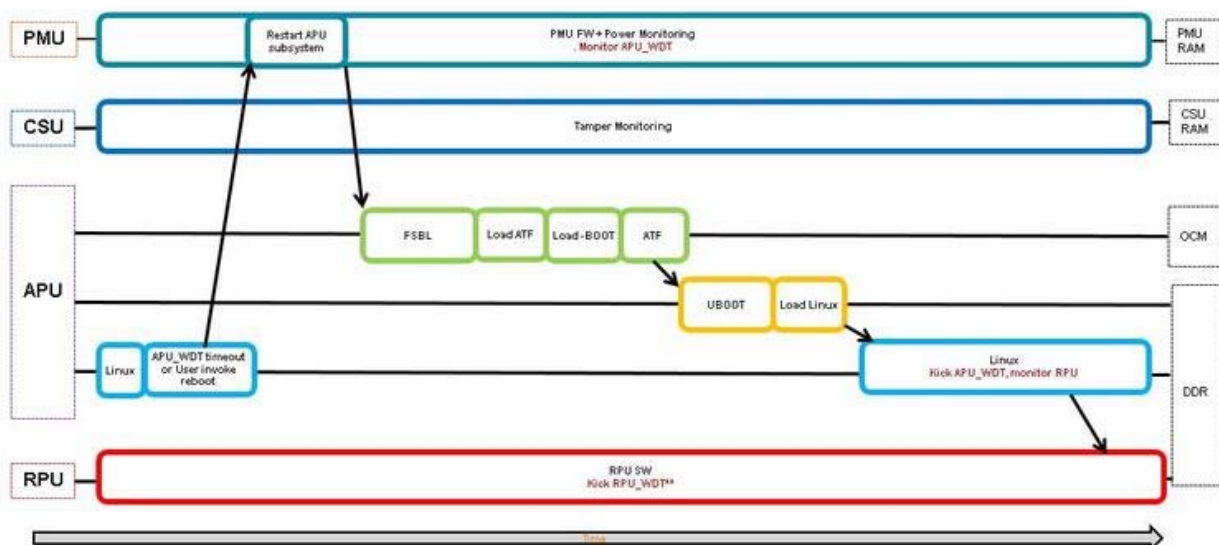


Figure 12-3: APU Subsystem Restart Process

The start of this flow diagram represents a clean running state. Linux, RPU, PMU, and CSU subsystems are in running status. The health of the APU subsystem is monitored by an APU WDT (watchdog timer). Linux runs a background application which periodically boosts the watchdog to prevent it from timing out. If an APU subsystem hangs, the WDT times out. The timeout interrupts the PMU and results in an APU subsystem restart. Alternatively, you can invoke the APU subsystem restart by directly calling for it in Linux.

Implementation

To support any subsystem restart, a subsystem must first be defined in the Vivado design tools using the Isolation Configuration view.

For an APU subsystem running Linux, the following APU subsystem are required in addition to the default PMU subsystem:

1. A secure APU system for running the FSBL and ATF
2. A non-secure APU subsystem for running Linux.

See [Isolation Configuration](#) for more information on subsystem configuration and an example of the APU only subsystem.



IMPORTANT: While APU subsystem consists solely of PS components, it is often the case that APU subsystem also includes IP peripherals implemented in PL. Unfortunately, isolation configuration menu does not include features to assign PL IPs to different subsystems. As a result, all IPs instantiated in Vivado are added to the generated device tree source (DTS) file. In order to properly define the APU subsystem, all PL IPs that do not belong in the APU subsystem need to be manually removed from the DTS file. Otherwise, drivers for all the soft IPs will be enabled for Linux, and APU will attempt to manage all the soft IPs even when the APU is going through a warm restart.



IMPORTANT: During a subsystem restart, all components in the subsystem must be in the idle state, followed by reset. This is implemented for supported components in the PS. For all IPs in PL of a subsystem that are AXI slaves, no additional tasks are required to idle them. You may supply code to reset these slaves if desired. For PL IPs that are AXI masters, you must provide the necessary code to stop and complete all AXI transactions from the master as well as to reset it. See *Idle and Reset of Peripherals* for details on adding the idle and reset code.

See GPIO Reset to PL for design issue and guidelines pertaining to using `resetn` signal from PS to PL (`ps_resetn`). You can optionally enable the recovery and escalation features as desired. Building Software for detailed instructions on building the software.

RPU Subsystem Restart

RPU subsystem restart requires the APU subsystem and one or more RPU subsystems running in lock-step or split mode. The APU subsystem running Linux is the master of the RPU subsystems and manages the life cycle of the subsystem using the `remoteproc` feature of OpenAMP. APU uses `remoteproc` to load, start, and stop the RPU application. It also re-syncs the APU subsystem with RPU subsystem after the restart. APU subsystem can trigger a RPU restart by following sequence:

1. First, it stops the RPU
2. Loads the new firmware
3. Then, it starts the RPU again.

Many events including user command, RPU watchdog timeout or message from the RPU to APU via message pipe may trigger the RPU subsystem restart. Then, APU issues `remoteproc` command to PMU to start or stop the RPU, and the PMU changes the state of the RPU subsystem.

Figure 12-4 shows the RPU subsystem restart process.

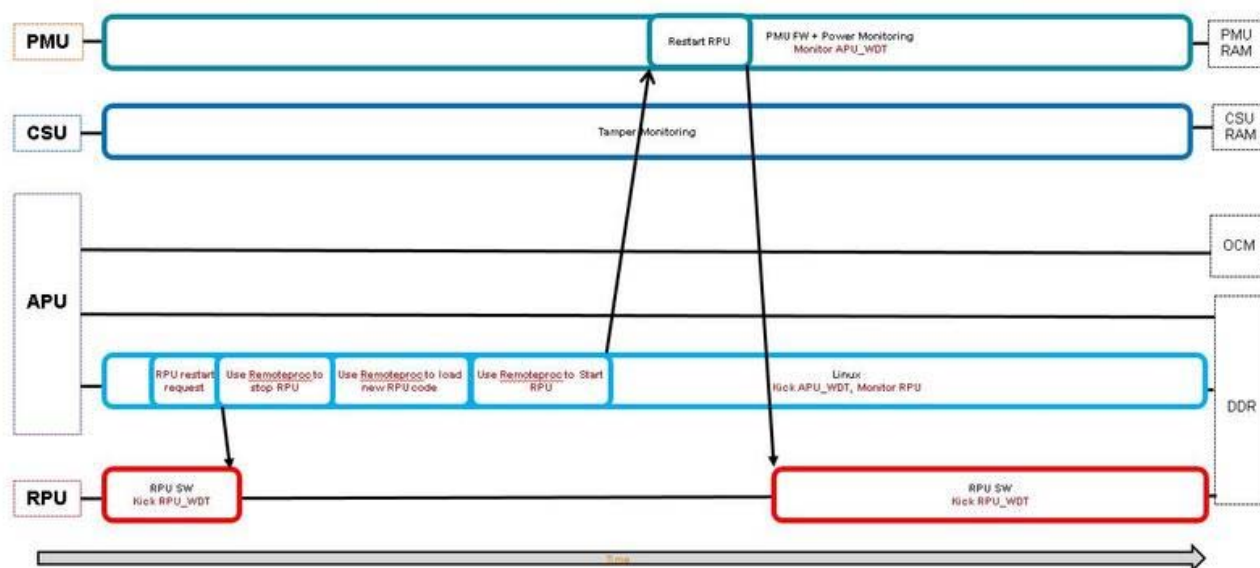


Figure 12-4: RPU Subsystem Restart

The start of the above diagram represents a clean running state for all subsystems, Linux, RPU, PMU and CSU. In the flowchart, APU receives a RPU subsystem restart request. When APU receives the restart request, it uses `remoteproc` features to stop the RPU subsystem, load new firmware code, and then starts the RPU subsystem again. The flow chart shows the use of a RPU WDT. The RPU periodically boosts the watch dog. If the RPU hangs, WDT times out. Linux will receive the timeout and restarts the RPU subsystem.

Implementation

You must define the RPU subsystem using the Isolation Configuration view in Vivado PCW, and both PMU and APU subsystems are required. In addition, two configurations are possible for the RPU subsystem: RPUs in lock step mode or in split mode. See the [Isolation Configuration Consideration wiki page](#) for more information on subsystem configuration. Sharing of peripherals between subsystems are not supported. Make sure that the peripherals in all subsystems are mutually exclusive.



IMPORTANT: *In the process of subsystem restart, all components in the subsystem must be in the idle state, followed by reset. This is implemented for supported components in the PS. For all IPs in PL of a subsystem that are AXI slaves, no additional tasks are required to idle them. User may supply code to reset the slaves if desired. For PL IPs that are AXI masters, user must provide the necessary code to stop and complete all AXI transactions from the master as well as to reset it. See *Idle and Reset of Peripherals* for details on adding the idle and reset code.*

RPU subsystem restart is supported with Linux kernel implementation of `remoteproc` on APU in conjunction with OpenAMP library on RPU. It is currently not supported with Linux userspace OpenAMP library on APU. RPU application must be written in accordance with the OpenAMP application requirements. See *Libmetal and OpenAMP User Guide* (UG1186) [Ref 16] for more information. Note that the `rpmsg` is not required for `remoteproc`. You can employ `rpmsg` feature to provide a communication pipe between the two processors. However, `remoteproc` is independent of `rpmsg`. To make `remoteproc` function properly with subsystem restart, RPU application needs to include a resource table with static shared memory allocation. Dynamic shared memory allocation is not supported for subsystem restart. You must implement the steps outlined in How to Write a Simple OpenAMP Application in *Libmetal and OpenAMP User Guide* (UG1186) [Ref 16] to satisfy the `remoteproc` requirement, but not beyond that.

after initialization, RPU application needs to signal to the PMU that it is Power Management (PM) aware by calling `XPm_InitFinalize()`.

Note: If you call `XPm_InitFinalize()` too early, then the slaves that are not yet initialized are powered off. They will be powered up again when the RPU application comes around to initialize them, which will incur some additional power-up latency. See [ZU+ Example - PM Hello World wiki page](#) for more information on how to write a PM aware RPU application.

Finally, you must ensure that the address of the reserved memory for RPU code is synchronized across all layers. It must be defined under memory for both APU and RPU subsystems in the isolation configuration of Vivado. The same address region should be used in the DTS file for OpenAMP overlay in Linux and again, in resource table and linker script for the RPU application.

See GPIO Reset to PL for design issue and guidelines pertaining to using `resetn` signal from PS to PL (`ps_resetn`). You can optionally enable the recovery and escalation features as desired. Building Software for detailed instructions on building the software.

PS-Only Reset

For a PS-only restart, the entire processor system is reset while PL continues to function. Prior to invoking PS-only reset, PMU turns on isolation between PS and PL, thus clamping the signals between them in well-defined states. After PS-only reset is released, PS executes the standard boot process starting from the PMU ROM, followed by CSU ROM, then FSBL and so on. During FSBL, the isolation between PS and PL is removed.



IMPORTANT: As the software has gone through a reset cycle, the state of the hardware IPs in PL which continue to run during the PS-only reset may become out of sync with the state of the software which interfaces or controls the IPs. It is your responsibility to make sure that the software and hardware states are properly re-synchronized. In a PS-only reset, you cannot download the bitstream again.

PS-only reset can be initiated by Linux command or watchdog timeout or PMU error management block.

If you are interested in PS-only reset without APU/RPU subsystem restart, subsystem/isolation configuration is not required. Linux commands for setting reboot type and reboot will work without additional modifications.

System Reset

In a system-reset, the entire hardware, both PS and PL are reset. After system reset is released, PS executes the standard boot process starting from the PMU ROM, followed by CSU ROM, then FSBL and so on. The following table shows the differences between system reset and POR:

Table 12-1: Differences between POR and System Reset

POR	System Reset
Reset persistent registers	Preserves persistent registers
Resamples boot mode pins	Does not resample boot mode pins
Reset debug states	Preserves debug states
Resample eFuse values	Requires explicit software action to refresh
Security state determined	Security state locked
Clear tamper response	Preserves tamper response
Select security key source	Security key source locked
Optional LBIST and/or SCAN/CLEAR	Does not run LBIST or SCAN/CLEAR
Run MBIST	Explicit software action needed to run MBIST

System reset can be initiated by Linux command or watchdog timeout or PMU error management block. If you are interested in only System reset without APU/RPU subsystem restart, subsystem/isolation configuration is not required.

Idle and Reset of Peripherals

It is necessary to stop/complete any ongoing transaction by any IP or processor of the subsystem before resetting them. Otherwise, it may lead to hanging of the interconnect and eventually hanging of the entire system. Also, to ensure proper operation by the IP after reboot, it is best to reset them and bring them to post bootrom state.

PMU firmware implements peripheral idling and resetting for the PS IPs that can be idled / reset during the subsystem reset. The IPs that will be attempted to idled/reset is based on isolation configuration of the Vivado.

Build PMU firmware with the following idling flags to enable subsystem node idling and resetting:

```
ENABLE_NODE_IDLING
IDLE_PERIPHERALS
```

Node Reset and Idle

During a subsystem restart, the PMU firmware makes sure that the associated PS peripheral nodes are idled and brought to reset state.

Following is the list of currently supported PS peripherals that will undergo idle/reset, if they are part of the subsystem that is undergoing reset:

- TTC
- Ethernet/EMAC
- I2C
- SD
- eMMC
- QSPI
- USB
- DP
- SATA

See GPIO reset to PL to understand the implication of GPIO reset.

Note: PS peripherals are idled prior to invoking resets for user invoked reboot of PS-only and system-reset command.

Custom Hooks

PMU firmware does not keep track of PL peripherals. Hence, there is no idle/reset function implementation available in the PMU firmware. However, it is necessary to treat those peripherals in the same the PS peripherals are treated. You can add a custom hook in the `idle_hooks.c` file to idle the PL peripherals and reset them. These hooks can be called from the `PmMasterIdleSlaves` function in the `pm_master.c` file of the PMU firmware.

```
lib/sw_apps/zynqmp_pmufw/src/pm_master.c
:dir:dir -769,6 +769,12 :dir:dir static void PmMasterIdleSlaves(PmMaster* const
master)

    PmDbg(DEBUG_DETAILED, "%s\r\n", PmStrNode(master->nid));

+    /*
+     * Custom hook to idle PL peripheral before PS peripheral idle
+     */
+
+    Xpfw_PL_Idle_HookBeforeSlaveIdle(master);
+
    while (NULL != req) {
        u32 usage = PmSlaveGetUsageStatus(req->slave, master);
        Node = &req->slave->node;
:dir:dir -783,6 +789,11 :dir:dir static void PmMasterIdleSlaves(PmMaster* const
master)
        }
        req = req->nextSlave;
    }

+    /*
+     * Custom hook to idle PL peripheral after PS peripheral idle
+     */
+    Xpfw_PL_Idle_HookAfterSlaveIdle(master);
#endif
}
```

The `Xpfw_PL_Idle_HookBeforeSlaveIdle` and `Xpfw_PL_Idle_HookAfterSlaveIdle` can contain the code to idle the PL peripherals and reset them if necessary. The implementation can be either of the following:

- Write AXI registers of PL IPs to bring them to idle state and reset. This is the preferred and a graceful way to idle PL peripherals.
- Implement a signal based handshake where PMU firmware signals PL to idle all PL IPs. This implementation should be used when there is no direct control to gracefully stop traffic. For example, you can use this implementation if there are non DMA PL IPs, which does not have reset control but are connected through a firewall IP. This implementation also allows stopping all traffic passing through it unlike the other where each IP needs to be idled individually.

Note: Implementation for these custom hooks is not provided by Xilinx.

GPIO Reset to PL

Vivado configuration allows you to enable fabric resets from PS to PL. Figure 12-5 shows that the Zynq UltraScale+ block outputs `p1_resetn0` and `p1_resetn1` signals with Fabric Reset Enabled and the Number of Fabric Resets set to 2, can be used to drive reset pins of PL components.

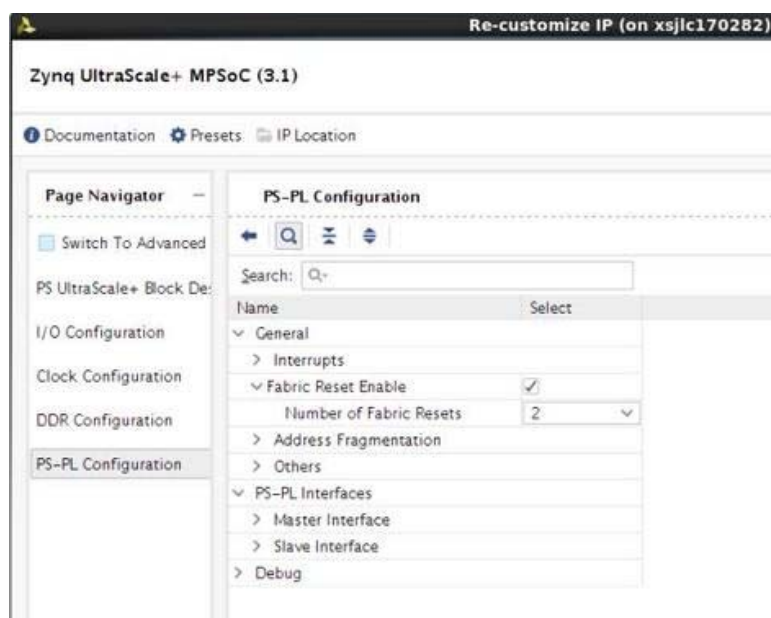


Figure 12-5: Resets from PS to PL

The `p1_resetn` signals are implemented with PS GPIOs. `p1_resetn` pins are released after bitstream configuration in software using the `psu_ps_p1_reset_config_data` function. In the case where a subsystem also uses GPIO for purpose other than reset, the GPIO block is included in the subsystem definition. The image below shows an example of an APU subsystem with GPIO as a slave peripheral.

Isolation Configuration

Please review **Known Limitations** under the **Isolation Configuration** Section of PG201.

☒ Enable Isolation ☐ Enable Secure Debug ☐ Lock Unused Memory

Search:

Name	Start Address	Size	Unit	TZ Settings
APU				
> Masters				
< Slaves				
> Memory				
< Peripherals				
CAN1	0xFF070000	64	KB	NonSecure
GEM3	0xFF0E0000	64	KB	NonSecure
I2C0	0xFF020000	64	KB	NonSecure
UART0	0xFF000000	64	KB	NonSecure
TTC0	0xFF110000	64	KB	NonSecure
TTC1	0xFF120000	64	KB	NonSecure
SWDT0	0xFF150000	64	KB	NonSecure
GPIO	0xFF0A0000	64	KB	NonSecure

Figure 12-6: APU Subsystem with GPIO

In the case where GPIO is a subsystem slave peripheral, the entire GPIO component will be reset as part of the restart process when the subsystem is being restarted. Since `p1_resetn` are implemented with GPIOs, `p1_resetn` will be forced low during subsystem restart. This behavior may be undesirable if the `p1_resetn` signals are being used to drive PL IPs in subsystems other than the one being reset. For example, if `p1_resetn0` drives resets to PL IP for APU subsystem and `p1_resetn1` drives PL IPs for RPU subsystem. During APU subsystem restart, both `p1_resetn0` and `p1_resetn1` will be forced into the reset state. Consequently, PL IPs in RPU subsystem will be reset. This is the wrong behavior since APU-restart should not affect the RPU subsystem as the GPIO is implicitly shared between the APU and RPU subsystem via `p1_resetn` signals. Since sharing of peripherals is not supported for subsystem restart, `p1_resetn` causes problems during subsystem reset. The work-around is to skip idling and resetting GPIO peripheral during any subsystem restart even if the component is assigned in the subsystem/isolation configuration.

To skip the GPIO reset during the node idling and reset, build the PMU firmware with following flag:

```
REMOVE_GPIO_FROM_NODE_RESET_INFO
```

Note: GPIO component goes through a reset cycle also during PS-only reset. PMU firmware enables PS-PL isolation prior to calling PS only reset which locks `p1_resetn` to High. However, as soon as FSBL removes the PS-PL isolation, the reset goes Low. FSBL then calls `psu_ps_p1_reset_config_data` to reconfigure `p1_resetn` back to High. This is needed since resetting the PL components allows proper synchronization of software and hardware states after reset.

Recovering from a Hang System

In an event of system hang, as indicated by FPT WDT timeout, PMU can be used to carry out a sequence of events to try and recover from the unresponsive condition. By default, when FPD WDT times out, PMU FW will not invoke any type of restart. This is so that user can specify the exact desired behavior. However, Xilinx provides a typical recovery scheme in which PMU firmware monitors the state of APU subsystem using FPD WDT and restart APU (Linux) subsystem if the timer expires, indicating problem with Linux.

Since RPU subsystem is managed by Linux using `remoteproc`, the life-cycle of the RPU subsystem is completely up to Linux. PMU is not involved in deciding when to restart RPU subsystem(s). RPU hang recovery can also be implemented with help of either software or hardware watchdog between APU and RPU subsystems. In that case, the watchdog is configured and handled by Linux but the heartbeats is provided by RPU application(s). The exact method of deciding when to restart RPU is up to the user, watchdog is simply one of many possibilities. To enable recovery, PMU firmware should be built with enabling error management and recovery. Following macros enable the Recovery feature:

```
ENABLE_EM
ENABLE_RECOVERY
```

It is also necessary to build ATF with following flags (see APU Idling for details):

```
ZYNQMP_WARM_RESTART=1
```



IMPORTANT: One TTC timer (timer 9) will be reserved for PMU's use when these compile flags are enabled.

Watchdog Management

The FPD WDT is used for monitoring APU state. Software running on APU periodically touch FPD WDT to keep it from timing out. The occurrence of WDT timeout indicates an unexpected condition on the APU which prevents the software from running properly and an APU restart is invoked. FPD WDT is configured by PMU firmware at initialization stage, but is periodically serviced by software running on APU.

The default timeout configured for WDT is 60 seconds and can be changed by `RECOVERY_TIMEOUT` flag in PMU firmware. When APU subsystem goes into a restart cycle, FPD WDT is kept running to ensure that the restart lands in a clean running state where software running on APU is able to touch the WDT again. Therefore, the timeout for the WDT must be long enough to cover the entire APU subsystem restart cycle to prevent the WDT from timing out in the middle of restart process. It is advisable to start providing the heartbeat as soon as is feasible in Linux. Petalinux's BSP includes recipe to add the watchdog management service in `init.d`. As FPD WDT is owned by PMU-firmware, it would be unsafe to use full fledged Linux driver for handling WDT. It is advisable to just pump the heartbeats by writing restart key (0x1999) to restart register (WDT base + 0x8) of the WDT. It can be done through C program daemon or it can be part of bash script daemon.

It is recommended to be part of idle thread or similar low priority thread, which if hangs we should consider the subsystem hang.

The following is the snippet of the single heartbeat stroke to the FPD WDT from command prompt. This can be included in the bash script which runs periodically.

```
# devmem 0xFD4D0008 32 0x1999
```

The following wdt-heartbeat application periodically provides the heartbeat to FPD WDT. For demo purpose this application is launched as daemon. The code from this application can be implemented in appropriate location such as Linux's Idle thread.

```
#include <stdio.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>

#define WDT_BASE                0xFD4D0000
#define WDT_RESET_OFFSET        0x8
#define WDT_RESET_KEY           0x1999

#define REG_WRITE(addr, off, val) (*(volatile unsigned int*)(addr+off)=(val))
#define REG_READ(addr,off) (*(volatile unsigned int*)(addr+off))

void wdt_heartbeat(void)
{
    char *virt_addr;
    int fd;
    int map_len = getpagesize();

    fd = open("/dev/mem", (O_RDWR | O_SYNC));

    virt_addr = mmap(NULL,
        map_len, PROT_READ|PROT_WRITE,
        MAP_SHARED,
        fd,
        WDT_BASE);

    if (virt_addr == MAP_FAILED)
        perror("mmap failed");

    close(fd);

    REG_WRITE(virt_addr,WDT_RESET_OFFSET, WDT_RESET_KEY);

    munmap((void *)virt_addr, map_len);
}

int main()
{
    while(1)
    {
        wdt_heartbeat();
        sleep(2);
    }
    return 0;
}
```

On the expiry of watchdog, PMU firmware receives and handles the WDT interrupt. PMU firmware idles the subsystem's master CPU i.e. all A53 cores (see APU Idling), and then carries out APU only restart flow which includes CPU reset and idling and resetting peripherals (see Peripheral Idling) associated to the subsystem reset.

Note: If ESCALATION is enabled PMU-firmware will trigger the appropriate restart flow (which can be other than APU only restart) as explained in Escalation section.

APU Idling

Each A53 is idled by taking them to the WFI state. This is done through Arm Trusted Firmware (ATF). For idling CPU, the PMU firmware raises TTC interrupt (timer 9) to ATF, which issues software interrupt to each alive A53 core. The respective cores then clears the pending SGI on itself and put itself into WFI.

The last core just before going into WFI issues `pm_system_shutdown` (PMU firmware API) to PMU firmware, which then performs APU only restart flow.

This feature must be enabled in ATF for recovery to work properly. It can be enabled by building ATF with `ZYNQMP_WARM_RESTART=1` flag.

Modifying Recovery Scheme

When `ENABLE_RECOVERY` is turned on, Xilinx provides a recovery implementation in which a FPD WDT timeout results in the invocation of APU subsystem restart. You can easily modify the recovery behavior by modifying the code. Alternatively, an example of PMU FW invoking system-reset on FPD WDT timeout is detailed in Xilinx Answer: [69423](#).

Escalation

If current recovery cannot bring the system back to the working state, the system must escalate to a more severe type of reset on the next WDT expiry in order to try and recover fully. It is up to you to decide on the escalation scheme. A commonly used scheme starts with APU-restart on the first watchdog expiration, followed by PS-only reset on the next watchdog expiration, then finally system-reset.

To enable Escalation, PMU firmware must be built with following flags:

```
ENABLE_ESCALATION
Escalation Scheme
```

Default Scheme

Default escalation scheme checks for the successful `pm_system_shutdown` call from ATF for APU-only restart which happens when the ATF is able to successfully idle all active CPUs. If ATF is not successful in idling the active cores (see blue boxes in [Figure 12-7](#)), WDT will time out again with the `WDT_in_Progress` flag set, resulting in do escalation.

Escalation will trigger System level reset. System level reset is defined as PS only reset if PL is present or System restart if PL is not present.

Figure 12-7 shows the flow of the control in case of default escalation scheme.

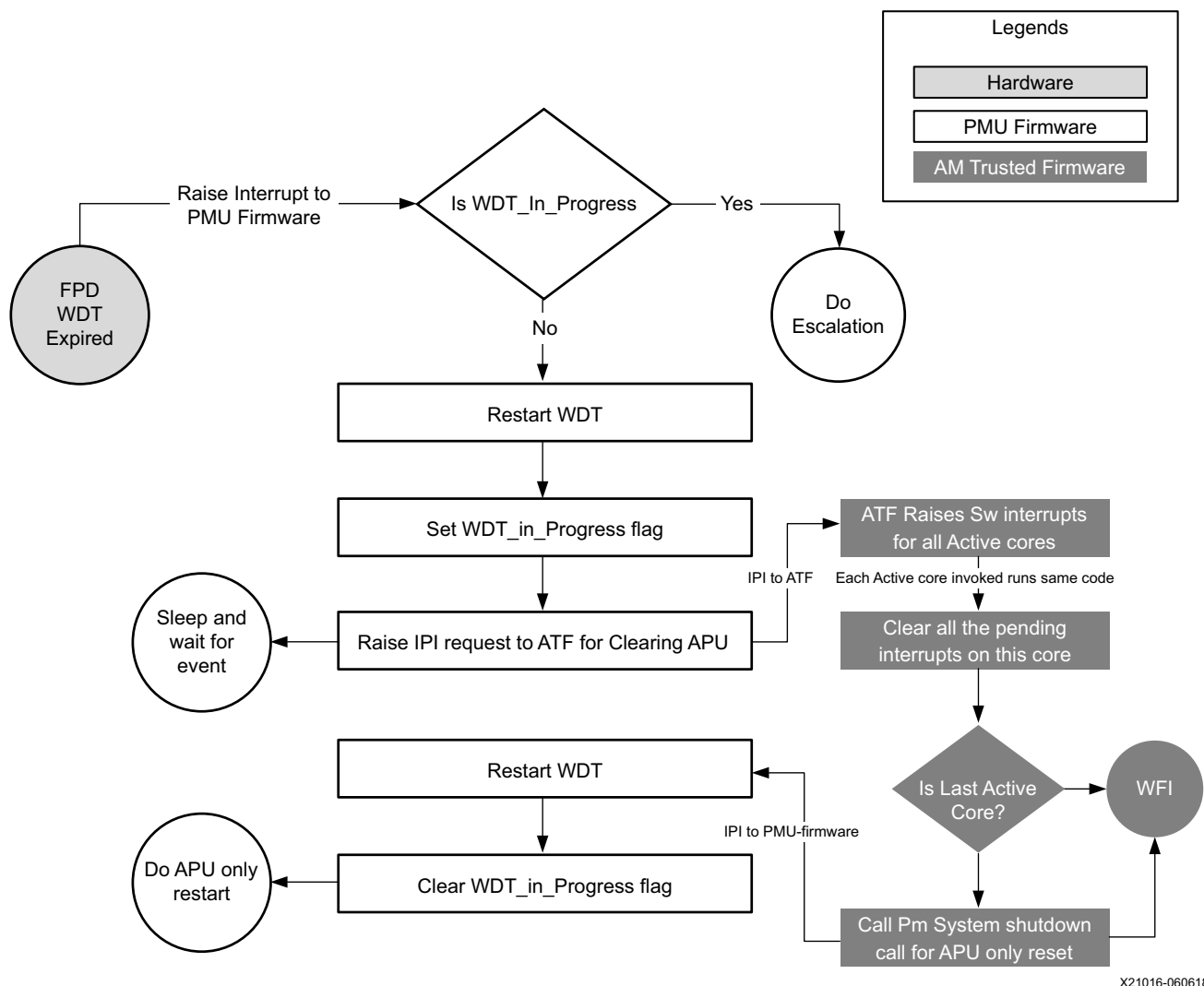


Figure 12-7:

Healthy Bit Scheme

Default scheme for escalation doesn't guarantee the successful reboot of the system. It only guarantees the successful role of ATF to idle the CPU during the recovery. Consider the scenario in which the FPD_WDT has timed out and APU subsystem restart is called in which ATF is able to successfully make the `pm_system_shutdown` call. However, APU subsystem restart is far from finished after `pm_system_shutdown` is called. The restart process can be stuck elsewhere, such as fsbl, u-boot or Linux init state. If the restart process is stuck in one of the aforementioned tasks, FPD_WDT will expire again, causing the same cycle to be repeated as long as ATF is loaded and functioning. This cycle can continue indefinitely without the system booting back into a clean running state.

The Healthy Bit scheme solves this problem. In addition to default scheme, the PMU firmware checks for a Healthy Bit, which is set by Linux on successful booting. On WDT expiry, if Healthy Bit is set, it indicates that Linux is able to boot into a clean running state, then no escalation is needed. However, if Healthy Bit is not set, that means the last restart attempt did not successfully boot into Linux and escalation is needed. There is no need to repeat the same type of restart. PMU firmware will escalate and call a system level reset.

Healthy Bit scheme is implemented using the bit-29 of PMU global general storage register (`PMU_GLOBAL_GLOBAL_GEN_STORAGE0[29]`). PMU firmware clears the bit before starting the recovery or normal reboot and Linux must set this bit to flag a healthy boot.

PMU global registers are accessed through sysfs interface from Linux. Hence, to set the healthy bit from the Linux, execute the following command (or include in the code):

```
# echo "0x20000000 0x20000000" > "/sys/devices/platform/firmware/ggs0"
```

To enable Healthy Bit based escalation scheme, build the PMU firmware with the following flag:

```
CHECK_HEALTHY_BOOT
```

Figure 12-8 shows the flow of the control in case of Healthy bit escalation scheme.

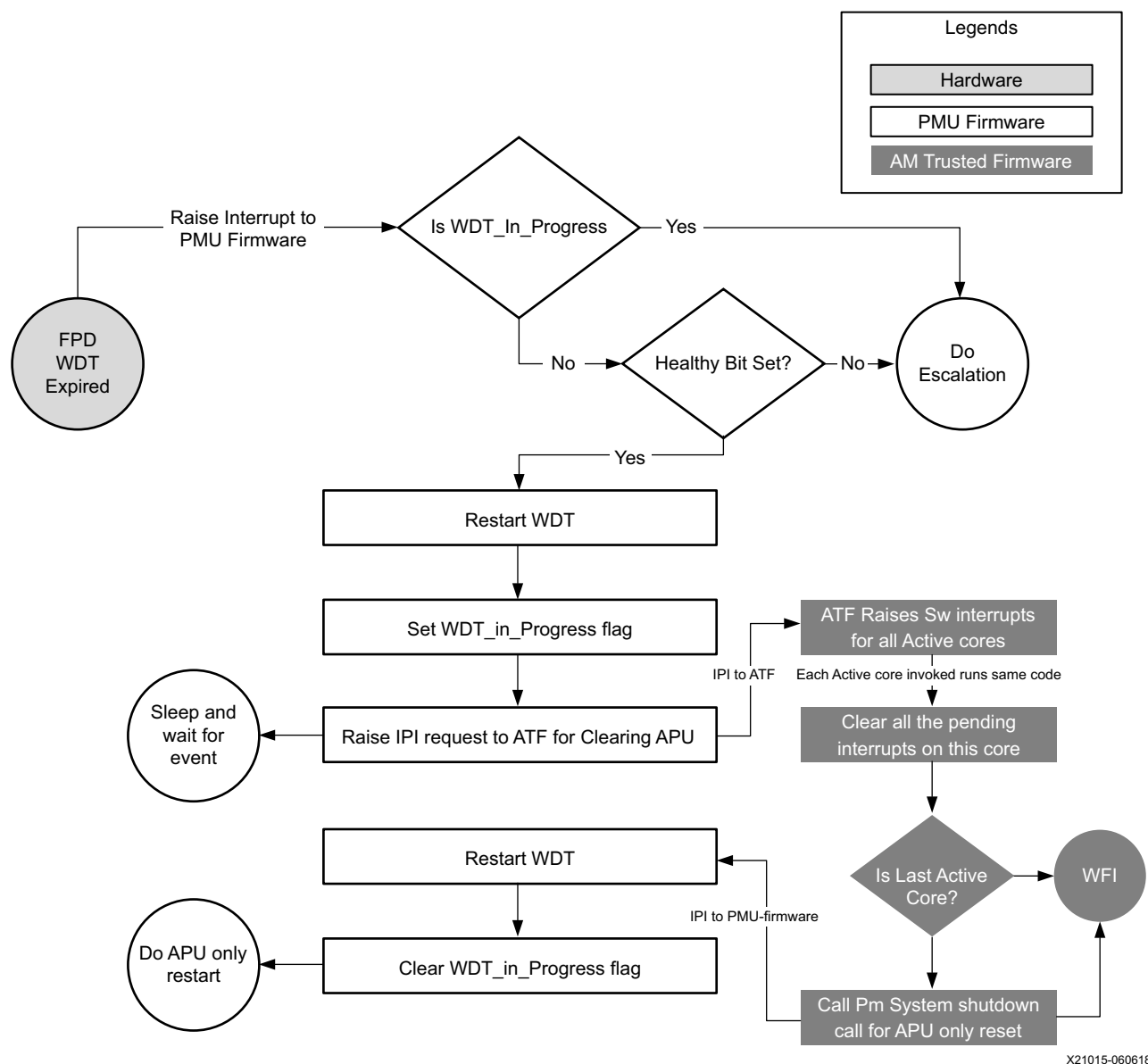


Figure 12-8: Healthy Bit Escalation Scheme

Customizing Recovery and Escalation Scheme

By default, when FPD WDT times out, PMU FW will not invoke any type of restart. While Xilinx has provided predefined RECOVERY and ESCALATION behaviors, users can easily customize different desired schemes.

When FPD_WDT times out, it calls `FpdSwdtHandler`. If `ENABLE_EM` is defined, `FpdSwdtHandler` calls `XPfw_recoveryHandler`. It is otherwise an empty function.

```

In xpfw_mod_em.c,

#ifdef ENABLE_EM
oid FpdSwdtHandler(u8 ErrorId)
{
    XPfw_Printf(DEBUG_ERROR, "EM: FPD Watchdog Timer Error (Error ID: %d)\r\n",
        ErrorId);
    XPfw_RecoveryHandler(ErrorId);
}

#else
void FpdSwdtHandler(u8 ErrorId) { }

```

Without `ENABLE_EM`, you can simply update `FpdSwdtHandler` which will be called at FPD Timeout. With `ENABLE_EM` turned on, you need to update `XPfw_recoveryHandler`.

Similarly, turning on `RECOVERY` defines the `XPfw_RecoveryHandler` (see `xpfw_restart.c`). Unless `RECOVERY` is turned on, `XPfw_RecoveryHandler` is an empty function and nothing will happen when `FPD_WDT` times out.

`RecoveryHandler` basically follows the flow chart detailed in the *Escalation Scheme* section. When `FPD_WDT` times out, the code follows the progression of orange boxes. If `WDT` is not already in progress, Restart `WDT`, Set `WDT_In_Progress` flag, Raise `TTC` (timer 9) interrupt to `ATF`. Then `ATF` takes over. It Raises `SW` interrupt for all active cores, clear pending interrupts, etc. (see blue boxes). Essentially, `PMU` restarts and boosts the `WDT`, then sends a request to `ATF`. `ATF` cleanly idles all four `APUs` and when they all get to `WFI` (Last Active Core is true), `ATF` issues `PMU` System Shutdown with `APU` subsystem as argument back to `PMU`. When `PMU` gets this command, it invokes `APU` subsystem restart.

If `ENABLE_ESCALATION` is not set, the code never takes the `Do Escalation` path. If the `RecoveryHandler` hangs for some reason (for example, something went wrong and `APU` cannot put all four `CPU` cores to `WFI`), it keeps retrying `APU` restart or hang forever. When `ENABLE_ESCLATION` is on and if anything goes wrong during execution of the flowchart, it will look like `WDT` is still in progress (since clear `WDT_in_progress` flag happens only as the last step), `Do Escalation` will call `SYSTEM_RESET` instead of trying `APU-restart` again and again.

To customize recovery and escalation behavior, use the provided `XPfw_recoveryHandler` as a template to provide a customized `XPfw_recoveryHandler` function.

Building Software

All the software components are built and packaged by Xilinx Petalinux tool. See [Petalinux wiki page](#) for more information on how to build and package software components.

Build Flag for Restart Solution

Following build time flags are not set by default and can alter the behavior of the restart in Zynq UltraScale+ MPSoC:

Table 12-2: Build Time Flags

Component	Flag Name	Description	Dependency
PMU firmware	ENABLE_EM	Enable error management and provide WDT interrupt handling. This is not directly related to restart solution but needed for recovery.	
	ENABLE_RECOVERY	Enable Recovery during WDT expiry	
	ENABLE_ESCALATION	Allow escalation on failure of boot or recovery	
	CHECK_HEALTHY_BOOT	Use Healthy bit to determine escalation	
	IDLE_PERIPHERALS	Both the flags must be used together to allow PMU firmware to attempt peripherals node idling (and reset).	
	ENABLE_NODE_IDLING		
	REMOVE_GPIO_FROM_NODE_RESET_INFO	Skips GPIO from the node idling and resetting list. This is needed when the system is using GPIO to provide reset (or similar) signals to PL or other peripherals outside current subsystem. If this flag is set, GPIO is not reset.	
ATF	ZYNQMP_WARM_RESTART=1	Enable WARM RESTART recovery feature in ATF that allow the CPU idling triggered from PMU firmware.	
FSBL	FSBL_PROT_BYPASS	Skip XMPU/XPPU based configuration for system except for DDR and OCM.	
Linux	CONFIG_SRAM	Needed for Remoteproc to work for load RPU images in the TCM.	

Modifying Component Recipes

Each component's recipe can be changed to either include the build time compilation flags or to include patches for custom code modification/addition. Petalinux provides meta-user yocto based layer for user specific modifications. The layer can be found in project directory *project-spec/meta-user/* location.

PMU firmware

User specific recipe for PMU firmware can be found in the following location:
dir:project-spec/meta-user/recipes-bsp/pmu/pmu-firmware_%.bbappend (if doesn't exist please create this file at this path).

The PMU firmware code can be modified by patches against [embeddedsw](#) GitHub repo. Location for the source code is *embeddedsw/tree/master/lib/sw_apps/zynqmp_pmufw*.

The patches should be copied to *project-spec/meta-user/recipes-bsp/pmu/files* directory and the same patch names should be added *pmu-firmware_%.bbappend* file.

Example:

If *my_changes.patch* (against PMU firmware source) is to be added and all the flags explained in [Table 12-2](#) are to be enabled (set), then

project-spec/meta-user/recipes-bsp/pmu/pmu-firmware_%.bbappend may look like the following file:

```
YAML_COMPILER_FLAGS_append = " -O2 -DENABLE_EM -DENABLE_RECOVERY
-DENABLE_ESCALATION -DENABLE_NODE_IDLING -DREMOVE_GPIO_FROM_NODE_RESET_INFO
-DCHECK_HEALTHY_BOOT -DIDLE_PERIPHERALS"

FILESEXTRAPATHS_prepend := "${THISDIR}/files:"

SRC_URI_append = " file://my_changes.patch"
```

FSBL

User specific recipe for fsbl can be found in the following location:
dir:project-spec/meta-user/recipes-bsp/fsbl/fsbl_%.bbappend (if does not exist, please create this file at this path). The fsbl code can be modified by patches against [embeddedsw](#) GitHub repo. Location for the source code is as follows:
embeddedsw/tree/master/lib/sw_apps/zynqmp_fsbl.

The patches should be copied to *project-spec/meta-user/recipes-bsp/fsbl/files* directory and the same patch names should be added to *fsbl_%.bbappend* file.

Example:

If *my_changes.patch* (against fsbl source) is to be added and all the flags explained in [Table 12-2](#) are to be enabled (set), then the modified

project-spec/meta-user/recipes-bsp/fsbl/fsbl_%.bbappend file will look like the following file(*XPS_BOARD_ZCU102* flag was already existing):

```
YAML_COMPILER_FLAGS_append = " -DXPS_BOARD_ZCU102 -DFSBL_PROT_BYPASS"
FILESEXTRAPATHS_prepend := "${THISDIR}/files:"
SRC_URI_append = " file://my_changes.patch"
```

ATF

User specific recipe for ATF can be found in the following location:

`:dirproject-spec/meta-user/recipes-bsp/arm-trusted-firmware/arm-trusted-firmware_%.bbappend` file (if it doesn't exist, create this file in this path). You can find the ATF files in [Git repository for arm trusted firmware](#).

Example:

To add warm restart flag to ATF,

`project-spec/meta-user/recipes-bsp/arm-trusted-firmware/arm-trusted-firmware_%.bbappend` will look like the following file:

```
#
# Enabling warm restart feature
#
EXTRA_OEMAKE_append = " ZYNQMP_WARM_RESTART=1 "
```

Linux

There are many ways to add /modify Linux configuration. See *PetaLinux Tools Documentation: Reference Guide* (UG1144)[\[Ref 27\]](#) for the same.

User specific recipe for Linux kernel can be found in the following location:

`project-spec/meta-user/recipes-kernel/linux/linux-xlnx_%.bbappend` (if it doesn't exist, create this file at this path).

You can find the Linux files at [Git Repository for Linux](#)

Example:

To add SRAM config to Linux, create the following `bsp.cfg` file:

`project-spec/meta-user/recipes-kernel/linux/linux-xlnx/bsp.cfg`

```
CONFIG_SRAM=y
```

Add this file in the following `bbappend` file of Linux:

`project-spec/meta-user/recipes-kernel/linux/linux-xlnx_%.bbappend`

```
SRC_URI += "file://bsp.cfg"
FILESEXTRAPATHS_prepend := "${THISDIR}/${PN}:"
```

Modifying Device Tree

User specific recipe for device tree can be found in the following location:

project-spec/meta-user/recipes-bsp/device-tree/device-tree-generation_%.bbappend. This file contains the following contents:

```
SRC_URI_append = "\
    file://system-user.dtsi \
"
FILESEXTRAPATHS_prepend := "${THISDIR}/files:"
```

The content of **system-user.dtsi** in *project-spec/meta-user/recipes-bsp/device-tree/files* directory is as follows:

```
/include/ "system-conf.dtsi"
/ {
};
```

This file can be modified to extend the device tree functionality by adding /removing / modifying the DTS nodes.

Example: Adding DT node(s) [remoteproc RPU split mode]

The overlay dtsi(s) can be added in files/ directory (remember to update bbappend file accordingly) and included in system-user.dtsi. For adding **remoteproc** related entries to enable RPU subsystem load / unload /restart; lets add new overlay file called **remoteproc.dtsi**

Note: This is for split mode. Check open amp documentation for lockstep and other possible configurations.

File: **remoteproc.dtsi**

```
/ {
    reserved-memory {
        #address-cells = <2>;
        #size-cells = <2>;
        ranges;
        rproc_0_reserved: rproc:dir3ed000000 {
            no-map;
            reg = <0x0 0x3ed00000 0x0 0x1000000>;
        };
    };
    power-domains {
```

```

pd_r5_0: pd_r5_0 {
    #power-domain-cells = <0x0>;

    pd-id = <0x7>;

};

pd_r5_1: pd_r5_1 {
    #power-domain-cells = <0x0>;

    pd-id = <0x8>;

};

pd_tcm_0_a: pd_tcm_0_a {
    #power-domain-cells = <0x0>;

    pd-id = <0xf>;

};

pd_tcm_0_b: pd_tcm_0_b {
    #power-domain-cells = <0x0>;

    pd-id = <0x10>;

};

pd_tcm_1_a: pd_tcm_1_a {
    #power-domain-cells = <0x0>;

    pd-id = <0x11>;

};

pd_tcm_1_b: pd_tcm_1_b {
    #power-domain-cells = <0x0>;

    pd-id = <0x12>;

};

};
amba {

    r5_0_tcm_a: tcm:dirffe00000 {

        compatible = "mmio-sram";

        reg = <0x0 0xFFE00000 0x0 0x10000>;

        pd-handle = <&pd_tcm_0_a>;

    };
};

```



```

r5_0_tcm_b: tcm:dirffe20000 {
    compatible = "mmio-sram";

    reg = <0x0 0xFFE20000 0x0 0x10000>;

    pd-handle = <&pd_tcm_0_b>;
};

r5_1_tcm_a: tcm:dirffe90000 {
    compatible = "mmio-sram";

    reg = <0x0 0xFFE90000 0x0 0x10000>;

    pd-handle = <&pd_tcm_1_a>;
};

r5_1_tcm_b: tcm:dirffeb0000 {
    compatible = "mmio-sram";

    reg = <0x0 0xFFEB0000 0x0 0x10000>;

    pd-handle = <&pd_tcm_1_b>;
};

elf_ddr_0: ddr:dir3ed00000 {
    compatible = "mmio-sram";

    reg = <0x0 0x3ed00000 0x0 0x40000>;
};

elf_ddr_1: ddr:dir3ed40000 {
    compatible = "mmio-sram";

    reg = <0x0 0x3ed40000 0x0 0x40000>;
};

test_r50: zynqmp_r5_rproc:dir0 {
    compatible = "xlnx,zynqmp-r5-remoteproc-1.0";

    reg = <0x0 0xff9a0100 0x0 0x100>, <0x0 0xff340000 0x0 0x100>, <0x0
0xff9a0000 0x0 0x100>;

    reg-names = "rpu_base", "ipi", "rpu_glbl_base";

    dma-ranges;

```

```

        core_conf = "split0";

        sram_0 = <&r5_0_tcm_a>;

        sram_1 = <&r5_0_tcm_b>;

        sram_2 = <&elf_ddr_0>;

        pd-handle = <&pd_r5_0>;

        interrupt-parent = <&gic>;

        interrupts = <0 29 4>;

    } ;
test_r51: zynqmp_r5_rproc:dir1 {
    compatible = "xlnx,zynqmp-r5-remoteproc-1.0";
    reg = <0x0 0xff9a0200 0x0 0x100>, <0x0 0xff340000 0x0 0x100>, <0x0 0xff9a0000 0x0
0x100>;

    reg-names = "rpu_base", "ipi", "rpu_glbl_base";

    dma-ranges;

    core_conf = "split1";

    sram_0 = <&r5_1_tcm_a>;

    sram_1 = <&r5_1_tcm_b>;

    sram_2 = <&elf_ddr_1>;

    pd-handle = <&pd_r5_1>;

    interrupt-parent = <&gic>;

    interrupts = <0 29 4>;

} ;
};
};

```

Now include this node in system-user.dtsi

```

#include/ "system-conf.dtsi"
#include/ "remoteproc.dtsi"
/ {
};

```

For information on OpenAMP and remoteproc, seen [OpenAmp wiki page](#).

Example: Removing DT node(s) [PL node]

It is necessary to remove PL nodes, which are not accessed or dependent on APU subsystem, from the device tree. Again, you can modify `system-user.dtsi` in `project-spec/meta-user/recipes-bsp/device-tree/files` to remove specific node or property.

For example, you can modify the `system-user.dtsi` as following, if you are willing to remove AXI DMA node from the dts:

```
/include/ "system-conf.dtsi"  
/include/ "remoteproc.dtsi"  
/  
{  
/delete-node/axi-dma;  
};
```

High-Speed Bus Interfaces

Introduction

The Zynq® UltraScale+™ MPSoC device has a serial input/output unit (SIOU) for a high-speed serial interface. It supports protocols such as PCIe™, USD 3.0, DisplayPort, SATA, and Ethernet protocols.

- The SIOU block is part of the full-power domain (FPD) in the PS.
- The USB and Ethernet controller blocks that are part of the low-power domain (LPD) in the Zynq UltraScale+ MPSoC device also share the PS-GTR transceivers.
- The interconnect matrix enables multiplexing of four PS-GTR transceivers in various combinations across multiple controller blocks.
- A register block controls or monitors signals within the SIOU.

This chapter explains the configuration flow of the high-speed interface protocols.

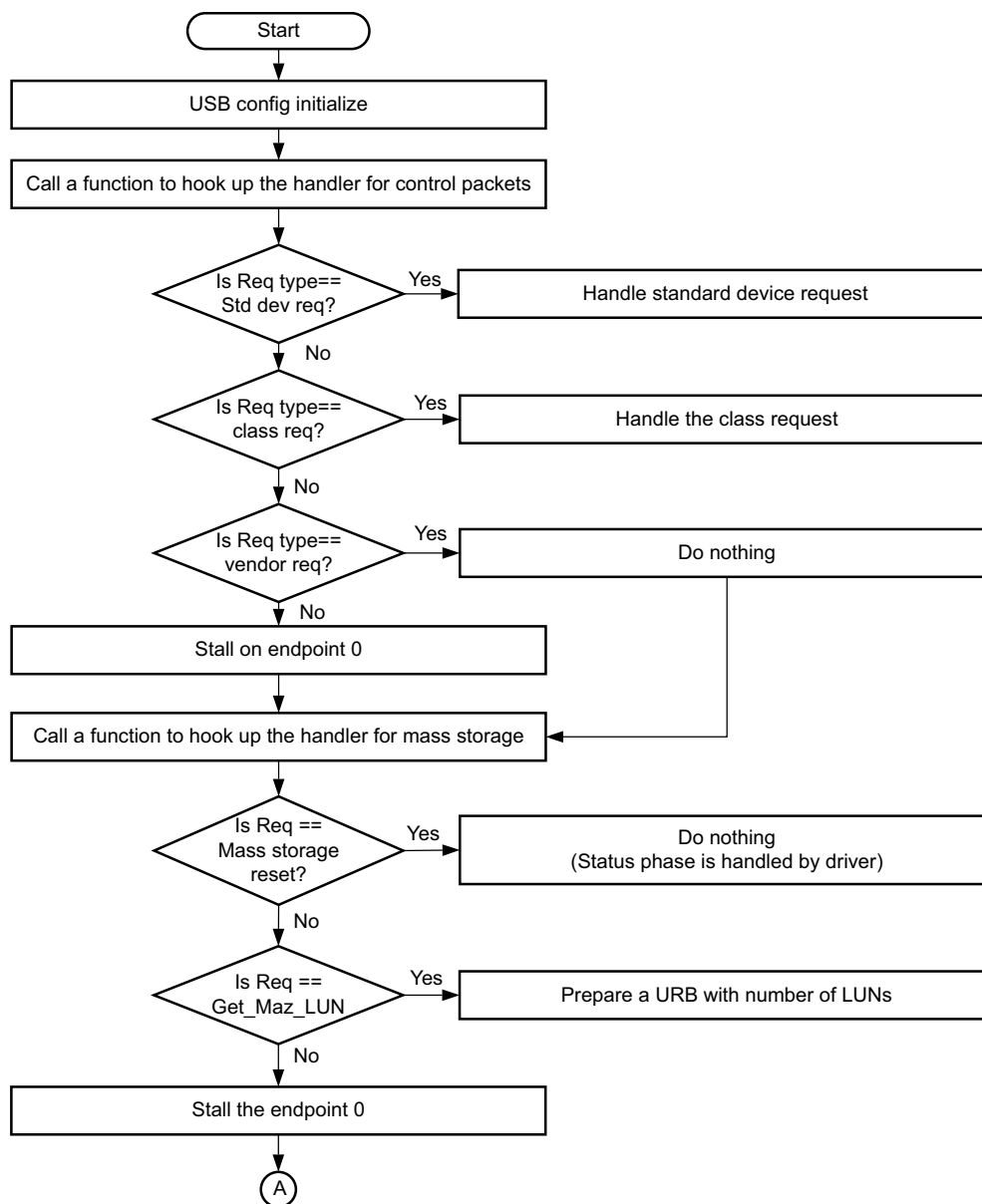
See this [link](#) to the “High-Speed PS-GTR Transceiver Interface” of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11] for more information.

USB 3.0

The Zynq UltraScale+ MPSoC USB 3.0 controller consists of two independent dual-role device (DRD) controllers. Both can be individually configured to work as host or device at any given time. The USB 3.0 DRD controller provides an eXtensible host controller interface (xHCI) to the system software through the advanced eXtensible interface (AXI) slave interface.

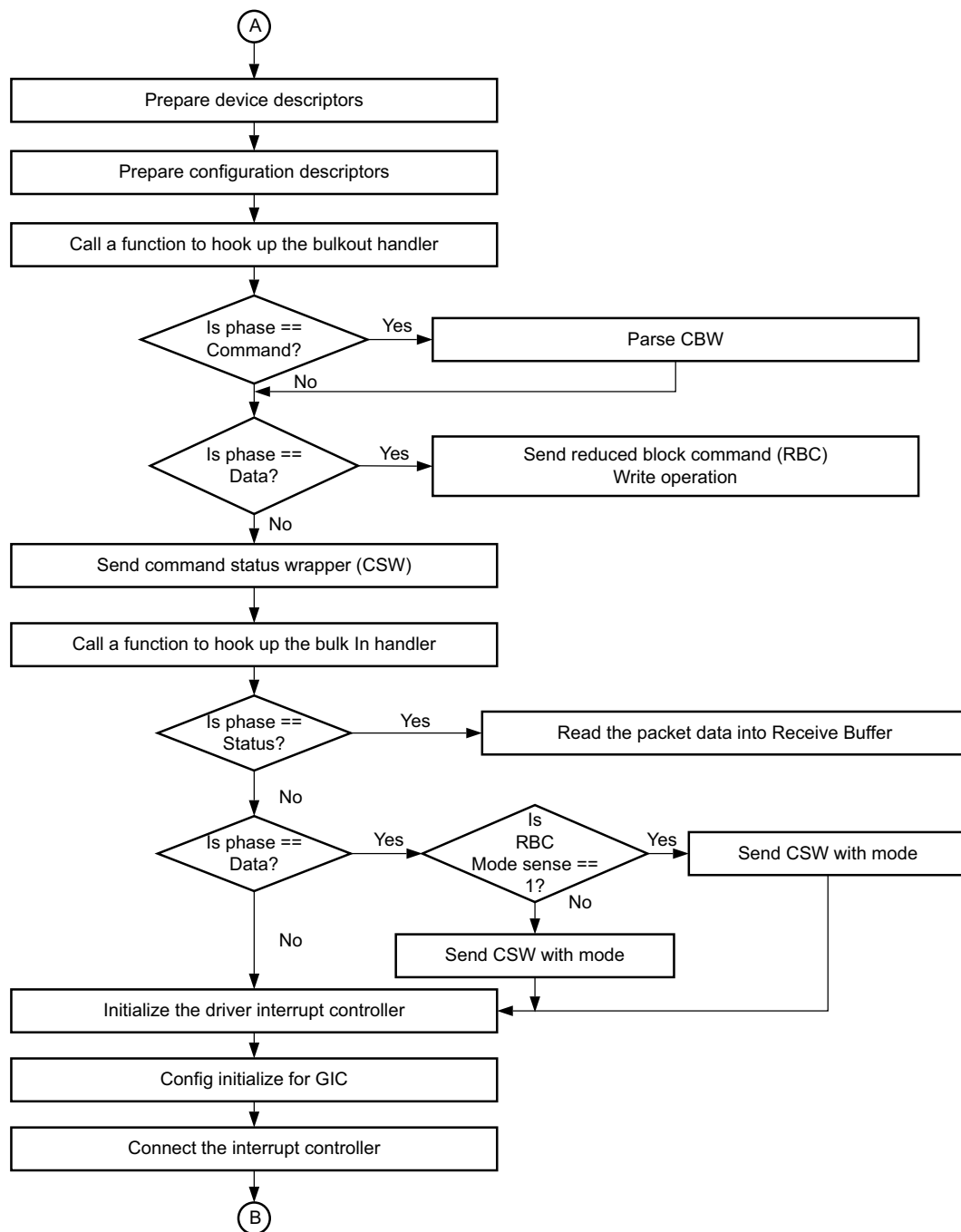
- An internal DMA engine is present in the controller and it uses the AXI master interface to transfer data.
- The three dual-port RAM configurations implement the RX data FIFO, TX data FIFO, and the descriptor/register cache.

The following flow diagrams illustrate how to configure USB as mass storage device.



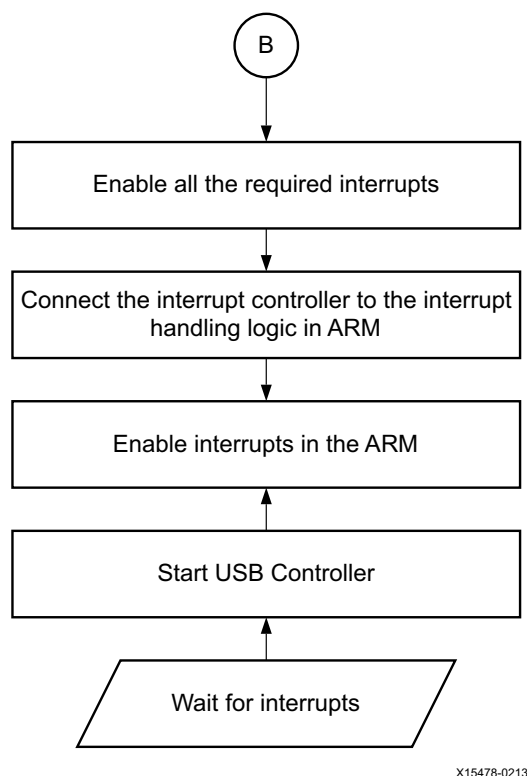
X15463-071017

Figure 13-1: USB Example Flow: USB Initialization



X15477-071017

Figure 13-2: Example USB Flow: Hookup Bulk in and Bulk out Handlers and Initialize Interrupt Controller



X15478-021317

Figure 13-3: Enable Interrupts and Start the USB Controller

For more information on USB controller, see this [link](#) to the “USB 2.0/3.0 Host, Device, and Controller,” chapter of the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

Gigabit Ethernet Controller

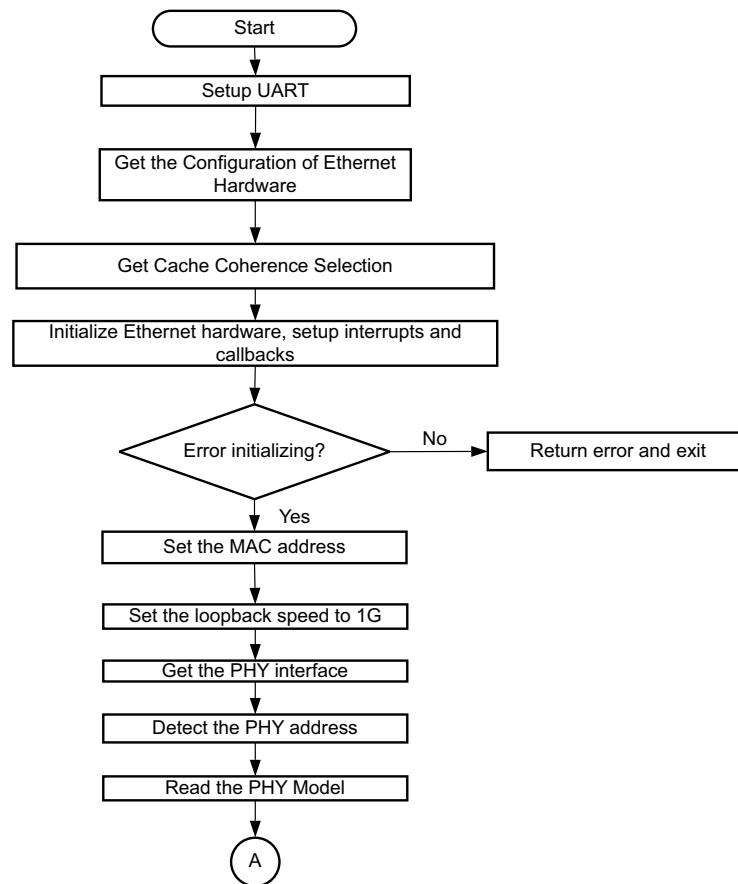
The gigabit Ethernet controller (GEM) implements a 10/100/1000 Mb/s Ethernet MAC compatible with *IEEE Standard for Ethernet* (IEEE Std 802.3-2008) and is capable of operating in either half or full-duplex mode in 10/100 mode and full-duplex in 1000 mode.

The processor system (PS) is equipped with four gigabit Ethernet controllers.

Registers are used to configure the features of the MAC, and select different modes of operation.

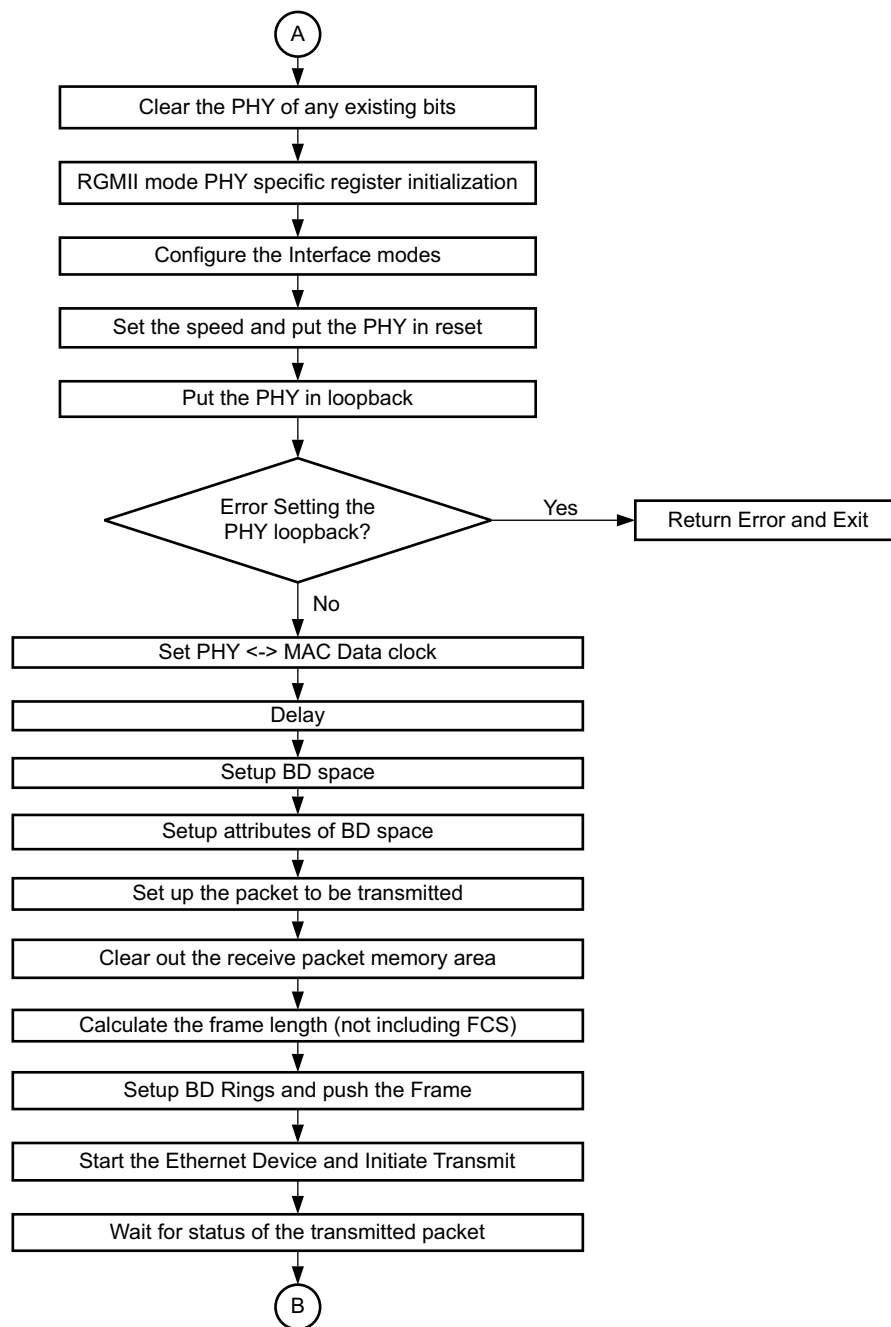
The DMA controller connects to memory through the advanced eXtensible interface (AXI). It is attached to the FIFO interface of the controller of the MAC to provide a scatter-gather type capability for packet data storage in an embedded processing system.

The following figures illustrate an example for configuring an Ethernet controller to send a single packet of data in RGMII mode.



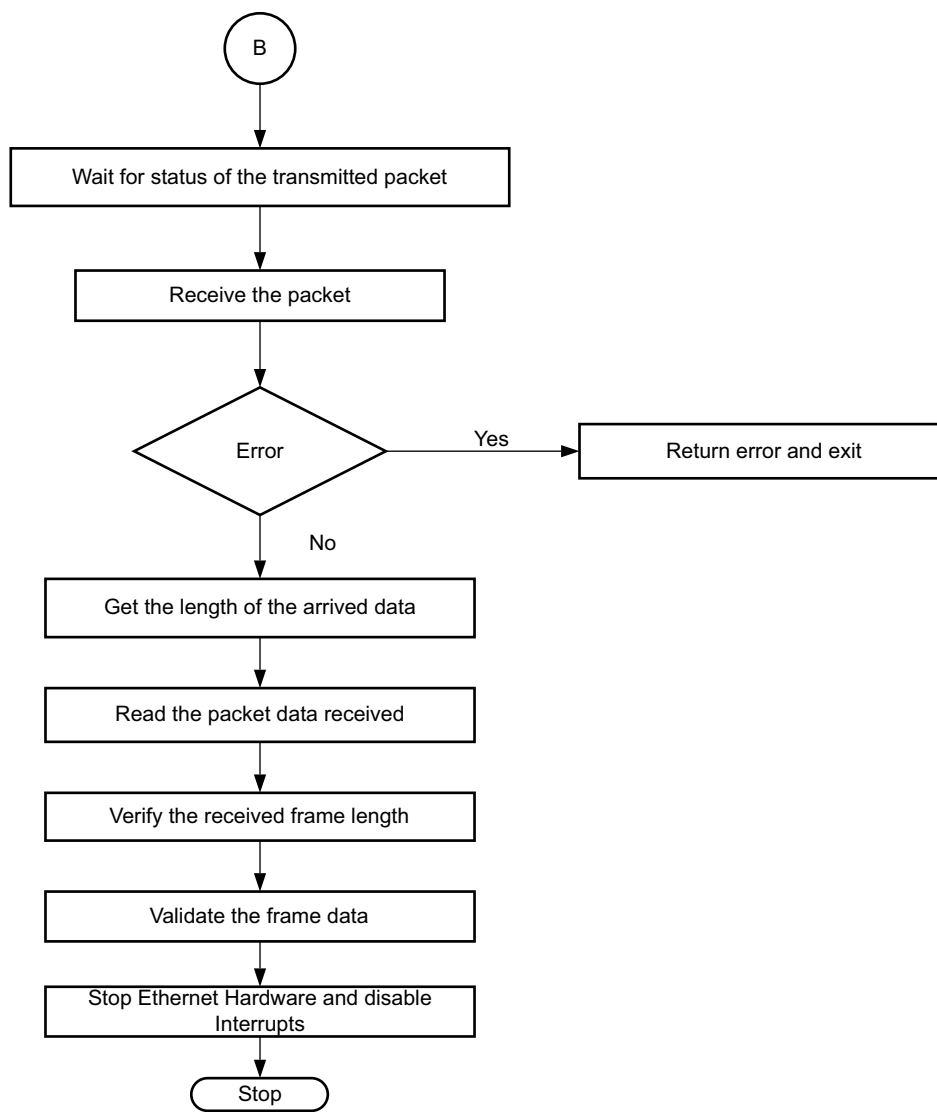
X15462-071017

Figure 13-4: Example Ethernet Flow: Initialize Ethernet Controller



X15479-071017

Figure 13-5: Example Ethernet Flow: Configure the Ethernet Parameters & Initiate the Transmit



X15480-021317

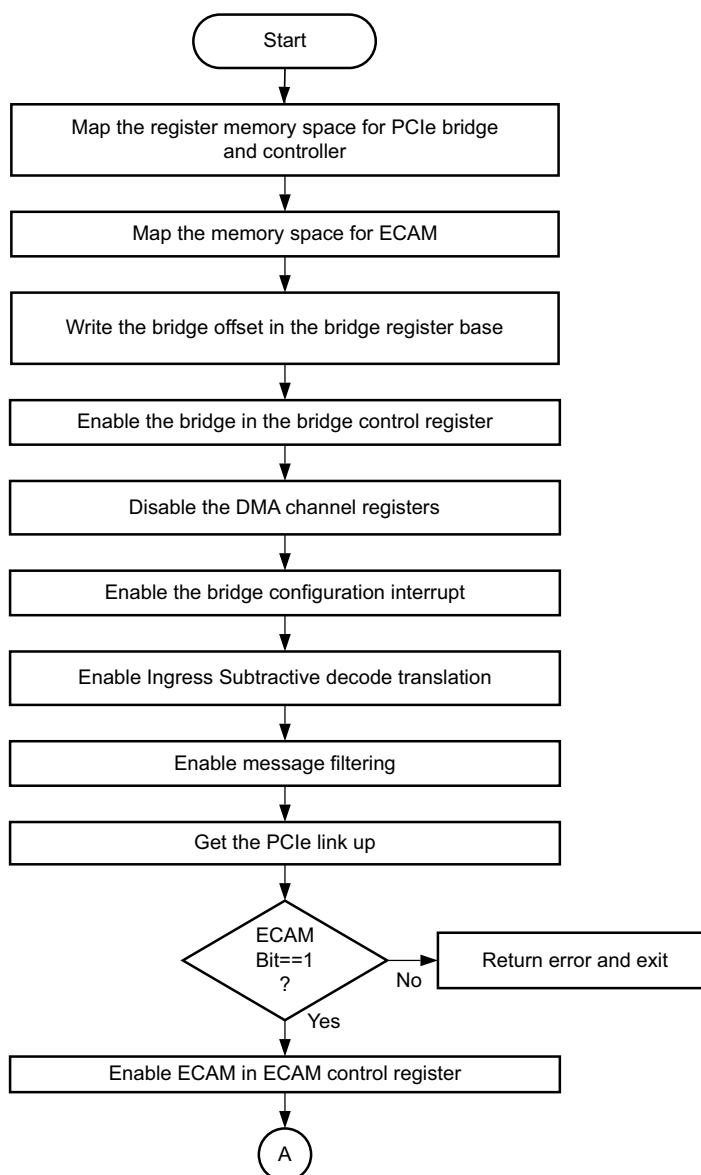
Figure 13-6: Example Ethernet Flow: Receive and Validate the Data

For more information on Ethernet Controller, see this [link](#) to the “Gigabit Ethernet Controller” chapter in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

PCI Express

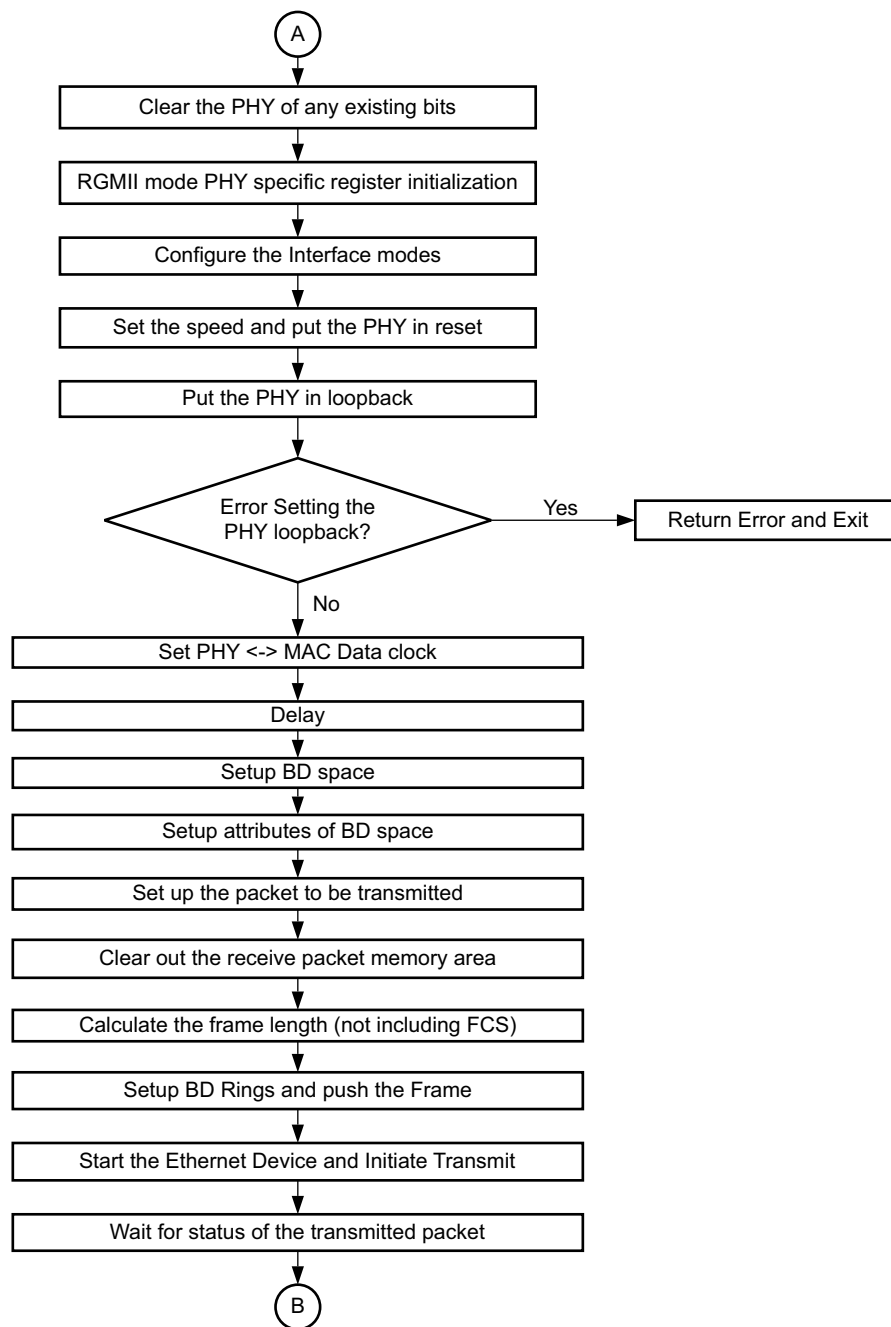
The Zynq UltraScale+ MPSoC device provides a controller for the integrated block for PCI Express™ v2.1 compliant, AXI-PCIe Bridge, and DMA modules. The AXI-PCIe Bridge provides high-performance bridging between PCIe and AXI.

The following flow diagrams illustrate an example for configuring PCIe root complex for a data transfer.



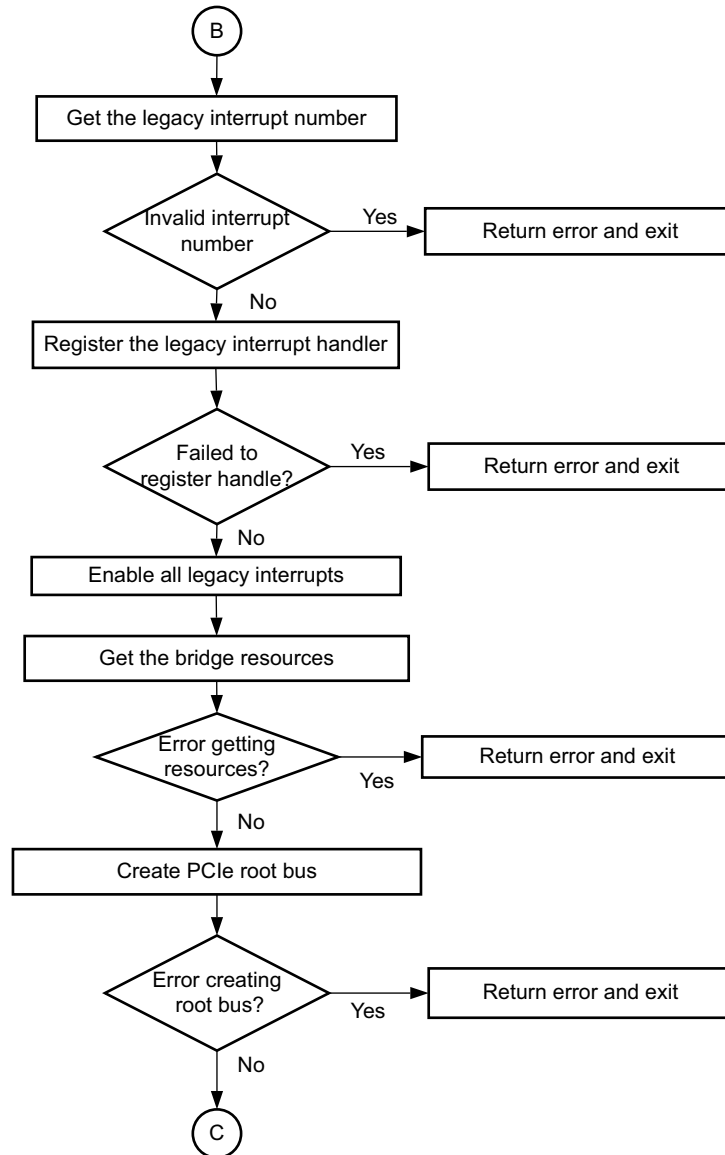
X15481-071217

Figure 13-7: Example PCIe Flow: Enable the Legacy Interrupts and Create PCIe Root Bus



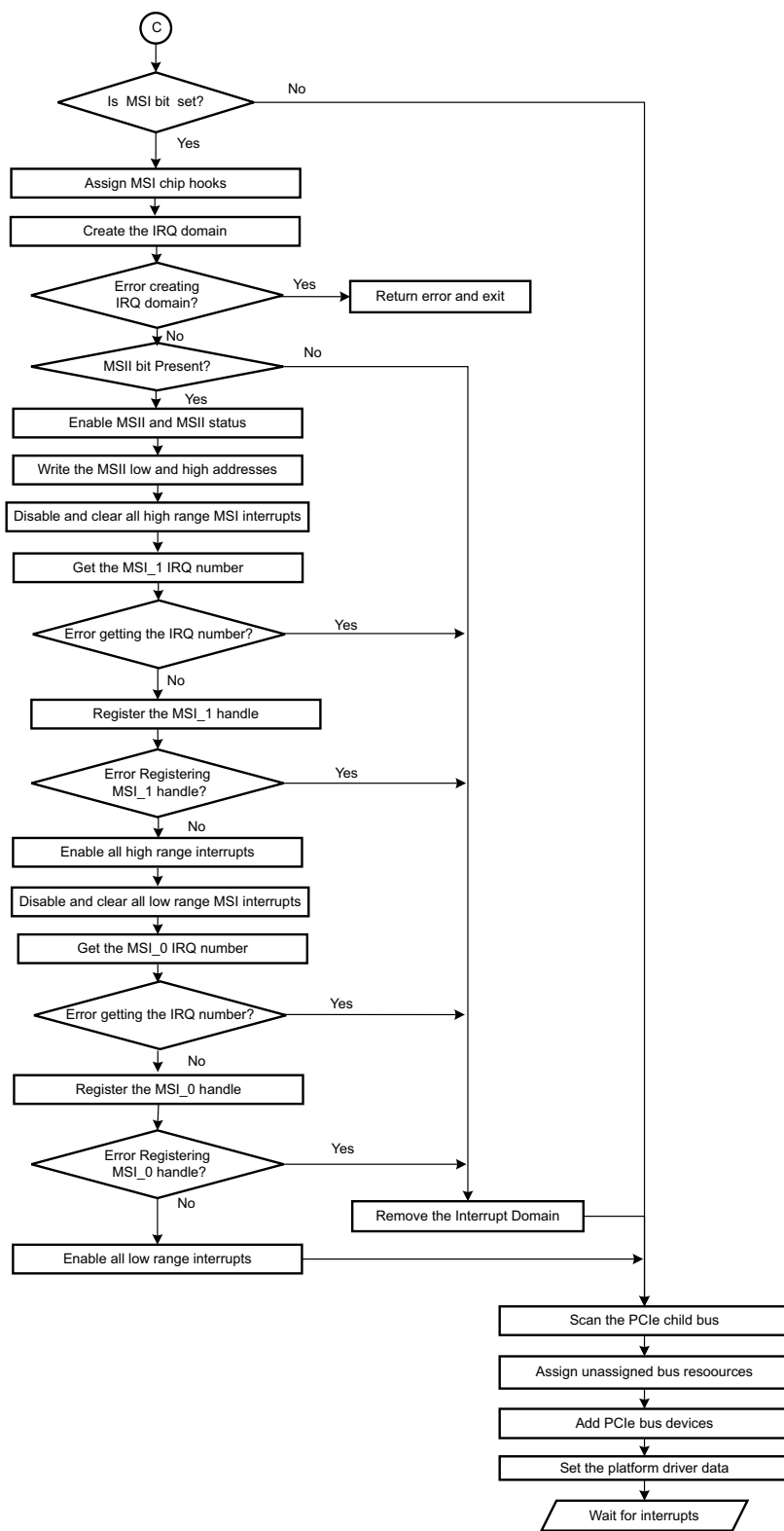
X15479-071017

Figure 13-8: Example PCIe Flow: Configure the PCIe Parameters and Initialize the Transmit



X15483-071217

Figure 13-9: Example PCIe Flow: Enable the Legacy Interrupts and Create PCIe Root Bus



X15484-071217

Figure 13-10: Example PCIe Flow: Enable MSI Interrupts and Wait for Interrupts

Note: For endpoint operation, refer to this [link](#) to “Controller for PCI Express” in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

After the memory space for PCIe bridge and ECAM is mapped, ECAM is enabled for ECAM translations. You then acquire the bus range to set up the bus numbers, and write the primary, secondary, and subordinate bus numbers.

The interrupt system must be set up by enabling all the miscellaneous and legacy interrupts. You can parse the ranges property of a PCI host bridge device node, and setup the resource mapping based on its content.

To create a root bus, allocate the PCIe root bus and add initial resources to the bus.

If the MSI bit is set, you must enable the message signaling interrupt (MSI).

After configuring the MSI interrupts, scan the PCIe slot and enumerate the entire PCIe bus and allocate bus resources to scanned buses.

Now, you can add PCIe devices to the system.

For more information on PCI Express, see this [link](#) to the “DMA Controller” section and this [link](#) to “Controller for PCI Express” in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

Clock and Frequency Management

Introduction

The Zynq® UltraScale+™ MPSoC device architecture includes a programmable clock generator that takes a clock of a definite input frequency and generates multiple-derived clocks using the phase-locked loop (PLL) blocks in the PS. The output clock from each of the PLLs is used as a reference clock for the different PS peripherals.

Unlike the USB and Ethernet peripherals, some peripherals like the UART and SD allow you to dynamically change the device frequency setting.

This chapter provides information about changing the operating frequency of these peripherals dynamically. See [Power Management Framework](#) for more information on reducing or adjusting the clock frequencies.

Changing the Peripheral Frequency

You can change the peripheral operation frequency by directly setting the frequency in the corresponding peripheral clock configuration register. The Zynq UltraScale+ MPSoC BSP provides APIs that aid in changing the peripheral clock frequency dynamically according to your requirements.

The following table shows the standalone APIs that can be used to change the frequency of the peripherals.

Table 14-1: Standalone APIs

APIs	Description
<code>XSDPS_change_clkfreq</code>	Change the clock frequency of SD.
<code>XSPIPS_setclkprescaler</code> <code>XSPIPS_getclkprescaler</code>	Pre-scale the SPI frequency.
<code>XRtcPSu_calculatecalibration</code>	Change the oscillator frequency.
<code>XQSPIPSU_setclkprescaler</code>	Change the clock frequency of QSPI.

In case of a Linux application, the frequency of all the peripherals is set in the device tree file. The following code snippet shows the setting of peripheral clock.

```
ps7_qspi_0: ps7-qspi:dir0xFF0F0000 {
    #address-cells = <0x1>;
    #size-cells = <0x0>;
    #bus-cells = <0x1>;
    clock-names = "ref_clk", "pclk";
    compatible = "xlnx,usmp-gqspi", "cdns,spi-r1p6";
    stream-connected-dma = <0x26>;
    clocks = <0x1e 0x1e>;
    dma = <0xb>;
    interrupts = <0xf>;
    num-chip-select = <0x2>;
    reg = <0x0 0xff0f0000 0x1000 0x0 0xc0000000 0x8000000>;
    speed-hz = <0xbefc200>;
    xlnx,fb-clk = <0x1>;
    xlnx,qspi-clk-freq-hz = <0xbefc200>;
    xlnx,qspi-mode = <0x2>;
}
```

To avoid any error condition, the peripheral needs to be stopped before changing the corresponding clock frequency.

The steps to follow before changing the clock frequency for any peripheral are as follows:

1. Stop the transition pertaining to the peripheral (IP) and make it idle.
2. Stop the IP by appropriately configuring the registers.
3. Change the clock frequency of the peripheral.
4. Issue soft reset to the IP.
5. Restart the IP.

For more information on Zynq UltraScale+ MPSoC clock generator, see this [link](#) in the "Clocking" chapter in the *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

Target Development Platforms

Introduction

This chapter describes various development platforms available for the Zynq® UltraScale+™ MPSoC device, such as Quick Emulators (QEMU) and the Zynq UltraScale+ MPSoC boards and kits.

QEMU

QEMU is a system emulation model that functions on an Intel-compatible Linux host system. Xilinx® QEMU implements a framework for generating custom machine models based on a device tree passed to it using the command line. See the *Zynq UltraScale+ MPSoC QEMU: User Guide* (UG1169) [\[Ref 8\]](#) for more information about QEMU.

Boards and Kits

Xilinx provides the Zynq UltraScale+ MPSoC ZCU102 Evaluation Kit for developers. To understand more about the ZCU102 evaluation kit, see the *Preliminary ZCU102 Getting Started Guide Answer Record: 66249* [\[Ref 36\]](#).

See the *Zynq UltraScale+ MPSoC Products Page* [\[Ref 7\]](#) to know the different Zynq UltraScale+ MPSoC devices.

Boot Image Creation

Introduction

Zynq® UltraScale+™ MPSoC supports both secure and non-secure booting. While deploying the devices in field, it is important to prevent unauthorized or modified code from being run on these devices. Zynq UltraScale+ MPSoC provides the required confidentiality, integrity, and authentication to host applications securely. For more information on security features, see *Zynq UltraScale+ MPSoC Technical Reference Manual* (UG1085) [Ref 11].

Zynq UltraScale+ MPSoC devices typically have many hardware and software binaries that are used to boot them to function as designed and expected. These binaries includes FPGA bitstreams, Firmware, boot loaders, operating system, and applications that you select. For example: FPGA bitstream files, first stage boot loader (FSBL), PMU firmware, ATF, U-Boot, Linux kernel, Rootfs, device tree, standalone or RTOS applications and so on). Xilinx provides a standalone tool, Bootgen, to stitch all these binary images together and generate a device bootable image in a specific format that Xilinx loader programs can interpret while loading.

Bootgen has multiple attributes and commands that define its behavior while generating boot images. They are secure boot image generation, non-secure boot image generation, Secure key generation, HMI Mode and so on. For complete details of how to get the Bootgen tool, the installation procedure, and details of Zynq Ultrascale+ Boot Image format, Bootgen commands, attributes, and boot image generation procedure with examples, see *Bootgen User Guide* (UG1283) [Ref 23].

XilPM Library Parameters

XilPM Argument Value Definitions

Introduction

The following are value definitions for the various arguments used in the power management APIs, as defined in the file `pm_defs.h`.

Node IDs: XPmNodeId

The following table lists the defined Node IDs in the Zynq® UltraScale™+ MPSoC device.

```
enum XPmNodeId {
```

Table A-1: XPmNodeIds

Node IDs for Zynq UltraScale+ MPSoC Devices		
NODE_UNKNOWN		0U
NODE_APU	APU Controller	1U
NODE_APU_0	APU Controller 0	2U
NODE_APU_1	APU Controller 1	3U
NODE_APU_2	APU Controller 2	4U
NODE_APU_3	APU Controller 3	5U
NODE_RPU	RPU Controller	6U
NODE_RPU_0	RPU Controller 0	7U
NODE_RPU_1	RPU Controller 1	8U
NODE_PLD	PLD Controller	9U
NODE_FPD	FPD Controller	10U
NODE_OCM_BANK_0	OCM Memory Tile 0	11U
NODE_OCM_BANK_1	OCM Memory Tile 1	12U
NODE_OCM_BANK_2	OCM Memory Tile 2	13U
NODE_OCM_BANK_3	OCM Memory Tile 3	14U
NODE_TCM_0_A	Tightly coupled memory (0A)	15U

Table A-1: XPmNodeIds (Cont'd)

Node IDs for Zynq UltraScale+ MPSoC Devices		
NODE_TCM_0_B	Tightly coupled memory (0B)	16U
NODE_TCM_1_A	Tightly coupled memory (1A)	17U
NODE_TCM_1_B	Tightly coupled memory (1B)	18U
NODE_L2	L2 Cache system	19U
NODE_GPU_PP_0	Graphics Processing Unit 0	20U
NODE_GPU_PP_1	Graphics Processing Unit 1	21U
NODE_USB_0	USB Controller 0	22U
NODE_USB_1	USB Controller 1	23U
NODE_TTC_0	Triple-timer Counter 0	24U
NODE_TTC_1	Triple-timer Counter 1	25U
NODE_TTC_2	Triple-timer Counter 2	26U
NODE_TTC_3	Triple-timer Counter 3	27U
NODE_SATA	SATA Controller	28U
NODE_ETH_0	Gigabit Ethernet Controller 0	29U
NODE_ETH_1	Gigabit Ethernet Controller 1	30U
NODE_ETH_2	Gigabit Ethernet Controller 2	31U
NODE_ETH_3	Gigabit Ethernet Controller 3	32U
NODE_UART_0	UART Controller 0	33U
NODE_UART_1	UART Controller 1	34U
NODE_SPI_0	SPI Controller 0	35U
NODE_SPI_1	SPI Controller 1	36U
NODE_I2C_0	SPI Controller 2	37U
NODE_I2C_1	SPI Controller 3	38U
NODE_SD_0	SD/SDIO Controller 0	39U
NODE_SD_1	SD/SDIO Controller 1	40U
NODE_DP	DisplayPort Controller	41U
NODE_GDMA	FPD DMA Controller	42U
NODE_ADMA	APU DMA	43U
NODE_NAND	NAND Controller	44U
NODE_QSPI	QSPI Controller	45U
NODE_GPIO	GPIO Controller	46U
NODE_CAN_0	CAN Controller 0	47U
NODE_CAN_1	CAN Controller 1	48U
NODE_EXTERN	Slave External Device	49U
NODE_APLL	APU PLL	50U

Table A-1: XPmNodeIds (Cont'd)

Node IDs for Zynq UltraScale+ MPSoC Devices		
NODE_VPLL	Video PLL	51U
NODE_DPLL	DDR Controller PLL	52U
NODE_RPLL	RPU PLL	53U
NODE_IOPLL	Peripheral I/O PLL	54U
NODE_DDR	DDR Controller	55U
NODE_IPI_APU	IPI APU Controller	56U
NODE_IPI_RPU_0	IPI APU Controller 0	57U
NODE_GPU	Graphics Processing Unit Controller	58U
NODE_PCIE	PCIE Controller	59U
NODE_PCAP	PCAP Controller	60U
NODE_RTC	RTC Controller	61U
NODE_LPD	LPD Controller	62U
NODE_VCU	VCU Controller	63U
NODE_IPI_RPU_1	IPI RPU Controller 1	64U
NODE_IPI_PL_0	IPI PL Controller 0	65U
NODE_IPI_PL_1	IPI PL Controller 1	66U
NODE_IPI_PL_2	IPI PL Controller 2	67U
NODE_IPI_PL_3	IPI PL Controller 3	68U
NODE_PL	PL Controller	69U

Acknowledge Request Types: XPmRequestAck

```
enum XPmRequestAck {
    REQUEST_ACK_NO = 1,
    REQUEST_ACK_BLOCKING,
    REQUEST_ACK_NON_BLOCKING,
};
```

Abort Reasons: XPmAbortReason

```
enum XPmAbortReason {
    ABORT_REASON_WKUP_EVENT = 100,
    ABORT_REASON_PU_BUSY,
    ABORT_REASON_NO_PWRDN,
    ABORT_REASON_UNKNOWN,
};
```

Suspend Reasons

```
enum XPmSuspendReason {
    SUSPEND_REASON_PU_REQ = 201,
    SUSPEND_REASON_ALERT,
    SUSPEND_REASON_SYS_SHUTDOWN,
};
```

Operating Characteristic Types: XPmOpCharType

```
enum XPmOpCharType {
    PM_OPCHAR_TYPE_POWER = 1,
    PM_OPCHAR_TYPE_ENERGY,
    PM_OPCHAR_TYPE_TEMP,
};
```

Notify Event Types: XPmNotifyEvent

```
enum XPmNotifyEvent {
    EVENT_STATE_CHANGE = 1,
    EVENT_ZERO_USERS = 2,
    EVENT_ERROR_CONDITION = 4,
};
```

Reset Line IDs

```
enum XPmReset {
    XILPM_RESET_PCIE_CFG = 1000,
    XILPM_RESET_PCIE_BRIDGE,
    XILPM_RESET_PCIE_CTRL,
    XILPM_RESET_DP,
    XILPM_RESET_SWDT_CRF,
    XILPM_RESET_AFI_FM5,
    XILPM_RESET_AFI_FM4,
    XILPM_RESET_AFI_FM3,
    XILPM_RESET_AFI_FM2,
    XILPM_RESET_AFI_FM1,
    XILPM_RESET_AFI_FM0,
    XILPM_RESET_GDMA,
    XILPM_RESET_GPU_PP1,
    XILPM_RESET_GPU_PP0,
    XILPM_RESET_GPU,
    XILPM_RESET_GT,
    XILPM_RESET_SATA,
    XILPM_RESET_ACPU3_PWRON,
    XILPM_RESET_ACPU2_PWRON,
    XILPM_RESET_ACPU1_PWRON,
```

```

XILPM_RESET_ACPU0_PWRON,
XILPM_RESET_APU_L2,
XILPM_RESET_ACPU3,
XILPM_RESET_ACPU2,
XILPM_RESET_ACPU1,
XILPM_RESET_ACPU0,
XILPM_RESET_DDR,
XILPM_RESET_APM_FPD,
XILPM_RESET_SOFT,
XILPM_RESET_GEM0,
XILPM_RESET_GEM1,
XILPM_RESET_GEM2,
XILPM_RESET_GEM3,
XILPM_RESET_QSPI,
XILPM_RESET_UART0,
XILPM_RESET_UART1,
XILPM_RESET_SPI0,
XILPM_RESET_SPI1,
XILPM_RESET_SDI00,
XILPM_RESET_SDI01,
XILPM_RESET_CAN0,
XILPM_RESET_CAN1,
XILPM_RESET_I2C0,
XILPM_RESET_I2C1,
XILPM_RESET_TTC0
XILPM_RESET_TTC1,
XILPM_RESET_TTC2,
XILPM_RESET_TTC3,
XILPM_RESET_SWDT_CRL,
XILPM_RESET_NAND,
XILPM_RESET_ADMA,
XILPM_RESET_GPIO,
XILPM_RESET_I0U_CC,
XILPM_RESET_TIMESTAMP,
XILPM_RESET_RPU_R50,
XILPM_RESET_RPU_R51,
XILPM_RESET_RPU_AMBA,
XILPM_RESET_OCM,
XILPM_RESET_RPU_PGE,
XILPM_RESET_USB0_CORERESSET,
XILPM_RESET_USB1_CORERESSET,
XILPM_RESET_USB0_HIBERRESET,

```



```

XILPM_RESET_USB1_HIBERRESET,
XILPM_RESET_USB0_APB,
XILPM_RESET_USB1_APB,
XILPM_RESET_IPI,
XILPM_RESET_APM_LPD,
XILPM_RESET_RTC,
XILPM_RESET_SYSMON,
XILPM_RESET_AFI_FM6,
XILPM_RESET_LPD_SWDT,
XILPM_RESET_FPD,
XILPM_RESET_RPU_DBG1,
XILPM_RESET_RPU_DBG0,
XILPM_RESET_DBG_LPD,
XILPM_RESET_DBG_FPD,
XILPM_RESET_APLL,
XILPM_RESET_DPLL,
XILPM_RESET_VPLL,
XILPM_RESET_IOPLL,
XILPM_RESET_RPLL,
XILPM_RESET_GP03_PL_0,
XILPM_RESET_GP03_PL_1,
XILPM_RESET_GP03_PL_2,
XILPM_RESET_GP03_PL_3,
XILPM_RESET_GP03_PL_4,
XILPM_RESET_GP03_PL_5,
XILPM_RESET_GP03_PL_6,
XILPM_RESET_GP03_PL_7,
XILPM_RESET_GP03_PL_8,
XILPM_RESET_GP03_PL_9,
XILPM_RESET_GP03_PL_10,
XILPM_RESET_GP03_PL_11,
XILPM_RESET_GP03_PL_12,
XILPM_RESET_GP03_PL_13,
XILPM_RESET_GP03_PL_14,
XILPM_RESET_GP03_PL_15,
XILPM_RESET_GP03_PL_16,
XILPM_RESET_GP03_PL_17,
XILPM_RESET_GP03_PL_18,
XILPM_RESET_GP03_PL_19,
XILPM_RESET_GP03_PL_20,
XILPM_RESET_GP03_PL_21,
XILPM_RESET_GP03_PL_22,

```

```
XILPM_RESET_GP03_PL_23,
XILPM_RESET_GP03_PL_24,
XILPM_RESET_GP03_PL_25,
XILPM_RESET_GP03_PL_26,
XILPM_RESET_GP03_PL_27,
XILPM_RESET_GP03_PL_28,
XILPM_RESET_GP03_PL_29,
XILPM_RESET_GP03_PL_30,
XILPM_RESET_GP03_PL_31,
};
```

XPm_Notifier struct

The `XPm_Notifier` struct is the structure to be passed in `XPm_RegisterNotifier`.

```
typedef struct XPm_Notifier {
    void (*const callback)(XPm_Notifier* const notifier);
    enum XPmNodeId node;
    enum XPmNotifyEvent event;
    u32 flags;
    volatile u32 oppoint;
    volatile u32 received;
    XPm_Notifier* next;
} XPm_Notifier;
```

Struct Members

Table A-2: Struct Members

Name	Description
callback	Custom callback handler to be called when the notification is received. The custom handler executes from the interrupt context; hence, it shall return quickly and must not block! (enables event-driven notifications),
node	Node argument (the node to receive notifications about).
event	Event argument (the event type to receive notifications about).
flags	Flags: Currently the flags only contain the wake option in bit0. - flags = 1: wake up on event - flags = 0: do not wake up (only notify if awake), no buffering or queueing will take place
oppoint	Operating point of node in question. Contains the value updated when the last event notification is received. User shall not modify this value while the notifier is registered.
received	How many times the notification has been received - to be used by application (enables polling). User shall not modify this value while the notifier is registered.
next	Pointer to next notifier in linked list. Must not be modified while the notifier is registered. User shall not ever modify this value.

XPm_NodeStatus struct

The `XPm_NodeStatus` struct is used to pass node status information.

```
typedef struct XPm_NodeStatus {
    u32 status;
    u32 requirements;
    u32 usage;
} XPm_NodeStatus;
```

Struct Members

Table A-3: Struct Members

Name	Description
<code>status</code>	Node power state.
<code>requirements</code>	Current requirements asserted on the node (slaves only).
<code>usage</code>	Usage information (which master is currently using the slave). This information is used for slave nodes only. It is encoded based on the IPI bits for the masters. If the respective bit is set, the corresponding master is currently using the node.

XiLPM Error Codes

Introduction

The following is a list of possible error codes returned by the PM API.

Table A-4: Error Codes and Explanations

Error Code	Explanation
<code>XST_FAILURE</code>	Power management controller has failed to comply with the request, because of a hardware/PMU-ROM failure or because the API cannot be processed in the given circumstances.
<code>XST_INVALID_PARAM</code>	An argument is either out-of-range or its value is not admissible in the respective API call.
<code>XST_NO_FEATURE</code>	The requested feature is not available for the selected PM slave.
<code>XST_PM_CONFLICT</code>	Conflicting requirements have been asserted when more than one PU is using the same PM slave.
<code>XST_PM_DOUBLE_REQ</code>	<code>XPm_RequestNode</code> : A PU has already been assigned access to a PM slave and has issued a duplicate request for that PM slave.
<code>XST_PM_INTERNAL</code>	Unexpected error in the PMU state machine. Should be reported as a bug.

Table A-4: Error Codes and Explanations (Cont'd)

Error Code	Explanation
XST_PM_INVALID_NODE	The API function does not apply to the node passed as argument.
XST_PM_NO_ACCESS	The PU does not have access to the requested node or operation.
XST_PM_ABORT_SUSPEND	The target PU has aborted suspend.

Appendix B:

Xilinx Standard C Libraries

Xilinx Standard C Libraries

Overview

The Xilinx® Software Development Kit (SDK) libraries and device drivers provide standard C library functions, as well as functions to access peripherals. The SDK libraries are automatically configured based on the Microprocessor Software Specification (MSS) file. These libraries and include files are saved in the current project lib and include directories, respectively. The -I and -L options of mb-gcc are used to add these directories to its library search paths.

Standard C Library (libc.a)

The standard C library, `libc.a`, contains the standard C functions compiled for the MicroBlaze™ processor or the Cortex A9 processor. You can find the header files corresponding to these C standard functions in the `<XILINX_SDK>/gnu/<processor>/<platform>/<processor-lib>/include` folder, where:

- `<XILINX_SDK>` is the Xilinx SDK installation path
- `<processor>` is ARM or MicroBlaze
- `<platform>` is Solaris (sol), Windows (nt), or Linux (lin)
- `<processor-lib>` is `arm-xilinx-eabi` or `microblaze-xilinx-elf`

The `libc` directories and functions are:

<code>_ansi.h</code>	<code>fastmath.h</code>	<code>machine/</code>	<code>reent.h</code>	<code>stdlib.h</code>	<code>utime.h</code>	<code>_syslist.h</code>	<code>fcntl.h</code>	<code>malloc.h</code>
<code>regdef.h</code>	<code>string.h</code>	<code>utmp.h</code>	<code>ar.h</code>	<code>float.h</code>	<code>math.h</code>	<code>setjmp.h</code>	<code>sys/</code>	<code>assert.h</code>
<code>grp.h</code>	<code>paths.h</code>	<code>signal.h</code>	<code>termios.h</code>	<code>ctype.h</code>	<code>ieeefp.h</code>	<code>process.h</code>	<code>stdarg.h</code>	<code>time.h</code>
<code>dirent.h</code>	<code>imits.h</code>	<code>pthread.h</code>	<code>stddef.h</code>	<code>nctrl.h</code>	<code>errno.h</code>	<code>locale.h</code>	<code>pwd.h</code>	<code>stdio.h</code>
<code>unistd.h</code>								

Programs accessing standard C library functions must be compiled as follows:

- For MicroBlaze processors:

```
mb-gcc <C files>
```

- For Cortex A9 processors:

```
arm-xilinx-eabi-gcc <C files>
```

The `libc` library is included automatically. For programs that access `libm` math functions, specify the `lm` option. For more information on the C runtime library, see *MicroBlaze Processor Reference Guide* (UG081).

Xilinx C Library (libxil.a)

The Xilinx C library, `libxil.a`, contains the following object files for the MicroBlaze processor embedded processor:

- `_exception_handler.o`
- `_interrupt_handler.o`
- `_program_clean.o`
- `_program_init.o`

Default exception and interrupt handlers are provided. The `libxil.a` library is included automatically. Programs accessing Xilinx C library functions must be compiled as follows:

```
mb-gcc <C files>
```

Memory Management Functions

The MicroBlaze processor and Cortex A9 processor C libraries support the standard memory management functions such as `malloc()`, `calloc()`, and `free()`. Dynamic memory allocation provides memory from the program heap. The heap pointer starts at low memory and grows toward high memory. The size of the heap cannot be increased at runtime. Therefore an appropriate value must be provided for the heap size at compile time. The `malloc()` function requires the heap to be at least 128 bytes in size to be able to allocate memory dynamically (even if the dynamic requirement is less than 128 bytes).

Note

The return value of `malloc` must always be checked to ensure that it could actually allocate the memory requested.

Arithmetic Operations

Software implementations of integer and floating point arithmetic is available as library routines in `libgcc.a` for both processors. The compiler for both the processors inserts calls to these routines in the code produced, in case the hardware does not support the arithmetic primitive with an instruction.

MicroBlaze Processor

Details of the software implementations of integer and floating point arithmetic for MicroBlaze processors are listed below:

Integer Arithmetic

By default, integer multiplication is done in software using the library function `__mulsi3`. Integer multiplication is done in hardware if the `-mno-xl-soft-mul mb-gcc` option is specified.

Integer divide and mod operations are done in software using the library functions `__divsi3` and `__modsi3`. The MicroBlaze processor can also be customized to use a hard divider, in which case the `div` instruction is used in place of the `__divsi3` library routine.

Double precision multiplication, division and mod functions are carried out by the library functions `__muldi3`, `__divdi3`, and `__moddi3` respectively.

The unsigned version of these operations correspond to the signed versions described above, but are prefixed with an `__u` instead of `__`.

Floating Point Arithmetic

All floating point addition, subtraction, multiplication, division, and conversions are implemented using software functions in the C library.

Thread Safety

The standard C library provided with SDK is not built for a multi-threaded environment. STDIO functions like `printf()`, `scanf()` and memory management functions like `malloc()` and `free()` are common examples of functions that are not thread-safe. When using the C library in a multi-threaded environment, proper mutual exclusion techniques must be used to protect thread unsafe functions.

Modules

- [Input/Output Functions](#)

Input/Output Functions

Overview

The SDK libraries contains standard C functions for I/O, such as `printf` and `scanf`. These functions are large and might not be suitable for embedded processors. The prototypes for these functions are available in the `stdio.h` file.

Note

The C standard I/O routines such as `printf`, `scanf`, `vfprintf` are, by default, line buffered. To change the buffering scheme to no buffering, you must call `setvbuf` appropriately. For example:

```
setvbuf (stdout, NULL, _IONBF, 0);
```

These Input/Output routines require that a newline is terminated with both a CR and LF. Ensure that your terminal CR/LF behavior corresponds to this requirement.

For more information on setting the standard input and standard output devices for a system, see *Embedded System Tools Reference Manual* (UG1043). In addition to the standard C functions, the SDK processors library provides the following smaller I/O functions:

Functions

- void `print` (char *)
- void `putnum` (int)
- void `xil_printf` (const *char ctrl1,...)

Function Documentation

void print (char *)

This function prints a string to the peripheral designated as standard output in the Microprocessor Software Specification (MSS) file. This function outputs the passed string as is and there is no interpretation of the string passed. For example, a `\n` passed is interpreted as a new line character and not as a carriage return and a new line as is the case with ANSI C `printf` function.

void putnum (int)

This function converts an integer to a hexadecimal string and prints it to the peripheral designated as standard output in the MSS file.

void xil_printf (const *char ctrl1, ...)

`xil_printf()` is a light-weight implementation of `printf`. It is much smaller in size (only 1 Kb). It does not have support for floating point numbers. `xil_printf()` also does not support printing of long (such as 64-bit) numbers.

About format string support:

The format string is composed of zero or more directives: ordinary characters (not %), which are copied unchanged to the output stream; and conversion specifications, each of which results in fetching zero or more subsequent arguments. Each conversion specification is introduced by the character %, and ends with a conversion specifier.

In between there can be (in order) zero or more flags, an optional minimum field width and an optional precision. Supported flag characters are:

The character % is followed by zero or more of the following flags:

- 0 The value should be zero padded. For d, x conversions, the converted value is padded on the left with zeros rather than blanks. If the 0 and - flags both appear, the 0 flag is ignored.
- - The converted value is to be left adjusted on the field boundary. (The default is right justification.) Except for n conversions, the converted value is padded on the right with blanks, rather than on the left with blanks or zeros. A - overrides a 0 if both are given.

About supported field widths

Field widths are represented with an optional decimal digit string (with a nonzero in the first digit) specifying a minimum field width. If the converted value has fewer characters than the field width, it is padded with spaces on the left (or right, if the left-adjustment flag has been given). The supported conversion specifiers are:

- d The int argument is converted to signed decimal notation.
- l The int argument is converted to a signed long notation.

- x The unsigned int argument is converted to unsigned hexadecimal notation. The letters abcdef are used for x conversions.
- c The int argument is converted to an unsigned char, and the resulting character is written.
- s The const char* argument is expected to be a pointer to an array of character type (pointer to a string).

Characters from the array are written up to (but not including) a terminating NULL character; if a precision is specified, no more than the number specified are written. If a precision s given, no null character need be present; if the precision is not specified, or is greater than the size of the array, the array must contain a terminating NULL character.

Appendix C:

Standalone v7.0

Xilinx Hardware Abstraction Layer API

Overview

This section describes the Xilinx® Hardware Abstraction Layer API, These APIs are applicable for all processors supported by Xilinx.

Modules

- [Assert APIs](#)
 - [IO interfacing APIs](#)
 - [Definitions for available xilinx platforms](#)
 - [Data types for Xilinx Software IP Cores](#)
 - [Customized APIs for memory operations](#)
 - [Xilinx software status codes](#)
 - [Test utilities for memory and caches](#)
-

Assert APIs

Overview

The `xil_assert.h` file contains the assert related functions.

Macros

- `#define Xil\_AssertVoid(Expression)`
- `#define Xil\_AssertNonvoid(Expression)`
- `#define Xil\_AssertVoidAlways()`
- `#define Xil\_AssertNonvoidAlways()`

Typedefs

- `typedef void(* Xil\_AssertCallback) (const char8 *File, s32 Line)`

Functions

- void [Xil_Assert](#) (const [char8](#) *File, s32 Line)
- void [XNullHandler](#) (void *NullParameter)
- void [Xil_AssertSetCallback](#) ([Xil_AssertCallback](#) Routine)

Variables

- u32 [Xil_AssertStatus](#)
- s32 [Xil_AssertWait](#)

Macro Definition Documentation

#define Xil_AssertVoid(*Expression*)

This assert macro is to be used for void functions. This in conjunction with the [Xil_AssertWait](#) boolean can be used to accomodate tests so that asserts which fail allow execution to continue.

Parameters

<i>Expression</i>	expression to be evaluated. If it evaluates to false, the assert occurs.
-------------------	--

Returns

Returns void unless the [Xil_AssertWait](#) variable is true, in which case no return is made and an infinite loop is entered.

#define Xil_AssertNonvoid(*Expression*)

This assert macro is to be used for functions that do return a value. This in conjunction with the [Xil_AssertWait](#) boolean can be used to accomodate tests so that asserts which fail allow execution to continue.

Parameters

<i>Expression</i>	expression to be evaluated. If it evaluates to false, the assert occurs.
-------------------	--

Returns

Returns 0 unless the [Xil_AssertWait](#) variable is true, in which case no return is made and an infinite loop is entered.

#define Xil_AssertVoidAlways()

Always assert. This assert macro is to be used for void functions. Use for instances where an assert should always occur.

Returns

Returns void unless the Xil_AssertWait variable is true, in which case no return is made and an infinite loop is entered.

#define Xil_AssertNonvoidAlways()

Always assert. This assert macro is to be used for functions that do return a value. Use for instances where an assert should always occur.

Returns

Returns void unless the Xil_AssertWait variable is true, in which case no return is made and an infinite loop is entered.

Typedef Documentation

typedef void(* Xil_AssertCallback) (const char8 *File, s32 Line)

This data type defines a callback to be invoked when an assert occurs. The callback is invoked only when asserts are enabled

Function Documentation

void Xil_Assert (const char8 * File, s32 Line)

Implement assert. Currently, it calls a user-defined callback function if one has been set. Then, it potentially enters an infinite loop depending on the value of the Xil_AssertWait variable.

Parameters

<i>file</i>	filename of the source
<i>line</i>	linenumber within File

Returns

None.

Note

None.

void XNullHandler (void * *NullParameter*)

Null handler function. This follows the XInterruptHandler signature for interrupt handlers. It can be used to assign a null handler (a stub) to an interrupt controller vector table.

Parameters

<i>NullParameter</i>	arbitrary void pointer and not used.
----------------------	--------------------------------------

Returns

None.

Note

None.

void Xil_AssertSetCallback (Xil_AssertCallback *Routine*)

Set up a callback function to be invoked when an assert occurs. If a callback is already installed, then it will be replaced.

Parameters

<i>routine</i>	callback to be invoked when an assert is taken
----------------	--

Returns

None.

Note

This function has no effect if NDEBUG is set

Variable Documentation

u32 Xil_AssertStatus

This variable allows testing to be done easier with asserts. An assert sets this variable such that a driver can evaluate this variable to determine if an assert occurred.

s32 Xil_AssertWait

This variable allows the assert functionality to be changed for testing such that it does not wait infinitely. Use the debugger to disable the waiting during testing of asserts.

IO interfacing APIs

Overview

The `xil_io.h` file contains the interface for the general IO component, which encapsulates the Input/Output functions for processors that do not require any special I/O handling.

Functions

- u16 [Xil_EndianSwap16](#) (u16 Data)
- u32 [Xil_EndianSwap32](#) (u32 Data)
- static INLINE u8 [Xil_In8](#) (UINTPTR Addr)
- static INLINE u16 [Xil_In16](#) (UINTPTR Addr)
- static INLINE u32 [Xil_In32](#) (UINTPTR Addr)
- static INLINE u64 [Xil_In64](#) (UINTPTR Addr)
- static INLINE void [Xil_Out8](#) (UINTPTR Addr, u8 Value)
- static INLINE void [Xil_Out16](#) (UINTPTR Addr, u16 Value)
- static INLINE void [Xil_Out32](#) (UINTPTR Addr, u32 Value)
- static INLINE void [Xil_Out64](#) (UINTPTR Addr, u64 Value)
- static INLINE u16 [Xil_In16LE](#) (UINTPTR Addr)
- static INLINE u32 [Xil_In32LE](#) (UINTPTR Addr)
- static INLINE void [Xil_Out16LE](#) (UINTPTR Addr, u16 Value)
- static INLINE void [Xil_Out32LE](#) (UINTPTR Addr, u32 Value)
- static INLINE u16 [Xil_In16BE](#) (UINTPTR Addr)
- static INLINE u32 [Xil_In32BE](#) (UINTPTR Addr)
- static INLINE void [Xil_Out16BE](#) (UINTPTR Addr, u16 Value)
- static INLINE void [Xil_Out32BE](#) (UINTPTR Addr, u32 Value)

Function Documentation



u16 Xil_EndianSwap16 (u16 Data)

Perform a 16-bit endian conversion.

Parameters

<i>Data</i>	16 bit value to be converted
-------------	------------------------------

Returns

converted value.

u32 Xil_EndianSwap32 (u32 Data)

Perform a 32-bit endian conversion.

Parameters

<i>Data</i>	32 bit value to be converted
-------------	------------------------------

Returns

converted value.

static INLINE u8 Xil_In8 (UINTPTR Addr) [static]

Performs an input operation for an 8-bit memory location by reading from the specified address and returning the Value read from that address.

Parameters

<i>Addr</i>	contains the address to perform the input operation at.
-------------	---

Returns

The Value read from the specified input address.

Note

None.

static INLINE u16 Xil_In16 (UINTPTR Addr) [static]

Performs an input operation for a 16-bit memory location by reading from the specified address and returning the Value read from that address.

Parameters

<i>Addr</i>	contains the address to perform the input operation at.
-------------	---

Returns

The Value read from the specified input address.

Note

None.

static INLINE u32 Xil_In32 (UINTPTR Addr) [static]

Performs an input operation for a 32-bit memory location by reading from the specified address and returning the Value read from that address.

Parameters

<i>Addr</i>	contains the address to perform the input operation at.
-------------	---

Returns

The Value read from the specified input address.

Note

None.

static INLINE u64 Xil_In64 (UINTPTR Addr) [static]

Performs an input operation for a 64-bit memory location by reading the specified Value to the the specified address.

Parameters

<i>Addr</i>	contains the address to perform the output operation at.
<i>Value</i>	contains the Value to be output at the specified address.

Returns

None.

Note

None.

static INLINE void Xil_Out8 (UINTPTR *Addr*, u8 *Value*) [static]

Performs an output operation for an 8-bit memory location by writing the specified Value to the the specified address.

Parameters

<i>Addr</i>	contains the address to perform the output operation at.
<i>Value</i>	contains the Value to be output at the specified address.

Returns

None.

Note

None.

static INLINE void Xil_Out16 (UINTPTR *Addr*, u16 *Value*) [static]

Performs an output operation for a 16-bit memory location by writing the specified Value to the the specified address.

Parameters

<i>Addr</i>	contains the address to perform the output operation at.
<i>Value</i>	contains the Value to be output at the specified address.

Returns

None.

Note

None.

static INLINE void Xil_Out32 (UINTPTR *Addr*, u32 *Value*) [static]

Performs an output operation for a 32-bit memory location by writing the specified Value to the the specified address.

Parameters

<i>Addr</i>	contains the address to perform the output operation at.
<i>Value</i>	contains the Value to be output at the specified address.

Returns

None.

Note

None.

static INLINE void Xil_Out64 (UINTPTR Addr, u64 Value) [static]

Performs an output operation for a 64-bit memory location by writing the specified Value to the the specified address.

Parameters

<i>Addr</i>	contains the address to perform the output operation at.
<i>Value</i>	contains the Value to be output at the specified address.

Returns

None.

Note

None.

static INLINE u16 Xil_In16LE (UINTPTR Addr) [static]

Perform a little-endian input operation for a 16-bit memory location by reading from the specified address and returning the value read from that address.

Parameters

<i>Addr</i>	contains the address at which to perform the input operation.
-------------	---

Returns

The value read from the specified input address with the proper endianness. The return value has the same endianness as that of the processor. For example, if the processor is big-endian, the return value is the byte-swapped value read from the address.

static INLINE u32 Xil_In32LE (UINTPTR Addr) [static]

Perform a little-endian input operation for a 32-bit memory location by reading from the specified address and returning the value read from that address.

Parameters

<i>Addr</i>	contains the address at which to perform the input operation.
-------------	---

Returns

The value read from the specified input address with the proper endianness. The return value has the same endianness as that of the processor. For example, if the processor is big-endian, the return value is the byte-swapped value read from the address.

static INLINE void Xil_Out16LE (UINTPTR Addr, u16 Value) [static]

Perform a little-endian output operation for a 16-bit memory location by writing the specified value to the specified address.

Parameters

<i>Addr</i>	contains the address at which to perform the output operation.
<i>Value</i>	contains the value to be output at the specified address. The value has the same endianness as that of the processor. For example, if the processor is big-endian, the byteswapped value is written to the address.

static INLINE void Xil_Out32LE (UINTPTR Addr, u32 Value) [static]

Perform a little-endian output operation for a 32-bit memory location by writing the specified value to the specified address.

Parameters

<i>Addr</i>	contains the address at which to perform the output operation.
<i>Value</i>	contains the value to be output at the specified address. The value has the same endianness as that of the processor. For example, if the processor is big-endian, the byteswapped value is written to the address.

static INLINE u16 Xil_In16BE (UINTPTR Addr) [static]

Perform an big-endian input operation for a 16-bit memory location by reading from the specified address and returning the value read from that address.

Parameters

<i>Addr</i>	contains the address at which to perform the input operation.
-------------	---

Returns

The value read from the specified input address with the proper endianness. The return value has the same endianness as that of the processor. For example, if the processor is little-endian, the return value is the byte-swapped value read from the address.

static INLINE u32 Xil_In32BE (UINTPTR Addr) [static]

Perform a big-endian input operation for a 32-bit memory location by reading from the specified address and returning the value read from that address.

Parameters

<i>Addr</i>	contains the address at which to perform the input operation.
-------------	---

Returns

The value read from the specified input address with the proper endianness. The return value has the same endianness as that of the processor. For example, if the processor is little-endian, the return value is the byte-swapped value read from the address.

static INLINE void Xil_Out16BE (UINTPTR Addr, u16 Value) [static]

Perform a big-endian output operation for a 16-bit memory location by writing the specified value to the specified address.

Parameters

<i>Addr</i>	contains the address at which to perform the output operation.
<i>Value</i>	contains the value to be output at the specified address. The value has the same endianness as that of the processor. For example, if the processor is little-endian, the byteswapped value is written to the address.

static INLINE void Xil_Out32BE (UINTPTR Addr, u32 Value) [static]

Perform a big-endian output operation for a 32-bit memory location by writing the specified value to the specified address.

Parameters

<i>Addr</i>	contains the address at which to perform the output operation.
<i>Value</i>	contains the value to be output at the specified address. The value has the same endianness as that of the processor. For example, if the processor is little-endian, the byteswapped value is written to the address.

Definitions for available xilinx platforms

Overview

The `xplatform_info.h` file contains definitions for various available Xilinx® platforms.

Functions

- u32 [XGetPlatform_Info](#) ()
- u32 [XGetPSVersion_Info](#) ()
- u32 [XGet_Zynq_UltraMp_Platform_info](#) ()

Function Documentation

u32 XGetPlatform_Info ()

This API is used to provide information about platform.

Parameters

<i>None.</i>	
--------------	--

Returns

The information about platform defined in `xplatform_info.h`



u32 XGetPSVersion_Info ()

This API is used to provide information about PS Silicon version.

Parameters

None.	
-------	--

Returns

The information about PS Silicon version.

u32 XGet_Zynq_UltraMp_Platform_info ()

This API is used to provide information about zynq ultrascale MP platform.

Parameters

None.	
-------	--

Returns

The information about zynq ultrascale MP platform defined in xplatform_info.h

Data types for Xilinx Software IP Cores

Overview

The `xil_types.h` file contains basic types for Xilinx® software IP cores. These data types are applicable for all processors supported by Xilinx.

Macros

- #define `XIL_COMPONENT_IS_READY`
- #define `XIL_COMPONENT_IS_STARTED`

New types

New simple types.

- typedef uint8_t `u8`
- typedef uint16_t `u16`
- typedef uint32_t `u32`
- typedef char `char8`
- typedef int8_t `s8`
- typedef int16_t `s16`

- typedef int32_t **s32**
- typedef int64_t **s64**
- typedef uint64_t **u64**
- typedef int **sint32**
- typedef intptr_t **INTPTR**
- typedef uintptr_t **UINTPTR**
- typedef ptrdiff_t **PTRDIFF**
- typedef long **LONG**
- typedef unsigned long **ULONG**
- typedef void(* [XInterruptHandler](#)) (void *InstancePtr)
- typedef void(* [XExceptionHandler](#)) (void *InstancePtr)
- #define **__XUINT64__**
- #define [XUINT64_MSW](#)(x)
- #define [XUINT64_LSW](#)(x)
- #define **ULONG64_HI_MASK**
- #define **ULONG64_LO_MASK**
- #define [UPPER_32_BITS](#)(n)
- #define [LOWER_32_BITS](#)(n)

Macro Definition Documentation

#define XIL_COMPONENT_IS_READY

component has been initialized

#define XIL_COMPONENT_IS_STARTED

component has been started

#define XUINT64_MSW(x)

Return the most significant half of the 64 bit data type.

Parameters

x	is the 64 bit word.
---	---------------------

Returns

The upper 32 bits of the 64 bit word.

#define XUINT64_LSW(x)

Return the least significant half of the 64 bit data type.

Parameters

<i>x</i>	is the 64 bit word.
----------	---------------------

Returns

The lower 32 bits of the 64 bit word.

#define UPPER_32_BITS(n)

return bits 32-63 of a number

Parameters

<i>n</i>	: the number we're accessing
----------	------------------------------

Returns

bits 32-63 of number

Note

A basic shift-right of a 64- or 32-bit quantity. Use this to suppress the "right shift count >= width of type" warning when that quantity is 32-bits.

#define LOWER_32_BITS(n)

return bits 0-31 of a number

Parameters

<i>n</i>	: the number we're accessing
----------	------------------------------

Returns

bits 0-31 of number

Typedef Documentation

typedef uint8_t u8

guarded against xbasic_types.h.

typedef char char8

xbasic_types.h does not typedef s* or u64

typedef void(* XInterruptHandler) (void *InstancePtr)

This data type defines an interrupt handler for a device. The argument points to the instance of the component

typedef void(* XExceptionHandler) (void *InstancePtr)

This data type defines an exception handler for a processor. The argument points to the instance of the component

Customized APIs for memory operations

Overview

The `xil_mem.h` file contains prototypes for function related to memory operations. These APIs are applicable for all processors supported by Xilinx®.

Functions

- void [Xil_MemCpy](#) (void *dst, const void *src, u32 cnt)

Function Documentation

void Xil_MemCpy (void * *dst*, const void * *src*, u32 *cnt*)

This function copies memory from one location to other.

Parameters

<i>dst</i>	pointer pointing to destination memory
<i>src</i>	pointer pointing to source memory
<i>cnt</i>	32 bit length of bytes to be copied

Xilinx software status codes

Overview

The `xstatus.h` file contains Xilinx® software status codes. Status codes have their own data type called `int`. These codes are used throughout the Xilinx device drivers.

Test utilities for memory and caches

Overview

The `xil_testcache.h`, `xil_testio.h` and the `xil_testmem.h` files contain utility functions to test cache and memory. Details of supported tests and subtests are listed below.

- **Cache test** : `xil_testcache.h` contains utility functions to test cache.
- **I/O test** : The `Xil_testio.h` file contains endian related memory IO functions. A subset of the memory tests can be selected or all of the tests can be run in order. If there is an error detected by a subtest, the test stops and the failure code is returned. Further tests are not run even if all of the tests are selected.
- **Memory test** : The `xil_testmem.h` file contains utility functions to test memory. A subset of the memory tests can be selected or all of the tests can be run in order. If there is an error detected by a subtest, the test stops and the failure code is returned. Further tests are not run even if all of the tests are selected. Following are descriptions of Memory test subtests:

- `XIL_TESTMEM_ALLMEMTESTS`: Runs all of the subtests.
- `XIL_TESTMEM_INCREMENT`: Incrementing Value Test. This test starts at `XIL_TESTMEM_INIT_VALUE` and uses the incrementing value as the test value for memory.
- `XIL_TESTMEM_WALKONES`: Walking Ones Test. This test uses a walking 1 as the test value for memory.

```
location 1 = 0x00000001
location 2 = 0x00000002
...
```

- `XIL_TESTMEM_WALKZEROS`: Walking Zero's Test. This test uses the inverse value of the walking ones test as the test value for memory.

```
location 1 = 0xFFFFFFFF
location 2 = 0xFFFFFFFFD
...
```

- `XIL_TESTMEM_INVERSEADDR`: Inverse Address Test. This test uses the inverse of the address of the location under test as the test value for memory.
- `XIL_TESTMEM_FIXEDPATTERN`: Fixed Pattern Test. This test uses the provided patterns as the test value for memory. If zero is provided as the pattern the test uses `0xDEADBEEF`.



WARNING: The tests are **DESTRUCTIVE**. Run before any initialized memory spaces have been set up. The address provided to the memory tests is not checked for validity except for the NULL case. It is possible to provide a code-space pointer for this test to start with and ultimately destroy executable code causing random failures.

Note

Used for spaces where the address range of the region is smaller than the data width. If the memory range is greater than 2 ** width, the patterns used in XIL_TESTMEM_WALKONES and XIL_TESTMEM_WALKZEROS will repeat on a boundary of a power of two making it more difficult to detect addressing errors. The XIL_TESTMEM_INCREMENT and XIL_TESTMEM_INVERSEADDR tests suffer the same problem. Ideally, if large blocks of memory are to be tested, break them up into smaller regions of memory to allow the test patterns used not to repeat over the region tested.

Functions

- s32 [Xil_TestIO8](#) (u8 *Addr, s32 Length, u8 Value)
- s32 [Xil_TestIO16](#) (u16 *Addr, s32 Length, u16 Value, s32 Kind, s32 Swap)
- s32 [Xil_TestIO32](#) (u32 *Addr, s32 Length, u32 Value, s32 Kind, s32 Swap)
- s32 [Xil_TestMem32](#) (u32 *Addr, u32 Words, u32 Pattern, u8 Subtest)
- s32 [Xil_TestMem16](#) (u16 *Addr, u32 Words, u16 Pattern, u8 Subtest)
- s32 [Xil_TestMem8](#) (u8 *Addr, u32 Words, u8 Pattern, u8 Subtest)

Memory subtests

- #define [XIL_TESTMEM_ALLMEMTESTS](#)
- #define [XIL_TESTMEM_INCREMENT](#)
- #define [XIL_TESTMEM_WALKONES](#)
- #define [XIL_TESTMEM_WALKZEROS](#)
- #define [XIL_TESTMEM_INVERSEADDR](#)
- #define [XIL_TESTMEM_FIXEDPATTERN](#)
- #define [XIL_TESTMEM_MAXTEST](#)

Macro Definition Documentation

#define XIL_TESTMEM_ALLMEMTESTS

See the detailed description of the subtests in the file description.

Function Documentation

s32 Xil_TestIO8 (u8 * *Addr*, s32 *Length*, u8 *Value*)

Perform a destructive 8-bit wide register IO test where the register is accessed using Xil_Out8 and Xil_In8, and comparing the written values by reading them back.

Parameters

<i>Addr</i>	a pointer to the region of memory to be tested.
<i>Length</i>	Length of the block.
<i>Value</i>	constant used for writting the memory.

Returns

- -1 is returned for a failure
- 0 is returned for a pass

s32 Xil_TestIO16 (u16 * *Addr*, s32 *Length*, u16 *Value*, s32 *Kind*, s32 *Swap*)

Perform a destructive 16-bit wide register IO test. Each location is tested by sequentially writing a 16-bit wide register, reading the register, and comparing value. This function tests three kinds of register IO functions, normal register IO, little-endian register IO, and big-endian register IO. When testing little/big-endian IO, the function performs the following sequence, Xil_Out16LE/Xil_Out16BE, Xil_In16, Compare In-Out values, Xil_Out16, Xil_In16LE/Xil_In16BE, Compare In-Out values. Whether to swap the read-in value before comparing is controlled by the 5th argument.

Parameters

<i>Addr</i>	a pointer to the region of memory to be tested.
<i>Length</i>	Length of the block.
<i>Value</i>	constant used for writting the memory.
<i>Kind</i>	Type of test. Acceptable values are: XIL_TESTIO_DEFAULT, XIL_TESTIO_LE, XIL_TESTIO_BE.
<i>Swap</i>	indicates whether to byte swap the read-in value.

Returns

- -1 is returned for a failure
- 0 is returned for a pass

s32 Xil_TestIO32 (u32 * *Addr*, s32 *Length*, u32 *Value*, s32 *Kind*, s32 *Swap*)

Perform a destructive 32-bit wide register IO test. Each location is tested by sequentially writing a 32-bit wide register, reading the register, and comparing value. This function tests three kinds of register IO functions, normal register IO, little-endian register IO, and big-endian register IO. When testing little/big-endian IO, the function perform the following sequence, Xil_Out32LE/ Xil_Out32BE, Xil_In32, Compare, Xil_Out32, Xil_In32LE/Xil_In32BE, Compare. Whether to swap the read-in value *before comparing is controlled by the 5th argument.

Parameters

<i>Addr</i>	a pointer to the region of memory to be tested.
<i>Length</i>	Length of the block.
<i>Value</i>	constant used for writing the memory.
<i>Kind</i>	type of test. Acceptable values are: XIL_TESTIO_DEFAULT, XIL_TESTIO_LE, XIL_TESTIO_BE.
<i>Swap</i>	indicates whether to byte swap the read-in value.

Returns

- -1 is returned for a failure
- 0 is returned for a pass

s32 Xil_TestMem32 (u32 * *Addr*, u32 *Words*, u32 *Pattern*, u8 *Subtest*)

Perform a destructive 32-bit wide memory test.

Parameters

<i>Addr</i>	pointer to the region of memory to be tested.
<i>Words</i>	length of the block.
<i>Pattern</i>	constant used for the constant pattern test, if 0, 0xDEADBEEF is used.
<i>Subtest</i>	test type selected. See xil_testmem.h for possible values.

Returns

- 0 is returned for a pass
- 1 is returned for a failure

Note

Used for spaces where the address range of the region is smaller than the data width. If the memory range is greater than 2 ** Width, the patterns used in XIL_TESTMEM_WALKONES and XIL_TESTMEM_WALKZEROS will repeat on a boundry of a power of two making it more difficult to detect addressing errors. The XIL_TESTMEM_INCREMENT and XIL_TESTMEM_INVERSEADDR tests suffer the same problem. Ideally, if large blocks of memory are to be tested, break them up into smaller regions of memory to allow the test patterns used not to repeat over the region tested.

s32 Xil_TestMem16 (u16 * *Addr*, u32 *Words*, u16 *Pattern*, u8 *Subtest*)

Perform a destructive 16-bit wide memory test.

Parameters

<i>Addr</i>	pointer to the region of memory to be tested.
<i>Words</i>	length of the block.
<i>Pattern</i>	constant used for the constant Pattern test, if 0, 0xDEADBEEF is used.
<i>Subtest</i>	type of test selected. See xil_testmem.h for possible values.

Returns

- -1 is returned for a failure
- 0 is returned for a pass

Note

Used for spaces where the address range of the region is smaller than the data width. If the memory range is greater than 2 ** Width, the patterns used in XIL_TESTMEM_WALKONES and XIL_TESTMEM_WALKZEROS will repeat on a boundry of a power of two making it more difficult to detect addressing errors. The XIL_TESTMEM_INCREMENT and XIL_TESTMEM_INVERSEADDR tests suffer the same problem. Ideally, if large blocks of memory are to be tested, break them up into smaller regions of memory to allow the test patterns used not to repeat over the region tested.

s32 Xil_TestMem8 (u8 * *Addr*, u32 *Words*, u8 *Pattern*, u8 *Subtest*)

Perform a destructive 8-bit wide memory test.

Parameters

<i>Addr</i>	pointer to the region of memory to be tested.
<i>Words</i>	length of the block.
<i>Pattern</i>	constant used for the constant pattern test, if 0, 0xDEADBEEF is used.
<i>Subtest</i>	type of test selected. See xil_testmem.h for possible values.

Returns

- -1 is returned for a failure
- 0 is returned for a pass

Note

Used for spaces where the address range of the region is smaller than the data width. If the memory range is greater than $2 \times \text{Width}$, the patterns used in `XIL_TESTMEM_WALKONES` and `XIL_TESTMEM_WALKZEROS` will repeat on a boundary of a power of two making it more difficult to detect addressing errors. The `XIL_TESTMEM_INCREMENT` and `XIL_TESTMEM_INVERSEADDR` tests suffer the same problem. Ideally, if large blocks of memory are to be tested, break them up into smaller regions of memory to allow the test patterns used not to repeat over the region tested.

Microblaze Processor API

Overview

This section provides a linked summary and detailed descriptions of the Microblaze Processor APIs.

Modules

- [Microblaze Pseudo-asm Macros and Interrupt handling APIs](#)
 - [Microblaze exception APIs](#)
 - [Microblaze Processor Cache APIs](#)
 - [MicroBlaze Processor FSL Macros](#)
 - [Microblaze PVR access routines and macros](#)
 - [Sleep Routines for Microblaze](#)
-

Microblaze Pseudo-asm Macros and Interrupt handling APIs

Overview

Standalone includes macros to provide convenient access to various registers in the MicroBlaze processor. Some of these macros are very useful within exception handlers for retrieving information about the exception. Also, the interrupt handling functions help manage interrupt handling on MicroBlaze processor devices. To use these functions, include the header file `mb_interface.h` in your source code.

Functions

- void [microblaze_register_handler](#) (XInterruptHandler Handler, void *DataPtr)
- void [microblaze_register_exception_handler](#) (u32 ExceptionId, [Xil_ExceptionHandler](#) Handler, void *DataPtr)

Microblaze pseudo-asm macros

The following is a summary of the MicroBlaze processor pseudo-asm macros.

- #define [mfgpr](#)(rn)
- #define [mfmsr](#)()
- #define [mfear](#)()
- #define [mfeare](#)()
- #define [mfesr](#)()
- #define [mffsr](#)()

Macro Definition Documentation

#define [mfgpr](#)(*rn*)

Return value from the general purpose register (GPR) rn.

Parameters

<i>rn</i>	General purpose register to be read.
-----------	--------------------------------------

#define [mfmsr](#)()

Return the current value of the MSR.

Parameters

<i>None</i>	
-------------	--

#define [mfear](#)()

Return the current value of the Exception Address Register (EAR).

Parameters

<i>None</i>	
-------------	--

#define [mfesr](#)()

Return the current value of the Exception Status Register (ESR).

Parameters

<i>None</i>	
-------------	--

#define mffsr()

Return the current value of the Floating Point Status (FPS).

Parameters

None	
------	--

Function Documentation

void microblaze_register_handler (XInterruptHandler *Handler*, void * *DataPtr*)

Registers a top-level interrupt handler for the MicroBlaze. The argument provided in this call as the *DataPtr* is used as the argument for the handler when it is called.

Parameters

<i>Handler</i>	Top level handler.
<i>DataPtr</i>	a reference to data that will be passed to the handler when it gets called.

Returns

None.

void microblaze_register_exception_handler (u32 *ExceptionId*, Xil_ExceptionHandler *Handler*, void * *DataPtr*)

Registers an exception handler for the MicroBlaze. The argument provided in this call as the *DataPtr* is used as the argument for the handler when it is called.

Parameters

<i>ExceptionId</i>	is the id of the exception to register this handler for.
<i>Top</i>	level handler.
<i>DataPtr</i>	is a reference to data that will be passed to the handler when it gets called.

Returns

None.

Note

None.

Microblaze exception APIs

Overview

The `xil_exception.h` file, available in the `<install-directory>/src/microblaze` folder, contains Microblaze specific exception related APIs and macros. Application programs can use these APIs for various exception related operations. For example, enable exception, disable exception, register exception handler.

Note

To use exception related functions, `xil_exception.h` must be added in source code

Data Structures

- struct [MB_ExceptionVectorTableEntry](#)

Typedefs

- typedef void(* [Xil_ExceptionHandler](#)) (void *Data)
- typedef void(* [XInterruptHandler](#)) (void *InstancePtr)

Functions

- void [Xil_ExceptionInit](#) (void)
- void [Xil_ExceptionEnable](#) (void)
- void [Xil_ExceptionDisable](#) (void)
- void [Xil_ExceptionRegisterHandler](#) (u32 Id, [Xil_ExceptionHandler](#) Handler, void *Data)
- void [Xil_ExceptionRemoveHandler](#) (u32 Id)

Data Structure Documentation

struct [MB_ExceptionVectorTableEntry](#)

Currently HAL is an augmented part of standalone BSP, so the old definition of [MB_ExceptionVectorTableEntry](#) is used here.

Typedef Documentation

typedef void(* Xil_ExceptionHandler) (void *Data)

This typedef is the exception handler function.

typedef void(* XInterruptHandler) (void *InstancePtr)

This data type defines an interrupt handler for a device. The argument points to the instance of the component

Function Documentation

void Xil_ExceptionInit (void)

Initialize exception handling for the processor. The exception vector table is setup with the stub handler for all exceptions.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

void Xil_ExceptionEnable (void)

Enable Exceptions.

Returns

None.

void Xil_ExceptionDisable (void)

Disable Exceptions.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

void Xil_ExceptionRegisterHandler (u32 *Id*, Xil_ExceptionHandler *Handler*, void * *Data*)

Makes the connection between the Id of the exception source and the associated handler that is to run when the exception is recognized. The argument provided in this call as the DataPtr is used as the argument for the handler when it is called.

Parameters

<i>Id</i>	contains the 32 bit ID of the exception source and should be XIL_EXCEPTION_INT or be in the range of 0 to XIL_EXCEPTION_LAST. See xil_mach_exception.h for further information.
<i>Handler</i>	handler function to be registered for exception
<i>Data</i>	a reference to data that will be passed to the handler when it gets called.

void Xil_ExceptionRemoveHandler (u32 *Id*)

Removes the handler for a specific exception Id. The stub handler is then registered for this exception Id.

Parameters

<i>Id</i>	contains the 32 bit ID of the exception source and should be XIL_EXCEPTION_INT or in the range of 0 to XIL_EXCEPTION_LAST. See xexception_l.h for further information.
-----------	--

Microblaze Processor Cache APIs

Overview

Cache functions provide access to cache related operations such as flush and invalidate for instruction and data caches. It gives option to perform the cache operations on a single cacheline, a range of memory and an entire cache.

Note

Macros

- void [Xil_L1DCacheInvalidate\(\)](#)
- void [Xil_L2CacheInvalidate\(\)](#)
- void [Xil_L1DCacheInvalidateRange](#)(Addr, Len)
- void [Xil_L2CacheInvalidateRange](#)(Addr, Len)
- void [Xil_L1DCacheFlushRange](#)(Addr, Len)
- void [Xil_L2CacheFlushRange](#)(Addr, Len)

- void [Xil_L1DCacheFlush\(\)](#)
- void [Xil_L2CacheFlush\(\)](#)
- void [Xil_L1ICacheInvalidateRange](#)(Addr, Len)
- void [Xil_L1ICacheInvalidate\(\)](#)
- void [Xil_L1DCacheEnable\(\)](#)
- void [Xil_L1DCacheDisable\(\)](#)
- void [Xil_L1ICacheEnable\(\)](#)
- void [Xil_L1ICacheDisable\(\)](#)
- void [Xil_DCacheEnable\(\)](#)
- void [Xil_ICacheEnable\(\)](#)

Functions

- void [Xil_DCacheDisable](#) (void)
- void [Xil_ICacheDisable](#) (void)

Macro Definition Documentation

void [Xil_L1DCacheInvalidate](#)()

Invalidate the entire L1 data cache. If the cacheline is modified (dirty), the modified contents are lost.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

Processor must be in real mode.

void [Xil_L2CacheInvalidate](#)()

Invalidate the entire L2 data cache. If the cacheline is modified (dirty), the modified contents are lost.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

Processor must be in real mode.

void Xil_L1DCacheInvalidateRange(*Addr*, *Len*)

Invalidate the L1 data cache for the given address range. If the bytes specified by the address (*Addr*) are cached by the L1 data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the modified contents are lost.

Parameters

<i>Addr</i>	is address of range to be invalidated.
<i>Len</i>	is the length in bytes to be invalidated.

Returns

None.

Note

Processor must be in real mode.

void Xil_L2CacheInvalidateRange(*Addr*, *Len*)

Invalidate the L1 data cache for the given address range. If the bytes specified by the address (*Addr*) are cached by the L1 data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the modified contents are lost.

Parameters

<i>Addr</i>	address of range to be invalidated.
<i>Len</i>	length in bytes to be invalidated.

Returns

None.

Note

Processor must be in real mode.

void Xil_L1DCacheFlushRange(Addr, Len)

Flush the L1 data cache for the given address range. If the bytes specified by the address (Addr) are cached by the data cache, and is modified (dirty), the cacheline will be written to system memory. The cacheline will also be invalidated.

Parameters

<i>Addr</i>	the starting address of the range to be flushed.
<i>Len</i>	length in byte to be flushed.

Returns

None.

void Xil_L2CacheFlushRange(Addr, Len)

Flush the L2 data cache for the given address range. If the bytes specified by the address (Addr) are cached by the data cache, and is modified (dirty), the cacheline will be written to system memory. The cacheline will also be invalidated.

Parameters

<i>Addr</i>	the starting address of the range to be flushed.
<i>Len</i>	length in byte to be flushed.

Returns

None.

void Xil_L1DCacheFlush()

Flush the entire L1 data cache. If any cacheline is dirty, the cacheline will be written to system memory. The entire data cache will be invalidated.

Returns

None.

void Xil_L2CacheFlush()

Flush the entire L2 data cache. If any cacheline is dirty, the cacheline will be written to system memory. The entire data cache will be invalidated.

Returns

None.



void Xil_L1ICacheInvalidateRange(Addr, Len)

Invalidate the instruction cache for the given address range.

Parameters

<i>Addr</i>	is address of range to be invalidated.
<i>Len</i>	is the length in bytes to be invalidated.

Returns

None.

void Xil_L1ICacheInvalidate()

Invalidate the entire instruction cache.

Parameters

<i>None</i>	
-------------	--

Returns

None.

void Xil_L1DCacheEnable()

Enable the L1 data cache.

Returns

None.

void Xil_L1DCacheDisable()

Disable the L1 data cache.

Returns

None.

Note

This is processor specific.



void Xil_L1ICacheEnable()

Enable the instruction cache.

Returns

None.

Note

This is processor specific.

void Xil_L1ICacheDisable()

Disable the L1 Instruction cache.

Returns

None.

Note

This is processor specific.

void Xil_DCacheEnable()

Enable the data cache.

Parameters

<i>None</i>	
-------------	--

Returns

None.

void Xil_ICacheEnable()

Enable the instruction cache.

Parameters

<i>None</i>	
-------------	--

Returns

None.

Note

Function Documentation



void Xil_DCachDisable (void)

Disable the data cache.

Parameters

None	
------	--

Returns

None.

void Xil_ICachDisable (void)

Disable the instruction cache.

Parameters

None	
------	--

Returns

None.

MicroBlaze Processor FSL Macros

Overview

Microblaze BSP includes macros to provide convenient access to accelerators connected to the MicroBlaze Fast Simplex Link (FSL) Interfaces. To use these functions, include the header file fsl.h in your source code

Macros

- #define [getfslx](#)(val, id, flags)
- #define [putfslx](#)(val, id, flags)
- #define [tgetfslx](#)(val, id, flags)
- #define [tputfslx](#)(id, flags)
- #define [getdfslx](#)(val, var, flags)
- #define [putdfslx](#)(val, var, flags)
- #define [tgetdfslx](#)(val, var, flags)
- #define [tputdfslx](#)(var, flags)

Macro Definition Documentation

#define getfslx(*val*, *id*, *flags*)

Performs a get function on an input FSL of the MicroBlaze processor

Parameters

<i>val</i>	variable to sink data from get function
<i>id</i>	literal in the range of 0 to 7 (0 to 15 for MicroBlaze v7.00.a and later)
<i>flags</i>	valid FSL macro flags

#define putfslx(*val*, *id*, *flags*)

Performs a put function on an input FSL of the MicroBlaze processor

Parameters

<i>val</i>	variable to source data to put function
<i>id</i>	literal in the range of 0 to 7 (0 to 15 for MicroBlaze v7.00.a and later)
<i>flags</i>	valid FSL macro flags

#define tgetfslx(*val*, *id*, *flags*)

Performs a test get function on an input FSL of the MicroBlaze processor

Parameters

<i>val</i>	variable to sink data from get function
<i>id</i>	literal in the range of 0 to 7 (0 to 15 for MicroBlaze v7.00.a and later)
<i>flags</i>	valid FSL macro flags

#define tputfslx(*id*, *flags*)

Performs a put function on an input FSL of the MicroBlaze processor

Parameters

<i>id</i>	FSL identifier
<i>flags</i>	valid FSL macro flags

#define getdfslx(*val*, *var*, *flags*)

Performs a getd function on an input FSL of the MicroBlaze processor

Parameters

<i>val</i>	variable to sink data from getd function
<i>var</i>	literal in the range of 0 to 7 (0 to 15 for MicroBlaze v7.00.a and later)
<i>flags</i>	valid FSL macro flags

#define putdfslx(*val*, *var*, *flags*)

Performs a putd function on an input FSL of the MicroBlaze processor

Parameters

<i>val</i>	variable to source data to putd function
<i>var</i>	literal in the range of 0 to 7 (0 to 15 for MicroBlaze v7.00.a and later)
<i>flags</i>	valid FSL macro flags

#define tgetdfslx(*val*, *var*, *flags*)

Performs a test getd function on an input FSL of the MicroBlaze processor;

Parameters

<i>val</i>	variable to sink data from getd function
<i>var</i>	literal in the range of 0 to 7 (0 to 15 for MicroBlaze v7.00.a and later)
<i>flags</i>	valid FSL macro flags

#define tputdfslx(*var*, *flags*)

Performs a put function on an input FSL of the MicroBlaze processor

Parameters

<i>var</i>	FSL identifier
<i>flags</i>	valid FSL macro flags

Microblaze PVR access routines and macros

Overview

MicroBlaze processor v5.00.a and later versions have configurable Processor Version Registers (PVRs). The contents of the PVR are captured using the `pvr_t` data structure, which is defined as an array of 32-bit words, with each word corresponding to a PVR register on hardware. The number of PVR words is determined by the number of PVRs configured in the hardware. You should not attempt to access PVR registers that are not present in hardware, as the `pvr_t` data structure is resized to hold only as many PVRs as are present in hardware. To access information in the PVR:

1. Use the [microblaze_get_pvr\(\)](#) function to populate the PVR data into a `pvr_t` data structure.
2. In subsequent steps, you can use any one of the PVR access macros list to get individual data stored in the PVR.
3. `pvr.h` header file must be included to source to use PVR macros.

Macros

- `#define MICROBLAZE_PVR_IS_FULL(_pvr)`
- `#define MICROBLAZE_PVR_USE_BARREL(_pvr)`
- `#define MICROBLAZE_PVR_USE_DIV(_pvr)`
- `#define MICROBLAZE_PVR_USE_HW_MUL(_pvr)`
- `#define MICROBLAZE_PVR_USE_FPU(_pvr)`
- `#define MICROBLAZE_PVR_USE_ICACHE(_pvr)`
- `#define MICROBLAZE_PVR_USE_DCACHE(_pvr)`
- `#define MICROBLAZE_PVR_MICROBLAZE_VERSION(_pvr)`
- `#define MICROBLAZE_PVR_USER1(_pvr)`
- `#define MICROBLAZE_PVR_USER2(_pvr)`
- `#define MICROBLAZE_PVR_D_LMB(_pvr)`
- `#define MICROBLAZE_PVR_D_PLB(_pvr)`
- `#define MICROBLAZE_PVR_I_LMB(_pvr)`
- `#define MICROBLAZE_PVR_I_PLB(_pvr)`
- `#define MICROBLAZE_PVR_INTERRUPT_IS_EDGE(_pvr)`
- `#define MICROBLAZE_PVR_EDGE_IS_POSITIVE(_pvr)`
- `#define MICROBLAZE_PVR_INTERCONNECT(_pvr)`
- `#define MICROBLAZE_PVR_USE_MUL64(_pvr)`
- `#define MICROBLAZE_PVR_OPCODE_0x0_ILLEGAL(_pvr)`

- #define MICROBLAZE_PVR_UNALIGNED_EXCEPTION(_pvr)
- #define MICROBLAZE_PVR_ILL_OPCODE_EXCEPTION(_pvr)
- #define MICROBLAZE_PVR_IPLB_BUS_EXCEPTION(_pvr)
- #define MICROBLAZE_PVR_DPLB_BUS_EXCEPTION(_pvr)
- #define MICROBLAZE_PVR_DIV_ZERO_EXCEPTION(_pvr)
- #define MICROBLAZE_PVR_FPU_EXCEPTION(_pvr)
- #define MICROBLAZE_PVR_FSL_EXCEPTION(_pvr)
- #define MICROBLAZE_PVR_DEBUG_ENABLED(_pvr)
- #define MICROBLAZE_PVR_NUMBER_OF_PC_BRK(_pvr)
- #define MICROBLAZE_PVR_NUMBER_OF_RD_ADDR_BRK(_pvr)
- #define MICROBLAZE_PVR_NUMBER_OF_WR_ADDR_BRK(_pvr)
- #define MICROBLAZE_PVR_FSL_LINKS(_pvr)
- #define MICROBLAZE_PVR_ICACHE_ADDR_TAG_BITS(_pvr)
- #define MICROBLAZE_PVR_ICACHE_ALLOW_WR(_pvr)
- #define MICROBLAZE_PVR_ICACHE_LINE_LEN(_pvr)
- #define MICROBLAZE_PVR_ICACHE_BYTE_SIZE(_pvr)
- #define MICROBLAZE_PVR_DCACHE_ADDR_TAG_BITS(_pvr)
- #define MICROBLAZE_PVR_DCACHE_ALLOW_WR(_pvr)
- #define MICROBLAZE_PVR_DCACHE_LINE_LEN(_pvr)
- #define MICROBLAZE_PVR_DCACHE_BYTE_SIZE(_pvr)
- #define MICROBLAZE_PVR_ICACHE_BASEADDR(_pvr)
- #define MICROBLAZE_PVR_ICACHE_HIGHADDR(_pvr)
- #define MICROBLAZE_PVR_DCACHE_BASEADDR(_pvr)
- #define MICROBLAZE_PVR_DCACHE_HIGHADDR(_pvr)
- #define MICROBLAZE_PVR_TARGET_FAMILY(_pvr)
- #define MICROBLAZE_PVR_MSR_RESET_VALUE(_pvr)
- #define MICROBLAZE_PVR_MMU_TYPE(_pvr)

Functions

- int [microblaze_get_pvr](#) (pvr_t *pvr)

Macro Definition Documentation

#define MICROBLAZE_PVR_IS_FULL(_pvr)

Return non-zero integer if PVR is of type FULL, 0 if basic

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_USE_BARREL(*_pvr*)

Return non-zero integer if hardware barrel shifter present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_USE_DIV(*_pvr*)

Return non-zero integer if hardware divider present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_USE_HW_MUL(*_pvr*)

Return non-zero integer if hardware multiplier present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_USE_FPU(*_pvr*)

Return non-zero integer if hardware floating point unit (FPU) present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_USE_ICACHE(*_pvr*)

Return non-zero integer if I-cache present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_USE_DCACHE(*_pvr*)

Return non-zero integer if D-cache present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_MICROBLAZE_VERSION(*_pvr*)

Return MicroBlaze processor version encoding. Refer to the MicroBlaze Processor Reference Guide (UG081) for mappings from encodings to actual hardware versions.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_USER1(*_pvr*)

Return the USER1 field stored in the PVR.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_USER2(*_pvr*)

Return the USER2 field stored in the PVR.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_D_LMB(*_pvr*)

Return non-zero integer if Data Side PLB interface is present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_D_PLB(*_pvr*)

Return non-zero integer if Data Side PLB interface is present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_I_LMB(*_pvr*)

Return non-zero integer if Instruction Side Local Memory Bus (LMB) interface present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_I_PLB(*_pvr*)

Return non-zero integer if Instruction Side PLB interface present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_INTERRUPT_IS_EDGE(*_pvr*)

Return non-zero integer if interrupts are configured as edge-triggered.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_EDGE_IS_POSITIVE(*_pvr*)

Return non-zero integer if interrupts are configured as positive edge triggered.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_INTERCONNECT(*_pvr*)

Return non-zero if MicroBlaze processor has PLB interconnect; otherwise return zero.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_USE_MUL64(*_pvr*)

Return non-zero integer if MicroBlaze processor supports 64-bit products for multiplies.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_OPCODE_0x0_ILLEGAL(*_pvr*)

Return non-zero integer if opcode 0x0 is treated as an illegal opcode. multiplies.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_UNALIGNED_EXCEPTION(*_pvr*)

Return non-zero integer if unaligned exceptions are supported.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_ILL_OPCODE_EXCEPTION(*_pvr*)

Return non-zero integer if illegal opcode exceptions are supported.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_IPLB_BUS_EXCEPTION(*_pvr*)

Return non-zero integer if I-PLB exceptions are supported.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_DPLB_BUS_EXCEPTION(*_pvr*)

Return non-zero integer if I-PLB exceptions are supported.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_DIV_ZERO_EXCEPTION(*_pvr*)

Return non-zero integer if divide by zero exceptions are supported.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_FPU_EXCEPTION(*_pvr*)

Return non-zero integer if FPU exceptions are supported.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_FSL_EXCEPTION(*_pvr*)

Return non-zero integer if FSL exceptions are present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_DEBUG_ENABLED(*_pvr*)

Return non-zero integer if debug is enabled.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_NUMBER_OF_PC_BRK(*_pvr*)

Return the number of hardware PC breakpoints available.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_NUMBER_OF_RD_ADDR_BRK(*_pvr*)

Return the number of read address hardware watchpoints supported.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_NUMBER_OF_WR_ADDR_BRK(*_pvr*)

Return the number of write address hardware watchpoints supported.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_FSL_LINKS(*_pvr*)

Return the number of FSL links present.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_ICACHE_ADDR_TAG_BITS(*_pvr*)

Return the number of address tag bits for the I-cache.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_ICACHE_ALLOW_WR(*_pvr*)

Return non-zero if writes to I-caches are allowed.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_ICACHE_LINE_LEN(*_pvr*)

Return the length of each I-cache line in bytes.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_ICACHE_BYTE_SIZE(*_pvr*)

Return the size of the D-cache in bytes.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_DCACHE_ADDR_TAG_BITS(*_pvr*)

Return the number of address tag bits for the D-cache.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_DCACHE_ALLOW_WR(*_pvr*)

Return non-zero if writes to D-cache are allowed.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_DCACHE_LINE_LEN(*_pvr*)

Return the length of each line in the D-cache in bytes.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_DCACHE_BYTE_SIZE(*_pvr*)

Return the size of the D-cache in bytes.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_ICACHE_BASEADDR(*_pvr*)

Return the base address of the I-cache.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_ICACHE_HIGHADDR(*_pvr*)

Return the high address of the I-cache.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_DCACHE_BASEADDR(*_pvr*)

Return the base address of the D-cache.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_DCACHE_HIGHADDR(*_pvr*)

Return the high address of the D-cache.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_TARGET_FAMILY(*_pvr*)

Return the encoded target family identifier.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_MSR_RESET_VALUE(*_pvr*)

Refer to the MicroBlaze Processor Reference Guide (UG081) for mappings from encodings to target family name strings.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

#define MICROBLAZE_PVR_MMU_TYPE(*_pvr*)

Returns the value of C_USE_MMU. Refer to the MicroBlaze Processor Reference Guide (UG081) for mappings from MMU type values to MMU function.

Parameters

<i>_pvr</i>	pvr data structure
-------------	--------------------

Function Documentation

int microblaze_get_pvr (pvr_t * pvr)

Populate the PVR data structure to which pvr points with the values of the hardware PVR registers.

Parameters

<i>pvr-</i>	address of PVR data structure to be populated
-------------	---

Returns

0 - SUCCESS -1 - FAILURE

Sleep Routines for Microblaze

Overview

microblaze_sleep.h contains microblaze sleep APIs. These APIs provides delay for requested duration.

Note

microblaze_sleep.h may contain architecture-dependent items.

Functions

- void [MB_Sleep](#) (u32 MilliSeconds) __attribute__((__deprecated__))

Function Documentation

void MB_Sleep (u32 *MilliSeconds*)

Provides delay for requested duration..

Parameters

<i>MilliSeconds-</i>	Delay time in milliseconds.
----------------------	-----------------------------

Returns

None.

Note

Instruction cache should be enabled for this to work.

Cortex R5 Processor API

Overview

Standalone BSP contains boot code, cache, exception handling, file and memory management, configuration, time and processor-specific include functions. It supports gcc compiler. This section provides a linked summary and detailed descriptions of the Cortex R5 processor APIs.

Modules

- [Cortex R5 Processor Boot Code](#)
 - [Cortex R5 Processor MPU specific APIs](#)
 - [Cortex R5 Processor Cache Functions](#)
 - [Cortex R5 Time Functions](#)
 - [Cortex R5 Event Counters Functions](#)
 - [Cortex R5 Processor Specific Include Files](#)
-

Cortex R5 Processor Boot Code

Overview

The boot .S file contains a minimal set of code for transferring control from the processor's reset location to the start of the application. The boot code performs minimum configuration which is required for an application to run starting from processor's reset state. Below is a sequence illustrating what all configuration is performed before control reaches to main function.

1. Program vector table base for exception handling
2. Program stack pointer for various modes (IRQ, FIQ, supervisor, undefine, abort, system)
3. Disable instruction cache, data cache and MPU
4. Invalidate instruction and data cache
5. Configure MPU with short descriptor translation table format and program base address of translation table
6. Enable data cache, instruction cache and MPU

7. Enable Floating point unit
8. Transfer control to _start which clears BSS sections and jumping to main application

Cortex R5 Processor MPU specific APIs

Overview

MPU functions provides access to MPU operations such as enable MPU, disable MPU and set attribute for section of memory. Boot code invokes Init_MPU function to configure the MPU. A total of 10 MPU regions are allocated with another 6 being free for users. Overview of the memory attributes for different MPU regions is as given below,

	Memory Range	Attributes of MPURegion	Note
DDR	0x00000000 - 0x7FFFFFFF	Normal write-back Cacheable	For a system where DDR is less than 2GB, region after DDR and before PL is marked as undefined in translation table
PL	0x80000000 - 0xBFFFFFFF	Strongly Ordered	
QSPI	0xC0000000 - 0xDFFFFFFF	Device Memory	
PCIe	0xE0000000 - 0xEFFFFFFF	Device Memory	
STM_CORESIGHT	0xF8000000 - 0xF8FFFFFF	Device Memory	

	Memory Range	Attributes of MPURegion	Note
RPU_R5_GIC	0xF9000000 - 0xF90FFFFFFF	Device Memory	
FPS	0xFD000000 - 0xFDFFFFFFF	Device Memory	
LPS	0xFE000000 - 0xFFFFFFFF	Device Memory	0xFE000000 - 0xFEFFFFFFF upper LPS slaves, 0xFF000000 - 0xFFFFFFFF lower LPS slaves
OCM	0xFFFC0000 - 0xFFFFFFFF	Normal write-back Cacheable	

Functions

- void [Xil_SetTlbAttributes](#) (INTPTR Addr, u32 attrib)
- void [Xil_EnableMPU](#) (void)
- void [Xil_DisableMPU](#) (void)
- void [Xil_SetMPURegion](#) (INTPTR addr, u64 size, u32 attrib)

Function Documentation

void [Xil_SetTlbAttributes](#) (INTPTR *addr*, u32 *attrib*)

This function sets the memory attributes for a section covering 1MB, of memory in the translation table.

Parameters

<i>Addr</i>	32-bit address for which memory attributes need to be set.
<i>attrib</i>	Attribute for the given memory region.

Returns

None.

void Xil_EnableMPU (void)

Enable MPU for Cortex R5 processor. This function invalidates I cache and flush the D Caches, and then enables the MPU.

Parameters

None.	
-------	--

Returns

None.

void Xil_DisableMPU (void)

Disable MPU for Cortex R5 processors. This function invalidates I cache and flush the D Caches, and then disables the MPU.

Parameters

None.	
-------	--

Returns

None.

void Xil_SetMPURegion (INTPTR addr, u64 size, u32 attrib)

Set the memory attributes for a section of memory in the translation table.

Parameters

<i>Addr</i>	32-bit address for which memory attributes need to be set..
<i>size</i>	size is the size of the region.
<i>attrib</i>	Attribute for the given memory region.

Returns

None.

Cortex R5 Processor Cache Functions

Overview

Cache functions provide access to cache related operations such as flush and invalidate for instruction and data caches. It gives option to perform the cache operations on a single cacheline, a range of memory and an entire cache.

Functions

- void [Xil_DCacheEnable](#) (void)
- void [Xil_DCacheDisable](#) (void)
- void [Xil_DCacheInvalidate](#) (void)
- void [Xil_DCacheInvalidateRange](#) (INTPTR adr, u32 len)
- void [Xil_DCacheFlush](#) (void)
- void [Xil_DCacheFlushRange](#) (INTPTR adr, u32 len)
- void [Xil_DCacheInvalidateLine](#) (INTPTR adr)
- void [Xil_DCacheFlushLine](#) (INTPTR adr)
- void [Xil_DCacheStoreLine](#) (INTPTR adr)
- void [Xil_ICacheEnable](#) (void)
- void [Xil_ICacheDisable](#) (void)
- void [Xil_ICacheInvalidate](#) (void)
- void [Xil_ICacheInvalidateRange](#) (INTPTR adr, u32 len)
- void [Xil_ICacheInvalidateLine](#) (INTPTR adr)

Function Documentation

void [Xil_DCacheEnable](#) (void)

Enable the Data cache.

Parameters

None.	
-------	--

Returns

None.

Note

None.

void [Xil_DCacheDisable](#) (void)

Disable the Data cache.

Parameters

None.	
-------	--

Returns

None.

Note

None.

void Xil_DCacheInvalidate (void)

Invalidate the entire Data cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

void Xil_DCacheInvalidateRange (INTPTR *adr*, u32 *len*)

Invalidate the Data cache for the given address range. If the bytes specified by the address (*adr*) are cached by the Data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the modified contents are lost and are NOT written to system memory before the line is invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be invalidated.
<i>len</i>	Length of range to be invalidated in bytes.

Returns

None.

void Xil_DCacheFlush (void)

Flush the entire Data cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

void Xil_DCacheFlushRange (INTPTR *adr*, u32 *len*)

Flush the Data cache for the given address range. If the bytes specified by the address (*adr*) are cached by the Data cache, the cacheline containing those bytes is invalidated. If the cacheline is modified (dirty), the written to system memory before the lines are invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be flushed.
<i>len</i>	Length of the range to be flushed in bytes

Returns

None.

void Xil_DCacheInvalidateLine (INTPTR *adr*)

Invalidate a Data cache line. If the byte specified by the address (*adr*) is cached by the data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the modified contents are lost and are NOT written to system memory before the line is invalidated.

Parameters

<i>adr</i>	32bit address of the data to be flushed.
------------	--

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

void Xil_DCacheFlushLine (INTPTR *adr*)

Flush a Data cache line. If the byte specified by the address (*adr*) is cached by the Data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the entire contents of the cacheline are written to system memory before the line is invalidated.

Parameters

<i>adr</i>	32bit address of the data to be flushed.
------------	--

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

void Xil_DCacheStoreLine (INTPTR *adr*)

Store a Data cache line. If the byte specified by the address (*adr*) is cached by the Data cache and the cacheline is modified (dirty), the entire contents of the cacheline are written to system memory. After the store completes, the cacheline is marked as unmodified (not dirty).

Parameters

<i>adr</i>	32bit address of the data to be stored
------------	--

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

void Xil_ICacheEnable (void)

Enable the instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

void Xil_ICacheDisable (void)

Disable the instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

void Xil_ICacheInvalidate (void)

Invalidate the entire instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

void Xil_ICacheInvalidateRange (INTPTR *adr*, u32 *len*)

Invalidate the instruction cache for the given address range. If the bytes specified by the address (*adr*) are cached by the Data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the modified contents are lost and are NOT written to system memory before the line is invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be invalidated.
<i>len</i>	Length of the range to be invalidated in bytes.

Returns

None.

void Xil_ICacheInvalidateLine (INTPTR *adr*)

Invalidate an instruction cache line. If the instruction specified by the address is cached by the instruction cache, the cacheline containing that instruction is invalidated.

Parameters

<i>adr</i>	32bit address of the instruction to be invalidated.
------------	---

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

Cortex R5 Time Functions

Overview

The `xtime_l.c` file and corresponding `xtime_l.h` include file provide access to the 32-bit counter in TTC. The `sleep.c`, `usleep.c` file and the corresponding `sleep.h` include file implement sleep functions. Sleep functions are implemented as busy loops.

Functions

- void [XTime_StartTimer](#) (void)
- void [XTime_SetTime](#) (XTime Xtime_Global)
- void [XTime_GetTime](#) (XTime *Xtime_Global)

Function Documentation

void XTime_StartTimer (void)

Starts the TTC timer 3 counter 0 if present and if it is not already running with desired parameters for sleep functionalities.

Parameters

None.	
-------	--

Returns

None.

Note

When this function is called by any one processor in a multi- processor environment, reference time will reset/lost for all processors.

void XTime_SetTime (XTime *Xtime_Global*)

TTC Timer runs continuously and the time can not be set as desired. This API doesn't contain anything. It is defined to have uniformity across platforms.

Parameters

<i>Xtime_Global</i>	32 bit value to be written to the timer counter register.
---------------------	---

Returns

None.

Note

In multiprocessor environment reference time will reset/lost for all processors, when this function called by any one processor.

void XTime_GetTime (XTime * *Xtime_Global*)

Get the time from the timer counter register.

Parameters

<i>Xtime_Global</i>	Pointer to the 32 bit location to be updated with the time current value of timer counter register.
---------------------	---

Returns

None.

Cortex R5 Event Counters Functions

Overview

Cortex R5 event counter functions can be utilized to configure and control the Cortex-R5 performance monitor events. Cortex-R5 Performance Monitor has 6 event counters which can be used to count a variety of events described in Cortex-R5 TRM. xpm_counter.h defines configurations XPM_CNTRCFGx which can be used to program the event counters to count a set of events.

Note

It doesn't handle the Cortex-R5 cycle counter, as the cycle counter is being used for time keeping.

Functions

- void [Xpm_SetEvents](#) (s32 PmcrCfg)
- void [Xpm_GetEventCounters](#) (u32 *PmCtrValue)

Function Documentation

void Xpm_SetEvents (s32 *PmcrCfg*)

This function configures the Cortex R5 event counters controller, with the event codes, in a configuration selected by the user and enables the counters.

Parameters

<i>PmcrCfg</i>	Configuration value based on which the event counters are configured. XPM_CNTRCFG* values defined in xpm_counter.h can be utilized for setting configuration
----------------	--

Returns

None.

void Xpm_GetEventCounters (u32 * *PmCtrValue*)

This function disables the event counters and returns the counter values.

Parameters

<i>PmCtrValue</i>	Pointer to an array of type u32 PmCtrValue[6]. It is an output parameter which is used to return the PM counter values.
-------------------	---

Returns

None.

Cortex R5 Processor Specific Include Files

Overview

The xpseudo_asm.h file includes xreg_cortexr5.h and xpseudo_asm_gcc.h.

The xreg_cortexr5.h include file contains the register numbers and the register bits for the ARM Cortex-R5 processor.

The xpseudo_asm_gcc.h file contains the definitions for the most often used inline assembler instructions, available as macros. These can be very useful for tasks such as setting or getting special purpose registers, synchronization, or cache manipulation. These inline assembler instructions can be used from drivers and user applications written in C.

ARM Processor Common API

Overview

This section provides a linked summary and detailed descriptions of the ARM Processor Common APIs.

Modules

- [ARM Processor Exception Handling](#)
-

ARM Processor Exception Handling

Overview

ARM processors specific exception related APIs for cortex A53,A9 and R5 can utilized for enabling/disabling IRQ, registering/removing handler for exceptions or initializing exception vector table with null handler.

Macros

- #define [Xil_ExceptionEnableMask](#)(Mask)
- #define [Xil_ExceptionEnable](#)()
- #define [Xil_ExceptionDisableMask](#)(Mask)
- #define [Xil_ExceptionDisable](#)()
- #define [Xil_EnableNestedInterrupts](#)()
- #define [Xil_DisableNestedInterrupts](#)()

Typedefs

- typedef void(* [Xil_ExceptionHandler](#)) (void *data)

Functions

- void [Xil_ExceptionRegisterHandler](#) (u32 Exception_id, [Xil_ExceptionHandler](#) Handler, void *Data)
- void [Xil_ExceptionRemoveHandler](#) (u32 Exception_id)

- void [Xil_ExceptionInit](#) (void)
- void [Xil_DataAbortHandler](#) (void *CallBackRef)
- void [Xil_PrefetchAbortHandler](#) (void *CallBackRef)
- void [Xil_UndefinedExceptionHandler](#) (void *CallBackRef)

Macro Definition Documentation

#define Xil_ExceptionEnableMask(*Mask*)

Enable Exceptions.

Parameters

<i>Mask</i>	for exceptions to be enabled.
-------------	-------------------------------

Returns

None.

Note

If bit is 0, exception is enabled. C-Style signature: void [Xil_ExceptionEnableMask\(Mask\)](#)

#define Xil_ExceptionEnable()

Enable the IRQ exception.

Returns

None.

Note

None.

#define Xil_ExceptionDisableMask(*Mask*)

Disable Exceptions.

Parameters

<i>Mask</i>	for exceptions to be enabled.
-------------	-------------------------------

Returns

None.

Note

If bit is 1, exception is disabled. C-Style signature: [Xil_ExceptionDisableMask\(Mask\)](#)

#define Xil_ExceptionDisable()

Disable the IRQ exception.

Returns

None.

Note

None.

#define Xil_EnableNestedInterrupts()

Enable nested interrupts by clearing the I and F bits in CPSR. This API is defined for cortex-a9 and cortex-r5.

Returns

None.

Note

This macro is supposed to be used from interrupt handlers. In the interrupt handler the interrupts are disabled by default (I and F are 1). To allow nesting of interrupts, this macro should be used. It clears the I and F bits by changing the ARM mode to system mode. Once these bits are cleared and provided the preemption of interrupt conditions are met in the GIC, nesting of interrupts will start happening. Caution: This macro must be used with caution. Before calling this macro, the user must ensure that the source of the current IRQ is appropriately cleared. Otherwise, as soon as we clear the I and F bits, there can be an infinite loop of interrupts with an eventual crash (all the stack space getting consumed).

#define Xil_DisableNestedInterrupts()

Disable the nested interrupts by setting the I and F bits. This API is defined for cortex-a9 and cortex-r5.

Returns

None.

Note

This macro is meant to be called in the interrupt service routines. This macro cannot be used independently. It can only be used when nesting of interrupts have been enabled by using the macro [Xil_EnableNestedInterrupts\(\)](#). In a typical flow, the user first calls the `Xil_EnableNestedInterrupts` in the ISR at the appropriate point. The user then must call this macro before exiting the interrupt service routine. This macro puts the ARM back in IRQ/FIQ mode and hence sets back the I and F bits.

Typedef Documentation

typedef void(* Xil_ExceptionHandler) (void *data)

This typedef is the exception handler function.

Function Documentation

void Xil_ExceptionRegisterHandler (u32 *Exception_id*, Xil_ExceptionHandler *Handler*, void * *Data*)

Register a handler for a specific exception. This handler is being called when the processor encounters the specified exception.

Parameters

<i>exception_id</i>	contains the ID of the exception source and should be in the range of 0 to XIL_EXCEPTION_ID_LAST. See xil_exception.h for further information.
<i>Handler</i>	to the Handler for that exception.
<i>Data</i>	is a reference to Data that will be passed to the Handler when it gets called.

Returns

None.

Note

None.

void Xil_ExceptionRemoveHandler (u32 *Exception_id*)

Removes the Handler for a specific exception Id. The stub Handler is then registered for this exception Id.

Parameters

<i>exception_id</i>	contains the ID of the exception source and should be in the range of 0 to XIL_EXCEPTION_ID_LAST. See xil_exception.h for further information.
---------------------	--

Returns

None.

Note

None.

void Xil_ExceptionInit (void)

The function is a common API used to initialize exception handlers across all supported arm processors. For ARM Cortex-A53, Cortex-R5, and Cortex-A9, the exception handlers are being initialized statically and this function does not do anything. However, it is still present to take care of backward compatibility issues (in earlier versions of BSPs, this API was being used to initialize exception handlers).

Parameters

None.	
-------	--

Returns

None.

Note

None.

void Xil_DataAbortHandler (void * *CallbackRef*)

Default Data abort handler which prints data fault status register through which information about data fault can be acquired

Parameters

None	
------	--

Returns

None.

Note

None.

void Xil_PrefetchAbortHandler (void * *CallbackRef*)

Default Prefetch abort handler which prints prefetch fault status register through which information about instruction prefetch fault can be acquired

Parameters

None	
------	--

Returns

None.

Note

None.



void Xil_UndefinedExceptionHandler (void * *CallbackRef*)

Default undefined exception handler which prints address of the undefined instruction if debug prints are enabled

Parameters

<i>None</i>	
-------------	--

Returns

None.

Note

None.

Cortex A9 Processor API

Overview

Standalone BSP contains boot code, cache, exception handling, file and memory management, configuration, time and processor-specific include functions. It supports gcc compilers.

Modules

- [Cortex A9 Processor Boot Code](#)
 - [Cortex A9 Processor Cache Functions](#)
 - [Cortex A9 Processor MMU Functions](#)
 - [Cortex A9 Time Functions](#)
 - [Cortex A9 Event Counter Function](#)
 - [PL310 L2 Event Counters Functions](#)
 - [Cortex A9 Processor and pl310 Errata Support](#)
 - [Cortex A9 Processor Specific Include Files](#)
-

Cortex A9 Processor Boot Code

Overview

The boot .S file contains a minimal set of code for transferring control from the processor reset location to the start of the application. The boot code performs minimum configuration which is required for an application to run starting from processor's reset state. Below is a sequence illustrating what all configuration is performed before control reaches to main function.

1. Program vector table base for exception handling
2. Invalidate instruction cache, data cache and TLBs
3. Program stack pointer for various modes (IRQ, FIQ, supervisor, undefine, abort, system)
4. Configure MMU with short descriptor translation table format and program base address of translation table
5. Enable data cache, instruction cache and MMU

6. Enable Floating point unit
7. Transfer control to `_start` which clears BSS sections, initializes global timer and runs global constructor before jumping to main application

The `translation_table.S` file contains a static page table required by MMU for cortex-A9. This translation table is flat mapped (input address = output address) with default memory attributes defined for zynq architecture. It utilizes short descriptor translation table format with each section defining 1MB of memory. The overview of translation table memory attributes is described below.

	Memory Range	Definition in Translation Table	Note
DDR	0x00000000 - 0x3FFFFFFF	Normal write-back Cacheable	For a system where DDR is less than 1GB, region after DDR and before PL is marked as undefined/reserved in translation table
PL	0x40000000 - 0xBFFFFFFF	Strongly Ordered	
Reserved	0xC0000000 - 0xDFFFFFFF	Unassigned	
Memory mapped devices	0xE0000000 - 0xE02FFFFFFF	Device Memory	
Reserved	0xE0300000 - 0xE0FFFFFFF	Unassigned	
NAND, NOR	0xE1000000 - 0xE3FFFFFFF	Device memory	
SRAM	0xE4000000 - 0xE5FFFFFFF	Normal write-back Cacheable	
Reserved	0xE6000000 - 0xF7FFFFFFF	Unassigned	
AMBA APB Peripherals	0xF8000000 - 0xF8FFFFFFF	Device Memory	0xF8000C00 - 0xF800FFF, 0xF8010000 - 0xF88FFFFFF and 0xF8F03000 to 0xF8FFFFFFF are reserved but due to granual size of 1MB, it is not possible to define separate regions for them

	Memory Range	Definition in Translation Table	Note
Reserved	0xF9000000 - 0xFBFFFFFF	Unassigned	
Linear QSPI - XIP	0xFC000000 - 0xFDFFFFFF	Normal write-through cacheable	
Reserved	0xFE000000 - 0xFFEFFFFFF	Unassigned	
OCM	0xFFF00000 - 0xFFFFFFF	Normal inner write-back cacheable	0xFFF00000 to 0xFFFB0000 is reserved but due to 1MB granual size, it is not possible to define separate region for it

Cortex A9 Processor Cache Functions

Overview

Cache functions provide access to cache related operations such as flush and invalidate for instruction and data caches. It gives option to perform the cache operations on a single cacheline, a range of memory and an entire cache.

Functions

- void [Xil_DCacheEnable](#) (void)
- void [Xil_DCacheDisable](#) (void)
- void [Xil_DCacheInvalidate](#) (void)
- void [Xil_DCacheInvalidateRange](#) (INTPTR adr, u32 len)
- void [Xil_DCacheFlush](#) (void)
- void [Xil_DCacheFlushRange](#) (INTPTR adr, u32 len)
- void [Xil_ICacheEnable](#) (void)
- void [Xil_ICacheDisable](#) (void)
- void [Xil_ICacheInvalidate](#) (void)
- void [Xil_ICacheInvalidateRange](#) (INTPTR adr, u32 len)
- void [Xil_DCacheInvalidateLine](#) (u32 adr)
- void [Xil_DCacheFlushLine](#) (u32 adr)
- void [Xil_DCacheStoreLine](#) (u32 adr)
- void [Xil_ICacheInvalidateLine](#) (u32 adr)
- void [Xil_L1DCacheEnable](#) (void)
- void [Xil_L1DCacheDisable](#) (void)
- void [Xil_L1DCacheInvalidate](#) (void)

- void [Xil_L1DCacheInvalidateLine](#) (u32 adr)
- void [Xil_L1DCacheInvalidateRange](#) (u32 adr, u32 len)
- void [Xil_L1DCacheFlush](#) (void)
- void [Xil_L1DCacheFlushLine](#) (u32 adr)
- void [Xil_L1DCacheFlushRange](#) (u32 adr, u32 len)
- void [Xil_L1DCacheStoreLine](#) (u32 adr)
- void [Xil_L1ICacheEnable](#) (void)
- void [Xil_L1ICacheDisable](#) (void)
- void [Xil_L1ICacheInvalidate](#) (void)
- void [Xil_L1ICacheInvalidateLine](#) (u32 adr)
- void [Xil_L1ICacheInvalidateRange](#) (u32 adr, u32 len)
- void [Xil_L2CacheEnable](#) (void)
- void [Xil_L2CacheDisable](#) (void)
- void [Xil_L2CacheInvalidate](#) (void)
- void [Xil_L2CacheInvalidateLine](#) (u32 adr)
- void [Xil_L2CacheInvalidateRange](#) (u32 adr, u32 len)
- void [Xil_L2CacheFlush](#) (void)
- void [Xil_L2CacheFlushLine](#) (u32 adr)
- void [Xil_L2CacheFlushRange](#) (u32 adr, u32 len)
- void [Xil_L2CacheStoreLine](#) (u32 adr)

Function Documentation

void [Xil_DCACHEEnable](#) (void)

Enable the Data cache.

Parameters

None.	
-------	--

Returns

None.

Note

None.

void Xil_DCacheDisable (void)

Disable the Data cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_DCacheInvalidate (void)

Invalidate the entire Data cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_DCacheInvalidateRange (INTPTR *adr*, u32 *len*)

Invalidate the Data cache for the given address range. If the bytes specified by the address range are cached by the Data cache, the cachelines containing those bytes are invalidated. If the cachelines are modified (dirty), the modified contents are lost and NOT written to the system memory before the lines are invalidated.

In this function, if start address or end address is not aligned to cache-line, particular cache-line containing unaligned start or end address is flush first and then invalidated the others as invalidating the same unaligned cache line may result into loss of data. This issue raises few possibilities.

If the address to be invalidated is not cache-line aligned, the following choices are available:

1. Invalidate the cache line when required and do not bother much for the side effects. Though it sounds good, it can result in hard-to-debug issues. The problem is, if some other variable are allocated in the same cache line and had been recently updated (in cache), the invalidation would result in loss of data.
2. Flush the cache line first. This will ensure that if any other variable present in the same cache line and updated recently are flushed out to memory. Then it can safely be invalidated. Again it sounds good, but this can result in issues. For example, when the invalidation happens in a typical ISR (after a DMA transfer has updated the memory), then flushing the cache line means, losing data that were updated recently before the ISR got invoked.

Linux prefers the second one. To have uniform implementation (across standalone and Linux), the second option is implemented. This being the case, following needs to be taken care of:

1. Whenever possible, the addresses must be cache line aligned. Please note that, not just start address, even the end address must be cache line aligned. If that is taken care of, this will always work.
2. Avoid situations where invalidation has to be done after the data is updated by peripheral/DMA directly into the memory. It is not tough to achieve (may be a bit risky). The common use case to do invalidation is when a DMA happens. Generally for such use cases, buffers can be allocated first and then start the DMA. The practice that needs to be followed here is, immediately after buffer allocation and before starting the DMA, do the invalidation. With this approach, invalidation need not to be done after the DMA transfer is over.

This is going to always work if done carefully. However, the concern is, there is no guarantee that invalidate has not needed to be done after DMA is complete. For example, because of some reasons if the first cache line or last cache line (assuming the buffer in question comprises of multiple cache lines) are brought into cache (between the time it is invalidated and DMA completes) because of some speculative prefetching or reading data for a variable present in the same cache line, then we will have to invalidate the cache after DMA is complete.

Parameters

<i>adr</i>	32bit start address of the range to be invalidated.
<i>len</i>	Length of the range to be invalidated in bytes.

Returns

None.

Note

None.

void Xil_DCacheFlush (void)

Flush the entire Data cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_DCacheFlushRange (INTPTR *adr*, u32 *len*)

Flush the Data cache for the given address range. If the bytes specified by the address range are cached by the data cache, the cachelines containing those bytes are invalidated. If the cachelines are modified (dirty), they are written to the system memory before the lines are invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be flushed.
<i>len</i>	Length of the range to be flushed in bytes.

Returns

None.

Note

None.

void Xil_ICacheEnable (void)

Enable the instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_ICacheDisable (void)

Disable the instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_ICacheInvalidate (void)

Invalidate the entire instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_ICacheInvalidateRange (INTPTR *adr*, u32 *len*)

Invalidate the instruction cache for the given address range. If the instructions specified by the address range are cached by the instruction cache, the cachelines containing those instructions are invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be invalidated.
<i>len</i>	Length of the range to be invalidated in bytes.

Returns

None.

Note

None.

void Xil_DCacheInvalidateLine (u32 *adr*)

Invalidate a Data cache line. If the byte specified by the address (*adr*) is cached by the Data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the modified contents are lost and are NOT written to the system memory before the line is invalidated.

Parameters

<i>adr</i>	32bit address of the data to be flushed.
------------	--

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

void Xil_DCacheFlushLine (u32 *adr*)

Flush a Data cache line. If the byte specified by the address (*adr*) is cached by the Data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the entire contents of the cacheline are written to system memory before the line is invalidated.

Parameters

<i>adr</i>	32bit address of the data to be flushed.
------------	--

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

void Xil_DCacheStoreLine (u32 *adr*)

Store a Data cache line. If the byte specified by the address (*adr*) is cached by the Data cache and the cacheline is modified (dirty), the entire contents of the cacheline are written to system memory. After the store completes, the cacheline is marked as unmodified (not dirty).

Parameters

<i>adr</i>	32bit address of the data to be stored.
------------	---

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

void Xil_ICacheInvalidateLine (u32 *adr*)

Invalidate an instruction cache line. If the instruction specified by the address is cached by the instruction cache, the cacheline containing that instruction is invalidated.

Parameters

<i>adr</i>	32bit address of the instruction to be invalidated.
------------	---

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.



void Xil_L1DCacheEnable (void)

Enable the level 1 Data cache.

Parameters

None.	
-------	--

Returns

None.

Note

None.

void Xil_L1DCacheDisable (void)

Disable the level 1 Data cache.

Parameters

None.	
-------	--

Returns

None.

Note

None.

void Xil_L1DCacheInvalidate (void)

Invalidate the level 1 Data cache.

Parameters

None.	
-------	--

Returns

None.

Note

In Cortex A9, there is no cp instruction for invalidating the whole D-cache. This function invalidates each line by set/way.

void Xil_L1DCacheInvalidateLine (u32 *adr*)

Invalidate a level 1 Data cache line. If the byte specified by the address (Addr) is cached by the Data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the modified contents are lost and are NOT written to system memory before the line is invalidated.

Parameters

<i>adr</i>	32bit address of the data to be invalidated.
------------	--

Returns

None.

Note

The bottom 5 bits are set to 0, forced by architecture.

void Xil_L1DCacheInvalidateRange (u32 *adr*, u32 *len*)

Invalidate the level 1 Data cache for the given address range. If the bytes specified by the address range are cached by the Data cache, the cachelines containing those bytes are invalidated. If the cachelines are modified (dirty), the modified contents are lost and NOT written to the system memory before the lines are invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be invalidated.
<i>len</i>	Length of the range to be invalidated in bytes.

Returns

None.

Note

None.

void Xil_L1DCacheFlush (void)

Flush the level 1 Data cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

In Cortex A9, there is no cp instruction for flushing the whole D-cache. Need to flush each line.

void Xil_L1DCacheFlushLine (u32 *adr*)

Flush a level 1 Data cache line. If the byte specified by the address (*adr*) is cached by the Data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the entire contents of the cacheline are written to system memory before the line is invalidated.

Parameters

<i>adr</i>	32bit address of the data to be flushed.
------------	--

Returns

None.

Note

The bottom 5 bits are set to 0, forced by architecture.

void Xil_L1DCacheFlushRange (u32 *adr*, u32 *len*)

Flush the level 1 Data cache for the given address range. If the bytes specified by the address range are cached by the Data cache, the cacheline containing those bytes are invalidated. If the cachelines are modified (dirty), they are written to system memory before the lines are invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be flushed.
<i>len</i>	Length of the range to be flushed in bytes.

Returns

None.

Note

None.

void Xil_L1DCacheStoreLine (u32 adr)

Store a level 1 Data cache line. If the byte specified by the address (adr) is cached by the Data cache and the cacheline is modified (dirty), the entire contents of the cacheline are written to system memory. After the store completes, the cacheline is marked as unmodified (not dirty).

Parameters

<i>Address</i>	to be stored.
----------------	---------------

Returns

None.

Note

The bottom 5 bits are set to 0, forced by architecture.

void Xil_L1ICacheEnable (void)

Enable the level 1 instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_L1ICacheDisable (void)

Disable level 1 the instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_L1ICacheInvalidate (void)

Invalidate the entire level 1 instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_L1ICacheInvalidateLine (u32 *adr*)

Invalidate a level 1 instruction cache line. If the instruction specified by the address is cached by the instruction cache, the cacheline containing that instruction is invalidated.

Parameters

<i>adr</i>	32bit address of the instruction to be invalidated.
------------	---

Returns

None.

Note

The bottom 5 bits are set to 0, forced by architecture.

void Xil_L1ICacheInvalidateRange (u32 *adr*, u32 *len*)

Invalidate the level 1 instruction cache for the given address range. If the instructions specified by the address range are cached by the instruction cache, the cacheline containing those bytes are invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be invalidated.
<i>len</i>	Length of the range to be invalidated in bytes.

Returns

None.

Note

None.



void Xil_L2CacheEnable (void)

Enable the L2 cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_L2CacheDisable (void)

Disable the L2 cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_L2CacheInvalidate (void)

Invalidate the entire level 2 cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_L2CacheInvalidateLine (u32 *adr*)

Invalidate a level 2 cache line. If the byte specified by the address (*adr*) is cached by the Data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the modified contents are lost and are NOT written to system memory before the line is invalidated.

Parameters

<i>adr</i>	32bit address of the data/instruction to be invalidated.
------------	--

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

void Xil_L2CacheInvalidateRange (u32 *adr*, u32 *len*)

Invalidate the level 2 cache for the given address range. If the bytes specified by the address range are cached by the L2 cache, the cacheline containing those bytes are invalidated. If the cachelines are modified (dirty), the modified contents are lost and are NOT written to system memory before the lines are invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be invalidated.
<i>len</i>	Length of the range to be invalidated in bytes.

Returns

None.

Note

None.

void Xil_L2CacheFlush (void)

Flush the entire level 2 cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_L2CacheFlushLine (u32 *adr*)

Flush a level 2 cache line. If the byte specified by the address (*adr*) is cached by the L2 cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the entire contents of the cacheline are written to system memory before the line is invalidated.

Parameters

<i>adr</i>	32bit address of the data/instruction to be flushed.
------------	--

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

void Xil_L2CacheFlushRange (u32 *adr*, u32 *len*)

Flush the level 2 cache for the given address range. If the bytes specified by the address range are cached by the L2 cache, the cacheline containing those bytes are invalidated. If the cachelines are modified (dirty), they are written to the system memory before the lines are invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be flushed.
<i>len</i>	Length of the range to be flushed in bytes.

Returns

None.

Note

None.

void Xil_L2CacheStoreLine (u32 *adr*)

Store a level 2 cache line. If the byte specified by the address (*adr*) is cached by the L2 cache and the cacheline is modified (dirty), the entire contents of the cacheline are written to system memory. After the store completes, the cacheline is marked as unmodified (not dirty).

Parameters

<i>adr</i>	32bit address of the data/instruction to be stored.
------------	---

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

Cortex A9 Processor MMU Functions

Overview

MMU functions equip users to enable MMU, disable MMU and modify default memory attributes of MMU table as per the need.

Functions

- void [Xil_SetTlbAttributes](#) (INTPTR *Addr*, u32 *attrib*)
- void [Xil_EnableMMU](#) (void)
- void [Xil_DisableMMU](#) (void)

Function Documentation

void Xil_SetTlbAttributes (INTPTR *Addr*, u32 *attrib*)

This function sets the memory attributes for a section covering 1MB of memory in the translation table.

Parameters

<i>Addr</i>	32-bit address for which memory attributes need to be set.
<i>attrib</i>	Attribute for the given memory region. <code>xil_mmu.h</code> contains definitions of commonly used memory attributes which can be utilized for this function.

Returns

None.

Note

The MMU or D-cache does not need to be disabled before changing a translation table entry.

void Xil_EnableMMU (void)

Enable MMU for cortex A9 processor. This function invalidates the instruction and data caches, and then enables MMU.

Parameters

None.	
-------	--

Returns

None.

void Xil_DisableMMU (void)

Disable MMU for Cortex A9 processors. This function invalidates the TLBs, Branch Predictor Array and flushed the D Caches before disabling the MMU.

Parameters

None.	
-------	--

Returns

None.

Note

When the MMU is disabled, all the memory accesses are treated as strongly ordered.

Cortex A9 Time Functions

Overview

xtime_l.h provides access to the 64-bit Global Counter in the PMU. This counter increases by one at every two processor cycles. These functions can be used to get/set time in the global timer.

Functions

- void [XTime_SetTime](#) (XTime Xtime_Global)
- void [XTime_GetTime](#) (XTime *Xtime_Global)

Function Documentation

void XTime_SetTime (XTime *Xtime_Global*)

Set the time in the Global Timer Counter Register.

Parameters

<i>Xtime_Global</i>	64-bit Value to be written to the Global Timer Counter Register.
---------------------	--

Returns

None.

Note

When this function is called by any one processor in a multi- processor environment, reference time will reset/lost for all processors.

void XTime_GetTime (XTime * *Xtime_Global*)

Get the time from the Global Timer Counter Register.

Parameters

<i>Xtime_Global</i>	Pointer to the 64-bit location which will be updated with the current timer value.
---------------------	--

Returns

None.

Note

None.

Cortex A9 Event Counter Function

Overview

Cortex A9 event counter functions can be utilized to configure and control the Cortex-A9 performance monitor events.

Cortex-A9 performance monitor has six event counters which can be used to count a variety of events described in Cortex-A9 TRM. `xpm_counter.h` defines configurations `XPM_CNTRCFGx` which can be used to program the event counters to count a set of events.

Note

It doesn't handle the Cortex-A9 cycle counter, as the cycle counter is being used for time keeping.

Functions

- void [Xpm_SetEvents](#) (s32 PmcrCfg)
- void [Xpm_GetEventCounters](#) (u32 *PmCtrValue)

Function Documentation

void Xpm_SetEvents (s32 *PmcrCfg*)

This function configures the Cortex A9 event counters controller, with the event codes, in a configuration selected by the user and enables the counters.

Parameters

<i>PmcrCfg</i>	Configuration value based on which the event counters are configured. XPM_CNTRCFG* values defined in xpm_counter.h can be utilized for setting configuration.
----------------	---

Returns

None.

Note

None.

void Xpm_GetEventCounters (u32 * *PmCtrValue*)

This function disables the event counters and returns the counter values.

Parameters

<i>PmCtrValue</i>	Pointer to an array of type u32 PmCtrValue[6]. It is an output parameter which is used to return the PM counter values.
-------------------	---

Returns

None.

Note

None.

PL310 L2 Event Counters Functions

Overview

xl2cc_counter.h contains APIs for configuring and controlling the event counters in PL310 L2 cache controller. PL310 has two event counters which can be used to count variety of events like DRHIT, DRREQ, DWHIT,

DWREQ, etc. xl2cc_counter.h contains definitions for different configurations which can be used for the event counters to count a set of events.

Functions

- void [XL2cc_EventCtrlInit](#) (s32 Event0, s32 Event1)
- void [XL2cc_EventCtrStart](#) (void)
- void [XL2cc_EventCtrStop](#) (u32 *EveCtr0, u32 *EveCtr1)

Function Documentation

void XL2cc_EventCtrlInit (s32 *Event0*, s32 *Event1*)

This function initializes the event counters in L2 Cache controller with a set of event codes specified by the user.

Parameters

<i>Event0</i>	Event code for counter 0.
<i>Event1</i>	Event code for counter 1.

Returns

None.

Note

The definitions for event codes XL2CC_* can be found in xl2cc_counter.h.

void XL2cc_EventCtrStart (void)

This function starts the event counters in L2 Cache controller.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void XL2cc_EventCtrStop (u32 * *EveCtr0*, u32 * *EveCtr1*)

This function disables the event counters in L2 Cache controller, saves the counter values and resets the counters.

Parameters

<i>EveCtr0</i>	Output parameter which is used to return the value in event counter 0. <i>EveCtr1</i> : Output parameter which is used to return the value in event counter 1.
----------------	--

Returns

None.

Note

None.

Cortex A9 Processor and pl310 Errata Support

Overview

Various ARM errata are handled in the standalone BSP. The implementation for errata handling follows ARM guidelines and is based on the open source Linux support for these errata.

Note

The errata handling is enabled by default. To disable handling of all the errata globally, un-define the macro `ENABLE_ARM_ERRATA` in `xil_errata.h`. To disable errata on a per-erratum basis, un-define relevant macros in `xil_errata.h`.

errata_definitions

The errata conditions handled in the standalone BSP are listed below

- #define **ENABLE_ARM_ERRATA**
- #define [CONFIG_ARM_ERRATA_742230](#)
- #define [CONFIG_ARM_ERRATA_743622](#)
- #define [CONFIG_ARM_ERRATA_775420](#)
- #define [CONFIG_ARM_ERRATA_794073](#)
- #define [CONFIG_PL310_ERRATA_588369](#)
- #define [CONFIG_PL310_ERRATA_727915](#)
- #define [CONFIG_PL310_ERRATA_753970](#)

Macro Definition Documentation

#define CONFIG_ARM_ERRATA_742230

Errata No: 742230 Description: DMB operation may be faulty

#define CONFIG_ARM_ERRATA_743622

Errata No: 743622 Description: Faulty hazard checking in the Store Buffer may lead to data corruption.

#define CONFIG_ARM_ERRATA_775420

Errata No: 775420 Description: A data cache maintenance operation which aborts, might lead to deadlock

#define CONFIG_ARM_ERRATA_794073

Errata No: 794073 Description: Speculative instruction fetches with MMU disabled might not comply with architectural requirements

#define CONFIG_PL310_ERRATA_588369

PL310 L2 Cache Errata Errata No: 588369 Description: Clean & Invalidate maintenance operations do not invalidate clean lines

#define CONFIG_PL310_ERRATA_727915

Errata No: 727915 Description: Background Clean and Invalidate by Way operation can cause data corruption

#define CONFIG_PL310_ERRATA_753970

Errata No: 753970 Description: Cache sync operation may be faulty

Cortex A9 Processor Specific Include Files

The `xpseudo_asm.h` includes `xreg_cortexa9.h` and `xpseudo_asm_gcc.h`.

The `xreg_cortexa9.h` file contains definitions for inline assembler code. It provides inline definitions for Cortex A9 GPRs, SPRs, MPE registers, co-processor registers and Debug registers.

The `xpseudo_asm_gcc.h` contains the definitions for the most often used inline assembler instructions, available as macros. These can be very useful for tasks such as setting or getting special purpose registers, synchronization, or cache manipulation etc. These inline assembler instructions can be used from drivers and user applications written in C.

Cortex A53 32-bit Processor API

Overview

Cortex-A53 standalone BSP contains two separate BSPs for 32-bit mode and 64-bit mode. The 32-bit mode of cortex-A53 is compatible with ARMv7-A architecture.

Modules

- [Cortex A53 32-bit Processor Boot Code](#)
 - [Cortex A53 32-bit Processor Cache Functions](#)
 - [Cortex A53 32-bit Processor MMU Handling](#)
 - [Cortex A53 32-bit Mode Time Functions](#)
 - [Cortex A53 32-bit Processor Specific Include Files](#)
-

Cortex A53 32-bit Processor Boot Code

Overview

The boot .S file contains a minimal set of code for transferring control from the processor reset location to the start of the application. The boot code performs minimum configuration which is required for an application to run starting from processor's reset state. Below is a sequence illustrating what all configuration is performed before control reaches to main function.

1. Program vector table base for exception handling
2. Invalidate instruction cache, data cache and TLBs
3. Program stack pointer for various modes (IRQ, FIQ, supervisor, undefine, abort, system)
4. Program counter frequency
5. Configure MMU with short descriptor translation table format and program base address of translation table
6. Enable data cache, instruction cache and MMU
7. Transfer control to _start which clears BSS sections and runs global constructor before jumping to main application

The `translation_table.S` file contains a static page table required by MMU for cortex-A53. This translation table is flat mapped (input address = output address) with default memory attributes defined for zynq ultrascale+ architecture. It utilizes short descriptor translation table format with each section defining 1MB of memory. The overview of translation table memory attributes is described below.

	Memory Range	Definition in Translation Table	Note
DDR	0x00000000 - 0x7FFFFFFF	Normal write-back Cacheable	For a system where DDR is less than 2GB, region after DDR and before PL is marked as undefined/reserved in translation table
PL	0x80000000 - 0xBFFFFFFF	Strongly Ordered	
QSPI, lower PCIe	0xC0000000 - 0xEFFFFFFF	Device Memory	
Reserved	0xF0000000 - 0xF7FFFFFF	Unassigned	
STM Coresight	0xF8000000 - 0xF8FFFFFF	Device Memory	
GIC	0xF9000000 - 0xF90FFFFF	Device memory	
Reserved	0xF9100000 - 0xFCFFFFFF	Unassigned	
FPS, LPS slaves	0xFD000000 - 0xFFBFFFFFF	Device memory	
CSU, PMU	0xFFC00000 - 0xFFDFFFFFF	Device Memory	This region contains CSU and PMU memory which are marked as Device since it is less than 1MB and falls in a region with device memory
TCM, OCM	0xFFE00000 - 0xFFFFFFFF	Normal write-back cacheable	

Cortex A53 32-bit Processor Cache Functions

Overview

Cache functions provide access to cache related operations such as flush and invalidate for instruction and data caches. It gives option to perform the cache operations on a single cacheline, a range of memory and an entire cache.

Functions

- void [Xil_DCacheEnable](#) (void)
- void [Xil_DCacheDisable](#) (void)
- void [Xil_DCacheInvalidate](#) (void)
- void [Xil_DCacheInvalidateRange](#) (INTPTR adr, u32 len)
- void [Xil_DCacheInvalidateLine](#) (u32 adr)
- void [Xil_DCacheFlush](#) (void)
- void [Xil_DCacheFlushLine](#) (u32 adr)
- void [Xil_ICacheEnable](#) (void)
- void [Xil_ICacheDisable](#) (void)
- void [Xil_ICacheInvalidate](#) (void)
- void [Xil_ICacheInvalidateRange](#) (INTPTR adr, u32 len)
- void [Xil_ICacheInvalidateLine](#) (u32 adr)

Function Documentation

void Xil_DCacheEnable (void)

Enable the Data cache.

Parameters

None.	
-------	--

Returns

None.

Note

None.

void Xil_DCacheDisable (void)

Disable the Data cache.

Parameters

None.	
-------	--

Returns

None.

Note

None.

void Xil_DCacheInvalidate (void)

Invalidate the Data cache. The contents present in the data cache are cleaned and invalidated.

Parameters

None.	
-------	--

Returns

None.

Note

In Cortex-A53, functionality to simply invalide the cachelines is not present. Such operations are a problem for an environment that supports virtualisation. It would allow one OS to invalidate a line belonging to another OS. This could lead to the other OS crashing because of the loss of essential data. Hence, such operations are promoted to clean and invalidate to avoid such corruption.

void Xil_DCacheInvalidateRange (INTPTR *adr*, u32 *len*)

Invalidate the Data cache for the given address range. The cachelines present in the adderss range are cleaned and invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be invalidated.
<i>len</i>	Length of the range to be invalidated in bytes.

Returns

None.

Note

In Cortex-A53, functionality to simply invalidate the cachelines is not present. Such operations are a problem for an environment that supports virtualisation. It would allow one OS to invalidate a line belonging to another OS. This could lead to the other OS crashing because of the loss of essential data. Hence, such operations are promoted to clean and invalidate to avoid such corruption.

void Xil_DCacheInvalidateLine (u32 *adr*)

Invalidate a Data cache line. The cacheline is cleaned and invalidated.

Parameters

<i>adr</i>	32 bit address of the data to be invalidated.
------------	---

Returns

None.

Note

In Cortex-A53, functionality to simply invalidate the cachelines is not present. Such operations are a problem for an environment that supports virtualisation. It would allow one OS to invalidate a line belonging to another OS. This could lead to the other OS crashing because of the loss of essential data. Hence, such operations are promoted to clean and invalidate to avoid such corruption.

void Xil_DCacheFlush (void)

Flush the Data cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_DCacheFlushLine (u32 *adr*)

Flush a Data cache line. If the byte specified by the address (*adr*) is cached by the Data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the entire contents of the cacheline are written to system memory before the line is invalidated.

Parameters

<i>adr</i>	32bit address of the data to be flushed.
------------	--

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

void Xil_ICacheEnable (void)

Enable the instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_ICacheDisable (void)

Disable the instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_ICacheInvalidate (void)

Invalidate the entire instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_ICacheInvalidateRange (INTPTR *adr*, u32 *len*)

Invalidate the instruction cache for the given address range. If the instructions specified by the address range are cached by the instruction cache, the cachelines containing those instructions are invalidated.

Parameters

<i>adr</i>	32bit start address of the range to be invalidated.
<i>len</i>	Length of the range to be invalidated in bytes.

Returns

None.

Note

None.

void Xil_ICacheInvalidateLine (u32 *adr*)

Invalidate an instruction cache line. If the instruction specified by the address is cached by the instruction cache, the cacheline containing that instruction is invalidated.

Parameters

<i>adr</i>	32bit address of the instruction to be invalidated..
------------	--

Returns

None.

Note

The bottom 4 bits are set to 0, forced by architecture.

Cortex A53 32-bit Processor MMU Handling

Overview

MMU functions equip users to enable MMU, disable MMU and modify default memory attributes of MMU table as per the need.

Functions

- void [Xil_SetTlbAttributes](#) (INTPTR Addr, u32 attrib)
- void [Xil_EnableMMU](#) (void)
- void [Xil_DisableMMU](#) (void)

Function Documentation

void Xil_SetTlbAttributes (INTPTR Addr, u32 attrib)

This function sets the memory attributes for a section covering 1MB of memory in the translation table.

Parameters

<i>Addr</i>	32-bit address for which the attributes need to be set.
<i>attrib</i>	Attributes for the specified memory region. xil_mmu.h contains commonly used memory attributes definitions which can be utilized for this function.

Returns

None.

Note

The MMU or D-cache does not need to be disabled before changing a translation table entry.

void Xil_EnableMMU (void)

Enable MMU for Cortex-A53 processor in 32bit mode. This function invalidates the instruction and data caches before enabling MMU.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

void Xil_DisableMMU (void)

Disable MMU for Cortex A53 processors in 32bit mode. This function invalidates the TLBs, Branch Predictor Array and flushed the data cache before disabling the MMU.

Parameters

None.	
-------	--

Returns

None.

Note

When the MMU is disabled, all the memory accesses are treated as strongly ordered.

Cortex A53 32-bit Mode Time Functions

Overview

The `xtime_l.c` file and corresponding `xtime_l.h` include file provide access to the 64-bit generic counter in Cortex-A53. The `sleep.c`, `usleep.c` file and the corresponding `sleep.h` include file implement sleep functions. Sleep functions are implemented as busy loops.

Functions

- void [XTime_StartTimer](#) (void)
- void [XTime_SetTime](#) (XTime Xtime_Global)
- void [XTime_GetTime](#) (XTime *Xtime_Global)

Function Documentation

void XTime_StartTimer (void)

Start the 64-bit physical timer counter.

Parameters

None.	
-------	--

Returns

None.

Note

The timer is initialized only if it is disabled. If the timer is already running this function does not perform any operation.

void XTime_SetTime (XTime *Xtime_Global*)

Timer of A53 runs continuously and the time can not be set as desired. This API doesn't contain anything. It is defined to have uniformity across platforms.

Parameters

<i>Xtime_Global</i>	64bit Value to be written to the Global Timer Counter Register.
---------------------	---

Returns

None.

Note

None.

void XTime_GetTime (XTime * *Xtime_Global*)

Get the time from the physical timer counter register.

Parameters

<i>Xtime_Global</i>	Pointer to the 64-bit location to be updated with the current value in physical timer counter.
---------------------	--

Returns

None.

Note

None.

Cortex A53 32-bit Processor Specific Include Files

The `xreg_cortexa53.h` file contains definitions for inline assembler code. It provides inline definitions for Cortex A53 GPRs, SPRs and floating point registers.

The `xpseudo_asm_gcc.h` contains the definitions for the most often used inline assembler instructions, available as macros. These can be very useful for tasks such as setting or getting special purpose registers, synchronization, or cache manipulation. These inline assembler instructions can be used from drivers and user applications written in C.

Cortex A53 64-bit Processor API

Overview

Cortex-A53 standalone BSP contains two separate BSPs for 32-bit mode and 64-bit mode. The 64-bit mode of cortex-A53 contains ARMv8-A architecture. This section provides a linked summary and detailed descriptions of the Cortex A53 64-bit Processor APIs.

Modules

- [Cortex A53 64-bit Processor Boot Code](#)
 - [Cortex A53 64-bit Processor Cache Functions](#)
 - [Cortex A53 64-bit Processor MMU Handling](#)
 - [Cortex A53 64-bit Mode Time Functions](#)
 - [Cortex A53 64-bit Processor Specific Include Files](#)
-

Cortex A53 64-bit Processor Boot Code

Overview

The boot.S file contains a minimal set of code for transferring control from the processor reset location to the start of the application. The boot code performs minimum configuration which is required for an application to run starting from processor's reset state. Cortex-A53 starts execution from EL3 and currently application is also run from EL3. Below is a sequence illustrating what all configuration is performed before control reaches to main function.

1. Program vector table base for exception handling
2. Set reset vector table base address
3. Program stack pointer for EL3
4. Routing of interrupts to EL3
5. Enable ECC protection
6. Program generic counter frequency
7. Invalidate instruction cache, data cache and TLBs

8. Configure MMU registers and program base address of translation table
9. Transfer control to `_start` which clears BSS sections and runs global constructor before jumping to main application

Cortex A53 64-bit Processor Cache Functions

Overview

Cache functions provide access to cache related operations such as flush and invalidate for instruction and data caches. It gives option to perform the cache operations on a single cacheline, a range of memory and an entire cache.

Functions

- void [Xil_DCacheEnable](#) (void)
- void [Xil_DCacheDisable](#) (void)
- void [Xil_DCacheInvalidate](#) (void)
- void [Xil_DCacheInvalidateRange](#) (INTPTR adr, INTPTR len)
- void [Xil_DCacheInvalidateLine](#) (INTPTR adr)
- void [Xil_DCacheFlush](#) (void)
- void [Xil_DCacheFlushLine](#) (INTPTR adr)
- void [Xil_ICacheEnable](#) (void)
- void [Xil_ICacheDisable](#) (void)
- void [Xil_ICacheInvalidate](#) (void)
- void [Xil_ICacheInvalidateRange](#) (INTPTR adr, INTPTR len)
- void [Xil_ICacheInvalidateLine](#) (INTPTR adr)

Function Documentation

void Xil_DCacheEnable (void)

Enable the Data cache.

Parameters

None.	
-------	--

Returns

None.

Note

None.

void Xil_DCacheDisable (void)

Disable the Data cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_DCacheInvalidate (void)

Invalidate the Data cache. The contents present in the cache are cleaned and invalidated.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

In Cortex-A53, functionality to simply invalidate the cachelines is not present. Such operations are a problem for an environment that supports virtualisation. It would allow one OS to invalidate a line belonging to another OS. This could lead to the other OS crashing because of the loss of essential data. Hence, such operations are promoted to clean and invalidate which avoids such corruption.

void Xil_DCacheInvalidateRange (INTPTR *adr*, INTPTR *len*)

Invalidate the Data cache for the given address range. The cachelines present in the address range are cleaned and invalidated.

Parameters

<i>adr</i>	64bit start address of the range to be invalidated.
<i>len</i>	Length of the range to be invalidated in bytes.

Returns

None.

Note

In Cortex-A53, functionality to simply invalidate the cachelines is not present. Such operations are a problem for an environment that supports virtualisation. It would allow one OS to invalidate a line belonging to another OS. This could lead to the other OS crashing because of the loss of essential data. Hence, such operations are promoted to clean and invalidate which avoids such corruption.

void Xil_DCacheInvalidateLine (INTPTR *adr*)

Invalidate a Data cache line. The cacheline is cleaned and invalidated.

Parameters

<i>adr</i>	64bit address of the data to be flushed.
------------	--

Returns

None.

Note

In Cortex-A53, functionality to simply invalidate the cachelines is not present. Such operations are a problem for an environment that supports virtualisation. It would allow one OS to invalidate a line belonging to another OS. This could lead to the other OS crashing because of the loss of essential data. Hence, such operations are promoted to clean and invalidate which avoids such corruption.

void Xil_DCacheFlush (void)

Flush the Data cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_DCacheFlushLine (INTPTR *adr*)

Flush a Data cache line. If the byte specified by the address (*adr*) is cached by the Data cache, the cacheline containing that byte is invalidated. If the cacheline is modified (dirty), the entire contents of the cacheline are written to system memory before the line is invalidated.

Parameters

<i>adr</i>	64bit address of the data to be flushed.
------------	--

Returns

None.

Note

The bottom 6 bits are set to 0, forced by architecture.

void Xil_ICacheEnable (void)

Enable the instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_ICacheDisable (void)

Disable the instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_ICacheInvalidate (void)

Invalidate the entire instruction cache.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

None.

void Xil_ICacheInvalidateRange (INTPTR *adr*, INTPTR *len*)

Invalidate the instruction cache for the given address range. If the instructions specified by the address range are cached by the instruction cache, the cachelines containing those instructions are invalidated.

Parameters

<i>adr</i>	64bit start address of the range to be invalidated.
<i>len</i>	Length of the range to be invalidated in bytes.

Returns

None.

Note

None.

void Xil_ICacheInvalidateLine (INTPTR *adr*)

Invalidate an instruction cache line. If the instruction specified by the parameter *adr* is cached by the instruction cache, the cacheline containing that instruction is invalidated.

Parameters

<i>adr</i>	64bit address of the instruction to be invalidated.
------------	---

Returns

None.

Note

The bottom 6 bits are set to 0, forced by architecture.

Cortex A53 64-bit Processor MMU Handling

Overview

MMU function equip users to modify default memory attributes of MMU table as per the need.

Functions

- void [Xil_SetTlbAttributes](#) (INTPTR Addr, u64 attrib)

Function Documentation

void Xil_SetTlbAttributes (INTPTR Addr, u64 attrib)

brief It sets the memory attributes for a section, in the translation table. If the address (defined by Addr) is less than 4GB, the memory attribute(attrib) is set for a section of 2MB memory. If the address (defined by Addr) is greater than 4GB, the memory attribute (attrib) is set for a section of 1GB memory.

Parameters

<i>Addr</i>	64-bit address for which attributes are to be set.
<i>attrib</i>	Attribute for the specified memory region. xil_mmu.h contains commonly used memory attributes definitions which can be utilized for this function.

Returns

None.

Note

The MMU and D-cache need not be disabled before changing an translation table attribute.

Cortex A53 64-bit Mode Time Functions

Overview

The xtime_l.c file and corresponding xtime_l.h include file provide access to the 64-bit generic counter in Cortex-A53. The sleep.c, usleep.c file and the corresponding sleep.h include file implement sleep functions. Sleep functions are implemented as busy loops.

Functions

- void [XTime_StartTimer](#) (void)
- void [XTime_SetTime](#) (XTime Xtime_Global)
- void [XTime_GetTime](#) (XTime *Xtime_Global)

Function Documentation

void XTime_StartTimer (void)

Start the 64-bit physical timer counter.

Parameters

<i>None.</i>	
--------------	--

Returns

None.

Note

The timer is initialized only if it is disabled. If the timer is already running this function does not perform any operation.

void XTime_SetTime (XTime *Xtime_Global*)

Timer of A53 runs continuously and the time can not be set as desired. This API doesn't contain anything. It is defined to have uniformity across platforms.

Parameters

<i>Xtime_Global</i>	64bit value to be written to the physical timer counter register.
---------------------	---

Returns

None.

Note

None.

void XTime_GetTime (XTime * *Xtime_Global*)

Get the time from the physical timer counter register.

Parameters

<i>Xtime_Global</i>	Pointer to the 64-bit location to be updated with the current value of physical timer counter register.
---------------------	---

Returns

None.

Note

None.

Cortex A53 64-bit Processor Specific Include Files

The `xreg_cortexa53.h` file contains definitions for inline assembler code. It provides inline definitions for Cortex A53 GPRs, SPRs and floating point registers.

The `xpseudo_asm_gcc.h` contains the definitions for the most often used inline assembler instructions, available as macros. These can be very useful for tasks such as setting or getting special purpose registers, synchronization, or cache manipulation. These inline assembler instructions can be used from drivers and user applications written in C.

Appendix D:

XiIMFS Library v2.3



Overview

The XiLMFS library provides the capability to manage program memory in the form of file handles. You can create directories and have files within each directory. The file system can be accessed from the high-level C language through function calls specific to the file system.

XiMFS Library API

Overview

This chapter provides a linked summary and detailed descriptions of the XiMSF library APIs.

Functions

- void [mfs_init_fs](#) (int numbytes, char *address, int init_type)
- void [mfs_init_genimage](#) (int numbytes, char *address, int init_type)
- int [mfs_change_dir](#) (const char *newdir)
- int [mfs_delete_file](#) (char *filename)
- int [mfs_create_dir](#) (char *newdir)
- int [mfs_delete_dir](#) (char *newdir)
- int [mfs_rename_file](#) (char *from_file, char *to_file)
- int [mfs_exists_file](#) (char *filename)
- int [mfs_get_current_dir_name](#) (char *dirname)
- int [mfs_get_usage](#) (int *num_blocks_used, int *num_blocks_free)
- int [mfs_dir_open](#) (const char *dirname)
- int [mfs_dir_close](#) (int fd)
- int [mfs_dir_read](#) (int fd, char **filename, int *filesize, int *filetype)
- int [mfs_file_open](#) (const char *filename, int mode)
- int [mfs_file_read](#) (int fd, char *buf, int buflen)
- int [mfs_file_write](#) (int fd, const char *buf, int buflen)
- int [mfs_file_close](#) (int fd)
- long [mfs_file_lseek](#) (int fd, long offset, int whence)
- int [mfs_ls](#) ()
- int [mfs_ls_r](#) (int recurse)
- int [mfs_cat](#) (char *filename)
- int [mfs_copy_stdin_to_file](#) (char *filename)
- int [mfs_file_copy](#) (char *from_file, char *to_file)

Function Documentation

```
void mfs_init_fs ( int numbytes, char * address, int init_type )
```

Initialize the file system.

This function must be called before any file system operations. Use [mfs_init_genimage\(\)](#) instead of this function for initializing with file images generated by mfs-gen.

Parameters

<i>numbytes</i>	Number of bytes allocated or reserved for this file system.
<i>address</i>	Starting address of the memory block. address must be word aligned (4 byte boundary).
<i>init_type</i>	• MFSINIT_NEW creates a new, empty file system for read/write • MFSINIT_IMAGE initializes a file system whose data has been previously loaded into memory at the base address. • MFSINIT_ROM_IMAGE initializes a Read-Only file system whose data has been previously loaded into memory at the base address.

```
void mfs_init_genimage ( int numbytes, char * address, int init_type )
```

Initialize the file system with a file image generated by mfs-gen.

This function must be called before any file system operations. Use [mfs_init_fs\(\)](#) instead of this function for other initialization.

Parameters

<i>numbytes</i>	Number of bytes allocated or reserved for this file system.
<i>address</i>	Starting address of the memory block. address must be word aligned (4 byte boundary).
<i>init_type</i>	• MFSINIT_IMAGE initializes a file system whose data has been previously loaded into memory at the base address. • MFSINIT_ROM_IMAGE initializes a Read-Only file system whose data has been previously loaded into memory at the base address.

int mfs_change_dir (const char * *newdir*)

Modify global mfs_current_dir to index of newdir if it exists.
mfs_current_dir is not modified otherwise.

Parameters

<i>newdir</i>	is the name of the new directory
---------------	----------------------------------

Returns

1 for success and 0 for failure

int mfs_delete_file (char * *filename*)

Delete a file from directory.



WARNING: This function does not completely free up the directory space used by the file. Repeated calls to create and delete files can cause the file system to run out of space.

Parameters

<i>filename</i>	Name of the file to be deleted. Delete the data blocks corresponding to the file and then delete the file entry from its directory.
-----------------	---

Returns

1 on success, 0 on failure

Note

Delete will not work on a directory unless the directory is empty.

int mfs_create_dir (char * *newdir*)

Create a new empty directory called newdir inside the current directory.

Parameters

<i>newdir</i>	is the name of the directory
---------------	------------------------------

Returns

index of new directory in the file system on success. 0 on failure

int mfs_delete_dir (char * *newdir*)

Delete the directory named *newdir* if it exists, and is empty.

Parameters

<i>newdir</i>	is the name of the directory
---------------	------------------------------

Returns

Index of new directory in the file system on success. 0 on failure.

int mfs_rename_file (char * *from_file*, char * *to_file*)

Rename *from_file* to *to_file*.

Rename works for directories as well as files. Function fails if *to_file* already exists. works for dirs as well as files cannot rename to something that already exists

Parameters

<i>from_file</i>	
<i>to_file</i>	

Returns

1 on success, 0 on failure

int mfs_exists_file (char * *filename*)

check if a file exists

Parameters

<i>filename</i>	is the name of the file
-----------------	-------------------------

Returns

- 0 if *filename* is not a file in the current directory
- 1 if *filename* is a file in the current directory
- 2 if *filename* is a directory in the current directory

int mfs_get_current_dir_name (char * *dirname*)

get the name of the current directory

Parameters

<i>dirname</i>	= pre_allocated buffer of at least MFS_MAX_FILENAME_SIZE+1 chars The directory name is copied to this buffer
----------------	--

Returns

1 if success, 0 if failure

int mfs_get_usage (int * *num_blocks_used*, int * *num_blocks_free*)

get the number of used blocks and the number of free blocks in the file system through pointers

Parameters

<i>num_blocks_used</i>	
<i>num_blocks_free</i>	the return value is 1 (for success) and 0 for failure to obtain the numbers

int mfs_dir_open (const char * *dirname*)

open a directory for reading each subsequent call to [mfs_dir_read\(\)](#) returns one directory entry until end of directory

Parameters

<i>dirname</i>	is the name of the directory to open
----------------	--------------------------------------

Returns

index of dir in array mfs_open_files or -1

int mfs_dir_close (int *fd*)

close a directory - same as closing a file

Parameters

<i>fd</i>	is the descriptor of the directory to close
-----------	---

Returns

1 on success, 0 otherwise

int mfs_dir_read (int *fd*, char ** *filename*, int * *filesize*, int * *filetype*)

read values from the next valid directory entry The last 3 parameters are output values

Parameters

<i>fd</i>	is the file descriptor for an open directory file
<i>filename</i>	is a pointer to the filename within the MFS itself
<i>filesize</i>	is the size in bytes for a regular file or the number of entries in a directory
<i>filetype</i>	is MFS_BLOCK_TYPE_FILE or MFS_BLOCK_TYPE_DIR

Returns

1 for success and 0 for failure or end of dir

int mfs_file_open (const char * *filename*, int *mode*)

open a file

Parameters

<i>filename</i>	is the name of the file to open
<i>mode</i>	is MFS_MODE_READ or MFS_MODE_WRITE or MFS_MODE_CREATE this function should be used for FILEs and not DIRs no error checking (is this FILE and not DIR?) is done for MFS_MODE_READ MFS_MODE_CREATE automatically creates a FILE and not a DIR MFS_MODE_WRITE fails if the specified file is a DIR

Returns

index of file in array mfs_open_files or -1

int mfs_file_read (int *fd*, char * *buf*, int *buflen*)

read characters to a file

Parameters

<i>fd</i>	is a descriptor for the file from which the characters are read
<i>buf</i>	is a pre allocated buffer that will contain the read characters
<i>buflen</i>	is the number of characters from buf to be read fd should be a valid index in mfs_open_files array Works only if fd points to a file and not a dir buf should be a pointer to a pre-allocated buffer of size buflen or more buflen chars are read and placed in buf if fewer than buflen chars are available then only that many chars are read

Returns

num bytes read or 0 for error=no bytes read

int mfs_file_write (int *fd*, const char * *buf*, int *buflen*)

write characters to a file

Parameters

<i>fd</i>	is a descriptor for the file to which the characters are written
<i>buf</i>	is a buffer containing the characters to be written out
<i>buflen</i>	is the number of characters from buf to be written out fd should be a valid index in mfs_open_files array buf should be a pointer to a pre-allocated buffer of size buflen or more buflen chars are read from buf and written to 1 or more blocks of the file

Returns

1 for success or 0 for error=unable to write to file

int mfs_file_close (int *fd*)

close an open file and recover the file table entry in mfs_open_files corresponding to the fd if the fd is not valid, return 0 fd is not valid if the index in mfs_open_files is out of range, or if the corresponding entry is not an open file

Parameters

<i>fd</i>	is the file descriptor for the file to be closed
-----------	--

Returns

1 on success, 0 otherwise

long mfs_file_lseek (int *fd*, long *offset*, int *whence*)

seek to a given offset within the file

Parameters

<i>fd</i>	should be a valid file descriptor for an open file
<i>whence</i>	is one of MFS_SEEK_SET, MFS_SEEK_CUR or MFS_SEEK_END
<i>offset</i>	is the offset from the beginning, end or current position as specified by the whence parameter if MFS_SEEK_END is specified, the offset can be either 0 or negative otherwise offset should be positive or 0 it is an error to seek before beginning of file or after the end of file

Returns

-1 on failure, value of the offset from the beginning of the file, on success

int mfs_ls ()

list contents of current directory

Returns

1 on success and 0 on failure

int mfs_ls_r (int *recurse*)

recursive directory listing list the contents of current directory if any of the entries in the current directory is itself a directory, immediately enter that directory and call [mfs_ls_r\(\)](#) once again

Parameters

<i>recurse</i>	If parameter recurse is non zero continue recursing else stop recursing recurse=0 lists just the current directory recurse = -1 allows unlimited recursion recurse = n stops recursing at a depth of n
----------------	--

Returns

1 on success and 0 on failure

int mfs_cat (char * *filename*)

print the file to stdout

Parameters

<i>filename</i>	- file to print
-----------------	-----------------

Returns

1 on success, 0 on failure

int mfs_copy_stdin_to_file (char * *filename*)

copy from stdin to named file

Parameters

<i>filename</i>	- file to print
-----------------	-----------------

Returns

1 on success, 0 on failure

int mfs_file_copy (char * *from_file*, char * *to_file*)

copy from_file to to_file to_file is created new copy fails if to_file exists already copy fails is from_file or to_file cannot be opened

Parameters

<i>from_file</i>	
<i>to_file</i>	

Returns

1 on success, 0 on failure



Utility Functions

This chapter provides a summary and detailed descriptions of the utility functions that can be used along with the MFS.

These functions are defined in `mfs_filesys_util.c` and are declared in `xilmfs.h`. /**

Library Parameters in MSS File

A memory file system can be integrated with a system using the following snippet in the Microprocessor Software Specification (MSS) file.

```
BEGIN LIBRARY
parameter LIBRARY_NAME = xilmfs
parameter LIBRARY_VER = 2.3
parameter numbytes= 50000
parameter base_address = 0xffe00000
parameter init_type = MFSINIT_NEW
parameter need_utils = false END
```

The memory file system must be instantiated with the name xilmfs. The following table lists the libgen customization parameters.

Parameter	Description
LIBRARY_NAME	Specifies the library name. Default is xilmfs
LIBRARY_VER	Specifies the library version. Default is 2.3
numbytes	Number of bytes allocated for file system.
base_address	Starting address for file system memory.
init_type	Options are: MFSINIT_NEW (default) creates a new, empty file system. MFSINIT_ROM_IMAGE creates a file system based on a pre-loaded memory image loaded in memory of size numbytes at starting address base_address. This memory is considered read-only and modification of the file system is not allowed. MFS_INIT_IMAGE is similar to the previous option except that the file system can be modified, and the memory is readable and writable.

Parameter	Description
need_utils	true or false (default=false) If true, this causes <code>stdio.h</code> to be included from <code>mfs_config.h</code>. The functions described in Utility Functions require that you have defined <code>stdin</code> or <code>stdout</code>. Setting the <code>need_utils</code> to true causes <code>stdio.h</code> to be included.



WARNING: *The underlying software and hardware platforms must support `stdin` and `stdout` peripherals for these utility functions to compile and link correctly.*

Appendix E:

LwIP 2.1.1 Library v2.3

Introduction

The lwIP is an open source TCP/IP protocol suite available under the BSD license. The lwIP is a standalone stack; there are no operating systems dependencies, although it can be used along with operating systems. The lwIP provides two APIs for use by applications:

- RAW API: Provides access to the core lwIP stack.
- Socket API: Provides a BSD sockets style interface to the stack.

The `lwip211_v1_0` is an SDK library that is built on the open source lwIP library version 2.1.1. The `lwip211_v1_0` library provides adapters for the Ethernetlite (`axi_ethernetlite`), the TEMAC (`axi_ethernet`), and the Gigabit Ethernet controller and MAC (GigE) cores. The library can run on MicroBlaze™, ARM Cortex-A9, ARM Cortex-A53, and ARM Cortex-R5 processors. The Ethernetlite and TEMAC cores apply for MicroBlaze systems. The Gigabit Ethernet controller and MAC (GigE) core is applicable only for ARM Cortex-A9 system (Zynq®-7000 processor devices) and ARM Cortex-A53 & ARM Cortex-R5 system (Zynq® UltraScale+™ MPSoC).

Features

The lwIP provides support for the following protocols:

- Internet Protocol (IP)
- Internet Control Message Protocol (ICMP)
- User Datagram Protocol (UDP)
- TCP (Transmission Control Protocol (TCP)
- Address Resolution Protocol (ARP)
- Dynamic Host Configuration Protocol (DHCP)
- Internet Group Message Protocol (IGMP)

References

- lwIP wiki:
<http://lwip.scribblewiki.com>
- Xilinx® lwIP designs and application examples:
http://www.xilinx.com/support/documentation/application_notes/xapp1026.pdf
- lwIP examples using RAW and Socket APIs:
<http://savannah.nongnu.org/projects/lwip/>
- FreeRTOS Port for Zynq is available for download from the [FreeRTOS](#) website

Using lwIP

Overview

The following sections detail the hardware and software steps for using lwIP for networking. The key steps are:

1. Creating a hardware system containing the processor, ethernet core, and a timer. The timer and ethernet interrupts must be connected to the processor using an interrupt controller.
2. Configuring `lwip211_v1_0` to be a part of the software platform. For operating with lwIP socket API, the Xilkernel library or FreeRTOS BSP is a prerequisite. See the Note below.

Note

The Xilkernel library is available only for MicroBlaze systems. For Cortex-A9 based systems (Zynq) and Cortex-A53 or Cortex-R5 based systems (Zynq® UltraScale™+ MPSoC), there is no support for Xilkernel. Instead, use FreeRTOS. A FreeRTOS BSP is available for Zynq systems and must be included for using lwIP socket API. The FreeRTOS BSP for Zynq is available for download from the the [FreeRTOS][freertos] website.

Setting up the Hardware System

This chapter describes the hardware configurations supported by lwIP. The key components of the hardware system include:

- **Processor:** Either a MicroBlaze™ or a Cortex-A9 or a Cortex-A53 or a Cortex-R5 processor. The Cortex-A9 processor applies to Zynq systems. The Cortex-A53 and Cortex-R5 processors apply to Zynq UltraScale+ MPSoC systems.
- **MAC:** LwIP supports `axi_etherlite`, `axi_ethernet`, and Gigabit Ethernet controller and MAC (GigE) cores.
- **Timer:** to maintain TCP timers, lwIP raw API based applications require that certain functions are called at periodic intervals by the application. An application can do this by registering an interrupt handler with a timer.
- **DMA:** For `axi_ethernet` based systems, the `axi_ethernet` cores can be configured with a soft DMA engine (AXI DMA and MCDMA) or a FIFO interface. For GigE-based Zynq and Zynq UltraScale+ MPSoC systems, there is a built-in DMA and so no extra configuration is needed. Same applies to `axi_etherlite` based systems, which have their built-in buffer management provisions.

The following figure shows a sample system architecture with a Kintex®-6 device utilizing the axi_ethernet core with DMA.

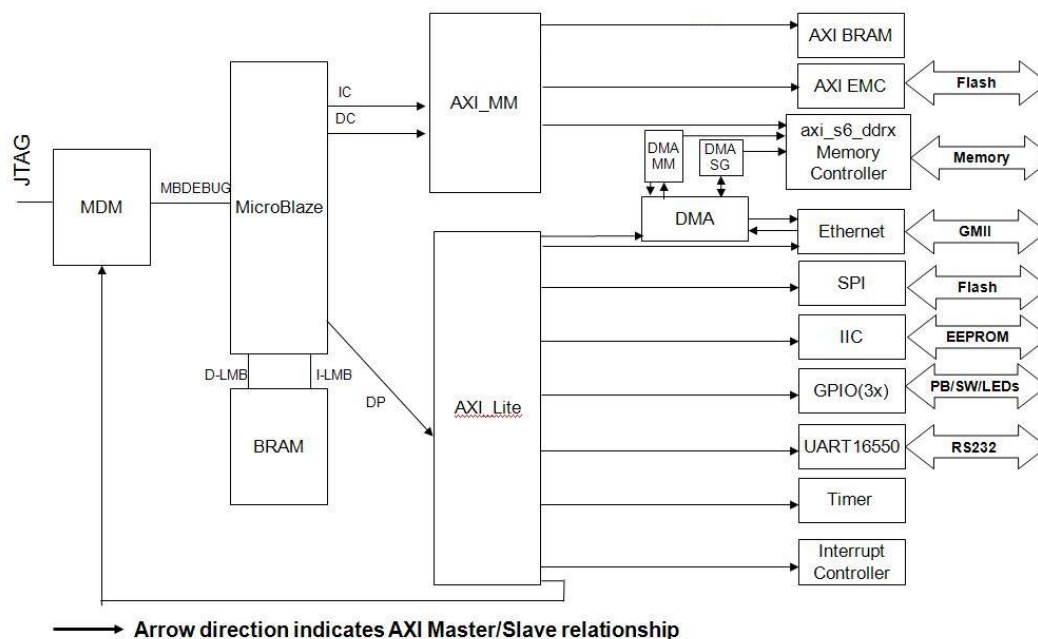


Figure 14.1: System Architecture using axi_ethernet core with DMA

Setting up the Software System

To use lwIP in a software application, you must first compile the lwIP library as a part of the software application. To move the hardware design to SDK, you must first export it from the Hardware tools.

1. Select Project > Export Hardware Design to SDK.
The **Export to SDK** dialog box appears.
2. Click **Export & Launch SDK**.
Vivado® exports the design to SDK. SDK opens and prompts you to create a workspace.
3. Compile the lwIP library:
 - (a) Select **File > New > Xilinx Board Support Package**.
The **New Board Support Package** wizard appears.
 - (b) Specify the project name and select a location for it.
 - (c) Select the BSP.
XilKernel is not supported for Zynq and Zynq UltraScale+ MPSoC devices. FreeRTOS must be used for Zynq. The FreeRTOS BSP for Zynq is available for download from the [FreeRTOS][freertos] website. For more information, see the help documentation provided provided with the port to use the FreeRTOS BSP.
 - (d) Click Finish.
The Board Support Package Settings window opens.

- (e) Select the lwip202 library with version 1_0 .
On the left side of the SDK window, lwip211_v1_0 appears in the list of libraries to be compiled.
- (f) Select lwip202 in the **Project Explorer** view.
The configuration options for lwIP are listed.
- (g) Configure the lwIP and click OK.
The board support package automatically builds with lwIP included in it.

Configuring lwIP Options

The lwIP library provides configurable parameters. The values for these parameters can be changed in SDK. There are two major categories of configurable options:

- Xilinx Adapter to lwIP options: These control the settings used by Xilinx adapters for the ethernet cores.
- Base lwIP options: These options are part of lwIP library itself, and include parameters for TCP, UDP, IP and other protocols supported by lwIP. The following sections describe the available lwIP configurable options.

Customizing lwIP API Mode

The lwip211_v1_0 supports both raw API and socket API:

- The raw API is customized for high performance and lower memory overhead. The limitation of raw API is that it is callback-based, and consequently does not provide portability to other TCP stacks.
- The socket API provides a BSD socket-style interface and is very portable; however, this mode is not as efficient as raw API mode in performance and memory requirements. The lwip211_v1_0 also provides the ability to set the priority on TCP/IP and other lwIP application threads.

The following table describes the lwIP library API mode options.

Attribute	Description	Type	Default
api_mode {RAW_API SOCKET_API}	The lwIP library mode of operation	enum	RAW_API

Attribute	Description	Type	Default
socket_mode_thread_prio	Priority of lwIP TCP/IP thread and all lwIP application threads. This setting applies only when Xilkernel is used in priority mode. It is recommended that all threads using lwIP run at the same priority level. Note For GigE based Zynq-7000 and Zynq UltraScale+ MPSoC systems using FreeRTOS, appropriate priority should be set. The default priority of 1 will not give the expected behaviour. For FreeRTOS (Zynq-7000 and Zynq UltraScale+ MPSoC systems), all internal lwIP tasks (except the main TCP/IP task) are created with the priority level set for this attribute. The TCP/IP task is given a higher priority than other tasks for improved performance. The typical TCP/IP task priority is 1 more than the priority set for this attribute for FreeRTOS.	integer	1

Attribute	Description	Type	Default
use_axieth_on_zynq	In the event that the AxiEthernet soft IP is used on a Zynq-7000 device or a Zynq UltraScale+ MPSoC device. This option ensures that the GigE on the Zynq-7000 PS (EmacPs) is not enabled and the device uses the AxiEthernet soft IP for Ethernet traffic. Note The existing Xilinx-provided lwIP adapters are not tested for multiple MACs. Multiple Axi Ethernet's are not supported on Zynq UltraScale+ MPSoC devices.	integer	0 = Use Zynq-7000 PS-based or ZynMP PS-based GigE controller 1= User AxiEthernet

Configuring Xilinx Adapter Options

The Xilinx adapters for EMAC/GigE cores are configurable.

Ethernetlite Adapter Options

The following table describes the configuration parameters for the axi_ethernetlite adapter.

Attribute	Description	Type	Default
sw_rx_fifo_size	Software Buffer Size in bytes of the receive data between EMAC and processor	integer	8192
sw_tx_fifo_size	Software Buffer Size in bytes of the transmit data between processor and EMAC	integer	8192

TEMAC Adapter Options

The following table describes the configuration parameters for the axi_ethernet and GigE adapters.

Attribute	Type	Description
n_tx_descriptors	integer	Number of Tx descriptors to be used. For high performance systems there might be a need to use a higher value. Default is 64.
n_rx_descriptors	integer	Number of Rx descriptors to be used. For high performance systems there might be a need to use a higher value. Typical values are 128 and 256. Default is 64.
n_tx_coalesce	integer	Setting for Tx interrupt coalescing. Default is 1.
n_rx_coalesce	integer	Setting for Rx interrupt coalescing. Default is 1.
tcp_rx_checksum_offload	boolean	Offload TCP Receive checksum calculation (hardware support required). For GigE in Zynq and Zynq UltraScale+ MPSoC, the TCP receive checksum offloading is always present, so this attribute does not apply. Default is false.
tcp_tx_checksum_offload	boolean	Offload TCP Transmit checksum calculation (hardware support required). For GigE cores (Zynq and Zynq UltraScale+ MPSoC), the TCP transmit checksum offloading is always present, so this attribute does not apply. Default is false.

Attribute	Type	Description
tcp_ip_rx_checksum_ofload	boolean	Offload TCP and IP Receive checksum calculation (hardware support required). Applicable only for AXI systems. For GigE in Zynq and Zynq UltraScale+ MPSoC devices, the TCP and IP receive checksum offloading is always present, so this attribute does not apply. Default is false.
tcp_ip_tx_checksum_ofload	boolean	Offload TCP and IP Transmit checksum calculation (hardware support required). Applicable only for AXI systems. For GigE in Zynq and Zynq UltraScale+ MPSoC devices, the TCP and IP transmit checksum offloading is always present, so this attribute does not apply. Default is false.
phy_link_speed	CONFIG_LINKSPEED_AUTODETECT	Link speed as auto-negotiated by the PHY. lwIP configures the TEMAC/GigE for this speed setting. This setting must be correct for the TEMAC/GigE to transmit or receive packets. The CONFIG_LINKSPEED_AUTODETECT setting attempts to detect the correct linkspeed by reading the PHY registers; however, this is PHY dependent, and has been tested with the Marvell and TI PHYs present on Xilinx development boards. For other PHYs, select the correct speed. Default is enum.

Attribute	Type	Description
temac_use_jumbo_frames_experimental	boolean	Use TEMAC jumbo frames (with a size up to 9k bytes). If this option is selected, jumbo frames are allowed to be transmitted and received by the TEMAC. For GigE in Zynq there is no support for jumbo frames, so this attribute does not apply. Default is false.

Configuring Memory Options

The lwIP stack provides different kinds of memories. Similarly, when the application uses socket mode, different memory options are used. All the configurable memory options are provided as a separate category. Default values work well unless application tuning is required. The following table describes the memory parameter options.

Attribute	Default	Type	Description
mem_size	131072	Integer	Total size of the heap memory available, measured in bytes. For applications which use a lot of memory from heap (using C library malloc or lwIP routine mem_malloc or pbuf_alloc with PBUF_RAM option), this number should be made higher as per the requirements.
memp_n_pbuf	16	Integer	The number of memp struct pbufs. If the application sends a lot of data out of ROM (or other static memory), this should be set high.
memp_n_udp_pcb	4	Integer	The number of UDP protocol control blocks. One per active UDP connection.

Attribute	Default	Type	Description
memp_n_tcp_pcb	32	Integer	The number of simultaneously active TCP connections.
memp_n_tcp_pcb_listen	8	Integer	The number of listening TC connections.
memp_n_tcp_seg	256	Integer	The number of simultaneously queued TCP segments.
memp_n_sys_timeout	8	Integer	Number of simultaneously active timeouts.
memp_num_netbuf	8	Integer	Number of allowed structure instances of type netbufs. Applicable only in socket mode.
memp_num_netconn	16	Integer	Number of allowed structure instances of type netconns. Applicable only in socket mode.
memp_num_api_msg	16	Integer	Number of allowed structure instances of type api_msg. Applicable only in socket mode.
memp_num_tcpip_msg	64	Integer	Number of TCPIP msg structures (socket mode only).

Note

Because Sockets Mode support uses Xilkernel services, the number of semaphores chosen in the Xilkernel configuration must take the value set for the `memp_num_netbuf` parameter into account. For FreeRTOS BSP there is no setting for the maximum number of semaphores. For FreeRTOS, you can create semaphores as long as memory is available.

Configuring Packet Buffer (Pbuf) Memory Options

Packet buffers (Pbufs) carry packets across various layers of the TCP/IP stack. The following are the pbuf memory options provided by the lwIP stack. Default values work well unless application tuning is required. The following table describes the parameters for the Pbuf memory options.

Attribute	Default	Type	Description
pbuf_pool_size	256	Integer	Number of buffers in pbuf pool. For high performance systems, you might consider increasing the pbuf pool size to a higher value, such as 512.
pbuf_pool_bufsize	1700	Integer	Size of each pbuf in pbuf pool. For systems that support jumbo frames, you might consider using a pbuf pool buffer size that is more than the maximum jumbo frame size.
pbuf_link_hlen	16	Integer	Number of bytes that should be allocated for a link level header.

Configuring ARP Options

The following table describes the parameters for the ARP options. Default values work well unless application tuning is required.

Attribute	Default	Type	Description
arp_table_size	10	Integer	Number of active hardware address IP address pairs cached.
arp_queueing	1	Integer	If enabled outgoing packets are queued during hardware address resolution. This attribute can have two values: 0 or 1.

Configuring IP Options

The following table describes the IP parameter options. Default values work well unless application tuning is required.

Attribute	Default	Type	Description
ip_forward	0	Integer	Set to 1 for enabling ability to forward IP packets across network interfaces. If running lwIP on a single network interface, set to 0. This attribute can have two values: 0 or 1.
ip_options	0	Integer	When set to 1, IP options are allowed (but not parsed). When set to 0, all packets with IP options are dropped. This attribute can have two values: 0 or 1.
ip_reassembly	1	Integer	Reassemble incoming fragmented IP packets.
ip_frag	1	Integer	Fragment outgoing IP packets if their size exceeds MTU.
ip_reass_max_pbufs	128	Integer	Reassembly pbuf queue length.
ip_frag_max_mtu	1500	Integer	Assumed max MTU on any interface for IP fragmented buffer.
ip_default_ttl	255	Integer	Global default TTL used by transport layers.

Configuring ICMP Options

The following table describes the parameter for ICMP protocol option. Default values work well unless application tuning is required.

Attribute	Default	Type	Description
icmp_ttl	255	Integer	ICMP TTL value.

For GigE cores (for Zynq and Zynq MPSoC) there is no support for ICMP in the hardware.

Configuring IGMP Options

The IGMP protocol is supported by lwIP stack. When set true, the following option enables the IGMP protocol.

Attribute	Default	Type	Description
imgp_options	false	Boolean	Specify whether IGMP is required.

Configuring UDP Options

The following table describes UDP protocol options. Default values work well unless application tuning is required.

Attribute	Default	Type	Description
lwip_udp	true	Boolean	Specify whether UDP is required.
udp_ttl	255	Integer	UDP TTL value.

Configuring TCP Options

The following table describes the TCP protocol options. Default values work well unless application tuning is required.

Attribute	Default	Type	Description
lwip_tcp	true	Boolean	Require TCP.
tcp_ttl	255	Integer	TCP TTL value.
tcp_wnd	2048	Integer	TCP Window size in bytes.
tcp_maxrtx	12	Integer	TCP Maximum retransmission value.
tcp_synmaxrtx	4	Integer	TCP Maximum SYN retransmission value.
tcp_queue_ooseq	1	Integer	Accept TCP queue segments out of order. Set to 0 if your device is low on memory.
tcp_mss	1460	Integer	TCP Maximum segment size.
tcp_snd_buf	8192	Integer	TCP sender buffer space in bytes.

Configuring DHCP Options

The DHCP protocol is supported by lwIP stack. The following table describes DHCP protocol options. Default values work well unless application tuning is required.

Attribute	Default	Type	Description
lwip_dhcp	false	Boolean	Specify whether DHCP is required.
dhcp_does_arp_check	false	Boolean	Specify whether ARP checks on offered addresses.

Configuring the Stats Option

lwIP stack has been written to collect statistics, such as the number of connections used; amount of memory used; and number of semaphores used, for the application. The library provides the stats_display() API to dump out the statistics relevant to the context in which the call is used. The stats option can be turned on to enable the statistics information to be collected and displayed when the stats_display API is called from user code. Use the following option to enable collecting the stats information for the application.

Attribute	Description	Type	Default
lwip_stats	Turn on lwIP Statistics	int	0

Configuring the Debug Option

lwIP provides debug information. The following table lists all the available options.

Attribute	Default	Type	Description
lwip_debug	false	Boolean	Turn on/off lwIP debugging.
ip_debug	false	Boolean	Turn on/off IP layer debugging.
tcp_debug	false	Boolean	Turn on/off TCP layer debugging.
udp_debug	false	Boolean	Turn on/off UDP layer debugging.
icmp_debug	false	Boolean	Turn on/off ICMP protocol debugging.
igmp_debug	false	Boolean	Turn on/off IGMP protocol debugging.

Attribute	Default	Type	Description
netif_debug	false	Boolean	Turn on/off network interface layer debugging.
sys_debug	false	Boolean	Turn on/off sys arch layer debugging.
pbuf_debug	false	Boolean	Turn on/off pbuf layer debugging

LwIP Library APIs

The lwIP library provides two different APIs: RAW API and Socket API.

Raw API

The Raw API is callback based. Applications obtain access directly into the TCP stack and vice-versa. As a result, there is no extra socket layer, and using the Raw API provides excellent performance at the price of compatibility with other TCP stacks.

Xilinx Adapter Requirements when using the RAW API

In addition to the lwIP RAW API, the Xilinx adapters provide the `xemacif_input` utility function for receiving packets. This function must be called at frequent intervals to move the received packets from the interrupt handlers to the lwIP stack. Depending on the type of packet received, lwIP then calls registered application callbacks.

The `$XILINX_SDK/sw/ThirdParty/sw_services/lwip211_v1_0/src/lwip-2.1.1/doc/rawapi.txt` file describes the lwIP Raw API.

LwIP Performance

The following table provides the maximum TCP throughput achievable by FPGA, CPU, EMAC, and system frequency in RAW modes. Applications requiring high performance should use the RAW API.

FPGA	CPU	EMAC	System Frequency	Max TCP Throughput in RAW Mode (Mbps)
Virtex®	MicroBlaze	axi-ethernet	100 MHz	RX Side: 182 TX Side: 100
Virtex	MicroBlaze	xps-ll-temac	100 MHz	RX Side: 178 TX Side: 100
Virtex	MicroBlaze	xps-ethernetlite	100 MHz	RX Side: 50 TX Side: 38

RAW API Example

Applications using the RAW API are single threaded. The following pseudo-code illustrates a typical RAW mode program structure.

```
int main()
{
    struct netif *netif, server_netif;
    ip_addr_t ipaddr, netmask, gw;

    /* the MAC address of the board.
     * This should be unique per board/PHY */
    unsigned char mac_ethernet_address[] =
        {0x00, 0x0a, 0x35, 0x00, 0x01, 0x02};

    lwip_init();

    /* Add network interface to the netif_list,
     * and set it as default */
    if (!xemac_add(netif, &ipaddr, &netmask,
        &gw, mac_ethernet_address,
        EMAC_BASEADDR)) {
        printf("Error adding N/W interface\n\r");
        return -1;
    }
    netif_set_default(netif);

    /* now enable interrupts */
    platform_enable_interrupts();

    /* specify that the network if is up */
    netif_set_up(netif);

    /* start the application, setup callbacks */
    start_application();

    /* receive and process packets */
    while (1) {
        xemacif_input(netif);
        /* application specific functionality */
        transfer_data();
    }
}
```

Socket API

The lwIP socket API provides a BSD socket-style API to programs. This API provides an execution model that is a blocking, open-read-write-close paradigm.

Xilinx Adapter Requirements when using the Socket API

Applications using the Socket API with Xilinx adapters need to spawn a separate thread called `xemacif_input_thread`. This thread takes care of moving received packets from the interrupt handlers to the `tcpip_thread` of the lwIP. Application threads that use lwIP must be created using the lwIP `sys_thread_new` API. Internally, this function makes use of the appropriate thread or task creation routines provided by XilKernel or FreeRTOS.

Xilkernel/FreeRTOS scheduling policy when using the Socket API

lwIP in socket mode requires the use of the Xilkernel or FreeRTOS, which provides two policies for thread scheduling: round-robin and priority based.

There are no special requirements when round-robin scheduling policy is used because all threads or tasks with same priority receive the same time quanta. This quanta is fixed by the RTOS (Xilkernel or FreeRTOS) being used.

With priority scheduling, care must be taken to ensure that lwIP threads or tasks are not starved. For Xilkernel, lwIP internally launches all threads at the priority level specified in `socket_mode_thread_prio`. For FreeRTOS, lwIP internally launches all tasks except the main TCP/IP task at the priority specified in `socket_mode_thread_prio`. The TCP/IP task in FreeRTOS is launched with a higher priority (one more than priority set in `socket_mode_thread_prio`). In addition, application threads must launch `xemacif_input_thread`. The priorities of both `xemacif_input_thread`, and the lwIP internal threads (`socket_mode_thread_prio`) must be high enough in relation to the other application threads so that they are not starved.

Socket API Example

XilKernel-based applications in socket mode can specify a static list of threads that Xilkernel spawns on startup in the Xilkernel Software Platform Settings dialog box. Assuming that `main_thread()` is a thread specified to be launched by Xilkernel, control reaches this first thread from application main after the Xilkernel schedule is started. In `main_thread`, one more thread (`network_thread`) is created to initialize the MAC layer.

For FreeRTOS (Zynq-7000 processor systems) based applications, once the control reaches application main routine, a task (can be termed as `main_thread`) with an entry point function as `main_thread()` is created before starting the scheduler. After the FreeRTOS scheduler starts, the control reaches `main_thread()`, where the lwIP internal initialization happens. The application then creates one more thread (`network_thread`) to initialize the MAC layer.

The following pseudo-code illustrates a typical socket mode program structure.

```
void network_thread(void *p)
{
    struct netif *netif;
    ip_addr_t ipaddr, netmask, gw;

    /* the MAC address of the board.
     * This should be unique per board/PHY */
    unsigned char mac_ethernet_address[] =
        {0x00, 0x0a, 0x35, 0x00, 0x01, 0x02};

    netif = &server_netif;

    /* initialize IP addresses to be used */
    IP4_ADDR(&ipaddr, 192, 168, 1, 10);
    IP4_ADDR(&netmask, 255, 255, 255, 0);
    IP4_ADDR(&gw, 192, 168, 1, 1);

    /* Add network interface to the netif_list,
     * and set it as default */
    if (!xemac_add(netif, &ipaddr, &netmask,
        &gw, mac_ethernet_address,
        EMAC_BASEADDR)) {
        printf("Error adding N/W interface\n\r");
        return;
    }
    netif_set_default(netif);
}
```

```

/* specify that the network if is up */
netif_set_up(netif);

/* start packet receive thread
- required for lwIP operation */
sys_thread_new("xemacif_input_thread", xemacif_input_thread,
    netif,
    THREAD_STACKSIZE, DEFAULT_THREAD_PRIO);

/* now we can start application threads */
/* start webserver thread (e.g.) */
sys_thread_new("httpd" web_application_thread, 0,
    THREAD_STACKSIZE DEFAULT_THREAD_PRIO);
}

int main_thread()
{
    /* initialize lwIP before calling sys_thread_new */
    lwip_init();

    /* any thread using lwIP should be created using
    * sys_thread_new() */
    sys_thread_new("network_thread" network_thread, NULL,
        THREAD_STACKSIZE DEFAULT_THREAD_PRIO);

    return 0;
}

```

Using the Xilinx Adapter Helper Functions

The Xilinx adapters provide the following helper functions to simplify the use of the lwIP APIs.

Appendix F:

XilFlash Library v4.6

Overview

The XilFlash library provides read/write/erase/lock/unlock features to access a parallel flash device. This library implements the functionality for flash memory devices that conform to the "Common Flash Interface" (CFI) standard. CFI allows a single flash library to be used for an entire family of parts and helps us determine the algorithm to utilize during runtime.

Note

All the calls in the library are blocking in nature in that the control is returned back to user only after the current operation is completed successfully or an error is reported.

Library Initialization

The `XFlash_Initialize()` function should be called by the application before any other function in the library. The initialization function checks for the device family and initializes the XFlash instance with the family specific data. The VT table (contains the function pointers to family specific APIs) is setup and family specific initialization routine is called.

Device Geometry

The device geometry varies for different flash device families. Following sections describes the geometry of different flash device families:

Intel Flash Device Geometry

Flash memory space is segmented into areas called blocks. The size of each block is based on a power of 2. A region is defined as a contiguous set of blocks of the same size. Some parts have several regions while others have one. The arrangement of blocks and regions is referred to by this module as the part's geometry. Some Intel flash supports multiple banks on the same device. This library supports single and multiple bank flash devices.

AMD Flash Device Geometry

Flash memory space is segmented into areas called banks and further in to regions and blocks. The size of each block is based on a power of 2. A region is defined as a contiguous set of blocks of the same size. Some parts have several regions while others have one. A bank is defined as a contiguous set of blocks. The bank

may contain blocks of different size. The arrangement of blocks, regions and banks is referred to by this module as the part's geometry.

The cells within the part can be programmed from a logic 1 to a logic 0 and not the other way around. To change a cell back to a logic 1, the entire block containing that cell must be erased. When a block is erased all bytes contain the value 0xFF. The number of times a block can be erased is finite. Eventually the block will wear out and will no longer be capable of erasure. As of this writing, the typical flash block can be erased 100,000 or more times.

Write Operation

The write call can be used to write a minimum of zero bytes and a maximum entire flash. If the Offset Address specified to write is out of flash or if the number of bytes specified from the Offset address exceed flash boundaries an error is reported back to the user. The write is blocking in nature in that the control is returned back to user only after the write operation is completed successfully or an error is reported.

Read Operation

The read call can be used to read a minimum of zero bytes and maximum of entire flash. If the Offset Address specified to write is out of flash boundary an error is reported back to the user. The read function reads memory locations beyond Flash boundary. Care should be taken by the user to make sure that the Number of Bytes + Offset address is within the Flash address boundaries. The write is blocking in nature in that the control is returned back to user only after the read operation is completed successfully or an error is reported.

Erase Operation

The erase operations are provided to erase a Block in the Flash memory. The erase call is blocking in nature in that the control is returned back to user only after the erase operation is completed successfully or an error is reported.

Sector Protection

The Flash Device is divided into Blocks. Each Block can be protected individually from unwarranted writing/erasing. The Block locking can be achieved using [XFlash_Lock\(\)](#) lock. All the memory locations from the Offset address specified will be locked. The block can be unlocked using [XFlash_UnLock\(\)](#) call. All the Blocks which are previously locked will be unlocked. The Lock and Unlock calls are blocking in nature in that the control is returned back to user only after the operation is completed successfully or an error is reported. The AMD flash device requires high voltage on Reset pin to perform lock and unlock operation. User must provide this high voltage (As defined in datasheet) to reset pin before calling lock and unlock API for AMD flash devices. Lock and Unlock features are not tested for AMD flash device.

Device Control

Functionalities specific to a Flash Device Family are implemented as Device Control. The following are the Intel specific device control:

- Retrieve the last error data.

- Get Device geometry.
- Get Device properties.
- Set RYBY pin mode.
- Set the Configuration register (Platform Flash only).

The following are the AMD specific device control:

- Get Device geometry.
- Get Device properties.
- Erase Resume.
- Erase Suspend.
- Enter Extended Mode.
- Exit Extended Mode.
- Get Protection Status of Block Group.
- Erase Chip.

Note

This library needs to know the type of EMC core (AXI or XPS) used to access the cfi flash, to map the correct APIs. This library should be used with the emc driver, v3_01_a and above, so that this information can be automatically obtained from the emc driver.

This library is intended to be RTOS and processor independent. It works with physical addresses only. Any needs for dynamic memory management, threads, mutual exclusion, virtual memory, cache control, or HW write protection management must be satisfied by the layer above this library.

All writes to flash occur in units of bus-width bytes. If more than one part exists on the data bus, then the parts are written in parallel. Reads from flash are performed in any width up to the width of the data bus. It is assumed that the flash bus controller or local bus supports these types of accesses.

XilFlash Library API

Overview

This chapter provides a linked summary and detailed descriptions of the LibXil Flash library APIs.

Functions

- int [XFlash_Initialize](#) (XFlash *InstancePtr, u32 BaseAddress, u8 BusWidth, int IsPlatformFlash)
 - int [XFlash_Reset](#) (XFlash *InstancePtr)
 - int [XFlash_DeviceControl](#) (XFlash *InstancePtr, u32 Command, DeviceCtrlParam *Parameters)
 - int [XFlash_Read](#) (XFlash *InstancePtr, u32 Offset, u32 Bytes, void *DestPtr)
 - int [XFlash_Write](#) (XFlash *InstancePtr, u32 Offset, u32 Bytes, void *SrcPtr)
 - int [XFlash_Erase](#) (XFlash *InstancePtr, u32 Offset, u32 Bytes)
 - int [XFlash_Lock](#) (XFlash *InstancePtr, u32 Offset, u32 Bytes)
 - int [XFlash_Unlock](#) (XFlash *InstancePtr, u32 Offset, u32 Bytes)
 - int [XFlash_IsReady](#) (XFlash *InstancePtr)
-

Function Documentation

int XFlash_Initialize (XFlash * *InstancePtr*, u32 *BaseAddress*, u8 *BusWidth*, int *IsPlatformFlash*)

This function initializes a specific XFlash instance.
The initialization entails:

- Check the Device family type.
- Issuing the CFI query command.
- Get and translate relevant CFI query information.
- Set default options for the instance.
- Setup the VTable.
- Call the family initialize function of the instance.

Initialize the Xilinx Platform Flash XL to Async mode if the user selects to use the Platform Flash XL in the MLD. The Platform Flash XL is an Intel CFI complaint device.

Parameters

<i>InstancePtr</i>	Pointer to the XFlash instance.
<i>BaseAddress</i>	Base address of the flash memory.
<i>BusWidth</i>	Total width of the flash memory, in bytes.
<i>IsPlatformFlash</i>	Used to specify if the flash is a platform flash.

Returns

- XST_SUCCESS if successful.
- XFLASH_PART_NOT_SUPPORTED if the command set algorithm or Layout is not supported by any flash family compiled into the system.
- XFLASH_CFI_QUERY_ERROR if the device would not enter CFI query mode. Either the device(s) do not support CFI, the wrong BaseAddress param was used, an unsupported part layout exists, or a hardware problem exists with the part.

Note

BusWidth is not the width of an individual part. Its the total operating width. For example, if there are two 16-bit parts, with one tied to data lines D0-D15 and other tied to D15-D31, BusWidth would be $(32 / 8) = 4$. If a single 16-bit flash is in 8-bit mode, then BusWidth should be $(8 / 8) = 1$.

int XFlash_Reset (XFlash * *InstancePtr*)

This function resets the flash device and places it in read mode.

Parameters

<i>InstancePtr</i>	Pointer to the XFlash instance.
--------------------	---------------------------------

Returns

- XST_SUCCESS if successful.
- XFLASH_BUSY if the flash devices were in the middle of an operation and could not be reset.
- XFLASH_ERROR if the device(s) have experienced an internal error during the operation. [XFlash_DeviceControl\(\)](#) must be used to access the cause of the device specific error. condition.

Note

None.

```
int XFlash_DeviceControl ( XFlash * InstancePtr, u32 Command, DeviceCtrlParam * Parameters )
```

This function is used to execute device specific commands.
For a list of device specific commands, see the xilflash.h.

Parameters

<i>InstancePtr</i>	Pointer to the XFlash instance.
<i>Command</i>	Device specific command to issue.
<i>Parameters</i>	Specifies the arguments passed to the device control function.

Returns

- XST_SUCCESS if successful.
- XFLASH_NOT_SUPPORTED if the command is not recognized/supported by the device(s).

Note

None.

```
int XFlash_Read ( XFlash * InstancePtr, u32 Offset, u32 Bytes, void * DestPtr )
```

This function reads the data from the Flash device and copies it into the specified user buffer.
The source and destination addresses can be on any alignment supported by the processor.
The device is polled until an error or the operation completes successfully.

Parameters

<i>InstancePtr</i>	Pointer to the XFlash instance.
<i>Offset</i>	Offset into the device(s) address space from which to read.
<i>Bytes</i>	Number of bytes to copy.
<i>DestPtr</i>	Destination address to copy data to.

Returns

- XST_SUCCESS if successful.
- XFLASH_ADDRESS_ERROR if the source address does not start within the addressable areas of the device(s).

Note

This function allows the transfer of data past the end of the device's address space. If this occurs, then results are undefined.

int XFlash_Write (XFlash * InstancePtr, u32 Offset, u32 Bytes, void * SrcPtr)

This function programs the flash device(s) with data specified in the user buffer.
The source and destination address must be aligned to the width of the flash's data bus.
The device is polled until an error or the operation completes successfully.

Parameters

<i>InstancePtr</i>	Pointer to the XFlash instance.
<i>Offset</i>	Offset into the device(s) address space from which to begin programming. Must be aligned to the width of the flash's data bus.
<i>Bytes</i>	Number of bytes to program.
<i>SrcPtr</i>	Source address containing data to be programmed. Must be aligned to the width of the flash's data bus. The SrcPtr doesn't have to be aligned to the flash width if the processor supports unaligned access. But, since this library is generic, and some processors(eg. Microblaze) do not support unaligned access; this API requires the SrcPtr to be aligned.

Returns

- XST_SUCCESS if successful.
- XFLASH_ERROR if a write error occurred. This error is usually device specific. Use [XFlash_DeviceControl\(\)](#) to retrieve specific error conditions. When this error is returned, it is possible that the target address range was only partially programmed.

Note

None.

int XFlash_Erase (XFlash * InstancePtr, u32 Offset, u32 Bytes)

This function erases the specified address range in the flash device.
The number of bytes to erase can be any number as long as it is within the bounds of the device(s).
The device is polled until an error or the operation completes successfully.

Parameters

<i>InstancePtr</i>	Pointer to the XFlash instance.
<i>Offset</i>	Offset into the device(s) address space from which to begin erasure.
<i>Bytes</i>	Number of bytes to erase.

Returns

- XST_SUCCESS if successful.
- XFLASH_ADDRESS_ERROR if the destination address range is not completely within the addressable areas of the device(s).

Note

Due to flash memory design, the range actually erased may be larger than what was specified by the Offset & Bytes parameters. This will occur if the parameters do not align to block boundaries.

int XFlash_Lock (XFlash * *InstancePtr*, u32 *Offset*, u32 *Bytes*)

This function Locks the blocks in the specified range of the flash device(s). The device is polled until an error or the operation completes successfully.

Parameters

<i>InstancePtr</i>	Pointer to the XFlash instance.
<i>Offset</i>	Offset into the device(s) address space from which to begin block locking. The first three bytes of every block is reserved for special purpose. The offset should be atleast three bytes from start of the block.
<i>Bytes</i>	Number of bytes to Lock in the Block starting from Offset.

Returns

- XST_SUCCESS if successful.
- XFLASH_ADDRESS_ERROR if the destination address range is not completely within the addressable areas of the device(s).

Note

Due to flash memory design, the range actually locked may be larger than what was specified by the Offset & Bytes parameters. This will occur if the parameters do not align to block boundaries.

int XFlash_Unlock (XFlash * *InstancePtr*, u32 *Offset*, u32 *Bytes*)

This function Unlocks the blocks in the specified range of the flash device(s). The device is polled until an error or the operation completes successfully.

Parameters

<i>InstancePtr</i>	Pointer to the XFlash instance.
<i>Offset</i>	Offset into the device(s) address space from which to begin block UnLocking. The first three bytes of every block is reserved for special purpose. The offset should be atleast three bytes from start of the block.
<i>Bytes</i>	Number of bytes to UnLock in the Block starting from Offset.

Returns

- XST_SUCCESS if successful.
- XFLASH_ADDRESS_ERROR if the destination address range is not completely within the addressable areas of the device(s).

Note

None.

int XFlash_IsReady (XFlash * *InstancePtr*)

This function checks the readiness of the device, which means it has been successfully initialized.

Parameters

<i>InstancePtr</i>	Pointer to the XFlash instance.
--------------------	---------------------------------

Returns

TRUE if the device has been initialized (but not necessarily started), and FALSE otherwise.

Note

None.

Library Parameters in MSS File

XilFlash Library can be integrated with a system using the following snippet in the Microprocessor Software Specification (MSS) file:

```
BEGIN LIBRARY
PARAMETER LIBRARY_NAME = xilflash
PARAMETER LIBRARY_VER = 4.6
PARAMETER PROC_INSTANCE = microblaze_0
PARAMETER ENABLE_INTEL = true
PARAMETER ENABLE_AMD = false
END
```

The table below describes the libgen customization parameters.

Parameter	Default Value	Description
LIBRARY_NAME	xilflash	Specifies the library name.
LIBRARY_VER	4.6	Specifies the library version.
PROC_INSTANCE	microblaze_0	Specifies the processor name.
ENABLE_INTEL	true/false	Enables or disables the Intel flash device family.
ENABLE_AMD	true/false	Enables or disables the AMD flash device family.

Appendix G:

Xillsf Library v5.13

Overview

The LibXil Isf library:

- Allows you to Write, Read, and Erase the Serial Flash.
- Allows protection of the data stored in the Serial Flash from unwarranted modification by enabling the Sector Protection feature.
- Supports multiple instances of Serial Flash at a time, provided they are of the same device family (Atmel, Intel, STM, Winbond, SST, or Spansion) as the device family is selected at compile time.
- Allows your application to perform Control operations on Intel, STM, Winbond, SST, and Spansion Serial Flash.
- Requires the underlying hardware platform to contain the axi_quad_spi, ps7_spi, ps7_qspi, psu_qspi, psv_ospi, or psu_spi device for accessing the Serial Flash.
- Uses the Xilinx® SPI interface drivers in interrupt-driven mode or polled mode for communicating with the Serial Flash. In interrupt mode, the user application must acknowledge any associated interrupts from the Interrupt Controller.

Additional information:

- In interrupt mode, the application is required to register a callback to the library and the library registers an internal status handler to the selected interface driver.
- When your application requests a library operation, it is initiated and control is given back to the application. The library tracks the status of the interface transfers, and notifies the user application upon completion of the selected library operation.
- Added support in the library for SPI PS and QSPI PS. You must select one of the interfaces at compile time.
- Added support for QSPIPSU and SPIPS flash interface on Zynq® UltraScale+™ MPSoC.
- Added support for OSPIPSV flash interface
- When your application requests selection of QSPIPS interface during compilation, the QSPI PS or QSPI PSU interface, based on the hardware platform, are selected.
- When the SPIPS interface is selected during compilation, the SPI PS or the SPI PSU interface is selected.
- When the OSPI interface is selected during compilation, the OSPIPSV interface is selected.

Supported Devices

The table below lists the supported Xilinx in-system and external serial flash memories.

Device Series	Manufacturer
AT45DB011D AT45DB021D AT45DB041D AT45DB081D AT45DB161D AT45DB321D AT45DB642D	Atmel
W25Q16 W25Q32 W25Q64 W25Q80 W25Q128 W25X10 W25X20 W25X40 W25X80 W25X16 W25X32 W25X64	Winbond
S25FL004 S25FL008 S25FL016 S25FL032 S25FL064 S25FL128 S25FL129 S25FL256 S25FL512 S70FL01G	Spansion
SST25WF080	SST

Device Series	Manufacturer
N25Q032 N25Q064 N25Q128 N25Q256 N25Q512 N25Q00AA MT25Q01 MT25Q02 MT25Q512 MT25QL02G MT25QU02G MT35XU512ABA	Micron
MX66L1G45G MX66U1G45G	Macronix
IS25WP256D IS25LP256D IS25LWP512M IS25LP512M IS25WP064A IS25LP064A IS25WP032D IS25LP032D IS25WP016D IS25LP016D IS25WP080D IS25LP080D IS25LP128F IS25WP128F	ISSI

Note

Intel, STM, and Numonyx serial flash devices are now a part of Serial Flash devices provided by Micron.

References

- Spartan-3AN FPGA In-System Flash User Guide (UG333):
http://www.xilinx.com/support/documentation/user_guides/ug333.pdf
- Winbond Serial Flash Page:
http://www.winbond.com/hq/product/code-storage-flash-memory/serial-nor-flash/?__locale=en



- Intel (Numonyx) S33 Serial Flash Memory, SST SST25WF080, Micron N25Q flash family :
<https://www.micron.com/products/nor-flash/serial-nor-flash>

Xllsf Library API

Overview

This chapter provides a linked summary and detailed descriptions of the Xllsf library APIs.

Functions

- int [Xlsf_Initialize](#) (Xlsf *InstancePtr, Xlsf_Iface *SpiInstPtr, u8 SlaveSelect, u8 *WritePtr)
- int [Xlsf_GetStatus](#) (Xlsf *InstancePtr, u8 *ReadPtr)
- int [Xlsf_GetStatusReg2](#) (Xlsf *InstancePtr, u8 *ReadPtr)
- int [Xlsf_GetDeviceInfo](#) (Xlsf *InstancePtr, u8 *ReadPtr)
- u32 [GetRealAddr](#) (Xlsf_Iface *QspiPtr, u32 Address)
- int [Xlsf_Write](#) (Xlsf *InstancePtr, Xlsf_WriteOperation Operation, void *OpParamPtr)
- int [Xlsf_Read](#) (Xlsf *InstancePtr, Xlsf_ReadOperation Operation, void *OpParamPtr)
- int [Xlsf_Erase](#) (Xlsf *InstancePtr, Xlsf_EraseOperation Operation, u32 Address)
- int [Xlsf_MicronFlashEnter4BAddMode](#) (Xlsf *InstancePtr)
- int [Xlsf_MicronFlashExit4BAddMode](#) (Xlsf *InstancePtr)
- int [Xlsf_SectorProtect](#) (Xlsf *InstancePtr, Xlsf_SpOperation Operation, u8 *BufferPtr)
- int [Xlsf_ioctl](#) (Xlsf *InstancePtr, Xlsf_ioctlOperation Operation)
- int [Xlsf_WriteEnable](#) (Xlsf *InstancePtr, u8 WriteEnable)
- void [Xlsf_RegisterInterface](#) (Xlsf *InstancePtr)
- int [Xlsf_SetSpiConfiguration](#) (Xlsf *InstancePtr, Xlsf_Iface *SpiInstPtr, u32 Options, u8 PreScaler)
- void [Xlsf_SetStatusHandler](#) (Xlsf *InstancePtr, Xlsf_Iface *XlfaceInstancePtr, Xlsf_StatusHandler Xllsf_Handler)
- void [Xlsf_IfaceHandler](#) (void *CallbackRef, u32 StatusEvent, unsigned int ByteCount)

Function Documentation

int Xlsf_Initialize (Xlsf * *InstancePtr*, Xlsf_Iface * *SpiInstPtr*, u8 *SlaveSelect*, u8 * *WritePtr*)

This API when called initializes the SPI interface with default settings.

With custom settings, user should call [Xlsf_SetSpiConfiguration\(\)](#) and then call this API. The geometry of the underlying Serial Flash is determined by reading the Joint Electron Device Engineering Council (JEDEC) Device Information and the Status Register of the Serial Flash.

Parameters

<i>InstancePtr</i>	Pointer to the Xlsf instance.
<i>SpiInstPtr</i>	Pointer to Xlsf_iface instance to be worked on.
<i>SlaveSelect</i>	It is a 32-bit mask with a 1 in the bit position of slave being selected. Only one slave can be selected at a time.
<i>WritePtr</i>	Pointer to the buffer allocated by the user to be used by the In-system and Serial Flash Library to perform any read/write operations on the Serial Flash device. User applications must pass the address of this buffer for the Library to work. • Write operations : ◦ The size of this buffer should be equal to the Number of bytes to be written to the Serial Flash + XISF_CMD_MAX_EXTRA_BYTES. ◦ The size of this buffer should be large enough for usage across all the applications that use a common instance of the Serial Flash. ◦ A minimum of one byte and a maximum of ISF_PAGE_SIZE bytes can be written to the Serial Flash, through a single Write operation. • Read operations : ◦ The size of this buffer should be equal to XISF_CMD_MAX_EXTRA_BYTES, if the application only reads from the Serial Flash (no write operations).

Returns

- XST_SUCCESS if successful.
- XST_DEVICE_IS_STOPPED if the device must be started before transferring data.
- XST_FAILURE, otherwise.

Note

- The [Xlsf_Initialize\(\)](#) API is a blocking call (for both polled and interrupt modes of the Spi driver). It reads the JEDEC information of the device and waits till the transfer is complete before checking if the information is valid.
- This library can support multiple instances of Serial Flash at a time, provided they are of the same device family (either Atmel, Intel or STM, Winbond or Spansion) as the device family is selected at compile time.

int Xlzf_GetStatus (Xlzf * *InstancePtr*, u8 * *ReadPtr*)

This API reads the Serial Flash Status Register.

Parameters

<i>InstancePtr</i>	Pointer to the Xlzf instance.
<i>ReadPtr</i>	Pointer to the memory where the Status Register content is copied.

Returns

XST_SUCCESS if successful else XST_FAILURE.

Note

The contents of the Status Register is stored at second byte pointed by the ReadPtr.

int Xlzf_GetStatusReg2 (Xlzf * *InstancePtr*, u8 * *ReadPtr*)

This API reads the Serial Flash Status Register 2.

Parameters

<i>InstancePtr</i>	Pointer to the Xlzf instance.
<i>ReadPtr</i>	Pointer to the memory where the Status Register content is copied.

Returns

XST_SUCCESS if successful else XST_FAILURE.

Note

The contents of the Status Register 2 is stored at the second byte pointed by the ReadPtr. This operation is available only in Winbond Serial Flash.

int Xlzf_GetDeviceInfo (Xlzf * *InstancePtr*, u8 * *ReadPtr*)

This API reads the Joint Electron Device Engineering Council (JEDEC) information of the Serial Flash.

Parameters

<i>InstancePtr</i>	Pointer to the Xlzf instance.
<i>ReadPtr</i>	Pointer to the buffer where the Device information is copied.

Returns

XST_SUCCESS if successful else XST_FAILURE.

Note

The Device information is stored at the second byte pointed by the ReadPtr.

u32 GetRealAddr (Xlzf_iface * QspiPtr, u32 Address)

Function to get the real address of flash in case dual parallel and stacked configuration.

Function to get the real address of flash in case dual parallel and stacked configuration.

This functions translates the address based on the type of interconnection. In case of stacked, this function asserts the corresponding slave select.

Parameters

<i>QspiPtr</i>	is a pointer to Xlzf_iface instance to be worked on.
<i>Address</i>	which is to be accessed (for erase, write or read)

Returns

RealAddr is the translated address - for single it is unchanged for stacked, the lower flash size is subtracted for parallel the address is divided by 2.

Note

None.

int Xlzf_Write (Xlzf * InstancePtr, Xlzf_WriteOperation Operation, void * OpParamPtr)

This API writes the data to the Serial Flash.

Parameters

<i>InstancePtr</i>	Pointer to the Xlzf instance.
<i>Operation</i>	Type of write operation to be performed on the Serial Flash. The different operations are • XISF_WRITE: Normal Write • XISF_DUAL_IP_PAGE_WRITE: Dual Input Fast Program • XISF_DUAL_IP_EXT_PAGE_WRITE: Dual Input Extended Fast Program • XISF_QUAD_IP_PAGE_WRITE: Quad Input Fast Program • XISF_QUAD_IP_EXT_PAGE_WRITE: Quad Input Extended Fast Program • XISF_AUTO_PAGE_WRITE: Auto Page Write • XISF_BUFFER_WRITE: Buffer Write • XISF_BUF_TO_PAGE_WRITE_WITH_ERASE: Buffer to Page Transfer with Erase • XISF_BUF_TO_PAGE_WRITE_WITHOUT_ERASE: Buffer to Page Transfer without Erase • XISF_WRITE_STATUS_REG: Status Register Write • XISF_WRITE_STATUS_REG2: 2 byte Status Register Write • XISF_OTP_WRITE: OTP Write.
<i>OpParamPtr</i>	Pointer to a structure variable which contains operational parameters of the specified operation. This parameter type is dependant on value of first argument(Operation). For more details, refer Operations .

Operations

- Normal Write(XISF_WRITE), Dual Input Fast Program (XISF_DUAL_IP_PAGE_WRITE), Dual Input Extended Fast Program(XISF_DUAL_IP_EXT_PAGE_WRITE), Quad Input Fast Program(XISF_QUAD_IP_PAGE_WRITE), Quad Input Extended Fast Program (XISF_QUAD_IP_EXT_PAGE_WRITE):
 - The OpParamPtr must be of type struct Xlzf_WriteParam.
 - OpParamPtr->Address is the start address in the Serial Flash.
 - OpParamPtr->WritePtr is a pointer to the data to be written to the Serial Flash.
 - OpParamPtr->NumBytes is the number of bytes to be written to Serial Flash.
 - This operation is supported for Atmel, Intel, STM, Winbond and Spansion Serial Flash.
- Auto Page Write (XISF_AUTO_PAGE_WRITE):

- The OpParamPtr must be of 32 bit unsigned integer variable.
 - This is the address of page number in the Serial Flash which is to be refreshed.
 - This operation is only supported for Atmel Serial Flash.
- Buffer Write (XISF_BUFFER_WRITE):
 - The OpParamPtr must be of type struct Xlsf_BufferToFlashWriteParam.
 - OpParamPtr->BufferNum specifies the internal SRAM Buffer of the Serial Flash. The valid values are XISF_PAGE_BUFFER1 or XISF_PAGE_BUFFER2. XISF_PAGE_BUFFER2 is not valid in case of AT45DB011D Flash as it contains a single buffer.
 - OpParamPtr->WritePtr is a pointer to the data to be written to the Serial Flash SRAM Buffer.
 - OpParamPtr->ByteOffset is byte offset in the buffer from where the data is to be written.
 - OpParamPtr->NumBytes is number of bytes to be written to the Buffer. This operation is supported only for Atmel Serial Flash.
- Buffer To Memory Write With Erase (XISF_BUF_TO_PAGE_WRITE_WITH_ERASE)/ Buffer To Memory Write Without Erase (XISF_BUF_TO_PAGE_WRITE_WITHOUT_ERASE):
 - The OpParamPtr must be of type struct Xlsf_BufferToFlashWriteParam.
 - OpParamPtr->BufferNum specifies the internal SRAM Buffer of the Serial Flash. The valid values are XISF_PAGE_BUFFER1 or XISF_PAGE_BUFFER2. XISF_PAGE_BUFFER2 is not valid in case of AT45DB011D Flash as it contains a single buffer.
 - OpParamPtr->Address is starting address in the Serial Flash memory from where the data is to be written. These operations are only supported for Atmel Serial Flash.
- Write Status Register (XISF_WRITE_STATUS_REG):
 - The OpParamPtr must be of type of 8 bit unsigned integer variable. This is the value to be written to the Status Register.
 - This operation is only supported for Intel, STM Winbond and Spansion Serial Flash.
- Write Status Register2 (XISF_WRITE_STATUS_REG2):
 - The OpParamPtr must be of type (u8 *) and should point to two 8 bit unsigned integer values. This is the value to be written to the 16 bit Status Register. This operation is only supported in Winbond (W25Q) Serial Flash.
- One Time Programmable Area Write(XISF_OTP_WRITE):
 - The OpParamPtr must be of type struct Xlsf_WriteParam.
 - OpParamPtr->Address is the address in the SRAM Buffer of the Serial Flash to which the data is to be written.
 - OpParamPtr->WritePtr is a pointer to the data to be written to the Serial Flash.
 - OpParamPtr->NumBytes should be set to 1 when performing OTPWrite operation. This operation is only supported for Intel Serial Flash.

Returns

XST_SUCCESS if successful else XST_FAILURE.

Note

- Application must fill the structure elements of the third argument and pass its pointer by type casting it with void pointer.
- For Intel, STM, Winbond and Spansion Serial Flash, the user application must call the [Xlsf_WriteEnable\(\)](#) API by passing XISF_WRITE_ENABLE as an argument, before calling the [Xlsf_Write\(\)](#) API.

int Xlsf_Read (Xlsf * InstancePtr, Xlsf_ReadOperation Operation, void * OpParamPtr)

This API reads the data from the Serial Flash.

Parameters

<i>InstancePtr</i>	Pointer to the Xlsf instance.
<i>Operation</i>	Type of the read operation to be performed on the Serial Flash. The different operations are • XISF_READ: Normal Read • XISF_FAST_READ: Fast Read • XISF_PAGE_TO_BUF_TRANS: Page to Buffer Transfer • XISF_BUFFER_READ: Buffer Read • XISF_FAST_BUFFER_READ: Fast Buffer Read • XISF_OTP_READ: One Time Programmable Area (OTP) Read • XISF_DUAL_OP_FAST_READ: Dual Output Fast Read • XISF_DUAL_IO_FAST_READ: Dual Input/Output Fast Read • XISF_QUAD_OP_FAST_READ: Quad Output Fast Read • XISF_QUAD_IO_FAST_READ: Quad Input/Output Fast Read
<i>OpParamPtr</i>	Pointer to structure variable which contains operational parameter of specified Operation. This parameter type is dependant on the type of Operation to be performed. For more details, refer Operations .

Operations

- Normal Read (XISF_READ), Fast Read (XISF_FAST_READ), One Time Programmable Area Read(XISF_OTP_READ), Dual Output Fast Read (XISF_CMD_DUAL_OP_FAST_READ), Dual Input/Output Fast Read (XISF_CMD_DUAL_IO_FAST_READ), Quad Output Fast Read (XISF_CMD_QUAD_OP_FAST_READ) and Quad Input/Output Fast Read (XISF_CMD_QUAD_IO_FAST_READ):
 - The OpParamPtr must be of type struct Xlsf_ReadParam.

- OpParamPtr->Address is start address in the Serial Flash.
- OpParamPtr->ReadPtr is a pointer to the memory where the data read from the Serial Flash is stored.
- OpParamPtr->NumBytes is number of bytes to read.
- OpParamPtr->NumDummyBytes is the number of dummy bytes to be transmitted for the Read command. This parameter is only used in case of Dual and Quad reads.
- Normal Read and Fast Read operations are supported for Atmel, Intel, STM, Winbond and Spansion Serial Flash.
- Dual and quad reads are supported for Winbond (W25QXX), Numonyx(N25QXX) and Spansion (S25FL129) quad flash.
- OTP Read operation is only supported in Intel Serial Flash.
- Page To Buffer Transfer (XISF_PAGE_TO_BUF_TRANS):
 - The OpParamPtr must be of type struct Xlzf_FlashToBufTransferParam .
 - OpParamPtr->BufferNum specifies the internal SRAM Buffer of the Serial Flash. The valid values are XISF_PAGE_BUFFER1 or XISF_PAGE_BUFFER2. XISF_PAGE_BUFFER2 is not valid in case of AT45DB011D Flash as it contains a single buffer.
 - OpParamPtr->Address is start address in the Serial Flash. This operation is only supported in Atmel Serial Flash.
- Buffer Read (XISF_BUFFER_READ) and Fast Buffer Read(XISF_FAST_BUFFER_READ):
 - The OpParamPtr must be of type struct Xlzf_BufferReadParam.
 - OpParamPtr->BufferNum specifies the internal SRAM Buffer of the Serial Flash. The valid values are XISF_PAGE_BUFFER1 or XISF_PAGE_BUFFER2. XISF_PAGE_BUFFER2 is not valid in case of AT45DB011D Flash as it contains a single buffer.
 - OpParamPtr->ReadPtr is pointer to the memory where data read from the SRAM buffer is to be stored.
 - OpParamPtr->ByteOffset is byte offset in the SRAM buffer from where the first byte is read.
 - OpParamPtr->NumBytes is the number of bytes to be read from the Buffer. These operations are supported only in Atmel Serial Flash.

Returns

XST_SUCCESS if successful else XST_FAILURE.

Note

- Application must fill the structure elements of the third argument and pass its pointer by type casting it with void pointer.
- The valid data is available from the fourth location pointed to by the ReadPtr for Normal Read and Buffer Read operations.
- The valid data is available from fifth location pointed to by the ReadPtr for Fast Read, Fast Buffer Read and OTP Read operations.
- The valid data is available from the (4 + NumDummyBytes)th location pointed to by ReadPtr for Dual/Quad Read operations.

int Xlzf_Erase (Xlzf * InstancePtr, Xlzf_EraseOperation Operation, u32 Address)

This API erases the contents of the specified memory in the Serial Flash.

Parameters

<i>InstancePtr</i>	Pointer to the Xlzf instance.
<i>Operation</i>	Type of Erase operation to be performed on the Serial Flash. The different operations are • XISF_PAGE_ERASE: Page Erase • XISF_BLOCK_ERASE: Block Erase • XISF_SECTOR_ERASE: Sector Erase • XISF_BULK_ERASE: Bulk Erase
<i>Address</i>	Address of the Page/Block/Sector to be erased. The address can be either Page address, Block address or Sector address based on the Erase operation to be performed.

Returns

XST_SUCCESS if successful else XST_FAILURE.

Note

- The erased bytes will read as 0xFF.
- For Intel, STM, Winbond or Spansion Serial Flash the user application must call [Xlzf_WriteEnable\(\)](#) API by passing XISF_WRITE_ENABLE as an argument before calling [Xlzf_Erase\(\)](#) API.
- Atmel Serial Flash support Page/Block/Sector Erase operations.
- Intel, Winbond, Numonyx (N25QXX) and Spansion Serial Flash support Sector/Block/Bulk Erase operations.
- STM (M25PXX) Serial Flash support Sector/Bulk Erase operations.

int Xlzf_MicronFlashEnter4BAddrMode (Xlzf * InstancePtr)

This API enters the Micron flash device into 4 bytes addressing mode.

As per the Micron spec, before issuing the command to enter into 4 byte addr mode, a write enable command is issued.

Parameters

<i>InstancePtr</i>	Pointer to the Xlzf instance.
--------------------	-------------------------------

Returns

XST_SUCCESS if successful else XST_FAILURE.

Note

Applicable only for Micron flash devices

int Xlzf_MicronFlashExit4BAddMode (Xlzf * InstancePtr)

This API exits the Micron flash device from 4 bytes addressing mode.

As per the Micron spec, before issuing this command a write enable command is first issued.

Parameters

<i>InstancePtr</i>	Pointer to the Xlzf instance.
--------------------	-------------------------------

Returns

XST_SUCCESS if successful else XST_FAILURE.

Note

Applicable only for Micron flash devices

int Xlzf_SectorProtect (Xlzf * InstancePtr, Xlzf_SpOperation Operation, u8 * BufferPtr)

This API is used for performing Sector Protect related operations.

Parameters

<i>InstancePtr</i>	Pointer to the Xlzf instance.
<i>Operation</i>	Type of Sector Protect operation to be performed on the Serial Flash. The different operations are • XISF_SPR_READ: Read Sector Protection Register • XISF_SPR_WRITE: Write Sector Protection Register • XISF_SPR_ERASE: Erase Sector Protection Register • XISF_SP_ENABLE: Enable Sector Protection • XISF_SP_DISABLE: Disable Sector Protection
<i>BufferPtr</i>	Pointer to the memory where the SPR content is read to/written from. This argument can be NULL if the Operation is SprErase, SpEnable and SpDisable.

Returns

- XST_SUCCESS if successful.
- XST_FAILURE if it fails.

Note

- The SPR content is stored at the fourth location pointed by the BufferPtr when performing XISF_SPR_READ operation.
- For Intel, STM, Winbond and Spansion Serial Flash, the user application must call the [Xlsf_WriteEnable\(\)](#) API by passing XISF_WRITE_ENABLE as an argument, before calling the [Xlsf_SectorProtect\(\)](#) API, for Sector Protect Register Write (XISF_SPR_WRITE) operation.
- Atmel Flash supports all these Sector Protect operations.
- Intel, STM, Winbond and Spansion Flash support only Sector Protect Read and Sector Protect Write operations.

int Xlsf_ioctl (Xlsf * InstancePtr, Xlsf_ioctlOperation Operation)

This API configures and controls the Intel, STM, Winbond and Spansion Serial Flash.

Parameters

<i>InstancePtr</i>	Pointer to the Xlsf instance.
<i>Operation</i>	Type of Control operation to be performed on the Serial Flash. The different control operations are • XISF_RELEASE_DPD: Release from Deep Power Down (DPD) Mode • XISF_ENTER_DPD: Enter DPD Mode • XISF_CLEAR_SR_FAIL_FLAGS: Clear Status Register Fail Flags

Returns

XST_SUCCESS if successful else XST_FAILURE.

Note

- Atmel Serial Flash does not support any of these operations.
- Intel Serial Flash support Enter/Release from DPD Mode and Clear Status Register Fail Flags.
- STM, Winbond and Spansion Serial Flash support Enter/Release from DPD Mode.
- Winbond (W25QXX) Serial Flash support Enable High Performance mode.

int Xlzf_WriteEnable (Xlzf * *InstancePtr*, u8 *WriteEnable*)

This API Enables/Disables writes to the Intel, STM, Winbond and Spansion Serial Flash.

Parameters

<i>InstancePtr</i>	Pointer to the Xlzf instance.
<i>WriteEnable</i>	Specifies whether to Enable (XISF_CMD_ENABLE_WRITE) or Disable (XISF_CMD_DISABLE_WRITE) the writes to the Serial Flash.

Returns

XST_SUCCESS if successful else XST_FAILURE.

Note

This API works only for Intel, STM, Winbond and Spansion Serial Flash. If this API is called for Atmel Flash, XST_FAILURE is returned.

void Xlzf_RegisterInterface (Xlzf * *InstancePtr*)

This API registers the interface SPI/SPI PS/QSPI PS.

Parameters

<i>InstancePtr</i>	Pointer to the Xlzf instance.
--------------------	-------------------------------

Returns

None

int Xlzf_SetSpiConfiguration (Xlzf * *InstancePtr*, Xlzf_Iface * *SpiInstPtr*, u32 *Options*, u8 *PreScaler*)

This API sets the configuration of SPI.

This will set the options and clock prescaler (if applicable).

Parameters

<i>InstancePtr</i>	Pointer to the Xlzf instance.
<i>SpiInstPtr</i>	Pointer to Xlzf_Iface instance to be worked on.
<i>Options</i>	Specified options to be set.
<i>PreScaler</i>	Value of the clock prescaler to set.

Returns

XST_SUCCESS if successful else XST_FAILURE.

Note

This API can be called before calling [Xlslf_Initialize\(\)](#) to initialize the SPI interface in other than default options mode. PreScaler is only applicable to PS SPI/QSPI.

void Xlslf_SetStatusHandler (Xlslf * *InstancePtr*, Xlslf_Iface * *XlslfInstancePtr*, Xlslf_StatusHandler *Xlslf_Handler*)

This API is to set the Status Handler when an interrupt is registered.

Parameters

<i>InstancePtr</i>	Pointer to the Xlslf Instance.
<i>XlslfInstancePtr</i>	Pointer to the Xlslf_Iface instance to be worked on.
<i>Xlslf_Handler</i>	Status handler for the application.

Returns

None

Note

None.

void Xlslf_IfaceHandler (void * *CallbackRef*, u32 *StatusEvent*, unsigned int *ByteCount*)

This API is the handler which performs processing for the QSPI driver.

It is called from an interrupt context such that the amount of processing performed should be minimized. It is called when a transfer of QSPI data completes or an error occurs.

This handler provides an example of how to handle QSPI interrupts but is application specific.

Parameters

<i>CallbackRef</i>	Reference passed to the handler.
<i>StatusEvent</i>	Status of the QSPI .
<i>ByteCount</i>	Number of bytes transferred.

Returns

None



Note

None.

Library Parameters in MSS File

Xilissf Library can be integrated with a system using the following snippet in the Microprocessor Software Specification (MSS) file:

```
BEGIN LIBRARY
PARAMETER LIBRARY_NAME = xilissf
PARAMETER LIBRARY_VER = 5.13
PARAMETER serial_flash_family = 1
PARAMETER serial_flash_interface = 1
END
```

The table below describes the libgen customization parameters.

Parameter	Default Value	Description
LIBRARY_NAME	xilissf	Specifies the library name.
LIBRARY_VER	5.13	Specifies the library version.
serial_flash_family	1	Specifies the serial flash family. Supported numerical values are: 1 = Xilinx In-system Flash or Atmel Serial Flash 2 = Intel (Numonyx) S33 Serial Flash 3 = STM (Numonyx) M25PXX/N25QXX Serial Flash 4 = Winbond Serial Flash 5 = Spansion Serial Flash/Micron Serial Flash/Cypress Serial Flash 6 = SST Serial Flash
Serial_flash_interface	1	Specifies the serial flash interface. Supported numerical values are: 1 = AXI QSPI Interface 2 = SPI PS Interface 3 = QSPI PS Interface or QSPI PSU Interface 4 = OSPIPSV Interface for OSPI

Note

Intel, STM, and Numonyx serial flash devices are now a part of Serial Flash devices provided by Micron.

Appendix H:

XiIFFS Library v4.1

Overview

The Xilinx fat file system (FFS) library consists of a file system and a glue layer.

This FAT file system can be used with an interface supported in the glue layer.

The file system code is open source and is used as it is. Currently, the Glue layer implementation supports the SD/eMMC interface and a RAM based file system.

Application should make use of APIs provided in ff.h. These file system APIs access the driver functions through the glue layer.

The file system supports FAT16, FAT32, and exFAT (optional). The APIs are standard file system APIs. For more information, see the http://elm-chan.org/fsw/ff/00index_e.html.

Note

The XilFFS library uses Revision R0.13b of the generic FAT filesystem module.

Library Files

The table below lists the file system files.

File	Description
ff.c	Implements all the file system APIs
ff.h	File system header
ffconf.h	File system configuration header – File system configurations such as READ_ONLY, MINIMAL, can be set here. This library uses FF_FS_MINIMIZE and FF_FS_TINY and Read/Write (NOT read only)

The table below lists the glue layer files.

File	Description
diskio.c	Glue layer – implements the function used by file system to call the driver APIs
ff.h	File system header
diskio.h	Glue layer header

Selecting a File System with an SD Interface

To select a file system with an SD interface:

1. Launch Xilinx SDK. Xilinx SDK prompts you to create a workspace.
2. Select **File > New > Xilinx Board Support Package**. The **New Board Support Package** wizard appears.
3. Specify a project name.
4. Select **Standalone** from the **Board Support Package OS** drop-down list. The **Board Support Package Settings** wizard appears.
5. Select the **xilffs** library from the list of **Supported Libraries**.
6. Expand the **Overview** tree and select **xilffs**. The configuration options for xilffs are listed.
7. Configure the xilffs by setting the `fs_interface = 1` to select the SD/eMMC. This is the default value. Ensure that the SD/eMMC interface is available, prior to selecting the `fs_interface = 1` option.
8. Build the bsp and the application to use the file system with SD/eMMC. SD or eMMC will be recognized by the low level driver.

Selecting a RAM based file system

To select a RAM based file system:

1. Launch Xilinx SDK. Xilinx SDK prompts you to create a workspace.
2. Select **File > New > Xilinx Board Support Package**. The **New Board Support Package** wizard appears.
3. Specify a project name.
4. Select **Standalone** from the **Board Support Package OS** drop-down list. The **Board Support Package Settings** wizard appears.
5. Select the **xilffs** library from the list of **Supported Libraries**.
6. Expand the **Overview** tree and select **xilffs**. The configuration options for xilffs are listed.
7. Configure the xilffs by setting the `fs_interface = 2` to select RAM.

8. As this project is used by LWIP based application, select `lwip` library and configure according to your requirements. For more information, see the LwIP Library documentation.
9. Use any lwip application that requires a RAM based file system - TCP/UDP performance test apps or tftp or webserver examples.
10. Build the bsp and the application to use the RAM based file system.

Library Parameters in MSS File

XilFFS Library can be integrated with a system using the following code snippet in the Microprocessor Software Specification (MSS) file:

```
BEGIN LIBRARY
PARAMETER LIBRARY_NAME = xilffs
PARAMETER LIBRARY_VER = 4.1
PARAMETER fs_interface = 1
PARAMETER read_only = false
PARAMETER use_lfn = 0
PARAMETER enable_multi_partition = false
PARAMETER num_logical_vol = 2
PARAMETER use_mkfs = true
PARAMETER use_strfunc = 0
PARAMETER set_fs_rpath = 0
PARAMETER enable_exfat = false
PARAMETER word_access = true
PARAMETER use_chmod = false
END
```

The table below describes the libgen customization parameters.

Parameter	Default Value	Description
LIBRARY_NAME	xilffs	Specifies the library name.
LIBRARY_VER	4.1	Specifies the library version.
fs_interface	1 for SD/eMMC 2 for RAM	File system interface. SD/eMMC and RAM based file system are supported.
read_only	False	Enables the file system in Read Only mode, if true. Default is false. For Zynq® UltraScale+™ MPSoC devices, sets this option as true.

Parameter	Default Value	Description
use_lfn	0	Enables the Long File Name(LFN) support if non-zero. 0: Disabled (Default) 1: LFN with static working buffer 2 (on stack) or 3 (on heap): Dynamic working buffer
enable_multi_partitio	False	Enables the multi partition support, if true.
num_logical_vol	2	Number of volumes (logical drives, from 1 to 10) to be used.
use_mkfs	True	Enables the mkfs support, if true. For Zynq UltraScale+ MPSoC devices, set this option as false.
use_strfunc	0	Enables the string functions (valid values 0 to 2). Default is 0.
set_fs_rpath	0	Configures relative path feature (valid values 0 to 2). Default is 0.
ramfs_size	3145728	Ram FS size is applicable only when RAM based file system is selected.
ramfs_start_addr	0x10000000	RAM FS start address is applicable only when RAM based file system is selected.
enable_exfat	false	Enables support for exFAT file system. 0: Disable exFAT 1: Enable exFAT(Also Enables LFN)
word_access	True	Enables word access for misaligned memory access platform.
use_chmod	false	Enables use of CHMOD functionality for changing attributes (valid only with read_only set to false).

Appendix I:

XilSecure Library v4.0

Overview

The XilSecure library provides APIs to access cryptographic accelerators on the Zynq® UltraScale+™ MPSoC devices. The library is designed to run on top of Xilinx standalone BSPs. It is tested for A53, R5 and MicroBlaze™. XilSecure is used during the secure boot process. The primary post-boot use case is to run this library on the PMU MicroBlaze with PMUFW to service requests from Uboot or Linux for cryptographic acceleration.

The XilSecure library includes:

- SHA-3/384 engine for 384 bit hash calculation.
- AES-GCM engine for symmetric key encryption and decryption using a 256-bit key.
- RSA engine for signature generation, signature verification, encryption and decryption. Key sizes supported include 2048, 3072, and 4096.



WARNING: SDK defaults to using a software stack in DDR and any variables used by XilSecure will be placed in DDR. For better security, change the linker settings to make sure the stack used by XilSecure is either in the OCM or the TCM.

Source Files

The source files for the library can be found at:

https://github.com/Xilinx/embeddedsw/blob/master/lib/sw_services/xilsecure/

AES-GCM

Overview

This software uses AES-GCM hardened cryptographic accelerator to encrypt or decrypt the provided data and requires a key of size 256 bits and initialization vector(IV) of size 96 bits.

XilSecure library supports the following features:

- Encryption of data with provided key and IV
- Decryption of data with provided key and IV
- Authentication using a GCM tag.
- Key loading based on key selection, the key can be either the user provided key loaded into the KUP key or the device key used during boot.

For either encryption or decryption the AES-GCM engine should be initialized first using the `XSecure_AesInitiaze` function.

AES Encryption Function Usage

When all the data to be encrypted is available, the `XSecure_AesEncryptData()` can be used. When all the data is not available, use the following functions in the suggested order:

1. `XSecure_AesEncryptInit()`
2. `XSecure_AesEncryptUpdate()` - This function can be called multiple times till input data is completed.

AES Decryption Function Usage

When all the data to be decrypted is available, the `XSecure_AesDecryptData()` can be used. When all the data is not available, use the following functions in the suggested order:

1. `XSecure_AesDecryptInit()`
2. `XSecure_AesDecryptUpdate()` - This function can be called multiple times till input data is completed.

During decryption, the passed in GCM tag will be compared to the GCM tag calculated by the engine. The two tags are then compared in the software and returned to the user as to whether or not the tags matched.



WARNING: when using the KUP key for encryption/decryption of the data, where the key is stored should be carefully considered. Key should be placed in an internal memory region that has access controls. Not doing so may result in security vulnerability.

PMU Firmware Board Support Package SECURE_ENVIRONMENT Flag

SECURE_ENVIRONMENT is a PMUFW board support package (BSP) flag. Set this flag if you plan to use the device key for encryption/decryption without authentication. By default, the SECURE_ENVIRONMENT flag is disabled.

Note

You can set the SECURE_ENVIRONMENT flag only for the post-boot use case when the XilSecure library is running on the PMU MicroBlaze™ and the PMUFW is performing the encryption/decryption service requests from the Uboot or the Linux operating system.

Modules

- [AES-GCM Error Codes](#)
- [AES-GCM API Example Usage](#)
- [AES-GCM Usage to decrypt Boot Image](#)

Function Documentation

s32 XSecure_AesInitialize (XSecure_Aes * InstancePtr, XCsuDma * CsuDmaPtr, u32 KeySel, u32 * IvPtr, u32 * KeyPtr)

This function initializes the instance pointer.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Aes instance.
<i>CsuDmaPtr</i>	Pointer to the XCsuDma instance.
<i>KeySel</i>	Key source for decryption, can be KUP/device key • XSECURE_CSU_AES_KEY_SRC_KUP :For KUP key • XSECURE_CSU_AES_KEY_SRC_DEV :For Device Key
<i>Iv</i>	Pointer to the Initialization Vector for decryption
<i>Key</i>	Pointer to Aes key in case KUP key is used. Pass Null if the device key is to be used.

Returns

XST_SUCCESS if initialization was successful.

Note

All the inputs are accepted in little endian format but the AES engine accepts the data in big endian format, The decryption and encryption functions in xsecure_aes handle the big endian to little endian conversion using the Xil_Htonl function, provided by the Xilinx xil_io library. If higher performance is needed, users can strictly use data in big endian format and modify the xsecure_aes functions to remove the use of the Xil_Htonl API.

u32 XSecure_AesDecryptInit (XSecure_Aes * InstancePtr, u8 * DecData, u32 Size, u8 * GcmTagAddr)

This function initializes the AES engine for decryption and is required to be called before calling XSecure_AesDecryptUpdate.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Aes instance.
<i>DecData</i>	Pointer in which decrypted data will be stored.
<i>Size</i>	Expected size of the data in bytes.
<i>GcmTagAddr</i>	Pointer to the GCM tag which needs to be verified during decryption of the data.

Returns

None

Note

If all of the data to be decrypted is available, the XSecure_AesDecryptData function can be used instead. Chunking will not be handled over here, it is handled by XSecure_AesChunkDecrypt()

s32 XSecure_AesDecryptUpdate (XSecure_Aes * InstancePtr, u8 * EncData, u32 Size)

This function decrypts the encrypted data passed in and updates the GCM tag from any previous calls. The size from XSecure_AesDecryptInit is decremented from the size passed into this function to determine when the GCM tag passed to XSecure_AesDecryptInit needs to be compared to the GCM tag calculated in the AES engine.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Aes instance.
<i>EncData</i>	Pointer to the encrypted data which needs to be decrypted.
<i>Size</i>	Expected size of data to be decrypted in bytes.

Returns

Final call of this API returns the status of GCM tag matching.

- XSECURE_CSU_AES_GCM_TAG_MISMATCH: If GCM tag is mismatched
- XSECURE_CSU_AES_ZEROIZATION_ERROR: If GCM tag is mismatched, zeroize the decrypted data and send the status of zeroization.
- XST_SUCCESS: If GCM tag is matching.

Note

When Size of the data equals to size of the remaining data that data will be treated as final data. This API can be called multiple times but sum of all Sizes should be equal to Size mention in init. Return of the final call of this API tells whether GCM tag is matching or not.

s32 XSecure_AesDecryptData (XSecure_Aes * InstancePtr, u8 * DecData, u8 * EncData, u32 Size, u8 * GcmTagAddr)

This function decrypts the encrypted data provided and updates the DecData buffer with decrypted data.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Aes instance.
<i>DecData</i>	Pointer to a buffer in which decrypted data will be stored.
<i>EncData</i>	Pointer to the encrypted data which needs to be decrypted.
<i>Size</i>	Size of data to be decrypted in bytes.

Returns

This API returns the status of GCM tag matching.

- XSECURE_CSU_AES_GCM_TAG_MISMATCH: If GCM tag was mismatched
- XST_SUCCESS: If GCM tag was matched.

Note

When using this function to decrypt data that was encrypted with XSecure_AesEncryptData, the GCM tag will be stored as the last sixteen (16) bytes of data in XSecure_AesEncryptData's Dst (destination) buffer and should be used as the GcmTagAddr's pointer.

s32 XSecure_AesDecrypt (XSecure_Aes * InstancePtr, u8 * Dst, const u8 * Src, u32 Length)

This function will handle the AES-GCM Decryption.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Aes instance.
<i>Src</i>	Pointer to encrypted data source location
<i>Dst</i>	Pointer to location where decrypted data will be written.
<i>Length</i>	Expected total length of decrypted image expected.

Returns

returns XST_SUCCESS if successful, or the relevant errorcode.

Note

This function is used for decrypting the Image's partition encrypted by Bootgen

u32 XSecure_AesEncryptInit (XSecure_Aes * InstancePtr, u8 * EncData, u32 Size)

This function is used to initialize the AES engine for encryption.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Aes instance.
<i>EncData</i>	Pointer of a buffer in which encrypted data along with GCM TAG will be stored. Buffer size should be Size of data plus 16 bytes.
<i>Size</i>	A 32 bit variable, which holds the size of the input data to be encrypted.

Returns

None

Note

If all of the data to be encrypted is available, the XSecure_AesEncryptData function can be used instead.

u32 XSecure_AesEncryptUpdate (XSecure_Aes * InstancePtr, const u8 * Data, u32 Size)

This function encrypts the clear-text data passed in and updates the GCM tag from any previous calls. The size from XSecure_AesEncryptInit is decremented from the size passed into this function to determine when the final CSU DMA transfer of data to the AES-GCM cryptographic core.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Aes instance.
<i>Data</i>	Pointer to the data for which encryption should be performed.
<i>Size</i>	A 32 bit variable, which holds the size of the input data in bytes.

Returns

None

Note

If all of the data to be encrypted is available, the XSecure_AesEncryptData function can be used instead.

u32 XSecure_AesEncryptData (XSecure_Aes * InstancePtr, u8 * Dst, const u8 * Src, u32 Len)

This function encrypts Len (length) number of bytes of the passed in Src (source) buffer and stores the encrypted data along with its associated 16 byte tag in the Dst (destination) buffer.

Parameters

<i>InstancePtr</i>	A pointer to the XSecure_Aes instance.
<i>Dst</i>	A pointer to a buffer where encrypted data along with GCM tag will be stored. The Size of buffer provided should be Size of the data plus 16 bytes
<i>Src</i>	A pointer to input data for encryption.
<i>Len</i>	Size of input data in bytes

Returns

None

Note

If data to be encrypted is not available in one buffer one can call [XSecure_AesEncryptInit\(\)](#) and update the AES engine with data to be encrypted by calling [XSecure_AesEncryptUpdate\(\)](#) API multiple times as required.

void XSecure_AesReset (XSecure_Aes * InstancePtr)

This function sets the and then clears the AES-GCM's reset line.

Parameters

<i>InstancePtr</i>	is a pointer to the XSecure_Aes instance.
--------------------	---

Returns

None

u32 XSecure_AesWaitForDone (XSecure_Aes * *InstancePtr*)

This function waits for AES completion or a timeout will occur, as indicated by the return of XST_FAILURE.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Aes instance.
--------------------	--------------------------------------

Returns

XST_SUCCESS if the AES operation has completed XST_FAILURE if a software timeout has occurred.

AES-GCM Error Codes

The table below lists the AES-GCM error codes.

Error Code	Error Value	Description
XSECURE_CSU_AES_GCM_TAG_MISMATCH	0x1	User provided GCM tag does not match with GCM calculated on data
XSECURE_CSU_AES_IMAGE_LEN_MISMATCH	0x2	When there is a Image length mismatch
XSECURE_CSU_AES_DEVICE_COPY_ERROR	0x3	When there is device copy error.
XSECURE_CSU_AES_ZEROIZATION_ERROR	0x4	When there is an error with Zeroization. Note In case of any error during Aes decryption, we perform zeroization of the decrypted data.
XSECURE_CSU_AES_KEY_CLEAR_ERROR	0x20	Error when clearing key storage registers after Aes operation.

AES-GCM API Example Usage

The following example illustrates the usage of AES encryption and decryption APIs.

```
static s32 SecureAesExample(void)
{
    XCsuDma_Config *Config;
    s32 Status;
    u32 Index;
    XCsuDma CsuDmaInstance;
    XSecure_Aes Secure_Aes;

    /* Initialize CSU DMA driver */
    Config = XCsuDma_LookupConfig(XSECURE_CSUDMA_DEVICEID);
    if (NULL == Config) {
        return XST_FAILURE;
    }

    Status = XCsuDma_CfgInitialize(&CsuDmaInstance, Config,
        Config->BaseAddress);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    /* Initialize the Aes driver so that it's ready to use */
    XSecure_AesInitialize(&Secure_Aes, &CsuDmaInstance,
        XSECURE_CSU_AES_KEY_SRC_KUP,
        (u32 *)Iv, (u32 *)Key);

    xil_printf("Data to be encrypted: \n\r");
    for (Index = 0; Index < XSECURE_DATA_SIZE; Index++) {
        xil_printf("%02x", Data[Index]);
    }
    xil_printf( "\r\n\n");

    /* Encryption of Data */
    /*
     * If all the data to be encrypted is contiguous one can call
     * XSecure_AesEncryptData API directly.
     */
    XSecure_AesEncryptInit(&Secure_Aes, EncData, XSECURE_DATA_SIZE);
    XSecure_AesEncryptUpdate(&Secure_Aes, Data, XSECURE_DATA_SIZE);

    xil_printf("Encrypted data: \n\r");
    for (Index = 0; Index < XSECURE_DATA_SIZE; Index++) {
        xil_printf("%02x", EncData[Index]);
    }
    xil_printf( "\r\n");

    xil_printf("GCM tag: \n\r");
    for (Index = 0; Index < XSECURE_SECURE_GCM_TAG_SIZE; Index++) {
        xil_printf("%02x", EncData[XSECURE_DATA_SIZE + Index]);
    }
    xil_printf( "\r\n\n");

    /* Decrypt's the encrypted data */
    /*
     * If data to be decrypted is contiguous one can also call
     * single API XSecure_AesDecryptData
     */
    XSecure_AesDecryptInit(&Secure_Aes, DecData, XSECURE_DATA_SIZE,
        EncData + XSECURE_DATA_SIZE);
    /* Only the last update will return the GCM TAG matching status */
    Status = XSecure_AesDecryptUpdate(&Secure_Aes, EncData,
        XSECURE_DATA_SIZE);
    if (Status != XST_SUCCESS) {
        xil_printf("Decryption failure- GCM tag was not matched\n\r");
    }
}
```

```

    return Status;
}

xil_printf("Decrypted data\n\r");
for (Index = 0; Index < XSECURE_DATA_SIZE; Index++) {
    xil_printf("%02x", DecData[Index]);
}
xil_printf( "\r\n");

/* Comparison of Decrypted Data with original data */
for (Index = 0; Index < XSECURE_DATA_SIZE; Index++) {
    if (Data[Index] != DecData[Index]) {
        xil_printf("Failure during comparison of the data\n\r");
        return XST_FAILURE;
    }
}

return XST_SUCCESS;
}

```

Note

Relevant examples are available in the <library-install-path>\examples folder. Where <library-install-path> is the XilSecure library installation path.

AES-GCM Usage to decrypt Boot Image

The Multiple key(Key Rolling) or Single key encrypted images will have the same format. The images include:

- Secure header - This includes the dummy AES key of 32byte + Block 0 IV of 12byte + DLC for Block 0 of 4byte + GCM tag of 16byte(Un-Enc).
- Block N - This includes the boot image data for the block N of n size + Block N+1 AES key of 32byte + Block N+1 IV of 12byte + GCM tag for Block N of 16byte(Un-Enc).

The Secure header and Block 0 will be decrypted using the device key or user provided key. If more than one block is found then the key and the IV obtained from previous block will be used for decryption.

Following are the instructions to decrypt an image:

1. Read the first 64 bytes and decrypt 48 bytes using the selected Device key.
2. Decrypt Block 0 using the IV + Size and the selected Device key.
3. After decryption, you will get the decrypted data+KEY+IV+Block Size. Store the KEY/IV into KUP/IV registers.
4. Using Block size, IV and the next Block key information, start decrypting the next block.
5. If the current image size is greater than the total image length, perform the next step. Else, go back to the previous step.
6. If there are failures, an error code is returned. Else, the decryption is successful.

RSA

Overview

The `xsecure_rsa.h` file contains hardware interface related information for the RSA hardware accelerator. This hardened cryptographic accelerator, within the CSU, performs the modulus math based on the Rivest-Shamir-Adelman (RSA) algorithm. It is an asymmetric algorithm.

Initialization & Configuration

The RSA driver instance can be initialized by using the `XSecure_RsaInitialize()` function. The method used for RSA implementation can take a pre-calculated value of $R^2 \bmod N$. If you do not have the pre-calculated exponential value pass NULL, the controller will take care of the exponential value.

Note

- From the RSA key modulus, the exponent should be extracted.
- For verification, PKCS v1.5 padding scheme has to be applied for comparing the data hash with decrypted hash.

Modules

- [RSA API Example Usage](#)

Function Documentation

s32 XSecure_RsaInitialize (XSecure_Rsa * *InstancePtr*, u8 * *Mod*, u8 * *ModExt*, u8 * *ModExpo*)

This function initializes a `XSecure_Rsa` structure with the default values required for operating the RSA cryptographic engine.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Rsa instance.
<i>Mod</i>	A character Pointer which contains the key Modulus of key size.
<i>ModExt</i>	A Pointer to the pre-calculated exponential ($R^2 \text{ Mod } N$) value. • NULL - if user doesn't have pre-calculated $R^2 \text{ Mod } N$ value, control will take care of this calculation internally.
<i>ModExpo</i>	Pointer to the buffer which contains key exponent.

Returns

XST_SUCCESS if initialization was successful.

Note

Modulus, ModExt and ModExpo are part of prtion signature when authenticated boot image is generated by bootgen, else the all of them should be extracted from the key.

u32 XSecure_RsaSignVerification (u8 * *Signature*, u8 * *Hash*, u32 *HashLen*)

This function verifies the RSA decrypted data provided is either matching with the provided expected hash by taking care of PKCS padding.

Parameters

<i>Signature</i>	Pointer to the buffer which holds the decrypted RSA signature
<i>Hash</i>	Pointer to the buffer which has the hash calculated on the data to be authenticated.
<i>HashLen</i>	Length of Hash used. • For SHA3 it should be 48 bytes • For SHA2 it should be 32 bytes

Returns

XST_SUCCESS if decryption was successful. XST_FAILURE in case of mismatch.

s32 XSecure_RsaPublicEncrypt (XSecure_Rsa * *InstancePtr*, u8 * *Input*, u32 *Size*, u8 * *Result*)

This function handles the RSA encryption with the public key components provided when initializing the RSA cryptographic core with the XSecure_RsaInitialize function.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Rsa instance.
<i>Input</i>	Pointer to the buffer which contains the input data to be encrypted.
<i>Size</i>	Key size in bytes, Input size also should be same as Key size mentioned. Inputs supported are • XSECURE_RSA_4096_KEY_SIZE • XSECURE_RSA_2048_KEY_SIZE • XSECURE_RSA_3072_KEY_SIZE
<i>Result</i>	Pointer to the buffer where resultant decrypted data to be stored .

Returns

XST_SUCCESS if encryption was successful.

Note

The Size passed here needs to match the key size used in the XSecure_RsaInitialize function.

s32 XSecure_RsaPrivateDecrypt (XSecure_Rsa * *InstancePtr*, u8 * *Input*, u32 *Size*, u8 * *Result*)

This function handles the RSA decryption with the private key components provided when initializing the RSA cryptographic core with the XSecure_RsaInitialize function.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Rsa instance.
<i>Input</i>	Pointer to the buffer which contains the input data to be decrypted.
<i>Size</i>	Key size in bytes, Input size also should be same as Key size mentioned. Inputs supported are XSECURE_RSA_4096_KEY_SIZE, XSECURE_RSA_2048_KEY_SIZE and XSECURE_RSA_3072_KEY_SIZE
<i>Result</i>	Pointer to the buffer where resultant decrypted data to be stored .

Returns

XST_SUCCESS if decryption was successful. Otherwise, an error code is returned.

- XSECURE_RSA_DATA_VALUE_ERROR - if input data is greater than modulus
- XST_FAILURE - on RSA operation failure

Note

The Size passed in needs to match the key size used in the XSecure_RsaInitialize function..

RSA API Example Usage

The following example illustrates the usage of the RSA library to encrypt data using the public key and to decrypt the data using private key.

Note

Application should take care of the padding.

```
u32 SecureRsaExample(void)
{
    u32 Index;

    /* RSA signature decrypt with private key */
    /*
     * Initialize the Rsa driver with private key components
     * so that it's ready to use
     */
    XSecure_RsaInitialize(&Secure_Rsa, Modulus, NULL, PrivateExp);

    if(XST_SUCCESS != XSecure_RsaPrivateDecrypt(&Secure_Rsa, Data,
        Size, Signature)) {
        xil_printf("Failed at RSA signature decryption\n\r");
        return XST_FAILURE;
    }

    xil_printf("\r\n Decrypted Signature with private key\r\n ");

    for(Index = 0; Index < Size; Index++) {
        xil_printf(" %02x ", Signature[Index]);
    }
    xil_printf(" \r\n ");

    /* Verification if Data is expected */
    for(Index = 0; Index < Size; Index++) {
        if (Signature[Index] != ExpectedSign[Index]) {
            xil_printf("\r\nError at verification of RSA signature"
                " Decryption\n\r");
            return XST_FAILURE;
        }
    }

    /* RSA signature encrypt with Public key components */

    /*
     * Initialize the Rsa driver with public key components
     * so that it's ready to use
     */
    XSecure_RsaInitialize(&Secure_Rsa, Modulus, NULL, (u8 *)&PublicExp);

    if(XST_SUCCESS != XSecure_RsaPublicEncrypt(&Secure_Rsa, Signature,
        Size, EncryptSignatureOut)) {
        xil_printf("\r\nFailed at RSA signature encryption\n\r");
        return XST_FAILURE;
    }
    xil_printf("\r\n Encrypted Signature with public key\r\n ");

    for(Index = 0; Index < Size; Index++) {
        xil_printf(" %02x ", EncryptSignatureOut[Index]);
    }
}
```



```
}

/* Verification if Data is expected */
for(Index = 0; Index < Size; Index++) {
    if (EncryptSignatureOut[Index] != Data[Index]) {
        xil_printf("\r\nError at verification of RSA signature"
            " encryption\n\r");
        return XST_FAILURE;
    }
}

return XST_SUCCESS;
}
```

Note

Relevant examples are available in the <library-install-path>\examples folder. Where <library-install-path> is the XilSecure library installation path.

SHA-3

Overview

This block uses the NIST-approved SHA-3 algorithm to generate a 384-bit hash on the input data. Because the SHA-3 hardware only accepts 104 byte blocks as the minimum input size, the input data will be padded with user selectable Keccak or NIST SHA-3 padding and is handled internally in the SHA-3 library.

Initialization & Configuration

The SHA-3 driver instance can be initialized using the [XSecure_Sha3Initialize\(\)](#) function. A pointer to CsuDma instance has to be passed during initialization as the CSU DMA will be used for data transfers to the SHA module.

SHA-3 Function Usage

When all the data is available on which the SHA3 hash must be calculated, the [XSecure_Sha3Digest\(\)](#) can be used with the appropriate parameters as described. When all the data is not available, use the SHA3 functions in the following order:

1. [XSecure_Sha3Start\(\)](#)
2. [XSecure_Sha3Update\(\)](#) - This function can be called multiple times until all input data has been passed to the SHA-3 cryptographic core.
3. [XSecure_Sha3Finish\(\)](#) - Provides the final hash of the data. To get intermediate hash values after each [XSecure_Sha3Update\(\)](#), you can call [XSecure_Sha3_ReadHash\(\)](#) after the [XSecure_Sha3Update\(\)](#) call.

Modules

- [SHA-3 API Example Usage](#)
-

Function Documentation

s32 XSecure_Sha3Initialize (XSecure_Sha3 * InstancePtr, XCsuDma * CsuDmaPtr)

This function initializes a XSecure_Sha structure with the default values required for operating the SHA3 cryptographic engine.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Sha3 instance.
<i>CsuDmaPtr</i>	Pointer to the XCsuDma instance.

Returns

XST_SUCCESS if initialization was successful

Note

The base address is initialized directly with value from xsecure_hw.h The default is NIST SHA3 padding, to change to KECCAK padding call [XSecure_Sha3PadSelection\(\)](#) after [XSecure_Sha3Initialize\(\)](#).

void XSecure_Sha3Start (XSecure_Sha3 * InstancePtr)

This function configures Secure Stream Switch and starts the SHA-3 engine.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Sha3 instance.
--------------------	---------------------------------------

Returns

None

u32 XSecure_Sha3Update (XSecure_Sha3 * InstancePtr, const u8 * Data, const u32 Size)

This function updates the hash with the input data buffer.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Sha3 instance.
<i>Data</i>	Pointer to the input data for hashing.
<i>Size</i>	Size of the input data in bytes.

Returns

XST_SUCCESS if the update is successful XST_FAILURE if there is a failure in SSS config

u32 XSecure_Sha3Finish (XSecure_Sha3 * InstancePtr, u8 * Hash)

This function updates SHA3 engine with final data which includes SHA3 padding and reads final hash on complete data.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Sha3 instance.
<i>Hash</i>	Pointer to location where resulting hash will be written

Returns

XST_SUCCESS if finished without any errors XST_FAILURE if Sha3PadType is other than KECCAK or NIST

u32 XSecure_Sha3Digest (XSecure_Sha3 * InstancePtr, const u8 * In, const u32 Size, u8 * Out)

This function calculates the SHA-3 digest on the given input data.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Sha3 instance.
<i>In</i>	Pointer to the input data for hashing
<i>Size</i>	Size of the input data
<i>Out</i>	Pointer to location where resulting hash will be written.

Returns

XST_SUCCESS if digest calculation done successfully XST_FAILURE if any error from Sha3Update or Sha3Finish.

void XSecure_Sha3_ReadHash (XSecure_Sha3 * InstancePtr, u8 * Hash)

This function reads the SHA3 hash of the data and it can be called between calls to XSecure_Sha3Update.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Sha3 instance.
<i>Hash</i>	Pointer to a buffer in which read hash will be stored.

Returns

None

s32 XSecure_Sha3PadSelection (XSecure_Sha3 *InstancePtr, XSecure_Sha3PadType Sha3PadType)

This function provides an option to select the SHA-3 padding type to be used while calculating the hash.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Sha3 instance.
<i>Sha3Type</i>	Type of SHA3 padding to be used. - For NIST SHA-3 padding - XSECURE_CSU_NIST_SHA3 - For KECCAK SHA-3 padding - XSECURE_CSU_KECCAK_SHA3

Returns

XST_SUCCESS if pad selection is successfull. XST_FAILURE if pad selecction is failed.

Note

The default provides support for NIST SHA-3. If a user wants to change the padding to Keccak SHA-3, this function should be called after [XSecure_Sha3Initialize\(\)](#)

u32 XSecure_Sha3WaitForDone (XSecure_Sha3 *InstancePtr)

This function waits till SHA3 completes its action.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Sha3 instance.
--------------------	---------------------------------------

Returns

XST_SUCCESS if SHA3 completes it action properly XST_FAILURE if Timeout happens

s32 XSecure_Sha3LastUpdate (XSecure_Sha3 * InstancePtr)

This function is to notify this is the last update of data where sha padding is also been included along with the data in the next update call.

Parameters

<i>InstancePtr</i>	Pointer to the XSecure_Sha3 instance.
--------------------	---------------------------------------

Returns

XST_SUCCESS if last update can be accepted

SHA-3 API Example Usage

The `xilsecure_sha_example.c` file is a simple example application that demonstrates the usage of SHA-3 accelerator to calculate a 384-bit hash on the Hello World string. A typical use case for the SHA3 accelerator is for calculation of the boot image hash as part of the authentication operation. This is illustrated in the `xilsecure_rsa_example.c`.

The contents of the `xilsecure_sha_example.c` file are shown below:

```
int SecureHelloWorldExample()
{
    u8 HelloWorld[4] = {'h','e','l','l'};
    u32 Size = sizeof(HelloWorld);
    u8 Out[384/8];
    XCsuDma_Config *Config;

    int Status;

    Config = XCsuDma_LookupConfig(0);
    if (NULL == Config) {
        xil_printf("config failed\n\r");
        return XST_FAILURE;
    }

    Status = XCsuDma_CfgInitialize(&CsuDma, Config, Config->BaseAddress);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    /*
     * Initialize the SHA-3 driver so that it's ready to use
     */
    XSecure_Sha3Initialize(&Secure_Sha3, &CsuDma);

    XSecure_Sha3Digest(&Secure_Sha3, HelloWorld, Size, Out);

    xil_printf(" Calculated Digest \r\n ");
    int i= 0;
    for(i=0; i< (384/8); i++)
    {
        xil_printf(" %0x ", Out[i]);
    }
    xil_printf(" \r\n ");

    return XST_SUCCESS;
}
```

Note

The `xilsecure_sha_example.c` and `xilsecure_rsa_example.c` example files are available in the `<library-install-path>\examples` folder. Where `<library-install-path>` is the XilSecure library installation path.

XilSecure Utilities

Overview

The `xsecure_utils.h` file contains common functions used among the XilSecure library like holding hardware crypto engines in Reset or bringing them out of reset, and secure stream switch configuration for AES and SHA3.

Function Documentation

`void XSecure_SetReset ( u32 BaseAddress, u32 Offset )`

This function places the hardware core into the reset.

Parameters

<i>BaseAddress</i>	Base address of the core.
<i>BaseAddress</i>	Offset of the reset register.

Returns

None

`void XSecure_ReleaseReset ( u32 BaseAddress, u32 Offset )`

This function takes the hardware core out of reset.

Parameters

<i>BaseAddress</i>	Base address of the core.
<i>BaseAddress</i>	Offset of the reset register.

Returns

None

void XSecure_SssInitialize (XSecure_Sss * InstancePtr)

This function initializes the secure stream switch instance.

Parameters

<i>InstancePtr</i>	Instance pointer to the XSecure_Sss.
--------------------	--------------------------------------

u32 XSecure_SssAes (XSecure_Sss * InstancePtr, XSecure_SssSrc InputSrc, XSecure_SssSrc OutputSrc)

This function configures the secure stream switch for AES engine.

Parameters

<i>InstancePtr</i>	Instance pointer to the XSecure_Sss
<i>InputSrc</i>	Input DMA to be selected for AES engine.
<i>OutputSrc</i>	Output DMA to be selected for AES engine.

Returns

- XST_SUCCESS - on successful configuration of the switch

Note

InputSrc, OutputSrc are of type XSecure_SssSrc.

u32 XSecure_SssSha (XSecure_Sss * InstancePtr, u16 Dmald)

This function configures the secure stream switch for SHA hardware engine.

Parameters

<i>InstancePtr</i>	Instance pointer to the XSecure_Sss
<i>Dmald</i>	Device ID of DMA which is to be used as an input to the SHA engine.

Returns

- XST_SUCCESS - on successful configuration of the switch.

u32 XSecure_SssDmaLoopBack (XSecure_Sss * InstancePtr, u16 Dmald)

This function configures secure stream switch to set DMA in loop back mode.

Parameters

<i>InstancePtr</i>	Instance pointer to the XSecure_Sss
<i>DmaId</i>	Device ID of DMA.

Returns

- XST_SUCCESS - on successful configuration of the switch.

Appendix J:

XilSKey Library v6.7

Overview

The XilSKey library provides APIs for programming and reading eFUSE bits and for programming the battery-backed RAM (BBRAM) of Zynq®-7000 SoC, UltraScale™, UltraScale+™ and the Zynq UltraScale+ MPSoC devices.

- In Zynq-7000 devices:
 - PS eFUSE holds the RSA primary key hash bits and user feature bits, which can enable or disable some Zynq-7000 processor features.
 - PL eFUSE holds the AES key, the user key and some of the feature bits.
 - PL BBRAM holds the AES key.
- In UltraScale or UltraScale+:
 - PL eFUSE holds the AES key, 32 bit and 128 bit user key, RSA hash and some of the feature bits.
 - PL BBRAM holds AES key with or without DPA protection enable or obfuscated key programming.
- In Zynq UltraScale+ MPSoC:
 - PS eFUSE holds:
 - Programming AES key and can perform CRC verification of AES key
 - Programming/Reading User fuses
 - Programming/Reading PPK0/PPK1 sha3 hash
 - Programming/Reading SPKID
 - Programming/Reading secure control bits
 - BBRAM holds the AES key.

Note

Due to the added support for the SSIT devices, it is recommended to use the updated library with updated examples only for the UltraScale and the UltraScale+ devices.

Hardware Setup

This section describes the hardware setup required for programming PL BBRAM or PL eFUSE.

Hardware setup for Zynq PL

This chapter describes the hardware setup required for programming BBRAM or eFUSE of Zynq PL devices. PL eFUSE or PL BBRAM is accessed through PS via MIO pins which are used for communication PL eFUSE or PL BBRAM through JTAG signals, these can be changed depending on the hardware setup.

A hardware setup which dedicates four MIO pins for JTAG signals should be used and the MIO pins should be mentioned in application header file (xilskey_input.h). There should be a method to download this example and have the MIO pins connected to JTAG before running this application. You can change the listed pins at your discretion.

MUX Usage Requirements

To write the PL eFUSE or PL BBRAM using a driver you must:

- Use four MIO lines (TCK,TMS,TDO,TDI)
- Connect the MIO lines to a JTAG port

If you want to switch between the external JTAG and JTAG operation driven by the MIOs, you must:

- Include a MUX between the external JTAG and the JTAG operation driven by the MIOs
- Assign a MUX selection PIN

To rephrase, to select JTAG for PL EFUSE or PL BBRAM writing, you must define the following:

- The MIOs used for JTAG operations (TCK,TMS,TDI,TDO).
- The MIO used for the MUX Select Line.
- The Value on the MUX Select line, to select JTAG for PL eFUSE or PL BBRAM writing.

The following graphic illustrates the correct MUX usage.

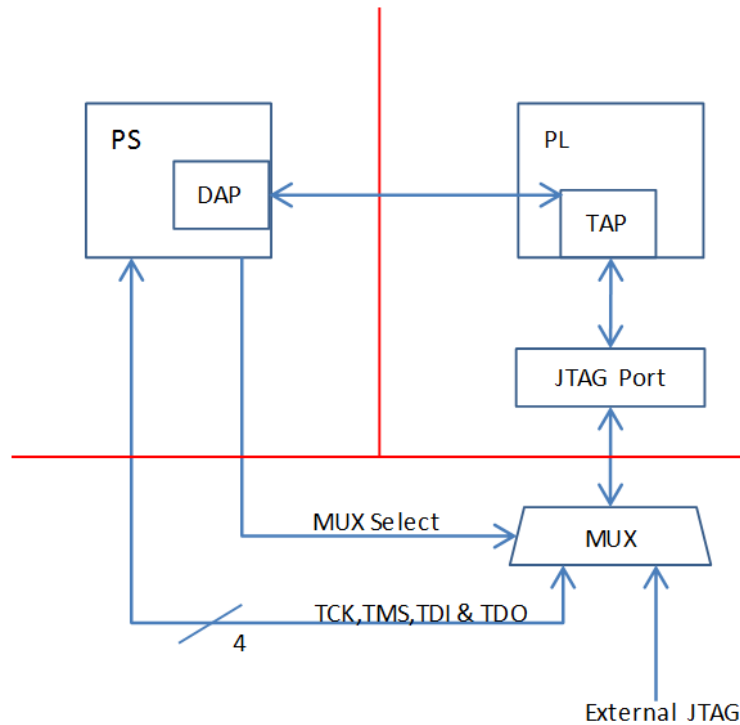


Figure 29.1: MUX Usage

Note

If you use the Vivado® Device Programmer tool to burn PL eFUSEs, there is no need for MUX circuitry or MIO pins.

Hardware setup for UltraScale or UltraScale+

This chapter describes the hardware setup required for programming BBRAM or eFUSE of UltraScale devices. Accessing UltraScale MicroBlaze eFuse is done by using block RAM initialization. UltraScale eFUSE programming is done through MASTER JTAG. Crucial Programming sequence will be taken care by Hardware module. It is mandatory to add Hardware module in the design. Use hardware module's vhd code and instructions provided to add Hardware module in the design.

- You need to add the Master JTAG primitive to design, that is, the MASTER_JTAG_inst instantiation has to be performed and AXI GPIO pins have to be connected to TDO, TDI, TMS and TCK signals of the MASTER_JTAG primitive.
- For programming eFUSE, along with master JTAG, hardware module(HWM) has to be added in design and it's signals XSK_EFUSEPL_AXI_GPIO_HWM_READY , XSK_EFUSEPL_AXI_GPIO_HWM_END and XSK_EFUSEPL_AXI_GPIO_HWM_START, needs to be connected to AXI GPIO pins to communicate with HWM. Hardware module is not mandatory for programming BBRAM. If your design has a HWM, it is not harmful for accessing BBRAM.

- All inputs (Master JTAG's TDO and HWM's HWM_READY, HWM_END) and all outputs (Master JTAG TDI, TMS, TCK and HWM's HWM_START) can be connected in one channel (or) inputs in one channel and outputs in other channel.
- Some of the outputs of GPIO in one channel and some others in different channels are not supported.
- The design should contain AXI BRAM control memory mapped (1MB).

Note

MASTER_JTAG will disable all other JTAGs.

For providing inputs of MASTER JTAG signals and HWM signals connected to the GPIO pins and GPIO channels, refer GPIO Pins Used for PL Master JTAG Signal and GPIO Channels sections of the UltraScale User-Configurable PL eFUSE Parameters and UltraScale User-Configurable PL BBRAM Parameters.

The procedure for programming BBRAM of eFUSE of UltraScale or UltraScale+ can be referred at UltraScale BBRAM Access Procedure and UltraScale eFUSE Access Procedure.

Source Files

The following is a list of eFUSE and BBRAM application project files, folders and macros.

- `xilskey_efuse_example.c`: This file contains the main application code. The file helps in the PS/PL structure initialization and writes/reads the PS/PL eFUSE based on the user settings provided in the `xilskey_input.h` file.
- `xilskey_input.h`: This file contains all the actions that are supported by the eFUSE library. Using the preprocessor directives given in the file, you can read/write the bits in the PS/PL eFUSE. More explanation of each directive is provided in the following sections. Burning or reading the PS/PL eFUSE bits is based on the values set in the `xilskey_input.h` file. Also contains GPIO pins and channels connected to MASTER JTAG primitive and hardware module to access Ultrascale eFUSE.

In this file:

- specify the 256 bit key to be programmed into BBRAM.
- specify the AES(256 bit) key, User (32 bit and 128 bit) keys and RSA key hash(384 bit) key to be programmed into UltraScale eFUSE.
- `XSK_EFUSEPS_DRIVER`: Define to enable the writing and reading of PS eFUSE.
- `XSK_EFUSEPL_DRIVER`: Define to enable the writing of PL eFUSE.
- `xilskey_bbram_example.c`: This file contains the example to program a key into BBRAM and verify the key.

Note

This algorithm only works when programming and verifying key are both executed in the recommended order.

- `xilskey_efuseps_zynqmp_example.c`: This file contains the example code to program the PS eFUSE and read back of eFUSE bits from the cache.

- `xilskey_efuseps_zynqmp_input.h`: This file contains all the inputs supported for eFUSE PS of Zynq UltraScale+ MPSoC. eFUSE bits are programmed based on the inputs from the `xilskey_efuseps_zynqmp_input.h` file.
- `xilskey_bbramps_zynqmp_example.c`: This file contains the example code to program and verify BBRAM key of Zynq UltraScale+ MPSoC. Default is zero. You can modify this key on top of the file.
- `xilskey_bbram_ultrascale_example.c`: This file contains example code to program and verify BBRAM key of UltraScale.

Note

Programming and verification of BBRAM key cannot be done separately.

- `xilskey_bbram_ultrascale_input.h`: This file contains all the preprocessor directives you need to provide. In this file, specify BBRAM AES key or Obfuscated AES key to be programmed, DPA protection enable and, GPIO pins and channels connected to MASTER JTAG primitive.
- `xilskey_puf_registration.c`: This file contains all the PUF related code. This example illustrates PUF registration and generating black key and programming eFUSE with PUF helper data, CHash and Auxiliary data along with the Black key.
- `xilskey_puf_registration.h`: This file contains all the preprocessor directives based on which read/write the eFUSE bits and Syndrome data generation. More explanation of each directive is provided in the following sections.



WARNING: *Ensure that you enter the correct information before writing or 'burning' eFUSE bits. Once burned, they cannot be changed. The BBRAM key can be programmed any number of times.*

Note

POR reset is required for the eFUSE values to be recognized.

BBRAM PL API

Overview

This chapter provides a linked summary and detailed descriptions of the battery-backed RAM (BBRAM) APIs of Zynq® PL and UltraScale™ devices.

Example Usage

- Zynq BBRAM PL example usage:
 - The Zynq BBRAM PL example application should contain the `xilskey_bbram_example.c` and `xilskey_input.h` files.
 - You should provide user configurable parameters in the `xilskey_input.h` file. For more information, refer [Zynq User-Configurable PL BBRAM Parameters](#).
- UltraScale BBRAM example usage:
 - The UltraScale BBRAM example application should contain the `xilskey_bbram_ultrascale_input.h` and `xilskey_bbram_ultrascale_example.c` files.
 - You should provide user configurable parameters in the `xilskey_bbram_ultrascale_input.h` file. For more information, refer [UltraScale or UltraScale+ User-Configurable BBRAM PL Parameters](#).

Note

It is assumed that you have set up your hardware prior to working on the example application. For more information, refer [Hardware Setup](#).

Functions

- int [XilSKey_Bbram_Program](#) (XilSKey_Bbram *InstancePtr)

Function Documentation

int XilSKey_Bbram_Program (XilSKey_Bbram * *InstancePtr*)

This function implements the BBRAM algorithm for programming and verifying key.
The program and verify will only work together in and in that order.

Parameters

<i>InstancePtr</i>	Pointer to XilSKey_Bbram
--------------------	--------------------------

Returns

- XST_FAILURE - In case of failure
- XST_SUCCESS - In case of Success

Note

This function will program BBRAM of Ultrascale and Zynq as well.

Zynq UltraScale+ MPSoC BBRAM PS API

Overview

This chapter provides a linked summary and detailed descriptions of the battery-backed RAM (BBRAM) APIs for Zynq® UltraScale+™ MPSoC devices.

Example Usage

- The Zynq UltraScale+ MPSoC example application should contain the `xilsky_bbramps_zynqmp_example.c` file.
- User configurable key can be modified in the same file (`xilsky_bbramps_zynqmp_example.c`), at the `XSK_ZYNQMP_BBRAMPS_AES_KEY` macro.

Functions

- u32 [XilSKey_ZynqMp_Bbram_Program](#) (u32 *AesKey)
- u32 [XilSKey_ZynqMp_Bbram_Zeroise](#) (void)

Function Documentation

u32 XilSKey_ZynqMp_Bbram_Program (u32 * AesKey)

This function implements the BBRAM programming and verifying the key written. Program and verification of AES will work only together. CRC of the provided key will be calculated internally and verified after programming.

Parameters

<i>AesKey</i>	Pointer to the key which has to be programmed.
---------------	--

Returns

- Error code from `XskZynqMp_Ps_Bbram_ErrorCodes` enum if it fails
- `XST_SUCCESS` if programming is done.



u32 XilSKey_ZynqMp_Bbram_Zeroise (void)

This function zeroize's Bbram Key.

Parameters

None.	
-------	--

Returns

None.

Note

BBRAM key will be zeroized.

Zynq eFUSE PS API

Overview

This chapter provides a linked summary and detailed descriptions of the Zynq eFUSE PS APIs.

Example Usage

- The Zynq eFUSE PS example application should contain the `xilsky_efuse_example.c` and the `xilsky_input.h` files.
- There is no need of any hardware setup. By default, both the eFUSE PS and PL are enabled in the application. You can comment 'XSK_EFUSEPL_DRIVER' to execute only the PS. For more details, refer [Zynq User-Configurable PS eFUSE Parameters](#).

Functions

- u32 [XilSKey_EfusePs_Write](#) (XilSKey_EPs *PsInstancePtr)
- u32 [XilSKey_EfusePs_Read](#) (XilSKey_EPs *PsInstancePtr)
- u32 [XilSKey_EfusePs_ReadStatus](#) (XilSKey_EPs *InstancePtr, u32 *StatusBits)

Function Documentation

u32 XilSKey_EfusePs_Write (XilSKey_EPs * *InstancePtr*)

PS eFUSE interface functions.

PS eFUSE interface functions.

Parameters

<i>InstancePtr</i>	Pointer to the PsEfuseHandle which describes which PS eFUSE bit should be burned.
--------------------	---

Returns

- XST_SUCCESS.
- In case of error, value is as defined in xilskey_utils.h. Error value is a combination of Upper 8 bit value and Lower 8 bit value. For example, 0x8A03 should be checked in error.h as 0x8A00 and 0x03. Upper 8 bit value signifies the major error and lower 8 bit values tell more precisely.

Note

When called, this initializes the timer, XADC subsystems. Unlocks the PS eFUSE controller. Configures the PS eFUSE controller. Writes the hash and control bits if requested. Programs the PS eFUSE to enable the RSA authentication if requested. Locks the PS eFUSE controller. Returns an error, if the reference clock frequency is not in between 20 and 60 MHz or if the system is not in a position to write the requested PS eFUSE bits (because the bits are already written or not allowed to write) or if the temperature and voltage are not within range.

u32 XilSKey_EfusePs_Read (XilSKey_EPs * InstancePtr)

This function is used to read the PS eFUSE.

Parameters

<i>InstancePtr</i>	Pointer to the PsEfuseHandle which describes which PS eFUSE should be burned.
--------------------	---

Returns

- XST_SUCCESS no errors occurred.
- In case of error, value is as defined in xilskey_utils.h. Error value is a combination of Upper 8 bit value and Lower 8 bit value. For example, 0x8A03 should be checked in error.h as 0x8A00 and 0x03. Upper 8 bit value signifies the major error and lower 8 bit values tell more precisely.

Note

When called: This API initializes the timer, XADC subsystems. Unlocks the PS eFUSE Controller. Configures the PS eFUSE Controller and enables read-only mode. Reads the PS eFUSE (Hash Value), and enables read-only mode. Locks the PS eFUSE Controller. Returns an error, if the reference clock frequency is not in between 20 and 60MHz. or if unable to unlock PS eFUSE controller or requested address corresponds to restricted bits. or if the temperature and voltage are not within range.

u32 XilSKey_EfusePs_ReadStatus (XilSKey_EPs * InstancePtr, u32 * StatusBits)

This function is used to read the PS efuse status register.

Parameters

<i>InstancePtr</i>	Pointer to the PS eFUSE instance.
<i>StatusBits</i>	Buffer to store the status register read.

Returns

- XST_SUCCESS.
- XST_FAILURE

Note

This API unlocks the controller and reads the Zynq PS eFUSE status register.

Zynq UltraScale+ MPSoC eFUSE PS API

Overview

This chapter provides a linked summary and detailed descriptions of the Zynq MPSoC UltraScale+ eFUSE PS APIs.

Example Usage

- For programming eFUSEs other than the PUF, the Zynq UltraScale+ MPSoC example application should contain the `xilsky_efuseps_zynqmp_example.c` and the `xilsky_efuseps_zynqmp_input.h` files.
 - For PUF registration, programming PUF helper data, AUX, chash, and black key, the Zynq UltraScale+ MPSoC example application should contain the `xilsky_puf_registration.c` and the `xilsky_puf_registration.h` files.
 - For more details on the user configurable parameters, refer [Zynq UltraScale+ MPSoC User-Configurable PS eFUSE Parameters](#) and [Zynq UltraScale+ MPSoC User-Configurable PS PUF Parameters](#).
-

Functions

- u32 [XilSKey_ZynqMp_EfusePs_CheckAesKeyCrc](#) (u32 CrcValue)
- u32 [XilSKey_ZynqMp_EfusePs_ReadUserFuse](#) (u32 *UseFusePtr, u8 UserFuse_Num, u8 ReadOption)
- u32 [XilSKey_ZynqMp_EfusePs_ReadPpk0Hash](#) (u32 *Ppk0Hash, u8 ReadOption)
- u32 [XilSKey_ZynqMp_EfusePs_ReadPpk1Hash](#) (u32 *Ppk1Hash, u8 ReadOption)
- u32 [XilSKey_ZynqMp_EfusePs_ReadSpkId](#) (u32 *SpkId, u8 ReadOption)
- void [XilSKey_ZynqMp_EfusePs_ReadDna](#) (u32 *DnaRead)
- u32 [XilSKey_ZynqMp_EfusePs_ReadSecCtrlBits](#) (XilSKey_SecCtrlBits *ReadBackSecCtrlBits, u8 ReadOption)
- u32 [XilSKey_ZynqMp_EfusePs_Write](#) (XilSKey_ZynqMpEPs *InstancePtr)
- u32 [XilSKey_ZynqMp_EfusePs_WritePufHelperData](#) (XilSKey_Puf *InstancePtr)
- u32 [XilSKey_ZynqMp_EfusePs_ReadPufHelperData](#) (u32 *Address)
- u32 [XilSKey_ZynqMp_EfusePs_WritePufChash](#) (XilSKey_Puf *InstancePtr)
- u32 [XilSKey_ZynqMp_EfusePs_ReadPufChash](#) (u32 *Address, u8 ReadOption)
- u32 [XilSKey_ZynqMp_EfusePs_WritePufAux](#) (XilSKey_Puf *InstancePtr)
- u32 [XilSKey_ZynqMp_EfusePs_ReadPufAux](#) (u32 *Address, u8 ReadOption)

- u32 [XilSKey_Write_Puf_EfusePs_SecureBits](#) (XilSKey_Puf_Secure *WriteSecureBits)
- u32 [XilSKey_Read_Puf_EfusePs_SecureBits](#) (XilSKey_Puf_Secure *SecureBitsRead, u8 ReadOption)
- u32 [XilSKey_Puf_Debug2](#) (XilSKey_Puf *InstancePtr)
- u32 [XilSKey_Puf_Registration](#) (XilSKey_Puf *InstancePtr)
- u32 [XilSKey_Puf_Regeneration](#) (XilSKey_Puf *InstancePtr)

Function Documentation

u32 XilSKey_ZynqMp_EfusePs_CheckAesKeyCrc (u32 CrcValue)

This function performs the CRC check of AES key.

Parameters

<i>CrcValue</i>	A 32 bit CRC value of an expected AES key.
-----------------	--

Returns

- XST_SUCCESS on successful CRC check.
- ErrorCode on failure

Note

For Calculating the CRC of the AES key use the [XilSKey_CrcCalculation\(\)](#) function or [XilSKey_CrcCalculation_AesKey\(\)](#) function

u32 XilSKey_ZynqMp_EfusePs_ReadUserFuse (u32 * UseFusePtr, u8 UserFuse_Num, u8 ReadOption)

This function is used to read a user fuse from the eFUSE or cache.

Parameters

<i>UseFusePtr</i>	Pointer to an array which holds the readback user fuse.
<i>UserFuse_Num</i>	A variable which holds the user fuse number. Range is (User fuses: 0 to 7)
<i>ReadOption</i>	Indicates whether or not to read from the actual eFUSE array or from the eFUSE cache. • 0(XSK_EFUSEPS_READ_FROM_CACHE) Reads from eFUSE cache • 1(XSK_EFUSEPS_READ_FROM_EFUSE) Reads from eFUSE array

Returns

- XST_SUCCESS on successful read
- ErrorCode on failure

u32 XilSKey_ZynqMp_EfusePs_ReadPpk0Hash (u32 * Ppk0Hash, u8 ReadOption)

This function is used to read the PPK0 hash from an eFUSE or eFUSE cache.

Parameters

<i>Ppk0Hash</i>	A pointer to an array which holds the readback PPK0 hash.
<i>ReadOption</i>	Indicates whether or not to read from the actual eFUSE array or from the eFUSE cache. • 0(XSK_EFUSEPS_READ_FROM_CACHE) Reads from eFUSE cache • 1(XSK_EFUSEPS_READ_FROM_EFUSE) Reads from eFUSE array

Returns

- XST_SUCCESS on successful read
- ErrorCode on failure

u32 XilSKey_ZynqMp_EfusePs_ReadPpk1Hash (u32 * Ppk1Hash, u8 ReadOption)

This function is used to read the PPK1 hash from eFUSE or cache.

Parameters

<i>Ppk1Hash</i>	Pointer to an array which holds the readback PPK1 hash.
<i>ReadOption</i>	Indicates whether or not to read from the actual eFUSE array or from the eFUSE cache. • 0(XSK_EFUSEPS_READ_FROM_CACHE) Reads from eFUSE cache • 1(XSK_EFUSEPS_READ_FROM_EFUSE) Reads from eFUSE array

Returns

- XST_SUCCESS on successful read
- ErrorCode on failure

u32 XilSKey_ZynqMp_EfusePs_ReadSpkId (u32 * *SpkId*, u8 *ReadOption*)

This function is used to read SPKID from eFUSE or cache based on user's read option.

Parameters

<i>SpkId</i>	Pointer to a 32 bit variable which holds SPK ID.
<i>ReadOption</i>	Indicates whether or not to read from the actual eFUSE array or from the eFUSE cache. 0(XSK_EFUSEPS_READ_FROM_CACHE) Reads from eFUSE cache 1(XSK_EFUSEPS_READ_FROM_EFUSE) Reads from eFUSE array

Returns

- XST_SUCCESS on successful read
- ErrorCode on failure

void XilSKey_ZynqMp_EfusePs_ReadDna (u32 * *DnaRead*)

This function is used to read DNA from eFUSE.

Parameters

<i>DnaRead</i>	Pointer to an array of 3 x u32 words which holds the readback DNA.
----------------	--

Returns

None.

u32 XilSKey_ZynqMp_EfusePs_ReadSecCtrlBits (XilSKey_SecCtrlBits * *ReadBackSecCtrlBits*, u8 *ReadOption*)

This function is used to read the PS eFUSE secure control bits from cache or eFUSE based on user input provided.

Parameters

<i>ReadBackSecCtrlBits</i>	Pointer to the XilSKey_SecCtrlBits which holds the read secure control bits.
<i>ReadOption</i>	Indicates whether or not to read from the actual eFUSE array or from the eFUSE cache. • 0(XSK_EFUSEPS_READ_FROM_CACHE) Reads from eFUSE cache • 1(XSK_EFUSEPS_READ_FROM_EFUSE) Reads from eFUSE array

Returns

- XST_SUCCESS if reads successfully
- XST_FAILURE if reading is failed

Note

Cache reload is required for obtaining updated values for ReadOption 0.

u32 XilSKey_ZynqMp_EfusePs_Write (XilSKey_ZynqMpEPs * *InstancePtr*)

This function is used to program the PS eFUSE of ZynqMP, based on user inputs.

Parameters

<i>InstancePtr</i>	Pointer to the XilSKey_ZynqMpEPs.
--------------------	-----------------------------------

Returns

- XST_SUCCESS if programs successfully.
- Errorcode on failure

Note

After eFUSE programming is complete, the cache is automatically reloaded so all programmed eFUSE bits can be directly read from cache.

u32 XilSKey_ZynqMp_EfusePs_WritePufHelprData (XilSKey_Puf * *InstancePtr*)

This function programs the PS eFUSEs with the PUF helper data.

Parameters

<i>InstancePtr</i>	Pointer to the XilSKey_Puf instance.
--------------------	--------------------------------------

Returns

- XST_SUCCESS if programs successfully.
- Errorcode on failure

Note

To generate PufSyndromeData please use XilSKey_Puf_Registration API

u32 XilSKey_ZynqMp_EfusePs_ReadPufHelprData (u32 * Address)

This function reads the PUF helper data from eFUSE.

Parameters

<i>Address</i>	Pointer to data array which holds the PUF helper data read from eFUSEs.
----------------	---

Returns

- XST_SUCCESS if reads successfully.
- Errorcode on failure.

Note

This function only reads from eFUSE non-volatile memory. There is no option to read from Cache.

u32 XilSKey_ZynqMp_EfusePs_WritePufChash (XilSKey_Puf * InstancePtr)

This function programs eFUSE with CHash value.

Parameters

<i>InstancePtr</i>	Pointer to the XilSKey_Puf instance.
--------------------	--------------------------------------

Returns

- XST_SUCCESS if chash is programmed successfully.
- An Error code on failure

Note

To generate the CHash value, please use XilSKey_PUF_Registration function.

u32 XilSKey_ZynqMp_EfusePs_ReadPufChash (u32 * Address, u8 ReadOption)

This function reads eFUSE PUF CHash data from the eFUSE array or cache based on the user read option.

Parameters

<i>Address</i>	Pointer which holds the read back value of the chash.
<i>ReadOption</i>	Indicates whether or not to read from the actual eFUSE array or from the eFUSE cache. 0(XSK_EFUSEPS_READ_FROM_CACHE) Reads from cache 1(XSK_EFUSEPS_READ_FROM_EFUSE) Reads from eFUSE array

Returns

- XST_SUCCESS if programs successfully.
- Errorcode on failure

Note

Cache reload is required for obtaining updated values for reading from cache..

u32 XilSKey_ZynqMp_EfusePs_WritePufAux (XilSKey_Puf * InstancePtr)

This function programs eFUSE PUF auxiliary data.

Parameters

<i>InstancePtr</i>	Pointer to the XilSKey_Puf instance.
--------------------	--------------------------------------

Returns

- XST_SUCCESS if the eFUSE is programmed successfully.
- Errorcode on failure

Note

To generate auxiliary data, please use XilSKey_Puf_Registration function.

u32 XilSKey_ZynqMp_EfusePs_ReadPufAux (u32 * Address, u8 ReadOption)

This function reads eFUSE PUF auxiliary data from eFUSE array or cache based on user read option.

Parameters

<i>Address</i>	Pointer which holds the read back value of PUF's auxiliary data.
<i>ReadOption</i>	Indicates whether or not to read from the actual eFUSE array or from the eFUSE cache. • 0(XSK_EFUSEPS_READ_FROM_CACHE) Reads from cache • 1(XSK_EFUSEPS_READ_FROM_EFUSE) Reads from eFUSE array

Returns

- XST_SUCCESS if PUF auxiliary data is read successfully.
- Errorcode on failure

Note

Cache reload is required for obtaining updated values for reading from cache.

u32 XilSKey_Write_Puf_EfusePs_SecureBits (XilSKey_Puf_Secure * WriteSecureBits)

This function programs the eFUSE PUF secure bits.

Parameters

<i>WriteSecureBits</i>	Pointer to the XilSKey_Puf_Secure structure
------------------------	---

Returns

- XST_SUCCESS if eFUSE PUF secure bits are programmed successfully.
- Errorcode on failure.

u32 XilSKey_Read_Puf_EfusePs_SecureBits (XilSKey_Puf_Secure * SecureBitsRead, u8 ReadOption)

This function is used to read the PS eFUSE PUF secure bits from cache or from eFUSE array.

Parameters

<i>SecureBits</i>	Pointer to the XilSKey_Puf_Secure structure which holds the read eFUSE secure bits from the PUF.
<i>ReadOption</i>	Indicates whether or not to read from the actual eFUSE array or from the eFUSE cache. • 0(XSK_EFUSEPS_READ_FROM_CACHE) Reads from cache • 1(XSK_EFUSEPS_READ_FROM_EFUSE) Reads from eFUSE array

Returns

- XST_SUCCESS if reads successfully.
- Errorcode on failure.

u32 XilSKey_Puf_Debug2 (XilSKey_Puf * *InstancePtr*)

This function Outputs distance metric that may be useful for software to determine impending key generation failures.

Distance metric also is useful to obtain a more stable provisioning syndrome value.

Parameters

<i>InstancePtr</i>	Pointer to the XilSKey_Puf instance.
--------------------	--------------------------------------

Returns

- XST_SUCCESS if debug 2 mode was successful.
- ERROR if registration was unsuccessful.

u32 XilSKey_Puf_Registration (XilSKey_Puf * *InstancePtr*)

This function performs registration of PUF which generates a new KEK and associated CHash, Auxilary and PUF-syndrome data which are unique for each silicon.

Parameters

<i>InstancePtr</i>	Pointer to the XilSKey_Puf instance.
--------------------	--------------------------------------

Returns

- XST_SUCCESS if registration/re-registration was successful.
- ERROR if registration was unsuccessful

Note

With the help of generated PUF syndrome data, it will be possible to re-generate same PUF KEK.

u32 XilSKey_Puf_Regeneration (XilSKey_Puf * *InstancePtr*)

This function regenerates the PUF data so that the PUF's output can be used as the key source to the AES-GCM hardware cryptographic engine.

Parameters

<i>InstancePtr</i>	is a pointer to the XilSKey_Puf instance.
--------------------	---

Returns

- XST_SUCCESS if regeneration was successful.
- ERROR if regeneration was unsuccessful

eFUSE PL API

Overview

This chapter provides a linked summary and detailed descriptions of the eFUSE APIs of Zynq eFUSE PL and UltraScale eFUSE.

Example Usage

- The Zynq eFUSE PL and UltraScale example application should contain the `xilsky_efuse_example.c` and the `xilsky_input.h` files.
- By default, both the eFUSE PS and PL are enabled in the application. You can comment 'XSK_EFUSEPL_DRIVER' to execute only the PS.
- For UltraScale, it is mandatory to comment 'XSK_EFUSEPS_DRIVER' else the example will generate an error.
- For more details on the user configurable parameters, refer [Zynq User-Configurable PL eFUSE Parameters](#) and [UltraScale or UltraScale+ User-Configurable PL eFUSE Parameters](#).
- Requires hardware setup to program PL eFUSE of Zynq or UltraScale.

Functions

- u32 [XilSKey_EfusePI_SystemInit](#) ([XilSKey_EPI](#) *InstancePtr)
- u32 [XilSKey_EfusePI_Program](#) ([XilSKey_EPI](#) *PIInstancePtr)
- u32 [XilSKey_EfusePI_ReadStatus](#) ([XilSKey_EPI](#) *InstancePtr, u32 *StatusBits)
- u32 [XilSKey_EfusePI_ReadKey](#) ([XilSKey_EPI](#) *InstancePtr)

Function Documentation

u32 XilSKey_EfusePI_SystemInit (XilSKey_EPI * *InstancePtr*)

Initializes PL eFUSE with input data given.

Parameters

<i>InstancePtr</i>	- Input data to be written to PL eFUSE
--------------------	--

Returns

- XST_FAILURE - In case of failure
- XST_SUCCESS - In case of Success

Note

Updates the global variable ErrorCode with error code(if any).

u32 XilSKey_EfusePI_Program (XilSKey_EPI * *InstancePtr*)

Programs PL eFUSE with input data given through InstancePtr.

Parameters

<i>InstancePtr</i>	Pointer to PL eFUSE instance which holds the input data to be written to PL eFUSE.
--------------------	--

Returns

- XST_FAILURE - In case of failure
- XST_SUCCESS - In case of Success

Note

When this API is called: Initializes the timer, XADC/xsysmon and JTAG server subsystems. Returns an error in the following cases, if the reference clock frequency is not in the range or if the PL DAP ID is not identified, if the system is not in a position to write the requested PL eFUSE bits (because the bits are already written or not allowed to write) if the temperature and voltage are not within range.

u32 XilSKey_EfusePI_ReadStatus (XilSKey_EPI * *InstancePtr*, u32 * *StatusBits*)

Reads the PL efuse status bits and gets all secure and control bits.

Parameters

<i>InstancePtr</i>	Pointer to PL eFUSE instance.
<i>StatusBits</i>	Buffer to store the status bits read.

Returns

- XST_FAILURE - In case of failure
- XST_SUCCESS - In case of Success

u32 XilSKey_EfusePI_ReadKey (XilSKey_EPI * *InstancePtr*)

Reads the PL efuse keys and stores them in the corresponding arrays in instance structure.

Parameters

<i>InstancePtr</i>	Pointer to PL eFUSE instance.
--------------------	-------------------------------

Returns

- XST_FAILURE - In case of failure
- XST_SUCCESS - In case of Success

Note

This function initializes the timer, XADC and JTAG server subsystems, if not already done so. In Zynq - Reads AES key and User keys. In Ultrascale - Reads 32 bit and 128 bit User keys and RSA hash But AES key cannot be read directly it can be verified with CRC check (for that we need to update the instance with 32 bit CRC value, API updates whether provided CRC value is matched with actuals or not). To calculate the CRC of expected AES key one can use any of the following APIs [XilSKey_CrcCalculation\(\)](#) or [XilSkey_CrcCalculation_AesKey\(\)](#)

CRC Calculation API

Overview

This chapter provides a linked summary and detailed descriptions of the CRC calculation APIs. For UltraScale and Zynq UltraScale+ MPSoC devices, the programmed AES cannot be read back. The programmed AES key can only be verified by reading the CRC value of AES key.

Functions

- u32 [XilSKey_CrcCalculation](#) (u8 *Key)
- u32 [XilSKey_CrcCalculation_AesKey](#) (u8 *Key)

Function Documentation

u32 XilSKey_CrcCalculation (u8 * Key)

This function Calculates CRC value based on hexadecimal string passed.

Parameters

<i>Key</i>	Pointer to the string contains AES key in hexadecimal of length less than or equal to 64.
------------	---

Returns

- On Success returns the Crc of AES key value.
- On failure returns the error code when string length is greater than 64

Note

If the length of the string provided is less than 64, this function appends the string with zeros.

u32 XilSkey_CrcCalculation_AesKey (u8 * Key)

Calculates CRC value of the provided key.
Key should be provided in hexa buffer.

Parameters

<i>Key</i>	Pointer to an array of 32 bytes, corresponds to AES key.
------------	--

Returns

Crc of provided AES key value.

Note

To calculate CRC on the AES key in string format please use XilSkey_CrcCalculation.

User-Configurable Parameters

Overview

This chapter provides detailed descriptions of the various user configurable parameters.

Modules

- [Zynq User-Configurable PS eFUSE Parameters](#)
 - [Zynq User-Configurable PL eFUSE Parameters](#)
 - [Zynq User-Configurable PL BBRAM Parameters](#)
 - [UltraScale or UltraScale+ User-Configurable BBRAM PL Parameters](#)
 - [UltraScale or UltraScale+ User-Configurable PL eFUSE Parameters](#)
 - [Zynq UltraScale+ MPSoC User-Configurable PS eFUSE Parameters](#)
 - [Zynq UltraScale+ MPSoC User-Configurable PS BBRAM Parameters](#)
 - [Zynq UltraScale+ MPSoC User-Configurable PS PUF Parameters](#)
-

Zynq User-Configurable PS eFUSE Parameters

Define the `XSK_EFUSEPS_DRIVER` macro to use the PS eFUSE.

After defining the macro, provide the inputs defined with `XSK_EFUSEPS_DRIVER` to burn the bits in PS eFUSE. If the bit is to be burned, define the macro as `TRUE`; otherwise define the macro as `FALSE`. For details, refer the following table.

Macro Name	Description
XSK_EFUSEPS_ENABLE_WRITE_PROTECT	Default = FALSE. TRUE to burn the write-protect bits in eFUSE array. Write protect has two bits. When either of the bits is burned, it is considered write-protected. So, while burning the write-protected bits, even if one bit is blown, write API returns success. As previously mentioned, POR reset is required after burning for write protection of the eFUSE bits to go into effect. It is recommended to do the POR reset after write protection. Also note that, after write-protect bits are burned, no more eFUSE writes are possible. If the write-protect macro is TRUE with other macros, write protect is burned in the last iteration, after burning all the defined values, so that for any error while burning other macros will not effect the total eFUSE array. FALSE does not modify the write-protect bits.
XSK_EFUSEPS_ENABLE_RSA_AUTH	Default = FALSE. Use TRUE to burn the RSA enable bit in the PS eFUSE array. After enabling the bit, every successive boot must be RSA-enabled apart from JTAG. Before burning (blowing) this bit, make sure that eFUSE array has the valid PPK hash. If the PPK hash burning is enabled, only after writing the hash successfully, RSA enable bit will be blown. For the RSA enable bit to take effect, POR reset is required. FALSE does not modify the RSA enable bit.
XSK_EFUSEPS_ENABLE_ROM_128K_CRC	Default = FALSE. TRUE burns the ROM 128K CRC bit. In every successive boot, BootROM calculates 128k CRC. FALSE does not modify the ROM CRC 128K bit.
XSK_EFUSEPS_ENABLE_RSA_KEY_HASH	Default = FALSE. TRUE burns (blows) the eFUSE hash, that is given in XSK_EFUSEPS_RSA_KEY_HASH_VALUE when write API is used. TRUE reads the eFUSE hash when the read API is used and is read into structure. FALSE ignores the provided value.

Macro Name	Description
XSK_EFUSEPS_RSA_KEY_HASH_VALUE	Default = 00000000000000000000000000000000 00000000000000000000000000000000 The specified value is converted to a hexadecimal buffer and written into the PS eFUSE array when the write API is used. This value should be the Primary Public Key (PPK) hash provided in string format. The buffer must be 64 characters long: valid characters are 0-9, a-f, and A-F. Any other character is considered an invalid string and will not burn RSA hash. When the Xilsky_EfusePs_Write() API is used, the RSA hash is written, and the XSK_EFUSEPS_ENABLE_RSA_KEY_HASH must have a value of TRUE.
XSK_EFUSEPS_DISABLE_DFT_JTAG	Default = FALSE TRUE disables DFT JTAG permanently. FALSE will not modify the eFuse PS DFT JTAG disable bit.
XSK_EFUSEPS_DISABLE_DFT_MODE	Default = FALSE TRUE disables DFT mode permanently. FALSE will not modify the eFuse PS DFT mode disable bit.

Zynq User-Configurable PL eFUSE Parameters

Overview

Define the XSK_EFUSEPL_DRIVER macro to use the PL eFUSE.

After defining the macro, provide the inputs defined with XSK_EFUSEPL_DRIVER to burn the bits in PL eFUSE bits. If the bit is to be burned, define the macro as TRUE; otherwise define the macro as FALSE. The table below lists the user-configurable PL eFUSE parameters for Zynq® devices.

Macro Name	Description
XSK_EFUSEPL_FORCE_PCYCLE_RECONFIG	Default = FALSE If the value is set to TRUE, then the part has to be power-cycled to be reconfigured. FALSE does not set the eFUSE control bit.
XSK_EFUSEPL_DISABLE_KEY_WRITE	Default = FALSE TRUE disables the eFUSE write to FUSE_AES and FUSE_USER blocks. FALSE does not affect the EFUSE bit.

Macro Name	Description
XSK_EFUSEPL_DISABLE_AES_KEY_READ	Default = FALSE TRUE disables the write to FUSE_AES and FUSE_USER key and disables the read of FUSE_AES. FALSE does not affect the eFUSE bit.
XSK_EFUSEPL_DISABLE_USER_KEY_READ	Default = FALSE. TRUE disables the write to FUSE_AES and FUSE_USER key and disables the read of FUSE_USER. FALSE does not affect the eFUSE bit.
XSK_EFUSEPL_DISABLE_FUSE_CNTRL_WRITE	Default = FALSE. TRUE disables the eFUSE write to FUSE_CTRL block. FALSE does not affect the eFUSE bit.
XSK_EFUSEPL_FORCE_USE_AES_ONLY	Default = FALSE. TRUE forces the use of secure boot with eFUSE AES key only. FALSE does not affect the eFUSE bit.
XSK_EFUSEPL_DISABLE_JTAG_CHAIN	Default = FALSE. TRUE permanently disables the Zynq ARM DAP and PL TAP. FALSE does not affect the eFUSE bit.
XSK_EFUSEPL_BBRAM_KEY_DISABLE	Default = FALSE. TRUE forces the eFUSE key to be used if booting Secure Image. FALSE does not affect the eFUSE bit.

Modules

- [MIO Pins for Zynq PL eFUSE JTAG Operations](#)
- [MUX Selection Pin for Zynq PL eFUSE JTAG Operations](#)
- [MUX Parameter for Zynq PL eFUSE JTAG Operations](#)
- [AES and User Key Parameters](#)

MIO Pins for Zynq PL eFUSE JTAG Operations

The table below lists the MIO pins for Zynq PL eFUSE JTAG operations. You can change the listed pins at your discretion.

Note

The pin numbers listed in the table below are examples. You must assign appropriate pin numbers as per your hardware design.

Pin Name	Pin Number
XSK_EFUSEPL_MIO_JTAG_TDI	(17)
XSK_EFUSEPL_MIO_JTAG_TDO	(21)
XSK_EFUSEPL_MIO_JTAG_TCK	(19)
XSK_EFUSEPL_MIO_JTAG_TMS	(20)

MUX Selection Pin for Zynq PL eFUSE JTAG Operations

The table below lists the MUX selection pin.

Pin Name	Pin Number	Description
XSK_EFUSEPL_MIO_JTAG_MUX_SELECT	(11)	This pin toggles between the external JTAG or MIO driving JTAG operations.

MUX Parameter for Zynq PL eFUSE JTAG Operations

The table below lists the MUX parameter.

Parameter Name	Description
XSK_EFUSEPL_MIO_MUX_SEL_DEFAULT_VAL	Default = LOW. LOW writes zero on the MUX select line before PL_eFUSE writing. HIGH writes one on the MUX select line before PL_eFUSE writing.

AES and User Key Parameters

The table below lists the AES and user key parameters.

Parameter Name	Description
XSK_EFUSEPL_PROGRAM_AES_AND_USER_LOW_KEY	Default = FALSE. TRUE burns the AES and User Low hash key, which are given in the XSK_EFUSEPL_AES_KEY and the XSK_EFUSEPL_USER_LOW_KEY respectively. FALSE ignores the provided values. You cannot write the AES Key and the User Low Key separately.
XSK_EFUSEPL_PROGRAM_USER_HIGH_KEY	Default =FALSE. TRUE burns the User High hash key, given in XSK_EFUSEPL_PROGRAM_USER_HIGH_KEY. FALSE ignores the provided values.
XSK_EFUSEPL_AES_KEY	Default = 00000000000000000000000000000000 00000000000000000000000000000000 This value converted to hex buffer and written into the PL eFUSE array when write API is used. This value should be the AES Key, given in string format. It must be 64 characters long. Valid characters are 0-9, a-f, A-F. Any other character is considered an invalid string and will not burn AES Key. To write AES Key, XSK_EFUSEPL_PROGRAM_AES_AND_USER_LOW_KEY must have a value of TRUE.
XSK_EFUSEPL_USER_LOW_KEY	Default = 00 This value is converted to a hexadecimal buffer and written into the PL eFUSE array when the write API is used. This value is the User Low Key given in string format. It must be two characters long; valid characters are 0-9,a-f, and A-F. Any other character is considered as an invalid string and will not burn the User Low Key. To write the User Low Key, XSK_EFUSEPL_PROGRAM_AES_AND_USER_LOW_KEY must have a value of TRUE.

Parameter Name	Description
XSK_EFUSEPL_USER_HIGH_KEY	Default = 000000 The default value is converted to a hexadecimal buffer and written into the PL eFUSE array when the write API is used. This value is the User High Key given in string format. The buffer must be six characters long: valid characters are 0-9, a-f, A-F. Any other character is considered to be an invalid string and does not burn User High Key. To write the User High Key, the XSK_EFUSEPL_PROGRAM_USER_HIGH_KEY must have a value of TRUE.

Zynq User-Configurable PL BBRAM Parameters

Overview

The table below lists the MIO pins for Zynq PL BBRAM JTAG operations.

Note

The pin numbers listed in the table below are examples. You must assign appropriate pin numbers as per your hardware design.

Pin Name	Pin Number
XSK_BBRAM_MIO_JTAG_TDI	(17)
XSK_BBRAM_MIO_JTAG_TDO	(21)
XSK_BBRAM_MIO_JTAG_TCK	(19)
XSK_BBRAM_MIO_JTAG_TMS	(20)

The table below lists the MUX selection pin for Zynq BBRAM PL JTAG operations.

Pin Name	Pin Number
XSK_BBRAM_MIO_JTAG_MUX_SELECT	(11)

Modules

- [MUX Parameter for Zynq BBRAM PL JTAG Operations](#)
- [AES and User Key Parameters](#)

MUX Parameter for Zynq BBRAM PL JTAG Operations

The table below lists the MUX parameter for Zynq BBRAM PL JTAG operations.

Parameter Name	Description
XSK_BBRAM_MIO_MUX_SEL_DEFAULT_VAL	Default = LOW. LOW writes zero on the MUX select line before PL_eFUSE writing. HIGH writes one on the MUX select line before PL_eFUSE writing.

AES and User Key Parameters

The table below lists the AES and user key parameters.

Parameter Name	Description
XSK_BBRAM_AES_KEY	Default = XX. AES key (in HEX) that must be programmed into BBRAM.
XSK_BBRAM_AES_KEY_SIZE_IN_BITS	Default = 256. Size of AES key. Must be 256 bits.

UltraScale or UltraScale+ User-Configurable BBRAM PL Parameters

Overview

Following parameters need to be configured.
Based on your inputs, BBRAM is programmed with the provided AES key.

Modules

- [AES Keys and Related Parameters](#)
- [DPA Protection for BBRAM key](#)
- [GPIO Device Used for Connecting PL Master JTAG Signals](#)
- [GPIO Pins Used for PL Master JTAG Signals](#)
- [GPIO Channels](#)

AES Keys and Related Parameters

The following table shows AES key related parameters.

Parameter Name	Description
XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR1_MONO	Default = FALSE By default, XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR1 is FALSE. BBRAM is programmed with a non-obfuscated key provided in XSK_BBRAM_AES_KEY_SLR1 and DPA protection can be either in enabled/disabled state. TRUE programs the BBRAM with key provided in XSK_BBRAM_OBFUSCATED_KEY_SLR1 and DPA protection cannot be enabled.
XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR2	Default = FALSE By default, XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR2 is FALSE. BBRAM is programmed with a non-obfuscated key provided in XSK_BBRAM_AES_KEY_SLR2 and DPA protection can be either in enabled/disabled state. TRUE programs the BBRAM with key provided in XSK_BBRAM_OBFUSCATED_KEY_SLR2 and DPA protection cannot be enabled.
XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR3	Default = FALSE By default, XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR3 is FALSE. BBRAM is programmed with a non-obfuscated key provided in XSK_BBRAM_AES_KEY_SLR3 and DPA protection can be either in enabled/disabled state. TRUE programs the BBRAM with key provided in XSK_BBRAM_OBFUSCATED_KEY_SLR3 and DPA protection cannot be enabled.
XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR4	Default = FALSE By default, XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR4 is FALSE. BBRAM is programmed with a non-obfuscated key provided in XSK_BBRAM_AES_KEY_SLR4 and DPA protection can be either in enabled/disabled state. TRUE programs the BBRAM with key provided in XSK_BBRAM_OBFUSCATED_KEY_SLR4 and DPA protection cannot be enabled.

Parameter Name	Description
XSK_BBRAM_OBFUSCATED_KEY_SLR1	Default = b1c276899d71fb4cdd4a0a7905ea46c2e11f9574d09c7ea23b70b67de713ccd1 The value mentioned in this will be converted to hex buffer and the key is programmed into BBRAM, when program API is called. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not program BBRAM. Note For writing the OBFUSCATED Key, XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR1 should have TRUE value.
XSK_BBRAM_OBFUSCATED_KEY_SLR2	Default = b1c276899d71fb4cdd4a0a7905ea46c2e11f9574d09c7ea23b70b67de713ccd1 The value mentioned in this will be converted to hex buffer and the key is programmed into BBRAM, when program API is called. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not program BBRAM. Note For writing the OBFUSCATED Key, XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR2 should have TRUE value.
XSK_BBRAM_OBFUSCATED_KEY_SLR3	Default = b1c276899d71fb4cdd4a0a7905ea46c2e11f9574d09c7ea23b70b67de713ccd1 The value mentioned in this will be converted to hex buffer and the key is programmed into BBRAM, when program API is called. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not program BBRAM. Note For writing the OBFUSCATED Key, XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR3 should have TRUE value.

Parameter Name	Description
XSK_BBRAM_OBFUSCATED_KEY_SLR4	Default = b1c276899d71fb4cdd4a0a7905ea46c2e11f9574d09c7ea23b70b67de713ccd1 The value mentioned in this will be converted to hex buffer and the key is programmed into BBRAM, when program API is called. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not program BBRAM. Note For writing the OBFUSCATED Key, XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR4 should have TRUE value.
XSK_BBRAM_PGM_AES_KEY_SLR1_OR_MONO	Default = FALSE TRUE will program BBRAM with AES key provided in XSK_BBRAM_AES_KEY_SLR1
XSK_BBRAM_PGM_AES_KEY_SLR2	Default = FALSE TRUE will program BBRAM with AES key provided in XSK_BBRAM_AES_KEY_SLR2
XSK_BBRAM_PGM_AES_KEY_SLR3	Default = FALSE TRUE will program BBRAM with AES key provided in XSK_BBRAM_AES_KEY_SLR3
XSK_BBRAM_PGM_AES_KEY_SLR4	Default = FALSE TRUE will program BBRAM with AES key provided in XSK_BBRAM_AES_KEY_SLR4
XSK_BBRAM_AES_KEY_SLR1	Default = 0000000000000000524156a63950bcedaf eadcdeabaadee34216615aaaabbbaa The value mentioned in this will be converted to hex buffer and the key is programmed into BBRAM,when program API is called. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not program BBRAM. Note For writing AES key, XSK_BBRAM_PGM_AES_KEY_SLR1_OR_MONO should have TRUE value , and XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR1_MONO should have FALSE value.

Parameter Name	Description
XSK_BBRAM_AES_KEY_SLR2	Default = 0000000000000000524156a63950bcdaf eadcdeabaadee34216615aaaabbbaaa The value mentioned in this will be converted to hex buffer and the key is programmed into BBRAM,when program API is called. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not program BBRAM. Note For writing AES key, XSK_BBRAM_PGM_AES_KEY_SLR2 should have TRUE value , and XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR2 should have FALSE value
XSK_BBRAM_AES_KEY_SLR3	Default = 0000000000000000524156a63950bcdaf eadcdeabaadee34216615aaaabbbaaa The value mentioned in this will be converted to hex buffer and the key is programmed into BBRAM, when program API is called. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not program BBRAM. Note For writing AES key, XSK_BBRAM_PGM_AES_KEY_SLR3 should have TRUE value , and XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR3 should have FALSE value

Parameter Name	Description
XSK_BBRAM_AES_KEY_SLR4	Default = 0000000000000000524156a63950bcedaf eadcdeabaadee34216615aaaabbbaaa The value mentioned in this will be converted to hex buffer and the key is programmed into BBRAM, when program API is called. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not program BBRAM. Note For writing AES key, XSK_BBRAM_PGM_AES_KEY_SLR4 should have TRUE value , and XSK_BBRAM_PGM_OBFUSCATED_KEY_SLR4 should have FALSE value
XSK_BBRAM_AES_KEY_SIZE_IN_BITS	Default= 256 Size of AES key must be 256 bits.

DPA Protection for BBRAM key

The following table shows DPA protection configurable parameter.

Parameter Name	Description
XSK_BBRAM_DPA_PROTECT_ENABLE	Default = FALSE By default, the DPA protection will be in disabled state. TRUE will enable DPA protection with provided DPA count and configuration in XSK_BBRAM_DPA_COUNT and XSK_BBRAM_DPA_MODE respectively. DPA protection cannot be enabled if BBRAM is been programmed with an obfuscated key.
XSK_BBRAM_DPA_COUNT	Default = 0 This input is valid only when DPA protection is enabled. Valid range of values are 1 - 255 when DPA protection is enabled else 0.

Parameter Name	Description
XSK_BBRAM_DPA_MODE	Default = XSK_BBRAM_INVALID_CONFIGURATIONS When DPA protection is enabled it can be XSK_BBRAM_INVALID_CONFIGURATIONS or XSK_BBRAM_ALL_CONFIGURATIONS If DPA protection is disabled this input provided over here is ignored.

GPIO Device Used for Connecting PL Master JTAG Signals

In hardware design MASTER JTAG can be connected to any one of the available GPIO devices, based on the design the following parameter should be provided with corresponding device ID of selected GPIO device.

Master JTAG Signal	Description
XSK_BBRAM_AXI_GPIO_DEVICE_ID	Default = XPAR_AXI_GPIO_0_DEVICE_ID This is for providing exact GPIO device ID, based on the design configuration this parameter can be modified to provide GPIO device ID which is used for connecting master jtag pins.

GPIO Pins Used for PL Master JTAG Signals

In Ultrascale the following GPIO pins are used for connecting MASTER_JTAG pins to access BBRAM. These can be changed depending on your hardware. The table below shows the GPIO pins used for PL MASTER JTAG signals.

Master JTAG Signal	Default PIN Number
XSK_BBRAM_AXI_GPIO_JTAG_TDO	0
XSK_BBRAM_AXI_GPIO_JTAG_TDI	0
XSK_BBRAM_AXI_GPIO_JTAG_TMS	1
XSK_BBRAM_AXI_GPIO_JTAG_TCK	2

GPIO Channels

The following table shows GPIO channel number.

Parameter	Default Channel Number	Master JTAG Signal Connected
XSK_BBRAM_GPIO_INPUT_CH	2	TDO

Parameter	Default Channel Number	Master JTAG Signal Connected
XSK_BBRAM_GPIO_OUTPUT_CH	1	TDI, TMS, TCK

Note

All inputs and outputs of GPIO should be configured in single channel. For example, XSK_BBRAM_GPIO_INPUT_CH = XSK_BBRAM_GPIO_OUTPUT_CH = 1 or 2. Among (TDI, TCK, TMS) Outputs of GPIO cannot be connected to different GPIO channels all the 3 signals should be in same channel. TDO can be a other channel of (TDI, TCK, TMS) or the same. DPA protection can be enabled only when programming non-obfuscated key.

UltraScale or UltraScale+ User-Configurable PL eFUSE Parameters

Overview

The table below lists the user-configurable PL eFUSE parameters for UltraScale™ devices.

Macro Name	Description
XSK_EFUSEPL_DISABLE_AES_KEY_READ	Default = FALSE TRUE will permanently disable the write to FUSE_AES and check CRC for AES key by programming control bit of FUSE. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPL_DISABLE_USER_KEY_READ	Default = FALSE TRUE will permanently disable the write to 32 bit FUSE_USER and read of FUSE_USER key by programming control bit of FUSE. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPL_DISABLE_SECURE_READ	Default = FALSE TRUE will permanently disable the write to FUSE_Secure block and reading of secure block by programming control bit of FUSE. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPL_DISABLE_FUSE_CNTRL_WRITE	Default = FALSE. TRUE will permanently disable the write to FUSE_CNTRL block by programming control bit of FUSE. FALSE will not modify this control bit of eFuse.

Macro Name	Description
XSK_EFUSEPL_DISABLE_RSA_KEY_READ	Default = FALSE. TRUE will permanently disable the write to FUSE_RSA block and reading of FUSE_RSA Hash by programming control bit of FUSE. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPL_DISABLE_KEY_WRITE	Default = FALSE. TRUE will permanently disable the write to FUSE_AES block by programming control bit of FUSE. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPL_DISABLE_USER_KEY_WRITE	Default = FALSE. TRUE will permanently disable the write to FUSE_USER block by programming control bit of FUSE. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPL_DISABLE_SECURE_WRITE	Default = FALSE. TRUE will permanently disable the write to FUSE_SECURE block by programming control bit of FUSE. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPL_DISABLE_RSA_HASH_WRITE	Default = FALSE. TRUE will permanently disable the write to FUSE_RSA authentication key by programming control bit of FUSE. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPL_DISABLE_128BIT_USER_KEY_WRITE	Default = FALSE. TRUE will permanently disable the write to 128 bit FUSE_USER by programming control bit of FUSE. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPL_ALLOW_ENCRYPTED_ONLY	Default = FALSE. TRUE will permanently allow encrypted bitstream only. FALSE will not modify this Secure bit of eFuse.
XSK_EFUSEPL_FORCE_USE_FUSE_AES_ONLY	Default = FALSE. TRUE then allows only FUSE's AES key as source of encryption FALSE then allows FPGA to configure an unencrypted bitstream or bitstream encrypted using key stored BBRAM or eFuse.
XSK_EFUSEPL_ENABLE_RSA_AUTH	Default = FALSE. TRUE will enable RSA authentication of bitstream FALSE will not modify this secure bit of eFuse.

Macro Name	Description
XSK_EFUSEPL_DISABLE_JTAG_CHAIN	Default = FALSE. TRUE will disable JTAG permanently. FALSE will not modify this secure bit of eFuse.
XSK_EFUSEPL_DISABLE_TEST_ACCESS	Default = FALSE. TRUE will disables Xilinx test access. FALSE will not modify this secure bit of eFuse.
XSK_EFUSEPL_DISABLE_AES_DECRYPTOR	Default = FALSE. TRUE will disables decoder completely. FALSE will not modify this secure bit of eFuse.
XSK_EFUSEPL_ENABLE_OBFUSCATION_EFUSEAES	Default = FALSE. TRUE will enable obfuscation feature for eFUSE AES key.

Modules

- [GPIO Device Used for Connecting PL Master JTAG Signals](#)
- [GPIO Pins Used for PL Master JTAG and HWM Signals](#)
- [GPIO Channels](#)
- [SLR Selection to Program eFUSE on MONO/SSIT Devices](#)
- [eFUSE PL Read Parameters](#)
- [AES Keys and Related Parameters](#)
- [USER Keys \(32-bit\) and Related Parameters](#)
- [RSA Hash and Related Parameters](#)
- [USER Keys \(128-bit\) and Related Parameters](#)
- [AES key CRC verification](#)

GPIO Device Used for Connecting PL Master JTAG Signals

In hardware design MASTER JTAG can be connected to any one of the available GPIO devices, based on the design the following parameter should be provided with corresponding device ID of selected GPIO device.

Master JTAG Signal	Description
XSK_EFUSEPL_AXI_GPIO_DEVICE_ID	Default = XPAR_AXI_GPIO_0_DEVICE_ID This is for providing exact GPIO device ID, based on the design configuration this parameter can be modified to provide GPIO device ID which is used for connecting master jtag pins.

GPIO Pins Used for PL Master JTAG and HWM Signals

In Ultrascale the following GPIO pins are used for connecting MASTER_JTAG pins to access eFUSE.

These can be changed depending on your hardware. The table below shows the GPIO pins used for PL MASTER JTAG signals.

Master JTAG Signal	Default PIN Number
XSK_EFUSEPL_AXI_GPIO_JTAG_TDO	0
XSK_EFUSEPL_AXI_GPIO_HWM_READY	0
XSK_EFUSEPL_AXI_GPIO_HWM_END	1
XSK_EFUSEPL_AXI_GPIO_JTAG_TDI	2
XSK_EFUSEPL_AXI_GPIO_JTAG_TMS	1
XSK_EFUSEPL_AXI_GPIO_JTAG_TCK	2
XSK_EFUSEPL_AXI_GPIO_HWM_START	3

GPIO Channels

The following table shows GPIO channel number.

Parameter	Default Channel Number	Master JTAG Signal Connected
XSK_EFUSEPL_GPIO_INPUT_CH	2	TDO
XSK_EFUSEPL_GPIO_OUTPUT_CH	1	TDI, TMS, TCK

Note

All inputs and outputs of GPIO should be configured in single channel. For example, XSK_EFUSEPL_GPIO_INPUT_CH = XSK_EFUSEPL_GPIO_OUTPUT_CH = 1 or 2. Among (TDI, TCK, TMS) Outputs of GPIO cannot be connected to different GPIO channels all the 3 signals should be in same channel. TDO can be a other channel of (TDI, TCK, TMS) or the same.

SLR Selection to Program eFUSE on MONO/SSIT Devices

The following table shows parameters for programming different SLRs.

Parameter Name	Description
XSK_EFUSEPL_PGM_SLR1	Default = FALSE TRUE will enable programming SLR1/MONO eFUSE. FALSE will disable programming.
XSK_EFUSEPL_PGM_SLR2	Default = FALSE TRUE will enable programming SLR2 eFUSE. FALSE will disable programming.

Parameter Name	Description
XSK_EFUSEPL_PGM_SLR3	Default = FALSE TRUE will enable programming SLR3 eFUSE. FALSE will disable programming.
XSK_EFUSEPL_PGM_SLR4	Default = FALSE TRUE will enable programming SLR4 eFUSE. FALSE will disable programming.

eFUSE PL Read Parameters

The following table shows parameters related to read USER 32/128bit keys and RSA hash.

Note

For only reading keys it is not required to enable XSK_EFUSEPL_PGM_SLR1, XSK_EFUSEPL_PGM_SLR2, XSK_EFUSEPL_PGM_SLR3, XSK_EFUSEPL_PGM_SLR4 macros, they can be in FALSE state.

By enabling any of the below parameters, by default will read corresponding hash/key associated with all the available SLRs. For example, if XSK_EFUSEPL_READ_USER_KEY is TRUE, USER key for all the available SLRs will be read.

Parameter Name	Description
XSK_EFUSEPL_READ_USER_KEY	Default = FALSE TRUE will read 32 bit FUSE_USER from eFUSE of all available SLRs and each time updates in XilSKey_EPI instance parameter UserKeyReadback, which will be displayed on UART by example before reading next SLR. FALSE 32-bit FUSE_USER key read will not be performed.
XSK_EFUSEPL_READ_RSA_KEY_HASH	Default = FALSE TRUE will read FUSE_USER from eFUSE of all available SLRs and each time updates in XilSKey_EPI instance parameter RSAHashReadback, which will be displayed on UART by example before reading next SLR. FALSE FUSE_RSA_HASH read will not be performed.
XSK_EFUSEPL_READ_USER_KEY128_BIT	Default = FALSE TRUE will read 128 bit USER key eFUSE of all available SLRs and each time updates in XilSKey_EPI instance parameter User128BitReadBack, which will be displayed on UART by example before reading next SLR. FALSE 128 bit USER key read will not be performed.

AES Keys and Related Parameters

Note

For programming AES key for MONO/SSIT device, the corresponding SLR should be selected and AES key programming should be enabled.

Example 1 Enable the following parameters if you want to program AES key for SLR2:

1. Enable programming for SLR:
 - XSK_EFUSEPL_PGM_SLR2 should have the TRUE value.
2. Enable AES key programming:
 - XSK_EFUSEPL_PROGRAM_AES_KEY should have the TRUE value.
3. Provide key to be programmed on SLR:
 - XSK_EFUSEPL_AES_KEY_SLR2 should have key to be programmed in the string format.

Example 2 Enable the following parameters if you want to program AES key on SLR_MONO and SLR3:

1. Enable programming for SLR:
 - XSK_EFUSEPL_PGM_SLR1 should have the TRUE value.
 - XSK_EFUSEPL_PGM_SLR3 should have the TRUE value.
2. Enable AES key programming:
 - XSK_EFUSEPL_PROGRAM_AES_KEY should have the TRUE value.
3. Provide key to be programmed on SLR:
 - XSK_EFUSEPL_AES_KEY should have key to be programmed in the string format.
 - XSK_EFUSEPL_AES_KEY_SLR3 should have key to be programmed in the string format.

The following table shows AES key and related parameters to be taken care while programming AES key.

Parameter Name	Description
XSK_EFUSEPL_PROGRAM_AES_KEY	Default = FALSE TRUE will burn the AES key provided in: XSK_EFUSEPL_AES_KEY if XSK_EFUSEPL_PGM_SLR1 is TRUE XSK_EFUSEPL_AES_KEY_SLR2 if XSK_EFUSEPL_PGM_SLR2 is TRUE XSK_EFUSEPL_AES_KEY_SLR3 if XSK_EFUSEPL_PGM_SLR3 is TRUE XSK_EFUSEPL_AES_KEY_SLR4 if XSK_EFUSEPL_PGM_SLR4 is TRUE FALSE will ignore the values given.

Parameter Name	Description
XSK_EFUSEPL_AES_KEY	Default = 00000000000000000000000000000000 00000000000000000000000000000000 The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR1/MONO when write API is used. This value should be the AES key given in string format. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn AES Key. Note For writing the AES Key, make sure XSK_EFUSEPL_PROGRAM_AES_KEY and XSK_EFUSEPL_PGM_SLR1 are enabled with the TRUE value.
XSK_EFUSEPL_AES_KEY_SLR2	Default = 00000000000000000000000000000000 00000000000000000000000000000000 The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR2 when write API is used. This value should be the AES key given in string format. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn AES Key. Note For writing the AES Key, make sure XSK_EFUSEPL_PROGRAM_AES_KEY and XSK_EFUSEPL_PGM_SLR2 are enabled with the TRUE value.

Parameter Name	Description
XSK_EFUSEPL_AES_KEY_SLR3	Default = 00000000000000000000000000000000 00000000000000000000000000000000 The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR3 when write API is used. This value should be the AES key given in string format. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn AES Key. Note For writing the AES Key, make sure XSK_EFUSEPL_PROGRAM_AES_KEY and XSK_EFUSEPL_PGM_SLR3 are enabled with TRUE value.
XSK_EFUSEPL_AES_KEY_SLR4	Default = 00000000000000000000000000000000 00000000000000000000000000000000 The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR4 when write API is used. This value should be the AES key given in string format. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn AES Key. Note For writing the AES Key, make sure XSK_EFUSEPL_PROGRAM_AES_KEY and XSK_EFUSEPL_PGM_SLR4 are enabled with TRUE value.

USER Keys (32-bit) and Related Parameters

Note

For programming USER key for MONO/SSIT device, the corresponding SLR should be selected and USER key programming should be enabled.

Example 1 Enable the following parameters if you want to program USER key for SLR2:

1. Enable programming for SLR:
 - XSK_EFUSEPL_PGM_SLR2 should have the TRUE value.

2. Enable USER key programming:

- XSK_EFUSEPL_PROGRAM_USER_KEY should have the TRUE value.

3. Provide key to be programmed on SLR:

- XSK_EFUSEPL_AES_USER_SLR2 should have key to be programmed in the string format.

Example 2 Enable the following parameters if you want to program USER key on SLR_MONO and SLR3:

1. Enable programming for SLR:

- XSK_EFUSEPL_PGM_SLR1 should have the TRUE value.
- XSK_EFUSEPL_PGM_SLR3 should have the TRUE value.

2. Enable USER key programming:

- XSK_EFUSEPL_PROGRAM_USER_KEY should have the TRUE value.

3. Provide key to be programmed on SLR:

- XSK_EFUSEPL_USER_KEY should have key to be programmed in the string format.
- XSK_EFUSEPL_USER_KEY_SLR3 should have key to be programmed in the string format.

The following table shows USER key and related parameters to be taken care while programming USER key.

Parameter Name	Description
XSK_EFUSEPL_PROGRAM_USER_KEY	Default = FALSE TRUE will burn 32 bit User key given in XSK_EFUSEPL_USER_KEY if XSK_EFUSEPL_PGM_SLR1 is TRUE XSK_EFUSEPL_USER_KEY_SLR2 if XSK_EFUSEPL_PGM_SLR2 is TRUE XSK_EFUSEPL_USER_KEY_SLR3 if XSK_EFUSEPL_PGM_SLR3 is TRUE XSK_EFUSEPL_USER_KEY_SLR4 if XSK_EFUSEPL_PGM_SLR4 is TRUE FALSE will ignore the values given.

Parameter Name	Description
XSK_EFUSEPL_USER_KEY	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR1/MONO when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY and XSK_EFUSEPL_PGM_SLR1 are enabled with TRUE value.
XSK_EFUSEPL_USER_KEY_SLR2	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array when the write API is used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY and XSK_EFUSEPL_PGM_SLR2 are enabled with TRUE value.
XSK_EFUSEPL_USER_KEY_SLR3	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY and XSK_EFUSEPL_PGM_SLR3 are enabled with TRUE value.

Parameter Name	Description
XSK_EFUSEPL_USER_KEY_SLR4	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, XSK_EFUSEPL_PROGRAM_USER_KEY and XSK_EFUSEPL_PGM_SLR4 are enabled with TRUE value.

RSA Hash and Related Parameters

Note

For programming RSA hash for MONO/SSIT device, the corresponding SLR should be selected and RSA hash programming should be enabled.

Example 1 Enable the following parameters if you want to program RSA hash for SLR2:

1. Enable programming for SLR:
 - XSK_EFUSEPL_PGM_SLR2 should have the TRUE value.
2. Enable RSA hash programming:
 - XSK_EFUSEPL_PROGRAM_RSA_KEY_HASH should have the TRUE value.
3. Provide hash to be programmed on SLR:
 - XSK_EFUSEPL_RSA_KEY_HASH_VALUE_SLR2 should have hash to be programmed in the string format.

Example 2 Enable the following parameters if you want to program RSA hash on SLR_MONO and SLR3:

1. Enable programming for SLR:
 - XSK_EFUSEPL_PGM_SLR1 should have the TRUE value.
 - XSK_EFUSEPL_PGM_SLR3 should have the TRUE value.
2. Enable RSA hash programming:
 - XSK_EFUSEPL_PROGRAM_RSA_KEY_HASH should have the TRUE value.

3. Provide hash to be programmed on SLR:

- XSK_EFUSEPL_RSA_KEY_HASH_VALUE should have hash to be programmed in the string format.
- XSK_EFUSEPL_RSA_KEY_HASH_VALUE_SLR3 should have hash to be programmed in the string format.

The following table shows RSA hash and related parameters to be taken care while programming RSA hash.

[illegible]

Parameter Name	Description
XSK_EFUSEPL_RSA_KEY_HASH_VALUE_SLR2	Default = 00000000000000000000000000000000 00000000000000000000000000000000 000000000000000000000000 The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR2 when write API used. This value should be the RSA Key hash given in string format. It should be 96 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn RSA hash value. Note For writing the RSA hash, make sure XSK_EFUSEPL_PROGRAM_RSA_KEY_HASH and XSK_EFUSEPL_PGM_SLR2 are enabled with TRUE value.
XSK_EFUSEPL_RSA_KEY_HASH_VALUE_SLR3	Default = 00000000000000000000000000000000 00000000000000000000000000000000 000000000000000000000000 The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR3 when write API used. This value should be the RSA Key hash given in string format. It should be 96 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn RSA hash value. Note For writing the RSA hash, make sure XSK_EFUSEPL_PROGRAM_RSA_KEY_HASH and XSK_EFUSEPL_PGM_SLR3 are enabled with TRUE value.

Parameter Name	Description
XSK_EFUSEPL_RSA_KEY_HASH_VALUE_SLR4	Default = 00000000000000000000000000000000 00000000000000000000000000000000 000000000000000000000000 The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR4 when write API used. This value should be the RSA Key hash given in string format. It should be 96 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn RSA hash value. Note For writing the RSA hash, make sure XSK_EFUSEPL_PROGRAM_RSA_KEY_HASH and XSK_EFUSEPL_PGM_SLR4 are enabled with TRUE value.

USER Keys (128-bit) and Related Parameters

Note

For programming USER key 128 bit for MONO/SSIT device, the corresponding SLR and programming for USER key 128 bit should be enabled.

Example 1 Enable the following parameters if you want to program USER key 128-bit for SLR2:

1. Enable programming for SLR:
 - XSK_EFUSEPL_PGM_SLR2 should have the TRUE value.
2. Enable USER key programming:
 - XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT should have the TRUE value. As:
 - XSK_EFUSEPL_USER_KEY_128BIT_0_SLR2 holds 31:0 bits,
 - XSK_EFUSEPL_USER_KEY_128BIT_1_SLR2 holds 63:32 bits,
 - XSK_EFUSEPL_USER_KEY_128BIT_2_SLR2 holds 95:64 bits and
 - XSK_EFUSEPL_USER_KEY_128BIT_3_SLR2 holds 127:96 bits of whole 128 bit User key.
3. Provide key to be programmed on SLR:
 - XSK_EFUSEPL_USER_KEY_128BIT_0_SLR2, XSK_EFUSEPL_USER_KEY_128BIT_1_SLR2, XSK_EFUSEPL_USER_KEY_128BIT_2_SLR2, XSK_EFUSEPL_USER_KEY_128BIT_3_SLR2 should have value to be programmed in the string format.

The following table shows USER key 128 bit and related parameters.

Parameter Name	Description
XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT	Default = FALSE TRUE will burn 128 bit User key given in: XSK_EFUSEPL_USER_KEY_128BIT_0, XSK_EFUSEPL_USER_KEY_128BIT_1, XSK_EFUSEPL_USER_KEY_128BIT_2, XSK_EFUSEPL_USER_KEY_128BIT_3 if XSK_EFUSEPL_PGM_SLR1 is TRUE XSK_EFUSEPL_USER_KEY_128BIT_0_SLR2, XSK_EFUSEPL_USER_KEY_128BIT_1_SLR2, XSK_EFUSEPL_USER_KEY_128BIT_2_SLR2, XSK_EFUSEPL_USER_KEY_128BIT_3_SLR2 if XSK_EFUSEPL_PGM_SLR2 is TRUE XSK_EFUSEPL_USER_KEY_128BIT_0_SLR3, XSK_EFUSEPL_USER_KEY_128BIT_1_SLR3, XSK_EFUSEPL_USER_KEY_128BIT_2_SLR3, XSK_EFUSEPL_USER_KEY_128BIT_3_SLR3 if XSK_EFUSEPL_PGM_SLR3 is TRUE XSK_EFUSEPL_USER_KEY_128BIT_0_SLR4, XSK_EFUSEPL_USER_KEY_128BIT_1_SLR4, XSK_EFUSEPL_USER_KEY_128BIT_2_SLR4, XSK_EFUSEPL_USER_KEY_128BIT_3_SLR4 if XSK_EFUSEPL_PGM_SLR4 is TRUE FALSE will ignore the values given.
XSK_EFUSEPL_USER_KEY_128BIT_0	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_0 holds 31:0 bits, of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR1/MONO when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR1 are enabled with TRUE value.

Parameter Name	Description
XSK_EFUSEPL_USER_KEY_128BIT_1	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_1 holds 63:32 bits, of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR1/MONO when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR1 are enabled with TRUE value.
XSK_EFUSEPL_USER_KEY_128BIT_2	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_2 holds 95:64 bits of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR1/MONO when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR1 are enabled with TRUE value.

Parameter Name	Description
XSK_EFUSEPL_USER_KEY_128BIT_3	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_3 holds 127:96 bits of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR1/MONO when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR1 are enabled with TRUE value.
XSK_EFUSEPL_USER_KEY_128BIT_0_SLR2	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_0_SLR2 holds 31:0 bits, of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR2 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR2 are enabled with TRUE value.

Parameter Name	Description
XSK_EFUSEPL_USER_KEY_128BIT_1_SLR2	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_1_SLR2 holds 63:32 bits, of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR2 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR2 are enabled with TRUE value.
XSK_EFUSEPL_USER_KEY_128BIT_2_SLR2	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_2_SLR2 holds 95:64 bits of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR2 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR2 are enabled with TRUE value.

Parameter Name	Description
XSK_EFUSEPL_USER_KEY_128BIT_3_SLR2	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_3_SLR2 holds 127:96 bits of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR2 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR2 are enabled with TRUE value.
XSK_EFUSEPL_USER_KEY_128BIT_0_SLR3	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_0_SLR3 holds 31:0 bits, of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR3 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR3 are enabled with TRUE value.

Parameter Name	Description
XSK_EFUSEPL_USER_KEY_128BIT_1_SLR3	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_1_SLR3 holds 63:32 bits, of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR3 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR3 are enabled with TRUE value.
XSK_EFUSEPL_USER_KEY_128BIT_2_SLR3	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_2_SLR3 holds 95:64 bits of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR3 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR3 are enabled with TRUE value.

Parameter Name	Description
XSK_EFUSEPL_USER_KEY_128BIT_3_SLR3	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_3_SLR3 holds 127:96 bits of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR3 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR3 are enabled with TRUE value.
XSK_EFUSEPL_USER_KEY_128BIT_0_SLR4	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_0_SLR4 holds 31:0 bits, of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR3 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR3 are enabled with TRUE value.

Parameter Name	Description
XSK_EFUSEPL_USER_KEY_128BIT_1_SLR4	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_1_SLR4 holds 63:32 bits, of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR4 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR4 are enabled with TRUE value.
XSK_EFUSEPL_USER_KEY_128BIT_2_SLR4	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_2_SLR4 holds 95:64 bits of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR4 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR4 are enabled with TRUE value.

Parameter Name	Description
XSK_EFUSEPL_USER_KEY_128BIT_3_SLR4	Default = 00000000 Provides 128-bit User key for XSK_EFUSEPL_USER_KEY_128BIT_3_SLR4 holds 127:96 bits of whole 128 bit User key. The value mentioned in this will be converted to hex buffer and written into the PL eFUSE array of SLR4 when write API used. This value should be the User Key given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn User Key. Note For writing the User Key, make sure XSK_EFUSEPL_PROGRAM_USER_KEY_128BIT and XSK_EFUSEPL_PGM_SLR4 are enabled with TRUE value.

WARNING: If you want to program USER key for SLR 1 and AES key for SLR2 then this should be done separately. For this you need to enable the XSK_EFUSEPL_PGM_SLR1, XSK_EFUSEPL_PGM_SLR2, XSK_EFUSEPL_PROGRAM_USER_KEY, and XSK_EFUSEPL_PROGRAM_AES_KEY parameters with the TRUE value. If you do all the settings in one single go and provide the USER key in XSK_EFUSEPL_USER_KEY and AES key in XSK_EFUSEPL_AES_KEY_SLR2 then:



- Enabling XSK_EFUSEPL_PROGRAM_USER_KEY will enable programming of USER key for both SLR1 And SLR2 as programming is enabled for both the SLR.
 - Enabling XSK_EFUSEPL_PROGRAM_AES_KEY will enable programming of AES key for both SLR1 And SLR2 as programming is enabled for both the SLR.
 - If you want to program USER key only for SLR1, then provided USER key will be programmed for SLR1 and Default key (all zeroes) will be programmed for SLR2.
 - If you want to program AES key only for SLR2, then provided AES key will be programmed for SLR2 and Default key will be programmed for SLR1.
- To avoid all the above mentioned scenarios, if programming is required for different key on different SLR, separate runs should be done.

AES key CRC verification

You cannot read the AES key.

You can verify only by providing the CRC of the expected AES key. The following lists the parameters that may help you in verifying the AES key:

Parameter Name	Description
XSK_EFUSEPL_CHECK_AES_KEY_CRC	Default = FALSE TRUE will perform CRC check of FUSE_AES with provided CRC value in macro XSK_EFUSEPL_CRC_OF_EXPECTED_AES_KEY. And result of CRC check will be updated in XilSKey_EPI instance parameter AESKeyMatched with either TRUE or FALSE. FALSE CRC check of FUSE_AES will not be performed.
XSK_EFUSEPL_CRC_OF_EXPECTED_AES_KEY	Default = XSK_EFUSEPL_AES_CRC_OF_ALL_ZEROS CRC value of FUSE_AES with all Zeros. Expected FUSE_AES key's CRC value has to be updated in place of XSK_EFUSEPL_AES_CRC_OF_ALL_ZEROS. For Checking CRC of FUSE_AES XSK_EFUSEPL_CHECK_AES_KEY_ULTRA macro should be TRUE otherwise CRC check will not be performed. For calculation of AES key's CRC one can use u32 XilSKey_CrcCalculation(u8_Key) API. For UltraScale, the value of XSK_EFUSEPL_AES_CRC_OF_ALL_ZEROS is 0x621C42AA(XSK_EFUSEPL_CRC_FOR_AES_ZEROS). For UltraScale+, the value of XSK_EFUSEPL_AES_CRC_OF_ALL_ZEROS is 0x3117503A(XSK_EFUSEPL_CRC_FOR_AES_ZEROS_ULTRA_PLUS)

Zynq UltraScale+ MPSoC User-Configurable PS eFUSE Parameters

Overview

The table below lists the user-configurable PS eFUSE parameters for Zynq UltraScale+ MPSoC devices.

Macro Name	Description
XSK_EFUSEPS_AES_RD_LOCK	Default = FALSE TRUE will permanently disable the CRC check of FUSE_AES. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_AES_WR_LOCK	Default = FALSE TRUE will permanently disable the writing to FUSE_AES block. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_ENC_ONLY	Default = FALSE TRUE will permanently enable encrypted booting only using the Fuse key. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_BBRAM_DISABLE	Default = FALSE TRUE will permanently disable the BBRAM key. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_ERR_DISABLE	Default = FALSE TRUE will permanently disables the error messages in JTAG status register. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_JTAG_DISABLE	Default = FALSE TRUE will permanently disable JTAG controller. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_DFT_DISABLE	Default = FALSE TRUE will permanently disable DFT boot mode. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_PROG_GATE_DISABLE	Default = FALSE TRUE will permanently disable PROG_GATE feature in PPD. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_SECURE_LOCK	Default = FALSE TRUE will permanently disable reboot into JTAG mode when doing a secure lockdown. FALSE will not modify this control bit of eFuse.

Macro Name	Description
XSK_EFUSEPS_RSA_ENABLE	Default = FALSE TRUE will permanently enable RSA authentication during boot. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_PPK0_WR_LOCK	Default = FALSE TRUE will permanently disable writing to PPK0 efuses. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_PPK0_INVLD	Default = FALSE TRUE will permanently revoke PPK0. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_PPK1_WR_LOCK	Default = FALSE TRUE will permanently disable writing PPK1 efuses. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_PPK1_INVLD	Default = FALSE TRUE will permanently revoke PPK1. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_USER_WRLK_0	Default = FALSE TRUE will permanently disable writing to USER_0 efuses. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_USER_WRLK_1	Default = FALSE TRUE will permanently disable writing to USER_1 efuses. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_USER_WRLK_2	Default = FALSE TRUE will permanently disable writing to USER_2 efuses. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_USER_WRLK_3	Default = FALSE TRUE will permanently disable writing to USER_3 efuses. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_USER_WRLK_4	Default = FALSE TRUE will permanently disable writing to USER_4 efuses. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_USER_WRLK_5	Default = FALSE TRUE will permanently disable writing to USER_5 efuses. FALSE will not modify this control bit of eFuse.

Macro Name	Description
XSK_EFUSEPS_USER_WRLK_6	Default = FALSE TRUE will permanently disable writing to USER_6 efuses. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_USER_WRLK_7	Default = FALSE TRUE will permanently disable writing to USER_7 efuses. FALSE will not modify this control bit of eFuse.
XSK_EFUSEPS_LBIST_EN	Default = FALSE TRUE will permanently enables logic BIST to be run during boot. FALSE will not modify this control bit of eFUSE.
XSK_EFUSEPS_LPD_SC_EN	Default = FALSE TRUE will permanently enables zeroization of registers in Low Power Domain(LPD) during boot. FALSE will not modify this control bit of eFUSE.
XSK_EFUSEPS_FPD_SC_EN	Default = FALSE TRUE will permanently enables zeroization of registers in Full Power Domain(FPD) during boot. FALSE will not modify this control bit of eFUSE.
XSK_EFUSEPS_PBR_BOOT_ERR	Default = FALSE TRUE will permanently enables the boot halt when there is any PMU error. FALSE will not modify this control bit of eFUSE.

Modules

- [AES Keys and Related Parameters](#)
- [User Keys and Related Parameters](#)
- [PPK0 Keys and Related Parameters](#)
- [PPK1 Keys and Related Parameters](#)
- [SPK ID and Related Parameters](#)

AES Keys and Related Parameters

The following table shows AES key related parameters.

Parameter Name	Description
XSK_EFUSEPS_WRITE_AES_KEY	Default = FALSE TRUE will burn the AES key provided in XSK_EFUSEPS_AES_KEY. FALSE will ignore the key provide XSK_EFUSEPS_AES_KEY.

Parameter Name	Description
XSK_EFUSEPS_AES_KEY	Default = 00000000000000000000000000000000 00000000000000000000000000000000 The value mentioned in this will be converted to hex buffer and written into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn AES Key. Note For writing the AES Key, XSK_EFUSEPS_WRITE_AES_KEY should have TRUE value.
XSK_EFUSEPS_CHECK_AES_KEY_CRC	Default value is FALSE. TRUE will check the CRC provided in XSK_EFUSEPS_AES_KEY. CRC verification is done after programming AES key to verify the key is programmed properly or not, if not library error outs the same. So While programming AES key it is not necessary to verify the AES key again. Note Please make sure if intention is to check only CRC of the provided key and not programming AES key then do not modify XSK_EFUSEPS_WRITE_AES_KEY (TRUE will Program key).

User Keys and Related Parameters

Single bit programming is allowed for all the user eFUSEs.

When you request to revert already programmed bit, the library will return an error. Also, if the user eFUSES is non-zero, the library will not throw an error for valid requests. The following table shows the user keys and related parameters.

Parameter Name	Description
XSK_EFUSEPS_WRITE_USER0_FUSE	Default = FALSE TRUE will burn User0 Fuse provided in XSK_EFUSEPS_USER0_FUSES. FALSE will ignore the value provided in XSK_EFUSEPS_USER0_FUSES
XSK_EFUSEPS_WRITE_USER1_FUSE	Default = FALSE TRUE will burn User1 Fuse provided in XSK_EFUSEPS_USER1_FUSES. FALSE will ignore the value provided in XSK_EFUSEPS_USER1_FUSES
XSK_EFUSEPS_WRITE_USER2_FUSE	Default = FALSE TRUE will burn User2 Fuse provided in XSK_EFUSEPS_USER2_FUSES. FALSE will ignore the value provided in XSK_EFUSEPS_USER2_FUSES
XSK_EFUSEPS_WRITE_USER3_FUSE	Default = FALSE TRUE will burn User3 Fuse provided in XSK_EFUSEPS_USER3_FUSES. FALSE will ignore the value provided in XSK_EFUSEPS_USER3_FUSES
XSK_EFUSEPS_WRITE_USER4_FUSE	Default = FALSE TRUE will burn User4 Fuse provided in XSK_EFUSEPS_USER4_FUSES. FALSE will ignore the value provided in XSK_EFUSEPS_USER4_FUSES
XSK_EFUSEPS_WRITE_USER5_FUSE	Default = FALSE TRUE will burn User5 Fuse provided in XSK_EFUSEPS_USER5_FUSES. FALSE will ignore the value provided in XSK_EFUSEPS_USER5_FUSES
XSK_EFUSEPS_WRITE_USER6_FUSE	Default = FALSE TRUE will burn User6 Fuse provided in XSK_EFUSEPS_USER6_FUSES. FALSE will ignore the value provided in XSK_EFUSEPS_USER6_FUSES
XSK_EFUSEPS_WRITE_USER7_FUSE	Default = FALSE TRUE will burn User7 Fuse provided in XSK_EFUSEPS_USER7_FUSES. FALSE will ignore the value provided in XSK_EFUSEPS_USER7_FUSES

Parameter Name	Description
XSK_EFUSEPS_USER0_FUSES	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn SPK ID. Note For writing the User0 Fuse, XSK_EFUSEPS_WRITE_USER0_FUSE should have TRUE value
XSK_EFUSEPS_USER1_FUSES	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn SPK ID. Note For writing the User1 Fuse, XSK_EFUSEPS_WRITE_USER1_FUSE should have TRUE value
XSK_EFUSEPS_USER2_FUSES	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn SPK ID. Note For writing the User2 Fuse, XSK_EFUSEPS_WRITE_USER2_FUSE should have TRUE value

Parameter Name	Description
XSK_EFUSEPS_USER3_FUSES	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn SPK ID. Note For writing the User3 Fuse, XSK_EFUSEPS_WRITE_USER3_FUSE should have TRUE value
XSK_EFUSEPS_USER4_FUSES	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn SPK ID. Note For writing the User4 Fuse, XSK_EFUSEPS_WRITE_USER4_FUSE should have TRUE value
XSK_EFUSEPS_USER5_FUSES	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn SPK ID. Note For writing the User5 Fuse, XSK_EFUSEPS_WRITE_USER5_FUSE should have TRUE value

Parameter Name	Description
XSK_EFUSEPS_USER6_FUSES	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn SPK ID. Note For writing the User6 Fuse, XSK_EFUSEPS_WRITE_USER6_FUSE should have TRUE value
XSK_EFUSEPS_USER7_FUSES	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn SPK ID. Note For writing the User7 Fuse, XSK_EFUSEPS_WRITE_USER7_FUSE should have TRUE value

PPK0 Keys and Related Parameters

The following table shows the PPK0 keys and related parameters.

Parameter Name	Description
XSK_EFUSEPS_WRITE_PPK0_SHA3_HASH	Default = FALSE TRUE will burn PPK0 sha3 hash provided in XSK_EFUSEPS_PPK0_SHA3_HASH. FALSE will ignore the hash provided in XSK_EFUSEPS_PPK0_SHA3_HASH.

Parameter Name	Description
XSK_EFUSEPS_PPK0_IS_SHA3	Default = TRUE TRUE XSK_EFUSEPS_PPK0_SHA3_HASH should be of string length 96 it specifies that PPK0 is used to program SHA3 hash. FALSE XSK_EFUSEPS_PPK0_SHA3_HASH should be of string length 64 it specifies that PPK0 is used to program SHA2 hash.
XSK_EFUSEPS_PPK0_HASH	Default = 00000000000000000000000000000000 00 The value mentioned in this will be converted to hex buffer and into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 96 or 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn PPK0 hash. Note that,for writing the PPK0 hash, XSK_EFUSEPS_WRITE_PPK0_SHA3_HASH should have TRUE value. While writing SHA2 hash, length should be 64 characters long XSK_EFUSEPS_PPK0_IS_SHA3 macro has to be made FALSE. While writing SHA3 hash, length should be 96 characters long and XSK_EFUSEPS_PPK0_IS_SHA3 macro should be made TRUE

PPK1 Keys and Related Parameters

The following table shows the PPK1 keys and related parameters.

Parameter Name	Description
XSK_EFUSEPS_WRITE_PPK1_SHA3_HASH	Default = FALSE TRUE will burn PPK1 sha3 hash provided in XSK_EFUSEPS_PPK1_SHA3_HASH. FALSE will ignore the hash provided in XSK_EFUSEPS_PPK1_SHA3_HASH.
XSK_EFUSEPS_PPK1_IS_SHA3	Default = TRUE TRUE XSK_EFUSEPS_PPK1_SHA3_HASH should be of string length 96 it specifies that PPK1 is used to program SHA3 hash. FALSE XSK_EFUSEPS_PPK1_SHA3_HASH should be of string length 64 it specifies that PPK1 is used to program SHA2 hash.

Parameter Name	Description
XSK_EFUSEPS_PPK1_HASH	Default = 00000000000000000000000000000000 00000000000000000000000000000000 The value mentioned in this will be converted to hex buffer and written into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 64 or 96 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn PPK1 hash. Note that,for writing the PPK11 hash, XSK_EFUSEPS_WRITE_PPK1_SHA3_HASH should have TRUE value. By default, PPK1 hash will be provided with 64 character length to program PPK1 hash with sha2 hash so XSK_EFUSEPS_PPK1_IS_SHA3 also will be in FALSE state. But to program PPK1 hash with SHA3 hash make XSK_EFUSEPS_PPK1_IS_SHA3 to TRUE and provide sha3 hash of length 96 characters XSK_EFUSEPS_PPK1_HASH so that one can program sha3 hash.

SPK ID and Related Parameters

The following table shows the SPK ID and related parameters.

Parameter Name	Description
XSK_EFUSEPS_WRITE_SPKID	Default = FALSE TRUE will burn SPKID provided in XSK_EFUSEPS_SPK_ID. FALSE will ignore the hash provided in XSK_EFUSEPS_SPK_ID.
XSK_EFUSEPS_SPK_ID	Default = 00000000 The value mentioned in this will be converted to hex buffer and written into the Zynq UltraScale+ MPSoC PS eFUSE array when write API used. This value should be given in string format. It should be 8 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn SPK ID. Note For writing the SPK ID, XSK_EFUSEPS_WRITE_SPKID should have TRUE value.

Note

PPK hash should be unmodified hash generated by bootgen. Single bit programming is allowed for User FUSEs (0 to 7), if you specify a value that tries to set a bit that was previously programmed to 1 back to 0, you will get an error. you have to provide already programmed bits also along with new requests.

Zynq UltraScale+ MPSoC User-Configurable PS BBRAM Parameters

The table below lists the AES and user key parameters.

Parameter Name	Description
XSK_ZYNQMP_BBRAMPS_AES_KEY	Default = 00000000000000000000000000000000 00000000000000000000000000000000 AES key (in HEX) that must be programmed into BBRAM.
XSK_ZYNQMP_BBRAMPS_AES_KEY_LEN_IN_BYTES	Default = 32. Length of AES key in bytes.
XSK_ZYNQMP_BBRAMPS_AES_KEY_LEN_IN_BITS	Default = 256. Length of AES key in bits.
XSK_ZYNQMP_BBRAMPS_AES_KEY_STR_LEN	Default = 64. String length of the AES key.

Zynq UltraScale+ MPSoC User-Configurable PS PUF Parameters

The table below lists the user-configurable PS PUF parameters for Zynq UltraScale+ MPSoC devices.

Macro Name	Description
XSK_PUF_INFO_ON_UART	Default = FALSE TRUE will display syndrome data on UART com port FALSE will display any data on UART com port.
XSK_PUF_PROGRAM_EFUSE	Default = FALSE TRUE will program the generated syndrome data, CHash and Auxilary values, Black key. FALSE will not program data into eFUSE.
XSK_PUF_IF_CONTRACT_MANUFACTURER	Default = FALSE This should be enabled when application is hand over to contract manufacturer. TRUE will allow only authenticated application. FALSE authentication is not mandatory.
XSK_PUF_REG_MODE	Default = XSK_PUF_MODE4K PUF registration is performed in 4K mode. For only understanding it is provided in this file, but user is not supposed to modify this.

Macro Name	Description
XSK_PUF_READ_SECUREBITS	Default = FALSE TRUE will read status of the puf secure bits from eFUSE and will be displayed on UART. FALSE will not read secure bits.
XSK_PUF_PROGRAM_SECUREBITS	Default = FALSE TRUE will program PUF secure bits based on the user input provided at XSK_PUF_SYN_INVALID, XSK_PUF_SYN_WRLK and XSK_PUF_REGISTER_DISABLE. FALSE will not program any PUF secure bits.
XSK_PUF_SYN_INVALID	Default = FALSE TRUE will permanently invalidate the already programmed syndrome data. FALSE will not modify anything
XSK_PUF_SYN_WRLK	Default = FALSE TRUE will permanently disable programming syndrome data into eFUSE. FALSE will not modify anything.
XSK_PUF_REGISTER_DISABLE	Default = FALSE TRUE permanently does not allow PUF syndrome data registration. FALSE will not modify anything.
XSK_PUF_RESERVED	Default = FALSE TRUE programs this reserved eFUSE bit. FALSE will not modify anything.
XSK_PUF_AES_KEY	Default = 00000000000000000000000000000000 00000000000000000000000000000000 The value mentioned in this will be converted to hex buffer and encrypts this with PUF helper data and generates a black key and written into the Zynq UltraScale+ MPSoC PS eFUSE array when XSK_PUF_PROGRAM_EFUSE macro is TRUE. This value should be given in string format. It should be 64 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string and will not burn AES Key. Note Provided here should be red key and application calculates the black key and programs into eFUSE if XSK_PUF_PROGRAM_EFUSE macro is TRUE. To avoid programming eFUSE results can be displayed on UART com port by making XSK_PUF_INFO_ON_UART to TRUE.

Macro Name	Description
XSK_PUF_BLACK_KEY_IV	Default = 000000000000000000000000 The value mentioned here will be converted to hex buffer. This is Initialization vector(IV) which is used to generated black key with provided AES key and generated PUF key. This value should be given in string format. It should be 24 characters long, valid characters are 0-9,a-f,A-F. Any other character is considered as invalid string.

Error Codes

Overview

The application error code is 32 bits long.
For example, if the error code for PS is 0x8A05:

- 0x8A indicates that a write error has occurred while writing RSA Authentication bit.
- 0x05 indicates that write error is due to the write temperature out of range.

Applications have the following options on how to show error status. Both of these methods of conveying the status are implemented by default. However, UART is required to be present and initialized for status to be displayed through UART.

- Send the error code through UART pins
- Write the error code in the reboot status register

Modules

- [PL eFUSE Error Codes](#)
- [PS eFUSE Error Codes](#)
- [Zynq UltraScale+ MPSoC BBRAM PS Error Codes](#)

PL eFUSE Error Codes

XSK_EFUSEPL_ERROR_NONE 0
No error.

XSK_EFUSEPL_ERROR_ROW_NOT_ZERO 0x10
Row is not zero.

XSK_EFUSEPL_ERROR_READ_ROW_OUT_OF_RANGE 0x11
Read Row is out of range.

XSK_EFUSEPL_ERROR_READ_MARGIN_OUT_OF_RANGE 0x12
Read Margin is out of range.

XSK_EFUSEPL_ERROR_READ_BUFFER_NULL 0x13
No buffer for read.

<i>XSK_EFUSEPL_ERROR_READ_BIT_VALUE_NOT_SET</i>	<i>0x14</i>
Read bit not set.	
<i>XSK_EFUSEPL_ERROR_READ_BIT_OUT_OF_RANGE</i>	<i>0x15</i>
Read bit is out of range.	
<i>XSK_EFUSEPL_ERROR_READ_TMEPERATURE_OUT_OF_RANGE</i>	<i>0x16</i>
Temperature obtained from XADC is out of range to read.	
<i>XSK_EFUSEPL_ERROR_READ_VCCAUX_VOLTAGE_OUT_OF_RANGE</i>	<i>0x17</i>
VCCAUX obtained from XADC is out of range to read.	
<i>XSK_EFUSEPL_ERROR_READ_VCCINT_VOLTAGE_OUT_OF_RANGE</i>	<i>0x18</i>
VCCINT obtained from XADC is out of range to read.	
<i>XSK_EFUSEPL_ERROR_WRITE_ROW_OUT_OF_RANGE</i>	<i>0x19</i>
To write row is out of range.	
<i>XSK_EFUSEPL_ERROR_WRITE_BIT_OUT_OF_RANGE</i>	<i>0x1A</i>
To read bit is out of range.	
<i>XSK_EFUSEPL_ERROR_WRITE_TMEPERATURE_OUT_OF_RANGE</i>	<i>0x1B</i>
To eFUSE write Temperature obtained from XADC is out of range.	
<i>XSK_EFUSEPL_ERROR_WRITE_VCCAUX_VOLTAGE_OUT_OF_RANGE</i>	<i>0x1C</i>
To write eFUSE VCCAUX obtained from XADC is out of range.	
<i>XSK_EFUSEPL_ERROR_WRITE_VCCINT_VOLTAGE_OUT_OF_RANGE</i>	<i>0x1D</i>
To write into eFUSE VCCINT obtained from XADC is out of range.	
<i>XSK_EFUSEPL_ERROR_FUSE_CNTRL_WRITE_DISABLED</i>	<i>0x1E</i>
Fuse control write is disabled.	
<i>XSK_EFUSEPL_ERROR_CNTRL_WRITE_BUFFER_NULL</i>	<i>0x1F</i>
Buffer pointer that is supposed to contain control data is null.	
<i>XSK_EFUSEPL_ERROR_NOT_VALID_KEY_LENGTH</i>	<i>0x20</i>
Key length invalid.	
<i>XSK_EFUSEPL_ERROR_ZERO_KEY_LENGTH</i>	<i>0x21</i>
Key length zero.	
<i>XSK_EFUSEPL_ERROR_NOT_VALID_KEY_CHAR</i>	<i>0x22</i>
Invalid key characters.	
<i>XSK_EFUSEPL_ERROR_NULL_KEY</i>	<i>0x23</i>
Null key.	
<i>XSK_EFUSEPL_ERROR_FUSE_SEC_WRITE_DISABLED</i>	<i>0x24</i>
Secure bits write is disabled.	
<i>XSK_EFUSEPL_ERROR_FUSE_SEC_READ_DISABLED</i>	<i>0x25</i>
Secure bits reading is disabled.	
<i>XSK_EFUSEPL_ERROR_SEC_WRITE_BUFFER_NULL</i>	<i>0x26</i>
Buffer to write into secure block is NULL.	
<i>XSK_EFUSEPL_ERROR_READ_PAGE_OUT_OF_RANGE</i>	<i>0x27</i>
Page is out of range.	
<i>XSK_EFUSEPL_ERROR_FUSE_ROW_RANGE</i>	<i>0x28</i>
Row is out of range.	

<i>XSK_EFUSEPL_ERROR_IN_PROGRAMMING_ROW</i>	<i>0x29</i>
Error programming fuse row.	
<i>XSK_EFUSEPL_ERROR_PRGRMG_ROWS_NOT_EMPTY</i>	<i>0x2A</i>
Error when tried to program non Zero rows of eFUSE.	
<i>XSK_EFUSEPL_ERROR_HWM_TIMEOUT</i>	<i>0x80</i>
Error when hardware module is exceeded the time for programming eFUSE.	
<i>XSK_EFUSEPL_ERROR_USER_FUSE_REVERT</i>	<i>0x90</i>
Error occurs when user requests to revert already programmed user eFUSE bit.	
<i>XSK_EFUSEPL_ERROR_KEY_VALIDATION</i>	<i>0xF000</i>
Invalid key.	
<i>XSK_EFUSEPL_ERROR_PL_STRUCT_NULL</i>	<i>0x1000</i>
Null PL structure.	
<i>XSK_EFUSEPL_ERROR_JTAG_SERVER_INIT</i>	<i>0x1100</i>
JTAG server initialization error.	
<i>XSK_EFUSEPL_ERROR_READING_FUSE_CNTRL</i>	<i>0x1200</i>
Error reading fuse control.	
<i>XSK_EFUSEPL_ERROR_DATA_PROGRAMMING_NOT_ALLOWED</i>	<i>0x1300</i>
Data programming not allowed.	
<i>XSK_EFUSEPL_ERROR_FUSE_CTRL_WRITE_NOT_ALLOWED</i>	<i>0x1400</i>
Fuse control write is disabled.	
<i>XSK_EFUSEPL_ERROR_READING_FUSE_AES_ROW</i>	<i>0x1500</i>
Error reading fuse AES row.	
<i>XSK_EFUSEPL_ERROR_AES_ROW_NOT_EMPTY</i>	<i>0x1600</i>
AES row is not empty.	
<i>XSK_EFUSEPL_ERROR_PROGRAMMING_FUSE_AES_ROW</i>	<i>0x1700</i>
Error programming fuse AES row.	
<i>XSK_EFUSEPL_ERROR_READING_FUSE_USER_DATA_ROW</i>	<i>0x1800</i>
Error reading fuse user row.	
<i>XSK_EFUSEPL_ERROR_USER_DATA_ROW_NOT_EMPTY</i>	<i>0x1900</i>
User row is not empty.	
<i>XSK_EFUSEPL_ERROR_PROGRAMMING_FUSE_DATA_ROW</i>	<i>0x1A00</i>
Error programming fuse user row.	
<i>XSK_EFUSEPL_ERROR_PROGRAMMING_FUSE_CNTRL_ROW</i>	<i>0x1B00</i>
Error programming fuse control row.	
<i>XSK_EFUSEPL_ERROR_XADC</i>	<i>0x1C00</i>
XADC error.	
<i>XSK_EFUSEPL_ERROR_INVALID_REF_CLK</i>	<i>0x3000</i>
Invalid reference clock.	
<i>XSK_EFUSEPL_ERROR_FUSE_SEC_WRITE_NOT_ALLOWED</i>	<i>0x1D00</i>
Error in programming secure block.	
<i>XSK_EFUSEPL_ERROR_READING_FUSE_STATUS</i>	<i>0x1E00</i>
Error in reading FUSE status.	

XSK_EFUSEPL_ERROR_FUSE_BUSY 0x1F00
Fuse busy.

XSK_EFUSEPL_ERROR_READING_FUSE_RSA_ROW 0x2000
Error in reading FUSE RSA block.

XSK_EFUSEPL_ERROR_TIMER_INITIALISE_ULTRA 0x2200
Error in initiating Timer.

XSK_EFUSEPL_ERROR_READING_FUSE_SEC 0x2300
Error in reading FUSE secure bits.

XSK_EFUSEPL_ERROR_PRGRMG_FUSE_SEC_ROW 0x2500
Error in programming Secure bits of efuse.

XSK_EFUSEPL_ERROR_PRGRMG_USER_KEY 0x4000
Error in programming 32 bit user key.

XSK_EFUSEPL_ERROR_PRGRMG_128BIT_USER_KEY 0x5000
Error in programming 128 bit User key.

XSK_EFUSEPL_ERROR_PRGRMG_RSA_HASH 0x8000
Error in programming RSA hash.

PS eFUSE Error Codes

XSK_EFUSEPS_ERROR_NONE 0
No error.

XSK_EFUSEPS_ERROR_ADDRESS_XIL_RESTRICTED 0x01
Address is restricted.

XSK_EFUSEPS_ERROR_READ_TEMPERATURE_OUT_OF_RANGE 0x02
Temperature obtained from XADC is out of range.

XSK_EFUSEPS_ERROR_READ_VCCPAUX_VOLTAGE_OUT_OF_RANGE 0x03
VCCAUX obtained from XADC is out of range.

XSK_EFUSEPS_ERROR_READ_VCCPINT_VOLTAGE_OUT_OF_RANGE 0x04
VCCINT obtained from XADC is out of range.

XSK_EFUSEPS_ERROR_WRITE_TEMPERATURE_OUT_OF_RANGE 0x05
Temperature obtained from XADC is out of range.

XSK_EFUSEPS_ERROR_WRITE_VCCPAUX_VOLTAGE_OUT_OF_RANGE 0x06
VCCAUX obtained from XADC is out of range.

XSK_EFUSEPS_ERROR_WRITE_VCCPINT_VOLTAGE_OUT_OF_RANGE 0x07
VCCINT obtained from XADC is out of range.

XSK_EFUSEPS_ERROR_VERIFICATION 0x08
Verification error.

XSK_EFUSEPS_ERROR_RSA_HASH_ALREADY_PROGRAMMED 0x09
RSA hash was already programmed.

XSK_EFUSEPS_ERROR_CONTROLLER_MODE 0x0A
Controller mode error

XSK_EFUSEPS_ERROR_REF_CLOCK 0x0B
Reference clock not between 20 to 60MHz

<i>XSK_EFUSEPS_ERROR_READ_MODE</i>	<i>0x0C</i>
Not supported read mode	
<i>XSK_EFUSEPS_ERROR_XADC_CONFIG</i>	<i>0x0D</i>
XADC configuration error.	
<i>XSK_EFUSEPS_ERROR_XADC_INITIALIZE</i>	<i>0x0E</i>
XADC initialization error.	
<i>XSK_EFUSEPS_ERROR_XADC_SELF_TEST</i>	<i>0x0F</i>
XADC self-test failed.	
<i>XSK_EFUSEPS_ERROR_PARAMETER_NULL</i>	<i>0x10</i>
Passed parameter null.	
<i>XSK_EFUSEPS_ERROR_STRING_INVALID</i>	<i>0x20</i>
Passed string is invalid.	
<i>XSK_EFUSEPS_ERROR_AES_ALREADY_PROGRAMMED</i>	<i>0x12</i>
AES key is already programmed.	
<i>XSK_EFUSEPS_ERROR_SPKID_ALREADY_PROGRAMMED</i>	<i>0x13</i>
SPK ID is already programmed.	
<i>XSK_EFUSEPS_ERROR_PPK0_HASH_ALREADY_PROGRAMMED</i>	<i>0x14</i>
PPK0 hash is already programmed.	
<i>XSK_EFUSEPS_ERROR_PPK1_HASH_ALREADY_PROGRAMMED</i>	<i>0x15</i>
PPK1 hash is already programmed.	
<i>XSK_EFUSEPS_ERROR_PROGRAMMING_TBIT_PATTERN</i>	<i>0x16</i>
Error in programming TBITS.	
<i>XSK_EFUSEPS_ERROR_BEFORE_PROGRAMMING</i>	<i>0x0080</i>
Error occurred before programming.	
<i>XSK_EFUSEPS_ERROR_PROGRAMMING</i>	<i>0x00A0</i>
Error in programming eFUSE.	
<i>XSK_EFUSEPS_ERROR_READ</i>	<i>0x00B0</i>
Error in reading.	
<i>XSK_EFUSEPS_ERROR_BYTES_REQUEST</i>	<i>0x00C0</i>
Error in requested byte count.	
<i>XSK_EFUSEPS_ERROR_RESRVD_BITS_PRGRMG</i>	<i>0x00D0</i>
Error in programming reserved bits.	
<i>XSK_EFUSEPS_ERROR_ADDR_ACCESS</i>	<i>0x00E0</i>
Error in accessing requested address.	
<i>XSK_EFUSEPS_ERROR_READ_NOT_DONE</i>	<i>0x00F0</i>
Read not done	
<i>XSK_EFUSEPS_ERROR_PS_STRUCT_NULL</i>	<i>0x8100</i>
PS structure pointer is null.	
<i>XSK_EFUSEPS_ERROR_XADC_INIT</i>	<i>0x8200</i>
XADC initialization error.	
<i>XSK_EFUSEPS_ERROR_CONTROLLER_LOCK</i>	<i>0x8300</i>
PS eFUSE controller is locked.	

<i>XSK_EFUSEPS_ERROR_EFUSE_WRITE_PROTECTED</i>	0x8400
PS eFUSE is write protected.	
<i>XSK_EFUSEPS_ERROR_CONTROLLER_CONFIG</i>	0x8500
Controller configuration error.	
<i>XSK_EFUSEPS_ERROR_PS_PARAMETER_WRONG</i>	0x8600
PS eFUSE parameter is not TRUE/FALSE.	
<i>XSK_EFUSEPS_ERROR_WRITE_128K_CRC_BIT</i>	0x9100
Error in enabling 128K CRC.	
<i>XSK_EFUSEPS_ERROR_WRITE_NONSECURE_INITB_BIT</i>	0x9200
Error in programming NON secure bit.	
<i>XSK_EFUSEPS_ERROR_WRITE_UART_STATUS_BIT</i>	0x9300
Error in writing UART status bit.	
<i>XSK_EFUSEPS_ERROR_WRITE_RSA_HASH</i>	0x9400
Error in writing RSA key.	
<i>XSK_EFUSEPS_ERROR_WRITE_RSA_AUTH_BIT</i>	0x9500
Error in enabling RSA authentication bit.	
<i>XSK_EFUSEPS_ERROR_WRITE_WRITE_PROTECT_BIT</i>	0x9600
Error in writing write-protect bit.	
<i>XSK_EFUSEPS_ERROR_READ_HASH_BEFORE_PROGRAMMING</i>	0x9700
Check RSA key before trying to program.	
<i>XSK_EFUSEPS_ERROR_WRTIE_DFT_JTAG_DIS_BIT</i>	0x9800
Error in programming DFT JTAG disable bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_DFT_MODE_DIS_BIT</i>	0x9900
Error in programming DFT MODE disable bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_AES_CRC_LK_BIT</i>	0x9A00
Error in enabling AES's CRC check lock.	
<i>XSK_EFUSEPS_ERROR_WRTIE_AES_WR_LK_BIT</i>	0x9B00
Error in programming AES write lock bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_USE_AESONLY_EN_BIT</i>	0x9C00
Error in programming use AES only bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_BBRAM_DIS_BIT</i>	0x9D00
Error in programming BBRAM disable bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_PMU_ERR_DIS_BIT</i>	0x9E00
Error in programming PMU error disable bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_JTAG_DIS_BIT</i>	0x9F00
Error in programming JTAG disable bit.	
<i>XSK_EFUSEPS_ERROR_READ_RSA_HASH</i>	0xA100
Error in reading RSA key.	
<i>XSK_EFUSEPS_ERROR_WRONG_TBIT_PATTERN</i>	0xA200
Error in programming TBIT pattern.	
<i>XSK_EFUSEPS_ERROR_WRITE_AES_KEY</i>	0xA300
Error in programming AES key.	

<i>XSK_EFUSEPS_ERROR_WRITE_SPK_ID</i>	<i>0xA400</i>
Error in programming SPK ID.	
<i>XSK_EFUSEPS_ERROR_WRITE_USER_KEY</i>	<i>0xA500</i>
Error in programming USER key.	
<i>XSK_EFUSEPS_ERROR_WRITE_PPK0_HASH</i>	<i>0xA600</i>
Error in programming PPK0 hash.	
<i>XSK_EFUSEPS_ERROR_WRITE_PPK1_HASH</i>	<i>0xA700</i>
Error in programming PPK1 hash.	
<i>XSK_EFUSEPS_ERROR_CACHE_LOAD</i>	<i>0xB000</i>
Error in re-loading CACHE.	
<i>XSK_EFUSEPS_ERROR_WRITE_USER0_FUSE</i>	<i>0xC000</i>
Error in programming USER 0 Fuses.	
<i>XSK_EFUSEPS_ERROR_WRITE_USER1_FUSE</i>	<i>0xC100</i>
Error in programming USER 1 Fuses.	
<i>XSK_EFUSEPS_ERROR_WRITE_USER2_FUSE</i>	<i>0xC200</i>
Error in programming USER 2 Fuses.	
<i>XSK_EFUSEPS_ERROR_WRITE_USER3_FUSE</i>	<i>0xC300</i>
Error in programming USER 3 Fuses.	
<i>XSK_EFUSEPS_ERROR_WRITE_USER4_FUSE</i>	<i>0xC400</i>
Error in programming USER 4 Fuses.	
<i>XSK_EFUSEPS_ERROR_WRITE_USER5_FUSE</i>	<i>0xC500</i>
Error in programming USER 5 Fuses.	
<i>XSK_EFUSEPS_ERROR_WRITE_USER6_FUSE</i>	<i>0xC600</i>
Error in programming USER 6 Fuses.	
<i>XSK_EFUSEPS_ERROR_WRITE_USER7_FUSE</i>	<i>0xC700</i>
Error in programming USER 7 Fuses.	
<i>XSK_EFUSEPS_ERROR_WRTIE_USER0_LK_BIT</i>	<i>0xC800</i>
Error in programming USER 0 fuses lock bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_USER1_LK_BIT</i>	<i>0xC900</i>
Error in programming USER 1 fuses lock bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_USER2_LK_BIT</i>	<i>0xCA00</i>
Error in programming USER 2 fuses lock bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_USER3_LK_BIT</i>	<i>0xCB00</i>
Error in programming USER 3 fuses lock bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_USER4_LK_BIT</i>	<i>0xCC00</i>
Error in programming USER 4 fuses lock bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_USER5_LK_BIT</i>	<i>0xCD00</i>
Error in programming USER 5 fuses lock bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_USER6_LK_BIT</i>	<i>0xCE00</i>
Error in programming USER 6 fuses lock bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_USER7_LK_BIT</i>	<i>0xCF00</i>
Error in programming USER 7 fuses lock bit.	

<i>XSK_EFUSEPS_ERROR_WRTIE_PROG_GATE0_DIS_BIT</i>	<i>0xD000</i>
Error in programming PROG_GATE0 disabling bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_PROG_GATE1_DIS_BIT</i>	<i>0xD100</i>
Error in programming PROG_GATE1 disabling bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_PROG_GATE2_DIS_BIT</i>	<i>0xD200</i>
Error in programming PROG_GATE2 disabling bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_SEC_LOCK_BIT</i>	<i>0xD300</i>
Error in programming SEC_LOCK bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_PPK0_WR_LK_BIT</i>	<i>0xD400</i>
Error in programming PPK0 write lock bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_PPK0_RVK_BIT</i>	<i>0xD500</i>
Error in programming PPK0 revoke bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_PPK1_WR_LK_BIT</i>	<i>0xD600</i>
Error in programming PPK1 write lock bit.	
<i>XSK_EFUSEPS_ERROR_WRTIE_PPK1_RVK_BIT</i>	<i>0xD700</i>
Error in programming PPK0 revoke bit.	
<i>XSK_EFUSEPS_ERROR_WRITE_PUF_SYN_INVLD</i>	<i>0xD800</i>
Error while programming the PUF syndrome invalidate bit.	
<i>XSK_EFUSEPS_ERROR_WRITE_PUF_SYN_WRLK</i>	<i>0xD900</i>
Error while programming Syndrome write lock bit.	
<i>XSK_EFUSEPS_ERROR_WRITE_PUF_SYN_REG_DIS</i>	<i>0xDA00</i>
Error while programming PUF syndrome register disable bit.	
<i>XSK_EFUSEPS_ERROR_WRITE_PUF_RESERVED_BIT</i>	<i>0xDB00</i>
Error while programming PUF reserved bit.	
<i>XSK_EFUSEPS_ERROR_WRITE_LBIST_EN_BIT</i>	<i>0xDC00</i>
Error while programming LBIST enable bit.	
<i>XSK_EFUSEPS_ERROR_WRITE_LPD_SC_EN_BIT</i>	<i>0xDD00</i>
Error while programming LPD SC enable bit.	
<i>XSK_EFUSEPS_ERROR_WRITE_FPD_SC_EN_BIT</i>	<i>0xDE00</i>
Error while programming FPD SC enable bit.	
<i>XSK_EFUSEPS_ERROR_WRITE_PBR_BOOT_ERR_BIT</i>	<i>0xDF00</i>
Error while programming PBR boot error bit.	
<i>XSK_EFUSEPS_ERROR_PUF_INVALID_REG_MODE</i>	<i>0xE000</i>
Error when PUF registration is requested with invalid registration mode.	
<i>XSK_EFUSEPS_ERROR_PUF_REG_WO_AUTH</i>	<i>0xE100</i>
Error when write not allowed without authentication enabled.	
<i>XSK_EFUSEPS_ERROR_PUF_REG_DISABLED</i>	<i>0xE200</i>
Error when trying to do PUF registration and when PUF registration is disabled.	
<i>XSK_EFUSEPS_ERROR_PUF_INVALID_REQUEST</i>	<i>0xE300</i>
Error when an invalid mode is requested.	
<i>XSK_EFUSEPS_ERROR_PUF_DATA_ALREADY_PROGRAMMED</i>	<i>0xE400</i>
Error when PUF is already programmed in eFUSE.	

XSK_EFUSEPS_ERROR_PUF_DATA_OVERFLOW 0xE500

Error when an over flow occurs.

XSK_EFUSEPS_ERROR_SPKID_BIT_CANT_REVERT 0xE600

Already programmed SPKID bit cannot be reverted

XSK_EFUSEPS_ERROR_PUF_DATA_UNDERFLOW 0xE700

Error when an under flow occurs.

XSK_EFUSEPS_ERROR_PUF_TIMEOUT 0xE800

Error when an PUF generation timedout.

XSK_EFUSEPS_ERROR_CMPLTD_EFUSE_PRGRM_WITH_ERR 0x10000

eFUSE programming is completed with temp and vol read errors.

XSK_EFUSEPS_ERROR_FUSE_PROTECTED 0x00080000

Requested eFUSE is write protected.

XSK_EFUSEPS_ERROR_USER_BIT_CANT_REVERT 0x00800000

Already programmed user FUSE bit cannot be reverted.

Zynq UltraScale+ MPSoC BBRAM PS Error Codes

XSK_ZYNQMP_BBRAMPS_ERROR_NONE 0

No error.

XSK_ZYNQMP_BBRAMPS_ERROR_IN_PRGRMG_ENABLE 0x01

If this error is occurred programming is not possible.

XSK_ZYNQMP_BBRAMPS_ERROR_IN_CRC_CHECK 0xB000

If this error is occurred programming is done but CRC check is failed.

XSK_ZYNQMP_BBRAMPS_ERROR_IN_PRGRMG 0xC000

programming of key is failed.

XSK_ZYNQMP_BBRAMPS_ERROR_IN_ZEROISE 0xE700

zeroize bbram is failed.

XSK_ZYNQMP_BBRAMPS_ERROR_IN_WRITE_CRC 0xE800

error write CRC value.

Status Codes

For Zynq® and UltraScale™, the status in the `xilsky_efuse_example.c` file is conveyed through a UART or reboot status register in the following format: `0xYYYYZZZZ`, where:

- `YYYY` represents the PS eFUSE Status.
- `ZZZZ` represents the PL eFUSE Status.

The table below lists the status codes.

Status Code Values	Description
<code>0x0000ZZZZ</code>	Represents PS eFUSE is successful and PL eFUSE process returned with error.
<code>0xYYYY0000</code>	Represents PL eFUSE is successful and PS eFUSE process returned with error.
<code>0xFFFF0000</code>	Represents PS eFUSE is not initiated and PL eFUSE is successful.
<code>0x0000FFFF</code>	Represents PL eFUSE is not initiated and PS eFUSE is successful.
<code>0xFFFFZZZZ</code>	Represents PS eFUSE is not initiated and PL eFUSE is process returned with error.
<code>0xYYYYFFFF</code>	Represents PL eFUSE is not initiated and PS eFUSE is process returned with error.

For Zynq UltraScale+ MPSoC, the status in the `xilsky_bbramps_zynqmp_example.c`, `xilsky_puf_registration.c` and `xilsky_efuseps_zynqmp_example.c` files is conveyed as 32 bit error code. Where Zero represents that no error has occurred and if the value is other than Zero, a 32 bit error code is returned.

Procedures

This chapter provides detailed descriptions of the various procedures.

Zynq eFUSE Writing Procedure Running from DDR as an Application

This sequence is same as the existing flow described below.

1. Provide the required inputs in `xilskey_input.h`, then compile the SDK project.
2. Take the latest FSBL (ELF), stitch the `<output>.elf` generated to it (using the bootgen utility), and generate a bootable image.
3. Write the generated binary image into the flash device (for example: QSPI, NAND).
4. To burn the eFUSE key bits, execute the image.

Zynq eFUSE Driver Compilation Procedure for OCM

The procedure is as follows:

1. Open the linker script (`lscript.ld`) in the SDK project.
2. Map all the chapters to point to `ps7_ram_0_S_AXI_BASEADDR` instead of `ps7_ddr_0_S_AXI_BASEADDR`. For example, Click the Memory Region tab for the `.text` chapter and select `ps7_ram_0_S_AXI_BASEADDR` from the drop-down list.
3. Copy the `ps7_init.c` and `ps7_init.h` files from the `hw_platform` folder into the example folder.
4. In `xilskey_efuse_example.c`, un-comment the code that calls the `ps7_init()` routine.
5. Compile the project.
The `<Project name>.elf` file is generated and is executed out of OCM.

When executed, this example displays the success/failure of the eFUSE application in a display message via UART (if UART is present and initialized) or the reboot status register.

UltraScale eFUSE Access Procedure

The procedure is as follows:

1. After providing the required inputs in `xilskey_input.h`, compile the project.
2. Generate a memory mapped interface file using TCL command `write_mem_info`

```
$Outfilename
```

3. Update memory has to be done using the tcl command `updatemem`.

```
updatemem -meminfo $file.mmi -data $Outfilename.elf -bit $design.bit  
-proc design_1_i/microblaze_0 -out $Final.bit
```

4. Program the board using `$Final.bit` bitstream.
5. Output can be seen in UART terminal.

UltraScale BBRAM Access Procedure

The procedure is as follows:

1. After providing the required inputs in the `xilskey_bbram_ultrascale_input.h` file, compile the project.
2. Generate a memory mapped interface file using TCL command

```
write_mem_info $Outfilename
```

3. Update memory has to be done using the tcl command `updatemem`:

```
updatemem -meminfo $file.mmi -data $Outfilename.elf -bit $design.bit  
-proc design_1_i/microblaze_0 -out $Final.bit
```

4. Program the board using `$Final.bit` bitstream.
5. Output can be seen in UART terminal.

Appendix K:

XiIPM Library v2.5

XilPM APIs

Overview

Xilinx Power Management(XilPM) provides Embedded Energy Management Interface (EEMI) APIs for power management on Zynq® UltraScale+™ MPSoC. For more details about power management on Zynq Ultrascale+ MPSoC, see the Zynq UltraScale+ MPSoC Power Management User Guide (UG1199). For more details about EEMI, see the Embedded Energy Management Interface (EEMI) API User Guide(UG1200).

Xilinx Power Management(XilPM) provides Embedded Energy Management Interface (EEMI) APIs for power management on Zynq® UltraScale+™ MPSoC. For more details about power management on Zynq Ultrascale+ MPSoC, see the Zynq UltraScale+ MPSoC Power Management User Guide (UG1199). For more details about EEMI, see the Embedded Energy Management Interface (EEMI) API User Guide(UG1200).

Modules

- [Error Status](#)

Data Structures

- struct [XPm_Notifier](#)
- struct [XPm_NodeStatus](#)
- struct [XPmClockSel2ClkIn](#)
- struct [XPmClockMux](#)
- struct [XPmClockModel](#)

Enumerations

- enum [XPmApild](#)
- enum [XPmApiCbld](#)
- enum [XPmNodeId](#)
- enum [XPmRequestAck](#)
- enum [XPmAbortReason](#)
- enum [XPmSuspendReason](#)
- enum [XPmRamState](#)
- enum [XPmOpCharType](#)

- enum [XPmBootTestStatus](#)
- enum [XPmResetAction](#)
- enum [XPmReset](#)
- enum [XPmNotifyEvent](#)
- enum [XPmClock](#)

Functions

- XStatus [XPm_InitXilpm](#) (XlpiPsu *lpiInst)
- void [XPm_SuspendFinalize](#) (void)
- enum [XPmBootTestStatus](#) [XPm_GetBootTestStatus](#) (void)
- XStatus [XPm_RequestSuspend](#) (const enum [XPmNodeId](#) target, const enum [XPmRequestAck](#) ack, const u32 latency, const u8 state)
- XStatus [XPm_SelfSuspend](#) (const enum [XPmNodeId](#) nid, const u32 latency, const u8 state, const u64 address)
- XStatus [XPm_ForcePowerDown](#) (const enum [XPmNodeId](#) target, const enum [XPmRequestAck](#) ack)
- XStatus [XPm_AbortSuspend](#) (const enum [XPmAbortReason](#) reason)
- XStatus [XPm_RequestWakeUp](#) (const enum [XPmNodeId](#) target, const bool setAddress, const u64 address, const enum [XPmRequestAck](#) ack)
- XStatus [XPm_SetWakeUpSource](#) (const enum [XPmNodeId](#) target, const enum [XPmNodeId](#) wkup_node, const u8 enable)
- XStatus [XPm_SystemShutdown](#) (u32 type, u32 subtype)
- XStatus [XPm_SetConfiguration](#) (const u32 address)
- XStatus [XPm_InitFinalize](#) (void)
- void [XPm_InitSuspendCb](#) (const enum [XPmSuspendReason](#) reason, const u32 latency, const u32 state, const u32 timeout)
- void [XPm_AcknowledgeCb](#) (const enum [XPmNodeId](#) node, const XStatus status, const u32 oppoint)
- void [XPm_NotifyCb](#) (const enum [XPmNodeId](#) node, const enum [XPmNotifyEvent](#) event, const u32 oppoint)
- XStatus [XPm_RequestNode](#) (const enum [XPmNodeId](#) node, const u32 capabilities, const u32 qos, const enum [XPmRequestAck](#) ack)
- XStatus [XPm_ReleaseNode](#) (const enum [XPmNodeId](#) node)
- XStatus [XPm_SetRequirement](#) (const enum [XPmNodeId](#) nid, const u32 capabilities, const u32 qos, const enum [XPmRequestAck](#) ack)
- XStatus [XPm_SetMaxLatency](#) (const enum [XPmNodeId](#) node, const u32 latency)
- XStatus [XPm_GetApiVersion](#) (u32 *version)
- XStatus [XPm_GetNodeStatus](#) (const enum [XPmNodeId](#) node, [XPm_NodeStatus](#) *const nodestatus)
- XStatus [XPm_RegisterNotifier](#) ([XPm_Notifier](#) *const notifier)
- XStatus [XPm_UnregisterNotifier](#) ([XPm_Notifier](#) *const notifier)
- XStatus [XPm_GetOpCharacteristic](#) (const enum [XPmNodeId](#) node, const enum [XPmOpCharType](#) type, u32 *const result)
- XStatus [XPm_ResetAssert](#) (const enum [XPmReset](#) reset, const enum [XPmResetAction](#) resetaction)
- XStatus [XPm_ResetGetStatus](#) (const enum [XPmReset](#) reset, u32 *status)
- XStatus [XPm_MmioWrite](#) (const u32 address, const u32 mask, const u32 value)

- XStatus [XPm_MmioRead](#) (const u32 address, u32 *const value)
- XStatus [XPm_ClockEnable](#) (const enum [XPmClock](#) clock)
- XStatus [XPm_ClockDisable](#) (const enum [XPmClock](#) clock)
- XStatus [XPm_ClockGetStatus](#) (const enum [XPmClock](#) clock, u32 *const status)
- XStatus [XPm_ClockSetDivider](#) (const enum [XPmClock](#) clock, const u32 divider)
- XStatus [XPm_ClockGetDivider](#) (const enum [XPmClock](#) clock, u32 *const divider)
- XStatus [XPm_ClockSetParent](#) (const enum [XPmClock](#) clock, const enum [XPmClock](#) parent)
- XStatus [XPm_ClockGetParent](#) (const enum [XPmClock](#) clock, enum [XPmClock](#) *const parent)
- XStatus [XPm_ClockSetRate](#) (const enum [XPmClock](#) clock, const u32 rate)
- XStatus [XPm_ClockGetRate](#) (const enum [XPmClock](#) clock, u32 *const rate)
- XStatus [XPm_PllSetParameter](#) (const enum [XPmNodeId](#) node, const enum XPmPllParam parameter, const u32 value)
- XStatus [XPm_PllGetParameter](#) (const enum [XPmNodeId](#) node, const enum XPmPllParam parameter, u32 *const value)
- XStatus [XPm_PllSetMode](#) (const enum [XPmNodeId](#) node, const enum XPmPllMode mode)
- XStatus [XPm_PllGetMode](#) (const enum [XPmNodeId](#) node, enum XPmPllMode *const mode)
- XStatus [XPm_PinCtrlRequest](#) (const u32 pin)
- XStatus [XPm_PinCtrlRelease](#) (const u32 pin)
- XStatus [XPm_PinCtrlSetFunction](#) (const u32 pin, const enum XPmPinFn fn)
- XStatus [XPm_PinCtrlGetFunction](#) (const u32 pin, enum XPmPinFn *const fn)
- XStatus [XPm_PinCtrlSetParameter](#) (const u32 pin, const enum XPmPinParam param, const u32 value)
- XStatus [XPm_PinCtrlGetParameter](#) (const u32 pin, const enum XPmPinParam param, u32 *const value)
- XStatus [XPm_NotifierAdd](#) ([XPm_Notifier](#) *const notifier)
- XStatus [XPm_NotifierRemove](#) ([XPm_Notifier](#) *const notifier)
- void [XPm_NotifierProcessEvent](#) (const enum [XPmNodeId](#) node, const enum [XPmNotifyEvent](#) event, const u32 oppoint)
- XStatus [XPm_GetClockParentBySelect](#) (const enum [XPmClock](#) clockId, const u32 select, enum [XPmClock](#) *const parentId)
- XStatus [XPm_GetSelectByClockParent](#) (const enum [XPmClock](#) clockId, const enum [XPmClock](#) parentId, u32 *const select)
- u8 [XPm_GetClockDivType](#) (const enum [XPmClock](#) clock)
- u8 [XPm_MapDivider](#) (const enum [XPmClock](#) clock, const u32 div, u32 *const div0, u32 *const div1)

PM Version Number macros

- #define **PM_VERSION_MAJOR** 1
- #define **PM_VERSION_MINOR** 1
- #define **PM_VERSION** ((PM_VERSION_MAJOR << 16) | PM_VERSION_MINOR)

Capabilities for RAM

- #define **PM_CAP_ACCESS** 0x1U
- #define **PM_CAP_CONTEXT** 0x2U
- #define **PM_CAP_WAKEUP** 0x4U

Node default states macros

- #define **NODE_STATE_OFF** 0
- #define **NODE_STATE_ON** 1

Processor's states macros

- #define **PROC_STATE_FORCEDOFF** 0
- #define **PROC_STATE_ACTIVE** 1
- #define **PROC_STATE_SLEEP** 2
- #define **PROC_STATE_SUSPENDING** 3

Maximum Latency/QOS macros

- #define **MAX_LATENCY** (~0U)
- #define **MAX_QOS** 100U

System shutdown/Restart macros

- #define **PMF_SHUTDOWN_TYPE_SHUTDOWN** 0U
- #define **PMF_SHUTDOWN_TYPE_RESET** 1U
- #define **PMF_SHUTDOWN_SUBTYPE_SUBSYSTEM** 0U
- #define **PMF_SHUTDOWN_SUBTYPE_PS_ONLY** 1U
- #define **PMF_SHUTDOWN_SUBTYPE_SYSTEM** 2U

PM API Min and Max macros

- #define **PM_API_MIN** PM_GET_API_VERSION

Payload Packets

Assigning of argument values into array elements. `pause` and `pm_dbg` are used for debugging and should be removed in final version.

- #define **PACK_PAYLOAD**(pl, arg0, arg1, arg2, arg3, arg4, arg5, rsvd)
- #define **PACK_PAYLOAD0**(pl, api_id) **PACK_PAYLOAD**(pl, (api_id), 0U, 0U, 0U, 0U, 0U, 0U)
- #define **PACK_PAYLOAD1**(pl, api_id, arg1) **PACK_PAYLOAD**(pl, (api_id), (arg1), 0U, 0U, 0U, 0U, 0U)
- #define **PACK_PAYLOAD2**(pl, api_id, arg1, arg2) **PACK_PAYLOAD**(pl, (api_id), (arg1), (arg2), 0U, 0U, 0U, 0U)
- #define **PACK_PAYLOAD3**(pl, api_id, arg1, arg2, arg3) **PACK_PAYLOAD**(pl, (api_id), (arg1), (arg2), (arg3), 0U, 0U, 0U)
- #define **PACK_PAYLOAD4**(pl, api_id, arg1, arg2, arg3, arg4) **PACK_PAYLOAD**(pl, (api_id), (arg1), (arg2), (arg3), (arg4), 0U, 0U)

- `#define PACK_PAYLOAD5(pl, api_id, arg1, arg2, arg3, arg4, arg5) PACK_PAYLOAD(pl, (api_id), (arg1), (arg2), (arg3), (arg4), (arg5), 0U)`

Data Structure Documentation

struct XPm_Notifier

[XPm_Notifier](#) - Notifier structure registered with a callback by app

Data Fields

- `void(*const callback )(struct XPm_Ntfier *const notifier)`
- `enum XPmNodeId node`
- `enum XPmNotifyEvent event`
- `u32 flags`
- `volatile u32 oppoint`
- `volatile u32 received`
- `struct XPm_Ntfier * next`

Field Documentation

void(*const [callback](#))(struct XPm_Ntfier *const notifier) Custom callback handler to be called when the notification is received. The custom handler would execute from interrupt context, it shall return quickly and must not block! (enables event-driven notifications)

enum [XPmNodeId](#) node Node argument (the node to receive notifications about)

enum [XPmNotifyEvent](#) event Event argument (the event type to receive notifications about)

u32 [flags](#) Flags

volatile u32 [oppoint](#) Operating point of node in question. Contains the value updated when the last event notification is received. User shall not modify this value while the notifier is registered.

volatile u32 [received](#) How many times the notification has been received - to be used by application (enables polling). User shall not modify this value while the notifier is registered.

struct XPm_Ntfier* [next](#) Pointer to next notifier in linked list. Must not be modified while the notifier is registered. User shall not ever modify this value.

struct XPm_NodeStatus

[XPm_NodeStatus](#) - struct containing node status information

Data Fields

- u32 [status](#)
- u32 [requirements](#)
- u32 [usage](#)

Field Documentation

u32 status Node power state

u32 requirements Current requirements asserted on the node (slaves only)

u32 usage Usage information (which master is currently using the slave)

struct XPmClockSel2ClkIn

Pair of multiplexer select value and selected clock input

Data Fields

- enum [XPmClock clkIn](#)
- const u8 [select](#)

Field Documentation

enum XPmClock clkIn ID of the clock that is selected with the 'select' value

const u8 select Select value of the clock multiplexer

struct XPmClockMux

MUX select values to clock input mapping

Data Fields

- const [XPmClockSel2ClkIn](#) *const [inputs](#)
- const u8 [size](#)
- const u8 [bits](#)
- const u8 [shift](#)

Field Documentation

const XPmClockSel2ClkIn* const inputs Mux select to pll mapping at the input of the multiplexer

const u8 size Size of the inputs array

const u8 bits Number of bits of mux select

const u8 shift Number of bits to shift 'bits' in order to get mux select mask

struct XPmClockModel

Clock model

Data Fields

- enum [XPmClock id](#)
- const [XPmClockMux](#) *const [mux](#)
- const u8 [type](#)
- const struct XPmClkModel *const [next](#)

Field Documentation

enum XPmClock id Clock ID

const XPmClockMux* const mux Pointer to the mux model

const u8 type Type specifying the available divisors

const struct XPmClkModel* const next Next clock in the list

Enumeration Type Documentation

enum XPmApild

APIs for Miscellaneous functions, suspending of PUs, managing PM slaves and Direct control.

enum XPmApiCbld

PM API Callback Id Enum

enum XPmNodeId

PM Node ID Enum

enum XPmRequestAck

PM Acknowledge Request Types

enum XPmAbortReason

PM Abort Reasons Enum

enum XPmSuspendReason

PM Suspend Reasons Enum

enum XPmRamState

PM RAM States Enum

enum XPmOpCharType

PM Operating Characteristic types Enum

enum XPmBootTestStatus

Boot Status Enum

enum XPmResetAction

PM Reset Action types

enum XPmReset

PM Reset Line IDs

enum XPmNotifyEvent

PM Notify Events Enum

enum XPmClock

PM Clock IDs

Function Documentation

XStatus XPm_InitXilpm (XlpiPsu * *IpInst*)

Initialize xilpm library.

Parameters

<i>IpInst</i>	Pointer to IPI driver instance
---------------	--------------------------------

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

None

void XPm_SuspendFinalize (void)

This Function waits for PMU to finish all previous API requests sent by the PU and performs client specific actions to finish suspend procedure (e.g. execution of wfi instruction on A53 and R5 processors).

Note

This function should not return if the suspend procedure is successful.

enum XPmBootStatus XPm_GetBootStatus (void)

This Function returns information about the boot reason. If the boot is not a system startup but a resume, power down request bitfield for this processor will be cleared.

Returns

Returns processor boot status

- PM_RESUME : If the boot reason is because of system resume.
- PM_INITIAL_BOOT : If this boot is the initial system startup.

Note

None

XStatus XPm_RequestSuspend (const enum XPmNodeId *target*, const enum XPmRequestAck *ack*, const u32 *latency*, const u8 *state*)

This function is used by a PU to request suspend of another PU. This call triggers the power management controller to notify the PU identified by 'nodeID' that a suspend has been requested. This will allow said PU to

gracefully suspend itself by calling `XPm_SelfSuspend` for each of its CPU nodes, or else call `XPm_AbortSuspend` with its PU node as argument and specify the reason.

Parameters

<i>target</i>	Node ID of the PU node to be suspended
<i>ack</i>	Requested acknowledge type
<i>latency</i>	Maximum wake-up latency requirement in us(micro sec)
<i>state</i>	Instead of specifying a maximum latency, a PU can also explicitly request a certain power state.

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

If 'ack' is set to `PM_ACK_NON_BLOCKING`, the requesting PU will be notified upon completion of suspend or if an error occurred, such as an abort. `REQUEST_ACK_BLOCKING` is not supported for this command.

XStatus XPm_SelfSuspend (const enum XPmNodeId *nid*, const u32 *latency*, const u8 *state*, const u64 *address*)

This function is used by a CPU to declare that it is about to suspend itself. After the PMU processes this call it will wait for the requesting CPU to complete the suspend procedure and become ready to be put into a sleep state.

Parameters

<i>nid</i>	Node ID of the CPU node to be suspended.
<i>latency</i>	Maximum wake-up latency requirement in us(microsecs)
<i>state</i>	Instead of specifying a maximum latency, a CPU can also explicitly request a certain power state.
<i>address</i>	Address from which to resume when woken up.

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

This is a blocking call, it will return only once PMU has responded

XStatus XPm_ForcePowerDown (const enum XPmNodeId *target*, const enum XPmRequestAck *ack*)

One PU can request a forced poweroff of another PU or its power island or power domain. This can be used for killing an unresponsive PU, in which case all resources of that PU will be automatically released.

Parameters

<i>target</i>	Node ID of the PU node or power island/domain to be powered down.
<i>ack</i>	Requested acknowledge type

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

Force power down may not be requested by a PU for itself.

XStatus XPm_AbortSuspend (const enum XPmAbortReason *reason*)

This function is called by a CPU after a XPm_SelfSuspend call to notify the power management controller that CPU has aborted suspend or in response to an init suspend request when the PU refuses to suspend.

Parameters

<i>reason</i>	Reason code why the suspend can not be performed or completed - ABORT_REASON_WKUP_EVENT : local wakeup-event received - ABORT_REASON_PU_BUSY : PU is busy - ABORT_REASON_NO_PWRDN : no external powerdown supported - ABORT_REASON_UNKNOWN : unknown error during suspend procedure
---------------	--

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

Calling PU expects the PMU to abort the initiated suspend procedure. This is a non-blocking call without any acknowledge.

XStatus XPm_RequestWakeUp (const enum XPmNodeId *target*, const bool *setAddress*, const u64 *address*, const enum XPmRequestAck *ack*)

This function can be used to request power up of a CPU node within the same PU, or to power up another PU.

Parameters

<i>target</i>	Node ID of the CPU or PU to be powered/woken up.
<i>setAddress</i>	Specifies whether the start address argument is being passed. 0 : do not set start address 1 : set start address
<i>address</i>	Address from which to resume when woken up. Will only be used if <i>set_address</i> is 1.
<i>ack</i>	Requested acknowledge type

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

If acknowledge is requested, the calling PU will be notified by the power management controller once the wake-up is completed.

XStatus XPm_SetWakeUpSource (const enum XPmNodeId *target*, const enum XPmNodeId *wkup_node*, const u8 *enable*)

This function is called by a PU to add or remove a wake-up source prior to going to suspend. The list of wake sources for a PU is automatically cleared whenever the PU is woken up or when one of its CPUs aborts the suspend procedure.

Parameters

<i>target</i>	Node ID of the target to be woken up.
<i>wkup_node</i>	Node ID of the wakeup device.
<i>enable</i>	Enable flag: 1 : the wakeup source is added to the list 0 : the wakeup source is removed from the list

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

Declaring a node as a wakeup source will ensure that the node will not be powered off. It also will cause the PMU to configure the GIC Proxy accordingly if the FPD is powered off.

XStatus XPm_SystemShutdown (u32 type, u32 subtype)

This function can be used by a privileged PU to shut down or restart the complete device.

Parameters

<i>restart</i>	Should the system be restarted automatically? • PM_SHUTDOWN : no restart requested, system will be powered off permanently • PM_RESTART : restart is requested, system will go through a full reset
----------------	---

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

In either case the PMU will call XPm_InitSuspendCb for each of the other PUs, allowing them to gracefully shut down. If a PU is asleep it will be woken up by the PMU. The PU making the XPm_SystemShutdown should perform its own suspend procedure after calling this API. It will not receive an init suspend callback.

XStatus XPm_SetConfiguration (const u32 address)

This function is called to configure the power management framework. The call triggers power management controller to load the configuration object and configure itself according to the content of the object.

Parameters

<i>address</i>	Start address of the configuration object
----------------	---

Returns

XST_SUCCESS if successful, otherwise an error code

Note

The provided address must be in 32-bit address space which is accessible by the PMU.

XStatus XPm_InitFinalize (void)

This function is called to notify the power management controller about the completed power management initialization.

Returns

XST_SUCCESS if successful, otherwise an error code

Note

It is assumed that all used nodes are requested when this call is made. The power management controller may power down the nodes which are not requested after this call is processed.

void XPm_InitSuspendCb (const enum XPmSuspendReason *reason*, const u32 *latency*, const u32 *state*, const u32 *timeout*)

Callback function to be implemented in each PU, allowing the power management controller to request that the PU suspend itself.

Parameters

<i>reason</i>	Suspend reason: • SUSPEND_REASON_PU_REQ : Request by another PU • SUSPEND_REASON_ALERT : Unrecoverable SysMon alert • SUSPEND_REASON_SHUTDOWN : System shutdown • SUSPEND_REASON_RESTART : System restart
<i>latency</i>	Maximum wake-up latency in us(micro secs). This information can be used by the PU to decide what level of context saving may be required.
<i>state</i>	Targeted sleep/suspend state.
<i>timeout</i>	Timeout in ms, specifying how much time a PU has to initiate its suspend procedure before it's being considered unresponsive.

Returns

None

Note

If the PU fails to act on this request the power management controller or the requesting PU may choose to employ the forceful power down option.

void XPm_AcknowledgeCb (const enum XPmNodeId *node*, const XStatus *status*, const u32 *oppoint*)

This function is called by the power management controller in response to any request where an acknowledge callback was requested, i.e. where the 'ack' argument passed by the PU was REQUEST_ACK_NON_BLOCKING.

Parameters

<i>node</i>	ID of the component or sub-system in question.
<i>status</i>	Status of the operation: • OK: the operation completed successfully • ERR: the requested operation failed
<i>oppoint</i>	Operating point of the node in question

Returns

None

Note

None

void XPm_NotifyCb (const enum XPmNodeId *node*, const enum XPmNotifyEvent *event*, const u32 *oppoint*)

This function is called by the power management controller if an event the PU was registered for has occurred. It will populate the notifier data structure passed when calling XPm_RegisterNotifier.

Parameters

<i>node</i>	ID of the node the event notification is related to.
<i>event</i>	ID of the event
<i>oppoint</i>	Current operating state of the node.

Returns

None

Note

None

XStatus XPm_RequestNode (const enum XPmNodeId *node*, const u32 *capabilities*, const u32 *qos*, const enum XPmRequestAck *ack*)

Used to request the usage of a PM-slave. Using this API call a PU requests access to a slave device and asserts its requirements on that device. Provided the PU is sufficiently privileged, the PMU will enable access to the memory mapped region containing the control registers of that device. For devices that can only be serving a single PU, any other privileged PU will now be blocked from accessing this device until the node is released.

Parameters

<i>node</i>	Node ID of the PM slave requested
<i>capabilities</i>	Slave-specific capabilities required, can be combined - PM_CAP_ACCESS : full access / functionality - PM_CAP_CONTEXT : preserve context - PM_CAP_WAKEUP : emit wake interrupts
<i>qos</i>	Quality of Service (0-100) required
<i>ack</i>	Requested acknowledge type

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

None

XStatus XPm_ReleaseNode (const enum XPmNodeId *node*)

This function is used by a PU to release the usage of a PM slave. This will tell the power management controller that the node is no longer needed by that PU, potentially allowing the node to be placed into an inactive state.

Parameters

<i>node</i>	Node ID of the PM slave.
-------------	--------------------------

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

None

XStatus XPm_SetRequirement (const enum XPmNodeId *nid*, const u32 *capabilities*, const u32 *qos*, const enum XPmRequestAck *ack*)

This function is used by a PU to announce a change in requirements for a specific slave node which is currently in use.

Parameters

<i>nid</i>	Node ID of the PM slave.
<i>capabilities</i>	Slave-specific capabilities required.
<i>qos</i>	Quality of Service (0-100) required.
<i>ack</i>	Requested acknowledge type

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

If this function is called after the last awake CPU within the PU calls SelfSuspend, the requirement change shall be performed after the CPU signals the end of suspend to the power management controller, (e.g. WFI interrupt).

XStatus XPm_SetMaxLatency (const enum XPmNodeId *node*, const u32 *latency*)

This function is used by a PU to announce a change in the maximum wake-up latency requirements for a specific slave node currently used by that PU.

Parameters

<i>node</i>	Node ID of the PM slave.
<i>latency</i>	Maximum wake-up latency required.

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

Setting maximum wake-up latency can constrain the set of possible power states a resource can be put into.

XStatus XPm_GetApiVersion (u32 * *version*)

This function is used to request the version number of the API running on the power management controller.

Parameters

<i>version</i>	Returns the API 32-bit version number. Returns 0 if no PM firmware present.
----------------	---

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

None

XStatus XPm_GetNodeStatus (const enum XPmNodeId *node*, XPm_NodeStatus *const *nodestatus*)

This function is used to obtain information about the current state of a component. The caller must pass a pointer to an [XPm_NodeStatus](#) structure, which must be pre-allocated by the caller.

Parameters

<i>node</i>	ID of the component or sub-system in question.
<i>nodestatus</i>	Used to return the complete status of the node.

- status - The current power state of the requested node.
 - For CPU nodes:
 - 0 : if CPU is powered down,
 - 1 : if CPU is active (powered up),
 - 2 : if CPU is suspending (powered up)
 - For power islands and power domains:
 - 0 : if island is powered down,
 - 1 : if island is powered up
 - For PM slaves:
 - 0 : if slave is powered down,
 - 1 : if slave is powered up,
 - 2 : if slave is in retention
- requirement - Slave nodes only: Returns current requirements the requesting PU has requested of the node.
- usage - Slave nodes only: Returns current usage status of the node:
 - 0 : node is not used by any PU,

- 1 : node is used by caller exclusively,
- 2 : node is used by other PU(s) only,
- 3 : node is used by caller and by other PU(s)

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

None

XStatus XPm_RegisterNotifier (XPm_Notifier *const *notifier*)

A PU can call this function to request that the power management controller call its notify callback whenever a qualifying event occurs. One can request to be notified for a specific or any event related to a specific node.

Parameters

<i>notifier</i>	Pointer to the notifier object to be associated with the requested notification. The notifier object contains the following data related to the notification:
-----------------	---

- nodeID : ID of the node to be notified about,
- eventID : ID of the event in question, '-1' denotes all events (- EVENT_STATE_CHANGE, EVENT_ZERO_USERS),
- wake : true: wake up on event, false: do not wake up (only notify if awake), no buffering/queueing
- callback : Pointer to the custom callback function to be called when the notification is available. The callback executes from interrupt context, so the user must take special care when implementing the callback. Callback is optional, may be set to NULL.
- received : Variable indicating how many times the notification has been received since the notifier is registered.

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

The caller shall initialize the notifier object before invoking the XPm_RegisterNotifier function. While notifier is registered, the notifier object shall not be modified by the caller.

XStatus XPm_UnregisterNotifier (XPm_Notifier *const *notifier*)

A PU calls this function to unregister for the previously requested notifications.

Parameters

<i>notifier</i>	Pointer to the notifier object associated with the previously requested notification
-----------------	--

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

None

XStatus XPm_GetOpCharacteristic (const enum XPmNodeId *node*, const enum XPmOpCharType *type*, u32 *const *result*)

Call this function to request the power management controller to return information about an operating characteristic of a component.

Parameters

<i>node</i>	ID of the component or sub-system in question.
<i>type</i>	Type of operating characteristic requested: • power (current power consumption), • latency (current latency in us to return to active state), • temperature (current temperature),
<i>result</i>	Used to return the requested operating characteristic.

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

None

XStatus XPm_ResetAssert (const enum XPmReset *reset*, const enum XPmResetAction *resetaction*)

This function is used to assert or release reset for a particular reset line. Alternatively a reset pulse can be requested as well.

Parameters

<i>reset</i>	ID of the reset line
<i>assert</i>	Identifies action: - PM_RESET_ACTION_RELEASE : release reset, - PM_RESET_ACTION_ASSERT : assert reset, - PM_RESET_ACTION_PULSE : pulse reset,

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

None

XStatus XPm_ResetGetStatus (const enum XPmReset *reset*, u32 * *status*)

Call this function to get the current status of the selected reset line.

Parameters

<i>reset</i>	Reset line
<i>status</i>	Status of specified reset (true - asserted, false - released)

Returns

Returns 1/XST_FAILURE for 'asserted' or 0/XST_SUCCESS for 'released'.

Note

None

XStatus XPm_MmioWrite (const u32 *address*, const u32 *mask*, const u32 *value*)

Call this function to write a value directly into a register that isn't accessible directly, such as registers in the clock control unit. This call is bypassing the power management logic. The permitted addresses are subject to restrictions as defined in the PCW configuration.

Parameters

<i>address</i>	Physical 32-bit address of memory mapped register to write to.
<i>mask</i>	32-bit value used to limit write to specific bits in the register.
<i>value</i>	Value to write to the register bits specified by the mask.

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

If the access isn't permitted this function returns an error code.

XStatus XPm_MmioRead (const u32 *address*, u32 **const value*)

Call this function to read a value from a register that isn't accessible directly. The permitted addresses are subject to restrictions as defined in the PCW configuration.

Parameters

<i>address</i>	Physical 32-bit address of memory mapped register to read from.
<i>value</i>	Returns the 32-bit value read from the register

Returns

XST_SUCCESS if successful else XST_FAILURE or an error code or a reason code

Note

If the access isn't permitted this function returns an error code.

XStatus XPm_ClockEnable (const enum XPmClock *clock*)

Call this function to enable (activate) a clock.

Parameters

<i>clock</i>	Identifier of the target clock to be enabled
--------------	--

Returns

Status of performing the operation as returned by the PMU-FW

Note

If the access isn't permitted this function returns an error code.

XStatus XPm_ClockDisable (const enum XPmClock *clock*)

Call this function to disable (gate) a clock.

Parameters

<i>clock</i>	Identifier of the target clock to be disabled
--------------	---

Returns

Status of performing the operation as returned by the PMU-FW

Note

If the access isn't permitted this function returns an error code.

XStatus XPm_ClockGetStatus (const enum XPmClock *clock*, u32 *const *status*)

Call this function to get status of a clock gate state.

Parameters

<i>clock</i>	Identifier of the target clock
<i>status</i>	Location to store clock gate state (1=enabled, 0=disabled)

Returns

Status of performing the operation as returned by the PMU-FW

XStatus XPm_ClockSetDivider (const enum XPmClock *clock*, const u32 *divider*)

Call this function to set divider for a clock.

Parameters

<i>clock</i>	Identifier of the target clock
<i>divider</i>	Divider value to be set

Returns

XST_INVALID_PARAM or status of performing the operation as returned by the PMU-FW

Note

If the access isn't permitted this function returns an error code.

XStatus XPm_ClockGetDivider (const enum XPmClock *clock*, u32 *const *divider*)

Call this function to get divider of a clock.

Parameters

<i>clock</i>	Identifier of the target clock
<i>divider</i>	Location to store the divider value

Returns

XST_INVALID_PARAM or status of performing the operation as returned by the PMU-FW

XStatus XPm_ClockSetParent (const enum XPmClock *clock*, const enum XPmClock *parent*)

Call this function to set parent for a clock.

Parameters

<i>clock</i>	Identifier of the target clock
<i>parent</i>	Identifier of the target parent clock

Returns

XST_INVALID_PARAM or status of performing the operation as returned by the PMU-FW.

Note

If the access isn't permitted this function returns an error code.

XStatus XPm_ClockGetParent (const enum XPmClock *clock*, enum XPmClock *const *parent*)

Call this function to get parent of a clock.

Parameters

<i>clock</i>	Identifier of the target clock
<i>parent</i>	Location to store clock parent ID

Returns

XST_INVALID_PARAM or status of performing the operation as returned by the PMU-FW.

XStatus XPm_ClockSetRate (const enum XPmClock *clock*, const u32 *rate*)

Call this function to set rate of a clock.

Parameters

<i>clock</i>	Identifier of the target clock
<i>rate</i>	Clock frequency (rate) to be set

Returns

Status of performing the operation as returned by the PMU-FW

Note

If the action isn't permitted this function returns an error code.

XStatus XPm_ClockGetRate (const enum XPmClock *clock*, u32 *const *rate*)

Call this function to get rate of a clock.

Parameters

<i>clock</i>	Identifier of the target clock
<i>rate</i>	Location where the rate should be stored

Returns

Status of performing the operation as returned by the PMU-FW

XStatus XPm_PllSetParameter (const enum XPmNodeId *node*, const enum XPmPllParam *parameter*, const u32 *value*)

Call this function to set a PLL parameter.

Parameters

<i>node</i>	PLL node identifier
<i>parameter</i>	PLL parameter identifier
<i>value</i>	Value of the PLL parameter

Returns

Status of performing the operation as returned by the PMU-FW

Note

If the access isn't permitted this function returns an error code.

XStatus XPm_PllGetParameter (const enum XPmNodeId *node*, const enum XPmPllParam *parameter*, u32 *const *value*)

Call this function to get a PLL parameter.

Parameters

<i>node</i>	PLL node identifier
<i>parameter</i>	PLL parameter identifier
<i>value</i>	Location to store value of the PLL parameter

Returns

Status of performing the operation as returned by the PMU-FW

XStatus XPm_PllSetMode (const enum XPmNodeId *node*, const enum XPmPllMode *mode*)

Call this function to set a PLL mode.

Parameters

<i>node</i>	PLL node identifier
<i>mode</i>	PLL mode to be set

Returns

Status of performing the operation as returned by the PMU-FW

Note

If the access isn't permitted this function returns an error code.

**XStatus XPm_PllGetMode (const enum XPmNodeId *node*,
enum XPmPllMode *const *mode*)**

Call this function to get a PLL mode.

Parameters

<i>node</i>	PLL node identifier
<i>mode</i>	Location to store the PLL mode

Returns

Status of performing the operation as returned by the PMU-FW

XStatus XPm_PinCtrlRequest (const u32 *pin*)

Call this function to request a pin control.

Parameters

<i>pin</i>	PIN identifier (index from range 0-77)
------------	--

Returns

Status of performing the operation as returned by the PMU-FW

XStatus XPm_PinCtrlRelease (const u32 *pin*)

Call this function to release a pin control.

Parameters

<i>pin</i>	PIN identifier (index from range 0-77)
------------	--

Returns

Status of performing the operation as returned by the PMU-FW

XStatus XPm_PinCtrlSetFunction (const u32 *pin*, const enum XPmPinFn *fn*)

Call this function to set a pin function.

Parameters

<i>pin</i>	Pin identifier
<i>fn</i>	Pin function to be set

Returns

Status of performing the operation as returned by the PMU-FW

Note

If the access isn't permitted this function returns an error code.

XStatus XPm_PinCtrlGetFunction (const u32 *pin*, enum XPmPinFn *const *fn*)

Call this function to get currently configured pin function.

Parameters

<i>pin</i>	PLL node identifier
<i>fn</i>	Location to store the pin function

Returns

Status of performing the operation as returned by the PMU-FW

XStatus XPm_PinCtrlSetParameter (const u32 *pin*, const enum XPmPinParam *param*, const u32 *value*)

Call this function to set a pin parameter.

Parameters

<i>pin</i>	Pin identifier
<i>param</i>	Pin parameter identifier
<i>value</i>	Value of the pin parameter to set

Returns

Status of performing the operation as returned by the PMU-FW

Note

If the access isn't permitted this function returns an error code.

XStatus XPm_PinCtrlGetParameter (const u32 *pin*, const enum XPmPinParam *param*, u32 *const *value*)

Call this function to get currently configured value of pin parameter.

Parameters

<i>pin</i>	Pin identifier
<i>param</i>	Pin parameter identifier
<i>value</i>	Location to store value of the pin parameter

Returns

Status of performing the operation as returned by the PMU-FW

XStatus XPm_NotifierAdd (XPm_Notifier *const *notifier*)

Add notifier into the list.

Parameters

<i>notifier</i>	Pointer to notifier object which needs to be added in the list
-----------------	--

Returns

Returns XST_SUCCESS if notifier is added / XST_INVALID_PARAM if given notifier argument is NULL

Note

None

XStatus XPm_NotifierRemove (XPm_Notifier *const *notifier*)

Remove notifier from the list.

Parameters

<i>notifier</i>	Pointer to notifier object to be removed from list
-----------------	--

Returns

Returns XST_SUCCESS if notifier is removed / XST_INVALID_PARAM if given notifier pointer is NULL / XST_FAILURE if notifier is not found

Note

None

void XPm_NotifierProcessEvent (const enum XPmNodeId *node*, const enum XPmNotifyEvent *event*, const u32 *oppoint*)

Call to process notification event.

Parameters

<i>node</i>	Node which is the subject of notification
<i>event</i>	Event which is the subject of notification
<i>oppoint</i>	Operating point of the node in question

Returns

None

Note

None

XStatus XPm_GetClockParentBySelect (const enum XPmClock *clockId*, const u32 *select*, enum XPmClock *const *parentId*)

Get parent clock ID for a given clock ID and mux select value.

Parameters

<i>clockId</i>	ID of the target clock
<i>select</i>	Mux select value
<i>parentId</i>	Location to store parent clock ID

Returns

Returns XST_SUCCESS if parent clock ID is found, XST_INVALID_PARAM otherwise.

Note

None

XStatus XPm_GetSelectByClockParent (const enum XPmClock *clockId*, const enum XPmClock *parentId*, u32 *const *select*)

Get mux select value for given clock and clock parent IDs.

Parameters

<i>clockId</i>	ID of the target clock
<i>parentId</i>	ID of the parent clock
<i>select</i>	Location to store mux select value

Returns

Returns XST_SUCCESS if select value is found, XST_INVALID_PARAM otherwise.

Note

None

u8 XPm_GetClockDivType (const enum XPmClock *clock*)

Get number of divider that a given clock has.

Parameters

<i>clock</i>	ID of the target clock
--------------	------------------------

Returns

Encoded clock divider types. If the clock ID is invalid zero is returned.

Note

None

u8 XPm_MapDivider (const enum XPmClock *clock*, const u32 *div*, u32 *const *div0*, u32 *const *div1*)

Map effective divider value for given clock on DIV0 and DIV1 dividers.

Parameters

<i>clock</i>	ID of the target clock
<i>div</i>	Effective divider value
<i>div0</i>	Location to store mapped DIV0 value
<i>div1</i>	Location to store mapped DIV1 value

Returns

Encoded mask of mapped dividers

Note

The effective divider value may not be mappable on 2x 6-bit wide dividers. This is the case if a given divider value is higher than 6-bit divider (requires 2xdividers), but its a prime number (cannot be divided to get 2x divider values).

Error Status

Overview

This section lists the Power management specific return error statuses.

Macros

- #define [XST_PM_INTERNAL](#) 2000L
- #define [XST_PM_CONFLICT](#) 2001L
- #define [XST_PM_NO_ACCESS](#) 2002L
- #define [XST_PM_INVALID_NODE](#) 2003L
- #define [XST_PM_DOUBLE_REQ](#) 2004L
- #define [XST_PM_ABORT_SUSPEND](#) 2005L
- #define [XST_PM_TIMEOUT](#) 2006L
- #define [XST_PM_NODE_USED](#) 2007L

Macro Definition Documentation

#define XST_PM_INTERNAL 2000L

An internal error occurred while performing the requested operation

#define XST_PM_CONFLICT 2001L

Conflicting requirements have been asserted when more than one processing cluster is using the same PM slave

#define XST_PM_NO_ACCESS 2002L

The processing cluster does not have access to the requested node or operation

#define XST_PM_INVALID_NODE 2003L

The API function does not apply to the node passed as argument

#define XST_PM_DOUBLE_REQ 2004L

A processing cluster has already been assigned access to a PM slave and has issued a duplicate request for that PM slave

#define XST_PM_ABORT_SUSPEND 2005L

The target processing cluster has aborted suspend

#define XST_PM_TIMEOUT 2006L

A timeout occurred while performing the requested operation

#define XST_PM_NODE_USED 2007L

Slave request cannot be granted since node is non-shareable and used

Appendix L:

XiIFPGA Library v5.0

Overview

The XilFPGA library provides an interface to the Linux or bare-metal users for configuring the programmable logic (PL) over PCAP from PS.

The library is designed for Zynq® UltraScale+™ MPSoC to run on top of Xilinx standalone BSPs. It is tested for A53, R5 and MicroBlaze. In the most common use case, we expect users to run this library on the PMU MicroBlaze with PMUFW to serve requests from either Linux or Uboot for Bitstream programming.

Note

XILFPGA does not support a DDR less system. DDR must be present for use of XilFPGA.

Supported Features

The following features are supported in Zynq® UltraScale+™ MPSoC platform.

- Full bitstream loading
- Partial bitstream loading
- Encrypted bitstream loading
- Authenticated bitstream loading
- Authenticated and encrypted bitstream loading
- Readback of configuration registers
- Readback of configuration data

XilFPGA library Interface modules

XilFPGA library uses the below major components to configure the PL through PS.

Processor Configuration Access Port (PCAP)

The processor configuration access port (PCAP) is used to configure the programmable logic (PL) through the PS.

CSU DMA driver

The CSU DMA driver is used to transfer the actual bitstream file for the PS to PL after PCAP initialization.

XilSecure Library

The XilSecure library provides APIs to access secure hardware on the Zynq UltraScale+ MPSoC devices.

Note

The current version of library supports only Zynq UltraScale MPSoC devices.

Design Summary

XilFPGA library acts as a bridge between the user application and the PL device. It provides the required functionality to the user application for configuring the PL Device with the required bitstream. The following figure illustrates an implementation where the XilFPGA library needs the CSU DMA driver APIs to transfer the bitstream from the DDR to the PL region. The XilFPGA library also needs the XilSecure library APIs to support programming authenticated and encrypted bitstream files.

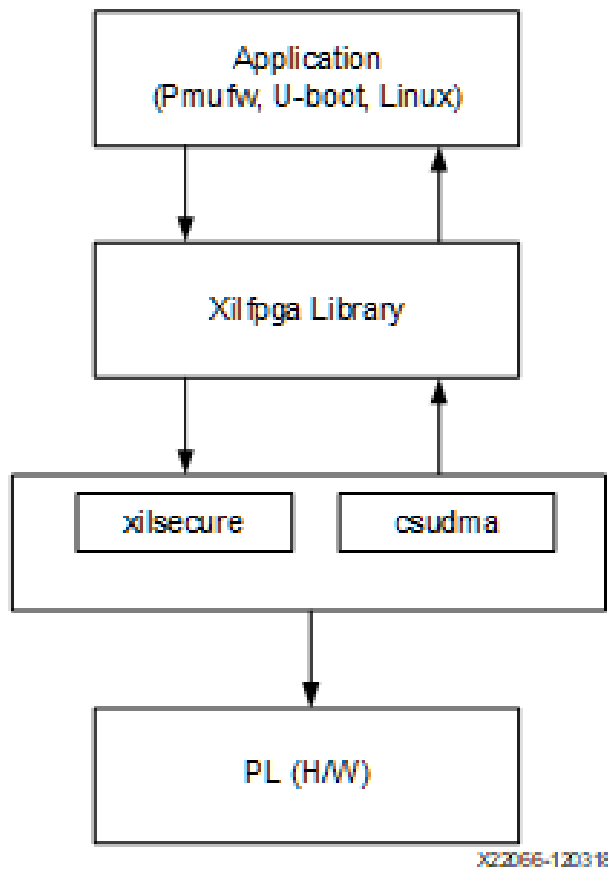


Figure 41.1: XilFPGA Design Summary

Flow Diagram

The following figure illustrates the Bitstream loading flow on the Linux operating system.

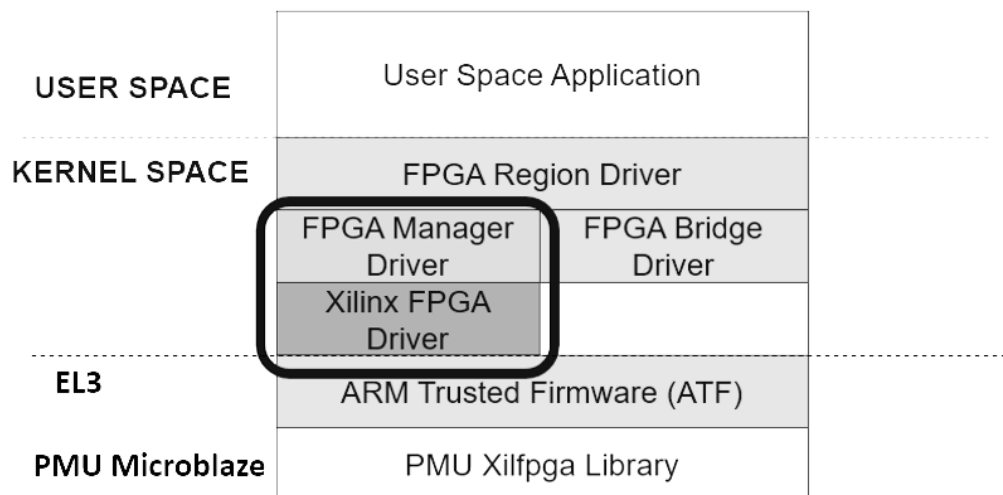


Figure 41.2: Bitstream loading on Linux:

The following figure illustrates the XilFPGA PL configuration sequence.

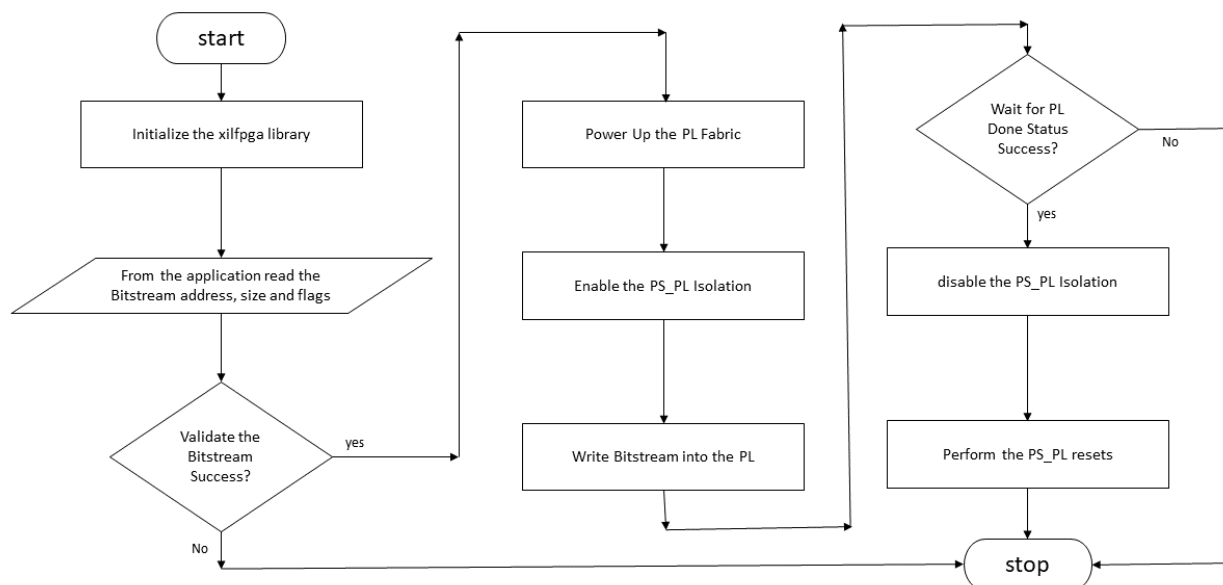


Figure 41.3: XilFPGA PL Configuration Sequence

The following figure illustrates the Bitstream write sequence.

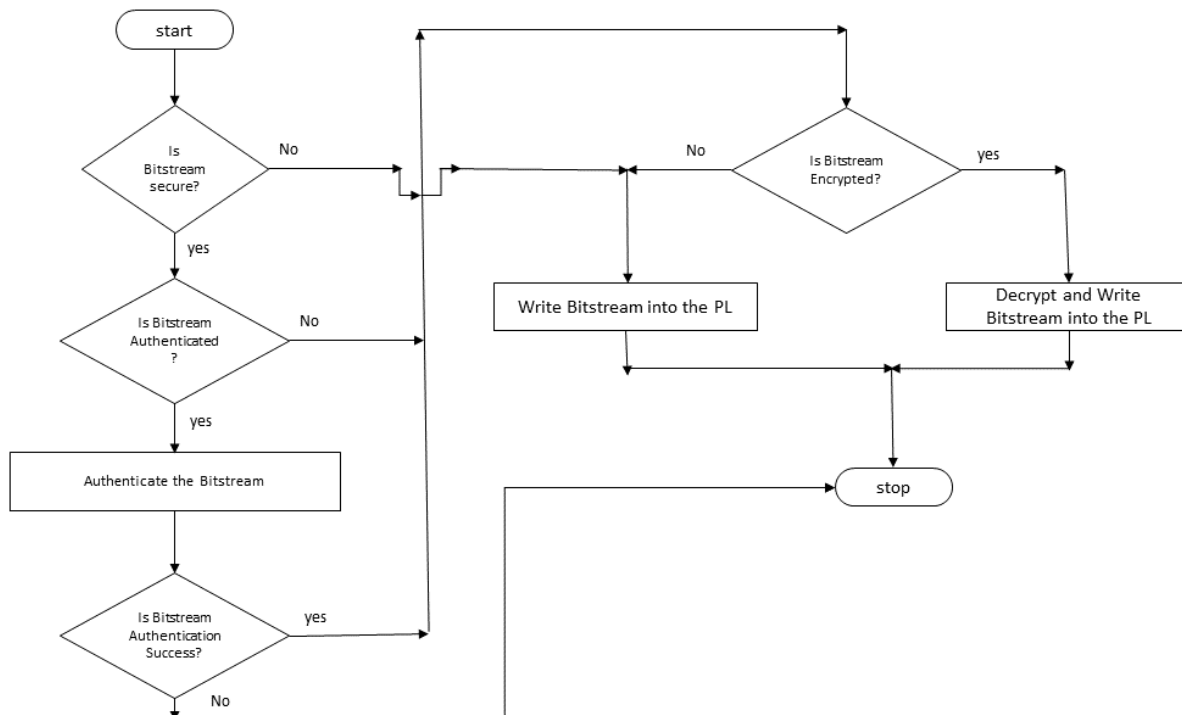


Figure 41.4: Bitstream write Sequence

Setting up the Software System

To use XilFPGA in a software application, you must first compile the XilFPGA library as part of software application.

1. Launch Xilinx SDK. Xilinx SDK prompts you to create a workspace.
2. Select **File > New > Xilinx Board Support Package**. The **New Board Support Package** wizard appears.
3. Specify a project name.
4. Select **Standalone** from the **Board Support Package OS** drop-down list. The **Board Support Package Settings** wizard appears.
5. Select the **xilfpga** library from the list of **Supported Libraries**.
6. Expand the **Overview** tree and select **xilfpga**. The configuration options for xilfpga are listed.
7. Configure the xilfpga by providing the base address of the Bit-stream file (DDR address) and the size (in bytes).

8. Click **OK**. The board support package automatically builds with XilFPGA library included in it.
9. Double-click the **system.mss** file to open it in the **Editor** view.
10. Scroll-down and locate the **Libraries** chapter.
11. Click **Import Examples** adjacent to the XilFPGA 5.0 entry.

Enabling Security

To support encrypted and/or authenticated bitstream loading, you must enable security in PMUFW.

1. Launch Xilinx SDK. Xilinx SDK prompts you to create a workspace.
2. Select **File > New > Application Project**. The **New Application Project** wizard appears.
3. Specify a project name.
4. Select **Standalone** from the **OS Platform** drop-down list.
5. Select a supported hardware platform.
6. Select **psu_pmu_0** from the **Processor** drop-down list.
7. Click **Next**. The **Templates** page appears.
8. Select **ZynqMP PMU Firmware** from the **Available Templates** list.
9. Click **Finish**. A PMUFW application project is created with the required BSPs.
10. Double-click the **system.mss** file to open it in the **Editor** view.
11. Click the **Modify this BSP's Settings** button. The **Board Support Package Settings** dialog box appears.
12. Select **xilfpga**. Various settings related to the library appears.
13. Select **secure_mode** and modify its value to **true**.
14. Click **OK** to save the configuration.

Note

By default the secure mode is enabled. To disable modify the secure_mode value to false.

Bitstream Authentication Using External Memory

The size of the Bitstream is too large to be contained inside the device, therefore external memory must be used. The use of external memory could create a security risk. Therefore, two methods are provided to authenticate and decrypt a Bitstream.

- The first method uses the internal OCM as temporary buffer for all cryptographic operations. For details, see [Authenticated and Encrypted Bitstream Loading Using OCM](#). This method does not require trust in external DDR.

- The second method uses external DDR for authentication prior to sending the data to the decryptor, there by requiring trust in the external DDR. For details, see [Authenticated and Encrypted Bitstream Loading Using DDR](#).

Bootgen

When a Bitstream is requested for authentication, Bootgen divides the Bitstream into blocks of 8MB each and assigns an authentication certificate for each block. If the size of a Bitstream is not in multiples of 8 MB, the last block contains the remaining Bitstream data.

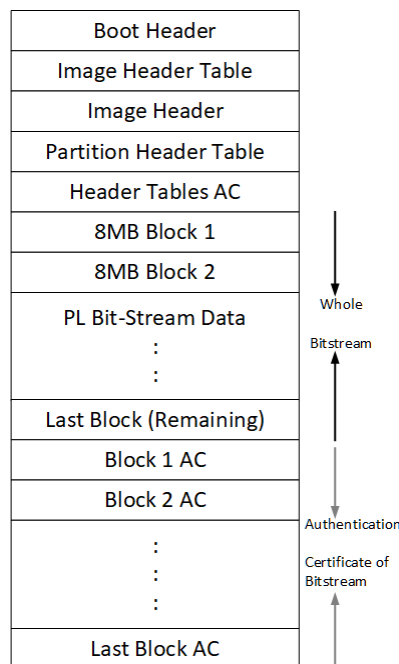


Figure 41.5: Bitstream Blocks

When both authentication and encryption are enabled, encryption is first done on the Bitstream. Bootgen then divides the encrypted data into blocks and assigns an Authentication certificate for each block.

Authenticated and Encrypted Bitstream Loading Using OCM

To authenticate the Bitstream partition securely, XilFPGA uses the FSBL chapter's OCM memory to copy the bitstream in chunks from DDR. This method does not require trust in the external DDR to securely authenticate and decrypt a Bitstream.

The software workflow for authenticating Bitstream is as follows:

1. XilFPGA identifies DDR secure Bitstream image base address. XilFPGA has two buffers in OCM, the Read Buffer is of size 56KB and hash of chunks to store intermediate hashes calculated for each 56 KB of every 8MB block.

2. XiIFPGA copies a 56KB chunk from the first 8MB block to Read Buffer.
3. XiIFPGA calculates hash on 56 KB and stores in HashsOfChunks.
4. XiIFPGA repeats steps 1 to 3 until the entire 8MB of block is completed.

Note

The chunk that XiIFPGA copies can be of any size. A 56KB chunk is taken for better performance.

5. XiIFPGA authenticates the 8MB Bitstream chunk.
6. Once the authentication is successful, XiIFPGA starts copying information in batches of 56KB starting from the first block which is located in DDR to Read Buffer, calculates the hash, and then compares it with the hash stored at HashsOfChunks.
7. If the hash comparison is successful, FSBL transmits data to PCAP using DMA (for un-encrypted Bitstream) or AES (if encryption is enabled).
8. XiIFPGA repeats steps 6 and 7 until the entire 8MB block is completed.
9. Repeats steps 1 through 8 for all the blocks of Bitstream.

Note

You can perform warm restart even when the FSBL OCM memory is used to authenticate the Bitstream. PMU stores the FSBL image in the PMU reserved DDR memory which is visible and accessible only to the PMU and restores back to the OCM when APU-only restart needs to be performed. PMU uses the SHA3 hash to validate the FSBL image integrity before restoring the image to OCM (PMU takes care of only image integrity and not confidentiality).

Authenticated and Encrypted Bitstream Loading Using DDR

The software workflow for authenticating Bitstream is as follows:

1. XiIFPGA identifies DDR secure Bitstream image base address.
2. XiIFPGA calculates hash for the first 8MB block.
3. XiIFPGA authenticates the 8MB block while stored in the external DDR.
4. If Authentication is successful, XiIFPGA transmits data to PCAP via DMA (for unencrypted Bitstream) or AES (if encryption is enabled).
5. Repeats steps 1 through 4 for all the blocks of Bitstream.

XilFPGA APIs

Overview

This chapter provides detailed descriptions of the XilFPGA library APIs.

Functions

- u32 [XFpga_PL_BitStream_Load](#) (XFpga *InstancePtr, UINTPTR BitstreamImageAddr, UINTPTR AddrPtr_Size, u32 Flags)
- u32 [XFpga_PL_PostConfig](#) (XFpga *InstancePtr)
- u32 [XFpga_PL_ValidateImage](#) (XFpga *InstancePtr, UINTPTR BitstreamImageAddr, UINTPTR AddrPtr_Size, u32 Flags)
- u32 [XFpga_GetPIConfigData](#) (XFpga *InstancePtr, UINTPTR ReadbackAddr, u32 ConfigReg_NumFrames)
- u32 [XFpga_GetPIConfigReg](#) (XFpga *InstancePtr, UINTPTR ReadbackAddr, u32 ConfigReg_NumFrames)
- u32 [XFpga_InterfaceStatus](#) (XFpga *InstancePtr)

Function Documentation

u32 XFpga_PL_BitStream_Load (XFpga * InstancePtr, UINTPTR BitstreamImageAddr, UINTPTR AddrPtr_Size, u32 Flags)

The API is used to load the bitstream file into the PL region.

It supports vivado generated Bitstream(*.bit, *.bin) and bootgen generated Bitstream(*.bin) loading, Passing valid Bitstream size (AddrPtr_Size) info is mandatory for vivado * generated Bitstream, For bootgen generated Bitstreams it will take Bitstream size from the Bitstream Header.

Parameters

<i>InstancePtr</i>	Pointer to the XFpga structure.
<i>BitstreamImageAddr</i>	Linear memory Bitstream image base address
<i>AddrPtr_Size</i>	Aes key address which is used for Decryption (or) In none Secure Bitstream used it is used to store size of Bitstream Image.
<i>Flags</i>	Flags are used to specify the type of Bitstream file. • BIT(0) - Bitstream type ◦ 0 - Full Bitstream ◦ 1 - Partial Bitstream • BIT(1) - Authentication using DDR ◦ 1 - Enable ◦ 0 - Disable • BIT(2) - Authentication using OCM ◦ 1 - Enable ◦ 0 - Disable • BIT(3) - User-key Encryption ◦ 1 - Enable ◦ 0 - Disable • BIT(4) - Device-key Encryption ◦ 1 - Enable ◦ 0 - Disable

Returns

- XFPGA_SUCCESS on success
- Error code on failure.
- XFPGA_VALIDATE_ERROR.
- XFPGA_PRE_CONFIG_ERROR.
- XFPGA_WRITE_BITSTREAM_ERROR.
- XFPGA_POST_CONFIG_ERROR.

u32 XFpga_PL_PostConfig (XFpga * *InstancePtr*)

This function set FPGA to operating state after writing.

Parameters

<i>InstancePtr</i>	Pointer to the XFpga structure
--------------------	--------------------------------

Returns

Codes as mentioned in xilfpga.h

u32 XFpga_PL_ValidateImage (XFpga * InstancePtr, UINTPTR BitstreamImageAddr, UINTPTR AddrPtr_Size, u32 Flags)

This function is used to validate the Bitstream Image.

Parameters

<i>InstancePtr</i>	Pointer to the XFpga structure
<i>BitstreamImageAddr</i>	Linear memory Bitstream image base address
<i>AddrPtr_Size</i>	Aes key address which is used for Decryption (or) In none Secure Bitstream used it is used to store size of Bitstream Image.
<i>Flags</i>	Flags are used to specify the type of Bitstream file. • BIT(0) - Bitstream type ◦ 0 - Full Bitstream ◦ 1 - Partial Bitstream • BIT(1) - Authentication using DDR ◦ 1 - Enable ◦ 0 - Disable • BIT(2) - Authentication using OCM ◦ 1 - Enable ◦ 0 - Disable • BIT(3) - User-key Encryption ◦ 1 - Enable ◦ 0 - Disable • BIT(4) - Device-key Encryption ◦ 1 - Enable ◦ 0 - Disable

Returns

Codes as mentioned in xilfpga.h

u32 XFpga_GetPIConfigData (XFpga * *InstancePtr*, UINTPTR *ReadbackAddr*, u32 *ConfigReg_NumFrames*)

This function provides functionality to read back the PL configuration data.

Parameters

<i>InstancePtr</i>	Pointer to the XFpga structure
--------------------	--------------------------------

Address which is used to store the PL readback data.

Configuration register value to be returned (or) The number of Fpga configuration frames to read

Returns

- XFPGA_SUCCESS if successful
- XFPGA_FAILURE if unsuccessful
- XFPGA_OPS_NOT_IMPLEMENTED if implementation not exists.

u32 XFpga_GetPIConfigReg (XFpga * *InstancePtr*, UINTPTR *ReadbackAddr*, u32 *ConfigReg_NumFrames*)

This function provides PL specific configuration register values.

Parameters

<i>InstancePtr</i>	Pointer to the XFpga structure
<i>ConfigReg</i>	Constant which represents the configuration register value to be returned.
<i>Address</i>	DMA linear buffer address.

Returns

- XFPGA_SUCCESS if successful
- XFPGA_FAILURE if unsuccessful
- XFPGA_OPS_NOT_IMPLEMENTED if implementation not exists.

u32 XFpga_InterfaceStatus (XFpga * *InstancePtr*)

This function provides the STATUS of PL programming interface.

Parameters

<i>InstancePtr</i>	Pointer to the XFgpa structure
--------------------	--------------------------------

Returns

Status of the PL programming interface.

Appendix M:

XiMailbox Library v1.0

XilMailbox

Overview

The XilMailbox library provides the top-level hooks for sending or receiving an inter-processor interrupt (IPI) message using the Zynq® UltraScale+™ MPSoC IPI hardware.

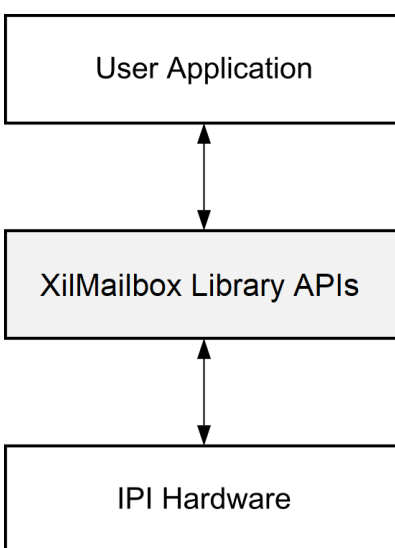


Figure 43.1: Overview

For more details on the IPI interrupts, see the Zynq UltraScale+ MPSoC Technical Reference Manual ([UG1085](#)). This library supports the following features:

- Triggering an IPI to a remote agent.
- Sending an IPI message to a remote agent.
- Callbacks for error and recv IPI events.
- Reading an IPI message.

Software Initialization

1. [XMailbox_Initialize\(\)](#) function initializes a library instance for the given IPI channel.
2. [XMailbox_Send\(\)](#) function triggers an IPI to a remote agent.
3. [XMailbox_SendData\(\)](#) function sends an IPI message to a remote agent, message type should be either XILMBOX_MSG_TYPE_REQ (OR) XILMBOX_MSG_TYPE_RESP.
4. [XMailbox_Recv\(\)](#) function reads an IPI message from a specified source agent, message type should be either XILMBOX_MSG_TYPE_REQ (OR) XILMBOX_MSG_TYPE_RESP.
5. [XMailbox_SetCallBack\(\)](#) using this function user can register call backs for receive and error events.

Data Structures

- struct [XMailbox](#)

Enumerations

- enum [XMailbox_Handler](#) {
 [XMAILBOX_RECV_HANDLER](#),
 [XMAILBOX_ERROR_HANDLER](#) }

Functions

- u32 [XMailbox_Send](#) ([XMailbox](#) *InstancePtr, u32 Remoteld, u8 Is_Blocking)
- u32 [XMailbox_SendData](#) ([XMailbox](#) *InstancePtr, u32 Remoteld, void *BufferPtr, u32 MsgLen, u8 BufferType, u8 Is_Blocking)
- u32 [XMailbox_Recv](#) ([XMailbox](#) *InstancePtr, u32 Sourceld, void *BufferPtr, u32 MsgLen, u8 BufferType)
- s32 [XMailbox_SetCallBack](#) ([XMailbox](#) *InstancePtr, [XMailbox_Handler](#) HandlerType, void *CallBackFuncPtr, void *CallBackRefPtr)
- u32 [XMailbox_Initialize](#) ([XMailbox](#) *InstancePtr, u8 DeviceId)

Data Structure Documentation

struct XMailbox

[XMailbox](#) structure.

Parameters

<i>XMbox_IPI_Send</i>	Triggers an IPI to a destination CPU
<i>XMbox_IPI_SendData</i>	Sends an IPI message to a destination CPU
<i>XMbox_IPI_Recv</i>	Reads an IPI message
<i>RecvHandler</i>	Callback for receive IPI event
<i>ErrorHandler</i>	Callback for error event
<i>ErroRef</i>	To be passed to the error interrupt callback
<i>RecvRef</i>	To be passed to the receive interrupt callback.
<i>Agent</i>	Used to store IPI Channel information.

Enumeration Type Documentation

enum XMailbox_Handler

This typedef contains XMAILBOX Handler Types.

Enumerator

- XMAILBOX_RECV_HANDLER*** For Recv Handler.
XMAILBOX_ERROR_HANDLER For Error Handler.

Function Documentation

u32 XMailbox_Send (XMailbox * InstancePtr, u32 Remoteld, u8 Is_Blocking)

This function triggers an IPI to a destination CPU.

Parameters

<i>InstancePtr</i>	Pointer to the XMailbox instance
<i>Remoteld</i>	Mask of the CPU to which IPI is to be triggered
<i>Is_Blocking</i>	If set, triggers notification in the blocking mode

Returns

- XST_SUCCESS if successful
- XST_FAILURE if unsuccessful

u32 XMailbox_SendData (XMailbox * InstancePtr, u32 Remoteld, void * BufferPtr, u32 MsgLen, u8 BufferType, u8 Is_Blocking)

This function sends an IPI message to a destination CPU.

Parameters

<i>InstancePtr</i>	Pointer to the XMailbox instance
<i>Remoteld</i>	Mask of the CPU to which IPI is to be triggered
<i>BufferPtr</i>	Pointer to Buffer which contains the message to be sent
<i>MsgLen</i>	Length of the buffer/message
<i>BufferType</i>	Type of buffer (XILMBOX_MSG_TYPE_REQ (OR) XILMBOX_MSG_TYPE_RESP)
<i>Is_Blocking</i>	If set, triggers the notification in blocking mode

Returns

- XST_SUCCESS if successful
- XST_FAILURE if unsuccessful

u32 XMailbox_Recv (XMailbox * InstancePtr, u32 Sourceld, void * BufferPtr, u32 MsgLen, u8 BufferType)

This function reads an IPI message.

Parameters

<i>InstancePtr</i>	Pointer to the XMailbox instance
<i>Sourceld</i>	Mask for the CPU which has sent the message
<i>BufferPtr</i>	Pointer to Buffer to which the read message needs to be stored
<i>MsgLen</i>	Length of the buffer/message
<i>BufferType</i>	Type of buffer (XILMBOX_MSG_TYPE_REQ or XILMBOX_MSG_TYPE_RESP)

Returns

- XST_SUCCESS if successful
- XST_FAILURE if unsuccessful

s32 XMailbox_SetCallback (XMailbox * InstancePtr, XMailbox_Handler HandlerType, void * CallbackFuncPtr, void * CallbackRefPtr)

This routine installs an asynchronous callback function for the given HandlerType.

HandlerType	Callback Function Type
XMAILBOX_RECV_HANDLER	Recv handler
XMAILBOX_ERROR_HANDLER	Error handler

Parameters

<i>InstancePtr</i>	Pointer to the XMailbox instance
<i>HandlerType</i>	Specifies which callback is to be attached
<i>CallbackFunc</i>	Address of the callback function
<i>CallbackRef</i>	User data item that will be passed to the callback function when it is invoked

Returns

- XST_SUCCESS when handler is installed.
- XST_INVALID_PARAM when HandlerType is invalid.

Note

Invoking this function for a handler that already has been installed replaces it with the new handler.

u32 XMailbox_Initialize (XMailbox * InstancePtr, u8 Deviceld)

Initialize the [XMailbox](#) Instance.

Parameters

<i>InstancePtr</i>	is a pointer to the instance to be worked on
<i>Deviceld</i>	is the IPI Instance to be worked on

Returns

XST_SUCCESS if initialization was successful XST_FAILURE in case of failure

Additional Resources and Legal Notices

Xilinx Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Xilinx Support](#).

Solution Centers

See the [Xilinx Solution Centers](#) for support on devices, software tools, and intellectual property at all stages of the design cycle. Topics include design assistance, advisories, and troubleshooting tips.

Documentation Navigator and Design Hubs

Xilinx® Documentation Navigator provides access to Xilinx documents, videos, and support resources, which you can filter and search to find information. To open the Xilinx Documentation Navigator (DocNav):

- From the Vivado® IDE, select **Help > Documentation and Tutorials**.
- On Windows, select **Start > All Programs > Xilinx Design Tools > DocNav**.
- At the Linux command prompt, enter `docnav`.

Xilinx Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In the Xilinx Documentation Navigator, click the **Design Hubs View** tab.
- On the Xilinx website, see the [Design Hubs](#) page.

Note: For more information on Documentation Navigator, see the [Documentation Navigator](#) page on the Xilinx website.

References

Xilinx References

1. *Xilinx Third-Party Licensing Solution Center*
2. [PetaLinux Product Page](#)
3. [Xilinx Vivado Design Suite – HLx Editions](#)
4. [Xilinx Third-Party Tools](#)
5. [Zynq UltraScale+ MPSoC Product Table](#)
6. [Zynq UltraScale+ MPSoC Product Advantages](#)
7. [Zynq UltraScale+ MPSoC Products Page](#)

Zynq Devices Documentation

8. *Quick EMUlator (QEMU) User Guide* ([UG1169](#))
9. *UltraScale Architecture and Product Overview* ([DS890](#))
10. *Isolation Methods in Zynq UltraScale+ MPSoCs* ([XAPP1320](#))
11. *Zynq UltraScale+ MPSoC Technical Reference Manual* ([UG1085](#))
12. *Zynq UltraScale+ MPSoC Register Reference* ([UG1087](#))
13. *Zynq UltraScale+ MPSoC: Embedded Design Tutorial* ([UG1209](#))
14. *Zynq UltraScale+ MPSoC Processing System LogiCORE IP Product Guide* ([PG201](#))
15. *UltraScale Architecture System Monitor Guide* ([UG580](#))
16. *Zynq UltraScale+ MPSoC OpenAMP Getting Started Guide* ([UG1186](#))
17. *Embedded Energy Management Interface Specification* ([UG1200](#))
18. *UltraFast Embedded Design Methodology Guide* ([UG1046](#))
19. *Zynq-7000 Embedded Design Tutorial* ([UG1165](#))
20. *Zynq-7000 SoC Software Developers Guide* ([UG821](#))
21. *UltraScale Architecture PCB Design* ([UG583](#))
22. [Vivado Design Suite Documentation](#)
23. *Bootgen User Guide* ([UG1283](#))

SDK and PetaLinux Documents

- 24. *Xilinx Software Developer Kit Help* ([UG782](#))
- 25. *OS and Libraries Document Collection* ([UG643](#))
- 26. *Embedded Design Tools* [Download](#)
- 27. *PetaLinux Tools Documentation Reference Guide* ([UG1144](#))
- 28. *Xilinx Software Development Kit: System Performance* ([UG1145](#))

Xilinx IP Documents

- 29. *LogiCORE IP AXI Central Direct Memory Access Product Guide* ([PG034](#))
- 30. *LogiCORE IP AXI Video Direct Memory Access Product Guide* ([PG020](#))

Miscellaneous Links

- 31. [Xilinx Github](#)
- 32. [Embedded Development](#)
- 33. [meta-xilinx](#)
- 34. [PetaLinux Software Development](#)
- 35. [Zynq UltraScale+ Silicon Devices Page](#)
- 36. Xilinx Answer: [66249](#)
- 37. [Vivado Quick Take Video: Vivado PS Configuration Wizard Overview](#)
- 38. [Xilinx Wiki](#)

Third-Party References

- 39. [Lauterbach Technologies](#)
- 40. [Arm Trusted Firmware](#)
- 41. [Xen Hypervisor](#)
- 42. [Arm Developer Center](#)
- 43. [Arm Cortex-A53 MPCore Processor Technical Reference Manual](#)
- 44. [Yocto Product Development](#)
- 45. [GNU FTP](#)
- 46. *Power State Coordination Interface* – Arm DEN 0022B.b, 6/25/2013

Please Read: Important Legal Notices

The information disclosed to you hereunder (the "Materials") is provided solely for the selection and use of Xilinx products. To the maximum extent permitted by applicable law: (1) Materials are made available "AS IS" and with all faults, Xilinx hereby DISCLAIMS ALL WARRANTIES AND CONDITIONS, EXPRESS, IMPLIED, OR STATUTORY, INCLUDING BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE; and (2) Xilinx shall not be liable (whether in contract or tort, including negligence, or under any other theory of liability) for any loss or damage of any kind or nature related to, arising under, or in connection with, the Materials (including your use of the Materials), including for any direct, indirect, special, incidental, or consequential loss or damage (including loss of data, profits, goodwill, or any type of loss or damage suffered as a result of any action brought by a third-party) even if such damage or loss was reasonably foreseeable or Xilinx had been advised of the possibility of the same. Xilinx assumes no obligation to correct any errors contained in the Materials or to notify you of updates to the Materials or to product specifications. You may not reproduce, modify, distribute, or publicly display the Materials without prior written consent. Certain products are subject to the terms and conditions of Xilinx's limited warranty, please refer to Xilinx's Terms of Sale which can be viewed at <https://www.xilinx.com/legal.htm#tos>; IP cores may be subject to warranty and support terms contained in a license issued to you by Xilinx. Xilinx products are not designed or intended to be fail-safe or for use in any application requiring fail-safe performance; you assume sole risk and liability for use of Xilinx products in such critical applications, please refer to Xilinx's Terms of Sale which can be viewed at <https://www.xilinx.com/legal.htm#tos>.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

© Copyright 2015-2019 Xilinx, Inc. Xilinx, the Xilinx logo, Artix, ISE, Kintex, Spartan, Virtex, Vivado, Zynq, and other designated brands included herein are trademarks of Xilinx in the United States and other countries. All other trademarks are the property of their respective owners. AMBA, AMBA Designer, Arm, ARM1176JZ-S, CoreSight, Cortex, PrimeCell, Mali, and MPCore are trademarks of Arm Limited in the EU and other countries. MATLAB and Simulink are registered trademarks of The MathWorks, Inc. PCI, PCIe, and PCI Express are trademarks of PCI-SIG and used under license.